

Agent Systems

Lucas Kim

한국어판 (Korean Edition)

2026년 8월

머리말

이 책은 대규모 언어 모델을 기반으로 하는 에이전트 시스템의 설계, 구축, 평가, 운영 전반을 다룬다.

지난 몇 년 사이 이 분야에는 자료가 넘치도록 쌓였다. 그런데 대체로 두 극단에 몰려 있다.

한쪽에는 특정 프레임워크의 사용법을 알려 주는 글이 있다. 빠르게 무언가를 돌아가게 해 주지만 프레임워크가 바뀌면 함께 낡는다. 다른 쪽에는 개별 기법을 제안하는 연구 논문이 있다. 정확하지만 시스템 전체를 어떻게 조립할지는 알려 주지 않는다.

그 사이에 있어야 할 것이 비어 있다. 왜 이 부품이 필요한지, 언제 그것을 쓰지 말아야 하는지, 고를 때 무엇을 치르는지다. 이 책은 그 자리를 채우려는 시도다.

누가 읽어야 하는가

이 책은 에이전트 시스템에 대한 큰 틀을 이해하고자 하는 독자를 염두에 두었다. 언어 모델 API를 한 번이라도 직접 불러 본 경험을 가정한다. 셋이면 된다. 호출이 이전 호출을 기억하지 않아 매번 이력 전체를 다시 보낸다는 것, 그 보내고 받는 분량만큼 요금이 붙는다는 것, 모델에게 함수 목록을 알려 주면 모델은 이름과 인자를 고를 뿐이고 실제로 실행하는 일은 바깥 코드의 몫이라는 것.

파이썬을 읽을 수 있으면 좋지만 이 책에 코드는 없고, 머신러닝 파이프라인 비유를 종종 사용하나, 깊이 알 필요도 없다.

이 책의 원칙

1. **프레임워크보다 개념을 먼저 다룬다.** 개념을 먼저 세워 두면 그것이 어느 라이브러리에서 어떤 이름으로 구현되어 있는 알아볼 수 있다. 이름을 먼저 외우면 프레임워크가 바뀔 때 남는 것이 없다. 마지막 장에서 실제 도구들을 짚는 것도 기능을 견주기 위해서가 아니라, 그 도구가 앞에서 세운 결정들 가운데 무엇을 대신 정해 주는지를 보기 위해서다.
2. **고를 때 무엇을 치르는지 적는다.** 어떤 기법을 소개할 때는 그것으로 무엇이 좋아지고 무엇이 늘어나는지를 함께 적는다. 각 장의 끝에는 그 기법을 쓰지 말아야 할 조건을 정리한 절을 둔다.

에이전트 시스템을 큰 틀에서 이해하려는 모든 이에게 이 책이 보탬이 되기를 바란다.

이 책의 지도

이 책이 다루는 것은 하나의 시스템이다. 모델이 행동을 내고, 환경이 관측을 돌려주고, 그 관측이 다른 재료들과 함께 컨텍스트로 조립되어 다시 모델로 들어가는 순환이다. 그리고 그 순환을 바깥에서 소유하는 제어가 있다.

이 구조에는 말을 대가 몇 군데 있고, 일곱 개의 부가 저마다 한 군데씩을 맡는다. 그런데 그것들이 한 층에 있지 않다. 순환의 안쪽에 있는 것도 있고 전체를 향해 서는 것도 있다.

아래 그림이 그 구조 위에 부를 얹은 지도다.

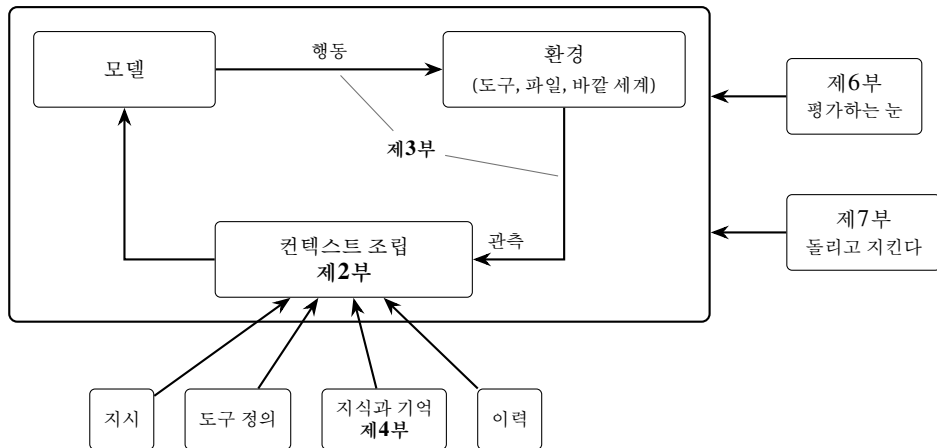
처음 읽을 때는 부의 순서대로 읽으면 된다. 앞의 부가 세운 것을 뒤의 부가 쓰기 때문이다.

다만 그 순서는 읽는 순서이지 시스템의 순서가 아니다. 앞 단계가 끝나야 뒤 단계가 시작되는 것이 아니라, 이것들은 실행 중에 동시에 살아 있고 서로를 물고 있다.

그래서 실무에서 문제를 만났을 때는 처음부터 다시 읽을 것이 아니라, 그 문제가 어느 부의 것인지 짚고 거기를 펴면 된다. 짚는 일을 돕는 것이 이 지도 끝의 표이고, 표의 마지막 열에 증상을 적어 두었다.

이 그림이 지금 낯설어도 괜찮다. 제1장이 이 구조를 제자리에서 세우고(그림 1.1), 그 뒤의 모든 장이 이 구조 위에서 말한다. 그 장을 읽고 여기로 돌아오면 같은 그림이 다시 설 것이다.

하네스 (루프의 소유, 제5부)



그림에 대해 밝혀 둘 것이 넷이다.

첫째, 부 번호는 읽는 순서이지 실행 순서가 아니다. 상자의 크기와 위치도 비중을 말하지 않는다. 옆에 세운 두 상자도 마찬가지다. 평가하는 일과 돌리고 지키는 일은 시스템이 선 다음에 오는 것이 아니라 처음부터 전체를 향해 서 있고, 화살표가 그 방향이다.

둘째, 조립 아래의 네 상자는 재료다. 그중 이력만 밖에서 오지 않고 순환 자신이 쌓는다.

셋째, 통로와 하네스는 다른 부다. 나가는 행동과 돌아오는 반환이 제3부이고, 그 순환을 바깥에서 소유하는 하네스가 제5부다. 재료가 먼저 서고 그것을 돌리는 장치가 나중에 온다. 지시와 도구와 지식이 다 선 뒤야 루프를 소유하는 일이 무엇을 소유하는지 말할 수 있기 때문이다.

넷째, 그림에 없는 부가 하나 있다. 제1부는 어느 한 상자의 부가 아니라 이 구조 자체를 세우는 부라서 그림 전체가 그 자리다. 순환도 루프의 형태도 거기서 선다. 여섯 부가 그림에 서고, 첫째 부가 그림을 세운다.

마지막 장인 제23장은 제7부에 들되 성격이 조금 다르다. 이 구조의 한 군데를 새로 맡는 장이 아니라, 실무에서 쓰이는 것들이 이 지도의 어디를 대신 채워 주는지를 짚는 장이다. 앞의 스물두 장이 지도를 그리고, 마지막 장은 그 지도를 들고 실물 앞에 선다.

부	맡는 것과 물음	이 부를 펴야 하는 증상
제1부 에이전트란 무엇인가	구조 전체와 루프의 형태. 무엇이 에이전트이고, 다음 스텝을 누가 정하며, 그 결정이 루프의 모양을 어떻게 바꾸는가	무엇을 만들고 있는지 이름이 서지 않는다. 워크플로로 될 일에 에이전트를 세웠다. 루프가 몇 스텝을 돌지 짐작할 수 없다
제2부 컨텍스트를 다루는 기술	예산의 단위와 조립의 안쪽. 무엇이 한 톨크이고 길이가 무엇을 무는가, 그리고 유한하고 길수록 나빠지는 컨텍스트에 무엇을, 언제, 어떤 확실성으로 실을 것인가	값과 지연이 과제 크기보다 빨리 자란다. 컨텍스트가 길어질수록 답이 나빠진다. 넣어 둔 지시가 지켜지지 않는다. 모델을 바꿨더니 전에 없던 실패가 나온다
제3부 도구와 외부 세계	환경으로 나가는 통로. 행동의 어휘인 도구, 그 반환, 능력이 공급되는 경로, 그리고 출력의 형식. 무엇으로 작용하게 하고 돌아온 것을 무엇으로 믿을 것인가	모델이 도구를 엉뚱하게 고르거나 없는 사용법을 지어낸다. 도구가 늘수록 고르기가 나빠진다. 출력의 형식이 자꾸 어긋난다. 도구는 성공했다는데 답이 틀린다
제4부 지식과 기억	조립의 재료 가운데 지식. 밖에 둔 것을 찾아 들이는 장치와 세션을 넘는 기억. 모델이 모르는 것과 남은 것을 어떻게 알게 하는가	남은 값을 확신 있게 말한다. 문서에 있는 답을 못 찾는다. 찾아 넣었는데도 답이 다른 데서 온다. 답이 틀렸는데 어느 지식이 틀렸는지 짚을 수 없다
제5부 루프를 돌리는 일	순환을 바깥에서 소유하는 제어. 언제 멈추고, 무엇을 남기고, 어떻게 다시 시작하고, 무엇을 믿을 것인가. 그리고 루프가 여럿일 때 언제 나누고 나눈 것들을 무엇으로 잇는가	루프가 멈추지 않거나 이상하게 일찍 멈춘다. 세션이 끊기면 처음부터 다시 한다. 한 컨텍스트에 담기지 않는 과제를 만났다. 나는 에이전트들이 서로 다른 전제 위에서 일한다
제6부 평가와 관측	이 시스템을 평가하는 일. 좋아졌다는 것을 무엇으로 아는가, 실행에서 무엇을 남겨야 나중에 볼 수 있는가	고쳤는데 나아졌는지 판정할 수 없다. 실패가 재현되지 않는다. 틀린 답의 원인을 찾을 기록이 없다
제7부 운영, 안전, 거버넌스	이것을 돌리고 지킬 때. 되돌릴 수 없는 결과 앞에 누구를 세우고, 밖에서 오는 텍스트를 어떻게 받으며, 어디까지가 이미 만들어져 있는가	실사용자 앞에 내보내기가 불안하다. 밖에서 온 텍스트가 지시처럼 작동한다. 무엇까지 허락할지 정해야 한다. 직접 만들 것과 가져다 쓸 것을 갈라야 한다

끝으로 인용에 관해 밝혀 둔다. 본문의 인용은 저자와 연도로 적었고, 1차 자료를 우선했다. 산업계의 기록을 인용할 때는 발행 주체와 시점을 함께 밝혔다. 이 분야는 빠르게 바뀌므로, 특정 제품이나 규격의 세부는 인용 시점의 것임을 감안해 읽기를 권한다.

차례

머리말	iii
이 책의 지도	v
I 에이전트란 무엇인가	1
1 에이전트란 무엇인가	3
1.1 고전적 정의: 지각과 행동의 순환	3
1.2 왜 하필 지금인가	3
1.3 워크플로와 에이전트	4
1.4 에이전트의 구성 요소	5
1.5 루프는 무엇으로 되어 있는가	6
1.6 언제 에이전트를 쓰지 말아야 하는가	7
1.7 이 책의 구조	7
1.8 요약	9
1.9 생각해 볼 문제	9
1.10 다시 읽을 때	10
1.11 더 읽을거리	10
2 추론과 계획 패턴 지도	11
2.1 어떤 패턴들이 있는가	11
2.2 선택의 결정 절차	13
2.3 패턴이 실패하는 방식	14
2.4 원인이 패턴 밖에 있을 때	15
2.5 요약	17
2.6 생각해 볼 문제	18
2.7 다시 읽을 때	18
2.8 더 읽을거리	18
II 컨텍스트를 다루는 기술	19
3 토큰: 예산의 단위	21
3.1 토큰화: 무엇이 세어지는가	21
3.2 같은 내용도 언어마다 값이 다르다	21
3.3 다루지 않은 것	22
3.4 요약	22
3.5 생각해 볼 문제	23
3.6 다시 읽을 때	23
3.7 더 읽을거리	23

4	컨텍스트 엔지니어링	25
4.1	조립 함수의 안쪽	25
4.2	무엇을 실을 것인가	25
4.3	언제 실을 것인가	26
4.4	왜 다 넣으면 안 되는가	27
4.5	요약	30
4.6	생각해 볼 문제	30
4.7	다시 읽을 때	31
4.8	더 읽을 거리	31
5	프롬프트 엔지니어링	33
5.1	프롬프트는 어디에 있는가	33
5.2	지시가 하는 일	33
5.3	프롬프트는 견고하지 않다	35
5.4	프롬프트를 더 고치지 말아야 할 때	36
5.5	요약	38
5.6	생각해 볼 문제	38
5.7	다시 읽을 때	39
5.8	더 읽을 거리	39
6	압축과 요약 전략	41
6.1	자라는 층	41
6.2	압축의 두 목적	42
6.3	무엇을 버릴 수 있는가	43
6.4	압축의 방법	44
6.5	압축은 공짜가 아니다	45
6.6	반복 압축의 문제	46
6.7	압축하지 말아야 할 때	47
6.8	요약	49
6.9	생각해 볼 문제	50
6.10	다시 읽을 때	50
6.11	더 읽을 거리	50
III	도구와 외부 세계	51
7	도구 설계 원칙	53
7.1	도구의 구성	53
7.2	도구명과 도구 설명이 성능을 정한다	55
7.3	최적화된 도구 설계	56
7.4	컨텍스트 안에서 도구의 출력	59
7.5	도구를 더 만들지 말아야 할 때	62
7.6	요약	64
7.7	생각해 볼 문제	65
7.8	다시 읽을 때	65
7.9	더 읽을 거리	65

8	모델 컨텍스트 프로토콜(MCP)	67
8.1	외부와의 연결	67
8.2	MCP	67
8.3	MCP가 미치는 영향	70
8.4	MCP 설계	71
8.5	프로토콜을 쓰지 말아야 할 때	72
8.6	요약	75
8.7	생각해 볼 문제	75
8.8	다시 읽을 때	76
8.9	더 읽을거리	76
9	구조화된 출력과 스키마 설계	77
9.1	왜 형식을 정하는가	77
9.2	스키마가 하는 일	78
9.3	얼마나 조일 것인가	80
9.4	모델이 형식을 어겼을 때	81
9.5	스키마로 덮을 수 없는 것	83
9.6	요약	84
9.7	생각해 볼 문제	85
9.8	다시 읽을 때	85
9.9	더 읽을거리	85
IV	지식과 기억	87
10	에이전트 메모리: 단기, 장기, 영속	89
10.1	남겨 두어야 하는 것	89
10.2	세 층위	90
10.3	사용되는 순간	91
10.4	잘못 누적되는 기억	92
10.5	기억을 두지 말아야 할 때	94
10.6	요약	96
10.7	생각해 볼 문제	97
10.8	다시 읽을 때	97
10.9	더 읽을거리	97
11	검색 증강 생성(RAG)	99
11.1	외부 자료의 사용	99
11.2	밖에 두면 무엇이 달라지는가	100
11.3	검색이라는 단계	101
11.4	칭킹, 임베딩, 벡터 검색	102
11.5	검색이 실패했을 때	107
11.6	검색으로 풀 수 없는 것	109
11.7	요약	111
11.8	생각해 볼 문제	112
11.9	다시 읽을 때	112
11.10	더 읽을거리	112

12 하이브리드 검색	115
12.1 두 검색을 겹치는 이유	115
12.2 두 목록을 하나로	116
12.3 재순위화: 후보를 다시 줄 세운다	117
12.4 문맥적 검색: 자를 때 맥락을 붙여 둔다	118
12.5 단계가 늘어난다는 것	119
12.6 여러 단계로 풀리지 않는 것	120
12.7 요약	122
12.8 생각해 볼 문제	123
12.9 다시 읽을 때	123
12.10 더 읽을 거리	123
13 온톨로지와 지식 그래프	125
13.1 검색으로 해결되지 않는 문제	125
13.2 지식의 그래프 구현	126
13.3 온톨로지 설계	127
13.4 적용 상황	129
13.5 그래프와 온톨로지로 풀리지 않는 것	130
13.6 요약	131
13.7 생각해 볼 문제	132
13.8 다시 읽을 때	132
13.9 더 읽을 거리	132
14 문서가 텍스트가 아닐 때	133
14.1 코퍼스가 텍스트라는 가정	133
14.2 텍스트가 아닐 경우	133
14.3 텍스트가 아닌 자료들의 활용	134
14.4 그 외 자료	137
14.5 요약	138
14.6 생각해 볼 문제	139
14.7 다시 읽을 때	139
14.8 더 읽을 거리	139
V 루프를 돌리는 일	141
15 에이전트 하네스 설계	143
15.1 하네스: 루프는 어떻게 돌아가는가	143
15.2 상태: 무엇을 남길 것인가	144
15.3 종료: 언제 멈출 것인가	146
15.4 재시작: 어떻게 다시 시작할 것인가	148
15.5 검증: 무엇을 믿을 것인가	149
15.6 하네스를 더 만들지 말아야 할 때	150
15.7 요약	152
15.8 생각해 볼 문제	153
15.9 다시 읽을 때	153
15.10 더 읽을 거리	153

16 단일 에이전트와 다중 에이전트	155
16.1 단일 에이전트의 운영	155
16.2 단일 에이전트로 부족할 경우	156
16.3 다중 에이전트	157
16.4 실무에서의 적용	159
16.5 요약	161
16.6 생각해 볼 문제	162
16.7 다시 읽을 때	162
16.8 더 읽을거리	162
17 에이전트 간 통신	163
17.1 에이전트의 상호작용	163
17.2 무엇을 주고받는가	164
17.3 어떻게 통신하는가	165
17.4 서로 모르는 것들끼리: A2A	167
17.5 설계할 때 주의할 점	168
17.6 요약	171
17.7 생각해 볼 문제	171
17.8 다시 읽을 때	172
17.9 더 읽을거리	172
VI 평가와 관측	173
18 에이전트를 어떻게 평가할 것인가	175
18.1 설계 그 이후에는	175
18.2 좋은 에이전트란	175
18.3 무엇을 평가하는가	176
18.4 어떻게 평가하는가	177
18.5 실제 사례	180
18.6 잘못된 평가	181
18.7 요약	183
18.8 생각해 볼 문제	183
18.9 다시 읽을 때	184
18.10 더 읽을거리	184
19 평가 지표의 설계	185
19.1 공식 지표의 부재	185
19.2 지표 설계	186
19.3 실 사용 예시	188
19.4 잘못된 지표의 설계	189
19.5 요약	192
19.6 생각해 볼 문제	193
19.7 다시 읽을 때	193
19.8 더 읽을거리	193

20 트레이싱과 관측	195
20.1 왜 남겨야 하는가	195
20.2 무엇을 어떻게 남기는가	195
20.3 남겨도 알 수 없는 것	197
20.4 실무에서 어떻게 하는가	198
20.5 남기는 값과 남기지 말아야 할 것	200
20.6 요약	202
20.7 생각해 볼 문제	202
20.8 다시 읽을 때	203
20.9 더 읽을 거리	203
 VII 운영, 안전, 거버넌스	 205
21 인간 개입 설계와 권한 모델	207
21.1 어디에 사람을 세울 것인가	207
21.2 실무자에게 묻는다	208
21.3 되돌릴 수 없는 자리를 찾는다	209
21.4 권한을 어떻게 나누는가	210
21.5 승인 결정을 세워도 남는 것	211
21.6 요약	213
21.7 생각해 볼 문제	214
21.8 다시 읽을 때	214
 22 프롬프트 인젝션과 가드레일	 215
22.1 어디로 들어오는가	215
22.2 무엇을 노리는가	217
22.3 가드레일을 어떻게 세우는가	217
22.4 뚫리는지 확인하는 법	218
22.5 막아도 남는 것	220
22.6 요약	222
22.7 생각해 볼 문제	223
22.8 다시 읽을 때	223
22.9 더 읽을 거리	223
 23 더 나아가기	 225
23.1 스스로 생각해 볼 주제들	225
23.2 OpenClaw	226
23.3 오픈소스 프레임워크	228
23.4 로컬 운영 에이전트	230
23.5 앞으로 지켜봐야 하는 것	232
23.6 맺음	233
 참고문헌	 235

제Ⅰ부

에이전트란 무엇인가

에이전트란 무엇인가

에이전트 설계는 결정 하나에서 갈린다. 다음에 무엇을 할지 코드가 정하는가, 모델이 정하는가. 이 하나가 비용과 지연과 실패의 모양과 평가 방법을 한꺼번에 정한다.

「에이전트」라는 말은 그 결정을 드러내지 않고 덮는다. 검색 결과를 요약하는 챗봇도, 며칠에 걸쳐 코드베이스를 고치는 시스템도 같은 이름으로 불린다. 어떤 시스템이 에이전트인 줄 알아도 그것이 얼마나 비싼지, 어떻게 실패하는지, 무엇으로 평가해야 하는지는 알 수 없다.

「이 시스템은 에이전트인가」는 답해도 달라지는 것이 없다. 「다음 스텝을 코드가 정하는가 모델이 정하는가」는 시스템을 뜯어보면 답이 나오고, 답에 따라 예산도 검증 방법도 달라진다.

1.1 고전적 정의: 지각과 행동의 순환

에이전트라는 개념은 언어 모델보다 훨씬 오래되었다. 1990년대 분산 인공지능 연구에서 Wooldridge and Jennings (1995)가 든 성질 넷 가운데 이 책이 계속 쓰는 것은 둘이다. 자율성은 곧 「다음에 무엇을 할지를 누가 정하는가」라는 물음으로 좁혀지고, 반응성은 「환경이 돌려준 것이 다음 입력이 된다」는 순환의 형태로 남는다.

Russell and Norvig (2020)의 표준 교과서가 그 순환을 간결하게 적는다. 에이전트란 센서로 환경을 지각하고 액추에이터로 환경에 작용하는 것이며, 그 행동은 지각 이력을 행동으로 옮기는 함수로 기술된다. 무엇으로 만들어졌는지는 묻지 않으므로 언어 모델도 이 틀에 그대로 들어간다.

그 순환을 되풀이한 횟수를 T 라 적고 종료 시각이라 부른다. 순환의 자리마다 붙는 이름은 절 1.5가 준다.

1.2 왜 하필 지금인가

에이전트라는 개념이 30년 된 것이라면, 왜 최근에야 실용화되었는가. 능력 셋과 조건 하나가 차례로 갖추어졌다. 지금 만들고 있는 시스템이 어디서 무너지는지 짚을 때 그대로 쓰이는 것들이다.

첫째는 **중간 추론의 외재화**다. Wei et al. (2022)는 모델에게 중간 추론 단계를 생성하도록 유도하면 산술, 상식, 기호 추론 과제에서 성능이 향상됨을 보였다. 이 책에 중요한 것은 정확도가 아니라 모델의 계획이 텍스트로 나온다는 점이다. 이것이 없으면 틀린 답을 받아 들고도 어디서 틀렸는지 짚을 자리가 없다.

둘째는 **추론과 행동의 결합**이다. Yao et al. (2023b)의 ReAct는 추론 흔적과 행동을 번갈아 생성하게 했다. 추론이 행동 계획을 갱신하고, 행동이 다시 추론에 필요한 정보를 가져오는 구조다. 이것이 없으면 계획은 시작할 때 한 번 세워지고 그것으로 끝난다. 도중에 들어온 관측이 계획을 고치지 못한다. 절 1.1의 순환이 실제로 닫힌 것이 이 지점이다. 오늘날 쓰이는 대부분의 에이전트 루프는 ReAct의 변형이거나 그에 대한 반작용이다(제2장).

셋째는 **도구 호출의 신뢰성**이다. Schick et al. (2023)는 모델이 어떤 API를 언제 호출하고 어떤 인자를 넘길지를 스스로 학습할 수 있음을 보였고, 이후 도구 호출은 프롬프트 기법이 아니라 모델과

API의 기본 기능이 되었다. 행동의 공간이 자유 텍스트에서 구조화된 호출의 집합으로 바뀐 것이 이 변화다. 이것이 없으면 모델이 무엇을 하려는지 알아내는 일부가 실패한다.

넷째는 능력이 아니라 조건이다. 컨텍스트 윈도우가 넓어져서, 이제 윈도우 한계는 더 이상 제약이 아니다. 다만 이 조건에는 함정이 달려 있다. 넓어진 윈도우는 더 긴 이력을 담게 해주었지만 동시에 「일단 다 넣는」 설계를 유혹한다. 그래서 실패의 모양도 옮겨 갔다. 예전에는 안 들어가서 실패했고, 지금은 들어갔는데 품질이 떨어져서 실패한다. 왜 그렇게 되는지를 제4장이 받는다.

1.3 워크플로와 에이전트

절 1.1의 틀은 아직 너무 넓다. 여기에 넣으면 고정된 두 단계 파이프라인도 에이전트가 된다.

가장 쓸모 있는 구분선은 Anthropic (2024a)이 제시한 것으로, 제어 흐름을 누가 소유하는가를 기준으로 삼는다. 실행 경로가 코드에 미리 적혀 있고 모델은 각 단계에서 텍스트를 생성하는 부품으로만 쓰이면 워크플로다. 이때 각 스텝의 행동은 하나로 고정되어 있고 종료 시간 T 는 입력과 무관한 상수다. 다음에 무엇을 할지와 언제 멈출지를 모델이 정하고 코드가 그 결정을 실행해 주면 에이전트다. 행동은 도구 집합 $\mathcal{T}$ 위에서 모델이 고르고, T 는 종료 판정 ρ 가 실행 중에 정한다.

왜 하필 소유권인가. 지능이나 과제의 복잡도로 가르는 쪽이 자연스러워 보이지만 그 기준으로는 선이 그어지지 않는다. 같은 모델이 워크플로의 부품으로도 에이전트의 정책으로도 쓰이므로 「얼마나 똑똑한가」는 두 구조를 가리지 못하고, 복잡한 과제는 정교한 워크플로로도 단순한 에이전트로도 풀린다. 소유권은 다르다. 다음 스텝을 코드가 정하는지 모델이 정하는지는 시스템을 뜯어보면 그 자리에서 판정된다.

여기서 갈라지는 것은 예측 가능성이다. 워크플로도 호출 횟수가 실행마다 똑같지는 않다. 재시도가 붙고, 속도 제한에 걸려 기다리고, 검사에 걸린 단계를 다시 돌린다. 다만 그 흔들림의 폭이 훨씬 좁아서 비용과 지연을 미리 어림할 수 있고, 실패 지점이 고정되어 있으므로 디버깅이 쉽다. 에이전트는 그 좁은 폭을 내주는 대신 미리 열거할 수 없는 경로를 다룰 수 있게 된다.

워크플로에도 이름이 있다

워크플로를 「에이전트가 아닌 것」으로만 부르면 설계에 쓸 수 없다. 실무에서 반복해 나타나는 형태에는 각각 이름이 있고, 그 이름이 곧 선택지다. Anthropic (2024a)이 든 다섯을 고객 문의의 한 건을 처리하는 일 위에 얹어 본다.

- **프롬프트 체이닝**. 정해진 순서의 단계로 쪼개고 앞 단계의 출력을 뒤 단계가 받는다. 단계 사이에 코드로 검사를 끼울 수 있다. 요구 사항을 뽑고, 답장 초안을 쓰고, 초안을 사내 문투로 다듬는 세 단계.
- **라우팅**. 입력을 분류해 전용 처리로 보낸다. 하나의 프롬프트가 모든 경우를 떠안느라 무너지는 것을 막는다. 환불인지 기술 문제인지 먼저 가르는 자리.
- **병렬화**. 나눌 것을 미리 알 때 갈라서 동시에 처리하고 결과를 코드로 합친다. 답장 초안과 긴급도 판정과 규정 위반 여부를 한꺼번에.
- **오케스트레이터-워커**. 나뉘 보기 전에는 몇 갈래인지 모를 때 중심 호출이 나누고 결과를 모은다. 얹힌 주문이 셋이면 셋으로, 하나면 하나로.
- **평가자-최적화**. 한 호출이 만들고 다른 호출이 평가해 되돌린다. 무엇이 좋은 결과인지 말로 적을 수 있을 때만 작동한다. 규정 문구를 빠뜨렸는지 검사하고 되돌리는 자리.

셋째와 넷째를 가르는 것은 규모가 아니라 시점이다. 답장과 긴급도와 규정 검사는 문의를 열기 전에 셋이라고 적어 둘 수 있다. 얹힌 주문이 몇 건인지는 열어야 안다.

이 다섯은 에이전트로 가기 전에 먼저 시도할 것들이다. 다섯 중 어느 것으로도 풀리지 않을 때 비로소 제어 흐름을 모델에게 넘길 이유가 생긴다.

표 1.1: 자율성의 스펙트럼. 아래로 갈수록 모델이 소유하는 결정이 늘어나고, 비용의 분산과 검증의 부담이 함께 늘어난다.

수준	모델이 결정하는 것	T 의 성질
단일 호출	출력 내용만	$T = 1$
고정 체인	각 단계의 출력 내용	$T = \text{상수}$
라우팅	어느 분기로 갈 것인가	$T = \text{상수}$, 경로 가변
도구 사용 루프	어떤 도구를 어떤 인자로 호출할 것인가	T 가변, 상한 존재
계획 수립 에이전트	하위 과제의 분해와 순서	T 가변, 분산 큼
다중 에이전트	어떤 에이전트를 언제 생성하고 종료할 것인가	T 가변, 분산 매우 큼

사이에 있는 칸들

실제 시스템은 두 극단 사이 어딘가에 있다. 모델에게 넘길 수 있는 결정이 하나가 아니기 때문이다. 출력의 내용, 분기의 선택, 도구와 인자, 멈출 시점, 새 에이전트의 생성. 이 결정들은 따로따로 넘길 수 있고, 하나를 넘길 때마다 시스템은 한 칸씩 에이전트 쪽으로 옮겨 간다. 모두 쥐고 있으면 단일 호출이고 모두 넘기면 다중 에이전트이며, 그 사이가 비어 있지 않다.

표 1.1이 그 칸들이다. 아래로 내려갈수록 모델이 소유하는 결정이 하나씩 늘고, 오른쪽 옆의 T 가 그 대가를 요약한다. 종료 시각을 미리 아는 일이 곧 비용을 미리 아는 일이기 때문이다.

표의 오른쪽 옆에 적힌 분산은 겪는 쪽에서는 흔들림이다. 행동이 표본인 이상 같은 과제라도 어떤 실행은 3스텝에, 어떤 실행은 30스텝에 끝난다. 아래로 갈수록 그 폭이 넓어지고, 그래서 예산도 평균이 아니라 분포로 잡아야 한다.

예 1.1 (같은 과제, 두 가지 구현). 사내 문서에서 답을 찾아 주는 시스템이다. 워크플로 구현은 질의를 임베딩하고, 상위 몇 개 청크를 검색하고, 그 청크와 질의를 하나의 프롬프트로 묶어 모델을 한 번 부른다. $T = 1$ 이고 비용은 거의 상수다.

에이전트 구현은 모델에게 검색 도구를 주고, 검색할 질의어를 스스로 정하게 하고, 결과가 부족하면 질의를 바꾸어 다시 검색하게 하고, 충분하다고 판단하면 답을 쓰게 한다. T 는 1일 수도 있고 8일 수도 있다.

두 번째는 여러 문서를 거쳐야 답이 나오는 질문, 질의어를 바꾸어야 원하는 문서가 나오는 질문에서 확실히 낫다. 그러나 첫 번째가 이미 잘 푸는 질문에도 몇 배의 토큰을 쓴다. 그래서 설계의 물음은 「어느 쪽이 더 좋은가」가 아니다. 내 질의 분포에서 두 번째가 필요한 질문이 몇 퍼센트인가다.

1.4 에이전트의 구성 요소

그림 1.1에는 상자가 아닌 자리가 몇 개 있다. 환경으로 나가는 화살표, 모델로 돌아오는 화살표, 하네스 상자의 이름표. 그 자리들까지 세면 여섯이 되고, 여섯이 이 책의 부를 하나씩 가져간다.

하나씩 빼 보면 왜 여섯인지가 보인다. 뽑는 것이 없으면 시스템 자체가 없다. 보는 것이 부실하면 매 스텝 눈을 감고 뽑는 셈이고, 통로가 없으면 말만 하는 시스템이며, 판정이 없으면 멈추지 않는다. 밖에 남는 상태를 버리면 세션이 끝날 때마다 처음부터 다시 시작한다. 남기지 않으면 여섯 중 어디가 고장 났는지 알 수 없다.

1. **모델** π_θ . 행동을 산출하는 정책이며, 제3장이 이 상자를 설계에 필요한 만큼만 연다. 특히 비용 구조가 중요하다.
2. **컨텍스트** c_t . 모델이 매 스텝 보는 전부이고, 유한하고, 비싸고, 길어질수록 품질이 떨어진다 (제4장). 조립 상자 안이 여러 층으로 갈린다고 적어 둔 것이 그 층들을 다루는 제2부다.

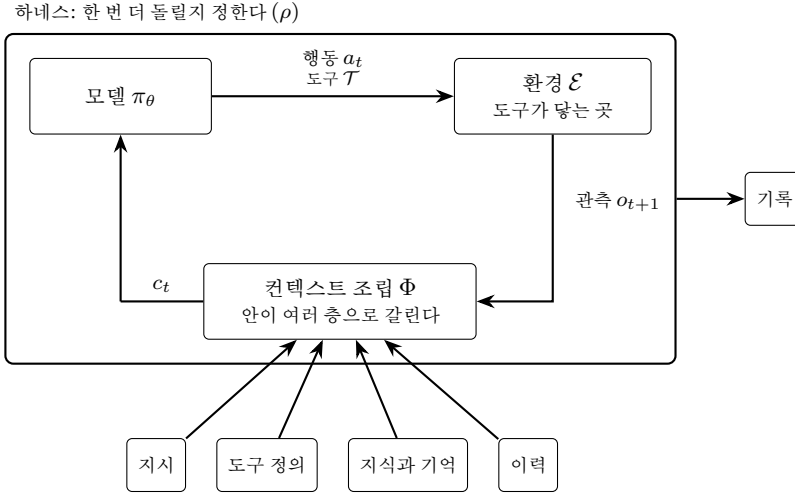


그림 1.1: 에이전트 시스템의 자리들. 모델과 환경이 도는 순환이 있고, 그 순환에 무엇을 실을지는 아래 넷을 받아 조립이 정한다. 순환 밖에는 둘이 있다. 한 번 더 돌릴지 정하는 하네스와, 사람에게로 나가는 기록이다.

3. 도구 $\mathcal{T}$. 상자가 아니라 통로인 데 뜻이 있다. 모델이 환경에 작용하는 길이 이것 하나뿐이므로 도구의 이름과 인자 구조가 사실상 그 도메인의 행위 모형이 되고, 이 설계가 나쁘면 어떤 프롭트으로도 복구되지 않는다(제3부).
4. 루프와 종료 조건 ρ . 언제 멈출 것인가. 실무에서 가장 흔한 사고는 틀린 답이 아니라 멈추지 않는 것이며, 하네스 설계(제15장)의 중심 주제다.
5. 기억. 컨텍스트 밖에 남은 상태이고 세션을 넘어 지속되며, 같은 상자에 든 지식과도 다른 문제다(제10장).
6. 기록. 스텝마다 무엇을 실었고 무엇을 불렀고 무엇이 돌아왔는지 남기는 자리이며, 앞의 다섯 어디에도 흡수되지 않는다. 모델이 보는 것이 아니니 컨텍스트가 아니고, 다음 실행이 쓰는 것도 아니니 기억이 아니다. 사람이 나중에 보려고 남기는 것이다.

1.5 루프는 무엇으로 되어 있는가

앞의 두 절이 말로 짚은 자리들을 형식으로 적으면 루프 한 바퀴가 네 자리로 나뉜다. 각 항목 끝의 괄호는 그 자리를 그림 1.1에서 부르는 이름이다.

1. 지금까지 있었던 일을 한 줄로 엮는다. 이력은 관측과 행동이 번갈아 쌓인 목록인데 모델의 입구는 토큰 열 하나뿐이다. 무엇을 남기고 무엇을 버리며 어떤 순서로 놓을지가 여기서 정해진다. (컨텍스트 조립 Φ)
2. 그 줄을 토큰으로 자른다. 자르는 규칙은 고를 수 없다. 모델을 고르는 순간 함께 달려 온다. 같은 이력이라도 모델을 바꾸면 토큰 수가 달라진다. (토큰나이저 enc)
3. 모델이 다음 행동을 뽑는다. 값이 아니라 표본이다. 같은 것을 넣어도 같은 것이 나온다는 보장이 없다. (언어 모델 π_θ)
4. 환경이 다음 관측을 돌려준다. 이쪽도 정해진 값이 아니다. 같은 명령을 보내도 네트워크는 끊기고 검색 결과는 바뀐다. 환경에는 앞 스텝의 흔적이 쌓여 있으므로, 돌아오는 것은 방금의

행동만이 아니라 그때까지의 이력 전체에 걸린다. (환경 $\mathcal{E}$)

루프는 종료 판정이 서거나 미리 정한 스텝 상한에 닿을 때까지 이 넷을 되풀이한다.

첫째와 둘째를 갈라 둔 것은 주인이 다르기 때문이다. 행동을 정하는 것은 모델 혼자가 아니다. 무엇을 실을지 정하는 쪽과 그것을 자르는 토크나이저와 다음 행동을 뽑는 모델, 셋이 함께 정한다. 그 가운데 우리 쪽 결정인 것은 첫째뿐이고 뒤의 둘은 모델을 고르는 하나의 결정에 묶여 온다. 셋을 뭉뚱그리면 같은 모델을 쓰고도 전혀 다른 에이전트가 되는 이유가 보이지 않는다. 다만 우리 몫이라는 말이 아무렇게나 해도 된다는 뜻은 아니다. 그 결정이 놓인 공간이 생각보다 좁다는 것은 제2부가 보인다.

핵심. 에이전트의 성능은 모델의 성질이 아니라 시스템의 성질이다. 개선의 여지가 큰 쪽은 대개 모델이 아니라 무엇을 컨텍스트에 실을지 정하는 자리다.

표기

그림 1.1의 이름들을 뒤의 장이 계속 부른다. 여기에 셋만 덧붙인다. 도구의 집합은 $\mathcal{T}$, 그 도구들로 모델이 고를 수 있는 행동 전체는 행동의 공간이고, 종료 판정은 ρ 다.

앞의 목록에서 「뽑는다」고 적은 자리가 둘인 것에 주의할 것. 같은 과제를 두 번 돌려도 스텝 수와 비용과 결과가 같으리라는 보장은 여기서 사라진다. 같은 내용인데 언어마다 토큰 수가 왜 달라지는지는 제3장이 받는다.

1.6 언제 에이전트를 쓰지 말아야 하는가

어떤 기법을 언제 쓰지 말아야 하는지를 아는 것이 그 기법을 아는 것의 절반이다. 다음 넷 중 하나라도 「그렇다」면, 에이전트가 아니라 워크플로를 먼저 시도한다.

1. 경로를 종이에 적을 수 있는가. 단계와 분기를 다섯 줄 안에 적을 수 있다면 그 다섯 줄을 코드로 쓰는 편이 언제나 더 싸고 더 안정적이다.
2. 되돌릴 수 없는 작용이 섞여 있는가. 자율성이 높아질수록 사전에 검토되지 않은 행동이 실행될 확률이 올라간다. 여기서 할 수 있는 일은 돌리고 서로 다른 결정이다. 그 행동을 아예 할 수 없게 만들거나, 할 수 있게 두고 그 앞에 사람을 세우거나. 앞쪽은 시스템이 할 수 있는 일의 상한을 내리고 뒤쪽은 실행할 때마다 값을 치른다. 어느 쪽을 어디에 쓸지는 제21장이 받는다.
3. 사용자가 화면 앞에서 기다리는가. 대화형 인터페이스에서 다중 스텝 루프는 사용자가 참아 주는 시간을 금방 넘긴다. 그 시간이 얼마인지는 기다리는 동안 무엇을 보여 주느냐에 따라 달라지므로, 물어야 할 것은 기다리는지 아닌지가 아니라 얼마나 기다릴 수 있는지다.
4. 성공을 자동으로 판정할 수 없는가. 판정할 수 없다면 개선할 방법도 없다. 이 경우 먼저 만들 것은 에이전트가 아니라 평가다.

넷째가 가장 자주 무시된다. 실무에서 실패하는 에이전트 프로젝트의 다수는 모델이 부족해서가 아니라, 무엇이 좋은 결과인지 정의하지 못한 채 시작했기 때문에 실패한다.

1.7 이 책의 구조

제1부는 두 장이다. 이 장이 구조를 세우고, 제2장이 그 구조 위에서 루프의 형태를 고르는 다섯 축을 세운다. 여기까지가 무엇을 만드는가에 대한 답이다.

값의 셸은 제2부가 받는다. 토큰이 예산의 단위이고 길이가 비용이라는 것을 제3장이 세우고, 그 위에서 컨텍스트에 무엇을 실을지가 나머지 세 장의 소재다. 제2장은 그 셸의 결과를 미리 쓰므로, 축의 값이 어디서 나왔는지 궁금해지는 자리가 있으면 제3장을 먼저 펴도 된다.

각 장은 독립적으로 읽을 수 있게 썼지만 제3장은 예외다. 토큰과 길이의 셸은 이후 거의 모든 장의 논거이므로 순서대로 읽기를 권한다.

1.8 요약

한 문장으로. 에이전트는 모델이 아니라 시스템이며, 어떤 결정을 모델에게 넘기는가에 따라 그 시스템이 다룰 수 있는 과제의 폭과 비용의 불확실성이 함께 자란다.

확실했던 것.

- 실제 정책은 Φ 와 enc 와 π_θ 의 합성이고, 셋 중 우리 쪽 결정인 것은 Φ 뿐이다(그림 1.1). 개선의 여지는 대개 그쪽에 있고, 그 자리가 얼마나 좁은지는 제2부가 보인다.
- 워크플로와 에이전트를 가르치는 것은 지능이 아니라 제어 흐름의 소유권이다. 행동의 선택지와 종료 시각을 코드가 고정하면 워크플로, 모델이 정하면 에이전트다.
- 에이전트로 가기 전에 시도할 워크플로가 다섯 있다. 프롬프트 체이닝, 라우팅, 병렬화, 오케스트레이터-워커, 평가자-최적화.
- 자율성은 이분법이 아니라 스펙트럼이며, 모델이 소유하는 결정이 늘수록 비용의 분산과 검증 부담이 함께 커진다(표 1.1).

아직 답하지 않은 것.

- enc 와 종료 판정 ρ 는 이름만 두었다. 같은 내용인데 언어마다 토큰 수가 왜 달라지는지는 제3장이, 종료 판정을 어디에 두어야 하는지는 제15장이 받는다.
- 루프가 길어질수록 값이 얼마나 빨리 커지는지는 세지 않았다. 그 셈과 캐시는 제3장이, 누적 자체를 줄이는 압축과 격리는 제6장과 제16장이 받는다.
- 컨텍스트가 길어질수록 품질이 떨어진다고 말했지만 근거는 대지 않았다. 제4장이 받는다.
- 평가 수단이 없으면 개선도 없다고 했지만, 평가를 어떻게 만드는지는 제6부가 받는다.

오늘 할 것. 다음 호출을 만드는 코드를 열어, 그 분기가 코드에 적혀 있는지 모델의 출력을 읽는지 본다. 코드에 적혀 있다면 이 책의 나머지 대부분은 아직 필요하지 않다(절 1.3). 다음으로 표 1.1에서 지금 만들고 있는 것이 어느 행인지 짚는다. 행이 내려갈수록 스텝 수의 폭이 넓어지고, 그 폭이 곧 예산의 불확실성이다. 마지막으로 절 1.6의 넷을 자기 과제에 대 본다. 하나라도 「그렇다」면 워크플로를 먼저 시도한다.

1.9 생각해 볼 문제

1. 모델이 사전에 정의된 세 개의 분기 중 하나를 고르는 라우터는 워크플로인가 에이전트인가. 이 경계 사례가 구분의 결함인지, 의미 있는 중간 지대인지 논하라.
2. 표 1.1의 마지막 두 행에서 스텝 수의 분산이 커지는 이유는 무엇인가. 그 분산을 줄이려면 모델에게서 어떤 결정을 도로 가져와야 하는가.
3. 예 1.1의 두 설계에 대해, 에이전트 구현이 워크플로 구현보다 기대 효용이 높아지는 조건을 질의 분포와 단가의 함수로 표현하라.
4. 절 1.6의 네 물음 중 실무에서 가장 자주 무시되는 것은 무엇이라고 생각하는가. 그 이유는 조직적인 것인가, 기술적인 것인가.

1.10 다시 읽을 때

- 절 3.1을 읽고 표 1.1로 돌아오면, 스텝 수의 분산이 왜 돈으로 바뀌는지가 보인다. 매 스텝 이력 전체를 다시 읽으므로 스텝 하나가 늘어날 때마다 늘어나는 값 자체가 커진다.
- 절 4.4를 읽고 「길어질수록 품질이 떨어진다」로 돌아오면, 그 구절이 유효 컨텍스트 길이라는 측정할 수 있는 양으로 바뀌어 있다. 비용과 별개로 품질에도 한계 길이가 있다.
- 제2장을 읽고 표 1.1로 돌아오면, 수준으로만 나눈 각 행이 구체적인 루프의 형태로 채워진다. 추론과 행동의 교대가 도구 사용 루프의 기본형이고, 계획 후 실행은 분산을 계획 시점으로 옮기는 선택이다.
- 제15장을 읽고 종료 판정 ρ 로 돌아오면, 루프 안의 이름으로 적어 둔 것이 사실은 루프 바깥의 결정임이 보인다. 그 이동이 하네스의 첫 번째 결정이다.

1.11 더 읽을거리

에이전트 개념의 고전적 정식화는 Wooldridge and Jennings (1995)와 Russell and Norvig (2020)에 있다. 후자는 이 책이 다루지 않는 탐색과 계획과 의사결정 이론의 배경을 준다.

언어 모델 에이전트의 직접적 계보는 Wei et al. (2022), Yao et al. (2023b), Schick et al. (2023) 세 편으로 요약된다. 세 편 모두 짧고, 원문을 읽을 가치가 있다.

워크플로와 에이전트의 구분과 다섯 가지 패턴은 Anthropic (2024a)에 나온다. 후속 자료인 Anthropic (2025c)은 Φ 를 「컨텍스트 엔지니어링」이라 부른다(제4장).

추론과 계획 패턴 지도

제1장은 제어 흐름의 소유권으로 워크플로와 에이전트를 갈랐다. 모델에게 넘기기로 했다면 다음 물음은 하나다. 그 루프를 어떤 형태로 돌릴 것인가.

형태에는 이름이 있다. **chain of thought**, **plan-and-execute**, **ReAct**, **self-reflection**, **best-of-N**, **tree search**. 해마다 변형이 더해지지만 뼈대는 이 여섯이다.

이름을 외우는 것으로는 아무 결정도 내리지 못한다. 목록은 목록이 늘어나는 속도만큼 낡는다. 그래서 이 장이 하려는 것은 여섯을 소개하는 일이 아니라 여섯을 가르는 물음을 세우는 일이다. 물음이 서면 내년에 나올 일곱째에도 같은 것을 물을 수 있다.

나열하는 순서에도 규칙이 있다. 비용이 저렴한 것부터다. 어느 패턴이든 능력을 호출을 늘려서 산다. 그러므로 먼저 시도할 것은 언제나 덜 늘리는 쪽이고, 더 늘리는 쪽은 덜 늘리는 쪽으로 안 될 때 비로소 이유가 생긴다(Anthropic, 2024a).

2.1 어떤 패턴들이 있는가

chain of thought. 답을 내기 전에 생각을 텍스트로 꺼내 놓게 하는 것이다. Wei et al. (2022)는 최종 답에 앞서 중간 추론 단계를 생성하게 하면 산술과 상식과 기호 추론 과제의 성능이 오른다는 것을 보였다. 그때는 프롬프트로 시키는 것이었고 지금은 대개 시키지 않아도 일어난다. 호출 수는 그대로이고 늘어나는 것은 출력의 길이뿐이다. 언제 쓰는가. 사실상 전부. 이것은 독립된 패턴이라기보다 아래 다섯에 들어가는 재료다. **ReAct**가 「추론」이라 부르는 것이 이것이고 **tree search**가 노드로 삼는 「사고」도 이것이다.

plan-and-execute. 첫 호출이 할 일의 목록을 만들고 나머지 호출이 그 목록을 하나씩 지운다. Wang et al. (2023)는 **chain of thought**만 지시했을 때 남는 함정을 계산 오류와 단계 누락과 의미 이해 오류로 나누고, 그중 단계 누락을 겨냥해 전체 과제를 더 작은 하위 과제로 먼저 쪼개게 했다. 원문의 이름은 **plan-and-solve**이고, 같은 구조를 루프에 얹은 것이 **plan-and-execute**다. 언제 쓰는가. 하위 과제의 구조가 실행 전에 보이고 몇 스텝이 걸릴지 미리 적어야 할 때. 다만 계획이 강한 것은 빠뜨림에 대해서다. 과제를 잘못 이해했다면 그 오해는 계획으로 자리를 옮길 뿐 사라지지 않는다.

ReAct. 생각 한 번과 행동 한 번을 번갈아 한다. Yao et al. (2023b)는 추론 흔적과 행동을 번갈아 생성하게 했다. 추론이 행동 계획을 세우고 갱신하며, 행동이 밖에서 정보를 가져와 다음 추론에 공급한다. 행동을 모델이 고르고 종료를 실행 중에 정한다는 제1장의 에이전트 조건을 처음으로 채우는 형태이기도 하다. 언제 쓰는가. 환경과 주고받을 것이 있고 몇 번 주고받아야 할지 미리 알 수 없을 때. 나머지 셋은 이 기본형 위에 얹는 층이다.

self-reflection. 실패한 시도를 스스로 되짚어 한 편의 글로 적어 두고, 컨텍스트를 비운 뒤 그 글만 들고 다시 시작한다. Shinn et al. (2023)의 **Reflexion**은 과제의 피드백 신호를 언어적 반성으로 바꾸어 에피소드 기억에 보관하고 다음 시도가 그것을 참조하게 한다. 안쪽에는 대개 **ReAct** 루프가 돈다. 언제 쓰는가. 성패를 자동으로 판정할 수단이 있고 재시도가 허용될 때. 판정이 없으면 반성할 거리 자체가 생기지 않는다.

best-of-N. 같은 과제를 처음부터 여러 번 따로 풀고 그중 하나를 고른다. 갈래가 중간에서 갈라지지 않고 뿌리에서 갈라져 끝까지 서로 만나지 않는다. 언제 쓰는가. 고르는 수단이 있고 시도들이

표 2.1: 여섯 패턴이 다섯 축 위에 앉는 자리. 값이 하나로 정해지지 않는 칸이 아래에서 다루는 자리다.

	세는 단위와 정해지는 때	남기는 것	길이 한계와 전출 것	앞부분	실패가 남기는 것
chain of thought	스텝 하나, 실제 시점	설정에 달렸다	호출 하나	뒤에만 덧붙는다	설정에 달렸다
plan-and- execute	계획과 하위 과제, 계획 시점	계획만큼의 준고정부	두는 자리가 정한다	두는 자리가 정한다	계획
ReAct	스텝 수 가변, 실행 종료	사고와 행동과 관측	루프 전체	뒤에만 덧붙는다	교대 기록 전부
self- reflection	시도마다 재시작, 시도 수는 상한	반성문만 이월	시도 하나	시도 안에서만 보존	반성문
best-of-N	시도 수 고정, 실제 시점	서로 나누지 않는다	시도 하나	고정부만 공유	버린 시도까지 전부
tree search	노드 수, 실행 중	경로에만 쌓인다	경로 하나	형제끼리 공유	나무 전체

서로의 상태를 건드리지 않을 때. 벌리는 두 패턴 가운데 이쪽이 비용이 저렴하므로 **tree search**를 생각하기 전에 먼저 시험한다.

tree search. 한 상태에서 여러 후보를 벌려 놓고, 유망한 것을 골라 내려가고, 막히면 다른 가지로 돌아온다. Yao et al. (2023a)의 출발점은 언어 모델이 되돌리기 없는 한 줄 쓰기에 머물러 있어서 탐색이나 전략적 예측이 필요한 과제와 초기 결정이 결정적인 과제에서 부족할 수 있다는 관찰이다. 사고 트리는 문제 해결의 중간 단계 역할을 하는 텍스트 단위를 노드로 두고 그 위에서 탐색한다. Zhou et al. (2024)의 LATS는 이 구조를 행동까지 넓히면서, 가지의 평가에 모델의 자기 평가만이 아니라 환경의 관측을 들인다. 언제 쓰는가. 초기 결정이 뒤를 결정적으로 가르고, 행동을 되돌리거나 격리할 수 있고, 가지를 평가할 기준이 있을 때. 셋 중 하나라도 없으면 고를 것이 아니다.

이 여섯을 무엇으로 가르는가

여섯이 전부 같은 것을 산다. 능력이다. 그리고 전부 같은 것으로 낸다. 호출이다. 그러면 새 패턴을 만났을 때 물을 것은 다섯으로 좁혀진다.

1. 무엇을 세는 단위로 삼고, 그 수가 언제 정해지는가. **chain of thought**는 스텝을 세고 그 수는 코드를 쓸 때 이미 정해져 있다. **tree search**는 노드를 세고 그 수는 돌려 봐야 안다.
2. 스텝이 무엇을 남기는가. 남긴 것은 다음 스텝의 입력이 되어 매 호출 다시 읽힌다. 그래서 스텝 하나가 남기는 양이 곧 스텝 하나의 값이다.
3. 길이의 한계와 전출 것이 무엇인가. 루프 전체인가, 시도 하나인가, 경로 하나인가. 모델이 쓸 만한 일을 해내는 길이에는 선언된 윈도우보다 짧은 한계가 있고, 무엇이 그 한계에 먼저 부딪히는지가 패턴마다 다르다.
4. 앞에서부터 똑같이 이어지는 구간을 그대로 두는가. 그대로 두면 다시 읽히는 값이 크게 짝인다. 흔들면 그 지점부터 뒤가 전부 무효가 된다.
5. 실패하면 무엇이 남는가. 진단할 재료다. 남지 않은 구간은 영영 없다.

첫째가 나머지와 다르다. 패턴이 바꾸는 것은 T 의 값만이 아니라 크기와 분산과 확정 시점이다. 실행 전에 아는 값과 첫 스텝이 끝나야 아는 값과 루프가 끝나야 아는 값은 예산 계획에서 전혀 다른 설계다. 여섯이 호출을 어떤 모양으로 늘리는지, 그리고 그 수를 실행 전에 알 수 있는지가 그림 2.1에 나란히 있다.

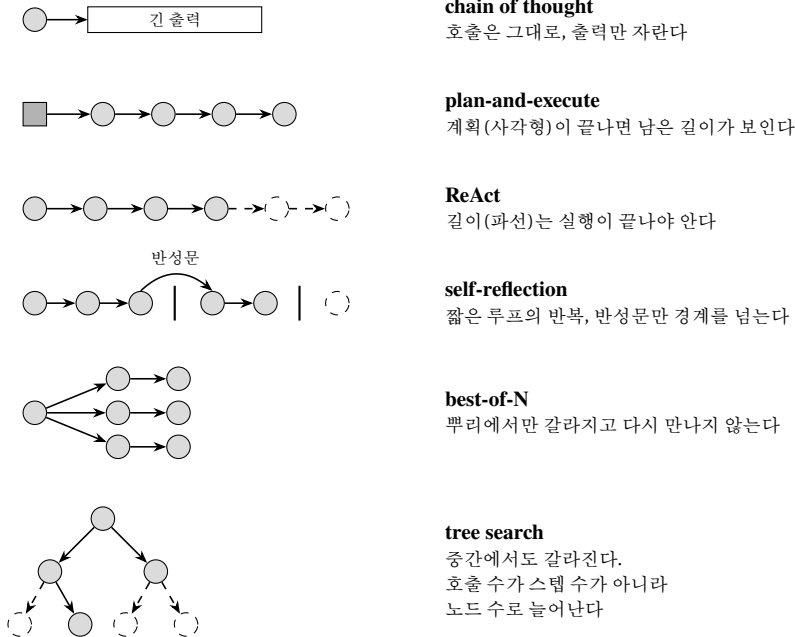


그림 2.1: 패턴이 만드는 호출의 모양. 원 하나가 모델 호출 하나, 파선 원은 실행 전에 개수를 알 수 없는 호출이다.

2.2 선택의 결정 절차

첫 번째 분기는 패턴 이전의 질문이다. 워크플로로 충분한가. 모든 패턴은 스텝이나 노드를 늘려서 능력을 산다. 실행 경로를 미리 열거할 수 있는 과제라면 살 것이 없다. 그 경로를 코드로 적는 쪽이 언제나 더 싸다(절 1.6). 이 분기를 건너뛰고 패턴부터 고르는 것이 실무에서 가장 흔하고 가장 비싼 실수다. 환경과 주고받을 것이 없다면 그다음도 루프의 문제가 아니라 호출 하나의 문제다.

그림 2.2가 전체 절차다. 각 분기에서 사는 것과 치르는 것은 이렇다.

1. **plan-and-execute**를 읽는다. 사는 것은 스텝 수의 분산이 계획 시점으로 옮겨 오는 것이고, 치르는 것은 계획 경직의 위험에 더해 앞의 구성에서는 준고정부만큼의 여유, 뒤의 구성에서는 계획을 읽어 오는 스텝이다. 계획을 컨텍스트에 돌지 밖에 돌지를 함께 정한다.
2. **self-reflection**을 읽는다. 사는 것은 실패의 자산화이고, 치르는 것은 총 스텝의 시도 수 배와 J 에 대한 의존이다. 사려는 것이 길이의 여유뿐이라면 이 층은 필요 없다.
3. **best-of-N**으로 벌린다. 시도 수를 우리가 정하므로 예산이 설계 시점에 확정되고, 지연은 병렬로 돌리면 한 배에 가깝다. 아래로 내려가기 전에 이쪽부터 시험한다.
4. **tree search**로 벌린다. 행동을 되돌리거나 격리할 수 있고, 상태를 평가할 수단이 있고, 노드 수의 예산을 감당할 수 있을 때에만 고른다. 하나라도 아니라면 고를 것이 아니다.

절차가 「고른다」가 아니라 「없는다」의 연속인 데는 근거가 있다. 두 선택지가 배타적이라면 같은 변수를 서로 다른 쪽으로 움직이려 해야 한다. **plan-and-execute**가 움직이는 것은 스텝 수가 확정되는 시점이고, **self-reflection**이 움직이는 것은 길이의 한계와 비교되는 것이며, **tree search**가 움직이는 것은 세는 단위 그 자체다. 움직이는 변수가 서로 다르면 충돌할 자리가 없다. Zhou et al. (2024)의 LATS가 그 실물이다. 추론과 행동과 계획을 결합하고 **self-reflection**까지 포함하는 그

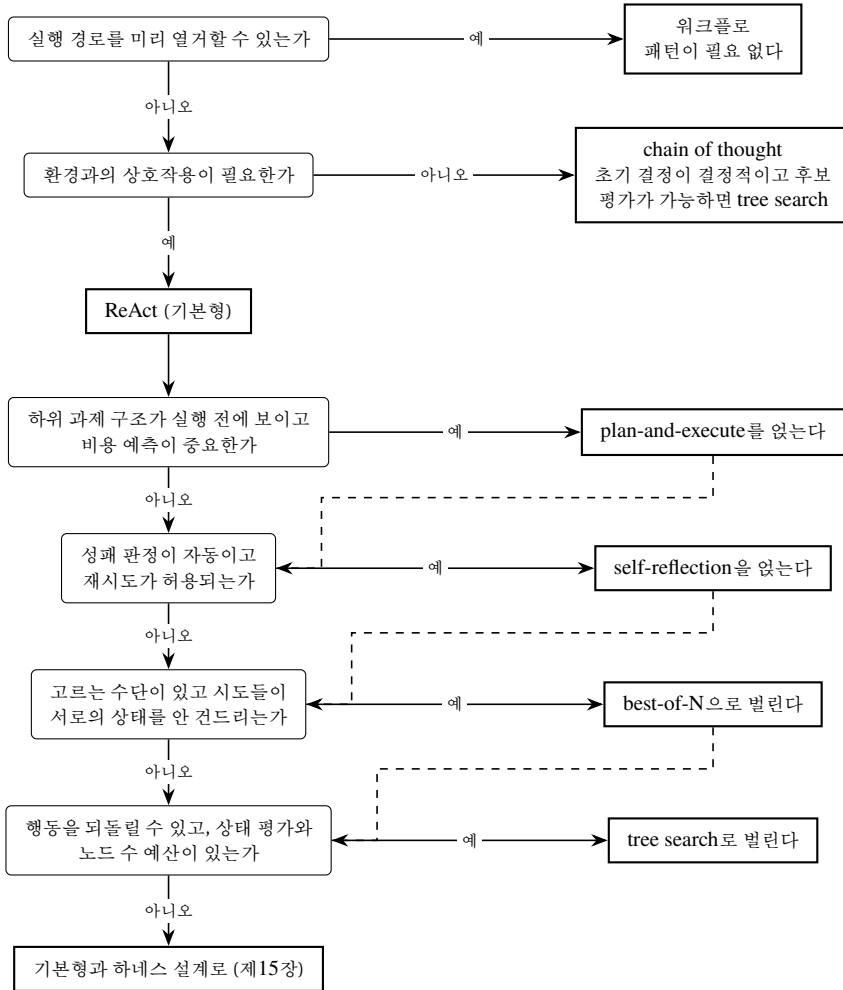


그림 2.2: 패턴 선택의 결정 절차. 오른쪽 상자는 종착점이 아니라 기본형 위에 겹치는 층이며, 층을 엮은 뒤에도 파선을 따라 다음 질문으로 내려간다.

구조는 이 절차의 층들이 한 시스템에 동시에 쌓인 사례다.

층을 엮을 때 건너낼 조건을 함께 적어 둔다. 엮은 층은 저절로 떨어지지 않는다. 판정 신호가 약해져 self-reflection이 헛돌아도 반성 루프는 남아 시도 수 배의 스텝을 계속 청구한다. 하위 과제 구조가 사라진 과제에서도 계획 스텝은 남아 준고정부를 계속 실행한다. 절차의 질문이 「예」였기 때문에 엮었다면, 그 답이 「아니오」가 되는 순간이 건너낼 시점이다. 그 감시는 층을 엮은 사람 말고는 아무도 하지 않는다.

2.3 패턴이 실패하는 방식

실무의 가장 흔한 사고는 틀린 답이 아니라 멈추지 않는 것이라고 절 1.4에서 했다. 패턴은 그 사고의 확률을 저마다 다른 방식으로 바꾼다.

- **chain of thought**는 유창하게 틀린다. 계획으로 고쳐지는 것은 단계 누락뿐이고 계산 오류와 의미 이해 오류는 사슬 위에 그대로 남는다. 최악의 형태는 추론이 결론을 검증하는 대신 정당화하는 서사가 될 때다. 남은 글은 많은데 신호는 적다.
- **plan-and-execute**는 경직과 동요 사이에서 실패한다. 관측이 계획의 전제를 무너뜨렸는데 계획을 고수하면 성실하게 틀린다. 반대로 관측마다 계획을 다시 쓰면 계획 비용이 매번 다시 든다. 계획을 컨텍스트에 둔 구성에서는 무효화까지 반복되어 사실상 비싼 **ReAct** 루프가 된다. 남은 설계 변수는 계획의 품질이 아니라 재계획의 빈도를 정하는 규칙이다. 쓰이는 형태는 대개 셋 중 하나다. 계획이 명시한 전제를 관측이 어겼을 때만, 같은 하위 과제가 정해 둔 횟수만큼 실패했을 때, 계획 항목이 다 소진되었는데 종료 판정이 나지 않았을 때.
- **ReAct**는 제자리를 걷는다. 같은 도구를 같은 인자로 다시 부르고, 이미 확인한 사실을 다시 확인한다. 저하가 스텝을 늘리고 늘어난 스텝이 저하를 키운다. 길이의 한계를 넘은 루프는 조금씩 나빠지는 것이 아니라 가속한다.
- **self-reflection**은 멈추지 못하는 첫 번째 사례다. J 가 실패를 성공으로 오판하면 조기에 멈추고, 성공을 실패로 오판하면 끝난 과제를 계속 되풀이한다. 판정의 오류율이 곧 패턴의 오류율이다. 반성이 원인을 잘못 짚으면 기억은 자라는데 행동은 변하지 않는다. 게다가 반성문이 시도마다 자라면 각 시도의 출발 크기가 커져 같은 되먹임이 시도 단위로 재현된다.
- **tree search**는 멈추지 못하는 두 번째 사례다. 가치를 평가하는 모델이 가치를 만든 모델과 같으므로 과대평가된 가치는 계속 유망해 보인다. 그리고 나무는 언제나 한 노드 더 자랄 수 있고, 확장의 기대 이득이 정확히 0이 되는 일은 드물다. 신뢰할 수 있는 종료는 둘뿐이다. 해를 찾거나 예산이 소진되거나다.

왜 판정이 바깥에서 와야 하는지는 답안을 쓴 사람이 그 자리에서 자기 답안을 채점하는 장면을 그려 보면 보인다. 답을 틀리게 만든 바로 그 이해가 채점 기준도 만든다. 계산을 잘못 이해해서 틀렸다면 채점에서도 같은 계산을 옳다고 볼 것이고, 그래서 오답일수록 쓴 사람에게는 정답처럼 보인다. 고칠 이유를 못 찾는 것이 아니라 찾을 수 없는 자리에서 있는 것이다. **best-of-N**의 고르는 일을 같은 모델에 맡길 때도 같은 일이 일어난다.

self-reflection의 실패에는 구조적 논증 이상의 근거가 있고, 갈림길은 외부 피드백의 유무다. Huang et al. (2024)는 외부 피드백에 의존하지 않고 모델이 자기 능력만으로 최초 응답을 고치려는 시도를 내재적 자기 교정이라 부르고, 추론 과제에서 모델이 그 교정에 어려움을 겪으며 때로는 교정 뒤에 성능이 오히려 떨어진다는 것을 보인다. 어려움을 보인 것은 판정과 반성을 모두 모델 자신이 맡는 경우다. 그러므로 이 인용이 밝히는 것은 패턴의 무용이 아니라 의존이다.

다섯 실패의 공통점은 방향이다. 모두 루프를 늘리는 쪽으로 실패한다. 그리고 비종료는 패턴 안에서 해결되지 않는다. 패턴은 계속할 이유를 만드는 데 능하고, 멈출 이유는 바깥에서 와야 하기 때문이다. 종료 판정 ρ 와 그것을 감싸는 구조가 그 바깥이며 제15장의 주제다. 그래서 패턴을 고르는 일에는 그 패턴의 정지 조건을 함께 고르는 일이 포함된다. **self-reflection**을 엮었다면 시도 횟수의 상한이, **tree search**를 벌였다면 노드 수의 예산이 그 순간 정지 조건 목록에 함께 올라야 한다.

2.4 원인이 패턴 밖에 있을 때

패턴을 바꿔도 아무것도 나아지지 않는 경우가 있다. 원인이 루프의 형태에 있지 않았기 때문이다. 다음 다섯 중 하나에 해당하면 더 정교한 패턴이 아니라 더 단순한 쪽을 골라야 한다.

1. **분포의 대부분을 단순한 쪽이 푼다.** 정교한 패턴이 이기는 질의의 비율이 얼마인가. 그 비율을 위해 나머지 전부에 몇 배를 지불하는가. 이 둘이 유일하게 의미 있는 비교다(예 1.1).
2. **성패를 판정할 수단이 없다.** **self-reflection**과 **tree search**는 판정을 입력으로 먹는 패턴이다. 반성할 신호가 없고 가치를 평가할 기준이 없으면 성립 자체가 안 된다. 개발 순서는 패턴이 아니라 평가가 먼저다(절 1.6).

3. 행동을 되돌릴 수 없다. **tree search**는 배제되고 재시도도 위험해진다. 갈래들이 상태를 나누지 않는 **best-of-N**만 남는데 그것도 각 갈래가 읽기만 할 때다. 자율성을 늘리는 패턴 대신 인간 개입 설계가 먼저다.
4. **지연 예산이 빠듯하다**. **self-reflection**은 시도 횟수만큼, **tree search**는 병렬화를 해도 깊이만큼 지연을 곱한다. 사용자가 기다리는 대화형 인터페이스에서 이 배수는 대개 감당되지 않는다.
5. 도구가 모호하거나 검색이 못 찾는다. 이런 시스템에 패턴을 얹으면 실패가 사라지는 것이 아니라 정교해진다. 게다가 정교한 패턴일수록 더 많은 관측과 평가 인프라를 요구한다.

다섯을 훑고 나면 공통점이 보인다. 패턴을 고르는 일은 루프 안쪽의 결정처럼 보이지만, 판단의 입력은 전부 루프 바깥에 있다. 질의 분포, 판정 수단, 환경의 가역성, 지연 예산, 도구와 검색의 품질. 어느 것도 패턴을 정교하게 만든다고 함께 생겨나지 않는다.

2.5 요약

한 문장으로. 어느 패턴이든 능력은 스텝이나 노드를 늘려 산다. 그래서 첫 질문은 언제나 그 지불이 필요한 과제인가다.

확실했던 것.

- 새 패턴은 다섯 축으로 관정한다. 세는 단위와 확정 시점, 스텝이 남기는 것, 길이의 한계와 견줄 것, 앞부분, 실패가 남기는 것이다(표 2.1). 그 축을 움직이는 것이 루프의 형태만은 아니다. 생각의 깊이를 정하는 요청 설정이 셋을 함께 움직이면서 구조를 바꾸지 않으므로, 순서는 그 짝 쪽이 먼저다.
- chain of thought는 호출 수를 바꾸지 않고 출력만 늘린다. 다만 둘째 축과 다섯째 축의 값은 이 패턴의 성질이 아니라 설정에 달렸다. plan-and-execute도 마찬가지로 계획을 컨텍스트에 두는지 밖에 두는지에 따라 셋째 축과 넷째 축의 값이 갈린다.
- 재시작하면 비용이 줄어든다. 매 스텝 다시 읽는 이력이 짧아지기 때문이다. 다만 이력을 문턱마다 줄여 다시 신는 장치가 재시작 없이 같은 여유를 사므로, self-reflection이 고유하게 사는 자리는 과제를 처음부터 다시 푸는 것이 말이 될 때다.
- 분기는 노드 수로 세어야 하고, 비용과 지연이 갈라진다(절 2.1). 다만 벌리는 방법이 tree search만은 아니다. 뿌리에서 갈라져 서로 만나지 않는 best-of-N은 깊이가 없어 폭발하지 않고, 갈래가 같은 상태를 공유하지 않으므로 되돌리기의 요구도 생기지 않는다.
- 패턴들의 실패는 공통적으로 루프를 늘리는 방향이고, self-reflection과 tree search는 자연스러운 종료가 없다. 패턴을 고르는 일에는 그 실패 방식이 요구하는 정지 조건을 함께 고르는 일이 포함된다(그림 2.2).

아직 답하지 않은 것.

- 멈출 이유가 와야 할 「바깥」이 무엇인지는 이름만 세웠다. 종료 판정과 그것을 감싸는 구조, 그리고 상한에 걸려 멈춘 실행이 사용자에게 무엇을 돌려주는가는 제15장이 받는다.
- 시도 사이에 넘기는 기억이 자랄 때의 계산은 제15장이, 그 기억을 일정 크기 안에 붙들어 두는 요약은 제6장이 받는다.
- 스텝이 남기는 것을 고르고 다듬어 스텝당 증분을 줄이는 일은 제4장부터 제6장이 받는다. 그 썸의 근거는 제3장이 세운다.
- tree search는 한 컨텍스트 안에서 벌리는 나무였다. 갈래마다 다른 일을 시키며 여러 컨텍스트로 벌리는 쪽은 제16장이 받는다.
- 판정 신호 J 를 어떻게 만드는가는 입력으로만 다루었다. 평가의 설계는 제6부가 받는다.

오늘 할 것. 지금 도는 루프에 여섯 중 하나의 이름을 붙이고, 기본형 위에 얹은 층이 몇인지 센다(절 2.1). 다음으로 예산서를 열어 단위가 그 이름과 맞는지 맞춰 본다. best-of-N이나 tree search를 돌리면서 예산이 스텝이나 지연으로 적혀 있으면 시도 수나 노드 수로 고쳐 적는다(절 2.1). 마지막으로 얹혀 있는 층마다 걷어낼 조건을 한 줄씩 붙인다. 없을 때 「예」였던 질문이 「아니오」가 되는 순간을 적어 두는 것이다(절 2.2).

2.6 생각해 볼 문제

1. chain of thought는 호출 수를 바꾸지 않으면서도 생각을 이력에 남기면 루프가 길이의 한계에 더 빨리 닿게 만든다. 그 경로를 한 문단으로 적고, 남기지 않는 설계에서 다섯 축 각각이 무엇을 얻고 무엇을 잃는지 함께 적어 보라.
2. 재시작으로 비용이 줄어드는 것은 반성문의 크기가 커지지 않을 때다. 반성문이 시도마다 조금씩 길어진다면 총비용과 각 시도가 쓸 수 있는 스텝 수는 어떻게 달라지는가. 반성문을 요약해 크기를 붙들어 두는 설계는 어느 축에서 대가를 치르는가.
3. 노드 수의 예산이 고정일 때, 넓고 얇은 나무와 좁고 깊은 나무 중 어느 쪽이 유리한지는 과제의 어떤 성질이 정하는가. 그 성질이 없는 과제라면 왜 tree search 대신 best-of-N이 나은가.

2.7 다시 읽을 때

- 제15장을 읽고 시도 사이에 남기는 반성문으로 돌아오면, 「커지지 않는 반성문」이라는 전제가 인제라는 이름을 얻고 그 자체가 설계 대상이 되어 있다. 인제가 시도마다 자라면 재시작으로 줄여 둔 비용이 그만큼 되돌아오므로, 인제는 누적이 아니라 갱신이어야 한다는 것이 그 장의 결론이다.
- 제2부를 읽고 다섯 축으로 돌아오면, 컨텍스트를 다루는 기술 전체가 둘째 축 하나를 움직이는 일이었다는 것이 보인다. 프롬프트와 컨텍스트 엔지니어링과 압축은 스텝당 증분을 줄이는 서로 다른 깊이의 손이며, 셋 다 패턴 선택이 정해 둔 증분의 상한 안에서 움직인다.
- 제6장을 읽고 self-reflection으로 돌아오면, 시도 경계에서 이력을 통째로 버리고 반성문만 남기는 구조가 압축의 거친 형태였음이 보인다. 원문은 버리고 결론은 남긴다는 그 장의 기준이 반성문에도 그대로 적용된다.
- 제16장을 읽고 벌리는 두 패턴으로 돌아오면, 한 컨텍스트 안에서 갈래를 벌리는 일과 컨텍스트 자체를 여럿으로 가르는 일의 경계가 보인다. 앞의 둘은 앞부분을 공유하지만 격리된 자리는 그것마저 나누지 않는다.

2.8 더 읽을거리

chain of thought와 ReAct의 원문인 Wei et al. (2022)와 Yao et al. (2023b)는 제1장에서 이미 권했다. 다시 읽으면 두 논문이 말하지 않는 것, 곧 비용의 축이 보인다.

계획 쪽은 Wang et al. (2023)를 보라. 여기 가져온 것은 구조다. 원문의 가치는 chain of thought의 실패를 세 갈래로 분류한 데 있고, 어떤 처방이 어떤 실패에만 듣는지를 가리는 감각이 그 분류에서 나온다.

self-reflection은 Shinn et al. (2023)이 출발점이다. 절 2.3에서 지적인 판정 의존성을 원문의 구성에서 직접 확인해 보기를 권한다. 그 한계 쪽은 Huang et al. (2024)가 다루는데, 외부 피드백이 있는 경우와 없는 경우를 구분하면서 읽어야 한다. 그 구분을 놓치면 self-reflection 전체에 대한 반증으로 오독하게 된다.

tree search는 두 편으로 충분하다. Yao et al. (2023a)는 좌에서 우로 진행하는 의사결정이라는 제약을 사고 단위의 탐색으로 넘고, Zhou et al. (2024)는 여기에 행동과 계획과 외부 피드백을 더한다. 나무가 자라는 동안 무엇이 비용을 내는지 절 2.1의 눈으로 세며 읽으라.

제Ⅱ부

컨텍스트를 다루는 기술

제3장

토큰: 예산의 단위

제1장의 루프는 한 바퀴 돌 때마다 값을 묻다. 그 값을 세는 단위가 있고, 이 장은 그 단위 하나만 세운다.

모델에 보내는 컨텍스트는 한 자도 무료가 아니다. 시스템 프롬프트도, 도구 설명문도, 방금 도구가 돌려준 파일의 내용도 값을 묻다. 보기 좋으라고 넣은 들여쓰기까지 값을 묻다.

에이전트를 만드는 일에서 설계만큼 자주 실패하는 자리가 비용이다. 그래서 이 장은 셋만 말한다. 왜 그 단위를 알아야 하는가. 같은 내용도 언어마다 값이 다르다는 것. 그리고 모델을 바꾸는 날 그 값을 처음부터 다시 세야 한다는 것. 셋 다 이론이 아니라 청구서의 이야기이고, 셋 다 프롬프트를 한글자도 쓰기 전에 이미 정해져 있다.

3.1 토큰화: 무엇이 세어지는가

청구서는 글자를 세지 않는다. 낱말도 세지 않는다. 그래서 프롬프트가 얼마나 긴지를 눈으로 가늠하는 일은 처음부터 빗나간다. 세는 단위가 따로 있고, 그 단위를 정하는 것은 우리가 아니다.

먼저 무엇이 세어지는지부터 본다. 한 번의 호출에 실려 가는 것은 방금 쓴 사용자의 말만이 아니다. 시스템 프롬프트가 실리고, 도구 정의가 통째로 실리고, 지금까지 오간 관측과 행동이 전부 실린다. 도구가 돌려준 300줄짜리 파일도 그 안에 있다. 실린 것은 전부 세어진다.

그리고 한 번만 세어지지 않는다. 모델 호출은 이전 호출을 기억하지 않는다. 그러니 열 번째 스텝은 아홉 번째까지 쌓인 것을 통째로 다시 읽는 일이다. 스무 번째 스텝은 열아홉 번째까지를 다시 읽는다. 한 번에 읽는 양이 스텝 수를 따라 늘고, 그 늘어난 읽기를 스텝 수만큼 되풀이한다.

과제가 네 배 길어지면 청구서는 여덟 배가 된다. 매 스텝 컨텍스트 전체가 다시 실리기 때문이다. 스텝 수를 두 배로 늘리면 값은 네 배 언저리가 된다. 스텝 사이에 아무것도 다시 쓰지 않는 루프의 이야기이고, 그 전체를 무너뜨리는 일은 제6장이 받는다.

3.2 같은 내용도 언어마다 값이 다르다

Petrov et al. (2023)는 같은 내용을 여러 언어로 옮긴 병렬 말뭉치에 널리 쓰이는 토큰라이저들을 적용해, 그 차이가 최대 15배까지 벌어진다는 것을 보였다. 다국어 지원을 의도하고 만든 토큰라이저에서도 사라지지 않았다. 영어로 답았을 때에 견준 토큰 수의 비를 토큰 팡창 배율이라 하자. 언어의 성질이 아니라 언어와 토큰라이저의 쌍이 갖는 성질이고, 모델을 고르는 순간 이미 정해져 있다.

여기서 나오는 결정은 하나다. 모델만 읽는 텍스트는 영어로 쓰고, 사용자가 읽는 텍스트는 그럴 수 없다. 매 스텝 다시 실리는 것부터 본다. 시스템 프롬프트와 도구 설명문이 그렇고, 그 자리가 배율에 가장 먼저, 가장 크게 아프다. 2,000토큰짜리 시스템 프롬프트를 배율 1.5인 언어로 쓰면 3,000토큰이 되고, 스무 스텝이면 그것만으로 20,000토큰이 더 든다. 반대쪽은 협상 대상이 아니다. 최종 응답은 사용자의 언어여야 하고, 모델이 답을 만들며 쓰는 중간 추론의 언어는 모델이 정한다.

그리고 옮겨 쓰는 일도 공짜는 아니다. 지침의 언어가 출력의 언어를 끌어당기므로 사용자의 언어로 답하라는 지시를 따로 넣어야 하고, 그 지시가 얼마나 지켜지는지는 모델마다 다르다(제5장).

3.3 다루지 않은 것

토큰에 관련해서는 「어디까지만 알면 되는가」가 문제다. 궁금한 독자를 위해 각각이 무엇인지 한 줄로 적고 어디를 보면 되는지를 붙인다.

- **어텐션의 계산 구조.** 각 토큰이 문맥 안의 다른 토큰들에게서 자기 표현을 보충할 재료를 모으는 연산이다. 어떤 토큰 쌍이 서로 관련될지는 내용을 맞춰 보기 전에 알 수 없으므로 기본형에서는 모든 토큰이 모든 토큰과 만나고, 그래서 만남의 수가 길이의 제곱이 된다. 질의와 키와 값이 어떻게 구성되는지까지는 Vaswani et al. (2017)에 있다.
- **위치를 어떻게 넣는가.** 어텐션 자체는 토큰의 순서를 모른다. 순서를 따로 넣어 주어야 하고, 그 방법이 긴 입력에서 얼마나 버티는지는 방법마다 다르다. Su et al. (2024)이 오늘날 널리 쓰이는 쪽을 다룬다.
- **인과 마스킹.** 다음 토큰을 예측하도록 학습하려면 어떤 토큰도 자기보다 뒤의 토큰을 미리 보아서는 안 된다. 그 가리개가 인과 마스킹이고, 출력이 한 번에 한 토큰씩 만들어지는 이유가 여기서 나온다. 같은 문헌이 다룬다.
- **학습.** 이 책의 전제는 θ_0 의 파라미터가 고정되어 있다는 것이다. 손댈 수 있는 것은 모델의 가중치가 아니라 무엇을 실을지와 무엇을 허락할지다. 배경은 Goodfellow et al. (2016)에 있다.

이 목록에서 이 책이 가져가는 것은 하나뿐이다. 길이가 늘면 값이 늘고, 그 값은 제공자의 하드웨어에서도 길이의 제곱으로 늘어난다는 사실이다. 우리가 손댈 수 있는 것은 그 길이뿐이다.

3.4 요약

한 문장으로. 모델에 보내는 컨텍스트는 한 자도 무료가 아니고, 그 값을 세는 단위는 우리가 고르지 못한 채 모델과 함께 팔려 오므로, 예산의 이야기는 무엇이 한 토큰이 되는가에서 시작한다.

확실해진 것.

- 한 번의 호출에는 시스템 프롬프트와 도구 정의와 지금까지의 이력이 전부 실리고, 매 스텝 그 전체가 다시 실린다. 그래서 과제가 네 배 길어지면 청구서는 여덟 배가 된다(절 3.1). 고정부의 크기와 스텝당 증분, 이 두 수는 지금 도는 루프의 기록에서 그대로 셀 수 있다.
- 영어에 건준 토큰 수의 비가 토큰 팽창 배율이고, 언어가 아니라 언어와 토큰나이저의 쌍이 갖는 성질이다. 여기서 나오는 결정은 하나다. 모델만 읽는 텍스트는 영어로 쓰고, 사용자가 읽는 텍스트는 그럴 수 없다.
- 모델 교체는 코드를 한 줄도 바꾸지 않으면서 토큰 수로 적어 둔 모든 결정의 뜻을 바꾼다. 문턱은 그대로인데 문턱이 잡아 주던 분량이 줄고, 프롬프트를 고치지 않아도 같은 과제의 값이 오른다.

아직 답하지 않은 것.

- 매 스텝 전부를 다시 읽는다는 전제는 무너뜨릴 수 있다. 이미 읽은 것을 다시 계산하지

않는 장치와 그 장치가 끝내 못 없애는 것은 제6장이 받는다.

- 무엇을 실을지 자체를 줄이는 일, 곧 컨텍스트를 층으로 갈라 다루는 일은 제4장이 받는다.
- 길이가 길어질수록 값만 오르는 것이 아니라 품질도 떨어진다는 것은 여기서 세지 않았다. 제4장이 받는다.
- 배율의 샘플 스텝 수가 언어와 무관하다고 가정했다. 언어가 바뀌어 모델이 같은 과제를 더 많은 스텝으로 풀면 그 효과가 상수배를 압도한다.

3.5 생각해 볼 문제

1. 팽창 배율의 샘플은 그 배율이 컨텍스트와 행동에 똑같이 적용된다고 가정했다. 도구 호출의 인자가 대부분 JSON과 파일 경로로 이루어져 있다면 이 가정은 어떻게 깨지는가. 그때 유효한 팽창 배율은 어떻게 정의해야 하는가.
2. 절 3.1의 샘플은 스텝 사이에 아무것도 다시 쓰지 않는다고 가정한다. 지난 호출에서 이미 읽은 앞쪽 구간을 그대로 다시 쓸 수 있다면 그 샘플의 무엇이 바뀌고 무엇이 그대로인가. 그 재사용이 불가능해지는 것은 컨텍스트에 어떤 변경이 가해졌을 때인가.
3. 시스템 프롬프트를 배율이 낮은 언어로 바꾸면 토큰 수는 줄지만 과제 성공률이 달라질 수 있다. 이 교환을 판정하려면 어떤 실험을 설계해야 하는가. 측정해야 할 양을 먼저 적어 보라.
4. 모델을 새 판으로 옮기면서 토큰 수로 적힌 상수를 그대로 두었다고 하자. 가장 먼저 관측되는 증상은 무엇이었는가. 그 증상이 모델 교체 때문이라는 것을 로그에서 어떻게 확인할 수 있는가.

3.6 다시 읽을 때

- 제4장을 읽고 이 장의 샘플로 돌아오면, 고정부의 크기와 스텝당 증분이 그냥 측정하는 값이 아니라 손잡이였음이 보인다. 그 장은 둘을 각각 어떻게 줄이는지를 층으로 갈라 적는다.
- 제6장을 읽고 「매 스텝 전체가 다시 실린다」로 돌아오면, 그 전체를 무너뜨리는 수단이 들어 있다는 것이 보인다. 다시 읽는 토큰의 단가를 내리는 쪽과 세어지는 개수 자체를 덜 자라게 만드는 쪽이다.
- 제5장을 읽고 팽창 배율로 돌아오면, 그 상수배가 컨텍스트 안에서 균일하게 아프지 않다는 것이 보인다. 시스템 프롬프트는 매 스텝 다시 실려 배율이 스텝 수만큼 곱해지는 자리다. 두 장을 겹쳐 읽어야 20,000토큰이라는 샘플이 완성된다.

3.7 더 읽을거리

토큰화의 계보는 Sennrich et al. (2016)과 Kudo and Richardson (2018) 두 편으로 충분하다. 전자는 서브워드라는 발상 자체를, 후자는 언어 독립적 구현과 원문 복원 가능성을 다룬다. 바이트를 기본 단위로 두는 쪽은 Wang et al. (2020)이다.

절 3.2의 근거는 Petrov et al. (2023)이다. 자신이 다루는 언어의 비용 구조를 논거로 삼아야 한다면 원문의 측정 방법을 확인하고 자신의 코퍼스에 적용해 보기를 권한다. 실제로 셀 창구는 Anthropic (2026i)에 있다. 가져갈 것은 호출의 형식이 아니라 두 단서다. 돌아오는 값은 어렵고, 세는 기준은 모델의 토큰אי저다.

모델 안쪽이 궁금하다면 절 3.3에 붙여 둔 문헌부터 보면 된다.

제4장

컨텍스트 엔지니어링

에이전트가 일하는 데 필요한 정보의 대부분은 실행이 시작된 뒤에야 존재한다. 검색으로 가져와야 할 사실, 도구가 돌려준 관측, 앞선 세션의 인계, 사용자가 방금 덧붙인 조건. 그 전부가 실행 전에 적어 두는 텍스트 밖에서 생긴다.

여기서 다루는 것이 그 나머지, 곧 실행 시점에 내려지는 결정이다.

Anthropic (2025c)은 그것을 한 줄로 말한다. 원하는 결과가 나올 확률을 가장 크게 만드는, 가장 작은 고신호 토큰 집합. 「가장 작은」은 길이가 값을 요구하기 때문이고, 「고신호」는 무관한 것이 관련된 것을 밀어내기 때문이며, 「집합」은 채우기가 아니라 고르기라는 뜻이다. 앞의 두 낱말이 서로 다른 두 축이고, 이 구분이 뼈대다.

4.1 조립 함수의 안쪽

매 스텝 모델이 보는 것은 하나의 긴 문자열이다. 그 문자열을 만드는 일을 제1장은 조립이라 불렀고, 그 일을 하는 것이 조립 함수다.

거기 실리는 것은 우리가 실행 전에 적어 둔 지시만이 아니다. 모델이 부를 수 있는 도구의 이름과 인자, 지금까지 오간 대화, 검색해 온 문서, 도구가 방금 돌려준 결과와 전부 한 덩어리의 텍스트로 이어져 한 번에 간다.

그리고 다음 스텝에서 같은 것이 통째로 다시 간다. 새로 생긴 관측 한 줄을 붙이려고 앞의 전부를 다시 실는 것이다. 대화가 이어지는 것처럼 보이지만 모델 쪽에서 보면 매번 처음부터 다시 읽는 일이다. 무엇을 싣고 무엇을 싣지 않을지, 어떤 형태로 실을지가 전부 이 함수의 설계 자유도다.

실행 전에 확정되는 뭇을 고정부, 실행이 만들어 내는 뭇을 가변부라 적는다. 고정부의 정확한 규격과 이름은 제5장이 준다(정의 5.1). 고정부를 뺀 나머지, 곧 매 시각 이력과 그 밖의 가용 정보 가운데 무엇을 어떤 형태로 실을지를 실행 시점에 정하는 일이 컨텍스트 엔지니어링(context engineering)이다.

이 이름은 산업계에서 왔다. Anthropic (2025c)이 실행 시점에 최적의 토큰 집합을 큐레이션하고 유지하는 전략들로 정의했고, 그 대상에는 시스템 지시와 도구와 외부 자료와 메시지 이력이 함께 들어간다.

컨텍스트는 성능에도 영향을 미치기에 설계하지 않는다는 선택지는 없다. 실제 질문은 「할 것인가」가 아니라 「누가 정한 값이, 지금 어떻게 실리고 있는가」이다.

4.2 무엇을 실을 것인가

컨텍스트는 균질한 덩어리가 아니라 성격이 다른 층들의 합이고, 층마다 내용이 정해지는 시점이 다르고 자라는 법칙이 다르고 줄이는 수단이 다르다.

첫째, 스텝에 비례해 자라는 층은 하나뿐이다. 이력과 관측만이 실행이 만드는 층이고, 이 층은 손대지 않으면 저절로 자란다. 다른 층들은 크기가 과제나 설정에 매여 있어 한 번 정해지면 대체로 유지된다. 제2부에서 압축이 한 장을 따로 얻는 이유가 여기 있다.

표 4.1: 컨텍스트의 층별 분해. 오른쪽 열이 제2부 이후의 독서 지도다.

층	정해지는 시점	크기를 정하는 것	줄이는 수단과 다루는 곳
지시 고정부	실행 전	덜으려는 과제 유형의 수 (절 5.3)	지시 우선의 규율(제5장), 필요할 때 불러들이는 스킴(제7장)
도구 정의	실행 전	도구의 수와 스키마의 상세함	도구 설계와 정의 비용의 관리, 코드 실행 우회(제7장)
검색된 지식	실행 시점	질의와 검색 설정	검색의 품질(제4부 전체), 넣을지 자체의 판단(제11장)
이력과 관측	실행이 만든다	스텝 수, 그리고 도구가 돌려주는 양이 정하는 스텝당 증분	반환의 설계(제7장), 압축(제6장), 컨텍스트 격리(제16장)
인계 산출물	세션 경계	남은 과제의 크기	갱신형 인계(제15장 절 15.2)

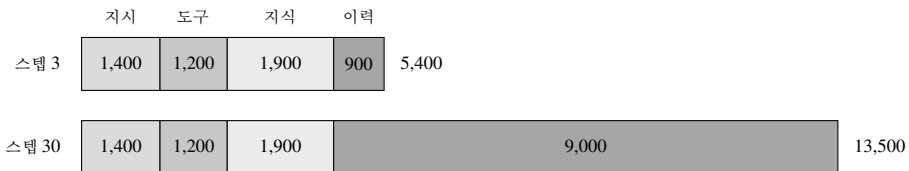


그림 4.1: 한 스텝의 컨텍스트를 층으로 쪼갠 모양. 두 줄이 같은 눈금이다. 앞의 세 층은 폭이 같고 마지막 층만 자란다. 인계 산출물은 세션을 이어받을 때만 실리므로 뺐다. 수는 한 시스템의 예시이며, 자기 값은 스텝 로그에서 나온다.

둘째, 어느 층이 예산을 지배하는지는 시스템마다 다르다. 도구가 많은 시스템은 도구 정의가, 문서를 다루는 시스템은 검색된 지식이, 긴 루프를 도는 시스템은 이력이 지배한다. 이 값은 표에서 읽는 값이 아니라 자기 시스템의 스텝 로그에서 측정하는 값이고, 측정하고 나면 그림 4.1처럼 그려진다.

이 그림에서 읽을 것은 비율이다. 이력이 차지하는 몫이 스텝 3에서 17%이고 스텝 30에서 67%다. 같은 구간에서 지시와 도구 정의를 합친 몫은 48%에서 19%로 내려간다. 줄어드는 것이 아니라 그대로인데 옆이 자란 것이다. 그래서 도구 정의를 다듬어 얻는 이득은 이른 스텝에서 크고 늦은 스텝에서 작으며, 짧은 과제만 측정해 보고 층의 순위를 정하면 긴 과제에서 틀린다.

셋째, 도구 정의는 실행 전에 적히고 도구의 반환은 실행이 만든다. 층마다 관리의 주체가 다를 수도 있다. 지시 고정부를 고치는 사람, 도구 정의를 관리하는 사람, 검색 설정을 권 사람, 이력의 압축 규칙을 정하는 사람은 다를 수 있다. 저마다 자기 층이 전체 예산에서 차지하는 비중을 모른 채 자기 층을 키운다. 비중이 측정되어 있어야 각 층의 관리자에게 자기 몫의 비율을 보여 줄 수 있다. 없으면 모든 층이 조금씩 자라는데 아무도 책임지지 않는다.

더불어 도구에 관해, *Anthropic (2025c)*은 기능이 겹치는 비대한 도구 묶음을 줄이라고 제안한다. 사람이 어느 도구를 써야 할지 단정하지 못하는 상황이라면 모델도 단정하지 못한다. 도구 이외의 지식과 이력의 압축은 이후 장들이 서술할 예정이다.

4.3 언제 실을 것인가

층은 다섯이지만 결정의 갈래는 둘이다.

한쪽은 우리가 실행 전에 적어 두는 것이다. 시스템 프롬프트가 그렇고 도구 정의가 그렇다. 무엇을 적을지 우리가 정하고, 다 적고 나면 크기가 확정되고, 그 크기가 첫 스텝부터 마지막 스텝까지 따라다닌다.

다른 한쪽은 컨텍스트 바깥에 두었다가 실행 중에 실어 오는 것이다. 도구가 돌려준 결과, 검색해 온 문서, 지난 세션이 남긴 인계가 그렇다. 여기서 바깥이란 파일 시스템이고, 검색 색인이고, 도구가 닿는 세계다. 실행이 시작될 때 이것들은 컨텍스트에 없다. 필요해진 시점에 들어온다.

그러면 물음이 하나 선다. 왜 바깥에 두는가. 처음부터 다 적어 두면 되지 않는가. 도구를 정의해 두는 것도 마찬가지다. 도구가 하는 일을 그냥 컨텍스트에 적어 두면 될 텐데 왜 부를 수 있는 이름으로 만들어 두는가.

이 장은 그 물음에 답을 시작하고 뒤의 부들이 마친다. 도구를 왜 어휘로 정의해 두는지, 그리고 도구가 돌려주는 것을 어떤 모양으로 받아야 하는지는 제3부가 받는다. 지식을 왜 바깥에 두었다가 가져오는지, 지난 세션의 것을 어디에 어떻게 남겨 두는지는 제4부가 받는다.

그 전에 여기서 셀 것이 하나 있다. 바깥에 두기로 했던 미리 적어 두기로 했던, 쓰려면 결국 컨텍스트에 실리는 순간이 온다. 그 순간을 언제로 잡느냐가 값을 정한다. 처음부터 실어 두는 것을 미리 실기, 필요해진 시점에 가져오는 것을 필요할 때 가져오기라 부른다.

사람도 그렇게 한다. 자료 전체를 외워 두지 않고 색인과 목차와 파일 이름을 두고 필요할 때 찾아 든다. 책상 위에는 지금 쓰는 것만 있다. 가져오기도 같은 모양이다. 데이터를 미리 처리해 두는 대신 파일 경로나 질의 같은 가벼운 식별자만 들고 있다가 실행 중에 도구로 불러들인다(Anthropic, 2025c).

4.4 왜 다 넣으면 안 되는가

다 넣는 설계에도 논리는 있다. 모델이 무엇을 필요로 할지 실행 전에는 알 수 없으니, 다 넣어 두면 적어도 부족 때문에 실패하는 일은 없으리라는 것이다. 그 밑에는 정보가 많을수록 좋고 무관한 것은 모델이 알아서 무시하리라는 가정이 깔려 있다. 하지만, 실제로는 그렇게 작동되지 않는다.

상한: 넣을 수 있는 양이 정해져 있다

모델마다 한 번의 호출에 받아들이는 토큰 수의 상한이 있다. 이것을 컨텍스트 윈도우라고 한다. 넘기면 요청이 거부되거나 앞이 잘려 나간다.

넷 가운데 이것이 가장 약한 제약이다. 세대가 바뀔 때마다 이 수는 커졌고, 오늘 좁아서 못 하는 일이 다음 세대에는 될 수도 있다. 컨텍스트에 관해 문제가 생기면 「더 큰 모델을 쓴다」로 끝난다. 물론 최대 컨텍스트를 사용하는 건, 같은 성능을 보장하지는 않는다.

부식: 상한에 닿기 전에 성능이 먼저 떨어진다

상한의 절반만 채워도 나빠진다. 실린 토큰이 늘어날수록 그 안의 것을 정확히 꺼내 쓰는 능력이 떨어진다(Anthropic, 2025c). 잘려서가 아니라 묻혀서다.

나쁜 컨텍스트에 두 종류가 있다. 하나는 길다. 실린 것마다 과제와 닿아 있는데 분량이 수백 장이어서, 섞인 것이 없는데도 일이 흐려진다. 다른 하나는 짧는데 엉뚱한 것이 섞여 있다. 관련된 조각이 전부 실려 있는데, 함께 실린 무관한 문장의 숫자를 답이 끝어다 쓴다. 앞의 것은 부피 때문이고 뒤의 것은 섞임 때문이며, 한쪽을 고쳐도 다른 쪽은 남는다.

길이가 값을 요구하는 방식도 둘이다. 하나는 자리다. Liu et al. (2024)은 담기는 내용은 그대로 두고 관련 정보의 위치만 바꾸어, 처음이나 끝에서 성능이 가장 높고 긴 컨텍스트의 중간에서 크게 떨어지는 것을 보였다(그림 4.2). 처방이 바로 나온다. 중요한 것은 앞과 뒤에 둔다. 중간에 넣어 둔 지침이 지켜지지 않을 때 먼저 의심할 것은 문구가 아니라 자리다.

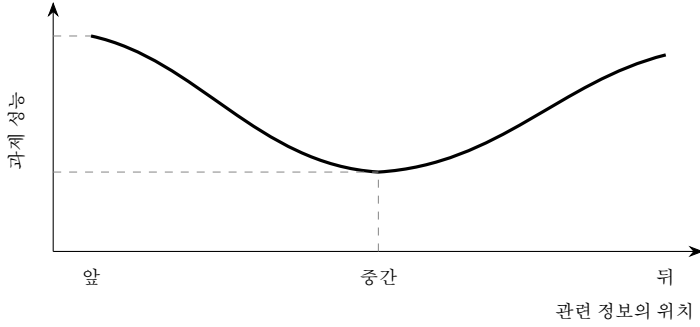


그림 4.2: 관련 정보의 위치에 따른 성능. 축에 눈금이 없는 것은 의도적이다. 곡선이 얼마나 깊게 파이는지는 모델과 과제마다 다르고, 이 그림이 나타내는 것은 값이 아니라 형태다.

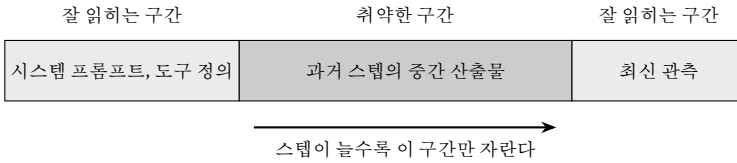


그림 4.3: 에이전트 컨텍스트의 전형적인 배치. 양 끝은 고정 크기이고 중간만 스텝 수에 따라 자란다.

다른 하나는 길이 그 자체다. 그러면 「윈도우 크기」라는 말이 서로 다른 두 양을 가리키고 있었다는 것이 드러난다. 들어가는 길이는 문서에 적힌 값이고, 일이 되는 길이는 측정해 보아야 하는 값이다.

정의 4.1 (유효 컨텍스트 길이). 과제 하나와 그 과제에서 요구하는 품질 기준을 고정했을 때, 그 기준을 여전히 만족하는 가장 긴 입력 길이를 유효 컨텍스트 길이라 한다.

긴 입력은 대개 내용도 더 많아서, 성능이 떨어졌을 때 길이 때문인지 과제가 어려워졌기 때문인지 가려지지 않는다. Levy et al. (2024)는 같은 표본을 길이와 종류와 위치가 다른 덧붙임 (padding) 으로 확장해 길이의 효과만 떼어 냈다. 결과는 이렇다. 기술적 최대치보다 훨씬 짧은 입력 길이에서 이미 추론 성능이 눈에 띄게 떨어진다. 무엇으로 채우든 어디에 채우든 채우는 것 자체가 값을 요구 한다는 뜻이다.

이력: 이 층만 저절로 자란다

문서 질의응답에서 입력 길이는 설계자가 정하는 값이다. 에이전트에서는 아니다. 컨텍스트는 스텝마다 관측과 행동이 덧붙으면서 자라므로 길이는 고르는 값이 아니라 스텝 수의 함수다. 한 선은 계속 자라고 유효 길이는 제자리에 있으니 두 선은 반드시 만난다.

참고 4.1 (누적 루프는 유효 길이를 저절로 넘어선다). 컨텍스트가 스텝마다 얼마간씩 누적되고 유효 컨텍스트 길이가 유한하다면, 저하 구간에 들어서는 스텝이 반드시 존재한다. 그 뒤의 모든 스텝에서 컨텍스트는 유효 길이를 넘는다.

자리도 나쁘다. 시스템 프롬프트는 맨 앞이고 새 관측은 맨 뒤이므로, 중간에 남는 것은 과거 스텝의 중간 산출물이다. 스텝 수에 비해 자라는 유일한 부분이 하필 잘 읽히지 않는 자리에 놓인다 (그림 4.3). 거기서 요구되는 것도 한 조각을 꺼내 오는 일이 아니라 여러 조각을 이어 따라가고 흠어진 것을 모아 종합하는 일이다(표 4.2).

표 4.2: 긴 컨텍스트에서 요구되는 능력의 층위. 위로 갈수록 시험하기 쉽고, 아래로 갈수록 에이전트가 실제로 하는 일에 가깝다.

능력	하는 일	에이전트에서의 대응
검색	한 조각을 찾아 꺼낸다	시스템 프롬프트의 한 규칙을 지킨다
다중 홉 추적	여러 조각을 이어 따라간다	앞선 스텝들의 결정을 이어 붙인다
집계	흩어진 것을 모아 종합한다	지금까지의 관측으로 다음 행동을 정한다

표 4.3: 컨텍스트 저하에 대한 네 가지 대응. 위의 둘은 샘의 인자를 건드리고 아래 둘은 샘의 전제를 건드린다. 아래로 갈수록 구조를 크게 바꾼다.

방향	하는 일	다루는 곳
실행 전에 실리는 것을 줄이기	루프가 돌기 전에 이미 실려 있는 것을 줄여 처음의 여유를 넓힌다	이 장, 제5장
덜 넣기 밖에 두기	한 스텝이 남기는 양을 줄인다 컨텍스트 대신 파일과 색인에 저장하고 필요할 때만 가져온다. 덧붙는 양의 상한을 설계자가 되찾는다	제5장, 이 장, 제6장 제10장, 제11장
나누기	하나의 긴 컨텍스트를 여러 개의 짧은 컨텍스트로 격리한다. 어느 하나도 스텝 수를 다 지지 않는다	제16장

그래서 줄여야 한다

셋을 이으면 결론이 하나다. 넣을 수 있는 양에 상한이 있고, 상한에 닿기 전에 이미 나빠지고, 다섯 층 가운데 하나는 손대지 않으면 저절로 그리로 간다. 그러니 줄여야 한다. 저하 시점을 뒤로 미루거나 그 시각이 와도 문제가 되지 않게 만드는 길은 넷이다.

절 4.3의 물음에도 여기서 답이 시작된다. 왜 바깥에 두는가. 다 넣을 수 없기 때문이다. 도구를 왜 어휘로 정의해 두는가. 도구가 하는 일을 전부 적어 두면 그 자체가 다 넣는 설계이기 때문이다. 이름만 실어 두고 결과는 부를 때 받는 편이 싸다.

물론 줄이면 잃는 것이 있다. 실려 있던 것을 요약으로 갈음하면 원본이 아닌 것이 남는다. 무엇을 잃는지, 얼마나 자주 줄여야 하는지, 줄인 것을 또 줄이면 어떻게 되는지가 제6장의 주제다.

4.5 요약

한 문장으로. 컨텍스트는 다 채우는 것이 아니라 골라 심는 것이다. 넣을 수 있는 양에 상한이 있고 상한에 닿기 전에 이미 나빠지므로, 어느 층에 무엇을 언제 실을지를 실행 시점에 정해야 한다. 토큰이 무료라 해도 해야 하는 일이다.

확실해진 것.

- 목표는 원하는 결과가 나올 확률을 가장 크게 만드는 가장 작은 고신호 토큰 집합이다. 프롬프트와의 경계는 기법이 아니라 결정 시점이다.
- 컨텍스트는 시점의 눈금을 따라 다섯 층으로 나뉜다(표 4.1). 스텝에 비례해 자라는 층은 이력뿐이고, 어느 층이 예산을 지배하는지는 시스템마다 다르다. 그것은 표에서 읽는 값이 아니라 자기 스텝 로그에서 측정하는 값이며, 측정하고 나면 이력의 몫이 스텝 3에서 17%, 스텝 30에서 67%로 보인다(그림 4.1).
- 결정의 갈래는 둘이다. 실행 전에 우리가 적어 두는 것과, 파일과 색인에 두었다가 실행 중에 실어 오는 것. 앞의 것이 미리 신기이고 뒤의 것이 필요할 때 가져오기다. 여기서 서는 물음이 「왜 바깥에 두는가」이고, 답은 제3부와 제4부가 채운다.
- 다 넣으면 안 되는 이유는 넷이고 뒤로 갈수록 무겁다. 상한이 있고, 상한에 닿기 전에 부식이 오고, 다섯 층 가운데 이력만 저절로 그리로 자라고(참고 4.1), 그래서 줄여야 한다. 일이 되는 길이는 선언된 크기가 아니라 측정하는 값이며(정의 4.1), 하필 스텝을 따라 자라는 구간이 가장 잘 안 읽히는 자리에 놓인다(그림 4.3).
- 그 저하에 대한 대응은 넷이다(표 4.3). 실행 전에 실리는 것을 줄이는 첫 행이 가장 싸고, 아래로 갈수록 구조를 크게 바꾼다.

아직 답하지 않은 것. 그림 4.1의 수는 실측이 아니라 예시이고, 층별 비중의 실측값은 속제로 남아 있다. 이 장이 답한 것은 무엇을 실을 것인가까지다. 이미 실린 것을 줄여 담는 기술은 제6장이, 도구를 왜 어휘로 정의해 두는지는 제3부가, 지식과 기억을 왜 바깥에 두는지는 제4부가 받는다. 애초에 넣을지의 의사결정은 제11장이고, 층별 비중을 측정하려면 있어야 하는 기록의 설계는 제20장이다.

오늘 할 것. 스텝 로그에서 한 스텝을 골라 그 스텝의 입력을 통째로 꺼낸다. 시스템 프롬프트가 몇 토큰이고 도구 정의가 몇 토큰이고 지금까지의 이력이 몇 토큰인지 나누어 센다. 이른 스텝에서 한번, 늦은 스텝에서 한번 하면 어느 것만 자라는지 보인다(절 4.2). 다음으로 그 입력에서 가장 큰 덩어리 하나를 골라 언제 처음 쓰였는지 찾는다. 첫 스텝부터 실려 있었는데 스무 번째 스텝에서 처음 쓰였다면, 열아홉 스텝 어치를 미리 실어서 버린 것이다(절 4.3). 마지막으로 실패한 실행 몇 건의 입력 길이와 성공한 실행의 입력 길이를 나란히 놓는다. 실패 쪽이 길다면 이 시스템은 이미 부식을 겪고 있다(절 4.4).

4.6 생각해 볼 문제

1. 나쁜 컨텍스트의 두 종류를 자기 과제에서 갈라 평가하는 실험을 설계하라. 내용을 고정하고 덧붙임으로 길이만 늘린 컨텍스트와, 길이를 거의 고정하고 과제와 무관한 내용을 섞은 컨텍스트를 각각 만들어야 한다. 길이에는 둔감하지만 섞인 것에 민감한 경우와 그 반대의 경우에 처방은 각각 어떻게 달라지는가.
2. 표 4.1의 다섯 층에 대해 자신의 시스템의 스텝 로그에서 층별 토큰 수를 측정하라. 이른 스텝과 늦은 스텝에서 각각 측정해 그림 4.1처럼 두 줄로 그려 보라. 어느 층이 예산을 지배하는가.

그 층의 크기를 정하는 사람이 조직에서 누구인지, 그 사람이 자기 층의 비중을 알고 있는지도 함께 적어 보라.

3. 지금 미리 읽고 있는 것 가운데 가장 큰 조각을 하나 골라, 그것이 실제로 쓰이는 스텝을 로그에서 뽑아 보라. 그 수는 로그에서 나오는가, 아니면 짐작인가. 그 조각을 바깥에 두고 필요할 때 가져오는 쪽으로 옮긴다면, 컨텍스트에는 무엇이 대신 남아 있어야 모델이 그것을 가져올 수 있는가.
4. 표 4.3의 네 방향 가운데 자신의 시스템에 지금 필요한 것은 어느 것인가. 첫 행이 가장 큰 이유를 자기 시스템의 수로 확인해 보라. 그리고 아래 두 행이 「셈의 인자」가 아니라 「셈의 전제」를 건드린다는 말이 자기 시스템에서 구체적으로 무엇을 바꾸는 일인지 적어 보라.

4.7 다시 읽을 때

- 제6장을 읽고 「그래서 줄여야 한다」로 돌아오면, 여기서 결론으로만 적어 둔 한 줄이 거기서 값을 문 기법이 되어 있다. 줄여 담기는 실린 양을 직접 줄이지만 원본이 아닌 것을 남기고, 잘못 줄이면 낡은 결론을 남겨 오히려 해친다. 넣을 수 있는 양에 상한이 있다는 이 장의 첫 이유가 거기서 「그래서 얼마나 자주, 무엇을 버리고」라는 물음으로 바뀐다.
- 제3부와 제4부를 읽고 「왜 바깥에 두는가」로 돌아오면, 여기서 물음으로만 세워 둔 것에 답이 붙어 있다. 도구는 제3부가, 지식과 기억은 제4부가 채운다. 표 4.1의 오른쪽 열이 그 장들의 목록이다. 제7장의 점진적 공개는 이름만 미리 읽고 본문은 부를 때 받는 구조를 지시 층에 그대로 적용한 것이다.
- 제16장을 읽고 표 4.3으로 돌아오면, 「나누기」가 왜 마지막 칸인지 보인다. 앞의 셋은 컨텍스트가 하나라는 전제를 그대로 두고 그 안에서 양을 다루지만, 나누기는 그 전제 자체를 버린다. 값이 가장 크게 드는 것도 그래서다.
- 제5장을 읽고 그림 4.1로 돌아오면, 여기서 폭이 같다고만 그린 앞의 두 층에 정의 5.1이 하나의 이름을 준다. 이 그림의 왼쪽 절반이 그 장의 대상 전체이고, 폭이 같은 이유도 거기서 나온다. 실행 전에 확정되기 때문이다.

4.8 더 읽을거리

자리 쪽은 Liu et al. (2024)을 보라. 담기는 내용을 그대로 둔 채 위치만 옮겨 U자를 얻어 내는 설계라, 이 장이 그림 하나로 줄여 둔 것의 실제 모양을 볼 수 있다. 길이 쪽은 Levy et al. (2024)를 보라. 같은 표본을 덧붙임으로 늘려 길이의 효과만 분리하는 설계가 명료해서, 자신의 과제로 같은 실험을 만들 때 그대로 본이 된다. 전통적 지표와 추론 성능이 음의 상관관을 보였다는 결과는 대리 지표에 대한 경고로 따로 기억해 둘 만하다.

산업계 쪽 정리로는 Anthropic (2025c)이 있다. 컨텍스트 엔지니어링의 정의를 가져온 문헌이며, 실무 기법의 목록이 정리되어 있다. 다만 태도를 정해 두어야 한다. 구체 기법의 목록은 모델 세대마다 갱신된다. 어떤 기법을 만나든 두 가지를 물어라. 표 4.1의 어느 층을 건드리는가. 실행 전에 적어 두는 쪽과 바깥에 두었다가 실어 오는 쪽 가운데 어디로 옮기는가. 목록이 낡아도 자리는 남는다.

제5장

프롬프트 엔지니어링

프롬프트는 실행이 시작되기 전에 적히는 텍스트다. 그래서 실행 중에 알게 되는 것은 무엇도 여기에 담기지 않는다.

제4장이 한 스텝의 컨텍스트를 층으로 쪼개고, 무엇을 언제 실을지가 설계의 자유도라고 했다. 그 가운데 우리가 실행 전에 적어 두는 몫이 이 장의 대상이다. 검색해 온 지식과 도구가 돌려준 결과는 뒤의 부들이 받는다.

담할 것은 셋이다. 컨텍스트 안에서 프롬프트는 어디에 있는가. 그것을 어떻게 명확하게 적는가. 고친 것이 나아졌는지 무엇으로 아는가.

5.1 프롬프트는 어디에 있는가

제4장은 한 스텝의 입력을 네 층으로 쪼개 그렸다. 지시, 도구, 검색해 온 지식, 이력이다. 스텝이 서른 번을 돌아도 폭이 그대로인 층이 앞의 둘이고, 그 둘이 이 장의 대상이다 (그림 4.1).

앞의 두 층은 셋으로 갈린다.

시스템 프롬프트가 맨 앞에 있다. 우리가 실행 전에 적어 두고, 첫 스텝에도 서른 번째 스텝에도 글자 하나 다르지 않은 채로 다시 실린다.

사용자의 입력이 그 뒤에 온다. 이쪽은 실행마다 다르다. 같은 시스템 프롬프트 뒤에 오늘은 이 문의가 붙고 내일은 저 문의가 붙는다.

그리고 도구 설명문이 있다. 모델이 부를 수 있는 도구마다 이름과 인자와 언제 쓰는 도구인지를 적은 문장이 딸려 간다. 이런 문장이다.

「고객의 최근 주문 내역을 돌려준다. 주문 번호를 알고 있을 때 쓴다. 이름만 있으면 먼저 고객을 조회한다.」

도구 설명문을 프롬프트로 세지 않는 사람이 많다. 도구 쪽 코드에 적혀 있고 도구를 만든 사람이 관리하기 때문이다. 그런데 모델이 보는 문자열에서 이 문장은 시스템 프롬프트 바로 옆에 있다. 실행 전에 확정되고, 매 스텝 다시 실리고, 매 스텝 값을 문다. 자리로 보면 같은 물건이다.

셋을 한 이름으로 부를 수 있다.

정의 5.1 (프롬프트). 이력이 무엇이 되든 매 스텝 같은 모양으로 실리는 부분을 프롬프트라 한다. 시스템 프롬프트와 도구 설명문과 출력 규약이 그 대부분이고, 이력을 어떤 순서로 배치하고 어떤 표기로 감싸는지의 틀이 나머진다. 제4장이 고정부라 부른 몫이 이것이다. 프롬프트 엔지니어링이란 이 고정부를 설계하는 일이다.

5.2 지시가 하는 일

고정부가 실제로 담는 것은 크게 셋이다. 첫째는 과제의 규정이다. 무엇을 하라는 것인지, 그리고 무엇이 되면 끝난 것인지. 제15장에서 완료 선언은 검증을 발동하는 입력이라 했고, 그 검증이 대조할 기준의 상당 부분은 프롬프트에 먼저 문장으로 적혀 있어야 한다. 둘째는 출력의 규약이다.

무엇을 어디에 어떤 구조로 적는지. 다만 이 몫의 상당 부분은 오늘날 문장이 아니라 요청에 딸린 형식 지정으로 옮겨 갔고, 그 이야기는 제9장이 받는다. 셋째는 행동의 경계다. 하지 말아야 할 것, 확인 없이 실행하면 안 되는 것, 도구를 쓸 때 지켜야 할 조건. 셋째에는 단서가 붙는다. 경계를 적어 두는 자리는 고정부이지만 그 경계를 밀어 보는 문장은 실행 중에 도착한다. 그래서 고정부는 계약인 동시에 방어선이고, 방어선의 설계는 제22장이 받는다.

셋이 섞여 있으면 어느 것도 지켜지지 않는다. Anthropic (2025c)은 구획을 나누고 XML 태그나 마크다운 머릿글로 경계를 표시하라고 권한다. 배경 정보, 지시, 도구 사용 지침, 출력 규약을 각각 제 자리에 두라는 것이다. 골격은 이런 모양이다.

```
<배경> 이 상담원은 통신 요금제 문의를 받는다. </배경>
<과제> 요금이 왜 올랐는지 한 문단으로 설명한다. </과제>
<도구> 청구 내역은 조회 도구로 확인하고 기억에 의존하지 않는다. </도구>
<출력> 금액은 원 단위로 적고 추정치에는 추정이라고 적는다. </출력>
```

이것이 문체의 권고가 아닌 이유는 검증에 있다. 구획이 있으면 「출력 규약이 무엇인가」에 문장 하나를 가리켜 답할 수 있고, 없으면 프롬프트 전문을 다시 읽어야 한다.

얼마나 구체적으로 쓸 것인가

같은 자료가 프롬프트 설계의 축을 하나 세운다. 구체성이다. 두 실패가 양 끝에 있다.

한쪽 끝은 너무 두루뭉술한 문장이다. 신호가 없다.

「사용자를 잘 도와라.」

어떤 출력도 배제하지 않으므로 계약이 아니다. 모델이 이미 아는 것을 반복하는 문장도 여기에 속한다. 같은 자리에 이렇게 적으면 배제되는 출력이 생긴다.

「답을 확정하기 전에 주문 상태를 조회해 확인한다. 조회하지 못했으면 확인하지 못했다고 적는다.」

금지로 적힌 문장은 한 번 더 뒤집어 본다. 하지 말라는 문장은 남은 것 전부를 허용하므로, 같은 뜻을 하라는 쪽으로 적으면 목표가 하나로 좁혀진다. 「목록으로 답하지 마라」보다 「문단으로 이어지는 산문으로 답한다」가 배제하는 출력이 없다.

다른 쪽 끝은 너무 구체적인 문장이다. 프롬프트가 분기를 하나씩 적어 둔 코드가 된다. 이 경우에는 이렇게, 저 경우에는 저렇게. 실행에서 마주치는 경우는 적어 둔 목록보다 언제나 많다. 목록에 없는 경우가 오면 모델은 가장 가까워 보이는 조항을 잘못 적용한다. 게다가 조항은 상황이 바뀔 때마다 손봐야 한다. 이것은 사실상 워크플로를 자연어로 쓰는 일이고, 그럴 바에는 제1장의 판정대로 코드로 쓰는 편이 싸고 안정적이다.

같은 실패가 오늘 더 혼한 다른 길로도 온다. 조항을 늘리는 대신 목소리를 키우는 것이다. 모델이 지시를 덜 따르던 시절에는 이렇게 적는 일이 잦을 했다.

「반드시 이 도구를 써야 한다. 의심스러우면 무조건 부른다.」

지금은 적힌 대로 따르므로, 같은 문장이 쓰지 않아도 될 자리에서 도구를 부르게 하고 예외로 두어야 할 경우까지 조항을 끌어다 붙인다. 강조는 배제하는 출력을 늘리지도 문장을 더 구체적으로 만들지도 않으면서 적용 범위만 넓힌다. 처방은 문장을 지우는 것이 아니라 목소리를 낮추는 것이다.

「주문 번호가 주어졌을 때 이 도구를 쓴다.」

조항은 남고 과잉만 사라진다.

가운데를 어떻게 찾는가. 판정은 문장 단위로 내린다. 이 문장이 배제하는 출력이 있는가. 없으면 두루뭉술한 것이다. 이 문장이 특정 상황에만 적용되는가. 그렇다면 지나치게 구체적인 것이고, 그 상황을 판별하는 일까지 모델에게 맡기려면 조건이 아니라 원칙으로 써야 한다. 문장을 읽는 대신 사람에게 시켜 보는 검사도 같은 일을 한다. 이 과제의 배경을 모르는 동료에게 프롬프트를 그대로 주고 시켰을 때 그가 헛갈린다면 모델도 헛갈린다.

표 5.1: 의미를 바꾸지 않고 흔들 수 있는 여섯 자리. 자리마다 값이 들이면 형식이 64가지가 된다.

흔드는 자리	한 값	다른 값
구획의 경계	태그로 감싼다	머리글로 연다
항목의 구분자	짧은 줄표	번호
이름과 값 사이	콜론과 빈칸	등호
항목 사이	빈 줄을 넣는다	빈 줄을 뺀다
예시를 감싸는 표기	따옴표	태그
예시의 순서	지금 순서	뒤집은 순서

5.3 프롬프트는 견고하지 않다

Sclar et al. (2024)은 의미를 보존하는 설계 선택의 전형인 프롬프트 형식에 대한 민감성을 측정했다. 구분자를 바꾸고, 공백과 대소문자를 바꾸는 수준의, 지시의 내용에는 아무 변화가 없는 변경들이다. 결과는 이렇다. 널리 쓰이는 여러 공개 모델이 퓨샷 상황에서 이런 미묘한 형식 변화에 극도로 민감했다. 이 민감성은 모델의 크기를 키워도, 예시의 수를 늘려도, 지시 조정을 거쳐도 사라지지 않는다.

이 민감성은 실무에서 이런 얼굴을 한다. 어느 날 프롬프트의 예시 구획을 손본다. 조항 하나를 다듬고, 검사검사 예시를 감싸던 표기를 통일하고, 항목 사이의 빈 줄을 정리한다. 평가를 돌리니 성공률이 올라 있다. 변경 기록에는 「지시를 명확히 함」이라고 적힌다. 다음 달, 다른 사람이 같은 조항을 더 명확히 고쳤는데 이번에는 점수가 내려간다. 앞의 개선이 조항에서 온 것이 아니라 빈 줄에서 온 것이었다면, 두 사람은 같은 실험을 두 번 한 것이 아니라 서로 다른 실험을 한 번씩 한 것이다. 그리고 기록만 보아서는 어느 쪽인지 알 길이 없다.

평가한 방식을 정확히 읽어야 한다. 여러 과제에 걸친 평균이 아니라, 하나의 과제를 고정하고 질문과 예시와 요구하는 답을 그대로 둔 채 형식만 바꾸어 같은 지표를 다시 평가한 것이다. 그리고 형식별 성능은 모델 사이에 약하게만 상관했다. 어떤 모델에 유리한 형식이 다른 모델에도 유리하리라는 보장이 없다는 뜻이다.

민감성은 형식이라는 한 축에서만 확인된 것이 아니다. Lu et al. (2022)는 퓨샷 설정에서 예시가 제시되는 순서만 바꾸어도 최신 수준에 가까운 성능과 무작위 추측 수준의 성능이 갈릴 수 있음을 보였다.

무엇을 흔들 것인가

같은 지시를 두 형식으로 적어 보면 이렇다.

(가) 「출력 규약: 한 줄에 한 항목을 적는다.」

(나) 「<출력 규약> 한 줄에 한 항목을 적는다 </출력 규약>」

뜻이 같다. 모델에게는 다른 문자열이다. 이런 자리가 여섯이고, 표 5.1이 그 목록이다.

그래서 고른다. 여섯 가운데 셋만 흔들고 나머지 셋은 지금 쓰는 값에 둔다. 고르는 규칙은 하나다. 고정부에서 여러 번 되풀이되는 자리를 고른다. 한 번만 나오는 표기는 흔들어도 차이가 작다. 예시 구획이 있는 프롬프트라면 예시의 순서가 대개 첫째로 뽑힌다.

나아졌다고 말할 수 있는 조건

판정은 점끼리가 아니라 범위끼리 한다. 수정 후의 가장 낮은 점수가 수정 전의 가장 높은 점수를 넘을 때만 나아졌다고 말한다. 두 범위가 겹치면 아직 모르는 것이다.

이 값을 문장 하나 고칠 때마다 치를 수는 없다. 치르는 자리는 셋이다. 프롬프트를 처음 세울 때, 고정부를 묶음으로 손볼 때, 그리고 모델을 바꿀 때다. 셋째가 가장 자주 잊힌다. 좋은 형식이 모델 사이에 이전되지 않으므로, 모델을 교체하면 프롬프트의 검증도 다시 해야 한다.

그 사이의 수정은 평가하지 않고 한다. 다만 기록에 「개선」이라 적지 않는다. 「변경」이라 적는다. 평가 없이 고친 프롬프트는 개선된 것이 아니라 달라진 것이다.

여기까지가 프롬프트의 이야기다. 그런데 같은 문제가 컨텍스트 전체에 걸린다. 무엇을 실을지 바꿨을 때, 무엇을 뺐지 바꿨을 때, 압축 주기를 바꿨을 때. 전부 「나아졌는가」를 물어야 하고 전부 같은 어려움을 만난다. 무엇을 성공이라 부를 것인지, 누가 판정할 것인지, 어떤 과제를 사례로 삼을 것인지가 정해져 있지 않기 때문이다.

그래서 컨텍스트 엔지니어링은 평가와 함께 굴러간다. 평가하지 않으면 무엇을 고쳤는지 알 수 없고, 무엇을 고쳤는지 모르면 다음 결정의 근거가 없다. 평가를 어떻게 세우는 지는 제6부가 받는다.

프롬프트로 고칠 수 없는 것

프롬프트에는 실행 전에 아는 것만 담을 수 있다. 검색으로 가져와야 할 사실도, 도구가 돌려준 관측도, 사용자가 방금 덧붙인 조건도 그때는 없다. 에이전트가 일하는 데 필요한 정보의 대부분은 실행이 시작된 뒤에야 존재한다. 아무리 공들여 써도 이 경계는 좁혀지지 않는다.

그래서 규정이 하나 나온다. 프롬프트로 고칠 수 있는 것은 모든 실행에 공통인 문제뿐이다. 같은 프롬프트로 어떤 실행은 되고 어떤 실행은 안 된다면, 두 실행에서 프롬프트는 글자 하나 다르지 않았다. 갈린 것은 프롬프트 밖이다.

이 규정이 실무에서 쓰이는 자리는 진단이다. 실패한 실행을 몇 건 놓고 물어본다. 전부 같은 자리에서 실패했는가, 아니면 실행마다 다른 자리에서 실패했는가. 앞이면 프롬프트를 본다. 뒤면 도구나 검색이나 과제 자체를 본다. 이 물음을 건너뛰면 프롬프트만 계속 고치게 된다. 시스템에서 손이 가장 쉽게 닿는 부품이기 때문이다.

경계를 무시했을 때 프롬프트가 어떻게 되는지는 정해진 경로를 따른다. 과제가 다양해질수록 하나의 프롬프트에 각 과제의 규정과 예외가 쌓인다. 어느 하나를 빼면 그 과제가 나빠지니 빼기는 어렵고 더하기는 쉽다. 그렇게 길어진 프롬프트는 매 스텝 값을 더 물고, 지금 과제와 무관한 조항으로 컨텍스트를 흐린다.

그러면 답은 프롬프트를 더 잘 쓰는 것이 아니다. 모든 과제의 지시를 언제나 실는 대신, 이 과제에 필요한 것을 실행 시점에 골라 넣는 것이다. 그것이 제4장이 다룬 컨텍스트 엔지니어링이고, 필요할 때만 불러들이는 지시 묶음이라는 형태로 같은 방향을 미는 구조는 제7장에서 만난다.

5.4 프롬프트를 더 고치지 말아야 할 때

프롬프트는 시스템에서 가장 쉽게 손이 가는 부품이다. 그래서 다른 부품의 문제가 가장 먼저 이리로 흘러온다. 고치기 전에 이것이 프롬프트의 문제인지부터 가른다.

물음 하나가 대부분을 가른다. 같은 프롬프트로 어떤 실행은 되고 어떤 실행은 안 되는가.

갈렸다면 다시 두 갈래다. 같은 입력으로 여러 번 돌려 보아서 이력까지 같은데도 결과가 갈리면 그것은 표집이고, 문구를 고쳐 없앨 수 없다. 검사와 재시도로 받는 수밖에 없으며 제15장의 일이다. 이력이 서로 달랐다면 무엇이 달랐는지가 답이다. 어떤 문서가 실렸는지, 도구가 무엇을 돌려주었는지, 몇 스텝을 돌았는지. 고칠 자리는 무엇을 언제 실는가이며 제4장의 일이다.

한 도구의 오용을 막는 지시가 쌓인다면 그 도구의 이름과 인자가 나쁜 것이고, 나쁜 도구는 어떤 프롬프트로도 복구되지 않는다. 근거를 확인하라는 지시가 늘어도 답이 지어내어진다면 컨텍스트에 근거가 안 들어온 것이다. 모델이 모르는 사실은 지시를 강하게 쓴다고 생기지 않는다.

표에 없는 자리도 있다. 자연이 길다거나 값이 비싸다는 요구는 어느 행에도 안 걸린다. 실패와 성공이 갈리는 문제가 아니라 모든 실행이 똑같이 느리고 똑같이 비싼 문제다. 프롬프트가 후보로 보이지만 문장을 다듬어 줄일 수 있는 뭇은 작다. 모델을 바꾸는 쪽이 대개 빠르다.

표 5.2: 프롬프트로 고칠 수 있는 것은 마지막 행뿐이다. 위의 넷은 다른 부품의 문제가 흘러온 것이다.

증상	무엇을 확인하는가	고칠 자리
이력이 같은데도 결과가 갈린다	같은 입력으로 여러 번 돌린다	검사와 재시도(제15장)
된 실행과 안 된 실행의 이력이 다르다	두 실행의 컨텍스트를 나란히 놓는다	무엇을 언제 실행하는가(제4장)
한 도구의 오용을 막는 지시가 계속 쌓인다	그 도구의 이름과 인자를 사람에게 보인다	도구의 정의(제7장)
근거를 확인하라는 지시가 늘어도 답이 지어내어진다	그 실행의 컨텍스트에 근거가 실렸는지 본다	검색(제4부)
모두 같은 자리에서 실패한다	그 자리를 다루는 문장이 고정부에 있는지 본다	프롬프트

예 5.1 (증상 하나를 셋으로 나눈다). 문서를 찾아 요약해 티켓에 붙이는 에이전트가 열 번 중 일곱 번 성공한다. 실패를 두고 프롬프트를 고쳐 달라는 요청이 들어왔다. 실패한 실행 열 개를 골라 표 5.2에 댄다.

같은 입력으로 세 번씩 다시 돌린다. 넷은 어떤 회차에서는 성공했다. 이력이 같은데도 갈렸으니 문구로 없앨 수 있는 것이 아니다.

남은 여섯의 컨텍스트를 성공한 실행과 나란히 놓는다. 다섯에서 요약할 문서가 아예 실려 있지 않았다. 근거가 없는데 답을 썼으니 지시를 강하게 쓸 문제가 아니다.

하나가 남는다. 문서도 실렸고 요약도 맞는데 티켓의 제목 자리에 본문이 들어갔다. 그리고 성공한 실행에서도 같은 자리가 같은 모양으로 틀렸다. 이것 하나가 고정부의 후보다. 열 개 가운데 하나다.

5.5 요약

한 문장으로. 프롬프트는 실행 전에 확정되어 매 스텝 같은 자리에 다시 실리는 부분이다. 그래서 미리 검증할 수 있지만, 견고하지도 않고 실행 중에 알게 되는 것을 담지도 못한다. 평가 하면서 고치고 나머지는 실행 시점의 결정으로 넘긴다.

확실해진 것.

- 프롬프트가 있는 자리는 컨텍스트의 맨 앞이다. 시스템 프롬프트와 도구 설명문이 거기 있다(정의 5.1). 도구 설명문은 도구 쪽 코드에 적혀 있어 프롬프트로 세지 않기 쉬운데, 실행 전에 확정되고 매 스텝 다시 실리고 매 스텝 값을 문다.
- 고정부가 담는 것은 셋이다. 과제의 규정, 출력의 규약, 행동의 경계. 구획을 나누어 적는 이유는 문체가 아니라 검증이다. 「출력 규약이 무엇인가」에 문장 하나를 가리켜 답할 수 있어야 한다.
- 지시를 적는 판정은 문장 단위다. 이 문장이 배제하는 출력이 있는가, 그리고 특정 상황에만 적용되는가. 목소리를 키우는 것은 둘 어디에도 걸리지 않으면서 적용 범위만 넓힌다.
- 프롬프트는 견고하지 않다. 뜻을 그대로 둔 형식 변경과 예시 순서 변경만으로 성능이 크게 달라지고, 좋은 형식은 모델 사이에 이식되지 않는다. 흔들 자리의 목록이 표 5.1이고, 고정부에서 여러 번 되풀이되는 자리부터 고른다. 그래서 판정은 점끼리가 아니라 범위끼리 하며, 평가하지 않고 고친 것은 개선이 아니라 변경이다.
- 프롬프트로 고칠 수 있는 것은 모든 실행에 공통인 문제뿐이다. 실패한 실행들이 전부 같은 자리에서 실패했는지 실행마다 다른 자리에서 실패했는지가 첫 물음이고, 표 5.2가 증상에서 고칠 자리로 가는 길을 적었다.

아직 답하지 않은 것. 평가하는 일이 이 장에서 끝나지 않는다. 무엇을 성공이라 부르고 누가 판정하며 어떤 과제를 사례로 삼을지가 정해져야 범위를 그릴 수 있고, 그 설계는 제6부가 받는다. 도구 설명문을 어떻게 쓰지는 제3부가 받는다.

오늘 할 것. 자기 시스템의 프롬프트를 연다. 시스템 프롬프트만 열면 절반이므로 도구 설명문도 함께 꺼내 놓고 둘의 토큰 수를 합쳐 센다(절 5.1). 다음으로 시스템 프롬프트를 문장 단위로 읽으며 각 문장이 배제하는 출력을 옆에 적는다. 아무것도 적히지 않는 문장이 첫 후보다(절 5.2). 마지막으로 들어와 있는 수정 요청 하나를 골라 표 5.2에 대 본다. 실패가 실행마다 다른 자리에서 나고 있다면 그 요청은 프롬프트의 것이 아니다(절 5.4).

5.6 생각해 볼 문제

1. 자기 시스템의 도구 설명문을 전부 꺼내 한자리에 모으고 시스템 프롬프트와 길이를 견주어 보라. 어느 쪽이 긴가. 그다음 그 설명문을 지난 분기에 누가 몇 번 고쳤는지, 고칠 때 나아졌다고 무엇으로 판정했는지 찾아보라. 정의 5.1에 따르면 둘은 같은 물건인데, 같은 규율 아래 있었는가.
2. 자기 프롬프트에서 표 5.1의 여섯 자리 가운데 실제로 있는 것을 세어 보라. 여러 번 되풀이되는 자리부터 골라 변형을 만들면 오늘 보고하는 점수는 그 범위의 어디에 있는가. 범위가 예상보다 넓게 나왔다면, 그 사실이 지난 분기의 개선 기록 가운데 무엇을 다시 읽게 만드는가.
3. 자기 시스템 프롬프트에서 「반드시」와 「절대」가 들어간 문장을 전부 뽑아라. 각각을 「이럴

때 이렇게 한다」로 고쳐 쓰면 배제되는 출력이 달라지는가. 달라지지 않는 문장이 있다면 그 낱말은 무엇을 하고 있었는지 적어 보라.

4. 과제 유형이 여럿인 시스템에서, 모든 유형을 덮는 하나의 프롬프트와 유형별로 갈라 쓰는 여러 프롬프트 사이의 선택을 생각하자. 갈라 쓰면 절 5.3이 그린 경로는 피하지만 유지할 프롬프트가 늘고 각각을 따로 평가해야 한다. 이 교환을 판정하는 데 필요한 것들을 나열하고, 유형의 수가 늘어날 때 어느 쪽이 유리해지는지 논하라.

5.7 다시 읽을 때

- 제7장을 읽고 절 5.1로 돌아오면, 도구 설명문이 프롬프트와 같은 자리에 있다고만 적어 둔 것에 설계가 붙어 있다. 이름과 인자를 어떻게 고르는지, 설명문을 얼마나 길게 쓰는지가 거기서는 프롬프트를 고치는 일과 같은 규율 아래 놓인다.
- 제22장을 읽고 절 5.2로 돌아오면, 고정부의 셋째 역할이 계약을 넘어 신뢰 경계였다는 것이 보인다. 행동의 경계를 적어 두는 일과 그 경계를 실행 중에 도착한 텍스트로부터 지키는 일은 같은 문장이 맡을 수 없다.
- 제18장과 제19장을 읽고 절 5.3으로 돌아오면, 형식 민감성이 평가 설계의 전제 조건이 되어 있는 것이 보인다. 여기서는 범위끼리 견주라고만 했는데, 그 범위를 몇 개의 변형과 몇 개의 과제로 그릴지, 두 범위가 겹쳤을 때 무엇을 하는지가 그 두 장의 주제다.
- 제6장을 읽고 절 5.3의 끝으로 돌아오면, 「나아졌는가」를 묻는 일이 프롬프트만의 사정이 아니었다는 것이 보인다. 무엇을 얼마나 자주 줄여 답을지도 같은 물음을 만나고, 평가하지 않으면 같은 자리에서 막힌다.

5.8 더 읽을거리

절 5.2가 구획을 나누라고 한 근거는 Anthropic (2025c)이다. 배경 정보와 지시와 도구 사용 지침과 출력 규약을 각각 제 자리에 두라는 권고이고, 그 밖의 실무 기법 목록도 함께 정리되어 있다.

절 5.3의 근거는 형식 축의 Sclar et al. (2024)과 순서 축의 Lu et al. (2022)다. 표 5.1의 여섯 자리는 그 변형 공간을 실무 프롬프트에 맞게 이 책이 줄여 적은 것이므로, 자기 시스템의 자리는 원문의 정의로 다시 세는 편이 낫다. 다만 둘 다 특정 시점의 모델들을 대상으로 한 실험이므로, 가져가야 할 것은 민감성의 크기가 아니라 단일 설정의 점수를 신뢰할 수 없다는 경고다.

프롬프트 작성 요령의 목록은 각 제공자의 최신 문서에 잘 정리되어 있고 모델 세대마다 갱신된다. 어떤 세대에서 값을 하던 문장이 다음 세대에서는 과잉이 되므로, 목록을 그대로 옮겨 적는 것은 이 장이 경고한 그 실수다. 어떤 요령을 만나든 물을 것은 셋이다. 그것이 고정부의 세 역할 가운데 어디에 속하는지, 그 문장이 배제하는 출력이 무엇인지, 그리고 그 효과를 점이 아니라 범위로 평가할 수 있는지. 목록은 낱아도 이 세 물음은 남는다.

압축과 요약 전략

압축은 손실이다.

이력의 한 구간을 더 짧은 것으로 바꾸면 원문은 컨텍스트에서 사라지고, 대개 돌아오지 않는다. 그래서 압축의 물음은「얼마나 줄일 것인가」가 아니라「무엇을 잃을 것인가」다.

그리고 이 손실은 피해 갈 수 있는 것이 아니다. 압축하지 않으면 이력은 스텝마다 자라고, 자란 이력은 값을 올리고 성능을 떨어뜨린다. 압축하면 그 사람은 멈추지만 버린 것은 돌아오지 않는다. 컨텍스트가 늘어나서 떨어지는 성능을 감당할 것인가, 이력을 압축하며 나오는 손실을 감당할 것인가. 둘 중 하나는 반드시 치른다. 안 치르는 길은 없다.

이 교환이 이 장 전체의 축이다. 아래의 모든 결정은 이 교환의 어느 쪽을 얼마나 치를지를 정하는 일이고, 잘된 압축과 잘못된 압축의 차이도 손실의 유무가 아니라 무엇을 손실로 골랐느냐에 있다.

6.1 자라는 층

제4장이 컨텍스트를 다섯 층으로 갈랐다. 지시 고정부, 도구 정의, 검색해 온 지식, 이력과 관측, 인계 산출물. 다섯 가운데 스텝을 따라 자라는 것은 이력 하나뿐이다. 나머지 넷은 실행 전이나 세션 경계에서 크기가 정해지고 그대로 있다. 이력만 스텝마다 관측과 행동이 덧붙으면서 자라고, 손대지 않으면 줄지 않는다(표 4.1).

자리도 나쁘다. 시스템 프롬프트는 맨 앞이고 새 관측은 맨 뒤이므로, 자라는 구간은 컨텍스트의 한가운데다. 하필 가장 잘 안 읽히는 자리다(그림 4.3).

그래서 압축을 들일지는 오늘의 비중이 아니라 스텝 수의 궤적을 보고 정한다. 도구 정의가 예산의 절반을 차지하는 그림은 루프가 길어지면 뒤집힌다. 기울기를 가진 층이 하나뿐이기 때문이다.

정의 6.1 (압축). 컨텍스트에 실린 이력의 한 구간을 그보다 짧은 표현으로 교체하는 변환을 압축(compression)이라 한다. 교체된 구간의 원문은 컨텍스트에서 사라지며 일반적으로 복원되지 않는다. 곧 압축은 손실 있는(lossy) 변환이다. 원문을 교체하기 전에 컨텍스트 밖으로 먼저 내보내 두면 손실의 범위가 컨텍스트 안으로 한정되며, 그 순서를 절 6.6이 규율로 세운다.

캐시가 남긴 것

압축으로 넘어가기 전에 치울 것이 하나 있다. 매 스텝 컨텍스트 전체를 처음부터 다시 읽는다는 말은 절반만 맞다. 두 호출의 토큰 열이 앞에서부터 똑같이 이어지는 구간은 다시 계산할 이유가 없고, 그 구간을 저장해 두었다가 훨씬싼 값에 다시 쓸 수 있다. 그렇게 이어지는 구간을 앞부분이라 부른다.

규칙은 하나다. 앞부분은 완전히 같아야 하므로 앞에서부터 처음 달라지는 자리를 만나는 순간 그 뒤가 전부 무효가 된다(Anthropic, 2026a). 그래서 변하지 않는 것을 앞에, 변하는 것을 뒤에 둔다. 매 호출 달라지는 값을 앞머리에 두면 토큰 하나 때문에 앞부분 전체를 잃는다.

여기서 국면이 하나 더 갈린다. 이미 주어진 토큰을 읽는 일과 새 토큰을 하나씩 만들어 내는 일은 병목이 다르고 값도 다르다. 캐시가 돕는 것은 읽는 쪽뿐이다. 쓰는 쪽, 곧 출력 토큰은 캐시가

표 6.1: 캐시가 없애는 값과 남기는 값. 남는 것 열이 비어 있는 행이 하나도 없다.

항목	캐시가 하는 일	남는 것	다루는 곳
같은 앞부분을 다시 읽기	다시 계산하지 않는다	새로 덧붙는 부분	이 절
실려 세어지는 토큰 수	아무것도 하지 않는다	스텝마다 자라는 이력 전부	이 장
출력 과금	아무것도 하지 않는다	전부. 요약도 여기서 나 간다	절 6.5
자라는 속도	낮추지 못한다	압축이 받거나 격리가 받는다	이 장, 제16장

아무것도 하지 못하며 세 축 가운데 가장 비싸다. 이 사실이 절 6.5에서 다시 나온다. 요약을 만드는 일이 바로 그 쓰는 쪽에서 일어나기 때문이다.

참고 6.1 (캐시는 단가만 낮추고 자라는 속도는 그대로 둔다). 앞부분이 스텝마다 보존되고 새 내용이 뒤에만 덧붙는 루프에서, 캐시가 작아지게 하는 것은 다시 읽는 토큰의 단가뿐이다. 적중한 토큰도 여전히 컨텍스트에 실려 세어지므로, 세어지는 개수가 스텝을 따라 늘어나는 속도는 캐시를 켜기 전과 같다.

6.2 압축의 두 목적

컨텍스트에는 넷이 함께 실린다. 무엇을 하라는 지시, 무엇을 부를 수 있는지 적은 도구 정의, 검색해 온 문서, 그리고 지금까지의 이력. 창은 하나이고 넷이 그 하나를 나누어 갖는다. 그러니 이력이 자라면 나머지 셋의 몫이 줄어든다. 줄어드는 것은 자리만이 아니다. 지시는 이력 더미에 묻혀 힘을 잃고, 방금 검색해 온 문서는 스무 스텝 전의 도구 결과와 같은 창에서 자리를 다룬다.

증상은 대개 이렇게 나타난다. 짧은 실행에서는 잘 지켜지던 규칙이 서른 스텝쯤 지나면 지켜지지 않는다. 검색은 제대로 된 문서를 가져오는데 답에 반영되지 않는다. 이럴 때 손이 먼저 가는 곳은 지시문이지만, 고쳐야 할 것은 대개 지시문이 아니라 이력의 크기다. 지시는 그대로인데 지시가 차지하는 몫이 달라진 것이다.

그러므로 압축은 이력을 위해 하는 것이 아니라 나머지 셋을 위해 하는 것이다. 이력을 줄여서 이력이 좋아지는 것이 아니다. 이력을 줄여야 지시가 다시 읽히고, 도구 정의가 제 이름값을 하고, 검색해 온 것이 제 몫을 한다. 압축의 성패를 평가하는 자리도 그래서 압축된 구간이 아니라 나머지 셋이 하던 일이다.

둘째 목적은 품질이고, 이쪽에는 실증이 있다. Jiang et al. (2024)는 모델의 성능이 입력에서 핵심 정보의 밀도와 그 위치에 달려 있다는 관찰에서 출발한다. 밀도를 올리는 압축으로 성능을 높이면서 비용과 지연도 함께 줄인다고 보고한다. 압축이 비용 절감 수단일 뿐 아니라 성능 수단이라는 것, 이것이 여기서 가져오는 핵심이다.

두 목적은 대체로 같은 방향을 가리킨다. 버릴 것을 잘 고르면 길이가 줄면서 남는 것의 밀도가 오른다. 어긋나는 지점이 설계가 실패하는 지점이다.

어긋나는 것은 목표를 압축률로 잡을 때다. 압축률은 길이 축 하나만 측정하는 값이다. 같은 비율로 줄인 두 컨텍스트가 관련성 축에서는 정반대일 수 있다. 하나는 무관해진 원문을 걷어내 밀도가 올랐고, 다른 하나는 결정의 이유를 걷어내 헛거워졌다. 지표는 둘을 같은 숫자로 보고한다. 손실은 청구서에 나타나지 않는다. 버린 것이 나중에 필요해져 다시 얻으러 가고, 한 스텝을 줄여 번 것을 늘어난 스텝 수가 도로 가져간다.

핵심. 압축의 척도는 압축률이 아니라 밀도다. 얼마나 줄였는가 수단의 지표이고, 남은 것이 얼마나 관련되어 있는가가 목적의 지표다. 압축률을 목표로 삼는 순간 관련된 것이 함께 잘려 나가고, 그 손실은 비용이 아니라 품질과 스텝 수로 뒤늦게 청구된다.

지시문의 문법이 여기서 갈린다. 요약 지시문에 이렇게 적으면 목표가 길이가 된다.

「지금까지의 이력을 절반으로 줄여라.」

무엇을 버릴지가 전부 요약자의 재량으로 넘어간다. 반대로 적어야 한다.

「고친 파일의 목록과 아직 통과하지 못한 시험의 이름은 그대로 남겨라. 길이는 그것을 지킨 다음에 맞춰라.」

남겨야 할 것의 목록을 주고, 길이는 그 목록을 지킨 뒤에 따라 나오는 결과로 둔다. 두 문장의 차이가 압축률과 밀도의 차이다.

6.3 무엇을 버릴 수 있는가

버릴 것을 고르는 기준은 셋이다.

- **되찾을 수 있는가.** 잘못 버려도 다시 가져오면 그만인 것부터 버린다. 환경에 남아 있는 것, 곧 파일의 내용이나 같은 인자로 다시 부를 수 있는 조희의 결과는 잘못 버려도 왕복 한 번이면 돌아온다.
- **최근 스텝이 참조했는가.** 여러 스텝 동안 아무도 다시 보지 않은 구간이 다음 후보다. 다만 이 기준은 과거를 보고 미래를 짐작하는 것이므로 혼자서는 약하다. 앞 기준과 겹쳐 쓴다. 뜸하게 참조되면서 되찾을 수 있는 것이 첫 후보다.
- **이미 남았는가.** 수정하기 전의 파일 내용, 폐기한 계획, 고치기 전의 오류 메시지. 부패한 것은 자리를 차지할 뿐 아니라 오도한다. 흔적 없이 버리기는 여기에만 쓴다.

스텝 단위로 옮기면 셋이 한 문장으로 접힌다. 이후의 스텝이 그것 없이 같은 결정을 다시 내릴 수 있는가.

손실은 두 부류로 갈린다. 같은 토큰 수를 버려도 대가가 다르다.

- **되찾을 수 있는 손실.** 원문이 환경이나 외부 저장에 남아 있고 도로 가져올 수단이 있는 경우다. 최악 비용이 재획득 왕복 한 번을 넘지 않는다. 압축은 이런 것부터 손대야 한다.
- **되찾을 수 없는 손실.** 컨텍스트에만 존재했던 것이다. 왜 이 경로를 골랐는지, 어떤 대안을 기각했는지, 무엇을 시도해 실패했는지, 지금의 계획이 어떤 가정 위에서 서 있는지. 환경 어디에도 없다. 다시 만들려면 그 판단과 시도를 다시 겪어야 한다.

되찾을 수 없다는 것이 손대지 말라는 뜻은 아니다. 이 안에서 갈림이 한 번 더 있다. 흔적 없이 버리는 것과 줄여 담는 것이 다르고, 되찾을 수 없는 것에 금지되는 것은 앞의 것뿐이다. 다만 줄여 담고 나면 그 줄인 표현이 유일한 사본이 된다. 그래서 무엇을 남길지가 요약자의 재량이 아니라 미리 적힌 목록이어야 하고, 그 목록을 절 6.6이 세운다.

가장 위험한 것은 실패의 기록이다

되찾을 수 없는 손실 가운데 가장 위험한 것은 실패의 기록이다. 이유는 셋이다.

첫째, 실패는 산출물을 남기지 않는다. 성공한 작업은 파일과 검증 결과라는 흔적을 환경에 남기지 만 실패한 시도는 그 반대다. 에이전트가 설정 파일을 고치는 접근을 시도했다가 그것이 다른 모듈을 깨뜨린다는 것을 확인하고 수정을 되돌렸다고 하자. 파일은 원래대로이고 검사는 다시 통과하므로

표 6.2: 압축의 네 방법. 위의 둘은 내용을 읽지 않고 종류만 보고 정할 수 있으므로 값이 싸다. 아래로 갈수록 판단이 깊어지고, 깊어진 판단이 곧 위험이다.

방법	무엇을 하는가	무엇을 잃는가	언제 쓰는가
도구 결과 비우기	이미 호출된 도구의 결과를 지우고 호출 자체는 남긴다	결과와 원문. 같은 인자로 다시 부르면 돌아온다	거의 언제나 첫 후보다. 판단이 필요 없고 손실이 되찾을 수 있는 쪽이다
그냥 잘라 내기	오래된 구간을 내용을 읽지 않고 통째로 건너낸다	그 구간 전부. 원문과 결론이 함께 간다	부패한 구간에만. 낡은 계획과 고치기 전의 오류가 그렇다
밖에 두고 가리키기	구간을 파일이나 저장소로 옮기고 포인터만 남긴다	즉시성. 다시 보려면 스텝 하나를 쓴다	크고, 되찾을 수 있고, 다시 볼지 모르는 것에
요약으로 갈음하기	모델이 자기 이력을 자연어로 요약해 원문과 바꾼다	요약자가 덜 중요하다고 판단한 전부. 실패의 기록이 가장 먼저 마모된다	앞의 셋으로 덮이지 않고 남는 서사에만

환경에는 시도의 흔적이 없다. 「그 길은 가 보았고 막혀 있었다」는 사실은 컨텍스트의 몇 문장에만 있다. 그러므로 실패 기록의 재획득 비용은 조화 한 번이 아니라 그 실패를 다시 겪는 비용이다.

둘째, 실패의 기록은 루프를 앞으로 나아가게 하는 바로 그 정보다. 제2장의 자기 반성 패턴은 실패를 언어로 요약해 다음 시도의 입력으로 삼는 구조였고, 그 패턴의 가치 전체가 실패 기록의 보존에 걸려 있다.

셋째, 잃었을 때의 증상이 조용하다. 오류는 나지 않고 같은 막다른 길을 다시 확인하는 스텝이 늘 뿐이며, 그 재방문은 제3장의 셈을 타고 비용과 지연으로 돌아온다.

여기서 압축의 기본 방향이 나온다. 이력의 한 구간은 대개 관측의 원문과 그 관측에서 내린 결론으로 이루어져 있다. 파일 300줄의 내용이 원문이라면 「이 파일에는 찾는 함수가 없다」가 결론이다. 원문은 크고 되찾을 수 있다. 결론은 작고 되찾을 수 없다. 그러므로 원문을 버리고 결론을 남긴다. Anthropic (2025c)이 이력을 요약할 때 보존하라고 꼽는 것도 같은 부류다. 구조에 관한 결정, 아직 풀리지 않은 문제, 구현의 세부를 남기고 중복된 도구 출력을 버린다.

6.4 압축의 방법

실제로 할 수 있는 일은 넷이다. 그냥 잘라 내거나, 도구 결과를 비우거나, 요약으로 갈음하거나, 밖에 두고 가리킨다. 표 6.2가 넷을 나란히 놓은 것이고, 아래는 각각을 언제 쓰는지다.

도구 결과 비우기가 첫 후보인 이유는 판단이 거의 필요 없다는 데 있다. Anthropic (2025c)은 이력 깊숙이에서 이미 호출된 도구의 원 결과를 계속 보여 줄 이유가 없다고 지적하고, 이것을 가장 안전하고 손이 덜 가는 압축으로 꼽는다. 앞 절의 기준으로도 그렇다. 도구 결과는 대개 되찾을 수 있는 손실이다. 도구의 호출은 남기고 결과만 지우는 것이 기본값이며, 무엇을 이미 해 보았는가의 기록은 그래서 남는다.

판단이 거의 필요 없는 대신 정할 값이 넷 있다. 언제 켜지는가, 최근 몇 개는 남기는가, 한 번에 최소 얼마를 비우는가, 어떤 도구의 결과는 비우지 않는가. 뒤의 둘이 실무에서 자주 빠진다. 조금씩 자주 비우면 캐시만 깨고 얻는 것이 없으므로 한 번 비울 때 캐시를 깨는 값어치가 있을 만큼을 비워야 하고(절 6.5), 결과가 곧 산출물인 도구는 비우지 않을 목록에 넣어 두어야 한다. 모델이 낸 중간 추론도 같은 방식으로 다룬다. 종류만 보고 들어낼 수 있고, 정할 값은 최근 몇 턴어치를 남기는가 하나다.

그냥 잘라 내기는 가장 거칠다. 무엇이 걸리는지 보지 않으므로 그 구간의 결론도 함께 간다. 그래

서 부패한 것에만 쓴다. 오래되었다는 이유만으로 이력의 앞쪽을 자르면 대개 가장 값진 것을 자른다. 초기 스텝에는 과제의 규정과 첫 판단이 들어 있기 때문이다.

밖에 두고 가리키기는 크기의 주인을 바꾼다. 대화의 길이는 스텝 수에 매여 있지만 파일의 크기는 과제의 구조에 매인다. 「기록이 아니라 산출물」이라는 제15장의 원리가 정확히 이것이고(절 15.2), Rajasekaran (2026)가 세션 사이의 인계 수단으로 꼽은 구조화된 산출물이 그 실물이다. 읽는 것은 즉시성뿐이고, 다시 보려면 스텝 하나를 쓴다.

요약으로 갈음하기는 마지막에 온다. 판단이 깊은 만큼 관련성 축을 개선할 여지가 가장 크고, 위험도 정확히 그 판단에 있다. 요약자는 대개 그 이력을 만든 모델 자신이다. 제2장이 「결론을 정당화하는 서사」라 부르고 제15장이 평가자에게 주지 말라고 한 그 서사가 요약에 스며든다. 진행은 부풀려지고 실패는 뭉개진다. 가장 위험하다고 한 실패의 기록이 가장 먼저 마모되는 자리가 여기도.

그래서 이 방법에서 실제로 설계하는 물건은 알고리즘이 아니라 요약 지시문이다. Anthropic (2025c)은 다듬는 순서를 준다. 관련된 것을 놓치지 않는 정도를 먼저 올리고, 군더더기가 없는 정도를 나중에 올린다. 검색에서 재현율과 정밀도라 부르는 그 둘이다. 처음에는 하나도 빠뜨리지 않도록 넉넉하게 쓰고, 그렇게 얻은 요약에서 덜어 내며 조인다. 순서가 반대면 무엇이 빠졌는지 알아낼 방법이 없다. 빠진 것은 요약에 남지 않기 때문이다. 같은 자료가 실물도 하나 든다. 이력을 요약하면서 가장 최근에 접근한 파일 다섯 개를 요약과 나란히 보존하는 구현이다. 손실 변환 옆에 손실 없는 조각을 몇 개 붙여 두는 설계이고, 무엇을 지시문에 맡기고 무엇을 규칙으로 못 박을지의 한 가지 답이기도 하다.

넷은 배타적이지 않고 순서가 있다. 종류만 보고 정할 수 있는 것을 먼저 하고, 밖으로 옮길 수 있는 것을 옮기고, 그러고도 남는 서사를 요약한다. 요약부터 하면 요약자가 상대할 대상이 필요 이상으로 커지고, 커진 대상은 판단을 더 많이 요구하고, 그 판단이 이 장에서 가장 비싼 것이다.

직접 짜는 것만이 아니다

넷 가운데 둘은 이제 코드로 짤 일이 아니다. 오래된 도구 결과를 걸어내는 일과 이력을 요약으로 갈음하는 일이 요청의 인자로 제공된다. 인자로 넘기는 값은 위에서 정한 것과 같다. 앞의 것은 문턱과 남길 개수와 한 번에 비율 최소량과 건드리지 않을 도구를 받고, 뒤의 것은 문턱 하나와 지시문 하나를 받는다. 요약 뒤에 무엇을 원문 그대로 붙일지도 따로 정하는 자리가 있다.

바뀐 것과 바뀌지 않은 것을 갈라 두는 편이 낫다. 알고리즘은 손을 떠났다. 지시문과 보존 목록과 문턱은 그대로 남는다. 그리고 하나 더, 아무 값도 넘기지 않으면 기본값이 일한다. 압축이 코드에도 프롬프트에도 없이 일어나는 구성에서도 압축은 일어나고 있다. 잘리는 자리를 정하지 않았다면 정하지 않기로 정한 것이다.

6.5 압축은 공짜가 아니다

압축 한 번에는 값이 세 군데서 든다.

첫째, 압축 호출 자체다. 요약을 만드는 것도 모델 호출이라서 압축할 이력을 읽는 입력과 요약을 쓰는 출력이 들고, 이 호출은 루프의 스텝과 별도로 과금된다. 그리고 요약은 출력이다. 출력은 캐시가 들지 못하는 가장 비싼 축이다(표 6.1). 압축이 아픈 이유가 이 순서에 있다. 요약은 가장 비싼 축에서 나오고, 요약이 지운 자리는 가장 싼 축을 읽는다.

둘째, 캐시 무효화다. 압축은 컨텍스트를 중간에서 고치는 일이라 교체 지점부터의 앞부분이 무효가 된다(그림 6.1). 이 값은 배치로 줄일 수 있다. 지시 고정부의 끝에 캐시 경계를 따로 잡아 두면 압축이 고정부의 캐시까지 깨지는 않는다. 무효가 되는 것은 교체 지점 뒤쪽뿐이다.

셋째, 정보 손실이다. 잘못 버리면 재획득과 재방문으로 스텝 수가 늘어난다. 이 값만 청구서에 나타나지 않는다.



그림 6.1: 앞부분 재사용과 무효화. 음영이 다시 쓰이는 구간이다. 덧붙기만 하는 스텝에서는 앞의 전부가 다시 쓰이고 새로 붙은 것만 새로 읽힌다. 압축은 중간을 고치는 일이라 고친 자리부터 뒤가 전부 새로 읽힌다. 그래서 압축의 값에는 요약을 쓰는 값 말고 이 무효화의 값이 함께 든다.

얼마나 자주 할 것인가

두 극단을 짚으면 구도가 보인다. 매 스텝 압축하면 캐시가 돕지 못하는 출력을 매번 지불하고, 컨텍스트의 중간을 매번 고치니 앞부분도 매번 무효가 된다. 한 번도 압축하지 않으면 컨텍스트가 스텝을 따라 자라고 매 스텝 그 전부를 다시 읽는다. 양쪽 끝이 다 손해인데 손해의 성격이 반대다.

명제 6.1 (압축 주기의 교환). 압축을 자주 할수록 압축 자체의 값이 커지고, 드물게 할수록 이력을 다시 읽는 값이 커진다. 두 값은 주기에 대해 반대 방향으로 움직이므로 그 합이 가장 작아지는 주기가 양 끝 사이에 하나 있다. 자주 압축할수록 좋은 것도, 미룰수록 좋은 것도 아니다.

주기를 하나로 고정해 두면 그다음이 따라 나온다. 압축 직후마다 컨텍스트가 같은 크기로 돌아오므로 스텝당 값이 어떤 선을 넘지 않고, 그래서 스텝 수가 아무리 늘어도 총 입력 비용이 스텝 수보다 빠르게 자라지 않는다. 캐시가 하지 못한 일이 이것이다.

그 자리가 어디인지는 세 값이 정한다. 스텝당 이력이 얼마나 붙는가, 요약 한 번이 얼마나 큰가, 다시 읽는 값과 새로 쓰는 값의 비가 얼마인가. 셋 다 오늘의 스텝 로그에서 측정할 수 있다.

셋 가운데 가장 크게 작용하는 것이 요약의 크기다. 요약이 크면 압축 한 번이 비싸고, 비싼 압축은 드물게 해야 한다. 요약을 길게 쓰는 시스템은 그 길이만큼 압축을 드물게 한다. 이 한 문장이 실무에서 가장 자주 쓰인다.

가져갈 것이 하나 더 있다. 계산은 스텝으로 하고 설정은 길이로 한다. 압축을 거는 자리는 대개 스텝 수가 아니라 길이의 문턱이어서 한 번 환산해야 한다. 압축 직후의 크기에 한 주기 동안 쌓일 양을 더하면 그 주기가 끝나는 자리의 길이다. 고정부가 2,000토큰이고 요약이 400토큰이고 스텝당 300토큰이 붙는데 열두 스텝에 한 번 압축하기로 했다면, 문턱에 적어 넣을 값은 6,000토큰 언저리다. 이 환산이 없으면 주기는 계산해 놓고 적어 넣을 자리가 없는 값이 된다.

압축 직전의 컨텍스트가 제4장의 유효 길이를 넘어서면, 비용이 최적이라 말하는 주기여도 그 주기는 쓸 수 없다. 잘못 버린 것의 기대 비용을 셈에 넣으면 압축 한 번의 값이 올라가고 최적 주기가 길어진다. 손실이 큰 압축일수록 드물게 해야 한다는, 방향이 직관과 일치하는 보정이다.

6.6 반복 압축의 문제

압축이 반복되면 두 번째부터 새 문제가 생긴다. 압축의 입력에 이전 압축의 요약이 들어 있다는 것이다.

주기가 올 때마다「지금까지의 이력을 요약하라」를 실행하면 회차마다 요약은 앞선 요약을 재료로 삼는다. 요약의 요약의 요약이다. 길이는 잘 통제된다. 문제는 손실 쪽이다. 요약 한 번은 원문에서 고른 것이다. 판단이 틀렸어도 틀림은 한 겹이고, 원문이 남아 있는 한 무엇이 잘못됐는지 대조할 수 있다. 10번째 요약은 9번의 판단을 통과해 살아남은 것에서 고른 것이다. 어느 회차의 판단도 뒤

회차가 무엇을 필요로 할지 모른 채 내려지고, 버려진 것은 다음 회차의 입력에 아예 없으므로 그 오판은 영구적이다. 손실은 회차가 늘수록 쌓이기만 하고 줄지 않는다. 요약은 회를 거듭할수록 무엇이 있었는가의 기록이 아니라 앞선 요약들이 무엇을 중요하다고 판단했는가의 기록이 된다.

이 실패는 제15장의 인계와 정확히 반대다. 요약을 덧붙이기만 하는 인계는 정보는 지켜지지만 회차가 늘수록 크기가 그만큼 자란다. 반복 요약은 크기가 붙들리는 대신 정보가 회차마다 깎인다. 무너지는 방식도 다르다. 자란 크기는 청구서에 곧바로 나타나지만, 깎인 정보는 같은 실패의 반복과 성공률로 뒤늦게 나타난다.

둘 중 무엇을 할지는 과제 경계가 정한다. 제15장이 문턱에서 할 수 있는 일을 둘로 갈랐다. 이력을 줄여 같은 세션을 이어 가거나, 세션을 닫고 인계를 남기거나. 하던 일이 끝났으면 닫는 쪽이 낫고, 끝나지 않았으면 압축하고 이어 간다. 갈림을 정하는 것은 문턱이 아니라 과제의 사정이다.

답은 두 극단 사이에서 적당히 버리는 것이 아니라 두 요구를 서로 다른 장치에 맡기는 것이다. 크기는 갱신형 요약이 붙든다. 요약 안에서 끝난 항목은 지워지고, 크기는 회차 수가 아니라 살아 있는 과제의 크기에 매인다. 정보의 보존은 컨텍스트 밖이 지킨다. 그 밖의 자리에 이름을 준다.

정의 6.2 (근원). 압축된 구간의 내용이 컨텍스트 밖에 그대로 남아 있는 자리를 그 구간의 근원(origin)이라 한다. 근원이 있으면 요약은 언제든 다시 만들 수 있는 파생물이고, 없으면 요약이 곧 원본이다.

근원이 있으면 재요약의 규칙이 정해진다. 압축의 입력은 앞선 요약이 아니라 근원이어야 한다. 앞선 요약을 다시 요약하는 것은 손실 위에 손실을 쌓는 일이다. 근원에서 다시 요약하면 때면 그 시점의 판단으로 새로 선별한다.

근원이 어디 있는지는 시스템마다 다르고, 없는 것이 기본값이다. 이력을 줄여 담는 장치는 대개 줄이기 전의 이력을 대신 보관해 주지 않는다. 줄인 표현 앞의 것은 다음 요청에 실리지 않고, 다음 요약은 이미 줄어든 상태 위에서 만들어진다. 곧 아무것도 하지 않으면 요약의 요약이 기본값으로 일어난다.

그래서 근원은 멀리 있지 않다. 검색 가능한 저장이나 세션을 넘는 기억이기 이전에, 하네스가 이미 쥐고 있는 원 메시지 목록이다. 그것을 그대로 붙들고 있으면 근원이 있고, 요청에 실을 것만 남기고 버리면 근원이 없다. 그 선택은 대개 명시적으로 내려지지 않는다. 더 오래 남겨야 하는 것은 제4부와 제10장이 받지만, 이 주기에 필요한 근원은 그 장들을 기다리지 않아도 이미 손안에 있다.

무엇이 회차와 무관하게 살아남아야 하는지도 요약자의 재량에 맡기지 않는다. 활성 과제의 규정, 실패의 목록과 그 이유, 미해결 결정. 되찾을 수 없는 손실의 목록이 그대로 보존 목록이 된다. 되찾을 수 없는 것은 버림의 대상이 아니라 보존의 대상이다. 줄여 담는 것까지 막는 규칙이 아니라, 줄인 표현이 유일한 사본이 되므로 무엇을 남길지가 미리 적혀 있어야 한다는 규칙이다.

압축은 루프의 표준 연산 가운데 유일하게 흔적을 남기지 않는 연산이다. 절단은 인계 산출물을 남기고, 파일 수정은 환경에 남으며, 틀린 행동조차 관측으로 기록된다. 되돌릴 수 없는 연산의 규율은 어디서나 같다. 실행하기 전에 원본의 행선지를 정한다. 순서를 하네스가 강제하면 된다. 압축할 구간을 먼저 컨텍스트 밖의 기록으로 내보내고, 그다음에 교체한다. 그러면 압축은 파괴가 아니라 이동이 되고, 근원에서 다시 요약한다는 전제도 그 순서에서 마련된다. 모델의 재량에 남는 것은 그 테두리 안에서 요약의 문장을 쓰는 일뿐이어야 한다.

가장 크게 비어 있는 자리는 손실 쪽이다. 무엇이 잘렸는지 알려면 잘리지 않은 실행과 견주어야 하는데 그 실행은 대개 존재하지 않는다. 그래서 주기를 계산으로 정한다는 말은 절반만 맞다. 반쪽인 비용은 계산되고 나머지 반쪽인 손실은 추정된다. 이 비대칭을 아는 설계자는 주기를 계산값보다 길게, 곧 덜 압축하는 쪽으로 잡는다.

6.7 압축하지 말아야 할 때

다음 조건에 해당하면 압축을 들일 때가 아니다.

1. **이력이 짧다.** 과제가 몇 스텝이면 압축할 시점이 오기 전에 끝난다. 압축 장치는 자기 값과 유지보수 복잡도만 청구하고 아무것도 절감하지 않는다. 컨텍스트가 자기 과제로 측정한 유효 길이에 한참 못 미친다면 품질 쪽 이득도 없다.
2. **잃은 것을 되찾을 수단이 없다.** 원 기록이 컨텍스트 밖 어디에도 없고 재획득 도구도 없다면 모든 손실이 되찾을 수 없는 쪽으로 격상된다. 순서를 바꾸어야 한다. 먼저 원 기록과 산출물을 밖에 두는 장치를 갖추고, 그다음에 압축한다. 압축은 근원이 있을 때 안전한 기법이다(정의 6.2).
3. **압축 품질을 측정할 수단이 없다.** 이득인 줄어든 토큰은 즉시 보이고 손실인 품질과 재방문은 뒤늦게 홀어져 나타난다. 평가하지 않으면 압축은 언제나 이득처럼 보인다. 평가해야 할 것은 과제 성공률, 압축 비용을 포함한 총비용, 스텝 수, 그리고 재획득 호출의 빈도다. 무효화 손실을 계산에 넣지 않으면 비용조차 늘 수 있다(참고 6.1).

셋에 공통된 형태가 있다. 압축은 이득과 손실이 관측되는 시점이 서로 다른 기법이다. 줄어든 토큰은 그 스텝에서 바로 세어지지만, 잘못 버린 것의 값은 한참 뒤에, 그것도 압축의 이름표를 달지 않고 찍힌다.

그래서 압축 장치는 컨 채로 완성하지 말고 끌 수 있게 들여야 한다. 장치를 꺼서 만료를 검사하라는 제15장의 처방이 여기서는 더 무겁게 적용된다. 압축의 손익은 장치를 끈 실행과 견주었을 때만 온전히 관측된다. 끄는 스위치가 없는 압축은 이득만 관측되는 채로 영구히 남는다. 결과를 즉시 볼 수 없는 연산일수록, 그 연산을 의심할 수단을 함께 들여야 한다.

6.8 요약

한 문장으로. 압축은 되돌릴 수 없는 손실 변환이므로 척도는 얼마나 줄였는가 아니라 무엇을 남겼는가다. 그리고 자주 할수록 좋은 것이 아니라 양쪽 끝이 다 손해라서 그 사이에 최적의 주기가 따로 있다.

확실해진 것.

- 손실은 선택지가 아니다. 컨텍스트가 자라서 떨어지는 성능을 감당하거나 압축의 손실을 감당하거나 둘 중 하나는 치른다. 스텝을 따라 자라는 층은 이력뿐이고, 캐시는 다시 읽는 단가를 깎을 뿐 자라는 속도를 낮추지 못한다(참고 6.1, 표 6.1).
- 창은 하나이고 지시와 도구 정의와 검색해 온 지식과 이력이 그 하나를 나누어 갖는다. 이력이 자라면서 나머지 셋의 몫이 줄어든다. 그러므로 압축은 이력을 위해 하는 일이 아니라 나머지 셋을 위해 하는 일이고, 성과도 압축된 구간이 아니라 나머지 셋이 하던 일에서 읽는다(절 6.2).
- 압축의 척도는 압축률이 아니라 밀도다. 압축률은 길이 축만 측정하는 값이라 같은 비율의 두 압축이 관련성 축에서 정반대일 수 있고, 잘못된 압축의 손실은 품질과 스텝 수로 뒤늦게 청구된다. 지시문에는 압축률이 아니라 보존 목록을 적는다.
- 버림의 기준은 셋이다. 되찾을 수 있는가, 최근 스텝이 참조했는가, 이미 남았는가. 한 문장으로는 「이후의 스텝이 그것 없이 같은 결정을 다시 내릴 수 있는가」이며, 방향은 원문을 버리고 결론을 남기는 것이다. 가장 위험한 손실은 실패의 기록이다. 환경에 흔적을 남기지 않아 되찾는 값이 그 실패를 다시 겪는 값이고, 잃어도 오류가 나지 않아 증상이 조용하다(절 6.3).
- 방법은 넷이고 순서가 있다(표 6.2). 도구 결과 비우기, 그냥 잘라 내기, 밖에 두고 가리키기, 요약으로 갈음하기. 종류만 보고 정할 수 있는 것을 먼저 하고 요약을 마지막에 둔다. 앞의 둘은 이제 요청의 인자로 제공되기도 하지만, 정할 값과 지시문과 보존 목록은 그대로 남는다.
- 압축 한 번의 값은 세 군데서 든다. 요약을 쓰는 출력, 교체 지점부터의 캐시 무효화, 그리고 잘못 버린 것. 앞의 둘이 주기에 반대로 작용하므로 최적의 주기가 하나 있다(명제 6.1). 그 자리를 정하는 것은 스텝당 증분과 요약의 크기와 단가의 비이고 셋 다 오늘의 로그에서 측정한다. 계산은 스텝으로 하고 설정은 길이로 하며, 주기의 상한은 비용이 아니라 유효 길이가 정한다(절 6.5).
- 반복 요약은 손실을 회차마다 쌓는다. 크기는 갱신형 요약이 지키고 정보는 밖의 근원이 지킨다(정의 6.2). 압축의 입력은 앞선 요약이 아니라 근원이어야 하고, 가장 가까운 근원은 하네스가 이미 쥐고 있는 원 메시지 목록이다. 아무것도 하지 않으면 요약의 요약이 기본값으로 일어난다. 되찾을 수 없는 것은 버림의 대상이 아니라 보존의 대상이다(절 6.6).

아직 답하지 않은 것.

- 한 주기의 근원은 손안의 원 메시지 목록으로 충분하지만, 그것은 실행이 끝나면 사라진다. 실행을 넘어 남기고 되찾는 장치, 곧 검색은 제4부가, 세션을 넘는 기억은 제10장이 받는다.
- 밖에 두고 가리키기가 기대는 산출물은 도구가 만든다. 관측을 처음부터 작고 구조화된 형태로 돌려주게 하는 도구 쪽 설계는 제3부, 특히 제7장이 받는다.
- 압축은 그릇 하나 안의 처방이다. 그릇 자체를 나누어 각 컨텍스트의 이력을 짧게 만드는

격리는 제16장이 받는다.

- 보존된 실패의 기록이 루프 밖에서 갖는 쓸모, 곧 평가 자료로서의 실행 기록은 제18장 부터의 제6부가 받는다.

오늘 할 것. 늦은 스텝 하나의 입력을 넷으로 나누어 쏜다. 지시, 도구 정의, 검색해 온 것, 이력이 각각 몇 토큰인가. 이력이 절반을 넘었다면 나머지 셋은 이미 밀리고 있다(절 6.2). 다음으로 그 이력에서 가장 큰 도구 결과 하나를 골라 마지막으로 쓰인 스텝을 찾는다. 그 뒤로 아무도 보지 않은 채 실행 다닌 스텝 수가 지금 당장 별 수 있는 양이다(절 6.4). 마지막으로 코드를 한 곳만 본다. 요청에 실패한 것만 남기고 원 메시지 목록을 버리고 있다면, 이 시스템은 줄인 것 위에서 다시 줄이고 있다(절 6.6).

6.9 생각해 볼 문제

1. 자기 시스템에서 압축이 잘못 버릴 만한 것을 하나 꼽아 보라. 그것을 잃었을 때 드는 값이 초회 한 번인가, 그 일을 다시 겪는 값인가. 그 값이 커지면 주기는 어느 쪽으로 움직여야 하며, 그 값을 미리 줄여 두는 수단은 이 장의 어느 절에 있는가.
2. 컨텍스트가 자라지 않게 붙드는 같은 일을 세션 안의 압축으로도, 세션의 절단과 인계(제15장의 그림 15.2)로도 이를 수 있다. 둘의 스텝당 비용을 나란히 적고 무엇이 선택을 정하는지 논하라.
3. 실제 이력 한 구간을 골라 표 6.2의 네 방법을 차례로 적용해 보라. 방법마다 줄어든 토큰 수를 측정하고, 사라진 정보를 되찾을 수 있는 손실과 없는 손실로 분류하라. 그 구간 이후의 로그에서 사라진 정보가 다시 필요해진 사례가 있었는가.
4. 보존 목록을 명시한 요약 지시문과 명시하지 않은 지시문으로 같은 이력을 요약시키는 비교 실험을 설계하라. 실패한 시도가 요약에 남았는지와 이후의 재방문 빈도를 어떻게 측정할 것인가. 요약자와 같은 모델이 그 요약을 판정해도 되는가를 절 15.5의 원리로 논하라.

6.10 다시 읽을 때

- 제4부와 제10장을 읽고 정의 6.2로 돌아오면, 근원이 실행보다 오래 사는 장치의 이름을 얻는다. 둘은 수명만 다르다.
- 제7장을 읽고 표 6.2로 돌아오면, 도구가 처음부터 산출물에 가까운 형태로 돌려줄 때 이력이 애초에 서사로 자라지 않는다는 것이 보인다.
- 제16장을 읽고 명제 6.1로 돌아오면, 그 계산이 그릇 하나를 전제한 것이었음이 보인다. 격리는 같은 문제를 주기가 아니라 그릇의 수로 풀고, 압축의 손실 대신 그릇 사이 인계의 비용을 치른다.
- 제18장을 읽고 보존 목록으로 돌아오면, 무엇을 시도했고 왜 실패했는지의 기록이 그 자체로 평가 사례의 원석임이 보인다. 보존 목록은 세션의 규율이면서 평가 자료의 수집 규칙이다.

6.11 더 읽을거리

새로 인용한 두 문헌은 압축을 서로 다른 문제로 다룬다. Jiang et al. (2023)는 예산 배분의 문제로 보고, Jiang et al. (2024)는 성능의 문제로 본다. 뒤엣것이 압축으로 품질을 올릴 수 있다는 주장의 실증적 근거다. 다만 두 편 모두 알고리즘의 세부는 빠르게 갱신되므로 기법이 아니라 문제 설정을 읽기를 권한다. 밖에 두고 가리키기의 실물은 Rajasekaran (2026)에 있다.

제 III 부

도구와 외부 세계

도구 설계 원칙

도구를 고치는 것이 프롬프트를 고치는 것보다 값을 하는 자리가 있다. 모델이 할 수 있는 일은 도구 목록이 정하고, 목록에 없는 행위는 어떤 지시로도 일어나지 않기 때문이다.

이 장이 답할 것은 둘이다. 도구 하나가 어떤 구조로 실려 와서 어떤 왕복을 거치는가. 그리고 그 구조의 각 자리를 어떻게 채워야 성능이 오르는가.

두 물음은 같은 넷을 놓고 묻는다. 도구명, 도구 설명, 인자, 반환이다. 도구가 어디서 공급되는지는 제8장이, 반환의 구조를 계약으로 만드는 일은 제9장이 받는다.

7.1 도구의 구성

도구 하나는 세 자리로 적힌다. 도구명, 도구 설명, 인자의 구조다. 반환은 어디에도 적히지 않는다. 세 자리를 실물로 채우면 이렇게 된다.

도구명: 「`search_orders`」

도구 설명: 「고객의 주문을 조회한다. 주문 번호나 고객 식별자를 알고 있을 때 쓴다. 최근 90 일치만 돌려주며 그보다 오래된 주문은 나오지 않는다. 결제 상세와 배송 추적은 담기지 않는다.」

인자: 「`customer_id`」는 문자열, 필수. 「`status`」는 「`pending`」, 「`shipped`」, 「`delivered`」 가운데 하나, 생략하면 전부. 「`limit`」은 정수, 생략하면 20.

도구명에는 제약이 있다. 영문자와 숫자와 밑줄과 불임표만 쓰고 64자를 넘지 못한다(Anthropic, 2026j). 도구 설명과 인자 구조에는 길이 제약이 없다. 제약이 없는 쪽이 값을 무는 쪽이라는 것을 절 7.2에서 본다.

반환이 정의에 없다는 것은 그냥 넘길 일이 아니다. 무엇이 어떤 모양으로 돌아오는지 모델이 미리 아는 길은 도구 설명 한 줄뿐이고, 나머지는 실제로 불러 본 뒤에야 안다. 설계자가 반환을 어떻게 잘라 보내든 그것을 검사할 자리가 없다는 뜻이기도 하다. 그래서 반환은 이 장에서 가장 늦게 세는 자리가 아니라 가장 무겁게 세는 자리이며, 절 7.4가 그것을 받는다.

이 셋이 어디에 실리는가

세 자리는 텍스트가 되어 요청에 실린다. 모델이 보는 것은 도구가 아니라 이 텍스트다. 코드도 못 보고 실제 동작도 모른다.

값도 여기서 시작된다. 도구를 하나라도 주면 도구를 쓰는 법을 알리는 안내문이 자동으로 따라 붙는다. 최근 모델에서 그 안내문만 300토큰 안팎이고, 도구를 만드시 쓰게 하면 400토큰을 넘는다(Anthropic, 2026j). 여기에 도구마다의 도구명과 도구 설명과 인자 구조가 더해진다. 이 전부가 실행 전에 확정되어 매 스텝 같은 자리에 다시 실린다.

왕복

모델은 도구를 실행하지 않는다. 도구명 하나와 인자 하나를 내놓을 뿐이다.

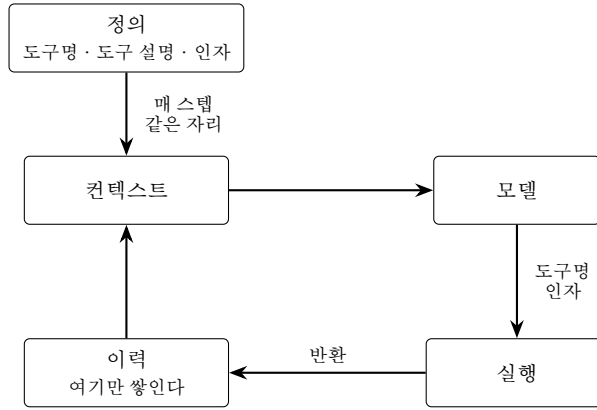


그림 7.1: 도구 하나의 왕복. 정의는 들어가기만 하고 반환은 쌓인다. 정의를 한 줄 늘리면 모든 스텝의 같은 자리가 그만큼 길어지고, 반환을 한 번 크게 돌려주면 그 뒤의 모든 스텝이 그것을 지고 간다.

그 출력력을 받아 실제 함수를 부르는 것은 하네스다. 하네스가 결과를 받아 다시 컨텍스트에 넣어 주어야 모델이 그것을 본다. 그림 7.1이 이 왕복이다.

이 왕복에서 두 자리의 회계가 다르다. 정의는 스텝마다 같은 크기로 다시 실리고 쌓이지 않는다. 반환은 한 번 실리고 그 뒤로 계속 실려 다닌다. 절 7.2가 앞의 자리를, 절 7.4가 뒤의 자리를 다룬다.

목록에 없는 행위

파일 전체를 읽는 도구와 파일 전체를 새로 쓰는 도구만 가진 에이전트가 있다. 과제는 300줄짜리 파일에서 3줄을 고치는 것이다.

프롬프트에 무엇을 적든 이 에이전트가 할 수 있는 일은 300줄을 읽고 300줄을 다시 쓰는 것뿐이다. 「필요한 부분만 고쳐라」라는 지시에 이 두 도구로 대응할 길이 없다. 지시가 나빠서가 아니다. 부분 수정이라는 행위가 이 시스템에 없기 때문이다.

정의 7.1 (행위 어휘). 에이전트가 실제로 할 수 있는 일은 둘이 정한다. 어떤 도구가 있는가, 그리고 각 도구가 어떤 인자를 받아들이는가. 하네스가 실행하는 것은 도구 하나와 그 도구의 인자 구조를 통과하는 인자 하나로 이루어진 호출뿐이다. 이 호출들에 환경에 작용하지 않는 자연어 응답을 더한 것이 시스템의 행위 어휘 (action vocabulary)이며, 제1장의 행동 공간이 바로 이것이다.

프롬프트로 고쳐지는 실패는 한 종류뿐이다. 있는 도구 가운데 덜 좋은 것이 뽑히는 실패다. 필요한 도구가 아예 없거나 도구들이 잘못 갈려 있는 실패는 지시를 아무리 베풀어도 남는다.

실증도 있다. Yang et al. (2024)는 모델을 그대로 두고 인터페이스만 바꾸었는데 파일을 다루고 저장소를 돌아다니고 프로그램을 실행하는 능력이 크게 달라졌다고 보고한다.

목록에는 언제나 자연어 응답이 남아 있다는 것도 짚어 둔다. 말하기는 금지할 수 없다. 그래서 필요한 도구가 없을 때 모델은 침묵하지 않고 말로 때운다. 조회하지 못한 것을 지어내는 실패다. 어휘의 구멍은 침묵이 아니라 지어내기로 메워진다.

실행이 실패했을 때 팀이 의심하는 것은 대개 프롬프트, 모델, 컨텍스트의 셋이다. 도구는 거기 없다. 도구 목록은 처음 만들 때 정해진 뒤로 시스템의 배경이 되어, 바뀌지 않으므로 의심되지 않는다. 진단 목록에 오르지 않는 원인은 고쳐지지 않는다. 실패 분석의 첫 물음은 「지시가 나빴는가」가 아니라 「이 과제에 필요한 도구가 있었는가」여야 한다.

7.2 도구명과 도구 설명이 성능을 정한다

모델이 도구를 고르는 근거는 도구명과 도구 설명뿐이다. 그러므로 도구명과 도구 설명이 곧 성능이다.

도구명

목록을 훑을 때 가장 먼저, 가장 자주, 가장 확실하게 읽히는 토큰이 도구명이다. 도구 설명은 도구명이 만든 기대를 통과한 뒤에야 읽힌다.

그리고 도구명은 선택만 정하는 것이 아니라 기대를 만든다. 모델은 학습에서 무수한 코드를 보았고 거기서 도구명이 어떤 행동과 짝지어지는지의 관례를 익혔다. 도구명은 그 관례를 통째로 끌고 들어온다.

나란히 놓으면 보인다.

나쁜 도구명: 「get_data」, 「process」, 「handle_request」, 「do_action」, 「helper」

좋은 도구명: 「search_orders」, 「cancel_subscription」,

「get_customer_context」, 「schedule_event」

왼쪽은 무엇을 하는지가 도구명에서 읽히지 않으므로 선택을 통째로 도구 설명에 떠넘긴다. 오른쪽은 동사와 대상이 붙어 있어서, 도구 설명을 읽기 전에 이미 후보에서 걸러진다.

기대가 실제와 어긋나면 대체로 도구명이 이긴다. 도구명은 짧고 목록에 언제나 실리며 굳은 관례를 등에 업고 있다. 도구 설명의 단서는 길고 여럿 사이에 묻힌다. Patil et al. (2024)가 짚은 두 실패 가운데 하나가 API 호출의 잘못된 사용법을 지어내는 경향인데, 지어내진 사용법은 무작위 잡음이 아니다. 모델은 아무렇게나 틀리는 것이 아니라 그럴듯하게 틀리며, 그럴듯함의 재료가 바로 도구명이 만든 기대다.

여기서 도구명 짓기의 규칙 셋이 나온다.

- 관례와 싸우지 않는다. 관례상 다른 뜻으로 읽힐 도구명을 쓰고 도구 설명으로 바로잡는 설계는 지는 싸움에 문장을 보태는 일이다. 고쳐야 할 것은 도구명이다.
- 목록에 대고 짓는다. 홀로 보면 좋은 도구명 둘이 한 목록에서 서로를 흐리는 일은 흔하다. 판정은 한 줄로 된다. 사람이 어느 도구를 써야 할지 단정하지 못하면 모델도 못 한다.
- 묶음에는 공통의 앞머리를 준다. 같은 서비스나 같은 자원을 다루는 도구들에 공통 앞부분을 붙인다. 「github_list_prs」와 「slack_send_message」처럼 서비스로 묶거나, 「orders_search」와 「users_search」처럼 자원으로 묶는다. 목록이 길어져도 어느 묶음의 물건인지가 도구명에서 갈린다(Anthropic, 2025g).

인자 이름에도 같은 규율이 걸린다. 「user」라고 적으면 이름이 올지 식별자가 올지 갈리지 않는다. 「user_id」라고 적으면 갈린다(Anthropic, 2025g).

도구 설명

도구 설명에 무엇을 적는가. 새로 온 동료에게 이 도구를 설명한다고 생각하면 목록이 나온다. 무엇을 하는 도구인가. 언제 쓰고 언제 쓰지 않는가. 각 인자가 무엇을 뜻하고 동작을 어떻게 바꾸는가. 그리고 무엇을 돌려주지 않는가.

마지막 항목이 자주 빠진다. 도구를 만든 사람에게는 당연해서 적히지 않고, 모델은 그것을 알 길이 없어서 있는 줄 알고 부른다.

공개된 대비를 그대로 옮기면 이렇다(Anthropic, 2026j). 같은 도구의 도구 설명 둘이다.

나쁜 도구 설명: 「종목의 주가를 가져온다.」

좋은 도구 설명: 「주어진 종목 기호의 현재 주가를 조회한다. 종목 기호는 주요 증권거래소에 상장된 기업의 유효한 기호여야 한다. 최근 체결가를 달러로 돌려준다. 특정 종목의 현재 또는 가장 최근 가격을 물을 때 쓴다. 그 밖에 종목이나 기업에 관한 정보는 제공하지 않는다.」

나쁜 쪽은 한 문장이다. 이 도구가 언제 맞는지, 무엇을 돌려주는지, 무엇을 돌려주지 않는지가 전부 열려 있다. 좋은 쪽은 다섯 문장이고 넷을 다 닫는다. 기준은 이 대비에서 읽힌다. 도구마다 서너 문장 이상, 복잡한 도구는 더.

7.3 최적화된 도구 설계

도구명이 행위를 고르게 했다면 인자는 고른 행위를 채우게 한다. 그리고 인자 구조는 설계자가 모델의 행동에 가할 수 있는 제약 가운데 유일하게 결정적으로 집행되는 것이다. 나머지는 전부 프롬프트, 곧 확률의 세계에 산다.

같은 제약을 두 자리에 적어 보면 차이가 분명해진다. 「이 도구를 부를 때는 반드시 대상 식별자를 밝혀라」를 프롬프트에 적으면 대체로 지켜지고 가끔 지켜지지 않으며, 지켜지지 않은 호출은 그대로 실행된다. 같은 제약을 필수 인자로 적으면 식별자 없는 호출은 실행에 도달하기 전에 기계적으로 거부된다.

참고 7.1 (강제와 지시는 위반을 다른 사건으로 만든다). 하네스가 도구 호출을 실행하기 전에 인자를 인자 구조와 대조하고 위반한 호출을 거부의 관측으로 돌려준다고 하자. 같은 제약을 프롬프트의 지시로 적으면 그것은 행동의 분포를 기울일 뿐이다. 이때 (a) 인자 구조가 강제한 제약의 위반은 환경에 도달하기 전에 차단되어 언제나 관측 가능한 거부가 되고, (b) 지시로 적은 제약의 위반은 0이 아닌 확률로 실행되며, 위반 여부는 실행의 결과에서만, 그것도 결과가 위반의 흔적을 남길 때만 알려진다.

무게는 위반의 확률 차이보다 위반의 처지 차이에 있다. 인자 위반은 실행 전의 사건이라 되돌릴 것이 없고, 거부라는 관측이 남아 모델이 곧바로 고칠 수 있다. 지시 위반은 실행 후의 사건이라 환경에 작용이 남는다.

인자를 채우기 쉽게

Patil et al. (2024)가 짚은 나머지 한 실패가 정확한 입력 인자를 생성하지 못하는 것이다. 이쪽을 줄이는 손질이 넷 있다.

인자를 줄인다. 인자가 늘면 채워야 할 자리가 늘고, 자리마다 틀릴 기회가 하나씩 생긴다. 필수 인자 하나에 선택 인자 한둘이 채우기 가장 쉬운 형태다. 다섯을 넘으면 대개 도구 하나가 두 가지 일을 하고 있다는 신호이니 가르는 쪽을 먼저 본다.

열거할 수 있는 값은 열거한다. 자유 문자열은 표집이 채우고 표집은 흔들린다. 같은 자리를 이렇게 적으면 흔들릴 자리가 없다.

열기 전: 「status」는 문자열. 도구 설명에 「주문 상태를 적는다」.

열고 난 뒤: 「status」는 「pending」, 「shipped」, 「delivered」, 「cancelled」 가운데 하나.

위쪽에서는 모델이 「배송중」이라고 적을 수도 있고 「shipping」이라고 적을 수도 있다. 아래쪽에서는 넷 밖의 값이 실행에 도달하지 못한다.

기본값을 둔다. 생략하면 무엇이 되는지가 정해져 있으면 모델은 그 자리를 비워도 된다. 기본값 하나가 인자 하나를 줄인다. 위의 「limit」이 그렇다.

표 7.1: 경계의 교환. 조개기와 합치기는 오류를 없애는 것이 아니라 오류의 자리를 선택과 인자 사이에서 옮긴다.

	잘게 쪼갬 목록	굵게 합친 목록
선택	후보가 많고 이웃이 비슷해져 잘못 고른다	후보가 적어 선택이 쉽다
인자	도구마다 적고 단순하다	모드 인자와 조건부 필수가 생겨 인자 오류의 표면이 넓어진다
정의 층	정의의 수가 늘어 매 스텝의 뭉치 자란다	정의는 적으나 하나가 길어진다
오류의 자리	틀린 도구를 고른다	옳은 도구를 틀리게 채운다
개정의 파급	행위 하나의 변경이 도구 하나에 갇힌다	행위 하나의 변경이 공유 인자 구조를 흔든다

틀릴 수 없는 형태로 바꾼다. 어떤 인자는 아무리 설명해도 틀린다. 그러면 설명을 늘릴 것이 아니라 인자를 바꾼다. 공개된 사례가 파일 경로다. 상대 경로를 받는 도구에서 모델이 디렉터리를 옮긴 뒤 경로를 자꾸 틀렸는데, 절대 경로만 받도록 바꾸자 그 오류가 사라졌다. 상대 경로는 「지금 어디에 있는가」를 기억해야 맞힐 수 있고, 절대 경로는 기억할 것이 없다. 틀릴 수 있는 자리를 없애는 편이 틀리지 말라고 적는 것보다 낫다.

형식을 고를 때도 같은 눈으로 본다. 모델이 만들기 쉬운 형태와 어려운 형태가 따로 있다. 파일을 고치는 도구에서 바뀐 줄만 받는 형식은 줄 번호를 미리 세어야 맞는다. 파일 전체를 받는 형식은 셀 것이 없다. 세기가 필요한 형식은 그 세기의 실패를 도구가 떠안는다.

모델이 채울 수 없는 인자는 인자가 아니다

인자 오류의 출처는 둘로 갈리고, 처방이 정반대다.

하나는 인자 구조가 말하지 않은 것이다. 형식이 열려 있고, 허용되는 값이 문안에만 있고, 단위가 암묵적인 경우다. 이쪽은 조여서 없앤다. 위의 넷이 그 일이다.

다른 하나는 모델이 알 수 없는 것이다. 대상의 식별자, 실행 환경의 경로처럼 컨텍스트에 실려 있지 않으면 알 길이 없는 값이다. 이쪽은 조일수록 나빠진다. 필수로 강제된 자리는 반드시 채워져야 실행에 도달하는데, 모델이 값을 모르면 그 자리는 지어낸 값으로 채워진다. 강제는 부재를 없애는 대신 지어내기를 부른다.

그리고 지어낸 값은 조용하다. 지어낸 식별자는 형식이 맞으므로 인자 검사를 통과하고 실행된다. 없는 대상에 대한 오류로, 더 나쁘게는 엉뚱한 대상에 대한 성공으로 돌아온다.

그래서 필수라는 표시를 붙이기 전에 물어야 할 것이 하나 생긴다. 이 값을 모델이 이 시점의 컨텍스트에서 알 수 있는가. 알 수 없다면 처방은 인자 구조가 아니라 공급 경로에 있다. 그 값을 알아낼 도구가 곁에 있어야 하고, 조회의 반환이 다음 호출의 인자가 되도록 도구들이 이어져야 한다.

도구를 어떻게 가를 것인가

여기까지는 도구 하나였다. 이제 목록이다.

행위를 잘게 갈라 도구를 여럿 두면 각 도구는 단순해진다. 도구명이 정확해지고 인자가 적어진다. 대신 목록이 길어져 고르기가 어려워진다. 크게 합쳐 도구를 하나로 두면 고르기는 쉬운데, 그 안에 모드 인자와 조건부 필수가 생겨 채우기가 어려워진다. 두 방향 모두 오류를 없애 주지 않는다. 오류가 나는 자리를 옮길 뿐이다. 표 7.1이 이 교환이다.

교환이라고 해서 기본값이 가운데 있는 것은 아니다. 방향은 분명하다. 도구가 많다고 결과가 좋아지지 않으며, 바깥 시스템의 끝점을 하나하나 감싸는 대신 값이 큰 작업 흐름을 겨냥한 도구 몇

개를 만드는 편이 낫다(Anthropic, 2025g). 공개된 대비 셋을 그대로 옮긴다.

「list_users」와「list_events」와「create_event」대신,
빈 시간을 찾아 일정을 잡아 주는「schedule_event」하나.
「read_logs」대신, 관련된 로그 줄과 그 앞뒤만 돌려주는「search_logs」.
「get_customer_by_id」와「list_transactions」와「list_notes」대신,
고객의 최근 정보를 한 번에 모아 주는「get_customer_context」.

셋 다 같은 일을 한다. 사람의 손으로 세 번 호출해야 끝나는 일을 도구 하나가 안에서 처리하면, 모델은 세 번의 선택과 세 번의 인자 채우기 대신 한 번의 결정을 한다.

그래서 경계의 기준은 도구의 크기가 아니라 결정의 단위다. 모델의 한 번의 결정으로 완결되는 행위가 도구 하나다. 언제나 함께 불리는 행위들을 갈라 두는 것은 한 결정을 여러 번의 선택으로 쪼개는 일이다. 경계가 결정의 단위와 맞으면 선택과 인자가 함께 좋아진다. 한쪽이 오르면 한쪽이 내리는 교환에서 벗어나는 유일한 길이다.

기본값이 어디서 오지도 의심해야 한다. 아무도 정하지 않으면 도구의 경계는 바깥 시스템의 경계를 물려받는다. 감싸는 API에 끝점이 다섯이면 도구도 다섯이 된다. 자연스러워 보이지만 그것은 구현의 경계이지 결정의 경계가 아니다. 사람 개발자를 위해 그어진 경계는 호출 사이에 상태를 들고 다닐 수 있는 사용자를 전제한다. 모델의 상태는 컨텍스트뿐이다. 경계를 설계했는가, 물려받았는가. 이것이 도구 목록을 감사할 때의 첫 물음이다.

정의의 다 신지 않는 길

도구를 줄여도 목록이 커지는 시스템이 있다. 그러면 남은 길은 전부를 신지 않는 것이다.

도구명과 한 줄 설명만 상시 신고, 인자 구조와 도구 설명 전문은 필요가 확인된 뒤에 읽어 들인다. 같은 장치가 지시 층에도 걸린다.

정의 7.2 (점진적 공개). 컨텍스트에 실을 수 있는 대상을 계층으로 나누어, 상위 계층의 요약은 모든 스텝에 상시 적재하고, 하위 계층의 본문은 그 요약을 근거로 필요가 확인된 시점에 읽어 들이며, 실행 가능한 부분은 컨텍스트에 들이지 않고 실행의 결과만 들이는 적재 방식을 점진적 공개(progressive disclosure)라 한다.

층은 셋이다(Anthropic, 2025f). 첫째 층은 이름과 설명이고 시작할 때부터 상시 실린다. 둘째 층은 지시 본문이고 관련된다고 판단했을 때 읽어 들인다. 셋째 층은 본문이 가리키는 참조 파일과 스크립트이며, 본문을 읽은 뒤 실제로 필요해진 것만 열린다. 셋째 층에 이 구조의 힘이 있다. 나머지는 파일 시스템에 있고 에이전트가 필요할 때 열어 보므로 한 묶음에 담을 수 있는 내용의 양에는 사실상 상한이 없다.

정의 7.3 (스킬). 특정 과제 유형을 수행하는 방법에 관한 지시와, 선택적으로 스크립트와 자원을 하나로 묶어 컨텍스트 밖에 두고, 요약을 통해 점진적 공개로 적재되는 단위를 스킬(skill)이라 한다.¹

공짜는 아니다. 상시 적재는 확실히 값을 치르고 확실히 거기 있다. 점진적 공개는 값을 덜 치르는 대신 필요할 때 불러 나온다는 보장이 없다.

선별이 실패하는 방식

실패는 셋이다. 부재는 요약이 지금 과제와의 관련을 알리지 못해 본문이 끝내 불러 나오지 않는 경우다. 실행은 지시 없이도 끝까지 돌고 결과만 다르다. 오선택은 반대다. 필요하지 않은 묶음이 불러 나와 예산과 주의를 함께 가져간다. 불이행은 본문이 실렸는데도 지시대로 되지 않는 경우이

¹ 이름과 구성은 Anthropic (2025f)에서 왔다. 구현의 파일 배치와 규약은 다루지 않는다. 가장 빨리 낚을 부분이기 때문이다.

표 7.2: 적재 방식으로 가른 네 갈래. 아래로 갈수록 실행의 소유권이 모델에서 환경으로, 다시 시스템 밖으로 나간다.

	누가 실행하는가	무엇이 실행되는가	언제 실행되는가
프롬프트	모델이 읽고 따른다	지시의 전문	실행 전에 확정, 모든 스텝에
스킬	지시는 모델이 따르고, 실행부는 환경이 실행한다	요약은 상시, 본문은 선택된 때만, 실행부는 결과만	요약은 첫 스텝에, 본문은 실행 시점에 모델의 선택으로
도구	환경이 실행한다	정의(도구명, 도구 설명, 인자 구조)와 호출의 결과	기본 배치에서 정의는 실행 전에, 결과는 호출 시점에
프로토콜	외부 공급자가 실행한다	공급된 도구의 정의와 결과, 그리고 자원	정의의 목록 자체가 실행 시점의 연결로 정해진다

며, 이것은 적재의 문제가 아니라 프롬프트의 문제로 제5장의 규율 아래 있다. 앞의 둘만이 요약의 책임이고, 그래서 공들일 자리는 본문이 아니라 요약이다. 도구 정의를 골라 실행 때도 같다. 실행하는 것은 도구명과 한 줄뿐이고, 절 7.2가 세운 도구명의 규율이 선별의 성과를 가른다.

무엇이 스킬이고 무엇이 도구인지는 기능이 아니라 적재로 물으면 흔들리지 않는다. 물을 것은 셋이다. 누가 실행하는가, 무엇이 컨텍스트에 실행되는가, 언제 실행되는가.

표 7.2 안에서 회색 지대가 자리를 찾는다. 스크립트를 품은 스킬은 둘째 행 안에서 두 성분으로 갈리고, 지침으로 비대해진 도구 설명은 셋째 행에 앉은 채 첫째 행의 비용을 묻다. 마지막 행의 실물이 제8장의 모델 컨텍스트 프로토콜이다. 행마다 새로 지불하는 것이 다르다. 첫째 행은 토큰과 검증의 값을, 둘째 행은 거기에 선택의 값을, 셋째 행은 실행 권한과 안전의 값을, 넷째 행은 공급자를 믿어야 하는 값을 더한다. 아래로 내려갈수록 우리가 검토하지 않은 것을 그대로 믿어 주어야 하는 범위가 넓어지며, 그 위험의 값은 제22장이 매긴다.

7.4 컨텍스트 안에서 도구의 출력

정의가 들어가는 쪽이라면 반환은 나오는 쪽이다. 그리고 나오는 쪽이 더 무겁다. 도구가 돌려주는 것은 곧 관측이고, 관측은 컨텍스트에 쌓이며, 다음 행동은 그 컨텍스트가 정한다. 반환의 형태를 정하는 일은 다음 스텝의 입력을 정하는 일이다.

무엇이 어떤 모양으로 오가는가

모델이 내놓는 것은 이런 모양이다. 호출 하나를 가리키는 식별자, 부를 도구명, 그리고 인자 객체다(Anthropic, 2026j).

「id」는「toolu_01A09q」,「name」은「search_orders」,
「input」은「{customer_id: “C-4471”, status: “shipped”}」

이것은 실행이 아니라 요청이다. 하네스가 이 도구명으로 실제 함수를 찾아 인자를 넘겨 부르고, 결과를 받아 짝이 되는 블록으로 되돌린다. 되돌리는 블록에는 셋이 실린다. 어느 호출에 대한 답인지 가리키는 식별자, 결과 본문, 그리고 오류였는지 아닌지의 표시다.

성공: 「tool_use_id」는「toolu_01A09q」,「content」는「주문 3건. C-4471. 배송중....」

실패: 같은「tool_use_id」에「is_error」는 참,「content」는

「호출 한도 초과. 60초 뒤에 다시 시도하라.」

세 가지가 여기서 확정된다. 첫째, 모델의 출력과 도구의 결과는 별개의 메시지이고, 결과는 반드시 그 호출 바로 뒤에 붙어야 한다. 둘째, 실패도 정상적인 반환이다. 예외를 던지는 자리가 아니라 표시를 하나 세우고 말로 적어 보내는 자리다. 셋째, 이 두 블록이 이력에 그대로 남는다.

셋째가 값이다. 스텝이 서른 번 도는 동안 이 쌓이 서른 벌 쌓인다. 열째 스텝의 반환은 열번째 스텝에도, 서른째 스텝에도 실려 있다. 한 번 크게 돌려준 반환은 한 번의 값이 아니라 남은 스텝 수만큼의 값을 묻는다.

그리고 도구에서 도구로 값을 옮기는 데 통로가 모델 하나뿐이라는 것도 여기서 나온다. 앞 도구의 출력이 뒤 도구의 입력이 되려면 그 값은 모델의 눈앞을 지나야 한다. 관측으로 한 번 읽고, 인자로 한 번 다시 쓰인다. 도구에서 도구로 건너가는 데이터는 두 번 값을 치른다. 게다가 다시 쓰는 일은 표집이므로, 인자에 적힌 값이 관측의 원본과 같다는 것은 보장이 아니라 확률이다.

길이: 상류에서 줄인다

조희 도구가 결과 전부를, 실행 도구가 로그 전체를 돌려주는 설계는 성실해 보인다. 그 성실의 값은 방금 센 대로다. 길이만 문제도 아니다. 반환에 섞인 무관한 부분은 정작 다음 결정에 필요한 신호를 제 안에 묻는다.

줄이는 수단은 넷이고, 넷 다 기본값을 함께 정해야 한다(Anthropic, 2025g). 쪽 나누기, 범위 고르기, 거르기, 잘라내기다. 어느 것도 도구를 쓰는 쪽의 재량으로 남기면 안 된다. 기본값이 없으면 첫 호출이 전부를 가져오고, 그 한 번으로 컨텍스트가 결정되기 때문이다.

반환의 크기에 상한을 두는 것도 같은 이유다. 공개된 실물에서는 도구 반환을 기본 25,000토큰으로 제한한다. 다만 잘라낼 때는 잘랐다는 사실과 나머지를 가져오는 법을 함께 돌려주어야 한다. 말없이 잘린 반환은 부분을 전부로 보이게 한다.

자세함을 모델이 고르게 하는 길도 있다. 반환의 형식을 「간단히」와 「자세히」 가운데 고르는 인자로 열어 두는 것이다. 공개된 대비에서 같은 결과가 자세히는 206토큰, 간단히는 72토큰이었다. 훑어보는 호출과 파고드는 호출이 같은 값을 묻지 않는다.

내용: 다음 호출이 채울 수 있게

모델이 알 수 없는 값이 지어내기를 부른다고 했다. 그 값의 대부분은 앞선 호출의 반환에서 온다. 그러므로 반환에 무엇을 담는가가 다음 호출의 인자 품질을 정한다.

규칙이 하나 나온다. 기계가 쓰는 식별자보다 사람이 읽는 이름을 돌려준다. Anthropic (2025g)은 뜻 없는 문자열 식별자를 뜻이 읽히는 이름으로 바꾸어 돌려주는 것만으로 정확도가 뚜렷이 올라간다고 보고한다. 이유는 앞의 논증으로 이어진다. 모델이 다음 호출에서 그 값을 다시 써야 하는데, 뜻 없는 문자열은 컨텍스트 안에서 서로 구별이 안 되고 이력이 길어질수록 뒤섞인다.

나란히 놓으면 이렇다.

줄일 것: 「uuid」, 「mime_type」, 「256px_image_url」

남길 것: 「name」, 「file_type」, 「image_url」

식별자가 꼭 필요하면 이름과 함께 돌려주면 된다.

반환의 구조 자체도 성능에 영향을 준다. 같은 내용을 표 형식으로 주는가, 중첩된 객체로 주는가, 줄 단위 텍스트로 주는가에 따라 평가 결과가 달라진다(Anthropic, 2025g). 어느 하나가 언제나 낫지는 않으므로, 이것은 자기 과제에서 평가해 볼 항목이지 외워 둘 답이 아니다.

제6장은 이력의 한 구간이 원문과 결론으로 이루어져 있고 압축의 방향은 원문을 버리고 결론을 남기는 것이라 했다. 반환 설계는 같은 일을 상류에서 한다. 도구가 결론에 가까운 형태를 돌려주면 버릴 원문이 애초에 컨텍스트에 실리지 않는다.

표 7.3: 실패 반환의 세 등급. 아래로 갈수록 실패에서 회복까지의 스텝 수가 줄어든다.

등급	반환이 말하는 것	모델의 다음 스텝	루프에 남는 것
조용한 실패	실패했다는 사실 자체가 없다. 빈 결과나 부분 결과가 성공의 형태로 돌아온다	거짓 관측 위에서 정상 진행	틀린 결론이 전제로 굳는다. 원인은 어느 로그에도 없다
뭉뚱그린 실패	실패했다는 사실뿐. 원인도 방향도 없다	원인을 추측해 재시도. 같은 호출의 반복이거나 무관한 수선	스텝이 소모되고, 진전 없음 감지(표 15.1)가 잡을 때까지 돈다
안내하는 실패	무엇이 왜 거부되었고, 무엇을 바꾸면 되는지	수정된 호출	실패가 한 스텝짜리 교정으로 끝난다

실패의 반환

성공의 반환보다 더 설계를 요구하는 것이 실패의 반환이다. 성공한 스텝의 다음 행동은 대개 과제가 정해 준다. 실패한 스텝의 다음 행동은 반환이 정해 준다. 모델이 실패에 대해 아는 전부가 그 반환이기 때문이다.

문서 검색 도구가 권한 문제로 검색을 수행하지 못했는데, 오류를 삼키고 빈 목록을 돌려준다고 하자. 모델이 보는 관측은「결과 없음」이고, 거기서 나오는 자연스러운 결론은「그런 문서는 없다」다. 이 결론은 거짓인데 관측과는 앞뒤가 맞으며, 이후 스텝들이 그 위에 쌓인다. 실패를 알리지 않는 반환은 실패를 없애는 것이 아니라 거짓 관측으로 바꾼다.

표 7.3이 세 등급이며, 가르는 축은 하나다. 반환이 다음 스텝을 얼마나 정해 주는가.

세 등급을 실물로 적으면 차이가 한눈에 보인다. 같은 실패에 대한 세 반환이다.

조용한 실패:「주문 0건」

뭉뚱그린 실패:「오류가 발생했습니다」

안내하는 실패:「이 고객의 주문을 볼 권한이 없다. 담당 팀 식별자를 함께 넘기거나 권한 조회 도구로 접근 가능한 고객 목록을 먼저 확인하라.」

참고 7.1의 (a)가 만든 거부의 관측은 이 눈금에서 자동으로 셋째 등급에 놓인다. 어느 인자가 어떤 검사에 걸렸는지가 기계적으로 알려지기 때문이며, 인자 강제의 숨은 이득이 여기 있다. 반대로 첫째 등급은 도구 안에서 실패를 처리해 준다는 선의로 자주 만들어진다. 오류를 우아하게 삼키는 코드는 사람 사용자에게는 친절일 수 있으나, 관측이 전부인 모델에게는 사실과 다른 세계를 보여 주는 일이다.

처방의 자리를 상류로 옮겨 보는 것이 반환 설계의 시험이다. 긴 이력에 시달리는 팀은 하류에 장치를 단다. 요약자를 붙이고 압축 주기를 조율한다. 전부 제6장이 다른 정당한 장치이지만, 그 장치가 상대하는 원문의 상당 부분은 도구가 만들어 보낸 것이다. 이 순서를 뒤집은 시스템은 도구가 쏟아 낸 것을 요약자가 도로 줄이는, 같은 정보를 두 번 값 치르는 구조가 된다.

정의를 고르고, 결과를 밖에 두기

지금까지 센 값이 둘이었다. 정의는 쓰지 않아도 나가고, 중간 결과는 쓸 때마다 나간다. 처방이 서로 다르다.

쓰지 않아도 나가는 값은 실을 것을 고르는 일로 줄인다. 절 7.3이 그 장치였다.

쓸 때마다 나가는 값에는 같은 처방이 통하지 않는다. 다음 호출의 인자로 필요한 값을 신지 않으면 사슬이 끊기므로 중간 결과는 선별의 대상이 아니다. 줄일 수 없다면 남는 길은 하나다. 통로를 바꾸는 것이다.

바꾸어도 되는지부터 물어야 한다. 분석의 대상이 되는 데이터는 모델이 읽고 그 내용이 다음 결정의 근거가 된다. 조작의 대상이 되는 데이터는 옮겨지고 걸러지고 세어질 뿐, 그 내용이 모델의 결정을 바꾸지 않는다. 판정 질문은 하나다. 이 값을 모델이 보지 않아도 다음 결정이 같은가. 같다면 조작이고, 다르다면 분석이다. 분석의 데이터는 컨텍스트에 있어야 한다. 조작의 데이터를 컨텍스트에 통과시키는 것은 값을 치르면서 오류의 기회까지 더하는 일이다.

정의 7.4 (코드 실행). 무엇을 부를지와 결과를 어떻게 엮을지를 모델이 코드로 쓰고, 그 코드를 실행 환경이 실행하는 방식을 코드 실행(code execution)이라 한다. 도구 목록을 컨텍스트에 실어 호출을 하나씩 뽑게 하는 대신, 도구를 부를 수 있는 실행 환경(execution environment)을 둔다. 중간 결과는 환경 안에 머물고, 컨텍스트에는 코드와 실행의 반환만 실린다.

Anthropic (2025b)가 제시하는 해법이 이 형태다. 도구를 직접 호출의 대상이 아니라 코드 API로 제시하고, 결과는 실행 환경 안에서 거른 뒤에 모델에게 넘긴다. 호출 다섯 번이 물려 있는 구간을 하나씩 부르면 중간 결과 다섯 벌이 전부 이력에 쌓이지만, 코드 실행에서는 코드 한 벌과 마지막 반환만 실린다. 스텝 수도 다섯에서 하나로 준다.

참고 7.2 (코드 실행은 어휘의 상한을 없애고 강제의 자리를 옮긴다). 행위 어휘(정의 7.1)는 도구 집합과 인자 구조가 함께 정하는 유효한 호출의 집합이고, 하네스는 실행 전에 인자를 대조한다. 그런데 코드 실행 도구의 인자는 프로그램이고, 실행 환경은 그 프로그램대로 환경 안의 도구들을 임의의 순서와 횟수로 호출한다. 이때 (a) 행동 하나가 담을 수 있는 도구 호출의 수와 조합에 상한이 없어지고, 행위 어휘는 열거 가능한 목록이기를 그친다. (b) 인자 대조가 확인할 수 있는 것은 프로그램의 형식까지다. 그 프로그램이 실행에서 어떤 호출 열을 만들지는 실행 전 검사로 강제되지 않는다. 결정적 강제가 설 수 있는 자리는 호출 전의 인자 구조에서 실행 환경의 경계로 옮겨 간다.

코드가 여는 것

코드는 톨로만 바꾸는 것이 아니다. 계획이 코드라는 형식을 얻는 순간 넷이 새로 열린다. 중간 결과를 파일로 써 두면 재개할 수 있고, 계획 전체가 실행되기 전에 텍스트로 존재하므로 실행 전에 검증할 수 있으며, 한 번 통한 처리를 환경에 함수로 남기면 다시 쓸 수 있고, 조작의 데이터가 환경에 머무르므로 민감한 값을 모델에 보이지 않을 수 있다(Anthropic, 2025b).

대신 관측이 뒤로 옮겨 간다. 하나씩 부르면 호출마다 관측이 이력에 남아 하네스의 장치들이 그 열을 보았다. 코드 실행은 그 열을 표현식 하나로 접는다. 환경 안의 호출 열은 컨텍스트가 아니라 환경의 기록으로 내려간다. 잃는 것이 아니라 옮겨지는 것이지만, 옮겨 간 자리에 기록 장치가 없으면 잃는 것이 된다.

신뢰 경계도 함께 열린다. 모델이 지어낸 코드를 돌리려면 격리와 자원 한도와 감시를 갖춘 실행 환경이 필요하고(Anthropic, 2025b), 이 운영 부담은 직접 호출에는 없던 것이다. 그래서 코드 실행의 도입은 도구 하나를 더하는 결정이 아니라 하네스의 가정 몇 개를 만료시키는 결정이다. 들이는 날 함께 세워야 할 것이 둘이다. 실행 전의 코드 검사, 그리고 실행 환경의 기록이다.

경계를 어디에 긋고 어떻게 지킬 것인가는 제22장의 소재다. 도구 반환이 대개 우리가 쓰지 않은 텍스트라는 것도 거기서 다시 만난다. 웹 페이지, 받은 메일, 외부 API의 응답은 전부 바깥에서 온 내용이고, 그 안에 모델을 돌려세우려는 문장이 섞여 있을 수 있다.

7.5 도구를 더 만들지 말아야 할 때

애매하게 정의된 도구를 여럿 만들 바에는 적은 수의 명확한 도구가 낫다.

이유는 목록이 선택의 무대이기 때문이다. 목록이 길어지면 고르는 일이 어려워진다. 비슷한 도구가 있으면 둘 다 안 불리거나 잘못 불린다. 도구 하나를 더하는 결정은 그 도구 하나의 결정이 아니다. 정의는 목록의 예산에 더해지고, 도구명은 목록의 구별에 얹히며, 선택은 목록 위의 사건이다.

그러므로 추가의 편익도 그 도구의 성공률이 아니라 목록 전체의 성공률로 평가해야 한다. 새 도구를 넣고 그 도구가 잘 쓰인다는 것만 확인한 팀은, 그 도구가 이웃의 선택 확률에서 무엇을 가져갔는지를 평가하지 않은 것이다.

모든 도구는 자기의 존재 이유를 정당화해야 한다(Anthropic, 2025g). 다음 셋 가운데 하나에 해당하면 새 도구를 만들 때가 아니다.

1. **실패의 원인이 도구의 배치다.** 이미 있는 도구가 오용되고 있을 때 그 오용을 피해 가는 새 도구를 더하는 것은 흔한 대응이지만 방향이 틀렸다. 원인이 도구명의 어긋난 기대라면 개명이(절 7.2), 경계가 결정의 단위와 어긋난 것이라면 재편이(절 7.3) 처방이다. 배치의 문제를 도구 추가로 풀려는 시도는 문제를 키우면서 목록을 늘린다.
2. **결정적으로 실행되어야 하는 절차다.** 같은 입력에 같은 동작이 나와야 하고 모델의 판단이 개입할 이유가 없다면, 그것은 도구가 아니라 코드나 워크플로의 뉘이다. 도구는 모델에게 선택을 맡기는 자리이므로, 선택이 필요 없는 절차를 도구로 만드는 것은 확실한 것을 확률에 넘기는 일이다.
3. **쓰이는 자리가 없다.** 만약을 위한 도구는 정의의 값을 확실히 지불하면서 목록을 흐리는 쪽으로만 작동한다. 어느 쪽으로도 값을 하지 못하는 도구다.

그러면 도구 몇 개까지가 괜찮은가. 실무에서 가장 자주 받는 이 질문에 수는 셀 수는 없다. 문턱은 도구의 수만으로 정해지지 않고, 도구명들이 서로 얼마나 갈리는가와 과제가 그중 몇을 실제로 부르는가에 함께 달려 있다. 공개된 자료도 목록이 커질 때 선택이 나빠진다는 방향까지만 말한다. 답은 수가 아니라 관측이다. 자기 시스템에서 도구별 호출 빈도와 오용 빈도를 세어 두면, 목록이 언제부터 나빠지는지는 남의 수가 아니라 자기 로그가 알려 준다.

7.6 요약

한 문장으로. 도구는 도구명과 도구 설명과 인자 구조로 실려 가고 반환으로 돌아오며, 모델이 보는 것은 그 텍스트뿐이다. 그래서 도구명이 선택을, 인자 구조가 강제를, 반환이 다음 스텝을 정하고, 도구 설계는 프롬프트보다 앞에 있는 행동 설계가 된다.

확실해진 것.

- 도구는 세 자리로 적히고 반환은 어디에도 적히지 않는다. 정의는 매 스텝 같은 자리에 다시 실리고 반환은 이력에 쌓인다(그림 7.1). 두 자리의 회계가 다르므로 처방도 다르다.
- 행위 어휘는 도구 목록과 인자 구조가 함께 정하는, 실행 가능한 행동의 전부다(정의 7.1). 프롬프트로 고쳐지는 것은 덜 좋은 도구가 뽑히는 실패뿐이고, 어휘의 구멍은 침묵이 아니라 지어내기로 메워진다.
- 모델이 고르는 근거는 도구명과 도구 설명뿐이다. 도구명은 학습된 관례의 기대를 끌고 오고, 어긋나면 대체로 도구명이 이긴다. 그러니 어긋난 도구명은 도구 설명으로 바로잡을 것이 아니라 개명한다. 도구 설명에는 무엇을 하고 언제 쓰고 무엇을 돌려주지 않는지를 서너 문장 이상 적는다. 마지막 항목이 가장 자주 빠진다(절 7.2).
- 인자는 줄이고, 열거할 수 있으면 열거하고, 기본값을 두고, 틀릴 수 있는 자리를 없앤다. 다만 강제는 모델이 알 수 있는 값에만 순수한 이득이고, 알 수 없는 값을 필수로 만들면 부재가 지어내기로 바뀐다(참고 7.1).
- 경계는 오류를 없애지 않고 선택 오류와 인자 오류 사이에서 자리를 옮긴다(표 7.1). 기준은 결정의 단위이고, 방향은 끝점을 하나씩 감싸는 대신 값이 큰 작업 흐름으로 합치는 쪽이다. 목록이 커지면 도구명과 한 줄만 상시 싣고 나머지를 나중에 읽어 들이는 길이 있으나(정의 7.2), 그 길은 값을 덜 치르는 대신 불러 나온다는 보장을 잃는다(절 7.3).
- 반환은 다음 스텝의 입력이다. 네 수단에 기본값과 상한을 주고, 뜻 없는 식별자보다 사람이 읽는 이름을 돌려주며, 실패는 세 등급을 따라 회복의 스텝 수를 정한다(표 7.3). 실패를 삼키는 반환은 실패를 거짓 관측으로 바꾼다.

아직 답하지 않은 것.

- 도구가 어디서 공급되고 그 목록이 언제 확정되는가, 곧 표 7.2의 마지막 행은 제8장이 받는다.
- 반환의 구조를 계약으로 만드는 일, 곧 출력 스키마의 일반론은 제9장이 받는다.
- 선택 오류와 인자 오류를 실제로 갈라 평가하는 설계는 제6부, 특히 제18장이 받는다.
- 어휘가 곧 권한의 목록이라는 관점, 곧 행위의 제한으로서의 도구 설계는 제21장이 받는다.

오늘 할 것. 도구 정의가 적힌 파일을 열어 도구마다 도구 설명이 몇 문장인지 센다. 한 문장짜리가 있으면 언제 쓰지 않는 도구인지와 무엇을 돌려주지 않는지를 거기에 적어 넣는다(절 7.2). 다음으로 필수로 표시된 인자를 하나씩 짚으며 이 값을 모델이 그 시점에 알 수 있는지 묻고, 알 수 없는 자리마다 그 값을 만들어 주는 도구가 목록에 있는지 확인한다(절 7.3). 마지막으로 로그에서 가장 큰 도구 반환 하나를 골라 토큰을 세고, 그 반환에서 다음 결정에 실제로 쓰인 줄에 표시한다(절 7.4).

7.7 생각해 볼 문제

1. 자기 도구 목록에서 도구 설명이 한 문장인 도구를 셋 고르고, 각각에 언제 쓰지 않는 도구인지와 무엇을 돌려주지 않는지를 적어 놓아라. 고치기 전과 후로 같은 과제 스무 건을 돌려 어느 도구가 불렀는지를 세어 보라. 도구 설명만 바뀌었는데 선택이 달라졌다면, 그 도구는 그동안 도구명만으로 뽑히고 있었던 것이다.
2. 자기 시스템의 도구 가운데 모델이 알 수 없는 값을 필수 인자로 요구하는 것을 찾아 보라. 그 값의 공급 경로, 곧 어느 도구의 반환이 그 값을 만들어 주는지를 그려 보고, 경로가 없다면 지어낸 값이 인자 검사를 통과해 실행되는 빈도를 측정하는 방법을 설계하라.
3. 실행 로그에서 선택 오류(틀린 도구가 불림)와 인자 오류(옳은 도구가 틀리게 채워짐)를 가르는 판정 절차를 적어 보라. 두 오류의 비가 표 7.1의 교환에서 어느 방향의 재편을 가리키는가.
4. 자기 도구들의 실패 반환을 표 7.3의 세 등급으로 분류하라. 조용한 실패가 있다면 그것이 만든 거짓 관측이 이후 스텝에서 어떤 결론으로 굳었는지 로그에서 추적해 보라. 등급을 한 단계 올리는 개수가 과제당 평균 스텝 수에 주는 효과를 평가하는 실험을 설계하라.

7.8 다시 읽을 때

- 참고 7.2를 읽고 표 7.1의 양 끝으로 돌아오면, 모든 것을 하는 도구 하나라는 끝점이 코드 실행이라는 이름을 얻는다. 어휘를 버리는 이 길과 어휘를 버리고 문법을 주는 그 길이 같은 교환의 양쪽이었다는 것이 보인다.
- 제9장을 읽고 질 7.4로 돌아오면, 반환을 결론의 형태로 돌려주라는 요구가 계약의 형태를 얻는다. 방향으로만 세운 「형태가 다음 스텝을 정한다」가, 그 형태를 실행 시점에 강제하는 장치로 채워진다.
- 제18장을 읽고 표 7.1로 돌아오면, 오류의 자리라는 행이 평가의 분해 축이 되어 있다. 과제 성공을 하나로 뭉뚱그려진 숫자를 선택과 인자와 반환 이해의 층으로 가르는 일이 그 장의 도구 평가다.
- 제21장을 읽고 정의 7.1로 돌아오면, 같은 어휘가 능력의 목록이 아니라 권한의 목록으로 읽힌다. 거기 없는 행위는 일어나지 않는다는 결론이, 일어나면 안 되는 행위를 목록에서 빼는 설계로 뒤집혀 쓰인다.

7.9 더 읽을거리

도구를 만드는 실무의 기록으로는 Anthropic (2025g)을 먼저 읽으라. 이 장의 실물 가운데 도구명 짓기와 묶음의 앞머리, 값이 큰 작업 흐름으로 합치는 세 예, 반환의 크기를 다루는 네 수단, 사람이 읽는 이름으로 돌려주기가 거기서 왔다. 이 장이 다루지 않은 것도 하나 있다. 도구 설명을 실제 과제 위에서 평가하면서 고치는 절차다. 구체 요령은 갱신될 것이므로 요구의 구조를 읽는 편이 오래간다.

규격 쪽의 실물은 Anthropic (2026j)에 있다. 도구명의 제약, 도구 안내문의 토큰 수, 호출과 결과 블록의 모양이 거기서 왔고, 좋은 도구 설명과 나쁜 도구 설명의 대비도 그 문서의 것이다.

도구 사용이 왜 어려운 과제인지의 실증은 Patil et al. (2024)에 있다. 여기서 가져온 것은 두 실패의 분류, 곧 인자를 정확히 생성하지 못하는 것과 사용법을 지어내는 경향이다. 실험의 수치는 남아 가지만 실패의 분류는 남는다. 자신의 로그에서 두 실패를 가려 세는 눈으로 읽기를 권한다.

제15장이 받은 Yang et al. (2024)도 함께 읽을 값어치가 있다. 여기의 언어로는 도구 목록의 재설계가 프롬프트의 조정을 우회해 성능을 움직인 실험이다.

모델 컨텍스트 프로토콜(MCP)

도구를 우리가 직접 만들지 않아도 된다. 남이 이미 만들어 둔 것을 끌어다 쓸 수 있다.

반대 방향도 있다. 우리가 만든 기능을 밖에서 쓰게 내놓을 수 있다. 옆 팀의 에이전트가, 또는 우리 회사 밖의 누군가가, 우리 시스템의 조회 하나를 자기 도구 목록에 얹는다.

두 방향을 함께 열어 주는 것이 하나의 공용 계약이고, 그 계약의 이름이 모델 컨텍스트 프로토콜이다. 계약이 정하는 것은 능력이 아니라 능력이 오가는 길이다. 프로토콜을 붙여도 시스템이 할 수 있는 일의 목록은 그대로다. 바뀌는 것은 그 목록에 하나를 더하는 값과 절차, 그리고 그 하나를 누가 썼는가다.

이 장은 그 값과 절차를 다룬다. 메시지가 어떤 형식으로 오가고 어떤 이름의 요청이 있는지는 다루지 않는다. 가장 빨리 남을 부분이고, 규격 자체가 개정되는 문서이기 때문이다.

8.1 외부와의 연결

제7장이 도구 하나를 어떻게 적는지를 세웠다. 도구명과 도구 설명과 인자를 짓고, 반환을 잘라 보내는 일이다. 거기서 답하지 않은 물음이 하나 남아 있다. 그 도구를 누가 만드는가.

한 조직에 에이전트 응용이 셋 있다고 하자. 코드를 다루는 조수, 사내 문서에 답하는 질의응답, 운영 지표를 지키는 당번 봇이다. 이들이 해야 할 자원은 넷이다. 코드 저장소, 사내 위키, 팀의 일정, 지표 화면. 응용 하나가 자원 하나에 닿으려면 이어 붙이는 코드가 하나 필요하다. 그 자원의 인증을 통과하고, 그 자원의 질의 형식으로 묻고, 그 자원이 내놓는 오류와 페이지 넘김을 다루는 코드다. 세 응용이 네 자원에 닿으니 이어 붙일 자리가 열두 곳이다.

이 코드가 왜 쌍마다 따로 필요한가. 두 사정이 겹치기 때문이다. 자원마다 말 거는 법이 다르고, 응용들은 서로 코드도 계약도 나누어 쓰지 않는다. 어느 한쪽만 성립하면 열두 곳까지 가지 않는데 둘이 함께 성립한다.

열두 곳이 아픈 것은 처음 만들 때가 아니라 그다음부터다. 위키가 인증 방식을 바꾸면 세 곳을 각각 고쳐야 한다. 넷째 응용을 만들면 이어 붙일 자리가 네 곳 늘고, 다섯째 자원을 들이면 세 곳이다. 한 걸음의 값이 언제나 반대쪽 목록의 크기만큼 커진다. 그리고 열두 개의 코드는 열두 곳에서 제각각의 품질로 쓰이고 제각각 남아 간다.

Anthropic (2024c)가 프로토콜을 공개하며 짙은 문제의식이 정확히 이것이다. 모델의 능력은 빠르게 나아졌으나 데이터와 끊겨 있고, 정보는 사일로와 낡은 시스템 뒤에 갇혀 있으며, 새 데이터 원천마다 고유한 구현이 필요해 연결된 시스템을 넓히기 어렵다는 것이다.

8.2 MCP

한 대의 컴퓨터에 기기를 붙이던 때를 떠올려 보자. 프린터에는 프린터의 선이 있었고 외장 저장장치에는 그것의 선이 있었다. 기기가 한 종류 늘면 컴퓨터 뒤에 꽂는 자리도 한 종류 늘었다. 그 뒤로 꽂는 자리의 모양이 하나로 정해졌다. 만드는 쪽은 그 모양에만 맞추면 되고, 쓰는 쪽은 무엇이 꽂히든 같은 방식으로 쓴다. 기기의 종류가 몇이든 컴퓨터가 익혀야 할 것은 하나다.

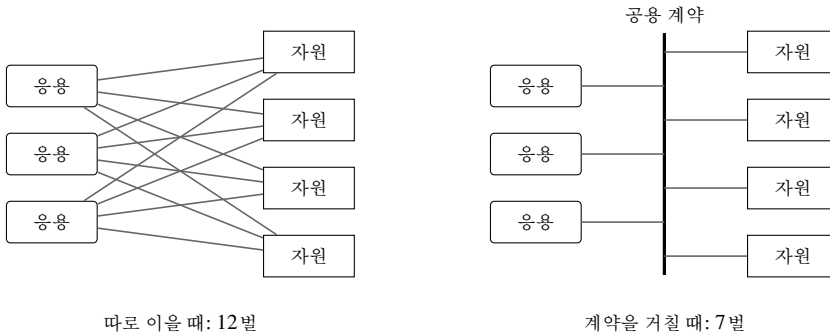


그림 8.1: 응용 셋과 자원 넷을 잇는 두 방식. 선 하나가 따로 짜야 하는 코드 한 벌이고, 오른쪽의 세로 막대가 모두가 맞추는 공용 계약이다.

같은 일을 자원과 응용 사이에서 한다. 모두가 맞출 공용 계약을 하나 정해 두면, 자원은 자기에게 맡겨는 법을 그 계약의 형식으로 한 번만 옮겨 두면 되고, 응용은 그 계약을 읽는 법을 한 번만 익힌다. 앞 절의 조직이라면 이어 붙일 자리 열두 곳이 만들 것 일곱 개가 된다. 자원마다 하나씩 넷, 응용마다 하나씩 셋이다. 그림 8.1이 두 경우를 나란히 놓은 것이다.

이 계약의 이름이 모델 컨텍스트 프로토콜이다.

계약의 양쪽

능력을 내놓는 쪽의 구현을 서버라고 부른다. 자기 시스템의 능력과 내용을 계약의 형식으로 내놓는다. 쓰는 쪽의 구현이 클라이언트다. 서버가 내놓는 것의 목록을 받아 오고, 모델의 호출을 서버로 넘기며, 결과를 받아 컨텍스트에 관측으로 놓는다. 셋째 이름이 하나 더 있다. 클라이언트 여럿을 거느리는 응용이 호스트다. 서버 하나에 클라이언트 하나가 붙으므로, 서버 열 개를 붙인 응용에는 클라이언트도 열 개다.

규격은 서버를 만드는 일이 아주 쉬워야 한다는 것을 설계 원칙으로 못 박아 두었다. 어려운 조율은 호스트가 맡고 서버는 자기 일 하나에만 집중한다는 뜻이다. 여기서 따라 나오는 제약이 있다. 서버는 대화 전체를 읽지 못하고, 같은 응용에 함께 붙은 다른 서버가 무엇을 하는지도 보지 못한다. 서버가 보는 것은 자기에게 온 호출 하나와 그 인자뿐이다.

무엇을 주고받는가

서버가 내놓는 것은 도구만이 아니다. 세 종류다. 도구와 자원과 프롭프트이며, 셋을 가르는 기준은 기능이 아니라 누가 고르는가다. 도구는 모델이 골라 부르고, 자원은 응용이 골라 신고, 프롭프트는 사용자가 골라 쓴다. 표 8.1이 그 대응이다.

이 목록을 처음 보는 목록으로 읽지 않는 것이 중요하다. 프로토콜이 능력의 발명이었다면 세 종류는 새로운 종류였을 것이다. 길의 표준화이므로, 셋은 컨텍스트를 층으로 갈라 놓은 표 4.1의 층들을 그대로 다시 만난다. 층마다 세워 둔 적재의 셈이 공급 경로가 바뀌어도 그대로 적용된다는 뜻이다.

오가는 방향도 한쪽이 아니다. 규격은 서버가 클라이언트를 거쳐 사용자에게 되묻는 길을 열어 둔다. 빠진 값을 채워 달라거나, 이 작용을 실행해도 되는지 확인해 달라라는 요청이다. 되돌릴 수 없는 작용 앞에 사람을 세우는 결정, 곧 제21장이 받을 결정이 여기서 공급자 쪽에서도 올라온다. 무엇을 보이고 무엇을 물을지는 응용의 결정으로 남는다.

표 8.1: 서버가 내놓는 세 종류. 셋을 가르는 것은 기능이 아니라 누가 고르는가다.

내놓는 것	무엇인가	누가 고르는가	어느 층에 실리는가
도구	불러서 실행하는 능력. 정의가 먼저 실리고 결과가 나중에 실린다	모델	도구 정의 층
자원	읽어서 컨텍스트에 넣는 내용. 주소 하나로 가리킨다	응용	검색된 지식 층
프롬프트	값을 채워 쓰는 지시 묶음	사용자	지시 고정부

연결이 어떻게 맺어지는가

규격이 정한 것 가운데 설계를 바꾸는 것이 셋이다.

첫째, 도구의 목록은 미리 알 수 없다. 클라이언트가 서버에 물어야 온다. 설계 시점의 시스템에는 도구 목록이 적혀 있지 않다. 연결이 열려야 목록이 오고, 목록이 와야 고정부가 조립된다. 게다가 목록은 바뀔 수 있다. 바뀌었다는 알림을 서버가 보낼 수는 있으나 그 전달이 보장되지는 않으므로 클라이언트가 다시 물어야 하고, 그래서 목록의 응답에는 얼마나 오래 재사용해도 되는지의 값이 함께 붙는다.

둘째, 양쪽은 연결을 붙들고 앉아 있지 않는다. 2026년 7월 개정판이 이 방향으로 크게 옮겨 갔다. 요청 하나하나가 자기 버전과 자기가 지원하는 것을 실어 보내므로, 서버 쪽에서 보면 평범한 웹 요청을 받는 일과 같아진다. 여러 대에 나누어 세워도 되고 앞에 부하 분산기를 두어도 된다. 서버를 내놓는 쪽에는 이것이 반가운 변화이고, 대신 여러 호출에 걸친 상태를 서버가 알아서 간수해야 한다는 짐이 따라온다.

셋째, 인증은 규격이 정하고 권한은 정하지 않는다. 네트워크로 붙는 서버는 자기 앞으로 발급된 토큰만 받는다. 남에게 발급된 토큰을 받아뒤로 그대로 흘려보내는 일은 규격이 하지 말라고 못 박은 몇 안 되는 것 가운데 하나다. 내 기계에서 프로세스로 도는 서버는 이 흐름을 타지 않고 환경에서 자격을 얻으며, 그래서 내 기계의 권한을 그대로 물려받는다.

규격이 정하지 않는 것도 분명하다. 무엇을 내놓을지, 내놓은 것 가운데 무엇을 실을지, 실은 것을 믿어도 되는지는 정하지 않는다. 말하지 않는 것이 아니라 말할 수 없다. 그것은 자원과 과제의 사정이 길의 사정이 아니기 때문이다.

규격도 개정된다

프로토콜은 2024년 11월에 열린 표준으로 공개되었고, 협업하는 오픈소스 프로젝트로 개발한다는 방침이 함께였다(Anthropic, 2024c). 2025년 12월에는 한 회사의 소유물이 아니라 공개 표준으로서 리눅스 재단 아래 새로 세워진 재단으로 이관되었으며, 2026년 7월에 개정판이 나왔다. 개정 주기가 한 회사의 제품 주기가 아니라 공개 표준의 주기를 따르게 되었다는 뜻이고, 바로 그렇기 때문에 계속 개정된다는 뜻이기도 하다.

규격은 폐기 예고 규칙을 함께 두었다. 없어질 기능은 예고된 뒤에도 적어도 열두 달은 규격에 남는다. 옮길 시간을 그만큼 준다는 약속이다. 설계자가 여기서 할 일은 자기 결정을 두 부류로 가르는 것이다. 메시지의 형식과 전송의 방식에 결합한 결정은 개정과 함께 낡는다. 공급 경로의 존재, 내놓는 것의 세 종류, 적재의 셈에 세운 결정은 개정을 건넌다.

8.3 MCP가 미치는 영향

영향은 두 자리에 미친다. 컨텍스트 안쪽과 루프 그 자체다. 앞의 것은 매 스텝 나가는 값의 문제이고, 뒤의 것은 시간과 재현의 문제다.

컨텍스트: 쓰지 않은 도구도 값을 묻다

외부에서 온 도구도 정의가 컨텍스트에 실린다. 이름과 설명문과 인자 구조가 텍스트가 되어 앞자리에 앉고, 실행이 도는 내내 매 스텝 다시 읽힌다. 스무 개짜리 서버를 붙였는데 실제로 쓰는 것이 둘이면, 나머지 열여덟의 설명문도 매 스텝 같은 값을 묻다.

그런데 프로토콜이 이 값을 올린 것은 아니다. 올린 것은 붙이는 개수다. 통합 하나에 며칠이 걸리던 때는 그 며칠이 문턱이었고, 값을 치를 만한 연결만 만들어졌다. 설정 몇 줄이면 되는 지금은 문턱이 사라졌다. 그래서 다섯을 붙이던 시스템이 스물을 붙인다. 그 스물의 정의가 전부 앞자리에 실린다.

공개된 사례 하나가 크기를 짐작하게 한다. 붙어 있는 서버의 도구 정의를 전부 미리 실은 실행이 15만 토큰을 썼고, 정의를 파일처럼 두고 필요할 때만 읽게 바꾼 실행이 2천 토큰을 썼다.

값이 절반이고 나머지 절반은 고르는 일이다. 도구가 하나면 고른다는 사건이 없다. 부르거나 안 부르거나. 백 개면 필요한 하나를 아흔아홉 사이에서 집어야 하고, 그때 모델이 보는 것은 이름과 설명문뿐이다. 후보가 늘수록, 그리고 비슷한 정의가 섞일수록 옳은 도구가 불릴 확률이 떨어진다. 규격은 도구 이름이 서버 하나 안에서만 유일하면 된다고 정하므로, 서버 셋을 붙이면 같은 이름이 셋이 되는 일이 일어난다.

그리고 그 설명문을 우리가 못 고친다. 직접 만들던 시절에는 설명문이 모델을 헛갈리게 하면 설명문을 고쳤다. 공급받은 도구의 텍스트는 남이 쓴 것이다. 우리 예산을 쓰고 우리 모델의 선택을 좌우하는데 고칠 권한이 없다. 제5장이 「고쳐야 할 것은 지시가 아니라 도구다」라고 했는데, 여기서는 그 출구가 막혀 있다.

그러니 연결을 승인하는 물음이 바뀐다. 예전에는 「만들 값이 있는가」를 물었고 아니오면 거기서 끝났다. 지금은 만들 값이 거의 언제나 있으므로 그 물음이 아무것도 거르지 못한다. 물어야 하는 것은 하나다. 이 공급자의 도구 정의가 모든 실행의 앞자리에 실릴 값이 있는가.

루프: 밖의 사정이 내 사정이 된다

호출이 밖으로 나가므로 왕복이 하나 더 생긴다. 모델이 도구명과 인자를 내놓고 하네스가 그것을 실제 함수로 옮기는 왕복은 그대로인데(그림 7.1), 그 함수가 이제 남의 기계에서 돈다. 서버가 느리면 스텝이 느려지고, 서버가 죽으면 루프가 선다. 시간 상한과 재시도를 누가 쥐는가는 하네스의 몫이며 제15장이 받는다.

더 조용한 쪽은 재현이다. 목록이 연결 시점에 온다는 것은 그 목록과 그 안의 정의들이 다음 연결에서는 다룰 수 있다는 뜻이다. 정의는 내 실행의 고정부에 실리는 텍스트인데, 그 텍스트를 고칠 권한과 고칠 시점이 공급자에게 있다. 개정이 내 배포 없이, 내게 통보되지 않은 채, 어제의 실행과 오늘의 실행 사이에 일어날 수 있다. 어제와 오늘이 같은 시스템이라는 보장이 내 배포 이력이 아니라 공급자의 관행에 걸린다.

그래서 성능이 흔들리는 날에 원인을 찾는 방식이 달라져야 한다. 팀은 원인을 자기가 바꾼 것의 목록에서 찾는다. 프롬프트를 고쳤던가, 모델을 올렸던가. 연결된 정의는 그 목록에 없으므로 의심 조차 되지 않고, 공급자의 개정은 모델의 성능 저하나 프롬프트의 문제로 오진된다. 처방은 둘이다. 실행의 기록에 연결의 상태, 곧 무엇이 붙어 있었고 각 정의가 어떤 내용이었는지를 함께 남긴다(제20장). 그리고 재현이 중요한 시스템에서는 검토를 통과한 시점의 정의를 고정해 두고, 그 갱신을 공급자의 사정이 아니라 엔지니어의 명시적 결정으로 만든다.

8.4 MCP 설계

설계는 두 방향이다. 가져다 쓰는 쪽과 내놓는 쪽이다. 물어야 할 것이 서로 다르다.

가져다 쓰는 쪽

서버 하나를 붙일 때 확인할 것은 넷이다.

- 이 서버는 어디서 도는가. 내 기계인가, 남의 인프라인가. 앞의 것은 내 기계의 권한을 그대로 물려받고, 뒤의 것은 지연과 가용성이 남의 사정에 걸린다.
- 무엇이 몇 개나 오는가. 도구의 수를 세었는가. 그 정의가 고정부에서 몇 토큰인지 세었는가.
- 목록이 바뀌면 어떻게 아는가. 알림이 오는가, 내가 물어야 하는가.
- 검토한 시점의 정의를 고정할 수 있는가.

여기에 하나를 덧붙여야 한다. 서버는 도구에 행동을 알리는 표시를 붙여 보낼 수 있다. 이 도구는 읽기만 한다거나, 이 도구는 되돌릴 수 없다는 표시다. 규격은 이 표시를 믿을 수 있는 서버에서 온 것이 아니면 믿지 말라고 적어 두었다. 표시를 붙이는 것도 서버이고, 표시가 사실인지 확인해 주는 장치는 계약 안에 없기 때문이다. 「읽기만 한다」는 표시를 보고 사람의 승인을 건너뛰는 설계는 그 표시를 쓴 사람을 믿는 설계다.

넷 다 연결의 물음이 아니라 적재와 재현의 물음이다. 그리고 붙인 뒤에 할 일이 남는다. 공급받은 정의를 주어진 사실이 아니라 검토하고 골라 싣고 필요하면 감싸서 고쳐 쓸 텍스트로 다루는 것이다. 무엇이 좋은 정의인가와 정의를 골라 싣는 장치는 둘 다 제7장에 있다(절 7.4).

내놓는 쪽: 회사 안

여기가 이 프로토콜이 가장 값을 하는 자리다. 자원 하나에 서버 하나를 세우면, 그 자원의 인증을 통과하고 페이지를 넘기고 오류를 다루는 코드를 조직 전체가 한 벌만 쓴다. 응용이 넷째 다섯째로 늘어도 그 코드는 늘지 않는다. 절 8.1의 조직이 열두 벌 대신 일곱 벌을 갖는 것이 이것이다.

그런데 제7장의 도구 설계가 그대로, 오히려 더 무겁게 걸린다. 이름과 설명을 읽는 사람이 이제 우리 팀이 아니기 때문이다. 사내 데이터 파이프라인을 다루는 팀이 도구를 하나 내놓으면서 도구 명을 「run_ingest」라고 붙였다고 하자. 그 팀 안에서는 무엇을 어디로 넣는 것인지 모두가 안다. 옆 팀의 에이전트는 모른다. 모델은 도구명과 설명문 말고는 볼 것이 없으므로, 그 도구는 붙려야 할 자리에서 붙리지 않거나 엉뚱한 자리에서 붙린다. 우리 팀만 아는 말로 쓴 도구 설명은 내놓는 순간 결함이 된다.

내놓는 쪽이 지켜야 할 것을 넷으로 적을 수 있다.

- 이름이 밖에서도 읽혀야 한다. 사내 약어와 내부 시스템의 별명을 도구명에 쓰지 않는다. 도구 명이 서버 안에서만 유일하면 된다는 것이 규격의 요구이므로, 여러 서버가 함께 붙는 자리를 생각해 우리 서비스를 가리키는 조각을 이름 앞에 붙여 둔다.
- 설명문에 경계를 적는다. 무엇을 돌려주지 않는지, 언제 쓰지 않는 도구인지가 설명문에 없으면 쓰는 쪽이 알아낼 길이 없다(절 7.2).
- 목록의 순서를 흔들지 않는다. 같은 도구 집합이면 같은 순서로 돌려준다. 순서가 흔들리면 쓰는 쪽의 캐시가 깨진다.
- 실패를 두 갈래로 돌려준다. 부르는 쪽이 고쳐서 다시 부를 수 있는 실패, 곧 낱짜 형식이 틀렸거나 값이 범위를 벗어났다는 것은 결과 안에 실패 표시를 달아 돌려준다. 모델이 그것을 읽고 스스로 고친다. 없는 도구를 불렀거나 요청의 형식이 깨졌다는 것은 프로토콜의 오류로 돌려준다. 모델이 고칠 수 있는 것이 아니기 때문이다(표 7.3).

서버 하나에 일 하나라는 것도 규칙의 설계 원칙이다. 우리 팀이 다루는 것이 여럿이면 큰 서버 하나에 전부 담기보다 갈라 세우는 편이 낫다. 쓰는 쪽이 필요한 것만 골라 붙일 수 있고, 붙이지 않은 것의 정의는 남의 고정부에 실리지 않는다.

부르는 이에 따라 목록을 달리 내놓을 수도 있다. 규칙은 도구의 집합이 연결마다 달라지는 것은 막지만, 부르는 이가 어떤 자격을 들고 왔느냐에 따라 달라지는 것은 허용한다. 자격이 읽기까지인 상대방에게는 읽는 도구만 보인다. 목록에 보이지 않는 도구는 볼릴 수 없을 뿐 아니라 남의 예산도 쓰지 않는다.

반환의 구조도 여기서 정해진다. 도구의 정의에는 반환이 적히지 않는 것이 원칙이었는데, 규칙은 출력의 형식을 함께 실을 자리를 따로 둔다. 그 자리에 형식을 적어 두면 쓰는 쪽이 결과를 검사할 수 있고, 적지 않으면 쓰는 쪽은 불러 본 뒤에야 안다. 무엇을 어떻게 적을지는 제9장이 받는다.

내놓는 쪽: 회사 밖

회사 밖으로 나가면 위의 넷 위에 보안이 없힌다. 물음은 셋이다. 누가 부르는가, 무엇까지 할 수 있는가, 무엇을 돌려주는가.

누가 부르는가. 네트워크로 여는 서버는 자기 앞으로 발급된 토큰만 받는다. 남에게 발급된 토큰을 받아 주지도 않고, 받은 토큰을 뒤에 있는 다른 시스템으로 그대로 흘려보내지도 않는다. 후자를 하면 뒤에 있는 시스템의 기록에는 우리가 아니라 엉뚱한 신원이 남고, 사고가 났을 때 누가 무엇을 했는지 가릴 수 없게 된다. 규칙이 이것을 금지로 적어 둔 이유다.

무엇까지 할 수 있는가. 권한은 작은 것부터 준다. 처음에는 읽기만 열어 두고, 쓰기가 필요한 자리에 이르렀을 때 그 자리에서 더 달라고 되묻는다. 규칙은 서버가 그 되묻는 응답에 「지금 이 일에 무엇이 더 필요한지」를 적어 보내게 한다. 한 번에 모든 권한을 요구하면 두 가지가 함께 나빠진다. 사용자가 승인 화면에서 물려서고, 승인을 받았다면 그 토큰 하나가 새어 나갔을 때 잃는 것이 전부가 된다. 승인 화면을 떠올리면 왜 그런지 보인다. 사내 문서를 찾는 도구 하나를 붙이려는 사람 앞에 문서 읽기와 쓰기와 삭제와 공유 설정 변경까지 스물댓 줄이 한꺼번에 뜬다. 이 사람에게 필요한 것은 읽기뿐이다. 목록이 길다는 것만으로 승인이 미뤄지고, 미뤄진 승인은 대개 돌아오지 않는다.

무엇을 돌려주는가. 우리가 돌려주는 텍스트는 남의 컨텍스트에 실린다. 넣지 말아야 할 것이 결과에 섞여 있으면 그 자리에서 우리 조직 밖으로 나간다. 반대 방향도 마찬가지여서, 받는 쪽에서 우리가 보낸 텍스트는 검토되지 않은 텍스트다. 어느 쪽에 서든 이 경계를 다루는 일은 제22장의 소재이고, 경계 자체의 정의는 정의 22.1에 있다.

상태를 두는 일에도 걸린다. 양쪽이 연결을 붙들고 있지 않으므로, 여러 호출에 걸친 상태가 필요한 서버는 손잡이를 하나 만들어 돌려주고 다음 호출에서 그것을 인자로 받는다. 장바구니 번호나 작업 번호 같은 것이다. 여기서 어긋나기 쉬운 판단이 하나 있다. 손잡이를 가지고 있다는 것을 본인 확인으로 여기는 것이다. 손잡이는 이름이지 열쇠가 아니다. 호출마다 부르는 이가 그 손잡이의 주인인지를 확인하지 않으면, 번호를 알아낸 누구든 남의 장바구니를 연다. 손잡이는 짐작할 수 없게 만들고, 저장할 때 부르는 이의 신원과 함께 묶어 둔다.

마지막으로 내 기계에서 도는 서버로 내놓는 경우다. 표준 입출력에 붙여 두면 그 서버를 띄운 응용만 닿는다. 네트워크 문으로 열면 같은 기계의 다른 프로세스도 닿으므로, 그때는 토큰을 요구하거나 접근이 제한된 통로를 쓴다.

8.5 프로토콜을 쓰지 말아야 할 때

다음에 해당하면 프로토콜이 아니라 직접 이어 붙이는 것이 답이다.

1. **이을 자리가 몇 곳 없다.** 응용이 하나뿐이고 자원도 하나 둘이면 계약이라는 중간 층의 복잡도만 남는다. 단서가 둘 있다. 이 판정은 지금의 수가 아니라 예상되는 수로 해야 한다. 응용이

표 8.2: 표준화가 남기는 결정. 오른쪽 열이 각 결정을 받는 장이다.

남는 결정	왜 규격이 답하지 못하는가	다루는 곳
무엇을 내놓을 것인가. 도구명, 인자, 도구 설명	정의는 여전히 누군가 써야 하고, 그 품질이 선택과 오용을 정한다. 길은 실어 나를 뿐 내용을 쓰지 않는다	제7장
내용인 것 가운데 무엇을 실을 것인가	계약에는 고르는 장치가 없다. 온 것을 그대로 다 실지 않는 일은 응용의 몫이다	제7장
관측과 출력이 어떤 형식으로 오가는가	계약은 봉투의 형태를 정할 뿐, 안에 담기는 내용의 구조는 도구와 응용 사이의 설계다	제9장
지식을 자원으로 실을 것인가, 검색으로 가져올 것인가	자원은 공급의 한 경로일 뿐, 지식을 어디에 둘지의 교환은 그대로 남는다	제4부, 제11장
공급자를 믿을 것인가	실행의 소유권이 경계 밖에 있고, 들어오는 정의와 내용은 검토되지 않은 텍스트다	제22장

하나에서 둘이 되는 순간 썸이 뒤집힌다. 그리고 자원 쪽 서버를 누군가 이미 만들어 두었다면 내가 만들 것은 클라이언트 하나뿐이다. 내가 만들 것의 수로 세는 것이 옳다.

2. **정의의 통제가 성패를 가른다.** 시스템의 병목이 되는 핵심 도구, 곧 선택과 관측의 품질이 과제 성공률을 직접 좌우하는 자리의 도구는 정의를 직접 쓰고 직접 다듬을 수 있어야 한다. 공급받은 정의는 그렇게 할 수 없고, 감싸는 층은 값과 복잡도를 더한다.
3. **신뢰 경계를 넓힐 수 없다.** 프로토콜은 실행의 소유권을 경계 밖에 두는 구조다. 되돌릴 수 없는 작용을 다루거나 민감한 데이터가 오가는 시스템에서, 검토할 수 없는 실행자와 검토되지 않은 정의를 들이는 것은 연결의 편의로 정당화되지 않는다.
4. **도구가 대화를 알아야 한다.** 서버는 대화 전체를 보지 못한다. 자기에게 온 호출 하나와 그 인자만 본다. 앞선 스텝에서 오간 것을 알아야 제대로 답하는 도구라면 그 내용을 인자에 실어 밖으로 내보내야 하고, 그러면 값이 두 번 든다. 나가는 왕복마다 그만큼의 토큰이 더 실리고, 우리 대화의 일부가 남의 시스템에 남는다. 이런 도구는 응용 안에 두는 편이 낫다.

넷 가운데 어느 것에도 걸리지 않아 붙이기로 했다면 거기서 판정이 끝나는 것이 아니라 절 8.4의 물음으로 넘어간다. 쓸 것인가를 가리는 일과 붙인 다음을 다루는 일은 다른 일이고, 앞의 판정을 통과했다는 것이 뒤의 검토를 면제해 주지 않는다.

그리고 붙이기로 했더라도 표준화가 끝내 주지 않는 결정이 남는다. 표 8.2가 그 목록이고, 행마다 그것을 받는 장이 오른쪽에 적혀 있다.

표에 적기 어려운 항목이 하나 더 있다. 규격의 세부에 닿는 코드가 우리 시스템의 어디에 있는지의 목록이다. 세부와의 결합이 응용마다 클라이언트 하나로 모여 있는 시스템은 규격이 개정될 때 응용당 한 곳을 고친다. 세부가 도구를 다루는 분기와 결과를 파싱하는 구석마다 스며 있는 시스템은 개정판 하나마다 응용 전부를 훑는다. 뒤의 것이 무엇의 재현인지 보라. 자원의 변경 하나가 사본의 수만큼의 수선이 되던 절 8.1의 세계가, 이번에는 규격 자신을 상대로 되살아난 것이다.

그러니 그 목록을 한 번은 만들어 두어야 한다. 찾을 것은 셋이다. 요청을 만들고 응답을 푸는 코드, 도구의 이름을 보고 갈라지는 코드, 그리고 반환이 어떤 형식으로 올 것이라고 가정하고 그 형식에 맞추어 읽는 코드다. 첫째는 클라이언트 한 곳에 모아 둘 수 있다. 뒤의 둘이 응용 코드 곳곳으로 번지기 쉽고, 번져 있으면 개정판이 나올 때마다 그 자리를 전부 다시 찾아야 한다. 개정에 드는 값은 규격이 얼마나 바뀌었는지가 아니라 이 목록이 얼마나 긴지가 정한다.

마지막으로 이 판정은 시스템 전체가 아니라 도구마다 내릴 수 있다. 전부 공급받거나 전부 직접 만들거나의 양자택일이 아니다. 성패를 가르는 소수의 도구는 직접 만들어 정의를 쥐고, 주변의

다수 자원은 공급받아 이를 자리를 줄이는 혼합이 보통의 답이다. 어느 도구가 소수 쪽인지는 이름의 목록이 아니라 두 값, 곧 쓰일 빈도와 선택에 미치는 영향을 평가해 본 사람만 가릴 수 있다.

8.6 요약

한 문장으로. 프로토콜은 능력이 아니라 공급의 길을 표준화해 이어 붙일 자리를 줄인다. 그러나 무엇을 실을지는 정해 주지 않으므로, 연결이 짜질수록 적재의 결정이 무거워진다.

확실해진 것.

- 응용 셋이 자원 넷에 닿으려면 따로 이을 때 코드가 열두 벌 필요하고, 공용 계약을 거치면 일곱 벌이면 된다(그림 8.1). 서버를 남이 이미 만들어 두었다면 내가 만들 것은 더 줄어든다.
- 서버가 내놓는 것은 도구와 자원과 프롬프트 셋이고, 가르치는 것은 누가 고르는가다. 도구는 모델이, 자원은 응용이, 프롬프트는 사용자가 고른다(표 8.1). 셋 다 컨텍스트의 층을 다시 만나므로 층마다의 셈이 그대로 적용된다.
- 프로토콜이 올린 것은 매 스텝의 값이 아니라 붙이는 개수다. 문턱이 사라지자 다섯을 붙이던 시스템이 스물을 붙이고, 그 스물의 정의가 전부 앞자리에 실린다. 공개된 사례 하나에서 미리 다 실은 실행이 15만 토큰, 필요할 때만 읽은 실행이 2천 토큰이었다.
- 후보가 늘수록 옳은 도구가 불릴 확률은 떨어지는데, 그 설명문을 쓴 사람이 밖에 있어 고칠 수도 없다. 그래서 물음이 하나로 좁혀진다. 이 공급자의 도구 정의가 모든 실행의 앞자리에 실릴 값이 있는가.
- 도구의 목록은 물어야 오고 실행 중에도 바뀐다. 정의를 고칠 권한이 공급자에게 있어 개정이 우리 배로 없이 일어나므로, 기록이 없으면 모델의 성능 저하로 오진된다(절 8.3).
- 내놓는 쪽에서는 이름과 설명이 남에게 읽히고, 목록의 순서가 남의 캐시를 정하며, 실패의 갈래가 남의 모델이 스스로 고칠 수 있는지를 정한다(절 8.4). 회사 밖으로 나가면 셋이 더해진다. 자기 앞으로 발급된 토큰만 받고, 권한은 작은 것부터 주고, 돌려주는 내용이 남의 컨텍스트에 실린다.
- 쓰지 말아야 할 자리는 넷이다. 이을 자리가 몇 곳 없을 때, 정의의 통제가 성패를 가를 때, 신뢰 경계를 넓힐 수 없을 때, 도구가 대화를 알아야 할 때다(절 8.5).

아직 답하지 않은 것.

- 정의라는 칸에 무엇을 어떻게 적어야 하는가, 그리고 내놓은 것 가운데 무엇을 실을 것인가는 제7장에 있다.
- 관측과 출력의 구조, 곧 형식을 계약으로 만드는 일은 제9장이 받는다.
- 자원으로 실을지 검색으로 가져올지의 판단은 제4부와 제11장이 받는다.
- 경계 밖의 실행자와 검토되지 않은 정의가 넓히는 신뢰 경계의 방어는 제22장이 받는다.

오늘 할 것. 지금 붙어 있는 서버의 목록을 열어 서버마다 도구가 몇 개인지 센다(절 8.3). 그 가운데 지난 한 달 동안 한 번도 불리지 않은 도구에 표시하고, 그 정의가 매 스텝 무슨 토큰을 적는다(절 8.3). 마지막으로 우리가 내놓은 서버가 있다면 그 도구명과 도구 설명을 그 팀 밖의 사람에게 보이고 무슨 일을 하는 도구인지 묻는다(절 8.4).

8.7 생각해 볼 문제

1. 자기 조직에서 응용과 자원의 쌍을 세어 이어 붙인 자리의 수를 적어 보라. 사내 공유 연결 라이브러리가 이미 있다면 그 수는 처음부터 열두 곳이 아니었을 것이다. 어떤 조건에서 표준 도입이 자기가 유지할 구현의 수를 오히려 늘리는가.

2. 붙어 있는 서버들의 도구 정의가 앞자리에서 차지하는 몫을 측정하고, 그 가운데 지난달에 한 번도 불리지 않은 도구의 몫을 따로 세어 보라. 15만 토큰이 2천 토큰이 된 사례를 자기 시스템에 대 보면 어디까지 줄어드는가. 그 도구들을 목록에서 빼면 남은 도구가 불릴 확률에는 무엇이 달라지는가.
3. 성능이 어제와 달라졌다. 바뀔 수 있었던 것은 모델, 프롬프트, 그리고 공급자의 정의다. 셋 중 어디가 바뀌었는지를 가리는 데 필요한 기록을 나열하라. 그 기록이 없는 시스템에서 이 물음이 어떤 오진으로 흘러가기도 적어 보라.
4. 우리 팀의 기능 하나를 서버로 내놓는다고 하자. 도구명과 도구 설명을 우리 팀 밖의 사람이 읽는다는 조건에서 다시 써 보라. 그 서버를 회사 밖으로 열 때 추가로 정해야 할 것을 절 8.4의 세 물음에 따라 적어 보라.

8.8 다시 읽을 때

- 제7장을 읽고 「설명문을 쓴 사람이 밖에 있다」로 돌아오면 그 문장의 무게가 달라져 있다. 좋은 도구명과 좋은 도구 설명의 기준을 알고 나면 공급받은 정의를 평가할 자가 생기고, 감싸서 고쳐 쓸 때 무엇을 고쳐야 하는지가 보인다.
- 제7장의 점진적 공개를 읽고 15만 토큰과 2천 토큰의 차이로 돌아오면, 그 차이가 관찰이 아니라 설계 대상이었다는 것이 보인다. 연결은 유지하되 정의를 골라 싣는 장치, 그리고 정의를 아예 신지 않는 우회까지가 거기서 채워진다.
- 제20장을 읽고 연결 상태의 기록으로 돌아오면, 실행 단위의 기록이라는 요구가 트레이스라는 설계 대상이 되어 있다. 무엇이 붙어 있었고 어떤 정의가 보였는지가 스텝의 기록에 남아야 공급자의 개정이 오진이 아니라 관측 가능한 사건이 된다.
- 제22장을 읽고 절 8.4의 세 물음으로 돌아오면, 내놓는 쪽의 규율이 상대방에서 어떻게 보이는지가 잡힌다. 공급자가 내놓은 정의와 자원의 내용은 받는 쪽에서 실행 시점에 들어오는 검토되지 않은 텍스트다.

8.9 더 읽을거리

프로토콜의 공개 문서인 Anthropic (2024c)는 짧고, 진단과 처방이 거기 다 있다. 고립된 모델과 사일로에 갇힌 정보와 원천마다의 고유한 구현이라는 진단, 파편화된 통합을 하나의 표준으로 대체한다는 처방, 그리고 도구와 자원과 프롬프트라는 세 종류다. 읽을 때는 어느 문장이 연결의 문제이고 어느 문장이 적재의 문제인지 가르며 읽기를 권한다. 문서가 앞의 것만을 말하고 있다는 사실이 곧 이 장의 논지다.

이 장은 규격의 세부를 다루지 않았다. 메시지를 어떤 형식으로 주고받는지, 인증과 권한을 어떻게 거는지, 전송을 무엇으로 하는지는 전부 규격이 정해 둔 것이고 여기서는 생략했다. 서버를 직접 만들 계획이라면 그 세부가 필요하다. 규격 문서에 전부 있으므로 거기서 확인하기를 권한다. 개정 이력과 앞으로의 계획도 함께 공개되어 있다.

다만 세부는 개정된다. 남겨야 할 것은 개정을 건너는 구조이고, 어떤 세부를 만나든 물을 것은 같다. 이것은 공용 계약이 정한 것인가, 아니면 여전히 내 설계의 몫인가.

제9장

구조화된 출력과 스키마 설계

모델은 같은 것을 두 번 물으면 두 번 다르게 답한다. 프롬프트를 아무리 다듬어도 그 차이는 사라지지 않는다. 그런데 에이전트에서 그 출력을 받는 것은 사람이 아니라 코드다. 한 스텝의 출력이 다음 스텝의 입력이 되므로, 모양이 흔들리면 그 흔들림이 루프를 타고 번진다.

여기서 다루는 것은 그 흔들림을 형식에서 없애는 일이다. 방법은 하나다. 받는 쪽이 기대하는 형식을 요청에 함께 실어, 그 형식을 벗어나는 출력이 아예 뿔치지 않게 만든다. 앞의 장들이 컨텍스트에 무엇을 넣을 것인가를 다루었다면 이것은 그 일의 반대쪽이다. 넣는 쪽을 아무리 다듬어도 나오는 쪽이 매번 다른 모양이면, 그 출력을 다시 컨텍스트에 넣는 다음 스텝에서 같은 다듬기를 처음부터 해야 한다.

강제되지 않는 것이 하나 있다. 값이 옳다는 것이다. 이것은 해석이 아니라 기능 문서가 스스로 좁게 적어 둔 범위다. 보장되는 것은 형식의 준수와 자리의 타입과 필수 자리의 존재까지이고, 값이 옳다는 보장은 그 목록에 없다(Anthropic, 2026h). 그래서 스키마 설계의 문제는 무엇을 강제할 것인가가 아니라 강제로 잡히지 않는 것이 무엇인지를 아는 것이다.

내용의 옳음을 평가하는 설계는 제6부가, 형식이 맞는 입력을 믿어도 되는가라는 방어의 물음은 제22장이 받는다.

9.1 왜 형식을 정하는가

아침마다 시스템을 훑고 이상이 있는지 보고하는 도구가 있다고 하자. 같은 요청을 두 번 보냈더니 이렇게 돌아왔다.

「점검을 마쳤다. 세 항목에서 이상이 보였고, 그중 둘은 어제와 같은 항목이다.」

「점검 결과 문제가 없지는 않았다. 확인이 필요한 곳이 3개 있다.」

사람이 읽으면 같은 보고다. 코드가 읽으면 다른 보고다.

받는 코드가 하는 일은 셋이다. 이상이 있었는지로 갈라지고, 건수를 문턱과 견주고, 항목을 다음 처리로 넘긴다. 값은 두 문장에 다 있다. 그런데 꺼낼 수가 없다. 「이상이 보였다」를 찾는 검사는 「문제가 없지는 않았다」 앞에서 무력하고, 「세 항목」에서 수를 꺼내던 규칙은 「3개」가 오는 날 헛돈다. 같은 뜻을 말하는 표현이 무한하다는 것이 사람에게서는 문장의 힘인데 코드에게는 곤란의 전부다.

다음 스텝의 모델이 받아도 곤란은 남는다. 앞 스텝의 결과에 있던 값을 다음 호출의 인자로 써야 한다면, 모델이 그 값을 다시 적어야 한다. 그런데 다시 적는 일은 베끼는 일이 아니다. 모델은 매번 새로 뽑으므로 원본과 다를 수 있다. 그리고 그 값이 문장 속에 섞여 있으면 찾아내는 일까지 더해진다. 「주문 번호는 A-1183입니다」에서 A-1183만 골라내야 하고, 골라낸 것을 틀리지 않게 옮겨야 한다.

프롬프트에 적어서는 풀리지 않는다. 「이 형식으로 답하라」는 잘 지켜진다. 다만 언제나는 아니다. 대체로 지켜지는 형식은 받는 코드의 눈으로 보면 가끔 무너지는 형식이고, 파서는 가장 드문 표현에서 무너진다.

그리고 흔들림은 사고가 아니라 정상 저동이다. 도구를 부르기 전에 무엇을 하려는지 모델이 한 마디 덧붙이는 일이 흔한데, 그 한마디의 표현이 매번 다르다. 공개된 지침도 그것을 특정 표현으로 가정하지 말라고 적는다. 사람에게 친절한 그 문장이 코드에게는 매번 다른 입력이다.

형식을 안 정해도 형식은 이미 있다. 받는 쪽의 코드에 파서의 모양으로 있다. 문장에서 건수를 꺼내는 규칙 하나하나가 「모델은 이렇게 말한다」는 가정이고, 그 가정 전부가 이 시스템의 실질적인 출력 규약이다. 그런데 그것이 어디에도 적혀 있지 않다. 받는 쪽마다 제각각 흩어져 있고, 내놓는 쪽은 자기가 거기 묶여 있다는 것조차 모른다. 그래서 모델을 바꾸거나 프롬프트를 다듬어 표현이 조금 달라지는 날, 어긋남은 받는 쪽의 오동작으로만 나타난다.

형식을 정한다는 것은 이미 있는 그 약속을 꺼내어 한 곳에 적는 일이다.

9.2 스키마가 하는 일

형식을 정하는 자리는 둘이고 힘이 다르다.

프롬프트에 적으면 「이렇게 답해 달라」는 부탁이다. 대체로 지켜지고 가끔 안 지켜진다. 스키마로 적으면 다르다. 모델이 낸 것을 받는 쪽에 넘기기 전에 코드가 대조하고, 안 맞으면 거기서 막힌다.

스키마는 이렇게 생겼다.

```
status: "ok" 또는 "issues" 또는 "unknown" 중 하나. 필수
count: 0 이상의 정수. 필수
items: 문자열의 목록. 비어 있어도 된다
```

그러면 앞 절의 두 보고가 이렇게 돌아온다.

```
{"status": "issues", "count": 3, "items": [...]}
```

받는 코드가 하는 일이 셋에서 하나로 준다. 값을 꺼내면 된다. 문장에서 찾아내는 일도, 표현이 바뀔까 걱정하는 일도 없다.

세 자리를 정한다. 필드의 이름과 타입, 그리고 필수인지 아닌지다. 이름은 받는 쪽이 부를 이름이고, 타입은 코드가 대조할 것이며, 필수 여부는 없어도 되는지를 정한다.

여기에 하나가 더 붙는다. 각 필드가 무엇인지 적는 설명이다. 모델은 필드 이름과 그 설명만 보고 값을 채운다. 제7장이 도구 설명문에 대해 말한 것이 그대로 걸린다. count 라고만 적으면 무엇의 개수인지 모르고, 「이상이 발견된 항목의 개수」라고 적으면 안다.

정의 9.1 (출력 스키마). 모델이 낸 것을 받는 쪽에 넘기기 전에 만족해야 하는 형식의 명세로서, 어졌는지를 사람이나 모델의 판단 없이 코드가 판정할 수 있는 것을 출력 스키마라 한다. 형식이 이렇게 강제된 출력을 구조화된 출력이라 한다.

가르는 기준은 무엇으로 적혔느냐가 아니라 누가 판정하느냐다. 자연어로 적힌 규약도 형식의 명세다. 다만 지켜졌는지를 사람이 보거나 모델이 판단하면, 그 확인 자체가 다시 흔들린다. 스키마의 힘은 대조하는 것이 코드라는 데서 온다.

어디에 적는가

적는 자리가 프롬프트가 아니라는 것이 첫째로 중요하다. 스키마는 요청에 딸린 인자다. 요청을 보낼 때 형식을 적는 칸이 따로 있고, 거기에 자리의 목록과 각 자리의 타입을 적어 넣는다(Anthropic, 2026h). 프롬프트에 적힌 「이렇게 답하라」와 이것은 같은 문장이 아니다. 앞의 것은 지시이고 뒤의 것은 요청의 인자다.

실물이 서는 자리는 둘이고 겹쳐 쓸 수 있다. 모델이 내놓는 응답 전체의 형식을 정하는 자리와, 도구 호출의 인자를 스키마와 엄격히 대조하는 자리다. 뒤의 것은 제7장의 인자 설계가 실행 시점의 장치를 얻은 것이다. 앞의 것이 여기의 소재다.

강제하는 방식도 둘이다. 생성한 뒤에 대조해 거부하는 길과, 생성 자체를 형식에 묶는 길이다. 뒤엎것은 스키마를 벗어나는 토큰을 아예 뽑지 못하게 만든다. 그 아래에서 형식의 위반은 거부되는 것이 아니라 일어나지 않는다. 어느 쪽을 쓰든 강제되는 것이 형식까지라는 사실은 같다. 다만 어졌을 때 무엇을 할 것인가(절 9.4)는 어느 쪽을 쓰느냐가 정한다.

스키마 하나, 그리고 그것으로 나온 출력

앞 절의 점검 도구에 형식을 정하면 이렇게 된다.

```
{
  "type": "object",
  "additionalProperties": false,
  "required": ["status", "count"],
  "properties": {
    "status": {
      "type": "string",
      "enum": ["ok", "issues", "unknown"],
      "description": "점검을 마쳤고 이상이 없으면 ok, 이상이 있으면 issues.
        점검 자체를 마치지 못했거나 이상 여부를 판단할 수 없으면 unknown."
    },
    "count": {
      "type": "integer",
      "description": "이상 항목의 수. status가 unknown이면 0을 적는다."
    },
    "items": {
      "type": "array",
      "items": {"type": "string"},
      "description": "이상 항목의 이름. 아는 만큼만 적는다."
    },
    "note": {
      "type": "string",
      "description": "위 자리에 담기지 않은 단서. 확신이 서지 않거나 결과가
        부분적이면 여기에 적는다."
    }
  }
}
```

이것을 지킨 출력은 이렇게 온다.

```
{
  "status": "issues",
  "count": 3,
  "items": ["인증서 만료 임박", "디스크 여유 부족", "야간 배치 지연"],
  "note": "야간 배치는 어제와 같은 이유로 걸렸다."
}
```

앞 절의 두 문장과 견주면 무엇이 달라졌는지가 보인다. 갈라질 값이 셋 중 하나이므로 조건문이 흔들리지 않는다. 건수가 수이므로 문턱과 바로 견줄 수 있다. 목록이 배열이므로 꺼내는 데 규칙이 필요 없다. 어제와의 비교라는 단서는 사라지지 않고 note로 갔다.

같은 내용을 형식만 어기면 이렇게 온다.

```
{
  "status": "이상 있음",
  "count": "세 개",
  "findings": ["인증서 만료 임박", "디스크 여유 부족", "야간 배치 지연"]
}
```

세 가지를 한꺼번에 어겼다. status의 값이 셋에 없고, count가 수가 아니라 문자열이고, 목록이 스키마에 없는 이름으로 왔다. 사람이 읽으면 셋 다 같은 보고인데 대조는 셋 다 거부한다. 대조가 형식 말고는 아무것도 보지 않기 때문이며, 그 성질이 다음 절부터 양쪽으로 작용한다.

자리 설명이 그 자리를 채운다

모델은 자리의 이름과 설명만 보고 그 자리를 채운다. 위 스키마에서 실제로 일을 하는 부분이 description이라는 뜻이다. status라는 이름과 세 값만 있으면 언제 「unknown」을 골라야

하는지를 모델이 알 길이 없다. 설명이 그것을 정한다.

제7장이 도구 설명문에 대해 세운 기준이 그대로 온다. 무엇을 하는지, 언제 쓰고 언제 쓰지 않는지, 각 인자가 무슨 뜻인지, 무엇을 돌려주지 않는지까지 적으라는 것이었다(Anthropic, 2025g). 출력 스키마의 자리 설명이 답해야 하는 것은 이렇다. 이 자리에 무엇을 적는가, 어떤 경우에 비워 두는가, 다른 자리와 어긋날 때 무엇을 우선하는가. 위 스키마의 count 설명이 마지막 물음에 답하고 있다. status가「unknown」인데 건수가 셋이라고 적히면 받는 쪽의 갈라짐이 어긋나기 때문이다.

자리가 여럿 겹쳐 있거나 값의 모양이 까다로우면 설명만으로는 모자란다. 그때는 올바른 입력의 예를 스키마와 함께 실을 수 있다. 예는 스키마에 유효해야 하고, 실린 만큼 입력 토큰을 문다. 설명을 늘릴 것인가 예를 실을 것인가는 그 두 값을 견주어 정한다.

9.3 얼마나 조일 것인가

조일수록 좋은 것이 아니다. 양 끝을 짚으면 보인다.

아무것도 정하지 않으면 앞 절의 두 문장이 돌아온다. 표현은 자유롭고 받는 쪽은 매번 고생한다. 반대로 모든 자리를 필수로 하고 모든 값을 정해진 것 중에서만 고르게 하면 받는 쪽은 편해진다. 그런데 세계가 그 목록에 맞춰 주지 않는다. 설계자가 내다보지 못한 일이 생기는 순간 모델 앞에는 맞는 답이 없고, 그중 가장 덜 틀린 것을 고르게 된다.

무엇이 오가는지는 분명하다. 조일수록 받는 쪽이 쉬워지고, 조일수록 답을 수 있는 것이 줄어든다. 그리고 줄어드는 자리에서 사라지는 것은 대개 값이 아니라 단서다. 확신이 서지 않는다는 것, 결과가 부분적이라는 것, 두 후보가 같았는데 하나를 골랐다는 것.

갈 곳 없는 단서는 셋 중 하나가 된다. 버려지거나, 아무 자리에나 구겨 놓여지거나, 형식을 어겨 거부된다. 셋째는 시끄러우니 차라리 낫다. 앞의 둘이 조용하다. 확신 없음을 적을 자리가 없는 모델은 확신 있는 모양으로 답하고, 받는 쪽은 그것을 온전한 값과 구별할 수 없다.

가르는 기준 셋

값이 정해져 있으면 정해진 것 중에 고르게 한다. 자유 문자열은 모델이 매번 새로 적으므로 흔들린다. 앞 절의 status가 그렇다. 열어 두면「이상 있음」이라고 적을 수도 있고「issues found」라고 적을 수도 있다. 셋으로 닫으면 그 밖의 값이 받는 쪽에 닿지 못한다.

모르는 경우를 답을 자리가 있어야 한다. 닫힌 값 셋 가운데 하나가「unknown」인 것이 그 자리다. 이것이 없으면 점검을 마치지 못한 아침에 고를 수 있는 것이「ok」와「issues」뿐이고 둘 다 거짓이다. 강제는 모델을 정확하게 만들지 않는다. 주어진 것 중 하나를 고르게 만들 뿐이다.

모델이 채울 수 없는 자리는 자리가 아니다. 필수로 표시한 자리를 하나씩 짚으며 이 값을 모델이 그 시점에 알 수 있는지 물어야 한다. 알 수 없으면 길은 셋이다. 선택 자리로 열거나, 모른다는 값을 넣거나, 그 값을 만들어 주는 도구를 목록에 넣거나. 알 수 없는 자리를 필수로 두는 것은 지어내라는 주문이다.

셋을 관통하는 한 줄이 있다. 조일지 말지는 받는 쪽이 정한다. 받는 쪽이 그 값으로 갈라지면 조인다. 받는 쪽이 읽고 판단하면 조여서 얻는 것이 없다. 그리고 모델만 아는 것, 곧 확신과 단서와 예외는 조이는 것이 아니라 답을 자리를 만들어 주어야 한다.

적었다고 다 지켜지지는 않는다

스키마에 적을 수 있는 것과 실제로 강제되는 것이 다르다. 타입과 필수 여부와 정해진 값의 목록은 강제된다. 수의 범위나 문자열의 길이는 대개 그렇지 않다.

강제되지 않는 조항은 사라지지 않고 설명문으로 내려간다. 그리고 설명문에 적힌 것은 다시 부탁이 된다.「100 이상이어야 한다」가 설명으로 내려가면 대체로 지켜지고 가끔 안 지켜진다. 범위를

지킨 값이 정말 필요하면 대조 뒤에 검사를 한 번 더 두어야 하고, 그 검사는 스키마가 아니라 받는 쪽 코드의 몫이다.

무엇이 강제되고 무엇이 안 되는지는 공급자마다 다르고 바뀐다. 쓰기 전에 문서에서 확인할 일이다.

조이면 답이 나빠지는가

형식을 강제하면 모델이 답을 적기 전에 생각을 펼칠 자리가 좁아진다. 공개된 지침이 형식을 고를 때 두 가지를 보라고 한다. 모델이 글에서 자연스럽게 보아 온 형태에 가까울 것, 그리고 형식을 지키는 일 자체가 부담이 되지 않을 것. 답을 적기 전에 생각할 토권을 남겨 두라는 항목이 그 옆에 있다(Anthropic, 2024a). 셋이 같은 곳을 가리킨다. 형식은 답을 담는 그릇이지 답을 만드는 과정이 아니다.

얼마나 나빠지는지는 아직 답이 없다. Tam et al. (2024)가 형식을 제한한 답과 자유로운 답을 여러 과제에서 견주어 제한 아래에서 추론이 떨어진다고 관찰했는데, 실험 설계를 두고 반론이 있다. 다투어지는 결과는 가져다 쓸 수 없다. 자기 과제에서 두 조건을 견주어 본 팀만 자기 답을 갖는다.

스키마도 값을 묻다

형식을 설명하는 텍스트가 입력에 함께 실린다. 스키마는 모델이 못 보는 검사 장치가 아니라 앞자리에 실리는 텍스트이기도 하다.

그리고 스키마를 고치면 캐시가 깨진다. 구조를 바꾸는 것과 도구 목록을 바꾸는 것이 그렇고, 이름과 설명만 고치는 것은 깨지 않는다(Anthropic, 2026h). 스키마 개정은 문구 수정이 아니라 캐시 경계의 변경이다.

고치는 일이 한쪽의 결정이 아니라는 것도 여기 걸린다. 받는 쪽이 새 자리를 기대하는데 내놓는 쪽이 그대로면 그 자리는 영원히 빈다. 내놓는 쪽이 자리의 뜻을 바꾸었는데 받는 쪽이 옛 뜻으로 읽으면 형식은 맞는 채로 값만 어긋난다. 뒤의 것이 더 나쁘다. 대조는 형식만 보므로 거부가 일어나지 않고, 양쪽 다 지키고 있다고 믿는 채로 시스템만 틀린다.

스키마는 언제 낡는가

자리의 목록은 그것을 쓴 날 그 팀이 알던 과제의 모양이다. 과제는 변하고, 변한 만큼 자리가 모자라게 된다. 그런데 이 어긋남은 여기는 일로 나타나지 않는다. 형식은 여전히 맞기 때문이다.

나타나는 곳은 열린 자리다. 갈 곳 없는 단서가 문자열 자리에 구겨져 들어오기 시작하면 그것은 모델의 일탈이 아니라 스키마가 낡았다는 신호다. 다음 판에 어떤 자리가 생겨야 하는지의 목록이 거기 쌓이고 있다.

9.4 모델이 형식을 어겼을 때

여기는 일이 일어나는지부터가 어느 방식을 쓰느냐에 달렸다. 생성 자체를 형식에 묶으면 여기는 일은 일어나지 않는다. 낸 것을 뒤에 대조하는 설계라면 여기는 일은 사고가 아니라 정상 거동이다. 모델이 내놓는 한 여기는 출력이 나올 확률은 0이 아니다.

언제 여기는가

여기는 자리는 대개 정해져 있다.

답을 수 없는 것을 담으라고 할 때 어긴다. 앞 절의「unknown」이 없는 스키마가 그렇다.

표 9.1: 어겼을 때의 세 길. 고르는 것이 아니라 순서로 잇는다.

길	하네스가 하는 일	얻는 것	값을 치르는 자리
고치다	어긴 출력을 가장 가까운 유효한 형식으로 고쳐 통과시킨다	스텝도 지연도 늘지 않는다	고치는 규칙은 뜻을 모른다. 형식에 가까운 것이 뜻에서 멀 수 있고, 어겼다는 사실이 기록에서 사라진다
다시 시킨다	거부를 관측으로 돌려주고 다시 내게 한다	모델이 무엇에 걸렸는지 알고 스스로 고칠 기회	왕복이 스텝을 쓰고 이력을 늘린다. 성공이 보장이 아니므로 상한이 필요하다
위로 올린다	실행을 멈추거나 사람에게 넘긴다	틀린 형식이 하류에 닿지 않는다는 보장	과제가 그 자리에서 멈춘다. 넘겨받는 쪽이 없으면 중단은 처리가 아니라 방치다

스키마가 길고 복잡할 때 어긴다. 채울 자리가 늘면 자리마다 틀릴 기회가 하나씩 생기고, 자리가 겹겹이 쌓여 있으면 그 구조를 유지하는 일 자체가 부담이 된다.

모르는 것을 모른다고 적을 자리가 없을 때 어긴다.

셋 다 스키마 쪽의 사정이지 모델 쪽의 무능이 아니다.

한 가지가 더 있는데 이쪽이 더 나쁘다. 어기지 않고 지어내는 경우다. 공개된 지침이 모델을 고르는 기준으로 그것을 든다. 복잡한 도구와 모호한 질의에는 가장 큰 모델을 쓰라고 하면서, 작은 모델은 빠진 인자를 스스로 추론해 채울 수 있다고 적어 두었다(Anthropic, 2026j). 빠진 자리를 비워 두고 되묻는 대신 그럴듯한 값을 넣는다는 뜻이다. 그런 출력은 형식을 지키므로 대조에 안 걸린다. 어기는 비율이 낮다는 것이 언제나 좋은 신호는 아닌 이유가 여기 있다.

어떻게 처리하는가

길은 셋뿐이다. 하네스가 몰래 고치거나, 다시 시키거나, 위로 올린다.

다시 시키는 것이 기본값이 되기 쉬우므로 그 값부터 세어 둔다. 어긴 출력과 거부문은 교정에 쓰인 뒤에도 이력에서 사라지지 않아 남은 스텝 동안 계속 실린다. 출력이 두 번 생성되므로 캐시가 못 돕는 쪽의 뭇도 두 배로 나간다. 다시 시키는 것은 공짜 재추침이 아니라 이력을 늘리는 왕복이다.

무엇을 알려 주는가가 회복률을 정한다. 「형식이 틀렸다」만 돌려주면 모델은 어느 자리를 어떻게 고칠지 모른 채 다시 뿜는다. 어느 자리가 무엇에 걸렸는지를 자리 단위로 돌려주면 고칠 수 있다. 앞 절의 예에서는 status의 값이 셋 중에 없다는 것, count가 수여야 한다는 것, findings라는 자리가 스키마에 없다는 것이 그 내용이다.

몇 번까지 시도할 것인가에는 공개된 답이 없다. 필요한 것은 두 수다. 자리별로 어기는 비율, 그리고 다시 시켰을 때 고쳐지는 비율. 두 번째 시도에서 급감하는 시스템과 세 번째까지 완만한 시스템은 상한이 다르다. 상한에 닿은 뒤의 길은 고치거나 위로 올리는 것이다. 세 길은 나란한 선택지가 아니라 순서다.

답은 괄호를 채워 주고 꼬리의 군더더기를 잘라 주는 관대한 하네스는 왕복을 아끼는 장치로 보인다. 값은 두 군데서 나간다. 첫째, 고치는 일은 형식의 거리로 하는 것이지 뜻의 거리로 하는 것이 아니다. 잘린 출력의 괄호를 채우면 유효한 형식이 되지만 담긴 것은 완성된 값이 아니라 잘린 값이다. 그리고 그것이 모델의 이름으로 소비된다. 둘째, 어겼다는 사실 자체가 사라진다.

표 9.2: 형식과 뜻의 네 조합. 스키마가 움직이는 것은 형식의 축뿐이다.

출력의 처지	판정	받는 쪽에서 일어나는 일	드러나는 방식
형식도 뜻도 맞다	통과	다음 스텝이 옳은 값 위에서 진행한다	드러날 것이 없다
형식은 맞고 뜻이 틀리다	통과	틀린 값이 검사를 통과했다는 표식을 달고 쓰이고, 이후 스텝의 전제가 된다	뒤의 결과가 어긋난 뒤에야, 그것도 결과가 흔적을 남길 때만
형식이 어긋나고 뜻은 맞다	거부	옳은 내용이 쓰이지 못하고 되돌아온다	즉시. 거부 자체가 관측이다
형식도 뜻도 어긋난다	거부	틀린 값이 쓰이기 전에 걸려진다	즉시. 다만 걸린 이유는 뜻이 아니라 형식이다

어긋난 것을 세어 둔다

여기는 비율은 비용이 아니라 진단이다. 자주 여기는 자리는 모델이 무능한 자리가 아니라 스키마와 모델의 이해가 어긋난 자리다. 이름이 만든 기대와 설명이 어긋났거나, 자리가 모자라 단서가 형식을 비집고 나온다. 요구된 값이 그 시점의 컨텍스트에 없을 수도 있다. 각각의 처방은 이름을 고치는 것, 자리를 만드는 것, 그 값을 주는 도구를 넣는 것이지, 재시도 상한을 올리는 것이 아니다. 여기는 비율이 높으면 나쁜 것은 모델이 아니라 스키마다.

그리고 몰래 고치는 설계는 정확히 이 수를 지운다. 고치더라도 무엇이 얼마나 자주 어떻게 고쳐졌는지는 남아야 한다. 그 기록이 다음 스키마 개정의 근거 목록이다. 어디에 어떤 형식으로 남길 것인가는 제20장의 소재다.

9.5 스키마로 덮을 수 없는 것

출력 하나에는 판정이 둘 있다. 형식이 스키마에 맞는가, 그리고 내용이 옳은가.

앞의 것은 코드가 판정한다. 뒤의 것은 실행의 결과나 사람이 판정한다. 스키마가 움직이는 것은 앞의 것뿐이다.

읽어야 할 것은 둘째 행이다. 앞 절의 점검 출력에서 status가「issues」이고 count가 3인데 그 셋이 실제로는 이상이 아니었다면, 대조는 통과시킨다. 옮겨 적다 바뀐 숫자도, 없는 것을 있다고 적은 보고도 형식만 갖추면 같은 문으로 지나간다.

핵심. 스키마는 형식이 틀린 출력을 쓰이기 전에 막는다. 형식이 맞는 틀린 답은 못 막는다. 위험한 것은 남은 실패가 아니라, 검사를 통과했다는 사실이 내용의 보증으로 읽히는 것이다.

검사가 없던 때 출력은 그저 출력이었고, 믿을지 말지는 받는 쪽의 몫이었다. 검사가 서면 통과라는 사건이 생기고, 통과한 출력은 덜 검토된다.「검증된 출력」이라는 말이 쓰이기 시작하고, 뒤의 코드가 통과한 값을 전제로 갈라진다. 그런데 그 검사가 본 것은 형식이 전부다.

모델이 답 대신 거절을 내놓는 경우도 다르지 않다. 거절의 문장이 스키마의 자리에 담기면 그것도 형식을 지킨 출력이다. 형식을 정했다고 답이 온다는 뜻은 아니다. 값이 오지 않는 경우의 길은 받는 쪽에 따로 있어야 한다.

그리고 검사가 서면 검토가 준다. 로그를 훑던 사람과 결과를 대조하던 절차가 줄고, 실패의 원인을 찾는 일이 거부의 기록만 뒤지는 쪽으로 기운다. 그런데 형식이 맞는 틀린 답은 거부의 기록에 없다. 통과한 것들 속에 있고, 뒤의 어긋난 결과에서 거슬러 올라와야 찾아진다.

조이지 말아야 할 때

넷 가운데 하나에 해당하면 스키마를 들일 때가 아니다.

받는 쪽이 사람뿐일 때. 스키마의 값은 코드가 받는 데서 나온다. 최종 산출물이 사람이 읽는 보고라면 꺼내는 문제가 애초에 없고, 강제는 없는 문제를 풀면서 표현만 짚는다.

형식을 아직 모를 때. 답의 구조를 찾는 것 자체가 과제인 단계에서 미리 조이면 여물지 않은 이해가 굳는다. 받는 코드가 아직 없는 출력의 스키마는 약속이 아니라 추측이다.

잡으려는 실패가 내용의 실패일 때. 시스템을 아프게 하는 것이 꺼내기의 실패가 아니라 틀린 내용이라면 스키마는 그 실패에 닿지 않는다. 게다가 검사가 검토를 줄이는 효과까지 세면 조임이 방어를 뒤로 물릴 수도 있다. 그 실패의 처방은 내용을 검증하는 것이고, 그 설계는 제6부의 일이다.

어졌을 때의 길을 안 정했을 때. 고칠 것인가, 다시 시킬 것인가, 멈출 것인가. 그 결정을 안 하고 스키마만 들이면 결정이 사라지는 것이 아니라 도구나 하네스의 기본값이 대신 내린다. 대개 그 기본값은 조용히 고치거나 상한 없이 다시 시키는 것이고, 어느 쪽이든 값이 정해진 적 없이 청구된다.

넷을 관통하는 것이 있다. 스키마는 받는 쪽을 위한 장치다. 코드가 받고, 형식이 여물었고, 잡으려는 실패가 형식의 실패이고, 어졌을 때의 길이 정해져 있을 때 제 값을 한다.

그리고 이 넷 가운데 어느 것도 스키마 문서에 적혀 있지 않다. 받는 쪽의 목록, 과제의 성숙도, 실패의 분포, 어졌을 때의 처리는 전부 스키마 밖의 사정이다.

9.6 요약

한 문장으로. 스키마는 내놓는 쪽과 받는 쪽 사이의 약속을 코드가 대조할 수 있는 형식으로 적어, 꺼내기의 실패를 쓰이기 전의 시끄러운 사건으로 바꾼다. 그러나 형식이 맞는 틀린 답은 검사를 통과했다는 표시를 달고 그대로 남는다. 그래서 스키마 설계의 중심은 무엇을 강제할 것인가가 아니라 강제로 잡히지 않는 것이 무엇인지를 아는 것이다.

확실했던 것.

- 형식을 안 정해도 형식은 이미 있다. 받는 쪽의 파서에 흩어진 가정의 모양으로 있으며, 내놓는 쪽의 표현이 달라지는 날 받는 쪽의 오동작으로만 알려진다. 출력 스키마는 그것을 한 곳에 적고 판정을 코드에 맡긴 것이다(정의 9.1).
- 스키마는 프롬프트가 아니라 요청의 인자로 적는다. 생성 뒤에 대조해 거부하거나 생성 자체를 형식에 묶는데, 어느 쪽이든 강제되는 것은 형식까지다. 스키마에 적은 제약이라고 다 강제되지도 않으며, 강제되지 않는 조항은 설명문으로 내려가 다시 부탁이 된다(절 9.3).
- 모델은 자리의 이름과 설명만 보고 그 자리를 채운다. 무엇을 적는가, 언제 비워 두는가, 다른 자리와 어긋나면 무엇을 우선하는가가 설명에 적혀 있어야 한다(절 9.2).
- 조임은 받는 쪽을 쉽게 하고 답을 수 있는 것을 줄인다. 자리가 없는 단서는 버려지거나 열린 자리에 구겨 놓여지며 어느 쪽도 대조에 걸리지 않는다. 조일지 말지는 받는 쪽이 정한다(절 9.3).
- 대조하는 설계에서 여기는 일은 정상 거동이므로 처리의 사슬이 설계의 절반이다. 몰래 고치면 신호가 지워지고, 다시 시키려면 상한이 필요하며, 위로 올리려면 받는 쪽이 있어야 한다(표 9.1). 여기는 비율이 높으면 나쁜 것은 스키마다.
- 스키마가 움직이는 것은 형식의 축뿐이다. 형식의 실패는 쓰이기 전의 안내된 거부가 되지만 형식이 맞는 틀린 답은 통과한다(표 9.2). 위험은 남은 실패가 아니라 통과가 만드는 안심이다.

아직 답하지 않은 것.

- 형식의 통과율과 과제의 성공률을 가르는 평가의 실체는 제6부, 특히 제18장이 받는다.
- 어긴 비율과 고친 기록을 실행의 기록에 어떻게 남길 것인가는 제20장이 받는다.
- 하위 에이전트가 돌려주는 요약의 형식과 그 약속은 제17장이 받는다.
- 형식이 맞는 텍스트가 지시로 읽히는 경로의 방어는 제22장이 받는다.

오늘 할 것. 자기 스키마가 적힌 파일을 열어 필수로 표시된 자리를 하나씩 짚으며 이 값을 모델이 그 시점에 알 수 있는지 묻는다. 알 수 없는 자리마다 모른다고 적을 값이 있는지 확인한다(절 9.3). 다음으로 설명이 한 줄도 없는 자리를 세고, 그 자리에 무엇을 적는지와 언제 비워 두는지를 거기에 적어 넣는다(절 9.2). 마지막으로 지난 한 주의 로그에서 형식을 어긴 것을 자리별로 세어, 가장 많이 어긴 자리 하나에 표시한다(절 9.4).

9.7 생각해 볼 문제

1. 표 9.2의 네 행이 자기 시스템에서 얼마나 자주 일어나는지 추정하는 절차를 설계하라. 아래 두 행은 거부의 기록에서 바로 세어지지만 둘째 행은 통과했으므로 기록에 표시가 없다. 무엇과 대조해야 둘째 행이 세어지는가.
2. 자기 스키마에 적힌 제약을 강제되는 것과 강제되지 않는 것으로 갈라 보라. 강제되지 않는 쪽이 지켜지지 않았을 때 무엇이 그것을 잡는가. 잡는 것이 없다면 그 제약은 약속인가 희망인가.
3. 자리 설명을 지운 스키마와 그대로 둔 스키마로 같은 과제 스무 건을 돌려, 여기는 비율과 답의 옳음을 견주어라. 여기는 비율은 그대로인데 답만 나빠지는 자리가 있다면, 그 자리에서 설명이 하던 일은 무엇이었는가.
4. 재시도 상한을 정하는 절차를 설계하라. 필요한 것은 자리별로 여기는 비율, 다시 시켰을 때 고쳐지는 비율, 그리고 왕복이 이력에 더하는 몫이다. 고쳐지는 비율이 상한까지 일정하다는 가정과 급감한다는 가정에서 절차가 어떻게 달라지는가.

9.8 다시 읽을 때

- 제17장을 읽고 정의 9.1로 돌아오면, 내놓는 쪽의 목록에 하위 에이전트가 추가되어 있다. 일 이 다른 컨텍스트에서 일어나고 요약만 돌아오면 그 요약이 반환이며, 내놓는 것은 모델이다.
- 제18장을 읽고 표 9.2로 돌아오면, 형식의 축과 뜻의 축이 평가를 가르는 축이 되어 있다. 둘을 한 숫자로 뭉뚱그린 지표는 둘째 행을 성공 쪽에 센다.
- 제20장을 읽고 절 9.4로 돌아오면, 어긴 비율과 고친 기록이 무엇을 남길 것인가의 대상이 되어 있다. 거부된 것과 고쳐진 것이 스텝의 기록에 남아야 스키마와 모델의 어긋남이 보인다.
- 제22장을 읽고 절 9.5를 다시 보면, 검사 통과가 보증이 아니라는 문장이 방어의 언어가 되어 있다. 형식이 맞는 텍스트도 통과하고, 들어온 뒤에는 지시로 읽힐 수 있다.

9.9 더 읽을거리

구조화된 출력의 기능 문서(Anthropic, 2026h)부터 읽는 편이 좋다. 가져갈 것은 표기법이 아니라 문서가 스스로 그은 세 경계다. 형식을 적는 자리가 프롬프트가 아니라 요청의 인자라는 것, 보장하는 것이 형식의 준수까지라는 것, 적을 수 있는 제약과 실제로 강제되는 제약이 다르다는 것이다.

자리 설명의 기준은 도구를 다루는 자료(Anthropic, 2025g)에서 가져온다. 무엇을 하는지, 언제 쓰고 언제 쓰지 않는지, 각 자리가 무슨 뜻인지, 무엇을 돌려주지 않는지까지 적으라는 것이었다. Anthropic (2024a)에서는 형식을 고르는 세 항목이 값지다. 모델이 글에서 자연스럽게 보아 온 형태에 가까울 것, 형식을 지키는 일 자체가 부담이 되지 않을 것, 답을 적기 전에 생각할 토권을 남겨 둘 것이다. Tam et al. (2024)는 결과가 다투어지고 있다는 것을 알고 읽어야 한다. 가져갈 것은 형식의 강제가 공짜인가라는 물음이다.

Patil et al. (2024)는 이 눈으로 다시 읽힌다. 정확한 인자를 생성하지 못한다는 첫째 실패는 내놓는 쪽이 약속을 어긴 것이고, 지어낸 사용법이라는 둘째 실패는 형식이 맞는 틀린 답의 원형이다. 같은 이유로 제7장의 반환 설계가 가장 값진 재독이다.

제3부는 여기서 닫힌다. 제7장이 도구의 설계를, 제8장이 공급의 경로를, 여기서 돌아오는 조각의 형식을 세웠다. 제4부는 모델이 알지 못하는 것을 컨텍스트에 들이는 일이며, 제10장에서 시작한다.

제Ⅳ부

지식과 기억

에이전트 메모리: 단기, 장기, 영속

세션은 끝난다. 완료로든 절단으로든 컨텍스트는 닫히고, 그 안에만 있던 것은 사라진다. 모델은 실행에서 배우지 않는다. 어제 실행이 무엇을 알아냈든 오늘의 모델은 어제의 모델과 같은 가중치다. 실행이 얻은 것이 살아남는 길은 하나뿐이다. 적히는 것이다.

적는 순간 할 일이 셋 생긴다. 무엇을 남길지 정하고, 남긴 것이 낡으면 고치고, 틀린 것은 걷어내는 일이다. 저장소를 만들고 그 위에 검색을 붙이는 일은 이 셋 가운데 아무것도 해 주지 않는다. 이 장이 다루는 것이 그 셋이다.

여기서 다루지 않는 것도 적어 둔다. 기록을 어디서 자르고 무엇으로 표현해 색인에 얹히는가, 찾아온 것에 순위를 어떻게 매기는가는 이어지는 네 장의 소재이고, 절 10.5가 그리로 넘긴다. 사람에 관한 기록을 남길지 지울지를 누가 정하는가는 규범의 물음이라 제21장이 받는다.

10.1 남겨 두어야 하는 것

어제 정한 것을 오늘 다시 정하는 자리가 있다.

한 조직의 에이전트가 석 달째 같은 코드 저장소에서 일한다. 어제 오후의 실행은 어떤 수정 방식이 다른 부분을 깨뜨린다는 것을 두 시간에 걸쳐 확인하고 그 방식을 접었다. 오늘 아침, 같은 사람이 이어지는 과제를 들고 온다. 오늘의 세션이 아는 것은 오늘의 컨텍스트에 실린 것뿐이다. 어제의 두 시간은 어디에도 없다. 그래서 오늘의 실행은 같은 수정 방식을 다시 유망하게 보고, 같은 자리에서 다시 깨뜨리고, 같은 결론에 다시 도착한다.

왜 이렇게 되는지는 얇이 살 수 있는 자리를 짚으면 보인다. 파라미터는 훈련이 닫힌 날 굳었다. 실행이 무언가를 알게 되었다면 그 얇이 산 곳은 컨텍스트뿐인데, 컨텍스트는 세션과 함께 죽는다.

세션이 닫히기를 기다릴 것도 없다. 컨텍스트는 도는 동안에도 유한하다. 과제가 길어지면 앞의 것을 줄이거나 버려야 하고(제6장), 버린 것 가운데 뒤에 다시 필요한 것이 있다. 공식 기억 도구가 시스템 지시로 못 박아 두는 문장이 그 사정이다. 컨텍스트는 언제든 초기화될 수 있으므로 기억에 적히지 않은 진행은 잃을 각오를 해야 한다(Anthropic, 2026e).

그러니 남길 것을 골라 컨텍스트 밖에 두어야 한다.

정의 10.1 (에이전트 기억). 에이전트 시스템이 실행 중에 얻은 사실, 결정, 실패의 기록을 세션과 과제의 경계를 넘어 컨텍스트 밖에 남기고, 뒤의 실행이 필요할 때 그것을 컨텍스트로 들여 쓸 수 있게 유지하는 장치와 그 기록 전체를 기억이라 한다.

힘이 실려야 할 곳은 「실행 중에 얻은」이다. 사내 규정 문서는 시스템이 얻은 것이 아니라 받은 것이고, 그래서 그 문서를 고치는 일은 시스템의 몫이 아니다. 기억은 실행이 낳은 기록이라 그 관리가 통째로 시스템 쪽에 남는다.

무엇을 남길 것인가에는 물음 하나만 된다. 뒤의 실행이 그것 없이 같은 것을 다시 겪어야 하는가. 다시 조회하면 되는 것은 남기지 않는다. 코드의 현재 상태, 문서에 적혀 있는 절차처럼 환경에서 언제든 다시 읽을 수 있는 것을 기억에 옮겨 적으면 그것은 얇이 아니라 사본이 되고, 사본은 원본이 바뀌는 날 조용히 틀려진다.

표 10.1: 기억의 세 층위. 층을 가르는 것은 저장 기술이 아니라 무엇을 위해 두는가다.

층위	무엇을 위해	무엇이 들어가는가	언제 사라지는가	누가 적는가
단기	지금 도는 과제를 이어 가려고	이 세션의 이력과 관측, 지금 잡고 있는 항목, 방금 읽은 파일의 내용	세션이 닫히면. 그 전 이 라 도 압축이 오면 그때	아무도 정하지 않는다. 루프가 쌓는다
장기	이 사람 또는 이 프로젝트를 이어 가려고	진행 기록, 남은 일의 점검표, 이 프로젝트에서만 통하는 절차, 원하는 보고 형식	과제가 끝나는 날. 사람이나 프로젝트가 바뀌는 날	세션을 닫는 실행이 갱신한다. 사람이 고쳐 준다
영속	시스템이 계속 참고하려고	과제를 넘는 사실과 결정, 접은 접근과 그 이유, 조직의 어휘	정하지 않으면 사라지지 않는다	정해야 한다. 모델이 후보를 적고 자격이 승격을 정한다

남겨야 하는 것은 다시 얻는 값이 다시 겪는 값인 것들이다. 공식 자료가 드는 두 예가 정확히 그 자리에 있다. 여러 번의 주고받음 끝에야 드러난 것, 그리고 사람이 대화 도중에 고쳐 준 것이다 (Anthropic, 2026e). 둘 다 다시 얻으려면 그 주고받음을 처음부터 다시 해야 한다. 그리고 첫자리는 실패의 기록이다. 성공한 일은 고쳐진 파일로 환경에 남지만 접은 접근은 아무것도 남기지 않는다. 그 존재의 증거가 컨텍스트의 서사뿐이었고, 세션이 닫히면 그 서사마저 사라진다.

「우리는 아직 기억이 없다」는 말부터 의심해야 한다. 설계하지 않았다는 것과 없다는 것은 다른 문장이다. 실행의 로그가 쌓이고 있고, 세션을 닫으며 남긴 인계문이 폴더에 있고, 산출물이 저장소에 앉아 있고, 뒤의 실행이 그것을 다시 읽는다면 기억은 이미 있다. 다만 설계되지 않은 채 있을 뿐이다.

설계되지 않은 기억이란 앞의 셋이 전부 기본값으로 진행되는 기억이다. 무엇을 남길지 아무도 정하지 않았으므로 전부 남고, 고치는 일이 없으므로 남은 채 남고, 걷어내는 일이 없으므로 틀린 채 남는다. 남기고 고치고 걷어내는 셋이 이 장의 뼈대다.

10.2 세 층위

층을 가르는 것은 저장 기술이 아니고, 얼마나 오래 사는가도 아니다. 무엇을 위해 두는가다.

수명으로 가르는 편이 깔끔해 보이지만 실무에서는 순서가 뒤집힌다. 어떤 기록을 얼마나 돌지는 그것을 무엇에 쓰려고 두었는지가 정한다. 목적을 먼저 적으면 사라질 날이 따라 나오고, 수명부터 정하면 왜 하필 그 날인지 뭇 근거가 없다.

용도는 셋이다. 지금 도는 과제를 이어 가려고 두는 것, 이 사람 또는 이 프로젝트를 이어 가려고 두는 것, 시스템이 계속 참고할 것으로 두는 것이다. 차례로 단기, 장기, 영속이라 부른다.

층이 셋이라는 것은 세션 하나가 열릴 때 실제로 세 종류를 읽는다는 뜻이다. 오늘 아침의 세션은 먼저 어제 세션이 닫히며 남긴 진행 기록을 연다. 어디까지 했고 다음이 무엇인지가 거기 있고, 이 과제가 끝나는 날 함께 접는다. 다음으로 접은 접근을 적어 둔 파일을 연다. 석 달 치가 쌓여 있고 이 과제가 끝나도 사라지지 않는다. 그리고 일하는 동안 읽은 파일과 실행한 명령의 결과가 컨텍스트에 쌓이는데, 이것은 오늘 저녁에 사라진다. 세 종류가 한 컨텍스트에 나란히 놓이지만 관리의 일은 셋 다 다르다.

누가 적는가와 짝이 되는 물음이 어디에 두는가다. 공식 기억 도구에서 저장은 전적으로 부르는 쪽에 있다. 모델은 파일을 열고 고쳐 달라고 청할 뿐이고, 그 청을 실제 저장소에 옮기는 것은 부르는 쪽의 코드다(Anthropic, 2026e). 그래서 사람마다 폴더를 따로 두는 일도, 어떤 폴더는 읽기만 되게 두는 일도 설계하는 쪽의 결정이다. 영속의 기록에 아무나 적지 못하게 막는 것이 남길 것을 정하는

가장 짠 방법이다.

금이 하나 더 지나간다. 단기는 세션 안의 일이고 장기와 영속은 세션 사이의 일이다. 세션 안에서 컨텍스트를 줄이는 수단은 압축과 정리이고 제6장이 다뤘다. 세션 사이를 잇는 수단이 기억이다. 둘은 서로 밀어내지 않고 짝으로 쓰인다. 공식 지침이 그렇게 적는다. 압축이 도는 컨텍스트를 작게 유지하고, 기억이 요약을 넘어 살아남아야 할 것을 보존한다(Anthropic, 2026e). 한 구절로 가르면 이렇다. 압축은 이력을 줄이면서 버리고, 기억은 버려도 남게 한다.

관리의 일이 층마다 다른 이유는 하나다. 끝이 정해진 층은 끝이 관리를 대신한다. 단기는 세션이 닫히며 사라지는 것이 기본값이라 관리란 사라지기 전에 무엇을 건질 것인가다. 장기는 과제가 끝나는 날 접으면 되므로 관리란 과제가 사는 동안 갱신을 유지하는 것이다. 영속에는 그 대신이 없다. 남아도 스스로 물러나지 않고, 틀려도 스스로 드러나지 않고, 쌓여도 스스로 줄지 않는다. 이 장의 나머지가 사실상 이 층의 이야기다.

극단을 짚으면 이 가름이 장식이 아니라는 것이 보인다. 전부 영속으로 두면 어떻게 되는가. 먼저 양이다. 기록이 실행 시간에 비례해 자라니, 이력이 컨텍스트를 지배하던 일이 이번에는 저장소에서 반복된다. 다음이 찾기 쪽에서 온다. 같은 주제의 기록이 수백이면 상위 몇 자리를 두고 남은 것들이 경쟁하고, 석 달 전의 사실과 어제의 사실이 같은 자격으로 실린다. 마지막이 소임을 다한 기록의 자리다. 끝난 과제의 인계가 새 과제의 질의에 걸려 후보의 자리를 차지한다. 전부 남기는 시스템은 아무것도 남기지 않는 시스템과 반대 극단에서 출발해 같은 자리에 도착한다. 필요한 것이 컨텍스트에 실리지 못하는 자리다.

그래서 잇는 일도 설계의 대상이다. 수명을 정하지 않은 기록의 기본값은 영속이다. 지우기는 누군가 결정해야 일어나는 사건인데, 결정하는 장치가 없으면 그 사건은 일어나지 않는다. 처방은 거창하지 않다. 기록을 적는 날에 그 기록이 언제 소임을 다하는지를 함께 적는 것이다. 이 기록은 과제가 끝나면 접힌다든지, 이 사실은 다음 검증에서 대체된단지 하는 한 줄이 지우는 날의 결정을 적는 날의 결정으로 옮겨 준다.

10.3 사용되는 순간

기억은 적히는 것만으로는 아무 일도 하지 않는다. 컨텍스트에 실려야 결정을 바꾼다. 그러니 설계의 절반은 언제, 어디에, 얼마나 실을 것인가다.

제4장이 컨텍스트를 층으로 갈랐을 때 이미 자리가 하나 잡혀 있었다. 앞선 세션이 다음 세션 보라고 남긴 것이 실리는 자리다. 기억은 그 자리를 넓힌 것이다. 새로운 종류의 무엇이 아니라 컨텍스트의 한 층이고, 층이라면 실리는 시점과 자리와 크기를 물어야 한다.

언제 실리는가. 제4장의 갈래는 둘이었다. 처음부터 실어 두는 것과, 필요해진 시점에 도구로 가져오는 것이다. 공식 기억 도구가 고르는 답은 둘을 섞은 것이다. 세션의 첫 일로 기억 폴더의 목록을 본다. 그리고 오늘의 과제에 걸리는 파일만 연다. 표지는 미리, 내용은 필요할 때다.

목록을 먼저 보는 것이 권고가 아니라 규율인 데는 이유가 있다. 지시가 「무엇을 하기 전에 기억 디렉터리부터 본다」로 못 박혀 있다(Anthropic, 2026e). 찾아 쓰는 일을 그때그때의 판단에 맡기면 필요한 줄 몰라 안 찾는 실패가 세션마다 되풀이되는데, 안 찾은 세션의 로그에는 아무 표시도 남지 않는다. 세어지지 않는 실패는 고쳐지지도 않는다.

어디에 실리는가. 세션 시작에 실린 목록은 앞자리에 온다. 앞자리는 매 스텝 다시 실리므로 작아야 하고, 그래서 목록이어야 한다. 도구로 열어 온 파일의 내용은 이력에 붙어 뒤에 온다. 뒤에 붙은 것은 그 스텝부터 세션이 끝날 때까지 따라다니므로, 큰 파일 하나를 통째로 여는 결정은 그 세션 내내 값을 묻는다.

얼마나 실리는가. 상한을 세 자리에 건다. 파일 하나가 자랄 수 있는 크기, 한 번에 돌려주는 글자 수, 그리고 한 세션이 여는 파일의 수다. 앞의 둘은 공식 운영 지침에 그대로 있다. 파일 크기를 측정하고 상한을 두고, 한 번에 돌려주는 글자 수를 잘라 나머지는 범위를 지정해 나눠 읽게 한다(Anthropic, 2026e).

실물을 보면 셋이 한꺼번에 보인다. 세션이 처음 받는 것은 이런 목록이다.

기억 폴더
1.2K 검증-절차
3.1K 접은-접근
0.8K 조직-어휘
0.4K 보고-형식

여기까지가 앞자리에 실리고, 크기는 몇십 톤이다. 오늘의 과제가 검증에 걸리는 일이면 그다음에 파일 하나를 연다.

접은 접근

[단정] 검증 단계에서 색인을 처음부터 다시 만들지 않는다
적은 날 3월 4일 / 근거 같은 날의 실행 기록 / 검증 통과
까닭: 색인을 다시 만드는 데 검증 전체보다 오래 걸렸다
다시 볼 조건: 색인 도구가 바뀐 부분만 갱신하게 되면

[후보] 야간 배치는 월요일에 밀린다
적은 날 3월 11일 / 근거 실행 하나의 관측 / 검증 없음

기록마다 한 줄이 더 붙어 있고, 그 줄에 세 가지가 들어 있다. 자격과 나이와 근거다. 자격은 이 문장을 단정으로 읽어도 되는지를 말하고, 나이는 얼마나 낡았을 수 있는지를 말하고, 근거는 의심이 들 때 어디로 거슬러 갈지를 말한다. 이 셋이 없으면 다음 절의 일이 서지 않는다. 무엇을 먼저 의심할지 정할 수 없고, 충돌이 났을 때 어느 쪽이 이기는지도 정할 수 없다.

실리는 순간과 짝이 되는 것이 적히는 순간이다. 어느 쪽도 그때그때의 판단에 맡기지 않는 편이 낫고, 실무가 쓰는 방아쇠가 하나 있다. 컨텍스트를 정리하는 문턱이다. 오래된 도구 결과를 지워 컨텍스트를 줄이는 구성에서는 문턱에 닿기 직전에 지워질 것이 있다는 경고가 먼저 가고, 그 경고를 받은 실행이 지워지기 전에 건질 것을 기억에 적는다(Anthropic, 2026e). 적는 시점을 판단이 아니라 사건에 건 것이다. 사건은 로그에 남으므로 안 적고 지나간 세션이 세어진다.

실린 뒤의 사정은 앞 장들이 말한 그대로다. 기억의 문장도 결국 컨텍스트에 실린 텍스트라, 넣는 일은 기울이는 일이지 결정하는 일이 아니다. 실었다고 지켜지는 것은 아니라는 뜻이다.

10.4 잘못 누적되는 기억

기억은 쌓기만 하면 어느 시점부터 시스템을 나쁘게 만든다. 늘어난 기록이 컨텍스트를 먹기 때문만은 아니다. 밖에서 받은 문서와 달리 기억은 스스로 상한다.

세 가지가 다르다. 첫째, 누가 적었는가. 사내 규정 문서는 시스템 밖의 사람이 적었고 그 사람이 관리한다. 문서가 틀리면 고칠 사람이 그쪽에 있다. 기억의 문장은 시스템 자신이 적었다. 둘째, 틀릴 수 있는가. 검색은 코퍼스가 대체로 옳다는 가정 위에서 성립한다. 기억은 그 가정을 세울 수 없다. 기억의 문장은 관측 그 자체가 아니라 관측 위에서 내린 결론이고, 결론은 매번 새로 뽑힌 것이다. 틀림은 예외가 아니라 셈에 넣어야 할 상수다. 셋째, 낡으면 누가 고치는가. 규정이 바뀌면 문서의 주인이 고친다. 기억이 낡는 가장 흔한 사정은 세계가 바뀐 것인데, 에이전트가 사는 세계에서 그 변경의 주범은 대개 자신이다. 검사를 빠르게 고친 것이 다음 달의 어느 실행이라면, 낡음을 만든 것도 알아챌 단서를 가진 것도 고쳐야 하는 것도 전부 시스템 쪽이다.

쌓이는 것은 네 모양으로 온다.

무엇을 적을지 정해 둔다

남길지의 결정은 누가 내리는가. 후보는 셋이다. 하네스의 규칙이 기계적 사건에 걸어 들 수 있다. 검증이 통과된 결정, 완료된 과제의 결정처럼 자격이 붙은 것만 적는 것이다. 모델이 실행 중에 판단할 수 있다. 그리고 사람이 적어 줄 수 있다. 셋 가운데 모델의 손이 가장 넓은 것을 줌고, 같은 이유로

표 10.2: 잘못 쌓이는 기억 네 모양. 오른쪽 두 열이 이 절의 처방이다.

모양	어떻게 생기는가	무엇이 알려 주는가	막는 자리
틀린 기록	이른 실행이 관측을 잘못 읽고 결론으로 적었다	아무것도 알려 주지 않는다. 다시 검증해야만 드러난다	적을 때 자격을 붙인다. 검증을 통과한 것만 단정으로 읽히게 한다
낡은 기록	세계가 바뀌었는데 옛 문장이 그대로 있다	같은 주제의 기록이 둘 이상이 될 때	덧붙이지 말고 옛 기록을 열어 고친다. 주제마다 파일 하나
값어치 없는 기록	남길 기준이 없어 꺾은 것을 다 적었다	오래 열리지 않은 파일	무엇을 적을지 미리 정해 둔다. 나이와 크기로 지운다
사본	환경에 이미 있는 것을 옮겨 적었다	원본과 어긋나는 날	다시 조회하면 되는 것은 적지 않는다(절 10.1)

가장 위험하다. 적은 대상이 자기 실행이기 때문이다. 요약하는 쪽이 자기 서사의 저자일 때 진행은 부풀려지고 실패는 뭉개진다는 것이 제6장의 경고였다. 기억은 그 요약이 세션을 넘어 굳는 자리다.

절충은 익숙한 형태다. 손은 빌리되 결정은 넘기지 않는다. 모델은 남길 후보를 적고, 승격은 자격이 정한다. 검증을 통과한 결론과 사람이 확인한 사실은 단정의 자격으로 적히고, 모델이 스스로 적은 것은 후보의 자격으로 남아 뒤의 실행에 실릴 때에도 그 자격을 달고 실린다. 자격이 없는 기억은 전부 단정으로 읽힌다. 적을 범위를 좁혀 거는 길도 있다. 이 주제에 관한 것만 기억에 적으라고 지시에 적어 두는 방식이 공식 문서에 예시로 있다(Anthropic, 2026e).

고치는 길과 지우는 길을 함께 만든다

남음부터 본다. 기억에 「이 저장소의 검사는 특정 모듈을 건드리면 오래 걸린다」가 적혀 있고, 어느 실행이 그 검사를 빠르게 고쳤다. 이 순간 기억은 낡았고, 아무 일도 일어나지 않으면 다음 실행은 낡은 기록을 받아 이제는 필요 없는 우회를 다시 짤다. 가장 쉬운 길, 곧 새 기록을 하나 더 적는 길의 끝은 뻥하다. 옛 문장과 새 문장이 같은 자격으로 찾아지고, 어느 쪽을 믿을지가 모델의 손에 넘어간다. 낡은 길은 옛 기록을 찾아 고치는 것이다.

그러려면 새 사실과 옛 기록이 같은 것에 관한 기록이라는 판정이 먼저 서야 한다. 기록을 조각으로 흩어 두면 그 판정이 갱신마다 필요하다. 주제마다 파일 하나를 두면 파일 이름이 그 판정을 미리 해 둔 것이 되어, 갱신은 그 파일을 열어 한 줄을 고치는 일이 된다. 저장의 형식이 관리의 값을 정한다.

공식 기억 도구가 색인이 아니라 파일 폴더인 것이 여기서 값을 한다. 모델이 파일을 열고 만들고 고치고 지우고 이름을 바꾼다(Anthropic, 2026e). 순위도 표현도 없고, 폴더의 짜임과 파일 이름이 어디를 열지의 단서가 된다. 이 형태에서는 고치는 일이 한 줄을 바꿔 쓰는 일이 되고, 건어내는 일이 파일 하나를 지우는 일이 된다. 정리해 두라는 요구를 모델에게 직접 걸 수도 있다. 내용을 최신으로 유지하고 앞뒤가 맞게 하며, 쓸모없어진 파일은 이름을 바꾸거나 지우고, 꼭 필요하지 않으면 새 파일을 만들지 않는다는 지시가 예시로 적혀 있다(Anthropic, 2026e).

충돌이 발견되었다고 해소가 자명한 것은 아니다. 시점이 가장 흔한 기준이다. 나중의 기록이 이긴다고 두고, 진 기록은 지우는 대신 기간을 달아 이력으로 남긴다. 대개는 이것으로 충분하지만 한계는 적어 두어야 한다. 날짜는 어느 쪽이 나중인지를 말해 줄 뿐 어느 쪽이 옳은지를 말해 주지 않는다.

처음부터 틀린 갈래는 더 나쁘다. 이른 실행 하나가 관측을 잘못 읽고 「이 설정은 성능 문제 때문에 꺼 두었다」라고 적었다고 하자. 실제 이유는 다른 것이었다. 뒤의 실행이 그 설정을 켜는 개선을 검토할 때마다 이 기록이 실리고, 실행은 제안을 접는다. 실린 기록을 근거로 삼는 것은 기억을 둔 목적 그 자체이고, 접는 결정은 그 정보 상태에서 합리적이다. 그리고 접는 결정이 다시 적힌다. 「성능

문제 때문에 이 설정은 켜지 않기로 함. 이제 틀린 문장은 돌리고, 뒤엣것은 앞엣것을 근거로 달고 있어 더 단단하다. 틀린 기억은 자기를 근거로 삼은 문장들을 거느린다.

틀린 기억이 검색의 틀린 조각보다 나쁜 이유는 고치는 길에 있다. 검색에서는 답에서 조각으로, 조각에서 문서로 거슬러 올라가 대조가 성립한다. 틀린 기억의 출처는 지나간 실행이고, 그 실행의 컨텍스트는 세션과 함께 사라졌다. 실행의 기록을 요약본 밖에 그대로 남겨 두었다면 (제6장) 그날의 관측으로 거슬러 가 어디서 결론이 미끄러졌는지 볼 수 있다. 남기지 않았다면 대조할 원본이 없고, 남은 길은 세계에 다시 물어보는 것뿐이다. 설정을 실제로 켜 보고, 문서를 다시 읽고, 그 팀에 다시 묻는 일이다.

걸어내기의 형태도 여기서 정해진다. 틀렸다고 판정된 기록을 흔적 없이 지우면 같은 관측을 같은 방식으로 잘못 읽는 조건이 그대로 남아, 시스템은 같은 틀림을 다시 적으려 간다. 그래서 기본형은 삭제가 아니라 정정이다. 틀렸음이 확인되었다는 것과 무엇으로 대체되었다는 것을 기록 위에 적어, 찾기에는 걸리되 단정으로는 읽히지 않게 만든다. 예외가 하나 있다. 사람에 관한 기록은 정정이 아니라 소거가 요구될 수 있고, 그 이유는 시스템의 셸 밖에 있다. 다음 절에서 굶는다.

얼마나 쌓였는지 센다

다시 검증하는 일은 비싸서 전부에 할 수 없으므로 의심에는 순서가 필요하다. 오래된 기록, 근거가 얇은 기록, 무거운 결정에 자주 실리는 기록부터다. 이 순서는 기록에 나이와 근거와 자격이 적혀 있어야 세워진다. 앞 절의 실물에 그 한 줄이 붙어 있던 이유가 이것이다.

무엇을 셀 것인가부터 정해 둔다. 내용을 읽지 않고 나오는 값이 셋 있다. 첫째는 기억의 수와 총 크기다. 지난달과 견주어 얼마나 늘었는지, 늘어난 만큼 열린 것도 늘었는지를 보면 쌓이기만 하는 저장소가 드러난다. 둘째는 한 세션이 여는 파일의 수다. 이 값이 0에 가까우면 기억이 있는데 쓰이지 않는 것이고, 상한 없이 늘어나면 목록이 이미 너무 길다는 뜻이다. 셋째는 마지막으로 고쳐진 날의 분포다. 전부 적힌 날 그대로라면 고치는 길이 아예 만들어지지 않은 것이다. 셋 다 판정 없이 세어진다.

손이 덜 가는 규율도 있다. 오래 열리지 않은 파일을 주기적으로 지우고, 파일 하나가 자랄 수 있는 크기에 상한을 둔다(Anthropic, 2026e). 나이와 크기는 내용을 읽지 않고도 측정할 수 있는 두 값이고, 판정 없이 걸 수 있는 유일한 규율이기도 하다.

측정하는 데에도 한계가 있다. 기억이 얼마나 틀렸는지는 다시 검증해 본 표본으로만 어렵할 수 있는데 그 표본의 크기와 주기에 일반해가 없고, 낱음의 속도도 영역마다 달라 어느 나이부터 의심할지의 문턱에도 일반해가 없다. 그래도 세지 않는 것과 어렵으로 세는 것은 다르다. 검사가 없는 시스템에서 틀린 기억의 수명은 시스템의 수명과 같다.

이 실패가 수선의 후보에 잘 오르지 않는다는 것도 적어 둔다. 실행이 이상해 로그를 열면 찾기는 관련된 기록을 돌려주었고 모델은 실린 기록에 충실했다. 아무도 잘못하지 않았고, 잘못은 석 달 전 실행에 있는데 그 실행은 오늘의 로그 어디에도 없다. 기억을 들인 시스템의 실패 분석 목록에 항목이 하나 늘어야 하는 이유다. 이 결정이 닫고 선 기억은 언제 누가 적었고, 검증된 적이 있는가.

10.5 기억을 두지 말아야 할 때

기억은 갖춰 두면 손해 볼 것 없는 장치처럼 보인다. 그래서 설계의 물음이 되지 못하고 기본으로 끼워 넣어지는 일이 잦다. 밖에서 오는 문의를 받아 답하는 시스템을 놓고 보자. 문의는 때때로 다른 사람이 보내고, 하나가 끝나면 다음 문의와 이을 것이 없다. 여기에 기억을 붙이면 남은 것은 늘어나는데 다시 열리는 것이 없고, 늘어난 것을 고치고 걸어내는 일만 그대로 남는다. 값은 치르고 값어치는 못 받는 구성이다.

다음 조건에 해당하면 기억을 들일 때가 아니다.

1. **이어 갈 반복이 없다.** 세 얼굴로 온다. 한 번에 끝나는 과제, 매번 다른 사람이 오는 자리, 앞의 실행과 이을 것이 없는 과제다. 기억의 값은 같은 저장소, 같은 규정, 같은 사람과의 반복에서 나온다. 반복이 없으면 이을 것이 없고, 세션 하나를 잇는 인계와 산출물로 충분하다. 반복 없는 시스템의 기억은 아무도 열지 않는 파일 더미이고, 유지비만 든다.
2. **환경이 이미 더 나은 기억이다.** 조직에 코드 저장소와 문서와 작업 목록이 살아 있다면, 결정과 사실은 그곳에 적는 쪽이 낫다. 사람과 시스템이 함께 읽고 고치는 기록은 낡음을 여럿이 감당한다. 환경에 적을 수 있는 것을 기억 저장소에 따로 적으면 사본이 하나 늘고, 사본은 낡는다. 기억이 맡을 것은 어디에도 적을 자리가 없는 것, 곧 시스템 자신의 경험이다.
3. **틀림을 감당할 수 없는데 검증할 수단이 없다.** 기억의 틀림은 상수이고 다시 검증하는 일만이 그것을 걷어낸다. 기록의 진위를 세계에 다시 물을 수단이 없는 영역에서, 또는 틀린 단정 하나가 무거운 행동으로 이어지는 과제에서 기억은 확신 있는 오염원이 된다. 그런 자리에서 기억의 뒤편을 주는 것이 아니라 어디를 다시 읽을지 가리키는 것까지다.
4. **사람에 관한 것이다.** 사용자의 선호를 기억하는 시스템은 편리해지지만, 사람에 관한 기록은 이 교환만으로 정할 수 없다. 다른 셋과 결이 다른 지점은 걷어내기의 주인이다. 무엇을 남기고 언제 지울지를 시스템의 선택이 아니라 그 사람이 정해야 하는 자리가 있고, 그 순간 지우기는 기술의 문제이기 때문이다. 동의와 삭제에 둘러싼 규범의 논의는 제21장이 받는다.

넷을 관통하는 형태가 있다. 반복이 없으면 남길 일이 없고, 환경이 대신하면 고칠 일을 나눌 수 있다. 검증이 없으면 걷어낼 수 없고, 사람에 관한 것이면 걷어냄의 주인이 바뀐다.

넷은 기억 전체를 둘 것이냐 말 것이냐의 판정이 아니다. 층마다 따로 물어야 맞다. 반복이 없는 시스템도 세션 하나를 잇는 장기의 기록은 필요하고, 영속만 두지 않으면 된다. 환경이 더 나은 기억인 조직에서도 시스템 자신이 겪은 실패는 어디에도 적을 자리가 없으니 영속으로 남는다. 물음은 기억을 둘 것인가가 아니라 어느 층까지 둘 것인가다.

마지막으로 가장 자주 헛갈리는 경계를 긋는다. 기억과 검색은 다른 물음이다. 기억은 시스템 자신이 겪은 것을 남기는 일이고, 검색은 이미 어딘가에 있는 것을 찾는 일이다. 남기는 쪽은 무엇을 적을지부터 정해야 하지만, 찾는 쪽은 무엇이 있는지가 이미 정해져 있다. 둘을 헛갈리면 기억 저장소에 문서를 옮겨 놓게 되고, 그 순간 표 10.2의 넷째 줄이 된다.

이 장이 다룬 것은 전부 남기는 쪽의 일이었다. 찾는 쪽의 일은 여기서 시작한다. 기록을 어디서 자르고 무엇으로 표현해 색인에 앞힐 것인가, 찾아온 것을 넣을지와 지식을 어디에 둘 것인가가 제11장, 두 검색을 겹치고 순위를 다시 매기는 일이 제12장, 개체와 관계로 세는 물음이 제13장, 텍스트가 아닌 원천이 제14장이다. 기억이 그 다섯을 앞질러 서는 것은 무엇을 남길지가 어떻게 찾을지보다 먼저 오는 결정이기 때문이다.

10.6 요약

한 문장으로. 기억은 시스템이 스스로 남긴 것을 세션과 과제를 넘어 이어 가는 장치이고, 기록의 주인이 자신이 되면서 남기고 고치고 걷어내는 일이 안으로 들어온다.

확실해진 것.

- 모델은 실행에서 배우지 않으므로 실행이 얻은 것이 살아남는 길은 적히는 것뿐이다. 컨텍스트는 세션과 함께 죽고 도는 동안에도 유한하다. 기록이 쌓이고 뒤의 실행이 그것을 다시 읽는 시스템에 기억은 이미 있으며, 결정은 그것을 설계의 대상으로 삼을 것인가다(절 10.1).
- 충위를 가르는 것은 수명이 아니라 무엇을 위해 두는가다. 지금 도는 과제를 위한 것, 이 사람과 이 프로젝트를 위한 것, 시스템이 계속 참고할 것의 셋이고, 층마다 들어가는 것과 사라지는 때와 적는 주체가 다르다(표 10.1). 끝이 있는 층은 끝이 관리를 대신하지만 영속의 층에는 그 대신이 없다.
- 적힌 기억은 실려야 값을 한다. 목록은 세션 시작에 앞자리로, 내용은 필요할 때 뒤에 붙는다. 상한은 파일 크기와 한 번에 돌려주는 양과 여는 파일의 수에 건다. 기록마다 자격과 나이와 근거를 함께 적는다(절 10.3).
- 기억은 스스로 상한다. 자신이 적었고, 틀림이 예외가 아니라 상수이고, 낡음을 만든 것이 대개 자신이기 때문이다. 쌓이는 것은 틀린 기록과 낡은 기록과 값어치 없는 기록과 사본의 네 모양이다(표 10.2). 갱신은 덧붙이기가 아니라 제자리 고치기여야 하고, 걷어내기의 기본형은 삭제가 아니라 정정이다(절 10.4).
- 기억을 두지 말아야 할 자리는 넷이다. 이어 갈 반복이 없을 때, 환경이 이미 더 나은 기억일 때, 틀림을 검증할 수 없을 때, 그리고 사람에 관한 것일 때다. 마지막 것은 걷어내기의 주인이 시스템이 아니게 되는 유일한 층이다(절 10.5).

아직 답하지 않은 것.

- 기억에 둘 것과 검색 코퍼스에 둘 것과 파라미터에 둘 것을 건주는 배치의 의사결정은 제11장이 받는다.
- 여러 에이전트가 기억을 나눠 갖거나 함께 쓸 때의 썸에는 답이 없다. 컨텍스트를 갈라도 기억은 공유되기 쉬우므로 한 에이전트가 적은 기록이 다른 에이전트의 전제가 되는데, 이 맞바꿈을 평가해 본 자료가 없다.
- 기억의 품질을 평가하는 일, 곧 다시 검증할 표본의 설계는 제6부가 받는다.
- 기억에 적힌 텍스트가 뒤의 실행을 조종하는 경로의 방어는 제22장이 받는다.
- 사람에 관한 기억의 동의, 열람, 삭제를 둘러싼 규범과 규제는 제21장이 받는다.

오늘 할 것. 뒤의 실행이 다시 읽는 파일이 지금 있는지 코드에서 찾는다. 로그를 적는 자리, 인계문을 쓰는 자리, 산출물을 두는 자리를 열어 그 가운데 뒤의 실행이 실제로 다시 여는 것에 표시한다(절 10.1). 표시한 것마다 언제 사라지는지 한 줄로 적는다. 사라질 날이 적히지 않는 것이 영속이고, 관리가 온전히 걸리는 층이 그것이다(절 10.2). 마지막으로 영속으로 판정된 파일의 마지막 수정 날짜와 크기를 측정한다(절 10.4).

10.7 생각해 볼 문제

1. 자기 시스템에서 설계되지 않은 기억의 목록을 만들어 보라. 로그, 인계문, 산출물, 결정 기록 가운데 뒤의 실행이 실제로 다시 읽은 것은 무엇인가. 그 각각에 대해 남기고 고치고 걷어내는 일이 설계되어 있는지 방치되어 있는지를 표로 정리하라.
2. 모델의 재량으로 적게 한 기억과 검증 통과만 승격하는 기억을 견주는 실험을 설계하라. 같은 과제 10개를 두 구성으로 돌린 뒤 무엇을 평가해야 두 기억의 품질이 같리는가. 재량 기억에서 자기 서사의 흔적, 곧 진행의 과장과 실패의 누락을 찾으려면 무엇과 대조해야 하는가.
3. 틀린 기억 하나를 일부러 심고 그것을 근거로 삼은 결정을 추적하는 실험을 설계하라. 몇 실행 뒤에 그런 결정이 나타나는가. 그 결정이 낳은 새 기록은 몇 개이고, 원래 문장을 정정하면 나머지는 어떻게 되는가. 이 실험이 자결과 나이와 근거 없이는 성립하지 않는 이유를 절 10.3으로 설명하라.
4. 수명을 적는 규칙을 연습해 보라. 자기 시스템이 남기는 기록의 종류마다 무엇을 위해 두는 것인지 먼저 적고, 거기서 사라질 날이 따라 나오는지 확인하라. 따라 나오지 않는 종류가 있다면 그것은 용도가 흐린 것인가, 아니면 영속이어야 하는 것인가.

10.8 다시 읽을 때

- 제11장을 읽고 절 10.5의 둘째 조건으로 돌아오면, 환경에 적을 것과 기억에 적을 것의 경계가 배치 의사결정의 한 조각이 되어 있다. 기억의 유지비가 검색 코퍼스의 유지비나 파라미터 갱신의 값과 나란히 놓이는 것이 거기다.
- 제15장을 읽고 표 10.1의 장기 줄로 돌아오면, 세션을 닫으며 무엇을 남길지의 규율이 곧 장기 층의 쓰기 규율이었음이 보인다. 인계는 기억의 가장 작은 사례이고, 수명이 과제에 매여 있어 관리가 갱신 하나로 줄어든 경우다.
- 제6부를 읽고 절 10.4의 미해결로 돌아오면, 다시 검증할 표본과 의심의 순서가 평가 설계의 문제가 되어 있다. 기억이 얼마나 틀렸는지를 모른 채 기억을 늘리는 일이 무엇을 쌓는지가 거기서 정확해진다.
- 제22장을 읽고 절 10.4의 쓰기 권한으로 돌아오면, 그 설계가 방어의 문제로 다시 보인다. 기억에 적히는 것은 뒤의 모든 실행의 컨텍스트에 실릴 수 있는 텍스트다. 신뢰 경계 밖의 내용이 기억에 적히는 순간, 한 번의 주입이 지속되는 주입이 된다.
- 제21장을 읽고 절 10.5의 넷째 조건으로 돌아오면, 걷어내기의 주인이 바뀐다는 한 문장이 권한 모델과 규제의 언어로 펼쳐져 있다.

10.9 더 읽을거리

Packer et al. (2023)은 언어 모델이 제한된 컨텍스트 윈도우에 매여 있어 긴 대화나 문서 분석에서 쓸모가 제한된다는 데서 출발해, 운영체제의 계층적 기억 관리에서 착안한 기법을 제안했다. 이 기법으로 만든 시스템은 모델이 함수 호출로 외부 저장소를 읽고 쓰며 자기 컨텍스트를 스스로 고친다. 가져갈 것은 특정 구성의 세부가 아니라 둘이다. 기억이라는 장치가 컨텍스트 윈도우의 한계에서 출발한다는 것, 그리고 모델이 자기 기억을 스스로 관리하는 구조가 이미 제안되어 있다는 것. 이 장의 눈으로 읽는다면 물을 것은 하나다. 그 구조에서 남기고 고치고 걷어내는 일은 각각 누구의 손에 있는가.

Anthropic (2026e)에서는 실물의 조건을 가져왔다. 기억이 색인이 아니라 파일 폴터이 수 있다는 것, 무엇을 하기 전에 기억부터 보라는 규율, 압축과 함께 쓰라는 권고, 무엇을 적을지 지시로 좁히

라는 예시, 그리고 오래 안 열린 파일을 지우고 크기에 상한을 두라는 운영 지침이다. 같은 자료가 여러 세션에 걸치는 일에 대해 짜임을 먼저 세우라고도 적는다. 첫 세션이 일을 시작하기 전에 진행 기록과 남은 일의 점검표를 만들어 두고, 뒤의 세션이 그것을 읽고 갱신하며, 한 번에 한 항목을 하되 끝에서 끝까지 검증된 뒤에만 완료로 적는다는 것이다. 여기에 적히지 않은 것도 하나 있다. 기억 파일의 경로를 검사하지 않으면 기억 밖의 파일이 열린다는 경고인데, 그 문제는 제22장의 소재다.

가장 값진 재독은 이 책 안에 있다. 제6장이 압축된 구간의 원문을 컨텍스트 밖에 그대로 남겨 두라고 요구한 것은, 여기 와서 보면 틀린 기억을 대조로 고칠 수 있는 유일한 길을 미리 사둔 일이었다.

검색 증강 생성(RAG)

사내 규정을 묻는다. 출장 결제 한도가 얼마인가. 모델이 답한다. 문장은 매끄럽고 확신도 붙어 있다. 그런데 그 규정은 지난달 개정되었고, 모델은 개정 전의 값을 말했다.

더 나쁜 것은 모델이 자기가 낡은 값을 말했다는 것을 모른다는 점이다. 그리고 이 답에는 어디서 가져온 값인지가 적혀 있지 않아, 확인하려는 사람이 열어 볼 문서가 없다.

검색 증강은 그 자리에 문서를 들여놓는다. 모델 안이 아니라 별도의 저장소에 규정을 두고, 질문이 올 때 관련된 부분만 꺼내 컨텍스트에 실은 다음 그 위에서 답하게 한다. 흔히「모르는 것을 알려주는 장치」로 소개되는데 그 규정은 썩다. 실제로 바뀌는 것은 셋이다. 시스템이 무엇을 아는가, 그 지식이 언제 것인가, 그리고 그것이 어디서 왔는가.

에이전트에서는 뒤의 둘이 앞의 것만큼 무겁다. 갱신할 수 없는 지식은 낡은 채로 확신을 유지하고, 출처를 댈 수 없는 답은 틀렸을 때 고치려 갈 자리가 없다.

그리고 이 장치의 힘은 컨텍스트에 넣는 데까지만 미친다. 넣었다고 쓰이는 것이 아니고, 못 찾은 것은 조용하며, 적혀 있는 것만 찾아진다.

여기서 다루는 것은 지형과 대표 방법까지다. 찾기를 겹으로 쌓고 후보를 다시 고르는 기술은 제12장, 검색으로 답할 수 없는 질문을 구조로 받는 장치는 제13장, 원천이 텍스트가 아닐 때 무엇을 잃는지는 제14장이 받는다.

11.1 외부 자료의 사용

제3부는 모델이 컨텍스트 밖에 닿는 두 길을 세웠다. 도구가 첫째 길이요, 그 도구를 공급하는 프로토콜이 둘째 길이였다. 셋째 길이 여기서 온다. 지식이다.

셋은 성질이 다르다. 도구는 무언가를 하게 하고, 지식은 무언가를 알게 한다. 파일을 고치는 도구는 세계를 바꾸고, 돌아오는 결과는 바뀐 세계의 보고다. 규정 문서를 가져오는 일은 세계를 바꾸지 않는다. 컨텍스트에 문장이 하나 더 실릴 뿐이다.

그래서 묻는 것이 다르다. 도구에서는 이 행동이 되돌릴 수 있는지를 물었다. 지식에서는 이 문장이 언제 것이고 어디서 왔는지를 묻는다.

모델이 모르는 것부터 좁힌다. 셋이다. 훈련이 닫힌 뒤에 일어난 일. 조직 안에서만 도는 일, 곧 사유와 코드와 회의록과 장애 이력. 그리고 지금 막 정해진 일, 이를테면 오늘 아침 회의에서 바꾸기로 한 배포 절차. 셋 다 모델 안에 실릴 방법이 없다.

문제는 모델이 가진 지식의 양이 아니라 성질이다. 두 질문을 나란히 놓아 본다.「물은 몇 도에서 끓는가」와「이 결계 연동 라이브러리의 최신 판에서 그 함수는 어떤 인자를 받는가」. 모델은 두 질문에 같은 유창함, 같은 확신으로 답한다. 그러나 첫째 답은 십 년 전에도 참이었고 십 년 뒤에도 참일 것이다. 둘째 답은 훈련이 닫힌 날의 것이고, 그 뒤로 몇 판이 지났을지 모른다. 어제 바뀐 사실과 십 년 전 사실이 모델 안에서는 같은 자리에, 같은 확신으로 실려 나온다.

곤란은 셋으로 갈린다.

첫째, 시점이 뭉개져 있다. 지식마다 언제 것이라는 날짜가 붙어 있지 않다. 훈련이 닫힌 시점 하나가 전체에 일괄로 적용된다. 그 뒤에 바뀐 것은 없는 것이 아니라 옛 값으로 있고, 옛 값은 부채와 달리 질문을 멈춰 세우지 않는다.

둘째, 한 항목만 고칠 수 없다. 모델 안에서 사실 하나만 골라 고치는 방법은 없다. 고침의 단위가 항목이 아니라 훈련 전체라서, 규정 한 줄이 바뀌었을 때 그것을 모델에게 알려 주는 경로가 없다.

셋째, 출처가 없다. 답이 어디서 왔는지 물을 자리가 없다. 검증하려는 쪽이 잡을 실마리도, 틀렸을 때 고치러 갈 주소도 없다.

이 곤란은 검색 증강이라는 이름이 생기던 때에 이미 문제로 적혀 있었다. Lewis et al. (2020)는 크게 사전학습된 언어 모델이 사실적 지식을 안에 저장하지만 그 지식에 접근하고 정밀하게 다루는 능력에는 한계가 있다는 데서 출발한다. 그리고 두 가지를 미해결로 명시했다. 결정에 대한 출처를 제시하는 것, 그리고 세계 지식을 갱신하는 것이다. 곧 출처와 갱신은 정확도를 좇다 나중에 발견된 부작용이 아니다. 이 장치의 출발점에서부터 정확도와 나란히 세워진 별개의 문제였다.

여기에 미해결이 하나 있다. 위의 서술은 모델이 아는 것과 모르는 것을 가릴 수 있는 것처럼 말했지만, 그 경계의 목록은 존재하지 않는다. 무엇을 아는지는 물어보아야 알 수 있고, 같은 것을 묻는 두 표현이 다른 답을 끌어내기도 한다. 그래서 「모르는 것을 검색으로 채운다」는 처방은 모르는 것의 목록을 전제하는데, 그 목록은 주어지지 않는다.

실무의 우회는 목록 만들기를 포기하는 것이다. 아는지 모르는지를 묻는 대신, 낱을 수 있거나 검증이 필요한 부류의 질문은 모델이 아는 것처럼 보여도 밖의 지식을 쓰게 한다.

모델을 바꾸는 날의 사정도 여기서 다시 계산해야 한다. 지식의 축에서 보면 모델 교체는 백과사전이 통째로 갈리는 일이다. 어느 항목이 어떻게 바뀌었는지의 목록은 이번에도 없어서, 모델 안의 지식에 기대어 서 있던 답들은 교체의 날 소리 없이 함께 바뀐다. 반면 저장소에 둔 문서는 모델이 갈려도 그대로다. 지식을 밖으로 옮기는 일은 값과 품질의 결정이기 전에, 모델 교체가 건드리는 면적을 줄이는 결정이다.

11.2 밖에 두면 무엇이 달라지는가

같은 지식을 모델 밖 저장소에 두고 필요할 때 컨텍스트로 들이면 세 가지가 달라진다.

고치는 값이 다르다. 규정이 바뀌면 문서를 고치고, 다음 검색이 새것을 가져온다. 고침의 단위가 훈련에서 문서 하나로 내려오고, 지식의 나이가 「훈련이 닫힌 날부터」에서 「문서가 마지막으로 고쳐진 날부터」로 내려온다. 저장소도 낡는다. 그러나 낡음이 보이고 고칠 수 있는 상태가 되었다는 것이 차이이다.

무엇을 근거로 답했는지 볼 수 있다. 검색으로 가져온 조각에는 주소가 있다. 출처 없는 시스템에서 틀린 답의 수선 대상은 모델뿐이다. 프롬프트를 다듬거나, 모델을 바꾸거나, 다시 훈련하는 것인데 셋 다 그 답 하나를 겨냥하지 못한다. 출처가 있으면 틀린 답에서 조각으로, 조각에서 문서로 거슬러 올라가는 길이 있고, 그 길의 끝에서 실패가 세 갈래로 갈린다. 조각 자체가 틀렸으면 문서를 고치면 된다. 조각은 옳은데 답이 그것을 쓰지 않았으면 문제는 답을 쓰는 쪽이다. 조각이 아예 없으면 문제는 찾는 쪽이다. 출처는 답에 붙는 장식이 아니라 실패의 원인을 찾기 시작하는 자리다.

넣고 빼는 일이 실행 시점의 결정이 된다. 모델 안에 든 지식은 언제나 거기 있다. 빼는 스위치가 없다. 저장소에 둔 지식은 이 실행에 실을지 말지를 매번 정할 수 있다. 권한이 다른 사용자에게 다른 문서만 보이게 하는 일, 실험을 위해 한 문서를 잠시 빼 보는 일, 과제에 따라 다른 묶음을 실는 일이 전부 이 축 위에 있다.

대가도 같은 자리에서 나온다. 찾는 단계가 하나 늘고, 그 단계가 실패할 수 있다. 모델 안에 든 지식은 부재라는 사건이 없다. 필요한 순간에 없을 수가 없기 때문이다. 저장소에 둔 지식은 찾아져야 쓰이고, 찾기는 어긋날 수 있다. 확실성을 팔아 갱신 가능성과 출처를 산 것이다.

지식이 놓일 자리는 셋이다

지식을 둘 곳은 셋뿐이다. 매 실행 언제나 실리는 프롬프트, 필요할 때만 실리는 저장소, 그리고 모델 안이다. 앞의 둘은 컨텍스트이고 셋째는 아니다.

표 11.1: 지식을 모델 밖에 둘 때 달라지는 세 축. 아래 두 행이 첫 행만큼 무겁다.

축	모델 안	저장소에 두면	에이전트에서 무거운 이유
고침	훈련이 닫힌 날로 일괄 고정. 바뀐 것은 옛 값으로 남는다	문서 단위로 갱신. 낡음이 보이고 고칠 수 있다	루프는 낡은 전제 위에서도 멈추지 않고 행동한다
근거	없다. 수선의 대상이 모델뿐이다	조각마다 주소가 있다. 실패가 세 갈래로 갈린다	틀린 답을 고칠 자리가 문서, 찾기, 쓰기로 좁혀진다
넣고 빼기	스위치가 없다. 언제나 실려 있다	실행마다 정한다. 사람과 과제에 따라 다른 묶음이 실린다	권한과 실험과 과제별 구성이 전부 이 축 위에 선다

표 11.2: 세 자리가 사는 것과 파는 것. 성능은 어느 열에도 없다.

자리	사는 것	파는 것
프롬프트	확실성. 필요한 순간에 없는 일이 일어나지 않는다	값. 매 스텝 실리고, 쓰지 않는 실행의 뭉까지 낸다
저장소	갱신 가능성과 출처. 문서 단위로 고쳐지고, 답에서 근거로 가는 주소가 있다	확실성. 찾아져야 쓰이고, 찾기와 쓰기가 둘 다 성립해야 한다
모델 안	컨텍스트. 크기가 값에 나타나지 않는다	갱신과 출처. 고침의 단위가 학습이고, 근거를 물을 자리가 없다

배치를 정할 때 가장 먼저 세우는 묶음은 대개 성능이다. 세 구성으로 만들어 견주고 잘 나온 쪽을 고른다. 그런데 코드 규약 하나를 놓고 그렇게 해 보면 대개 셋 다 규약을 지킨다. 결정이 서지 않는다.

질문을 바꾸면 답이 갈린다. 규약이 개정되는 날 무슨 일이 일어나는가. 프롬프트는 문장을 고치면 되고, 저장소는 문서를 고치면 된다. 모델 안에 넣었으면 아무 일도 일어나지 않는다. 고칠 자리가 없고, 다시 학습하기 전까지 옛 규약을 같은 확신으로 지킨다.

셋의 정확도가 완전히 같아도 결정은 사라지지 않는다. 성능은 이 결정 위에 얹혀 있는 값이지 결정의 축이 아니다. 성능으로 고른 배치는 같은 잘못을 되풀이하게 만든다. 모델 안에 넣었다 하자. 몇 달 뒤 규약이 개정되어도 시스템은 옛 규약을 지키는 코드를 예전과 같은 품질로 계속 내놓고, 평가하는 것이 성능뿐이므로 지표에는 아무것도 걸리지 않는다.

무엇을 사야 하는지는 대개 배치를 정하는 날에 알 수 없으므로 순서가 하나 붙는다. 무르는 값이 짠 자리부터 쓴다. 프롬프트는 문장 하나를 고치면 되고, 저장소는 문서 하나에서 코퍼스 전체까지 벌어지며, 모델 안은 다시 학습하는 값이다. Hu et al. (2022)가 보고한 것은 고치는 값이 줄었다는 사실이지 무르는 값이 줄었다는 사실이 아니다. 모델 안에 넣은 것은 걷어낼 수 없다.

11.3 검색이라는 단계

밖에 둔 지식은 저절로 쓰이지 않는다. 필요한 순간에 필요한 부분이 컨텍스트로 들어와야 하고, 그 들임이 검색이다.

정의 11.1 (검색 증강 생성). 지식을 모델 밖 저장소에 두고, 실행 시점에 그 가운데 과제와 관련된 조각만 골라 컨텍스트에 싣는다. 모델은 그 컨텍스트 위에서 답을 생성한다. 이 구성을 검색 증강 생성이라 한다.

한 번의 검색이 하는 일은 네 걸음이다. 질의를 만든다. 후보를 찾는다. 몇 개를 고른다. 컨텍스트에 넣는다. 걸음마다 정해야 하는 것이 있다.

- **질의를 만든다.** 사용자의 말을 그대로 쓸 것인가, 모델이 다시 쓰게 할 것인가, 여러 개로 늘릴 것인가. 루프에서는 질의를 만드는 것도 모델이라 이 걸음도 매번 달라진다.
- **후보를 찾는다.** 글자로 찾을 것인가 뜻으로 찾을 것인가, 겹칠 것인가. 다음 질이 이 걸음을 연다.
- **몇 개를 고른다.** 상위 몇 개를 실을 것인가. 점수가 낮으면 빼는 문턱을 둘 것인가. 공식 지침이 보고한 방향은 뚜렷하다. 상위 다섯 개보다 스무 개를 넘기는 쪽이 나았다(Anthropic, 2024b). 다만 후보를 늘리면 무관한 것도 함께 실리므로 이 눈금은 과제마다 다시 측정해야 한다.
- **컨텍스트에 넣는다.** 어느 자리에 실을 것인가. 검색된 조각이 실리는 자리는 대개 앞자리도 최근 이력도 아닌 그 사이 어딘가인데, 그 자리가 가장 덜 읽히는 자리다.

그 앞에 물음이 하나 더 있다. 애초에 검색이 필요한가. 문서 집합 전체가 컨텍스트에 들어갈 만큼 작으면 통째로 실는 편이 낫다. 앞자리에 실린 것은 캐시가 받아 주므로 반복 호출에서 값이 크게 내려가고, 찾기라는 단계가 통째로 없어져 못 찾는 실패도 없어진다. 공식 지침이 드는 눈금은 20만 토큰, 곧 500쪽 남짓이다(Anthropic, 2024b). 이 눈금은 컨텍스트가 커지면 함께 올라가므로 가져갈 것은 숫자가 아니라 물음이다. 내 문서가 예산 안에 들어오는가.

검색은 도구의 한 종류다

누가 검색을 부르는가에 따라 구성이 둘로 갈린다.

하네스가 미리 찾아 넣는다. 질문이 도착하면 하네스가 먼저 검색을 돌리고, 결과를 컨텍스트에 얹은 다음 모델을 부른다. 모델은 검색이 있었다는 것도 모른다. 왕복이 한 번이라 빠르고, 무엇이 실렸는지가 로그에 그대로 남는다. 대신 모델이 결과가 나쁘다고 판단해도 다시 찾을 길이 없다.

모델이 검색 도구를 부른다. 검색이 도구 목록에 하나로 서 있고, 모델이 필요할 때 질의를 지어 부른다. 결과를 보고 질의를 바꾸어 다시 부를 수도 있다. 대신 왕복이 늘고, 부르지 않기로 하는 실패가 새로 생긴다.

어느 구성이든 이 절에 필요한 성질은 하나다. 검색이 돌려주는 것은 답이 아니라 후보다. 질의를 닮은 조각들이 순서를 달고 늘어선 목록이다. 그 순서는 관련이 있다는 판단 자체가 아니라 그 판단을 훑내 낸 것이다. 질의는 필요의 표현이고 표현은 필요와 어긋날 수 있으며, 닮았다는 것과 관련이 있다는 것은 다르다. 그 어긋남이 이 장의 중심이고, 절 11.4이 가까움이 관련성인지 묻는 데서 그것을 연다.

11.4 청킹, 임베딩, 벡터 검색

앞 절은 검색이 무엇을 하는지까지 세웠다. 여기서는 그 안을 연다. 가장 널리 쓰이는 한 갈래를 따라가면서, 그것이 무엇을 사고 무엇을 파는지 적는다.

왜 자르는가

200쪽짜리 사규를 저장소에 넣었다. 출장 결제 한도를 묻는 질문 하나에 200쪽을 다 실을 수는 없다. 그래서 문서를 잘라 둔다. 이 일을 자르기라 부르고, 영어로는 청킹이라 한다. 이유는 둘이며 둘이 다르다.

첫째 이유는 컨텍스트의 예산이고 익숙한 쪽이다. 길면 나빠지고 섞이면 나빠지므로 문서 전부를 실는 설계는 이미 기각되었다. 그런데 예산만이 이유라면 자르기는 실을 때의 문제이지 저장할 때의 문제가 아니다. 문서를 통째로 저장해 두고 찾은 뒤에 필요한 부분만 오려 실으면 되기 때문이다.

자르기가 저장하는 날의 결정이 되는 이유는 둘째에 있다. 뒤에 볼 벡터 검색은 조각 하나를 표현 하나로 바꾸어 두고 그 표현으로 찾는데, 조각이 크든 작든 표현의 크기는 같다. 한 문장도 표현 하나가 되고 100쪽짜리 문서도 표현 하나가 된다.

같은 크기의 그릇에 담기는 내용이 다르니, 긴 글의 표현은 그 글이 말하는 여러 가지를 뭉개 평균 같은 것이 된다. 사보 한 부를 통째로 표현 하나로 만들었다고 하자. 인사 발령과 보안 규정 개정과 구내식당 공지가 한 점에 뭉개진다. 보안 규정을 묻는 질의에 그 점은 어중간하게 가깝고, 보안 이야기만 적힌 짧은 쪽지가 있다면 그쪽이 훨씬 가깝다. 규정 개정의 자세한 내용은 사보 쪽에 적혀 있어도 그렇다.

곧 자르기는 실을 수 있는 크기로 줄이는 일이기 전에 찾을 수 있는 또렷함을 지키는 일이다. 컨텍스트가 큰 모델을 들인 날 자르기를 그만두고 싶어지지만, 그날 사라지는 것은 실기의 제약이지 뭉개짐이 아니다.

잘라 둔 것의 실물은 이렇게 생겼다.

[조각 4172] 출처 사유/경비.md 4장 3절 2026-07-01 개정

출장 결제는 사전 승인 없이 오백만 원까지 가능하다.

초과분은 부서장 승인을 받는다.

조각 하나에 본문과 주소가 함께 들어 있다. 주소가 앞 절이 말한 출처이고, 답이 틀렸을 때 거슬러 올라갈 길이다. 그런데 바로 다음 조각이 문제를 만든다.

[조각 4173] 출처 사유/경비.md 4장 3절 2026-07-01 개정

다만 해외 출장에는 위 조항을 적용하지 않는다.

「해외 출장 결제 한도」라는 질의에 앞 조각은 가깝다. 낱말이 겹치고 주제가 같다. 뒤 조각은 「해외 출장」을 담고 있으나 짧고, 「위 조항」이 무엇인지가 조각 안에 없어 질의와 그리 닮지 않았다. 앞 조각만 실리면 답은 오백만 원이 되고, 그 답은 문서의 절반 위에서 있다.

이 실패가 성가신 것은 실린 조각에 흠이 없다는 점이다. 관련 있고, 내용도 문서에 적힌 그대로다. 다만 문서가 그 문장에 달아 둔 예외가 경계 밖에 남았다. 옳은 것의 반쪽이 온전한 형태로 실린다.

어디서 자를 것인가에는 길이 둘 있다.

크기로 자른다. 몇백 토큰마다 기계적으로 끊는다. 어떤 문서에도 적용되고 구현이 몇 줄이다. 대신 경계가 문장 한가운데나 표 중간에 떨어진다.

구조로 자른다. 저자가 그은 경계를 빌린다. 문단은 저자가 한 덩어리로 읽히기를 바란 단위다. 절의 제목은 여기서부터 화제가 바뀐다는 표지이고, 표는 함께 있어야 하는 값들의 묶음이다. 저자는 문단을 닫을 때 대명사의 주인을 같은 문단 안에 두는 경향이 있고, 그 경향만큼 조각은 혼자서도 읽힌다. 대신 대화 기록이나 이어 붙은 로그처럼 저자가 경계를 긋지 않은 텍스트 앞에서는 빌릴 것이 없고, 문단 하나가 열 쪽인 문서 앞에서는 크기 문제가 그대로 남는다.

겹쳐 자르기가 혼한 것은 값이 거의 들지 않기 때문이다. 앞 조각의 꼬리를 뒤 조각의 머리에도 실는다. 구현은 잘린 자리를 몇 문장 되돌리는 것이 전부이고, 경계 근처의 문장이 두 조각에 담기므로 한쪽에서 놓쳐도 다른 쪽에서 걸린다. 대가는 셋이다. 저장이 겹친 만큼 늘고, 같은 문장이 후보 두 자리를 차지하며, 무엇보다 경계는 사라지지 않고 옮겨질 뿐이다. 겹침의 폭보다 멀리 떨어진 규정과 예외는 여전히 갈라진다.

값을 더 치르면서 사는 길이 하나 더 있다. 조각마다 맥락을 적어 붙이는 것이다. 색인을 만들 때 원문 전체를 본 모델이 조각마다 짧은 설명을 지어 앞에 붙이고, 붙인 채로 표현으로 옮긴다(Anthropic, 2024b). 앞의 조각 4173은 이렇게 된다.

[조각 4173] 출처 사유/경비.md 4장 3절 2026-07-01 개정

불인 맥락 이 조각은 출장 결제 한도 규정의 예외 조항이다.

다만 해외 출장에는 위 조항을 적용하지 않는다.

「위 조항」이 무엇인지 몰라도 이 조각은 해외 출장 한도라는 질의에 닿는다. 공식 지침이 보고한 값은 조각마다 몇십 토큰짜리 설명이고, 그것만으로 검색이 놓치는 비율이 내려갔다(Anthropic, 2024b).

표 11.3: 조각 크기의 양 극단. 실제 선택은 두 열 사이 어딘가에 선다.

	잘게 자른다 (극단: 한 문장)	크게 자른다 (극단: 문서 전체)
표현	또렷하다. 한 가지만 말하므로 뭉개질 것이 없다	뭉개진다. 여러 주제의 평균이 되어 어느 질의와도 어중간하게 가깝다
실리는 것	맥락을 잃은 반쪽. 대명사, 생략, 참조가 경계에서 끊긴다	온전하지만 대부분 무관물. 실은 순간 컨텍스트가 흐려진다
실패의 모습	답이 여러 조각에 걸리고, 걸렸다는 사실이 보이지 않는다	검색 자체가 흐려지고, 실린 뒤에는 답이 무관물에 끌려간다

값은 문서 수에 비례하는 모델 호출이다. 원문을 캐시에 없으면 크게 내려가지만 0이 되지는 않는다. 그리고 불인 맥락은 모델이 지어낸 글이라 틀릴 수 있다. 틀린 맥락은 조각을 엉뚱한 자리로 옮겨 놓고, 그 잘못된 원문 어디에도 없으므로 원본과 대조해도 보이지 않는다.

조각의 크기와 검색의 폭에 정답은 없다. 어디에 설지는 어떤 질문이 오느냐에 달렸고, 얼마나 좁은 사실을 묻는 질문이 얼마나 오는지가 그것을 정한다. 그런데 자르기는 질문이 도착하기 전에 해야 한다. 기법이 아니라 순서의 문제라서 어떤 자르기 기법도 이 사정 자체는 풀지 못한다.

경계가 만든 실패가 매번 되풀이된다는 것도 무겁다. 경계는 저장하는 날 한 번 그어지고 질문마다 다시 그어지지 않는다. 해외 출장의 한도를 묻는 질문은 올 때마다 예외 없는 오백만 원을 받는다. 그리고 이 실패는 수선의 후보에 잘 오르지 않는다. 검색은 관련 있는 조각을 돌려주었고 답은 그 조각에 충실하니, 둘 다 문제없어 보이기 때문이다.

뜻으로 찾기

자른 것을 어떻게 찾는가. 가장 오래된 방법은 낱말로 찾는 것이다. 질의의 낱말이 문서에 그대로 나타나는가를 본다. 곤란은 하나다. 문서는「연차」라 적었는데 질의는「휴가」라 묻고, 문서는「타임아웃 값」이라 적었는데 질의는「응답이 없다」고 묻는다. 낱말로 찾는 한 같은 뜻의 다른 표현은 남남이다.

원하는 것은 분명하다. 같은 뜻을 담은 두 문장이 서로 가까운 자리에 놓이는 지도가 있으면 된다. 그러면「휴가」로 물어도「연차」가 적힌 조각이 그 근처에 있다.

이런 지도는 낯선 물건이 아니다. 서점에서 제목을 모르는 책을 찾을 때 사람은 서가를 짚어 간다. 요리책 옆에는 요리책이 있고, 그 옆 칸에 식재료 도감이 있다. 꽃은 사람이 그렇게 꽃았기 때문이다. 짚어 가다 보면 찾던 책이 아니어도 찾던 것에 가까운 책이 손에 잡힌다.

글에 대해 그런 자리를 만들어 두는 일에 이름이 붙어 있다.

정의 11.2 (임베딩과 벡터 검색). 텍스트 조각을 고정된 크기의 수 목록으로 옮기되, 같은 뜻을 담은 서로 다른 표현이 가까운 값으로 옮겨지도록 학습된 옮김을 임베딩이라 한다. 문서의 조각들을 임베딩으로 옮겨 저장해 두고, 질의를 옮긴 값과 가까운 조각을 찾는 검색을 벡터 검색이라 한다.

한 번 돌려 보면 오가는 것이 무엇인지 보인다.

질의 해외 출장 결제 한도

받아 온 것 (상위 3)

- 0.83 [4172] 출장 결제는 사전 승인 없이 오백만 원까지...
- 0.79 [0918] 해외 출장 항공권은 이코노미를 원칙으로 한다...
- 0.61 [4173] 다만 해외 출장에는 위 조항을 적용하지 않는다

조각마다 수가 하나씩 붙어 있고 그 수의 순서로 늘어서 있다. 이 수가 가까움이다.

힘이 실려야 할 낱말은 정의의 「학습된」이다. 무엇과 무엇이 가까운지는 누가 규칙으로 적은 것이 아니라 보기로 가르친 것이다. 가까움을 측정하는 방법도 저절로 정해지지 않는다. 어느 것을 쓸지는 표현과 짝을 이루어 저장하는 낱 정해진다.

그리고 그 수는 읽을 수 없다. 0.83이 왜 0.79보다 큰지 물을 자리가 없다. 옮겨진 수 하나하나에 뜻이 없기 때문이다. 가까움은 설명 없이 수 하나로 주어지는 판정이다. 낱말 검색에서는 어느 낱말이 겹쳐 걸렸는지가 언제나 보였다는 것과 대비된다.

이 방식이 실제로 통한다는 것은 유도가 아니라 실증의 문제이고, 근거가 있다. Karpukhin et al. (2020)는 낱말 검색이 사실상의 표준이던 자리에서 뜻으로 찾는 방법만으로 그 기준선을 넘어섰다. 여기서 가져오는 것은 둘이다. 질의와 문서를 각각 옮겨 건준다는 구조, 그리고 그 구조가 통한다는 사실.

여기가 가장 정직해야 할 자리다. 왜 통하는가. 같은 뜻의 다른 표현이 가까워지도록 학습했더니, 그 가까움이 학습에 쓰인 보기를 넘어 처음 보는 질의와 문서에서도 사람의 판단과 대체로 맞더라는 것. 이것이 말할 수 있는 전부다.

대가도 관찰의 형태로만 말할 수 있다. 표현은 학습이 본 세계의 사본이므로, 학습이 드물게 본 문자열에서는 가까움이 제 노릇을 못 한다. 오류 코드, 사내의 식별자, 제품의 코드명이 그 자리다. 공식 지침의 예가 정확하다. 특정 오류 코드를 그대로 물으면 벡터 검색은 오류 코드 일반을 다루는 글을 가져오고, 정작 그 코드가 적힌 자리는 놓칠 수 있다. 낱말 검색은 그 문자열을 그대로 찾는다 (Anthropic, 2024b). 그래서 실무는 낱말 검색을 버리지 않고 겹쳐 쓴다. 겹치는 방법은 제12장의 소재다.

표현의 선택에는 청구서 하나가 붙어 있고, 정하기 전에 읽어야 한다. 다시 저장하는 값이다. 서로 다른 임베딩이 만드는 지도는 서로 다른 지도라서, 옛 표현으로 옮긴 값과 새 표현으로 옮긴 값은 건줄 수 없다. 표현을 바꾸는 날은 문서 전체를 새 표현으로 다시 옮기는 날이다. 자르는 방식을 바꾸는 날도 같은 값을 치른다. 조각이 달라지면 그 조각들의 표현도 다시 만들어야 하기 때문이다.

이 값이 「굳는다」의 실체다. 되돌리는 장치가 없어서 굳는 것이 아니라, 되돌림의 값이 문서 수에 비례해서 굳는다. 그리고 에이전트의 문서는 자기 기록이 쌓이며 계속 자라므로 그 값은 매일 오른다.

가까움이 관련성인가

앞의 목록을 다시 본다. 실제로 필요했던 조각은 셋째 자리의 4173이었다. 해외 출장에는 그 한도가 적용되지 않는다는 문장이다. 그런데 그것보다 위에 항공권 등급을 말하는 조각이 앉아 있다. 왜 이런 일이 일어나는지가 이 절의 물음이다.

검색이 실제로 원하는 것부터 정확히 한다. 질의가 있을 때 원하는 조각은 질의를 닮은 조각이 아니라 과제를 진전시키는 조각이다. 그런데 그 판정에는 두 가지가 필요하다. 질의와 조각을 함께 보아야 하고, 지금 무슨 일을 하고 있는지도 알아야 한다.

반면 벡터 검색이 측정하는 가까움은 두 값 사이의 거리다. 그 둘 중 하나인 조각의 값은 질의가 오기 전에, 저장하는 날, 조각 혼자서 계산된 것이다. 둘을 함께 보아야 하는 자리에서 홀로 미리 굳은 값 둘의 뒤늦은 만남을 측정하고 있는 셈이다.

그래서 어긋나는 자리가 있다. 셋을 짝는다.

첫째, 가까움은 방향을 보지 않는다. 「이 옵션은 운영 환경에서 켜야 한다」와 「이 옵션은 운영 환경에서 켜면 안 된다」는 낱말이 거의 같고 주제가 정확히 같다. 지도에서 두 문장은 이웃이다. 그러나 어느 쪽이 실리는가에 따라 에이전트의 행동은 반대가 된다. 부정 하나, 예외 한 마디가 가르는 차이는 문장 전체의 답음 속에서 작은 차이이다. 그리고 가까움은 작은 차이를 작게 측정한다. 위의 목록에서 4173이 밀린 것도 이 형태다.

둘째, 주제가 같다고 답이 있는 것은 아니다. 「왜 배포가 자주 실패하는가」라는 질의에 가장 가까운 조각들은 배포 실패를 이야기하는 조각들이다. 배포가 또 실패했다고 적힌 회의록, 실패를 보고한 장에 티켓, 같은 질문을 하고 답을 받지 못한 채 닫힌 스레드. 전부 질의와 주제가 같아서 가깝고, 전부

표 11.4: 가까움이 어긋나는 세 자리. 셋 다 검색은 성공의 형태로 돌아온다.

자리	가까움이 보는 것	실제로 필요한 것	뒤에서 일어나는 일
답았는데 반대	같은 낱말, 같은 주제	방향. 긍정과 부정, 규정과 예외의 구분	반대 처방이 근거로 실리고, 행동이 뒤집힌다
주제만 같다	질의와 같은 것을 이야기한 다는 사실	답, 곧 과제를 진전시키는 내용	한탄과 미해결이 상위를 채우고, 답이 든 조각이 밀려난다
질문과 답이 다르다	질문을 닮은 것, 곧 또 다른 질문	질문을 닮지 않은 답	답 없는 질문 스프레드가 답 있는 문서를 이긴다

답이 없다. 정작 원인이 적힌 조각, 이를테면 실정 값 하나를 무미건조하게 서술한 문서는 이들보다 멀 수 있다.

셋째, 질문과 답은 닮지 않는다. 「왜 응답이 느려지는가」의 답은 「연결 풀의 기본 크기가 10이다」 같은 문장이다. 질문은 결핍을 말하고 답은 사실을 말하며, 좋은 답일수록 질문을 되풀이하지 않는다. 두 문장이 지도에서 이웃이라는 보장은 어디서도 오지 않는다. 오히려 질의와 가장 닮은 것은 같은 질문이 적힌 다른 자리다.

핵심. 관련이 있는지는 질의와 과제가 도착한 뒤에야 판정되는데, 조각 쪽 절반은 질의가 오기 전에 홀로 굶는다. 가까움은 관련성을 흉내 낸 것이고, 어긋나는 자리는 기술의 미숙이 아니라 이 구조의 성질이다.

위의 셋이 어긋남의 전부라는 근거는 없다. 목록은 아마 더 길 것이다. 그리고 자기 시스템에서 이 어긋남이 얼마나 자주 일어나는지를 점수로 알 수는 없다. 0.83이라는 수는 검색이 자기 자를 대고 측정한 값이라 자기 성적표가 되지 못한다. 평가하려면 질의와 조각의 쌍마다 사람이 판정을 달아 둔 평가 집합이 있어야 하고, 그 설계는 제6부의 소재다. 여기서 말할 수 있는 것은 방향뿐이다. 점수의 축과 판정의 축은 다르다.

루프에서는 겹이 하나 더 얹힌다. 사람이 검색창에 묻는 자리에서 질의는 사람의 필요에서 나오지만, 루프에서 질의를 쓰는 것은 모델이다. 질의도 매번 달라지는 것이다. 그리고 이 실패도 되풀이된다. 같은 유형의 필요는 같은 지시문 아래에서 같은 표현의 질의를 만들고, 그 질의가 어긋남에 걸리면 그 유형 전체가 걸린다.

사람은 결과가 이상하면 낱말을 바꾸어 다시 묻는다. 이 회복이 루프 안에서 성립하려면 두 가지가 필요하다. 지금의 결과가 최선이 아닐 수 있다는 신호가 관측에 실려야 하고, 질의를 바꾸어 다시 찾는 것이 행동의 목록에 있어야 한다. 질의를 한 번 던지고 마는 워크플로에서 이 어긋남은 확정된 실패이고, 질의를 바꿀 수 있는 에이전트에서는 회복될 수 있는 사건이다.

실무에서 무엇이 쓰이는가

여기까지가 이 갈래의 뼈대다. 실제로 돌아가는 시스템에는 값을 짚는 결정이 하나 더 있고, 그 바깥에는 이 갈래를 아예 쓰지 않는 구성이 있다.

가까운 조각을 찾는 가장 정직한 방법은 전부 측정해 보는 것이다. 질의의 값을 모든 조각의 값과 하나씩 견주어 가장 가까운 몇을 고른다. 이 방법이 정확한 이유와 비싼 이유는 같다. 아무것도 건너뛰지 않기 때문이다.

그래서 실무의 벡터 검색은 다른 선택을 한다. 저장하는 낱 값들을 미리 정리해 두어, 질의가 왔을 때 전부가 아니라 유망한 동네만 뒤진다. 정리하는 방식은 여럿이고 그 세부는 여기의 경계 밖이지만,

표 11.5: 지식을 찾아 오는 세 구성. 어느 것도 기본값이 아니다.

구성	사는 것	파는 것
통째로 실기	못 찾는 실패가 없다. 캐시가 값을 받아 준다	문서가 예산을 넘는 순간 쓸 수 없다
자르고 저장하기	속도와 규모. 문서가 아무리 많아도 값이 질의마다 일정하다	저장하는 날 결정이 굳고, 사본이 낡고, 농침이 조용하다
도구로 열기	최신성. 사본이 없으므로 낡지 않고, 굳는 결정도 없다	속도. 왕복이 늘고, 열 자리를 모델이 정하므로 안 여는 실패가 생긴다

어떤 방식이든 사는 것과 파는 것은 같다.

사는 것은 속도다. 건준의 수가 전체에서 일부로 내려온다. 파는 것은 보장이다. 진짜 가장 가까운 조각이 건너편 동네에 있었다면 찾지 못하고, 건너편 곳은 보지 않았으므로 놓쳤다는 것도 모른다. 「가장 가까운 것을 반드시 찾는다」가 「대체로 찾는다」로 내려온다.

이 포기가 어떤 실패로 나타나는가가 요점이다. 농침은 조용하다. 가장 가까운 조각을 건너편 질 의에서도 목록은 채워져 온다. 문서에 있었고 표현도 질의와 가까웠는데 찾는 마지막 걸음이 건너편 것이라, 앞선 두 결정을 아무리 다듬어도 닿지 않는다. 놓쳤다는 것을 알려면 전부 측정해 본 결과와 견주어야 하는데, 전부 측정하는 값을 피하려고 이 방식을 들었으므로 그 기준선은 일상의 실행에 없다. 표본 질의 몇에 대해 이따금 전부 측정해 보는 것이 이 실패를 보는 유일한 길이다.

이 갈래를 쓰지 않는 구성

여기까지 따라온 갈래는 여전히 가장 널리 쓰이지만 유일한 길이 아니고, 최근에는 밀리는 자리가 생겼다. 둘이 겹쳤다.

하나는 컨텍스트가 커진 것이다. 문서가 예산 안에 들어오면 자르지도 저장하지도 않고 통째로 실는 쪽이 낫다는 논금을 앞 절에서 세웠다. 그 논금이 올라가는 만큼 이 갈래를 열지 않아도 되는 자리가 늘어난다.

다른 하나는 에이전트가 도구로 직접 파일을 여는 구성이다. 미리 전부 훑어 저장하는 대신, 파일 경로나 링크 같은 가벼운 표지만 들고 다니다 필요해진 것만 도구로 연다. 폴더의 짜임과 이름 규칙과 갱신 시각이 어디를 열지의 단서가 되고, 한 번 연 결과가 다음에 무엇을 열지를 정한다(Anthropic, 2025c). 제10장의 기억 도구가 이 형태였다. 세션이 처음 받는 것은 색인이 아니라 폴더의 목록이었다.

실무의 답은 대개 섞은 것이다. 빠르게 있어야 할 것은 미리 가져오고, 나머지는 모델의 재량으로 열게 한다(Anthropic, 2025c). 그러니 이 절을 「이것이 정답이다」로 읽으면 안 된다. 자르고 옮기고 가까운 것을 찾는 길은 가장 널리 쓰이는 한 갈래이고, 위 표의 가운데 행이 그 값이다.

11.5 검색이 실패했을 때

검색은 항상 성공하지 않는다. 실패의 모습은 셋이고, 셋 다 오류를 일으키지 않는다.

빈손이 두 가지다

검색이 아무것도 돌려주지 않았을 때 가능한 사정은 둘이다. 문서에 정말 없거나, 있는데 찾기가 어긋났거나.

표 11.6: 검색 실패의 세 형태. 어느 것도 오류를 일으키지 않는다.

형태	실제 사정	모델이 보는 것	뒤에서 일어나는 일
부재의 빈손	문서에 정말 없다	결과 없음	「없다」가 옳지만, 아래 행과 구분되지 않으므로 없다는 증명으로 쓸 수 없다
실패의 빈손	문서에 있는데 질의와 표현이 어긋나 못 찾았다	똑같은 「결과 없음」	「없다」는 거짓 결론이 전제로 굳고, 없다고 믿은 것을 새로 만들려 간다
그럴듯한 무관물	답이 없거나 못 찾았는데, 상위 후보는 언제나 채워진다	순위가 붙은 조각들. 성공과 같은 형태	가장 덜 무관한 것이 근거로 올라서고, 답이 그 위에 선다

곤란은 두 사정이 같은 관측으로 온다는 것이다. 「결과 없음」이라는 관측 어디에도 둘을 가릴 정보가 없고, 모델에게는 관측이 전부이므로 모델은 가르지 못한 채 하나를 고른다. 대개 고르는 쪽은 「없다」이고, 그 결론은 행동으로 이어진다. 그런 규정은 없다는 전제로 계획이 바뀌고, 없다고 결론 낸 것을 새로 만들려 가기도 한다.

더 흔하고 더 나쁜 쪽은 빈손이 아니다. 검색기는 대개 빈손으로 돌아오지 않는다. 상위 몇 개를 달라는 요청에는 문서에 답이 없어도 가장 덜 무관한 몇 개가 온다. 점수는 후보들 사이의 순위이지 「이만하면 관련 있다」는 판정이 아니기 때문이다.

그래서 실패한 검색은 성공의 형태로 온다. 조각들이 있고, 순서가 있고, 점수가 붙어 있다.

여기서 물음이 하나 선다. 못 찾았다는 것을 누가 아는가. 셋을 하나씩 지워 보면 그 자리가 비어 있다. 검색기가 아는 것은 후보들의 순위까지다. 자기가 돌려준 최선이 과제에 충분한지는 검색기가 답할 수 있는 물음이 아니다. 모델이 보는 것은 관측뿐이고, 관측은 없는 것과 못 찾은 것을 구분해 주지 않는다. 저장소는 무엇이 있는지 알지만 질의가 무엇을 필요로 했는지 모른다.

실패를 알아보는 데 필요한 두 정보, 무엇이 필요했는가와 무엇이 있는가가 시스템의 서로 다른 자리에 나뉘어 있다.

찾았는데 안 쓰이는 실패

셋째 실패는 찾기가 성공한 뒤에 온다. 검색의 설계는 대개 찾기에서 끝난다. 좋은 것을 찾아 넣었으면 일이 끝났다는 전제가 조용히 깔려 있다.

그 전제를 장면 하나로 시험한다. 앞의 조각 4172가 컨텍스트에 실려 있다. 출장 결제 한도가 오백만 원이라는 문장이다. 그리고 모델이 배운 세계에서 이런 규정의 흔한 값이 백만 원이었다고 하자.

모델의 답은 세 갈래 중 하나다. 조각을 그대로 쓴 「오백만 원」. 배운 통념을 쓴 「백만 원」. 그리고 가장 상가신 셋째, 둘을 섞은 「오백만 원, 다만 백만 원을 넘는 건은 별도 승인이 필요하다」.

셋째 답은 어느 문서에도 없다. 조각의 숫자가 들어 있으니 출처 표시까지 그럴듯하게 달리고, 확인하는 사람이 문서를 열어 보아도 절반은 맞으니 넘어가기 쉽다. 반쯤 쓰인 지식은 안 쓰인 지식보다 잡기 어렵다.

왜 이런 일이 가능한가. 조각은 규칙이 아니라 조건이기 때문이다. 컨텍스트에 들어간 텍스트는 답을 기울일 뿐 정하지 않는다. 지시에 대해 이미 세운 문장과 같은데, 지시에는 출구가 하나 있었다. 지켜지지 않으면 안 되는 형식을 스키마로 내려앉히는 길이다. 사실에는 그 출구가 없다. 「이 조각과 모순되게 답하지 말라」에서 검사되어야 하는 것은 형식이 아니라 뜻이고, 뜻은 코드가 대조하지 못한다.

핵심. 검색의 성공은 두 사건의 곱이다. 옳은 것을 찾아 넣는 것, 그리고 답이 그것을 쓰는 것. 앞의 확률은 검색의 품질로 올릴 수 있다. 뒤의 것을 강제하는 장치는 없다. 조각이 컨텍스트에 있다는 것은 답이 그 위에 섰다는 증거가 아니다.

컨텍스트의 새 것과 모델이 배운 옛 것이 충돌할 때 어느 쪽이 이기는지에는 일반화된 답이 없다. 어느 한 쪽이 항상 이기지는 않는다는 것까지가 말할 수 있는 전부다.

대신 확실한 것이 하나 있다. 검색은 갱신을 위해 쓰는 장치이므로 컨텍스트의 지식이 모델이 배운 지식과 다르기를 바라는 구성이다. 낡은 통념을 새 사실로 덮으려고 넣는 것이니, 두 지식의 충돌은 예외 상황이라 아니라 목적에 내장된 정상 상태다. 그리고 그 해소는 아무 강제 장치 없이 모델의 재량에 맡겨져 있다.

어떻게 줄이고 어떻게 보는가

셋을 줄이는 자리는 각각 다르다.

질의를 하나로 두지 않는다. 못 찾은 실패의 상당수는 질의와 문서의 낱말이 어긋난 것이다. 사용자의 말을 그대로 던지는 대신 모델이 질의를 여러 개로 늘리게 하면 어긋남이 전부 겹칠 확률이 내려간다. 값은 검색 횟수와 왕복이다.

찾는 방법을 겹친다. 뜻으로 찾는 길이 놓치는 것은 대개 글자로 찾는 길이 잡는다. 오류 코드와 사내 식별자가 그 자리였다. 겹치는 방법은 제12장이 받는다.

조각을 혼자서도 읽히게 만든다. 반쪽이 실리는 실패는 검색 시점에 고칠 수 없다. 저장하는 날 구조로 자르거나 겹쳐 자르거나 맥락을 붙여야 한다.

쓰였는지를 답에서 확인한다. 셋째 실패에는 강제 장치가 없으므로 남는 것은 확인뿐이다. 답의 문장마다 근거 조각의 주소를 달게 하고, 그 주소의 조각과 답을 대조한다.

그리고 마지막 물음이 남는다. 이 실패들이 보이는가. 못 찾은 것은 로그에 안 남는다. 로그에 남는 것은 검색이 무엇을 돌려주었는가고, 돌려주지 못한 것에는 줄이 없다.

그래서 검색의 실패를 볼 수 있는 사건으로 만드는 것이 하네스의 일이다. 빈 목록만 돌려주는 대신 검색이 자기 사정을 함께 보고하게 한다. 몇 건을 훑었고, 최고 점수가 얼마였고, 그 점수가 이 문서 집합에서 잘 맞았을 때 나오는 값과 견주어 어디쯤인지. 이런 신호가 관측에 실리면 모델은 「없다」와 「못 찾았다」를 가를 재료를 얻고, 질의를 바꾸어 다시 찾는 것이 선택지가 된다. 신호가 없으면 빈손은 없다는 증명으로 굳는다.

문턱은 미해결로 남는다. 점수가 얼마 아래면 조각을 신지 않고 빈손을 보고하는 쪽이 나은가. 문턱을 두면 그럴듯한 무관물이 걸러지는 대신 아슬아슬하게 관련된 조각도 함께 걸리고, 문턱을 안 두면 그 반대다.

셋째 실패는 보기가 한 겹 더 어렵다. 답이 틀려 로그를 열면 옳은 조각이 컨텍스트에 실려 있는 것이 보인다. 실려 있으므로 검색은 문제없다고 판정된다. 수선은 더 많이, 더 정확히, 더 길게 실는 쪽으로 흘러간다. 그러나 문제가 쓰기에 있다면 가져오기의 개선은 이 실패에 닿지 않고, 후보가 더 실리는 만큼 컨텍스트는 흐려진다. 컨텍스트에 있었는가가 아니라 답에 쓰였는가를 평가해야 한다.

11.6 검색으로 풀 수 없는 것

다음에 해당하면 검색으로 풀 때가 아니다. 자르기와 표현을 다듬는 일도 여기에는 닿지 않는다.

1. **답이 어느 조각에도 적혀 있지 않다.** 흩어진 것을 이어야 답이 되는 질문이다. 검색은 적혀 있는 것을 가져올 뿐 잊지 않는다. 그리고 자르기는 이 사정을 돕기는 커녕 악화시킨다. 잘게 자를수록 흩어짐은 늘기 때문이다. 이 질문을 구조로 받는 장치가 제13장이다.

2. **세어야 한다.** 계약이 몇 건인가, 이 조건에 걸리는 고객이 몇인가. 세려면 전부를 보아야 하는데 검색이 돌려주는 것은 상위 몇 개다. 빠진 것이 있는 채로 센 수는 셈이 아니고, 무엇보다 빠졌다는 표시가 목록 어디에도 없다.
3. **모델이 그 말을 몰라 질의조차 못 만든다.** 사내에서 새 기능을 「등대」라 부른다고 하자. 무엇을 찾아야 하는지 모르는 모델은 그 낱말을 질의에 쓰지 못하고, 쓰지 않은 낱말은 검색에 걸리지 않는다. 문서에 그것이 있어도 그렇다. 이 실패는 검색기 바깥에 있다. 조직의 어휘를 어딘가에 미리 적어 두어야 풀린다.
4. **사실이 아니라 방법이다.** 절차, 규범, 판단 기준은 조각으로 가져와 참조하는 것이 아니라 행동으로 지켜져야 한다. 늘 지켜져야 할 규범을 검색이 맞히기를 바라는 것은 위험한 내기다.
5. **적힌 곳이 없다.** 조직의 암묵지, 코드의 구조 속에만 있는 사실, 아무도 문서화하지 않은 결정. 검색의 상한은 문서의 상한이다. 적혀 있는 것만 찾아지며, 적는 일은 검색이 해 주지 않는다.
6. **세계가 색인보다 빨리 변한다.** 색인은 만든 시점의 사본이므로, 다시 저장하는 주기보다 빠르게 변하는 값, 곧 지금의 재고, 실행 중인 작업의 상태, 방금 고친 파일의 내용은 색인에서 찾을 것이 아니라 도구로 그 자리에서 읽을 것이다.

여섯을 관통하는 형태가 있다. 검색은 적혀 있는 것을 가져오는 장치다. 여섯 중 어느 것도 검색 품질을 올려서 풀리지 않는다. 제12장이 올려 주는 것은 적혀 있는 것을 찾을 확률이고, 이 목록은 그 확률이 1 이어도 남는다.

다섯째는 밖에서 주어지는 사정이 아니라 설계의 대상이다. 에이전트 시스템의 문서는 주어진 것만이 아니기 때문이다. 시스템이 스스로 만드는 기록, 곧 제10장의 기억이 미래 실행의 문서가 된다.

그렇다면 「적혀 있는가」는 운의 문제가 아니라 오늘의 기록 규율의 문제다. 요구가 둘이다. 오늘 남기는 결정 기록이 내일의 질의로 찾아질 낱말과 형태로 적혀 있는가. 그리고 잘라도 혼자서 읽히는 크기로 적혀 있는가. 결정 하나와 그 근거를 한 자리에 적는 것, 대명사 대신 이름을 부르는 것이 그 규율이다. 남이 쓴 글에는 경계를 짐작해 그을 수밖에 없지만 자기가 적는 기록에는 짐작할 필요가 없다. 적는 날에 만들 수 있기 때문이다.

11.7 요약

한 문장으로. 검색 증강은 지식의 범위만 넓히는 장치가 아니라 지식에 시점과 출처를 되돌려 주는 장치다. 그 힘은 컨텍스트에 넣는 데까지만 미치므로 설계의 중심은 잘 찾는 것 너머에 있다. 넣은 것이 쓰였는지를 물어야 하고, 못 찾았다는 것을 볼 수 있게 만들어야 한다.

확실해진 것.

- 도구는 무언가를 하게 하고 지식은 무언가를 알게 한다. 모델 안에 든 지식의 곤란은 셋이다. 시점이 뭉개져 있고, 한 항목만 고칠 수 없으며, 출처가 없다(절 11.1).
- 지식을 저장소에 두면 고치는 값과 근거와 넣고 빼기가 달라지고, 대신 확실성을 판다(표 11.1). 세 자리는 서로 다른 것을 사므로 성능으로 고를 수 없다(표 11.2).
- 한 번의 검색은 질의를 만들고, 후보를 찾고, 몇 개를 고르고, 컨텍스트에 넣는 네 걸음이다(정의 11.1). 문서가 작으면 통째로 신는 편이 낫고, 검색이 돌려주는 것은 답이 아니라 후보다.
- 자르는 이유는 둘이다. 컨텍스트의 예산, 그리고 조각이 크든 작든 표현의 크기가 같아 긴 글의 표현이 뭉개진다는 사정이다. 대가는 옳은 것의 반쪽이 온전한 형태로 실리는 것이고, 크기에 정답은 없다(표 11.3).
- 임베딩은 같은 뜻의 다른 표현이 가까워지도록 학습된 율김이다(정의 11.2). 가까움은 설명 없는 판정이다. 학습이 드물게 본 문자열에서는 듣지 않으므로 낱말 검색을 겹쳐 쓴다. 표현을 바꾸는 날은 문서 전체를 다시 율기는 날이다.
- 관련이 있는지는 질의와 과제가 도착한 뒤에야 판정되는데 조각의 표현은 그 전에 홀로 굳는다. 그래서 가까움은 흉내이고 어긋남은 구조의 성질이다. 방향을 보지 못하고, 주제가 같은 것을 답이 있는 것으로 잘못 측정한다(표 11.4).
- 자르고 저장하는 길은 가장 널리 쓰이는 한 갈래이지 기본값이 아니다. 통째로 신는 구성과 도구로 여는 구성이 나란히 있고, 셋이 파는 것이 다르다(표 11.5).
- 실패는 셋이다. 두 빈손은 같은 관측으로 오고, 무관물은 성공의 형태로 오며, 셋째는 찾기도 안 쓰이는 것이다(표 11.6). 성공은 찾기와 쓰기의 곱인데 뒤의 항을 강제하는 장치가 없고, 못 찾은 것은 로그에 남지 않는다.
- 검색으로 풀 수 없는 것은 여섯이다. 이어야 하는 답, 세어야 하는 물음, 물을 낱말을 모르는 경우, 사실 아닌 방법, 적히지 않은 지식, 색인보다 빠른 세계(절 11.6).

아직 답하지 않은 것.

- 두 찾기의 결합과 후보의 재정렬, 그리고 문턱의 설계는 제12장이 받는다.
- 이어야 하는 답과 세어야 하는 물음을 구조로 받는 장치는 제13장이 받는다.
- 원천이 텍스트가 아닐 때 무엇을 잃는지는 제14장이 받는다.
- 넣은 것이 쓰였는지를 평가하는 일과 판정이 달린 평가 집합의 설계는 제6부가 받는다.
- 검색된 조각이 지시로 읽히는 경로의 방어는 제22장이 받는다.

오늘 할 것. 자기 문서 집합의 크기를 토큰으로 측정한다. 컨텍스트에 통째로 들어가는 크기라면 검색을 건너낼 후보로 표시한다(절 11.3). 다음으로 어제 로그에서 검색이 돌아온 자리 열 개를 열어, 실린 조각이 답에 실제로 쓰였는지 하나씩 표시한다(절 11.5). 마지막으로 검색이 빈손으로 돌아온 횟수를 센다. 세어지지 않는다면 그것이 오늘의 결과다.

11.8 생각해 볼 문제

1. 찾았는데 안 쓰이는 실패를 자기 시스템에서 관측하는 실험을 설계하라. 같은 질문을 세 조건, 곧 검색 없이, 옳은 조각과 함께, 일부러 틀린 조각과 함께 물어 답을 건주면 컨텍스트가 이기는 경우와 모델이 배운 것이 이기는 경우의 경계를 더듬을 수 있다. 셋째 조건이 왜 필요한가.
2. 자기 문서에서 경계의 실패를 관측하는 실험을 설계하라. 답이 규정과 예외처럼 두 문단에 걸쳐 있는 질문의 목록을 만들고, 겹침의 폭만 바꾸어 가며 그 질문들을 던져 보라. 무엇을 지표로 삼아야 「반쪽의 실패」이 세어지는가.
3. 질문과 답이 닮지 않는다는 관찰을 확인하는 실험을 설계하라. 같은 필요에 대해 질의를 두 가지로 만든다. 하나는 질문 문장 그대로, 다른 하나는 답이 있을 법한 문서의 문장을 지어낸 것이다. 두 질의의 결과가 다르다면 그 차이는 무엇의 증거인가.
4. 자기 문서의 크기를 측정하고, 통째로 신는 설계와 자르고 저장하는 설계와 도구로 여는 설계의 값을 표 11.5의 세 행으로 건주어 보라. 문서가 자라 경계를 넘는 날을 무엇으로 알아차릴 것인가.

11.9 다시 읽을 때

- 제12장을 읽고 절 11.4이 가까움이 관련성인지 묻는 데를 다시 보면, 세 어긋남 각각을 어떤 장치가 겨누고 있는지가 보인다. 그때 확인할 것은 여기서 세운 문장이다. 그 장치들이 늘리는 것도 전부 찾기의 항이다.
- 제13장을 읽고 절 11.6의 앞 두 항목으로 돌아오면, 이어야 하는 답과 세어야 하는 물음이 장치들 얻는다. 자를수록 흠어진다는 관찰이, 애초에 조각이 아니라 관계를 저장했어야 하는 질문의 표지였다는 것이 거기서 보인다.
- 제10장을 다시 열고 절 11.6의 마지막 문단을 보면, 적는 쪽과 찾는 쪽이 같은 시스템이라는 관찰이 이미 기억이라는 설계 대상이 되어 있었다는 것이 보인다.
- 제22장을 읽고 절 11.5로 돌아오면, 넣기가 기울이기라는 문장이 방어의 언어로 다시 보인다. 문서에 적힌 것이 모델의 행동을 기울일 수 있다는 뜻이다. 문서가 밖에서 오는 시스템에서 그 기울임은 공격의 통로가 된다.

11.10 더 읽을거리

Lewis et al. (2020)는 검색 증강 생성이라는 이름의 출처다. 오늘의 실무는 이 논문이 제시한 방법을 거의 쓰지 않으므로 방법을 배우러 읽는 것은 길을 잘못 든 것이다. 가져갈 것은 문제의 규정이다. 출처와 갱신이 미해결로 명시되던 자리, 곧 이 장치가 정확도만이 아니라 검증 가능성과 갱신 가능성을 위해 세워졌다는 규정이 첫 쪽에 있다.

Karpukhin et al. (2020)에서 가져갈 것은 둘이다. 질의와 문서를 각각 옮겨 건준다는 구조, 그리고 낱말 검색이 사실상의 표준이던 자리에서 그 기준선을 넘어섰다는 사건이다. 학습 방법의 세부를 배우러 읽는 것은 아니다.

Anthropic (2024b)에서는 넷을 가져왔다. 문서가 작으면 통째로 신는 편이 낫다는 눈금, 조각으로 자르는 순간 조각이 제 맥락을 잃는다는 진단과 그 처방, 벡터 검색이 정확한 문자열 앞에서 듣지 않는다는 관찰, 그리고 후보를 몇 개 실을 것인가의 방향이다. 이 문헌은 수치를 함께 보고하는데, 가져갈 것은 수치가 아니라 방향이다.

Anthropic (2025c)은 미리 저장하지 않고 표지만 들고 다니다 도구로 여는 구성을 적어 둔 자리다. 폴더의 짜임과 이름 규칙과 갱신 시점이 신호가 된다는 관찰, 그리고 실행 중에 찾는 일이 미리 만들어 둔 것을 꺼내는 것보다 느리다는 대가가 함께 있다.

넣은 것이 쓰이지 않을 수 있다는 이 장의 중심은 새 실증이 아니라 제4장을 반대쪽에서 읽은 것이다. Shi et al. (2023)를 이 장의 눈으로 다시 읽으면, 무관한 것이 답을 끌어간다는 결과가 곧 컨텍스트는 기울일 뿐 정하지 않는다는 성질의 실증이다. Liu et al. (2024)이 보인 자리별 차이는 검색된 조각을 어디에 실을 것인가라는 배치의 질문으로 돌아온다.

하이브리드 검색

날말 검색은 오류 코드처럼 드문 문자열을 정확히 찾아내지만, 같은 뜻을 다른 낱말로 쓴 문서는 찾지 못한다. 벡터 검색은 그 반대다. 「연차」와 「휴가」를 이어 주지만, 사내 코드명처럼 학습이 보지 못한 문자열에는 듣지 않는다. 그리고 둘 다 조각을 자를 때 끊긴 앞뒤 맥락은 되돌려 주지 못한다.

그래서 실무는 찾기를 한 번에 끝내지 않는다. 세 가지를 더 한다. 두 검색을 겹쳐 한쪽이 놓치는 것을 다른 쪽이 잡게 하고, 싸게 좁힌 뒤 비싼 관정으로 다시 고르고, 잘리며 잃은 맥락은 저장하는 날 미리 되돌려 놓는다.

셋 다 겨는 것을 맞힌다. 공개된 측정 하나에서 검색이 옳은 조각을 놓치는 비율이 내려갔다 (Anthropic, 2024b). 그런데 같은 자료가 그 값을 낸 구성을 함께 적어 두었다. 조각마다 모델을 한 번씩 돌려 맥락을 적어 붙이고, 저장할 두 묶음 만들고, 질의마다 후보 150개를 받아 다시 세우고, 그중 20개를 싣는다. 산 것과 치른 것이 나란히 적혀 있다.

이 장은 그 순서로 간다. 왜 겹치는지, 실제로 어떻게 하는지, 그리고 그렇게 쌓았을 때 무엇을 치르는지다. 마지막으로 단계를 몇 개 쌓아도 안 되는 것으로 닫는다.

구현은 다루지 않는다. 다시 세우는 모델의 구조와 학습, 특정 제품의 이름은 실지 않는다. 검색으로 답할 수 없는 질문은 제13장으로 미룬다.

12.1 두 검색을 겹치는 이유

뿌리는 하나다. 벡터 검색이 쓰는 것은 학습된 지도이고, 학습은 세상의 글을 보았다. 사내 시스템이 지어 붙인 오류 코드는 그 세상에 없던 문자열이라 지도에 자리가 없다. 사내에서 새 기능을 「등대」라 부르는 경우도 뿌리가 같다. 이 낱말은 학습이 많이 보았으되 세상의 뜻으로 보았고, 코드명과 기능 사이의 다리는 조직의 관례이지 낱말의 뜻이 아니다.

날말 검색은 정확히 그 자리에서 강하다. 질의의 낱말이 문서에 그대로 있는가만 보므로, 드문 낱말일수록 좋은 질의다. 그리고 날말 검색이 놓치는 것은 이미 아는 그 실패다. 문서는 「연차」라 적었는데 질의는 「휴가」라 묻는 빈손이고, 벡터 검색은 바로 그것을 메우려고 온 장치였다.

서로 다른 것을 놓치니 겹치면 덜 놓친다. 겹침의 근거는 이 한 줄이 전부다.

한쪽을 다듬어 둘을 겹치게 만드는 길은 양쪽 다 막혀 있다. 벡터 검색이 사내 어휘를 배우려면 학습이 그것을 보아야 하는데 사내 어휘는 오늘도 새로 기재된다. 날말 검색에 동의어 사전을 달아주는 길은 벡터 검색이 학습으로 얻은 것을 사람 손으로 베끼는 일이라 끝이 없다. 그래서 실무는 겹치는 대신 겹친다.

여기서 이 장이 무엇을 늘리는지 못 박아 둔다. 제11장은 검색의 성공을 두 사건의 곱으로 세웠다. 옳은 것을 찾아 넣는 것, 그리고 답이 그것을 쓰는 것.

이 장의 세 장치는 전부 찾는 쪽만 손댄다. 겹쳐도, 다시 세워도, 맥락을 붙여도 오르는 것은 옳은 조각이 컨텍스트에 실릴 확률까지다. 실린 뒤에 그것이 쓰이는지는 그대로 남는다.

표 12.1: 두 검색의 분업. 아래 두 행이 서로 엇갈려 있다는 것이 겹침의 근거다.

	날말 검색	벡터 검색
측정하는 것	질의의 날말이 문서에 그대로 있는가. 드문 날말의 겹침일수록 무겁게 센다	학습된 지도에서 질의와 조각의 자리가 가까운가
잘 찾는 것	오류 코드, 코드명, 식별자, 규정 번호. 그 날말이 적힌 바로 그 문서	「연차」를「휴가」로 몰어도 닿는, 표현이 다른 같은 뜻
놓치는 것	같은 것을 다른 날말로 부르는 질의와 문서	학습 밖에서 태어난 어휘. 지도에 자리가 없는 문자열

12.2 두 목록을 하나로

두 검색을 각각 돌려 두 목록을 얻고 합치는 구성을 하이브리드 검색이라 하고, 오늘의 권장 구성이다(Anthropic, 2024b).

역사의 순서 때문에 오해를 사기 쉽다. 날말 쪽이 오래 표준이던 자리를 벡터 검색이 뒤늦게 넘겨받았으니 이긴 쪽만 남기면 될 것 같다. 그러나 새 검색은 옛 검색의 실패를 메우면서 옛 검색에 없던 실패를 새로 들여왔다. 겹침은 미련이 아니라 분업이다.

실물로 본다. 「해외 출장 결제 한도」를 묻는다.

뜻으로 찾은 목록

- 1위 출장 결제는 사전 승인 없이 오백만 원까지 가능하다
- 2위 해외 출장 항공권은 이코노미를 원칙으로 한다
- 3위 다만 해외 출장에는 위 조항을 적용하지 않는다

글자로 찾은 목록

- 1위 다만 해외 출장에는 위 조항을 적용하지 않는다
- 2위 해외 출장 결제 한도 변경은 다음 절차를 따른다
- 3위 출장 결제는 사전 승인 없이 오백만 원까지 가능하다

우리가 실제로 원한 것은 예외 조항이다. 해외 출장에는 그 한도가 적용되지 않는다는 문장이다. 그것이 한쪽에서는 3위이고 다른 쪽에서는 1위다.

왜 갈랐는지 물어 둔다. 그 조각은 한문장이고 「위 조항」이 무엇인지 조각 안에 없다. 뜻으로 찾는 쪽에서는 그래서 밀렸다. 글자로 찾는 쪽에서는 「해외」와 「출장」이 짧은 문장 안에 다 들어 있어 1위다.

이제 둘을 합쳐야 하는데, 점수를 더할 수는 없다. 한쪽 점수는 날말이 몇 번 겹쳤는지이고 다른 쪽 점수는 지도 위에서 얼마나 가까운지다. 측정하는 것이 다르니 몸무게와 체온을 더하는 셈이다.

그래서 널리 쓰이는 방법은 점수를 버리고 등수만 쓴다. 1위에게 큰 몫을 주고 아래로 갈수록 조금씩 줄인다. 다만 줄어드는 폭을 완만하게 둔다. 1위와 3위의 차이를 크게 두지 않는다는 뜻이다.

그러면 어떻게 되는가. 한 목록에서만 1위인 조각은 몫을 한 번 받는다. 두 목록에 다 든 조각은 두 번 받는다. 폭이 완만하니 한 번 크게 받는 것보다 두 번 나눠 받는 쪽이 이긴다.

위의 예외 조항이 그 경우다. 3위와 1위를 하나씩 받아 합계에서 맨 위로 올라온다. 벡터 검색만 쓰던 구성에서 이 조각은 상위 둘 안에 든 적이 없었고, 그것이 제11장에서 오백만 원이라는 반쪽 답이 나온 경로였다.

남는 결정이 하나 있다. 두 목록을 대등하게 볼 것인가, 한쪽을 더 믿을 것인가다. 오류 코드가 많이 오가는 곳이면 날말 쪽을 더 믿는 편이 낫고, 개념을 묻는 질문이 많으면 반대다. 그런데 그 비율은 질문이 도착하기 전에는 모른다. 저장소마다 기본값이 다르다는 사실 자체가 답이 정해져 있지 않다는 증거다.

그리고 이 결정을 어디서 내릴지도 갈린다. 시스템에 한 번 정해 두면 모든 질문이 같은 비중을 지낸다. 검색 도구의 인자로 열어 두면 질문마다 다르게 정할 수 있는데, 정하는 것이 모델이 된다.

모델은 자기가 무엇을 묻는지 아니까 유리하고, 대신 그 선택이 매번 달라진다. 한 번 정할 것인가, 매번 정하게 할 것인가.

12.3 재순위화: 후보를 다시 줄 세운다

겹침이 많은 것을 세고 넘어간다. 오류 코드도 등대도 이제 낱말 검색이 낯는다.

그런데 「배포가 왜 자주 실패하는가」를 묻는 질의는 그대로다. 실패를 한탄하는 회의록이 낱말로 봐도 질의와 가장 많이 겹치는 조각이기 때문이다. 낱말 검색도 문서를 질의가 오기 전에 혼자 훑어 두므로, 겹치는 것으로는 이 실패가 안 풀린다.

질의와 조각을 함께 읽어야 한다. 그리고 그것은 검색을 몇 개 겹쳐도 나오지 않는다.

그래서 단계를 하나 더 세운다. 질의와 조각을 나란히 놓고 모델에게 읽히는 것이다. 이 조각이 이 질문에 답이 되는가를 묻는다.

함께 읽으면 보이는 것이 있다. 한탄의 회의록에는 원인이 없다는 것. 설정 문서는 질문을 하나도 답지 않았지만 그 안에 답이 있다는 것. 「켜야 한다」와 「켜면 안 된다」가 반대라는 것. 전부 따로따로 평가하던 눈에는 안 보이던 것들이다.

이 방법이 기본이 되지 못하는 이유는 하나뿐이고, 그것이 Khattab and Zaharia (2020)의 출발점이다. 깊은 모델로 순위를 매기는 방법은 매우 효과적이지만 계산 값이 이전 방식보다 여러 자릿수 높다. 질의와 조각의 쌍 하나마다 커다란 모델을 한 번씩 통과시키기 때문이다. 문서 전체에 이것을 대면 질문 하나의 값이 문서 수에 비례한다.

그래서 실무는 이 판정을 버리는 대신 마지막에 세운다. 찬 검색이 문서 전체를 훑어 후보를 좁히고, 이 판정은 그 후보만 다시 본다. 이 단계에 붙은 이름이 재순위화다.

공개된 구성 하나를 보면 수가 셋이다. 두 검색이 섞여 준 후보 150개를 받아, 150개를 전부 다시 읽고, 그중 20개를 컨텍스트에 신는다. 같은 자료가 상위 5개와 10개와 20개를 견주어 20개를 실을 때가 가장 좋았다고 적었다(Anthropic, 2024b).

다시 읽는 수를 실는 수보다 훨씬 넓게 잡는 것이 요점이다. 넓게 받는 것이 앞 단계의 농침을 되살릴 유일한 기회이고, 좁히는 일은 어차피 이 단계가 하기 때문이다.

앞의 예에서 무엇이 달라지는지 본다. 예외 조항은 이미 위에 있었으니 큰 변화가 없다. 대신 「해외 출장 결제 한도 변경은 다음 절차를 따른다」가 크게 내려간다. 낱말은 질의와 여럿 겹치지만 한도가 얼마인지는 아무것도 말하지 않는 조각이다. 겹침만 세는 눈에는 안 보이던 것이 함께 읽는 눈에는 보인다.

언제 계산하는가

여기까지 오면 축 하나가 보인다. 조각 쪽 계산을 언제 하느냐다.

벡터 검색은 미리 다 한다. 조각을 수로 옮겨 두는 일이 저장하는 날 끝나고, 질문이 오면 그 수와 질문의 수를 견주기만 한다. 그래서 질문 하나에 드는 계산이 적다. 대신 조각을 옮길 때 질문을 몰랐으므로 함께 읽는 힘이 없다.

재순위화는 미리 아무것도 안 한다. 질문이 온 뒤에 조각을 처음부터 읽는다. 함께 읽으니 정확한 데, 질문 하나마다 모델을 후보 수만큼 돌린다.

그 사이에 중간 지점이 있다. Khattab and Zaharia (2020)가 도입한 방법이다. 조각을 미리 옮겨 두는 것은 벡터 검색과 같은데, 수 하나로 뭉치지 않고 세밀한 채로 남겨 둔다. 그리고 질문이 온 뒤에 가벼운 계산으로 그 세밀한 것들을 견준다. 미리 할 수 있는 것은 미리 하되 뭉개지는 않는 것이다.

여기가 제11장의 진단이 처방으로 이어지는 자리다. 가까움이 관련성을 훑내 낸 것이라던 진단은, 함께 읽어야 하는 자리에서 조각 쪽이 질문을 보기 전에 계산된다는 사실을 가리킨 것이었다. 그러니 처방도 그 시점 위에 선다. 전부 미리 하는 벡터 검색, 아무것도 미리 안 하는 재순위화, 그 사이의 중간 지점.

다만 미리 하는 것을 줄일수록 흉내는 정교해져도 관련성 자체가 되지는 못한다. 같은 조각이라도 지금 무슨 일을 하느냐에 따라 답이 되기도 하고 안 되기도 하는데, 그 사정은 질문과 조각만 읽는 어느 단계에도 실리지 않는다.

앞에서 읽은 것은 되찾지 못한다

다시 세어 본다. 배포 실패의 질의가 이제 산다. 한탄과 원인이 함께 읽는 눈앞에서 갈라진다.

그러나 예외 조항은 위태롭다. 그것이 목록 위로 올라온 것은 낱말 검색이 살려 준 덕이었다. 살아 오르지 못했다면 이야기가 다르다.

「위 조항」의 주인을 읽은 한 문장은 질의와 나란히 놓아도 이어 주는 낱말이 없다. 무관해 보인다. 그리고 무관해 보이는 것을 끌어내리는 것이 이 단계의 일이다. 판정이 정확해질수록 반쪽은 오히려 더 단호하게 밀려난다.

다시 세우기는 앞에 놓인 것을 잘 읽을 뿐이다. 앞에 놓이기 전에 잃어버린 것은 읽을 수 없다.

12.4 문맥적 검색: 자를 때 맥락을 붙여 둔다

그러면 반쪽은 어디서 살려야 하는가. 맥락이 끊긴 것은 검색할 때가 아니라 자를 때다. 되돌리는 일도 자를 때 해야 한다.

각 조각에 그 조각이 놓여 있던 맥락을 한 줄 적어 붙여 저장하는 것, 곧 문맥적 검색이다(Anthropic, 2024b). 앞의 예외 조항에 이런 줄이 붙는다.

붙인 맥락 이 조각은 출장 결재 한도 규정의 예외 조항이다.

본문 다만 해외 출장에는 위 조항을 적용하지 않는다.

「위 조항」이 무엇인지 몰라도 이제 이 조각은 「해외 출장 결재 한도」라는 질문에 닿는다.

여기서 볼 것은 그 한 줄이 겹치기와 어떻게 맞물리는가다. 붙인 맥락은 두 저장에 모두 들어간다. 뜻으로 찾는 쪽에서는 조각의 자리에 출장과 결재와 한도가 들어서서 질문이 닿을 곳이 생긴다. 글자로 찾는 쪽에서는 본문에 없던 낱말이 겹침에 걸린다. 한 번의 손질이 두 검색을 동시에 고친다.

앞의 예에서 이 조각이 뜻으로 찾은 목록의 3위로나마 올라와 있었던 것도 이 맥락 덕이다. 맥락 없이는 목록에 아예 없었을 조각이다.

왜 하필 자를 때인가

조각이 찾아지려면 읽은 맥락이 찾는 데 쓰이는 표현 안에 들어 있어야 한다. 그리고 그 표현은 조각을 저장할 때 한 번 만들어지고 그다음부터 바뀌지 않는다.

찾은 뒤에 복원하는 길이라면 이미 다른 장치가 있다. 조각의 주소로 원본을 열어 읽는 것이다. 두 장치는 분업한다. 맥락을 붙이는 일은 조각이 찾아지게 만들고, 원본을 여는 일은 찾아진 조각이 온전히 읽히게 만든다.

순서에 주의할 것. 원본을 여는 일은 찾기가 성공한 다음에만 있다. 찾기에서 미끄러진 조각에게는 그 길이 열리지 않는다.

무엇이 얼마나 드는가

맥락은 조각만 보고 쓸 수 없다. 그 조각이 문서 어디에 서서 무엇을 받치고 있었는지는 문서 전체를 놓고 보아야 한다. 그래서 조각 하나마다 문서 전체가 함께 입력된다. 조각이 수십인 문서라면 같은 문서가 수십 번 들어가는 셈이라 이 구성은 오래 값이 많이 들었다. 같은 앞부분이 되풀이되는 입력에서 그 값을 줄이는 캐싱이 이것을 쓸 만한 범위로 끌어내렸다.

그리고 이 호출은 문서 수만큼 도는 기계적인 요약이다. 작은 모델이 맞는 자리다. 판단의 품질보다 처리량이 정하는 일이기 때문이다.

표 12.2: 단계 하나를 읽을 때 사는 것과 무는 값. 어느 단계도 겨는 것만 들고 오지 않는다.

없는 단계	사는 것	무는 값	새로 생기는 결정
날말 검색을 하나 더 세운다	학습 밖에서 태어난 어휘로 묻는 질의	저장 한 벌과 그 벌의 갱신. 나란히 도니 지연은 거의 없다	두 목록을 어느 쪽으로 기울여 섞을 것인가
받은 후보를 다시 줄 세운다	질의와 조각을 함께 읽는 판정	후보 수에 비례하는 모델 호출과 대기. 검색하는 스텝마다	몇 개를 넘겨받고 그중 몇 개를 실을 것인가
저장하는 날 조각에 맥락을 붙인다	잘리며 끊긴 앞뒤 맥락	문서 수에 비례하는 모델 호출. 조각마다 늘어나는 저장	무엇을 얼마나 적을 것인가. 언제 다시 적을 것인가

새 위험도 함께 온다. 맥락을 적는 손은 모델이고, 모델은 매번 다르게 답한다. 틀린 맥락이 붙은 조각은 원문에 없는 날말로 검색에 걸려드는 조각이 된다. 그리고 그 잘못은 성공의 형태로 온다. 검색 로그에서 어느 적중과 구별되지 않는다.

어떤 맥락이 좋은 맥락인지에도 정답이 없다. 맥락은 도착할 질문이 조각에 닿는 데 필요한 만큼이면 된다. 그런데 무엇이 필요할지는 질문이 와야 알고, 저장은 그 전에 끝난다. 조각의 크기에 정답이 없던 것과 같은 문제가 맥락의 내용에서 반복된다.

12.5 단계가 늘어난다는 것

셋을 다 없으면 질문 하나가 지나는 길이 이렇게 된다.

문서를 저장할 때 이미 세 가지를 한다. 자르고, 조각마다 모델을 돌려 맥락을 적어 붙이고, 두 방식으로 각각 저장한다. 질문이 오면 두 검색이 각각 훑고 두 목록이 하나로 섞인다. 시점이나 출처를 보고 후보를 거르는 단계가 끼기도 한다. 그다음 재순위화가 순서를 바꾸고, 그 위에서 실을 몇 개가 골라진다. 한 번의 훑기였던 것이 예닐곱 단계가 되었다.

늘어나는 것이 넷이다.

지연. 두 검색은 나란히 돌릴 수 있어 기다림이 한 번으로 끝난다. 재순위화는 다르다. 앞 단계가 넘긴 것을 받아야 시작하므로 순서가 강제되고, 그 기다림이 그대로 지연이 된다. 그리고 이 기다림은 질문마다 한 번이 아니다. 검색창에서는 사람이 한 번 묻고 한 번 기다리지만, 루프에서는 검색하는 스텝마다 되풀이된다.

토큰. 재순위화는 모델 호출이고, 후보가 많을수록 호출이 많아진다. 맥락을 붙이는 일도 모델 호출인데 이쪽은 문서 수만큼이다. 그런데 무는 때가 다르다. 재순위화는 질문이 올 때마다 무는 값이고, 맥락 붙이기는 저장할 때 한 번 무는 값이다. 하나는 매일 나가고 하나는 처음에 나간다. 갈래가 다르니 따로 세야 한다.

저장 공간. 두 방식으로 찾으려면 두 벌을 만들어 둔다. 여기에 맥락을 붙이면 조각마다 몇십 토큰이 앞에 더 붙으므로 양쪽이 다 커진다. 그리고 둘은 함께 낡는다. 옮기는 방식을 바꾸면 문서 전체를 다시 옮겨야 한다는 제11장의 청구서가 이제 두 배가 된다.

고칠 자리. 단계마다 정할 값이 생긴다. 두 목록을 어느 쪽으로 기울여 섞을 것인가, 후보를 몇 개나 넘길 것인가, 그중 몇 개를 실을 것인가, 맥락을 얼마나 적을 것인가. 하나하나가 따로 정해지는데 서로 물려 있다. 넘기는 후보를 줄이면 섞는 단계에서 밀린 조각을 되살릴 기회가 줄고, 거르기를 조이면 검색이 더 넓게 훑어야 한다.

넷과 성질이 다른 것이 하나 더 는다. 실패할 자리다.

답이 틀렸을 때 가능한 사정을 세어 본다. 두 검색이 다 놓쳤을 수 있다. 한쪽은 찾았는데 섞는 단계에서 밀렸을 수 있다. 거기까지 살았는데 거르는 단계가 표시를 잘못 읽었을 수 있다. 재순위화가

옳은 후보를 끌어내렸을 수 있다. 저장할 때 잘못 적힌 맥락이 엉뚱한 조각을 위로 밀어 올렸을 수 있다.

한 번의 찾기에서 의심할 곳은 제11장의 세 어긋남뿐이었는데, 이제 단계마다 하나씩 늘어난다.

그리고 이 자리들은 대등하지 않다. 각 단계는 앞 단계가 넘긴 것 안에서만 일한다. 재순위화는 받은 목록의 순서를 바꿀 뿐 목록 밖의 것을 불러오지 못한다.

극단을 짚어 둔다. 틀리는 법이 없는 재순위화를 상상해 보자. 앞 단계가 답을 안 넘겼다면 그것이 하는 일은 답 없는 후보들을 완벽한 순서로 세우는 것이다. 앞에서 잃은 것은 뒤에서 되찾지 못한다.

무엇보다 단계 사이의 농침은 조용하다. 앞 단계가 무엇을 안 넘겼는지는 넘겨받은 쪽 어디에도 적히지 않고, 재순위화 앞의 목록은 언제나 부탁한 수만큼 채워져 있다.

그래서 어느 단계를 의심할 것인가가 기록의 문제로 돌아온다. 제11장은 검색이 자기 사정을 함께 보고하게 하자고 했다. 몇 건을 훑었고 최고 점수가 얼마인지가 함께 오면 못 찾음이 볼 수 있는 사건이 된다는 것이었다.

단계가 여럿인데 그 보고가 최종 목록 하나에 대한 것뿐이면, 방금 센 사정들은 답이 틀렸을 때 「검색이 이상하다」라는 한 문장으로 도로 몽게진다. 갈래를 가르는 것은 중간 결과의 기록이다. 검색별 목록, 섞인 목록, 거른 것, 재순위화 앞뒤의 순위가 남아 있어야 실패를 단계로 거슬러 올라갈 수 있다.

이 기록이 값을 하는 이유는 실패가 되풀이되기 때문이다. 앞 단계의 농침은 질문 유형책다. 같은 유형의 질문은 같은 자리에서 같은 이유로 빠지고, 같은 잘못된 답이 반복된다.

기록이 없을 때 어디를 고치게 되는지는 예측할 수 있다. 마지막 단계를 고친다. 재순위화는 끝에서 있어 바꾸면 순위가 바로 달라지고 갈아 끼우기도 쉬워서 언제나 먼저 손이 간다. 그런데 문제가 앞에 있으면 그 수선은 아무 일도 하지 않는다. 그리고 앞 단계의 실패는 뒤 단계의 로그에 흔적이 없으니 고칠 후보로 떠오르지도 않는다.

12.6 여러 단계로 풀리지 않는 것

여기까지가 단계 안의 일이다. 다음에 해당하면 단계를 쌓을 때가 아니다.

1. 실패가 쓰는 쪽에 있다. 옳은 조각이 실렸는데 답이 그것을 쓰지 않았다면, 이 장의 어느 장치도 만질 것이 없다. 찾는 쪽을 완벽하게 만들어도 전체는 쓰는 쪽을 넘지 못한다. 이때 단계를 엮는 수선은 품질 대신 값과 지연만 얹는다.
2. 답이 문서에 적혀 있지 않다. 아예 없는 것, 그리고 저장 이전의 처리에서 사라진 것은 어느 단계 앞에도 나타나지 않는다. 문맥적 검색조차 있는 조각에 결말을 붙이는 장치이지 없는 내용을 지어 넣는 장치가 아니다. 흩어진 것을 세거나 이어야 답이 되는 질문도 그대로 남고, 그것은 제13장의 소재다.
3. 관련성을 가르는 것이 텍스트 밖에 있다. 아무리 비싼 판정도 읽는 것은 질의와 조각의 텍스트다. 같은 내용을 담은 문서가 둘일 때 그중 낡은 쪽이라는 사실이 본문에 적혀 있지 않으면 어느 검색에도 어느 판정에도 보이지 않는다. 절반의 처방은 단계 안에 있다. 시점과 출처의 표지로 거르는 일은 표지가 적혀 있는 한 가장싼 단계다. 나머지 절반은 단계 밖에 있다. 표지가 아예 붙어 있지 않으면 어느 단계도 그것을 만들어 주지 못한다.
4. 실패의 빈도가 단계의 값을 밀돈다. 단계는 상시 비용이다. 모든 질의가 지나가며 값을 내고, 잡아 주는 실패는 일부 질의에만 있다. 첫 훑기가 이미 잘 맞는 시스템에 단계를 엮는 것은 드문 실패를 사려고 모든 질의에 세를 무는 일이다. 자기 검색의 실패율을 평가해 본 적 없이 단계부터 엮는 팀은 순서가 거꾸로 된 것이다.

넷을 관통하는 것은 도입의 문장이다. 이 장의 장치는 찾는 쪽의 것이다. 찾고도 안 쓰이는 것, 적혀 있지 않은 것, 텍스트 밖의 것은 그 바깥이라 단계를 몇 개 쌓아도 닿지 않고, 마지막 항목이 보이듯 안에서조차 단계는 공짜가 아니다.

몇 단계로 세울 것인가에도 정답이 없다. 단계 하나가 줄여 주는 실패의 빈도와 그것이 보태는 값은 질의와 문서에 따라 달라진다.

마지막으로 그림에 잘 그려지지 않는 단계 하나를 마저 센다. 골라져 실린 후보들 위에서 답을 만드는 모델 자신이다. 모델은 컨텍스트 안의 후보를 읽고 그 가운데 쓸 것을 고르므로, 설계자가 무엇을 세웠든 마지막으로 고르는 것은 언제나 모델이다.

이 단계는 셋으로 특별하다. 값이 스텝의 모델 호출 전체라서 가장 비싼 판정이다. 컨텍스트에 과제가 실려 있으니 유일하게 지금 무슨 일을 하는지 아는 판정자다. 그리고 이 단계의 실패가 곧 쓰는 쪽의 실패다.

12.7 요약

한 문장으로. 뜻으로 찾는 길과 글자로 찾는 길은 서로 다른 것을 놓치므로 겹치면 덜 놓친다. 거기에 질의와 조각을 함께 읽어 다시 줄 세우는 단계와, 잘리며 끊긴 맥락을 저장하는 날 미리 되돌려 놓는 단계를 없으면 찾기는 여러 단계가 된다. 셋 다 겨는 것을 맞히지만 셋 다 찾는 쪽만 움직이고, 단계가 늘어난 만큼 값과 지연과 실패할 자리도 함께 늘어난다.

확실해진 것.

- 낱말 검색과 벡터 검색은 놓치는 자리가 서로 엇갈린다(표 12.1). 겹침의 근거는 그 엇갈림 하나이고, 한쪽을 다듬어 둘을 겹치게 만드는 길은 양쪽 다 막혀 있다. 그리고 이 장의 세 장치는 전부 찾는 쪽만 손댄다. 실린 뒤에 그것이 쓰이는지는 그대로 남는다.
- 두 점수는 측정하는 것이 달라 더할 수 없다. 그래서 널리 쓰이는 방법은 점수를 버리고 등수만 쓰되 줄어드는 쪽을 완만하게 둔다. 그러면 한 목록에서 한 번 크게 받는 것보다 두 목록에서 두 번 나눠 받는 쪽이 이긴다. 두 목록을 대등하게 볼 것인가에는 정답이 없고, 그것을 한 번 정해 둘 것인가 질문마다 모델이 정하게 할 것인가가 남는 결정이다.
- 질의와 조각을 함께 읽는 판정은 검색을 몇 개 겹쳐도 나오지 않는다. 그 판정은 값이 문서 수에 비례하므로 문서 전체에 딸 수 없고, 싸게 좁힌 뒤 마지막에 세운다. 다시 읽는 수는 실는 수보다 훨씬 넓게 잡는다. 넓게 받는 것이 앞 단계의 놓침을 되살릴 유일한 기회이기 때문이다.
- 세 장치를 가르는 축은 조각 쪽 계산을 언제 하느냐다. 벡터 검색은 저장할 때 다 하고, 재순위화는 질문이 온 뒤에 다 하고, 그 사이에 미리 하되 뭉개지 않는 중간 지점이 있다. 다만 미리 하는 것을 줄일수록 흉내는 정교해져도 관련성 자체가 되지는 못한다. 지금 무슨 일을 하는지는 질문과 조각만 읽는 어느 단계에도 실리지 않는다.
- 맥락이 끊긴 것은 검색할 때가 아니라 자를 때이므로 되돌리는 일도 자를 때 해야 한다. 붙인 맥락은 두 저장에 모두 들어가 한 번의 손질로 두 검색을 함께 고친다. 대신 그 한 줄을 적는 손이 모델이라, 틀린 맥락이 저장된 것 안에 놓고 그 잘못은 성공의 형태로 온다.
- 단계 하나를 없으면 지연과 토큰과 저장 공간과 고칠 자리가 함께 늘고(표 12.2), 실패할 자리가 하나 더 늘는다. 각 단계는 앞 단계가 넘긴 것 안에서만 일하므로 틀리는 법이 없는 재순위화도 앞에서 읽은 답을 세우지 못한다. 그 놓침은 어디에도 적히지 않아서, 중간 기록이 없으면 수선은 눈에 보이는 마지막 단계로 쏠린다.
- 여러 단계로 풀리지 않는 것은 넷이다. 쓰는 쪽의 실패, 적혀 있지 않은 답, 텍스트 밖의 판단, 그리고 실패의 빈도가 밀도는 상시 비용. 그리고 골라져 실린 후보 위에서 마지막으로 고르는 것은 언제나 모델 자신이며, 그 단계의 실패가 곧 쓰는 쪽의 실패다.

아직 답하지 않은 것.

- 흩어진 것을 세거나 이어야 답이 되는 질문을 구조로 받는 장치는 제13장이 받는다.
- 두 목록을 어느 쪽으로 기울일지, 후보를 몇 개 넘길지, 그중 몇 개를 실을지, 맥락을 얼마나 적을지의 근거는 여기에 없다. 그 값들을 정하려면 판정이 달린 평가 집합과 단계별 지표가 있어야 하고, 그 설계는 제6부가 받는다.

오늘 할 것. 답이 틀린 실행 열 건을 골라, 옳은 조각이 컨텍스트에 실려 있었는지 가른다. 실려 있었던 건은 이 장의 어느 장치로도 고쳐지지 않으므로 명단에서 빼는 것이 먼저다(절 12.6). 남은 건에서 질의를 열어 식별자나 코드명이나 규정 번호가 들어 있는데 못 찾은 건이 몇인지 센다. 그 수가 두 검색을 겹쳐 살아날 뚝이고, 0에 가까우면 겹치기는 지금 살 것이 아니다(절 12.1). 마지막으로 지금

의 검색 로그에 최종 목록 말고 무엇이 남는지 연다. 검색별 목록과 재순위화 앞뒤의 순위가 없으면, 단계를 엮은 다음 날부터 어느 단계를 의심할지 알 수 없다(절 12.5).

12.8 생각해 볼 문제

1. 자기 문서에서 두 검색이 서로 엇갈린다는 것을 관측하는 실험을 설계하라. 식별자와 코드명을 묻는 질의 묶음과, 같은 것을 다른 낱말로 묻는 질의 묶음을 만들어 낱말 검색만, 벡터 검색만, 겹침의 세 구성에 던지는 설계다. 두 목록을 한쪽으로 기울여 섞을 때 어느 묶음이 먼저 상하는가. 두 묶음의 비율을 미리 알 수 없다는 것이 왜 이 결정을 어렵게 만드는지 적어 보라.
2. 재순위화에 넘기는 후보 수를 늘려 가며 최종 품질과 값의 곡선을 그리는 절차를 적어 보라. 품질이 어느 지점부터 늡는다면 그것은 무엇의 신호인가. 앞 단계의 농침이 지배하는 시스템과 재순위화의 오판이 지배하는 시스템에서 이 곡선이 어떻게 다르게 생길지 예측하고, 그 예측을 「앞에서 잃은 것은 뒤에서 되찾지 못한다」로 설명하라.
3. 답이 틀린 사례 열 개를 골라 단계에 귀속시키는 절차를 설계하라. 「앞 단계가 놓쳤다」와 「재순위화가 끌어내렸다」를 가르는 데 필요한 최소의 기록은 무엇인가. 최종 목록만 남는 로그에서 구분할 수 없는 갈래는 어느 것들인가. 자기 팀의 수선 이력에서 마지막 단계로 수선이 풀린 흔적을 찾아보라.
4. 문맥적 검색의 맥락이 문서를 잘못 간추린 조각을 일부를 몇 개 심고, 어떤 질의가 그 조각을 잘못 찾아오는지 관측하는 실험을 설계하라. 이 실패가 검색 로그에서 어느 적중과 구별되지 않는 이유를 적고, 원본 대조 말고 이 실패를 볼 창구가 있는지 논하라.

12.9 다시 읽을 때

- 제13장을 읽고 절 12.6의 둘째 조건으로 돌아오면, 단계 밖이라며 넘긴 질문이 구조라는 장치를 얻는다. 여기의 장치가 전부 가져오기의 개선이었다면 그 장의 장치는 질문을 다른 저장소 앞으로 보내는 것이라는 대비가 거기서 선다.
- 절 12.3의 「언제 계산하는가」로 돌아와 제11장의 진단과 나란히 놓으면, 세 방법이 서로 다른 기술이기 전에 한 물음에 대한 세 위치라는 것이 보인다. 조각 쪽 계산을 저장할 때 다 할 것인가, 질문이 온 뒤에 다 할 것인가, 미리 하되 뭇개지 않을 것인가.
- 제11장을 읽고 여기의 값들로 돌아오면, 질문마다 무는 값과 저장할 때 한 번 무는 값이 배치 결정의 향으로 놓여 있다. 단계를 어디까지 쌓는가라는 물음이 지식을 어디에 두는가라는 더 큰 교환의 일부라는 것이 거기서 보인다.
- 제6부를 읽고 절 12.5의 단계 귀속으로 돌아오면, 중간 기록이 평가의 재료가 되어 있다. 어느 쪽으로 기울일지와 몇 개를 넘길지와 단계를 더 엮을지라는 미해결들이 전부 판정이 달린 평가 집합이라는 하나의 부재에서 나온다는 것이 거기서 정확해진다.

12.10 더 읽을거리

Khattab and Zaharia (2020)에서 확인할 것은 계산법이 아니라 축이다. 조각 쪽을 저장할 때 미리 계산해 두는 길과 질문이 온 뒤에 처음부터 읽는 길 사이에 중간 지점이 설 수 있다는 것, 그리고 미리 하되 수 하나로 뭇개지 않는다는 것이 그 지점의 정확한 이름이라는 것이다.

Anthropic (2024b)에서 확인할 것은 문제의 규정과 구성의 합류다. 조각이 맥락을 잃는다는 문제를 저장하는 날 받는다는 방향, 그리고 그 제안이 맥락 부착 위에 두 검색과 재순위화를 함께

쌓는다는 사실이다. 보고된 값은 그 문서와 그 질의의 것이므로 가져갈 것은 수치가 아니라 구성이고, 그 구성이 이 장의 세 절과 같은 순서로 서 있다.

온톨로지와 지식 그래프

「결제 개편을 누가 담당하나」는 문서를 뒤지면 나온다. 회의록이든 인수인계 문서든, 답이 적힌 줄이 어딘가에 있다. 그런데 「김 부장이 담당하는 프로젝트가 몇 개인가」는 안 나온다.

프로젝트 하나 하나는 제 문서에 충실히 적혀 있다. 몇 개라는 사실만 어느 문서에도 없다. 그 수는 문서를 모아 세어야 비로소 생겨난다. 세려면 전부 봐야 하는데, 검색은 상위 몇 개만 준다.

처방은 찾는 법이 아니라 저장을 바꾸는 것이다. 김 부장을 노드 하나로 두고, 프로젝트를 노드 하나로 두고, 담당한다는 사실을 그 둘을 연결하는 엣지 하나로 적는다. 그러면 셀 수 있다. 김 부장에서 나가는 「담당한다」 엣지를 세면 된다. 그리고 연결할 수 있다. 엣지를 두 번 타면 된다.

대신 무엇이 노드가 될 수 있고 어떤 엣지를 놓을 수 있는지를 미리 정해야 한다. 제9장이 출력의 형식을 미리 정했던 것과 같은 성질의 일이고, 같은 교환이 걸린다.

13.1 검색으로 해결되지 않는 문제

규정 문서와 회의록과 배포 기록과 장애 보고서가 쌓여 있다. 낱말 검색과 벡터 검색이 겹쳐 있고, 후보는 질의와 함께 다시 읽혀 줄이 다시 서고, 조각마다 잘릴 때 앞뒤 맥락이 붙었다. 검색의 성공률로 가져올 수 있는 것은 다 가져온 시스템이다. 여기에 두 질문이 나란히 온다.

첫째는 「결제 개편을 누가 담당하나」다. 이 질문은 풀린다. 답이 한 문장 안에 있고, 그 문장이 든 조각을 찾아 오면 끝난다.

둘째는 「김 부장이 담당하는 프로젝트가 몇 개인가」다. 이 질문은 안 풀린다. 답은 수 하나인데, 그 수가 적힌 문장이 저장소에 없다.

세려면 전부 봐야 하는데 검색은 상위 몇 개만 준다. 김 부장이 맡은 프로젝트가 실제로 일곱인데 후보를 다섯 받았다면, 받은 것을 아무리 충실히 세어도 답은 다섯이다. 후보를 늘리면 되지 않는가. 몇으로 늘려야 하는지 문제다.

전부를 받아도 셈은 끝나지 않는다. 세는 단위는 프로젝트인데 저장의 단위는 조각이다. 한 프로젝트가 계획서와 회의록과 주간 보고에 세 번 적히고, 두 프로젝트가 한 문서에 함께 적힌다. 세려면 같은 프로젝트를 하나로 접는 판정이 있어야 하는데, 검색은 그 판정을 하지 않는다.

세는 질문 옆에 연결된 질문이 있다. 「승인 거부를 일으킨 설정을 바꾼 것은 어느 팀인가」. 이번에는 답이 적혀 있다. 다만 세 문서에 나뉘어 적혀 있다. 장애 보고서가 원인이 된 설정을 가리키고, 배포 기록이 그 설정을 바꾼 작업을 가리키고, 회의록이 그 작업을 맡은 팀을 가리킨다. 문서마다 다리의 한 칸씩은 있는데 다리 전체는 어디에도 없다.

에이전트에는 이 간격을 메울 수단이 있다. 찾고, 읽고, 읽은 것으로 다음 질의를 쓰면 된다. 그런데 걸음마다 값을 묻다. 스텝이 하나 늘고 이력이 그만큼 자란다.

그리고 만약 실패한다면, 이 실패는 조용하다. 답이 틀린 실행의 로그를 열어도 검색은 관련된 조각을 돌려주었고 모델은 받은 것을 충실히 세었다. 아무도 잘못하지 않았으니 루프는 계속 진행된다.

13.2 지식의 그래프 구현

해결은 지식과 지식의 연결이다. 그럼 문장과 문장 사이에 있는 사실을 어디에 둘 것인가. 길은 둘뿐이다. 문장 안에 그대로 두거나, 문장 밖으로 꺼내 따로 적어 두거나.

문장 안에 두면 그 사실은 물을 때마다 다시 읽혀야 한다. 「이번 분기 결제 개편은 김 부장이 맡기로 했다」에서 누가 무엇을 담당하는지를 꺼내려면 그 문장을 읽는 손이 있어야 하고, 그 손은 모델이다. 모델은 같은 문장을 두 번 읽으면 두 번 다르게 읽을 수 있다. 이번에는 「담당한다」로, 다음에는 「맡았다」로 읽는다. 일곱을 세려는데 일곱이 저마다 다른 말로 나오면 셀 수가 없다.

문장 밖으로 꺼내 두면 그 읽기가 한 번으로 끝난다. 한 번 읽어 적어 두면 그다음부터 묻는 일은 읽기가 아니라 조회다. 무엇이 오가는지 보이게 늘어놓으면 이렇다. 세 문서에서 온 문장이 이것이고,

[회의록]

이번 분기 결제 개편은 김 부장이 맡기로 했다.
정산 이관도 같은 사람이 맡는다.

[배포 기록]

결제 개편에서 승인 한도 설정을 바꾸었다. 작업은 정산팀이 했다.

[장애 보고서]

결제 승인이 거부된 것은 한도 설정이 0으로 바뀐 탓이다.

같은 것을 꺼내 적으면 이렇게 된다.

노드

김 부장 (사람), 결제 개편 (프로젝트), 정산 이관 (프로젝트),
정산팀 (팀), 한도 설정 (설정), 승인 거부 (장애)

엣지

김 부장 -- [담당한다] --> 결제 개편
김 부장 -- [담당한다] --> 정산 이관
정산팀 -- [수행한다] --> 결제 개편
결제 개편 -- [바꾼다] --> 한도 설정
한도 설정 -- [일으킨다] --> 승인 거부

문장 다섯 줄에서 노드 여섯과 엣지 다섯이 나왔다. 노드에는 이름으로 가리킬 수 있는 것이 오고, 엣지에는 두 노드 사이에 성립하는 사실이 온다. 이렇게 노드와 엣지를 저장의 단위로 삼은 저장소를 지식 그래프라 한다.

정의 13.1 (지식 그래프). 글의 조각 대신 개체(entity)와 관계(relation)를 저장의 단위로 삼은 저장소를 지식 그래프(knowledge graph)라 한다. 개체는 이름으로 가리킬 수 있는 것이고, 관계는 두 개체 사이에 성립하는 사실이다.

상위 몇 개라는 자름이 없고, 어디까지가 전부인지 어렵하지 않아도 된다. 나가는 엣지가 전부이기 때문이다. 연결을 따라가는 일에서 달라지는 것은 걸음마다 놓칠 자리가 없다는 점이다. 검색이 아니라 조회라서 스텝이 셋이든 다섯이든 그렇다.

「이 계약에 위약 조항이 있는가」도 여기서 모양이 바뀐다. 검색으로 물으면 답은 빈손으로 오는데, 그 빈손이 정말 없다는 뜻인지 있는데 못 찾았다는 뜻인지 갈리지 않는다. 계약을 노드로 두고 조항을 엣지로 두면 답은 그 노드에 위약 엣지가 붙어 있는지를 보는 일이 된다. 다만 그 「없다」는 뿔을 때 빠뜨린 것이 없다는 약속 위에 선다. 그 약속이 깨지는 자리는 절 13.5에서 다시 본다.

대신 잃는 것이 있다. 「원칙적으로 오백만 원이되 사전 승인이 있으면 예외로 한다」라는 문장이 「한도 오백만」이라는 엣지 하나로 늘리면, 유보와 조건이 성공의 형태로 사라진다. 노드와 엣지는 글을 버리고 사실만 남긴 저장이다.

표의 마지막 줄이 이 장치의 자리를 정한다. 노드와 엣지는 더 나은 검색이 아니다. 검색 앞으로 오던 질문 가운데 애초에 검색의 것이 아니었던 부류를 다른 저장소 앞으로 보내는 일이다. 그래서 실무는 둘 중 하나를 고르지 않고 둘을 함께 두며, 질문마다 어디로 보낼지를 정하는 결정이 하나

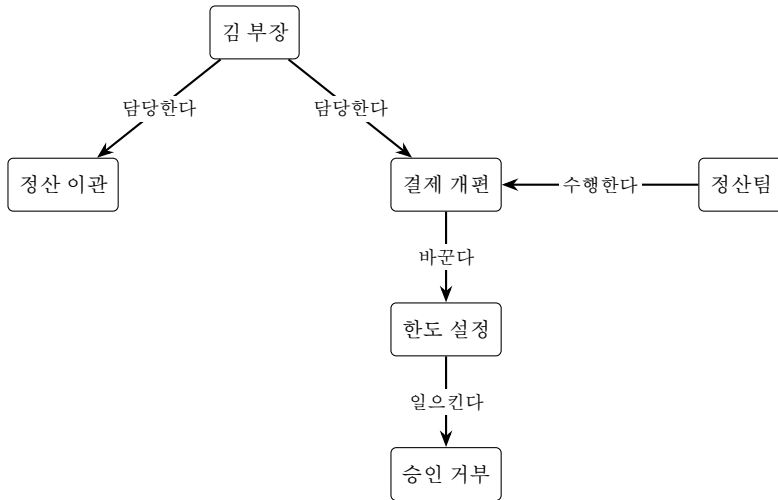


그림 13.1: 세 문서의 문장에서 뽑아낸 노드 여섯과 엣지 다섯. 「김 부장이 담당하는 프로젝트가 몇 개인가」는 맨 위의 노드에서 나가는 같은 이름의 엣지를 세는 일이 되고, 「승인 거부를 일으킨 설정을 바꾼 것은 어느 팀인가」는 맨 아래에서 엣지를 거꾸로 세 번 타는 일이 된다.

표 13.1: 조각으로 저장할 때와 노드와 엣지로 저장할 때, 어느 쪽도 다른 쪽의 개선판이 아니다.

	조각으로 저장	노드와 엣지로 저장
저장의 단위	글의 조각. 자르는 자리만 정하면 된다	노드와 엣지. 무엇이 같은 노드인지를 항목마다 판정해야 한다
문서를 넣는 일	잘라 넣으면 끝난다. 문서끼리 서로 독립이다	새 노드가 기존의 어느 노드인지 매번 맞춘다
답이 오는 모양	후보의 목록. 순위와 점수가 붙는다	단정. 둘이다, 연결된다, 없다
잘 되는 질문	이것에 대해 적힌 대목을 다오	몇 개인가, 무엇과 연결되는가, 있는가 없는가
안 되는 질문	세기, 연결	문구, 맥락, 유보. 글이 글로서 담던 것

새로 생긴다. 하네스의 규칙으로 굳히면 경계 위의 질문이 잘못 간다. 도구의 선택으로 모델에게 열면 고르는 일이 표집이 된다. 잘못 보내는 값은 대칭이 아니다. 적힌 대목을 묻는 질문이 노드와 엣지로 가면 빈손이 오고 빈손은 신호다. 세는 질문이 검색으로 가면 세지 못한 수가 성공의 형태로 온다.

13.3 온톨로지 설계

앞 절의 노드와 엣지는 이미 뽑힌 뒤의 모습이다. 뽑기 전에 정해야 하는 것이 있다. 첫 줄을 적으려면 「김 부장」이 사람이라는 것, 「결제 개편」이 프로젝트라는 것, 그리고 사람과 프로젝트 사이에 「담당한다」라는 엣지가 있을 수 있다는 것이 먼저 서 있어야 한다.

정할 것은 무엇이 노드가 될 수 있는가, 어떤 엣지가 있을 수 있는가, 그리고 어떤 엣지는 어떤 노드 사이에만 놓을 수 있는가다.

마지막 것이 앞의 둘보다 힘이 세다. 「담당한다」가 사람에서 프로젝트로만 갈 수 있다고 정해 두면, 김 부장의 프로젝트를 셀 때 프로젝트가 아닌 것이 섞여 들어올 길이 없다. 정해 두지 않으면 「담당한다」의 양끝에 무엇이든 올 수 있고, 세는 일이 다시 흐려진다.

앞 절의 여섯 노드와 다섯 엣지 아래에 깔려 있던 것을 적어 보면 이렇다.

노드의 종류와 그 노드가 지니는 것
 사람 (이름, 소속), 팀 (이름), 프로젝트 (이름, 시작, 끝),
 설정 (이름), 장애 (일어난 때)

엣지의 종류와 그 엣지를 놓을 수 있는 자리
 담당한다: 사람에서 프로젝트로
 수행한다: 팀에서 프로젝트로
 바꾼다: 프로젝트에서 설정으로
 일으킨다: 설정에서 장애로

이렇게 미리 정해 둔 목록을 온톨로지(ontology)라 한다.

제9장이 출력에 대해 한 일이 이것이다. 어떤 자리가 있고 각 자리에 무엇이 들어갈 수 있는지를 미리 적어 두면, 모델이 무엇을 내놓든 그 형식만은 지켜졌다. 여기서는 같은 일을 저장에 한다. 어떤 노드가 있고 어떤 엣지를 놓을 수 있는지를 미리 적어 두면, 뽑는 손이 무엇을 뽑든 그 형식만은 지켜진다.

같은 교환도 따라온다. 조일수록 세기 쉬워지고, 조일수록 담을 수 없는 것이 는다. 「담당한다」의 양끝을 사람과 프로젝트로 못 박으면 세기가 정확해진다. 그런데 외주 업체가 담당하는 프로젝트가 생기면 적을 자리가 없다. 열어 두면 적을 자리는 생기는데 세는 일이 다시 흐려진다. 제9장에서 값을 몇 개로 못 박을 것인가와 문자열로 열어 둘 것인가의 갈림이었던 것이, 여기서는 엣지의 양끝을 얼마나 좁힐 것인가가 된다.

목록을 정하는 자리가 사람인가 모델인가는 그다음 결정이다. 코퍼스를 모델에 보이고 목록을 뽑게 하면 저절로 생기는 것처럼 보인다. 그때 일어나는 일을 정확히 적으면 이렇다. 모델이 뽑으면 매번 다르게 뽑는다. 이 문서에서는 「담당한다」로, 저 문서에서는 「맡았다」로, 또 다른 문서에서는 「책임진다」로 나온다. 사람이 읽으면 셋이 같은 말인데 세는 쪽에서는 세 종류다. 김 부장의 프로젝트를 세면 「담당한다」 엣지만 세어 하나가 나온다.

같은 애매함이 노드에서도 나타난다. 장애 보고서에는 「김민수」가, 회의록에는 「민수 님」이, 배포 기록에는 영문 계정명이 나온다. 노드 하나인가 셋인가. 이 판정은 낱말의 문제가 아니다. 그 조직에 김민수가 한 사람뿐이라는 것과 그 계정이 그 사람의 것이라는 것을 알아야 내릴 수 있다.

틀리는 방향이 둘이고 실패의 모양이 셋이다. 하나를 셋으로 가르면 한 사람의 일이 세 노드에 흩어져 세는 수가 적게 나온다. 셋을 하나로 합쳤는데 그중 하나가 동명이인이면 남의 기록이 한 노드에 붙어 엉뚱한 연결을 따라가는 일이 발생한다. 그리고 어느 묶음에도 안 들어간 이름은 조용히 사라진다. 앞의 둘은 틀린 답으로 나타나지만 해당 경우는 아무 답이 없다.

반대 방향의 실패도 있다. 제12장의 코드명 「등대」가 작년에는 검색 기능의 이름이었는데 올해 다른 팀이 새 기획에 같은 이름을 다시 썼다. 이름만 보면 가를 수 없다. 실무의 처방은 뽑을 때 이름만 뽑지 않는 것이다. 노드마다 그 문서에 근거한 한 줄 설명을 함께 적어 두면, 나중에 합칠지 가를지 판정할 때 그 설명이 재료가 된다. 이름은 같아도 설명은 다르기 때문이다.

문서가 몇 장일 때는 이름을 전부 늘어놓고 한 번에 판정할 수 있다. 이름이 만 개가 되면 그렇게 못 한다. 한 번에 다 보여 줄 수가 없기 때문이다. 그래서 먼저 거칠게 묶는다. 글자가 비슷하거나 뜻이 가까운 것끼리 선에서 백 개짜리 묶음으로 나누고, 합칠지 가를지는 묶음 안에서만 판정한다. 그러면 서로 다른 묶음에 떨어진 둘은 영영 합쳐지지 않는다. 거칠게 묶는 규칙이 무엇을 같은 노드로 만들 수 있는지의 상한을 먼저 정해 버리는 셈이다.

그래서 실무의 구성이 이렇게 선다. 종류의 목록은 사람이 정하고, 그 목록 안에서 무엇이 어디에 해당하는지는 모델이 뽑는다. 제9장에서 자리를 사람이 정하고 채우기를 모델이 하던 분업이 여기서 반복된다. 모델이 후보를 만들고 사람이 판정하는 것과, 판정 자체를 모델에게 넘기는 것은 다른 이야기다.

표 13.2: 노드와 엣지를 세우고 유지하는 데 드는 것. 사람 품이 드는 자리가 첫 줄과 마지막 줄이다.

하는 일	손	언제 다시 드는가
무엇이 노드가 되고 어떤 엣지를 놓을 수 있는지 정한다	사람	종류가 늘 때마다
문서를 읽어 노드와 엣지를 뽑는다	모델	문서마다 한 번
새로 뽑은 노드가 이미 있는 어느 노드인지 맞춘다	모델과 사람	문서가 들어올 때마다
엣지의 종류를 하나 더한 뒤 과거 문서를 다시 읽는다	모델과 사람	목록을 고칠 때마다

핵심. 조각으로 저장하는 시스템은 무엇이 같은 것인지 판정하지 않고도 선다. 노드와 엣지로 저장하는 시스템은 그 판정 없이 첫 줄도 못 적는다. 무엇이 노드가 되고 어떤 엣지를 놓을 수 있는지를 정하는 일은 그 조직에서 무엇이 같고 무엇이 다른지를 정하는 일이다. 물을 수 있는 질문의 상한은 저장 용량이 아니라 그 목록을 정한 사람의 이해가 정한다.

옳은 목록의 판정 기준은 이 책에 없다. 목록의 좋음은 도착할 질문이 정하는데, 목록은 질문이 도착하기 전에 정해진다. 제11장이 어디서 자를지 물으며 일반해를 못 찾은 것과 같은 순서의 문제이되, 걸린 것이 크기 하나가 아니라 목록 전체다.

13.4 적용 상황

목록을 정하는 일이 사람의 업무라면, 해당 업무를 언제 들일 것인가. 먼저 판단할 것이 둘이다.

세거나 연결해야 하는 질문이 실제로 오는가. 이 장치가 판단하는 것은 그 질문에 대한 답 하나 뿐이다. 그 질문이 안 오는 시스템에 노드와 엣지를 세우면 모든 문서에 세를 물리기도 아무것도 판단하지 못한다. 최근 백 건에서 「몇 개인가」와 「무엇이 무엇을 일으켰나」로 읽히는 질의가 몇 건인지 세어 본 팀만 이 결정의 재료를 가진다.

관계가 안정되어 있는가. 조직 개편이 반 년에 한 번이면 담당 관계도 반년에 한 번 손보면 된다. 개편이 분기마다 있고 겸직과 외주가 섞이면, 담당 관계를 고치는 일이 문서를 넣는 일보다 커진다. 문서는 낱아도 그대로 두면 되지만 노드와 엣지는 낱으면 틀린 답을 단정으로 내놓기 때문이다. 그리고 고칠 때가 되었다고 알려 주는 것이 없다. 새 문서가 들어오면 색인은 그것을 알지만, 담당이 바뀌었다는 사실은 아무 문서에도 안 적히는 경우가 흔하다.

둘 다 그렇다면 다음은 무엇이 드는가다.

문서를 새로 넣을 때가 조각 색인과 가장 크게 갈린다.

조각으로 저장하면 새 문서를 잘라 넣으면 끝이다. 이미 들어 있는 문서를 볼 필요가 없다. 노드와 엣지로 저장하면 다르다. 새 문서에 나온 「김민수」가 이미 있는 노드인지 다른 사람인지를 매번 확인해야 한다. 문서가 늘수록 확인할 것도 함께 는다.

종류를 하나 더하는 일은 값이 늦게 청구된다. 목록에 「검해서 담당한다」를 추가하는 것 자체는 몇 줄이면 된다. 무거운 것은 그다음이다. 이미 들어 있는 문서에는 그 엣지가 하나도 없으므로, 전부 다시 읽어야 그래프가 완성된다.

답이 단정으로 온다

값보다 먼저 볼 것이 있다. 답의 모양이 달라진다.

검색의 답에는 순위와 점수가 붙어 있다. 그 형태 자체가 확실하지 않다는 표시다. 그래프의 답은 그렇지 않다. 「둘이다」로 온다. 「없다」로 온다.

그 확실함이 위험이 되는 자리가 있다. 뽑을 때 놓친 엣지 하나가 「둘이다」를 조용히 「하나다」로 만든다. 검색의 빈손은 의심할 여지를 달고 오지만 이 답은 그것을 안 달고 온다.

그러면 무엇을 봐야 하는가. 뽑은 것과 원문을 표본으로 대조하는 일 말고 세어 볼 수 있는 것이 하나 있다. 붙은 엣지가 하나도 없는 노드를 센다. 이름은 뽑혔는데 관계가 하나도 안 뽑힌 노드는 대개 절반만 뽑힌 자리다. 그 수가 갑자기 늘면 새로 들어온 문서가 목록에 없는 종류의 사실을 신고 있다는 신호다.

다 만들 필요는 없다

그래서 이것은 전부 만들거나 안 만들거나의 선택이 아니다.

문서에 날짜와 작성자와 부서를 붙여 두는 것도 이미 종류 몇 개짜리 구조다. 오는 질문이 담당을 세는 것뿐이라면 「담당한다」 엣지 하나만 두고 나머지는 검색에 맡기면 된다. 그래프는 통째로 들이는 장치가 아니라 질문 하나씩 사는 장치다.

13.5 그래프와 온톨로지로 풀리지 않는 것

다음에 해당하면 노드와 엣지를 세울 때가 아니다.

1. **미리 정한 것 밖은 안 들어간다.** 목록에 없는 종류의 사실을 만나면 뽑는 손은 그것을 버린다. 버려진 주장은 어디에도 세어지지 않으므로, 목록이 남아 가는 일은 관측되지 않는다. 목록을 정할 때 자리 없는 주장을 어디에 모을지까지 함께 정해 두지 않으면, 낚음은 조용히 진행된다.
2. **뽑아내는 일이 틀릴 수 있다.** 뜻을 읽어야 하는 일이므로 손은 모델이고, 모델은 틀린다. 잘못 놓인 엣지는 노드와 엣지의 형태로는 온전해 검색의 어느 실패보다 단정하게 읽힌다. 그리고 이 실패를 비추는 지표가 없다. 뽑은 것과 원문을 건주는 일은 결국 표본을 골라 사람이 나란히 놓는 일이고, 그 대조를 얼마나 자주 할 것인가에도 일반해가 없다.
3. **관계가 문서에 적혀 있지 않으면 꺼낼 것이 없다.** 누가 무엇을 담당하는지 아무 문서에도 안 적힌 조직에서는 뽑을 엣지가 없다. 이때 지어야 할 것은 노드와 엣지가 아니라 그 사실을 적는 자리다. 사실의 원천이 이미 다른 시스템에 줄로 있는 경우도 같다. 프로젝트 담당이 인사 시스템에, 설정 변경이 배포 기록에 이미 있으면 처방은 그 시스템으로 가는 도구이고, 세는 일은 컨텍스트 안의 종합이 아니라 코드에 맡긴다(절 7.4). 백 건을 세는 일은 백 건을 싣는 일이 아니다.
4. **질문이 결국 원문을 요구한다.** 노드와 엣지는 글을 버린 저장이다. 조항의 해석, 결정의 맥락, 표현의 결이 답의 재료인 과제에서 노드와 엣지가 도울 수 있는 것은 어느 문서를 읽을지 찾는 데까지다. 읽히는 것은 원문이어야 한다.

넷을 관통하는 것은 이 장의 첫 대비다. 노드와 엣지는 더 나은 검색이 아니라 다른 질문을 받는 장치이므로, 그 다른 질문이 실제로 오는가가 판정의 전부다. 그리고 정직하게 적어 둘 것이 있다. 대부분의 시스템에서 이 판정은 세우지 않는 쪽으로 기운다. 세는 질문과 연결된 질문은 실재하지만 대개 전체 질의의 작은 뚝이고, 그중 상당수는 이미 구조로 된 원천이 따로 있으며, 코퍼스는 생각보다 오래 전부 읽기가 가능한 크기에 머문다. 이 장을 읽고 목록부터 세우는 팀은 순서가 거꾸로 된 것이다.

그러면 세우지 않는 쪽에서는 무엇을 하는가. 답은 루프다. 코퍼스가 전부 읽을 만하면 읽고 코드로 세고, 아니면 찾고 읽고 다시 찾는 걸음을 몇 번 밟게 한다. 노드와 엣지는 그 걸음을 색인을 만들 때 미리 치러 두는 쪽이고, 루프는 물을 때마다 치르는 쪽이다. 질문이 찾고 코퍼스가 크면 앞의 것이, 드물고 작으면 뒤의 것이 적게 든다.

13.6 요약

한 문장으로. 「몇 개인가」와 「무엇이 무엇을 일으켰나」는 답이 한 문장 안에 없고 문장과 문장 사이에 있어서, 검색을 아무리 잘해도 답해지지 않는다. 이런 질문은 저장을 노드와 엮기로 바꾸면 세는 일과 연결을 따라가는 일이 되는데, 무엇이 노드가 되고 어떤 엮기를 놓을 수 있는지를 미리 정하는 사람의 일이 그 앞에 선다.

확실해진 것.

- 세는 질문이 검색에 걸리는 자리는 검색의 품질이 아니다. 상위 몇 개를 받은 것으로 늘려야 하는지 알 길이 없고, 전부를 받아도 세는 단위는 프로젝트인데 저장의 단위는 조각이라 셈이 끝나지 않는다. 연결된 질문은 답이 세 문서에 나뉘어 있어 문서마다 다리의 한 칸씩만 있다(절 13.1).
- 그리고 이 실패는 조용하다. 답이 틀린 실행의 로그를 열어도 검색은 관련된 조각을 돌려 주었고 모델은 받은 것을 충실히 세었다. 아무도 잘못하지 않은 채로 루프가 계속된다.
- 관계를 문장 안에 두면 물을 때마다 모델이 다시 읽어야 하고, 읽을 때마다 「담당한다」가 「말았다」가 된다. 문장 밖으로 꺼내 적어 두면 그 읽기가 한 번으로 끝나고 그다음부터 묻는 일은 조희다(정의 13.1, 그림 13.1). 대신 유보와 조건이 저장을 떠난다(표 13.1).
- 미리 정할 것은 무엇이 노드가 될 수 있는가, 어떤 엮기가 있을 수 있는가, 어떤 엮지는 어떤 노드 사이에만 놓을 수 있는가다. 마지막 것이 가장 세다. 제9장의 스키마와 같은 성질이고 같은 교환이 걸린다. 조일수록 세기 쉬워지고 조일수록 답을 수 없는 것이 는다(절 13.3).
- 이 목록을 모델에게 통째로 맡기면 매번 다르게 뽑는다. 같은 사실이 세 종류의 엮지가 되고 같은 사람이 세 노드로 갈린다. 노드마다 한 줄 설명을 함께 적어 두는 것이 나중에 합칠지 가를지 판정하는 재료가 되고, 이름이 만 개가 되면 거칠게 묶는 규칙이 무엇을 같은 노드로 만들 수 있는지의 상한을 먼저 정해 버린다.
- 드는 것은 네 자리다(표 13.2). 문서를 새로 넣는 일이 더하기가 아니라 확인이 되고, 종류를 하나 더하면 과거 문서를 전부 다시 읽어야 한다. 그리고 답이 단정으로 온다. 놓친 엮지 하나가 「둘이다」를 조용히 「하나다」로 만들므로, 붙은 엮지가 하나도 없는 노드를 세어 두어야 한다(절 13.4).
- 풀리지 않는 것은 넷이다. 목록 밖의 사실, 틀리는 뽑기, 애초에 적혀 있지 않은 관계, 원문을 요구하는 질문(절 13.5). 그리고 이것은 통째로 들이는 장치가 아니라 질문 하나씩 사는 장치다.

아직 답하지 않은 것.

- 질문을 검색으로 보낼지 그래프로 보낼지 정하는 자리를 하네스에 둘 것인가 모델에 열 것인가, 그리고 잘못 보낸 것과 잘못 뽑은 것을 관측 가능하게 만드는 설계는 제6부가 받는다.
- 검색해 온 조각과 그래프의 답이 한 컨텍스트에서 충돌할 때 어느 쪽이 이기는가는 제11장이 세운 충돌의 미해결에 그대로 포함된다.
- 원천이 텍스트가 아닐 때, 곧 표와 그림에서 무엇을 뽑을 수 있고 무엇이 첫 칸에서 사라지는지는 제14장이 받는다.

오늘 할 것. 질의 기록에서 최근 백 건을 열어, 「몇 개인가」와 「무엇이 무엇을 일으켰나」로 읽히는 질의가 몇 건인지 센다. 그 수가 0에 가까우면 이 장의 장치는 지금 들일 것이 아니다(절 13.4). 걸린 건을 하나씩 열어, 그 답이 이미 다른 시스템에 줄로 있는지 가른다. 있으면 지을 것은 그래프가 아니

라 그 시스템으로 가는 도구다(절 13.5). 남은 건에서 같은 사람과 같은 프로젝트를 몇 가지 이름으로 부르고 있는지 문서 열 장에서 손으로 적는다. 그 목록이 길수록 뽑기보다 확인에 사람이 먼저 붙어야 한다(절 13.3).

13.7 생각해 볼 문제

1. 자기 질의 기록에서 세는 질문과 연결된 질문을 가려내는 절차를 설계하라. 찾기로도 세기로도 읽을 수 있는 질의가 반드시 나올 텐데, 그 모호함은 무엇의 신호인가. 가려낸 것 가운데 답이 이미 다른 시스템에 줄로 있는 건의 비율을 함께 세어 보라.
2. 자기 문서 열 장을 놓고 온톨로지 초안을 적어 보라. 노드의 종류, 엣지의 종류, 그리고 각 엣지의 양끝이다. 다 적은 뒤 같은 문서를 다시 읽으며 그 목록으로는 적을 자리가 없는 문장을 세어 보라. 양끝을 넓히면 그 수가 줄어드는 대신 무엇이 나빠지는가.
3. 자기 코퍼스에서 같은 대상을 부르는 이름들의 목록을 손으로 만들고, 같은 관정을 모델에게 시켜 불일치를 세어 보라. 하나를 여럿으로 가른 오류, 여럿을 하나로 합친 오류, 그리고 어느 묶음에도 못 들어가 사라진 이름이 각각 어떤 답을 만드는가. 어느 것이 가장 조용한가.
4. 엣지 하나를 일부를 빼고 「몇 개인가」를 물어 보는 설계를 적어 보라. 검색의 빈손과 그래프의 「하나다」가 그 뒤의 결정에 미치는 영향이 어떻게 다른가. 붙은 엣지가 없는 노드를 세는 것 말고, 뽑기가 놓쳤다는 것을 볼 수 있는 신호를 하나 더 제안하라.

13.8 다시 읽을 때

- 제11장의 검색으로 답할 수 없는 것의 목록(절 11.6)으로 돌아오면, 그 목록의 앞 두 항목이 이 장의 두 질문 유형이 되어 있다. 조각을 어디서 자를지에 일반해가 없던 자리와 노드와 엣지의 목록에 일반해가 없는 자리가 같은 순서의 문제라는 것도 거기서 보인다.
- 제9장의 스키마로 돌아가 절 13.3의 목록과 견주면, 자리를 사람이 정하고 채우기를 모델이 하는 분업이 출력과 저장 두 곳에서 같은 형태로 서 있다. 열거된 값과 열린 문자열의 갈림이 여기서는 엣지의 양끝을 얼마나 좁힐 것인가로 옮겨 온다.
- 제7장으로 돌아가 절 7.4의 규율과 나란히 놓으면, 세는 일을 코드에 맡기라는 그 문장이 이 장의 첫 처방이 되어 있다. 백 건을 세는 일과 백 건을 심는 일이 다르다는 것을 거기서 이미 세웠다.
- 제6부를 읽고 절 13.1의 조용한 실패로 돌아오면, 세지 못한 수와 잘못 놓인 엣지를 잡는 일이 평가 설계의 문제가 되어 있다. 정답 건수를 아는 평가 집합 없이는 세는 질문의 실패가 보이지 않는다는 것이 거기서 정확해진다.

13.9 더 읽을거리

Edge et al. (2024)에서 가져갈 것은 둘이다. 첫째는 진단이 겹친다는 사실이다. 검색 증강이 코퍼스 전체를 향한 질문에서 실패하며 그것이 찾기의 과제가 아니라 본래 질의 중심 요약의 과제라는 규정은, 절 11.6의 목록과 따로 출발해 같은 자리에 도달한 관찰이다. 둘째는 구성이다. 원본 문서에서 노드와 엣지를 이끌어 내는 것을 색인의 첫 단계로 두고, 가깝게 묶인 무리마다 요약을 미리 만들어 두는 단계를 그 위에 얹는다. 뽑는 손이 모델이고 그 값이 코퍼스의 크기를 따라 색인 단계에서 치러진다는 계산이 실물로서 있는 모습이다. 다만 이 문헌은 하나의 방식을 제안하는 논문이므로, 방식을 처방으로 옮겨 읽지 말 것이다.

문서가 텍스트가 아닐 때

저장소를 열어 보면 이런 것들이 있다. 상대방이 서명해 보내온 계약서의 스캔본, 쪽마다 표가 절반인 월간 보고서, 여백에 손으로 주석을 단 설비 도면, 낱말 몇 개와 그림 하나로 된 발표 자료. 사람의 눈에 이것들은 전부 같은 문서다. 열면 읽히고, 찾는 답이 그 안에 있다.

파이프라인의 눈에는 두 부류다. 글로 열리는 문서와, 글을 먼저 만들어 내야 하는 문서.

제4부의 앞 장들은 텍스트 자료만 다루었다. 무엇을 남길지 고르는 일, 조각으로 잘라 뜻이 가까운 것을 찾는 일, 낱말 검색과 겹쳐 다시 줄 세우는 일, 개체와 관계를 뽑아 이어 두는 일이 전부 글자 위에서 일어났다. 이 장은 텍스트가 아닐 경우는 어떻게 하는 물음에 답한다. 코퍼스에 무엇이 들어오고, 그것을 읽힐 때 어떤 형태로 모델 앞에 놓고, 답에 그림이 들어가야 할 때 그것을 어떻게 붙이는가.

14.1 코퍼스가 텍스트라는 가정

앞 네 장의 모든 장치가 적히지 않은 문장 하나를 담고 있다. 저장소에 놓인 것이 글이라는 문장이다. 자르기는 글을 자르고, 뜻으로 찾기는 글을 옮기고, 낱말 검색은 글자를 맞춘다. 개체와 관계를 뽑는 일도 문장을 읽어야 한다. 그런데 이 문장은 코드 어디에도 적힐 필요가 없다. 개발 중에 넣어 보는 문서는 대개 설계자 자신이 적은 글이고, 모든 단계가 글을 받아 글을 내놓는 동안은 어길 일이 생기지 않는다.

가정이 실제로 세워지는 자리는 따로 있다. 문서를 저장소에 넣는 첫 걸음이다. 거기서 글이 아닌 문서는 글로 바뀌어 들어오거나, 반쪽만 들어오거나, 파일 이름만 남고 내용 없이 들어온다.

14.2 텍스트가 아닐 경우

들어오는 것을 네 부류로 갈라 보면 각각 다른 데서 걸린다.

스캔한 PDF. 파일은 PDF인데 안에 글자가 없다. 종이를 찍은 그림이 쪽마다 들어 있을 뿐이다. 사람이 열면 읽히지만 집어 낼 글자가 없어서, 그림에서 글을 만들어 내는 단계를 먼저 거치지 않으면 코퍼스에 들어가는 것은 빈자리다.

표와 도면. 글자는 있다. 그런데 뜻의 절반이 글자가 아니라 자리에 있다. 어느 행과 어느 열이 만나는 자리인가, 어느 상자에서 어느 상자로 화살표가 가는가가 그렇다. 글자만 가져오면 그 절반이 남지 않는다.

그림과 사진. 확인해야 할 것이 그림에만 있는 문서가 있다. 설비의 상태를 찍은 사진, 손상 부위를 표시한 도면, 화면을 갈무리해 붙인 장애 기록이 그렇다. 여기서 글자로 가져올 것은 대개 그림 언저리의 설명 한 줄이고, 답은 그림 안에 있다.

손으로 쓴 것. 서명, 여백의 주석, 결재란에 적힌 한 줄. 인쇄된 글자와 달리 읽어 내는 일이 어렵이 되고, 어렵이 틀려도 틀렸다는 표시가 나지 않는다.

표에서 무엇이 사라지는지는 값 하나를 따라가 보면 보인다. 월간 보고서의 지출 표에서「오백만」이라는 칸을 읽는 순간, 눈은 왼쪽으로 가서 그 행의 머리인「삼월」을 확인하고 위로 가서 그 열의

표 14.1: 문서가 글자 밖으로 전하는 것들과 글만 가져온 뒤의 행방. 어느 행에서도 오류는 나지 않는다.

자리	사람이 읽는 법	뿔뿔 글에 남는 것	뒷단에서 일어나는 일
표	행 머리와 열 머리가 만나는 자리에서 값의 뜻을 읽는다	머리와 값이 순서만 남긴 낱말의 나열	값은 찾아지는데 무엇의 값인지가 없다. 옆 칸의 값이 대신 답이 된다
배치	크기와 자리에서 제목, 본문, 각주, 절까지의 위계를 읽는다	같은 굵기로 이어진 한 줄의 흐름	각주가 본문의 자격으로 실리고, 예외 조항이 원칙과 같은 무게로 찾아진다
그림	도면, 도표, 그림 속 글자와 화살표를 내용으로 읽는다	대개 아무것도, 좋아야 그림 언저리의 설명 한 줄	문서에 있던 답이 코퍼스에 없다. 없어진 것인데 처음부터 없던 것으로 보인다
표시	취소선, 강조, 서명란, 도장에 서 글의 지위와 상태를 읽는다	표시가 사라진 본문 그대로	철회된 조항이 유효한 조항과 같은 글로 실려, 낡은 값이 확인 있게 찾아진다

머리인「출장비」를 확인한다. 오백만의 뜻은 칸 안에 있지 않다. 행과 열이 만나는 자리에 있다. 표라는 형식이 하는 일이 정확히 그것이다. 값마다「삼월의 출장비는」을 되풀이해 적지 않고 자리로 그 문장을 대신한다.

글자만 가져오면 그 자리가 없어진다. 뿔뿔 글에서 삼월과 출장비와 오백만은 한 줄로 늘어선 낱말들이다. 나쁜 것은 여기서 오류가 나지 않는다는 점이다. 뿔뿔 글은 문장이고 문단이며 글로서 모양에 흠이 없다. 색인은 정상으로 만들어지고, 삼월의 출장비를 물으면 오백만이 든 조각이 돌아 오기도 한다. 옆 칸의 사백만이 돌아오는 것도 똑같이 자연스럽다. 두 경우를 가를 근거가 코퍼스에 남아 있지 않기 때문이다.

표 14.1의 네 행에 공통된 것이 하나 있다. 어느 행에서도 오류가 나지 않는다는 것이다. 그리고 이 상실은 코퍼스에 고르게 퍼지지 않는다. 줄글로만 된 보고서는 읽을 것이 적고, 표가 절반인 보고서는 절반을 읽는다. 그래서 코퍼스 하나의 평균 품질이라는 수는 말해 주는 것이 별로 없다. 잘 뿔뿔하는 다수의 줄글이 평균을 끌어올리는 동안 어떤 부류의 문서는 통째로 반쪽이 되어 들어가 있을 수 있다.

이 부류는 조직에서 예외가 아니다. 서명해 받은 계약서, 표로 된 실적 보고, 설비 도면, 화면을 갈무리해 붙인 장에 기록이 어느 조직에나 있다. 게다가 코퍼스의 형태는 설계자가 고르는 것이 아니라 도착하는 것이다. 문서를 저장소에 넣는 것은 조직의 구성원들이고, 연결한 저장소에 무엇이 들어 있는지는 연결해 보기 전에 다 알 수 없다. 문서가 글로만이 아니라 그림과 쪽의 배치와 표로도 정보를 전한다는 데서 출발하는 연구가 그래서 나온다(Faysse et al., 2025).

14.3 텍스트가 아닌 자료들의 활용

읽는 것을 알았으니 다음은 무엇을 할 것인가다. 결정은 두 자리에서 따로 내려진다. 문서를 읽힐 때 어떤 형태로 모델 앞에 놓을 것인가, 그리고 답에 그림이 들어가야 할 때 그것을 어떻게 붙일 것인가.

읽을 때

갈래가 셋이다. 글로 뽑아낸다, 쪽을 그림 그대로 보인다, 둘을 섞는다.

글로 뽑으면 앞 네 장의 결정이 그대로 선다. 뜻의 경계를 골라 자를 수 있고, 낱말 검색과 뜻으로 찾기를 겹칠 수 있고, 개체와 관계를 뽑아 이어 둘 수 있다.

표 14.2: 읽힐 때 문서를 어떤 형태로 놓을 것인가. 오른쪽 열이 앞의 두 열을 고르는 문제로 두지 않는다.

	뽑아서 글로	쪽을 그림으로	둘을 함께
읽는 것	표의 교차, 배치의 위계, 그림의 내용, 표시의 상태	축소된 뒤의 작은 글자. 세기는 어렵이 된다	글 쪽의 상실은 남지만 읽을 때 그림이 메운다
자르는 단위	뜻의 경계를 골라 자른다	쪽이라는 그릇의 경계가 먼저 그어져 있다	찾기는 뜻의 경계로, 읽기는 쪽으로
토큰	내용만 실는다	내용이 놓인 종이까지 싣고 스텝마다 되풀이된다	좁힌 뒤의 몇 쪽에만 그림 값을 낸다
찾기	앞 네 장의 방법이 그대로 선다	쪽 그림을 바로 표현으로 옮기는 방식이 보고되어 있다	뽑은 글로 찾고 찾아낸 쪽을 읽는다

그림으로 보이면 자리가 살아 있다. 표는 표의 모양으로, 도면은 도면의 모양으로 모델 앞에 놓인다. 하지만 단점이 셋이다. 첫째, 찾을 수가 없다. 그림은 낱말 검색에 걸리지 않는다. 둘째, 토큰이 많이 든다. 셋째, 저장의 단위가 뜻의 경계가 아니라 쪽이라는 경계로 먼저 그어져 있다. 한 조항이 두 쪽에 걸치고, 한 쪽에서 서로 무관한 절 두 개가 함께 있다. 제11장이 어디서 자를지 묻는 자리는 시작하기도 전에 형식에 의해 정해져 있다.

섞으면 글로 찾고 그림으로 읽는다. 이것이 실무의 답이고, 문서를 받아 처리하는 기본 형태가 이미 그렇다. PDF를 보내면 쪽마다 그림으로 바꾸고, 같은 쪽에서 뽑은 글을 그 그림과 나란히 실어 함께 보낸다(Anthropic, 2026g). 그래서 표는 표의 모양으로 놓이면서 동시에 글로도 놓인다.

다만 섞는 방법에는 주의해야 할 점들이 함께 따라온다.

첫째, 뽑아낸 글이 틀릴 수 있고 그 잘못은 조용하다. 숫자 하나가 다른 숫자로 읽히거나 행이 한 칸 밀려도 결과물은 여전히 문장이다. 그래서 아는 길이 하나뿐이다. 문서 표본을 유형별로 골라 원본과 뽑은 글을 사람이 나란히 놓고 대조하는 것이다.

둘째, 원본으로 돌아갈 길을 남겨야 한다. 뽑은 글은 파생물이고, 파생물의 결합은 원본이 있는 한 다시 뽑아 고칠 수 있다. 뽑고 나서 원본을 버리는 파이프라인은 저장을 아끼는 대신 파생물을 원본 자리에 올린다. 그 순간 앞 절의 상실은 미뤄 둔 손해에서 확정된 손해로 바뀐다. 되돌릴 수 없는 변환을 실행하기 전에 원본의 행선지를 정한다는 제6장의 규율이 여기에 글자 그대로 적용된다. 뽑기도 루프 밖에서 한 번 일어나는 되돌릴 수 없는 변환이다.

셋째, 어느 쪽을 근거로 답했는지 구별되어야 한다. 이것은 요청에 켜 두면 답과 함께 돌아온다(Anthropic, 2026b). 근거가 붙은 답에서 근거 한 건은 이런 모양이다.

```
{
  "type": "page_location",
  "cited_text": "3월 출장비 5,000,000",
  "document_index": 0,
  "document_title": "3월 지출 보고",
  "start_page_number": 7,
  "end_page_number": 8
}
```

근거가 가리키는 것은 일곱번째 쪽 하나다. 조각에서 원본으로 돌아가는 다리가 여기서 놓인다. 답이 이상할 때 열어야 하는 것은 그 문서가 아니라 그 쪽이다. 시스템이 실제로 읽은 자료를 사람이 열어 보는 창구가 없으면, 원인 찾기는 언제나 엉뚱한 곳에서 이루어진다.

출력에 쓸 때

모델은 그림을 내놓지 못한다. 읽는 쪽에서는 그림을 받지만 내놓는 쪽은 토큰 열 하나뿐이고, 그 열에 픽셀은 실리지 않는다(Anthropic, 2026k). 그래서 답에 그림이 들어가야 하는 시스템에서 모델이 하는 일은 그림을 만드는 것이 아니라 가리키는 것이다.

가리킬 대상이 어디서 오는지는 두 갈래다. 하나는 이미 있는 그림이다. 코퍼스에 든 도면과 사진과 쪽 그림이 그렇다. 다른 하나는 그 자리에서 만들어지는 그림이다. 모델이 그림을 그리는 코드를 적고 실행 환경이 그 코드를 돌린다(정의 7.4). 나온 파일도 픽셀로 돌아오지 않고 식별자로 돌아온다. 두 갈래 다 모델이 내놓는 것은 가리키는 말이다.

어떤 그림이 있는지를 모델이 어떻게 아는가. 컨텍스트에 실려 있어야 한다. 저장소에 그림이 만 장 있어도 목록이 실려 있지 않으면 없는 것과 같다. 목록의 한 줄에 무엇을 적을 것인가가 곧 설계이며, 이 물음은 제7장에서 이미 한 번 나왔다. 도구를 고르게 하는 것이 도구명과 도구 설명이었듯이(절 7.2), 그림을 고르게 하는 것은 이름과 한 줄 설명이다.

쓸 수 있는 그림

```
3월-지출-표
3월 부서별 지출을 항목으로 가른 표
file_011CPMxVD3fHLUhvTqtsQA5w

설비-배치도
2공장 라인 배치와 점검 지점
file_011CNha8iCJcU1wXNR6q4V8w
```

한 줄에 두 가지가 함께 있는 것이 이 목록의 요점이다. 고르는 데 쓰이는 것은 이름과 설명이고, 붙이는 데 쓰이는 것은 그 아래 식별자다. 사람도 모델도 식별자만 보고는 무엇인지 알 수 없고, 이름만 있으면 실제로 붙일 수가 없다. 모델이 내놓는 것은 이 모양이다.

```
{
  "answer": "3월 출장비가 2월보다 늘었고, 늘어난 몫의 대부분이 2공장 점검 출장이다.",
  "figures": ["3월-지출-표"]
}
```

답에는 이름만 들어 있다. 이름을 식별자로 바꾸어 실제로 그림을 붙이는 일은 받는 쪽 코드가 한다. 식별자를 모델에게 직접 적게 하면, 스물몇 글자를 한 글자도 틀리지 않게 옮겨 적는 일을 모델에게 시키는 것이 된다.

잘못 가리키면 어떻게 되는가. 두 갈래로 갈리고, 가르는 것은 어긋남을 누가 먼저 보는가다. 식별자가 어긋나면 요청이 추론에 닿기 전에 거절된다. 없는 식별자를 실은 요청은 이렇게 돌아온다.

```
{
  "type": "error",
  "error": {
    "type": "not_found_error",
    "message": "File `file_011CNha8iCJcU1wXNR6q4V8w` not found."
  }
}
```

시끄러운 실패다. 답도 나오지 않고 입력 토큰도 물지 않는다. 반면 이름이 어긋나면 조용하다. 모델이 목록에 없는 이름을 적어도 그 자리에서는 아무 일도 일어나지 않고, 받는 쪽 코드가 이름을 찾지 못해 그림 자리를 비운 채 답을 내보낸다. 사용자가 보는 것은 멀쩡한 답과 빈자리다. 그림이 근거였다면 근거 없는 답이 근거 있는 답의 모양으로 나간 것이다.

조용한 쪽을 시끄러운 쪽으로 옮기는 길이 있다. 목록에 있는 이름만 값으로 허용하도록 출력의 형식에 열거해 두는 것이다(절 9.3). 그 아래에서 목록에 없는 이름은 거절되는 것이 아니라 뺏히지 않는다.

표 14.3: 코퍼스에 함께 들어오는 것들. 마지막 줄만 코퍼스의 소재가 아니다.

자료	글과 다른 점	어느 갈래로
표 형태의 자료	글자로 열리지만 뜻이 행과 열이 만나는 자리에 있다	글로 두되 자르지 않는다. 세고 거르는 일은 코드에 맡긴다(절 7.4)
코드	글자인데 자립하는 단위가 문단이 아니라 함수와 정의다	글로 두고 자르는 자리를 함수의 경계로 옮긴다
로그	한 줄에는 뜻이 없고 이어진 구간에 뜻이 있다	글로 두되 조각을 시간 구간으로 세운다
소리와 영상	모델의 입구에 그대로 들어가지 않는다	글로 바꾼 자료가 코퍼스가 된다. 돌아갈 자리는 쪽이 아니라 시각으로 남긴다
화면 갈무리	그림인데 기록이 아니라 지금의 상태다	색인의 소재가 아니라 관측의 소재다(제7장)

14.4 그 외 자료

문서 말고도 코퍼스에 들어오는 것이 있다. 앞 절의 세 갈래 가운데 어디로 보낼지가 자료마다 다르다.

표 14.3의 마지막 줄만 성격이 다르다. 화면은 방금의 행동으로 바뀌고 다음 스텝이면 낡는다. 색인의 주기보다 빠르게 변하는 것은 색인에서 찾을 것이 아니라 그 자리에서 읽어야 한다는 경계를 제11장이 그었고(절 11.6), 화면은 그 경계의 극단에 있다.

규율 하나가 여기서 남는다. 코퍼스에는 도표와 차트처럼 시스템이 사람에게 보이려고 그림으로 만든 산출물도 들어온다. 그것이 미래 실행의 코퍼스가 될 것이라면, 만들 때 기계가 읽을 자료를 함께 남긴다. 뿔을 필요가 없는 자료를 함께 적으라는 것이다.

14.5 요약

한 문장으로. 앞 네 장의 모든 장치는 저장소에 놓인 것이 글이라는 가정 위에서 있다. 실무의 문서 상당수가 그 가정을 어기고, 어긴 문서에서 글을 만들어 내는 단계는 문서가 글자 밖으로 전하던 것을 오류 없이 버린다. 그 단계가 자르기보다 먼저 오픈로 여기서 읽은 것은 뒤의 어떤 기술로도 되찾을 수 없다.

확실해진 것.

- 코퍼스가 글이라는 가정은 코드 어디에도 적히지 않은 채 모든 단계가 닫고 있다. 가정이 세워지는 자리는 문서를 저장소에 넣는 첫 걸음이고, 그 걸음이 지난 뒤에는 어느 장치도 읽은 것을 되돌리지 못한다. 수집은 파일 단위로 성공하고 뽑기는 내용 단위로 실패한다(절 14.1).
- 걸리는 것이 넷이다. 글자가 없는 스캔본, 뜻의 절반이 자리에 있는 표와 도면, 답이 그림 안에 있는 사진, 읽기가 어렵이 되는 손글씨다. 표의 값 하나가 어느 행과 어느 열의 것인지 줄을 세우는 순간 사라지고, 그 상실은 오류가 아니라 멸절한 글의 형태로 코퍼스에 들어간다(표 14.1).
- 읽힐 때의 갈래는 셋이다. 글로 뽑으면 앞 네 장의 방법이 그대로 서는 대신 네 자리를 잃고, 그림으로 두면 자리가 사는 대신 찾을 수 없고 토큰이 들고 쪽으로 잘린다. 실무의 답은 섞는 쪽이다. 뽑은 글로 찾고 찾아낸 쪽의 원본을 읽는다(표 14.2).
- 섞기에는 셋이 따라온다. 뽑은 글의 잘못된 조용하브로 표본 대조 말고는 아는 길이 없다. 원본을 보관해야 다시 뽑을 수 있다. 그리고 답에 붙어 오는 근거가 원본의 몇째 쪽인지까지 가리켜야 원인 찾기가 시스템이 실제로 읽은 자료 위에서 이루어진다. 다만 그 근거는 뽑힌 글에만 붙으므로, 글자가 없는 스캔본에서는 확인의 장치가 먼저 없어진다(Anthropic, 2026b)(절 14.3).
- 출력 쪽에서 모델은 그림을 내놓지 못하고 가리킬 뿐이다. 그래서 어떤 그림이 있는지를 이름과 한 줄 설명으로 컨텍스트에 실어야 하고, 붙이는 데 쓸 식별자를 그 옆에 함께 두어야 한다. 식별자가 어긋나면 요청이 추론 전에 거절되고, 이름이 어긋나면 그림 자리가 빈 채로 답이 나간다. 뒤엣것은 이름을 열거로 좁혀 시끄럽게 만들 수 있다.
- 코퍼스에 함께 들어오는 자료마다 갈래가 다르다. 표 형태의 자료는 자르지 말고 코드에 맡기고, 코드는 함수의 경계로 자르고, 로그는 시간 구간으로 세우고, 소리와 영상은 글로 바꾼 자료가 코퍼스가 된다. 화면 갈무리만 코퍼스의 소재가 아니라 관측의 소재다(표 14.3).

아직 답하지 않은 것.

- 어떤 문서를 뽑은 글만으로 감당하고 어떤 문서에서 원본을 열 것인가, 원본을 여는 문턱을 어디에 둘 것인가에 일반해는 없다. 세 갈래를 같은 과제 위에서 나란히 견준 자료를 저자는 알지 못한다.
- 쪽 그림을 바로 검색의 표현으로 옮기는 방식이 어떤 코퍼스와 질의에서 어디까지 통하는지도 견줄 자료가 아직 없다. 확실하게 말할 수 있는 것은 그 값의 형태까지다.
- 표본 대조로만 보이는 뽑기의 손실을 측정으로 바꾸는 일, 곧 판정이 달린 평가 집합의 설계는 제6부가 받는다.

오늘 할 것. 연결된 저장소에서 파일 백 개를 골라 형식별로 몇 개인지 센다. PDF가 몇이고 그중 열어서 글자가 잡히지 않는 것이 몇인지까지 가르면, 뽑기 앞에서 반쪽이 되는 몫이 그 자리에서 나온다(절 14.2). 그다음 표가 든 문서 다섯 장을 골라 원본과 뽑은 글을 나란히 놓고 대조한다. 값이

어느 행과 어느 열의 것인지가 남아 있는지만 보면 된다(절 14.3). 마지막으로 지금 색인에 든 조각 하나를 열어 원본의 몇째 쪽에서 왔는지가 적혀 있는지 확인한다. 없으면 다음에 색인을 만들 때 그 자리부터 채운다(절 14.3).

14.6 생각해 볼 문제

1. 자기 코퍼스에서 뽑기의 손실을 평가하는 절차를 설계하라. 문서 유형별로 표본을 뽑아 원본과 뽑은 글을 사람이 대조하고, 표 14.1의 네 자리별로 사라진 정보를 세는 설계다. 이 측정이 검색 품질 지표로 대체될 수 없는 이유를 절 14.1의 논증으로 설명하라.
2. 표가 든 문서에서 답을 찾는 과제 하나를 골라, 뽑은 글만 쓰는 구성과 뽑은 글로 찾고 원본 쪽을 읽는 구성을 건주는 실험을 절차로 적어 보라. 두 구성의 실패를 찾기의 실패와 읽기의 실패로 갈라 세려면 무엇을 기록해야 하는가.
3. 쪽 그림을 보낼 때 해상도를 얼마로 할 것인가를 결정 문제로 세워 보라. 크게 보내면 상한에서 어차피 줄어 들고, 작게 보내면 작은 글자가 읽히지 않는다. 자기 문서의 가장 작은 글자를 기준으로 이 값을 정하는 절차를 적고, 잘못 정했을 때의 실패가 어떤 모습으로 관측될지 예측하라.
4. 답에 그림을 붙이는 시스템에서 모델이 없는 이름을 적었을 때 무엇이 일어나는지 자기 코드에서 따라가 보라. 그 경로에 기록이 남는가. 남지 않는다면 절 9.3의 열거와 받는 쪽의 검사 가운데 어느 것을 먼저 넣을 것인가.

14.7 다시 읽을 때

- 제11장을 읽고 절 14.1로 돌아오면, 거기서 굳는 결정이라 부른 자르기와 표현 앞에 더 일찍 굳는 결정이 있었다는 것이 순서로 보인다. 표현 모델을 바꿀 때 드는 것은 색인을 다시 만드는 일인데, 뽑기를 무를 때 드는 것은 원본이 보관되어 있으면 다시 뽑는 일이고 보관되어 있지 않으면 무를 길 자체가 없다.
- 제12장을 읽고 절 14.3으로 돌아오면, 값이 적게 드는 방법으로 후보를 좁히고 값이 많이 드는 판정으로 다시 고른다는 거기의 뼈대가 여기의 분업과 같은 형태라는 것이 보인다. 뽑은 글로 좁히고 원본 쪽으로 읽는 구성은 같은 분업을 두 자료로 나눈 것이다.
- 제10장을 읽고 산출물을 두 자료로 남기는 대목으로 돌아오면, 만들 때 찾을 때를 준비한다는 규율이 기억의 쓰기 규율이 되어 있다. 무엇을 남길지만이 아니라 어떤 형태로 남길지가 기억의 설계라는 것이 거기서 채워진다.
- 제9장을 읽고 절 14.3의 출력 대목으로 돌아오면, 스키마가 형식까지만 강제한다는 거기의 결론이 여기서 두 번 쓰인다. 이름을 열거로 좁히면 없는 이름은 뽑히지 않지만, 그 이름이 답에 맞는 그림인지는 형식이 보지 않는다.

14.8 더 읽을거리

Faysse et al. (2025)에서 가져갈 것은 둘이다. 하나는 문제의 규정이다. 문서는 시각적으로 풍부한 구조인데 오늘의 색인은 뽑아낸 글자에 기대며 그 과정이 길고 깨지기 쉽다는 첫 쪽의 관찰은, 방식의 우열과 무관하게 여기의 출발점 그대로 남는다. 다른 하나는 평가하는 틀의 존재다. 이 논문은 시각적으로 풍부한 문서의 검색을 평가하기 위해 여러 영역과 언어에 걸친 쪽 단위 검색 과제들로 이루어진 기준을 함께 내놓았다. 자기 코퍼스에서 두 길을 건주려는 팀에게 필요한 것이 정확히 그런 틀이다.

문서를 그림으로 다룰 때의 실물 조건은 **Anthropic (2026g)**과 **Anthropic (2026k)**에 있다. 쪽마다 글과 그림을 함께 준다는 처리 방식과 두 형태의 토큰 차이, 그림이 토큰이 되는 눈금과 상한, 그리고 상한에서 줄어들면 작은 글자가 읽히지 않는다는 한계다. 근거가 뿔힌 글에만 붙고 쪽 그림에는 붙지 않는다는 제약은 **Anthropic (2026b)**에 쪽 근거의 모양과 함께 있다. 숫자는 모델과 함께 움직이므로 가져갈 것은 형태다. 그림의 값은 넓이를 따라 자라되 상한에서 멈추고, 상한에서 멈추는 대가는 해상도다.

제4부는 여기서 닫힌다. 다섯 장이 하나의 물음이었다. 질문이 도착하기 전에 무엇을 어떤 형태로 어디에 적어 둘 것인가. 무엇을 남길지, 밖에 두는 일이 무엇을 사고 무엇을 파는지, 찾기를 어떻게 겹으로 쌓을지, 조각에 답이 없는 질문을 어떤 구조로 받을지, 그리고 원천이 글이 아닐 때 첫 걸음에서 무엇을 잃는지가 전부 그 물음의 서로 다른 층이었다. 제5부는 방향을 바꾼다. 지금까지의 네 부가 모은 것은 루프에 실릴 재료다. 지시와 도구와 지식이 다 섰으니, 남은 것은 그 재료로 루프를 돌리는 장치다.

제 V 부

루프를 돌리는 일

에이전트 하네스 설계

에이전트가 마흔 번째 스텝에서 「다 했습니다」라고 말한다. 그 말을 받아 루프를 끝낼 것인가, 한번 더 돌릴 것인가.

이 결정을 내리는 것은 모델이 아니다. 관측을 이력에 붙이는 일도, 종료 조건을 확인하는 일도, 다음 호출을 만들거나 만들지 않기로 정하는 일도 모델 안에서 일어나지 않는다.

이 장이 다루는 것이 그 바깥이다. 실무에서 성능을 가르는 것은 모델이 아니라 이쪽인 경우가 많다.

15.1 하네스: 루프는 어떻게 돌아가는가

모델은 한 번 답하고 끝난다. 요청을 받아 다음 행동 하나를 내놓으면 그 호출은 거기서 닫힌다. 다시 부를지, 언제 그만둘지, 무엇을 남길지는 그 답 안에 없다. 밖에서 정한다.

제1장이 한 번의 호출을 셋으로 갈랐다. 이력을 재료로 컨텍스트를 조립하고, 그것을 토큰으로 바꾸고, 토큰을 받아 다음 행동을 내놓는다. 셋 다 넣으면 하나가 나오는 한 방향의 일이고, 셋 다 호출 하나의 안쪽에 있다.

그런데 에이전트는 그 호출을 반복한다. 반복에는 그 셋이 답할 수 없는 물음이 따라온다. 한번 더 돌 것인가. 호출은 자기가 다시 불릴지를 정하지 못한다. 부르는 쪽이 정하고, 그 부르는 쪽은 코드다.

반복이 한 컨텍스트 위에서 이어지는 동안을 세션이라 부른다. 세션이 열리면 컨텍스트가 스텝마다 자라고, 세션이 닫히면 그 컨텍스트는 통째로 사라진다. 한 과제가 세션 하나로 끝날 수도 있고 여러 세션에 걸쳐질 수도 있다.

반복문을 쥐고 이 결정들을 내리는 자리에 이름을 준다.

정의 15.1 (하네스). 에이전트 시스템에서 모델 호출의 바깥에 있으면서 루프의 실행을 소유하는 코드를 하네스라 한다. 하네스는 네 가지를 정한다. 무엇을 남기는가, 언제 멈추는가, 어떻게 다시 시작하는가, 그리고 무엇을 믿는가.

제1장이 워크플로와 에이전트를 갈랐던 기준을 여기 놓아 본다. 에이전트란 언제 멈출지를 실행 중에 정하는 시스템이었다. 그런데 그 관정을 실행하는 것은 여전히 코드다. 모델은 완료를 선언하고 다음 행동을 고르지만, 그 선언을 받아들일지는 하네스가 정한다. 제어의 소유권이 모델로 넘어간 에이전트에서도 반복문 하나는 끝내 코드에 남는다.

앞 장들이 여기로 넘긴 것이 있다. 가장 혼한 사고가 틀린 답이 아니라 멈추지 않는 것이라는 관찰, 그리고 과제를 여러 세션으로 나누면 값이 줄어든다는 셈이다.

뒤엎것이 이 장의 발판이다. 같은 총 스텝을 여러 세션으로 나누면 한 세션이 매 스텝 다시 읽는 이력이 그만큼 짧아진다. 다만 그 이득은 세션 사이에 넘기는 것이 커지지 않을 때만 성립한다. 남은 물음은 그다음이다. 어디서 자르고, 잘린 자리에 무엇을 남기고, 어떻게 다시 잇는가.

「우리는 아직 하네스가 없다」는 말부터 의심해야 한다. 모델을 호출하는 반복문이 있다면 하네스는 이미 있다. 재시도 횟수를 박아 둔 상수, 시간 제한, 예외를 잡고 한번 더 돌리는 분기가 전부 멈추고 다시 시작하는 결정이다. 다만 설계되지 않은 채 코드 여기저기에 흩어져 있을 뿐이다.

하네스

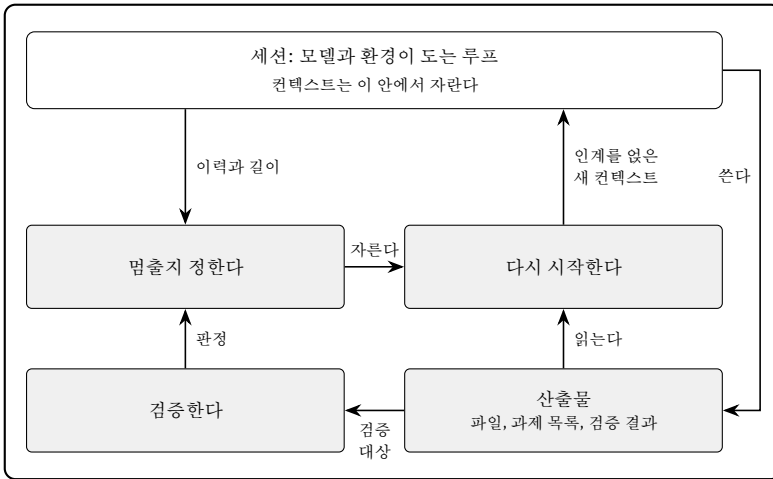


그림 15.1: 하네스와 그것이 정하는 네 가지. 세션 상자는 그림 1.1의 루프를 통째로 넣은 것이다.

실제 결정은 하네스를 만들 것인가가 아니라, 이미 내려져 있는 넷을 한자리에 모아 설계의 대상으로 삼을 것인가다.

15.2 상태: 무엇을 남길 것인가

세션이 닫히면 그 컨텍스트는 통째로 사라진다. 어떤 경로를 시도했는지, 왜 접었는지, 어떤 사실을 확인했는지가 컨텍스트에만 있었다면 전부 없던 일이 된다.

그러니 자를 자리를 정하기 전에 정할 것이 있다. 잘린 자리에 무엇을 놓아두어야 다음 세션이 일을 이어 갈 수 있는가.

제10장에서 다룬 기억과는 남는 기간이 다르다. 기억은 과제가 끝나도 남아 다음 과제에서 다시 불러 나온다. 여기서 남기는 것은 같은 과제의 다음 세션이 손에 쥐면 그만이고, 과제가 끝나면 소임도 끝난다.

세션과 세션 사이로 넘어가는 그 묶음을 인계라 한다. 크기에 제약이 하나 걸려 있다. 세션을 나눈 이득은 세션마다 넘기는 양이 커지지 않을 때만 성립하므로, 인계는 정해진 크기 안에 들어와야 한다.

기록이 아니라 산출물

첫 번째 충동은 대화 기록을 요약해 넘기는 것이다. 아예 틀린 것은 아니지만 원리로 삼을 것은 다른 쪽이다. 상태는 대화의 기록이 아니라 작업의 산출물로 남긴다.

3스텝짜리 세션에서는 기록과 산출물이 거의 같다. 세션을 1000스텝으로 늘리면 기록은 1000스텝 어치로 자라 있는데, 그 끝에서 남는 산출물은 고쳐진 파일 몇 개와 갱신된 목록 몇 줄일 수 있다. 기록의 길이는 걸은 걸음 수를 따라가고, 산출물의 크기는 일의 결과만 따라간다.

다음 세션에 필요한 것은 뒤쪽이고, 이유는 셋이다.

- **크기가 과제에 매인다.** 대화 기록은 스텝 수에 비례해 자란다. 반면 과제 목록의 크기는 과제의 구조에, 파일의 크기는 작업 결과의 크기에 매인다. 넘기는 양이 커지지 않아야 한다는 조건이 저절로 성립하는 쪽은 산출물이다.

- **확인할 수 있다.** 기록은「했다고 말한 것」이고 산출물은「된 것」이다. 파일은 열어 볼 수 있고, 검증 결과는 다시 돌려 확인할 수 있다. 다음 세션이 물려받은 상태를 믿을 근거는 서술의 설득력이 아니라 다시 확인할 수 있다는 데 있어야 한다.
- **성능이 떨어지지 않는다.** 컨텍스트는 길어지면 앞뒤만 읽고 가운데가 묻히지만 파일 시스템은 그렇지 않다.

기본형은 넷이다. 파일은 작업의 결과이고, 과제 목록은 전체 계획과 진행 상태이며, 검증 결과는 무엇이 통과되어 있는지의 기록이고, 결정 기록은 접은 경로와 그 이유다. 여기에 되돌릴 수 있는 변경 이력을 더하면 세션 하나가 망가졌을 때 버릴 단위가 생긴다.

넷 중 결정 기록만이 대화 기록의 성격을 물려받는데, 그마저도 갱신되는 요약이어야 한다.

넘기는 양이 커질 때

세션을 나눈 이득은 넘기는 양이 계속 커지지 않는다는 전제에서 나왔고, 이 전제는 저절로 지켜지지 않는다.

세션이 끝날 때마다 요약을 하나 더 덧붙이는 방식이 이 조건을 정확히 깨뜨린다. 매 세션 요약이 하나씩 덧붙으니 다음 세션은 그만큼 큰 컨텍스트로 시작한다. 출발이 커지면 한 세션이 감당하는 일이 줄고, 같은 일에 드는 세션 수가 늘어난다. 나중에는 인계만으로 컨텍스트가 차서 세션이 시작과 동시에 성능이 떨어지는 구간에 있게 된다.

말로 하면 이렇다. 열 번째 세션은 앞선 아홉을 모두 지고 출발한다.

그래서 인계는 쌓는 것이 아니라 고쳐 쓰는 것이어야 한다. 과제 목록에서 끝난 항목은 지워지거나 접히고, 결정 기록은 다시 쓰이고, 요약은 교체된다. 크기가 세션의 수가 아니라 남은 과제의 크기에 매이도록 만드는 것이다.

참고 15.1. 고쳐 쓰는 일은 공짜가 아니다. 지우고 다시 쓰는 것은 정보의 손실이다. 접은 경로의 이유를 지운 시스템은 같은 경로를 다시 접으려 간다. 그 재방문은 스택을 늘리고, 늘어난 스택은 값으로 돌아온다. 무엇을 버릴지 고르는 문제는 제6장이 압축의 이름으로 정면으로 다룬 것이고, 여기서 세우는 기준은 하나다. 다음 세션이 그것 없이 같은 결정을 다시 내릴 수 있는가.

인계의 형식은 하네스가 고정한다. 형식을 모델의 재량에 맡기면 세션마다 다른 모양의 인계가 나오고, 다음 세션은 매번 처음 보는 문서를 해독하는 것부터 시작한다. 갱신되었는지, 크기가 예산 안인지, 필수 항목이 차 있는지는 모델에게 묻지 않고 코드가 검사할 수 있다. 내용은 모델의 몫이고 그릇은 하네스의 몫이다.

필수 항목이란 앞의 넷이 답하는 물음들이다. 지금 무엇이 되어 있는가, 무엇이 아직 아닌가, 다음에 집을 것이 무엇인가, 무엇을 시도했다가 접었는가.

형식으로 옮기면 이렇게 된다.

- **완료.** 항목 이름과 각각을 무엇으로 확인했는지.
- **남은 것.** 항목 이름과 순서.
- **다음에 집을 것.** 남은 것 가운데 하나.
- **접은 경로.** 시도한 것과 접은 이유를 한 줄씩.

첫째 자리에 확인 수단을 함께 적게 하는 것이 요점이다. 항목 이름만 적는 자리라면「했다고 말한 것」이 그대로 통과한다. 무엇으로 확인했는지를 옆에 적어야 하면 확인하지 않은 항목이 빈칸으로 드러난다.

앞의 셋이 비면 다음 세션은 상태를 알아내는 데서 시작하고, 넷째가 비면 접은 경로를 다시 걷는다. 그런데 자리가 비어도 인계문은 그럴듯하게 읽힌다. 비었는지는 사람의 눈이 아니라 검사가 보아야 한다.

표 15.1: 종료 조건의 갈래. 넷째 줄만 성격이 다르다. 길이 문턱은 루프를 끝내는 신호가 아니라 세션을 자르는 신호다.

갈래	무엇을 보는가	주된 실패 방식
완료 판정	산출물이 기준을 통과했는가	판정 수단이 없으면 성립하지 않는다. 판정이 틀리는 비율이 곧 종료가 틀리는 비율이 된다
자원 상한	스텝 수, 토큰, 시간, 값	상한이 크면 낭비를 다 치른 뒤에야 멈추고, 작으면 풀 수 있던 과제를 자른다
진전 없음	행동과 산출물의 변화	탐색하는 것을 맵도는 것으로 오관한다. 신호의 설계가 과제에 달렸다
길이 문턱	컨텍스트가 얼마나 길어졌는가	종료가 아니라 자르는 신호다. 문턱이 낮으면 이미 흐려진 컨텍스트로 인계를 쓰게 된다
외부 개입	사람 또는 상위 시스템의 신호	개입 지점이 없으면 되돌릴 수 없는 행동이 검토 없이 실행된다

다만 크기를 검사하는 것과 안에 옳은 것이 담겼는지를 검사하는 것은 다르다. 크기는 예산 안인데 다음 세션이 앞 세션의 결정을 되풀이하는 실패는 실행 중에 잡을 방법이 없고, 사후에 두 세션의 기록을 건주는 것이 최선이다.

15.3 종료: 언제 멈출 것인가

남길 것이 정해졌으니 이제 언제 넘길지를 정한다.

멈출지 판정하는 일을 세션 바깥으로 옮기는 것, 그 이동 자체가 이 절의 결정이다. 세션이 자기 종료로 스스로 판단하지 못하는 경우가 많기 때문이다.

갈래가 다섯인 것은 하네스가 볼 수 있는 것에서 나온다. 하네스는 컨텍스트를 조립하므로 그 안에 무엇이 들어 있는지도 알고 길이가 얼마인지도 측정한다. 읽지 못하는 것은 모델의 머릿속이다. 지금 무엇을 놓치고 있는지, 자기가 맵돌고 있는지를 모델은 말로만 알려 주고 그 말은 검사할 수 없다.

그래서 기댈 수 있는 것은 루프를 오가는 것과 컨텍스트의 길모양뿐이다. 볼 수 있는 자리가 다섯이다. 세션이 만들어 내는 산출물, 쓴 자원의 양, 지나가는 행동의 열, 자라는 컨텍스트의 길이, 그리고 루프 밖에서 들어오는 신호.

완료 판정과 자원 상한

가장 좋은 종료는 과제가 끝났음을 확인하고 멈추는 것이다. 그런데 「끝났음」은 판정 수단이 있어야 정의되는 사건이다. 돌릴 수 있는 검사가 없으면 「끝나 보인다」가 유일한 신호이고, 그때 확인하는 일은 사람이 한다.

자원 상한은 반대쪽 끝에 있다. 과제에 대해 아무것도 모르는 종료 조건이며, 그래서 언제나 있어야 한다.

토큰 예산, 시간 제한, 스텝 수 상한은 성공을 만들어 주는 장치가 아니라 실패를 값싸게 만드는 장치다. 값을 정하기 어렵다는 반론은 순서가 틀렸다. 값은 측정해 보고 조정하면 되지만, 장치가 없으면 조정할 대상도 없다.

진전 없음

같은 도구를 같은 인자로 다시 부르고 확인한 사실을 다시 확인하는 것이 제2장에서 본 실패다.

이 신호의 좋은 점은 하네스가 모델에게 묻지 않고 잡을 수 있다는 데 있다. 행동의 열은 하네스를 지나가므로 같은 호출의 반복이나 산출물의 무변화는 세어서 안다.

다만 잘못 잡을 수 있다. 반복처럼 보이는 것이 실제로는 탐색일 수 있다. 그래서 이 갈래는 즉시 종료보다 개입의 신호로 쓰는 편이 안전하다. 잘라서 다시 시작하거나, 사람을 부르거나, 최소한 기록을 남긴다.

길이 문턱

제4장의 요구는 이랬다. 종료 장치가 스텝 수만 보고 컨텍스트 길이를 보지 않으면, 성능이 떨어져 스텝이 늘어나는 동안 아무도 개입하지 않는다.

그 요구를 장치로 옮기면 이렇다. 하네스는 컨텍스트 길이를 감시하고, 성능이 떨어지기 시작하는 지점보다 앞선 문턱에 닿으면 세션을 닫는다.

두 가지가 이 문장에 들어 있다.

첫째, 이것은 루프를 끝내는 조건이 아니다. 과제는 남아 있으므로 세션을 자르고 인계를 남겨 다시 시작하는 조건이다.

둘째, 문턱이 그 지점보다 앞이어야 하는 이유는 하나다. 인계문을 쓰는 것도 모델이다. 성능이 떨어지기 시작한 뒤에 세션을 닫으면 무엇이 중요했고 무엇이 끝났는지를 요약하는 일 자체가 그 구간에서 일어난다. 흐려진 컨텍스트로 쓴 인계는 다음 세션에도 그것을 물려준다. 문턱이 남기는 여유는 흐려지기 전에 인계를 쓰기 위한 예산이다.

문턱에 닿았을 때 할 수 있는 일은 둘이다. 이력을 줄여 같은 세션을 이어 가거나, 세션을 닫고 인계를 남겨 다시 시작하거나. 앞쪽이 압축이고 뒤쪽이 자르기다.

흔히 먼저 걸리는 것은 압축 쪽이다. 자르기는 과제가 한 세션에 담기지 않을 만큼 길거나, 컨텍스트가 이미 흐려져 줄여서 이어 갈 값어치가 없을 때 고른다. 둘은 같은 문턱을 두고 경쟁하며, 압축 쪽의 섀은 제6장이 세워 두었다.

성능이 떨어지기 시작하는 지점은 사양표에서 읽는 값이 아니라 자기 과제로 측정하는 값이다. 측정하지 않았다면 길이 문턱을 세울 기준이 없다.

외부 개입

앞의 넷은 하네스가 혼자 판정한다. 다섯째만 판정하는 주체가 밖에 있고, 필요한 자리도 정해져 있다. 되돌릴 수 없는 행동 앞이다.

지우는 일, 돈이 오가는 일, 밖으로 나가는 메시지. 여기서는 판정이 틀렸을 때 무를 방법이 없으므로, 앞의 네 갈래가 모두 통과시킨 행동이라도 한 번 더 세울 자리가 있어야 한다.

성격도 나머지와 다르다. 넷은 낭비를 막는 장치이고 이것은 사고를 막는 장치다. 다만 하네스가 소유하는 것은 그 자리를 만드는 일까지다. 어떤 행동 앞에 사람을 세우고 무엇을 자동으로 허락할지는 권한의 설계이며 제21장이 받는다.

재시도가 남기는 것

도구 호출은 실패한다. 시간이 초과되고, 절반만 성공하고, 요청 한도에 걸린다. 그때 하네스가 하는 일이 종료 결정의 가장 작은 사례다. 한 번 더 부를 것인가.

몇 번까지 다시 부를지는 자원 상한과 같은 이야기다. 새로운 것은 남기는 쪽이다.

오류 전문을 그대로 이어 붙이면 추적 하나가 수천 토큰이고, 세 번 되풀이하면 세 벌이 쌓인다. 스텝마다 늘어나는 양이 그렇게 부풀고, 부풀 만큼 성능이 떨어지는 지점이 앞당겨진다. 도구가 자주 실패하는 시스템은 같은 과제를 같은 스텝 수로 풀고도 더 이른 스텝에서 흐려진다.

그러므로 재시도의 설계에는 항목이 둘이다. 몇 번 다시 부를 것인가, 그리고 그 실패에서 무엇을 남길 것인가. 뒤쪽의 기준은 다음 결정에 쓰이는 것만 남기는 것이다. 같은 호출이 같은 이유로 세 번 실패했다면 남길 것은 세 번이 아니라 한 번과 비슷하다.

완료 선언은 입력이지 판정이 아니다

모델은 스스로 「다 했습니다」라고 말한다. 그 같은 문장이 나오는 두 상태를 나란히 놓아 보면 문제가 보인다.

하나는 과제가 실제로 끝난 상태다. 다른 하나는 컨텍스트가 흐려진 상태다. 남은 항목은 컨텍스트 가운데 어딘가에 묻혀 있고, 모델은 그것을 놓친 채 눈에 들어오는 일이 끝났다는 이유로 완료를 선언한다.

두 상태는 겉으로 같다. 선언의 문장만 보고는 가릴 수 없고, 하필 스텝이 쌓여 선언에 기대고 싶어지는 뒤쪽일수록 둘째 상태일 확률이 올라간다. 컨텍스트가 한계에 가까워졌다고 여길수록 일을 서둘러 마무리하려는 경향이 보고된 적도 있다(Rajasekaran, 2026).

스스로 끝났다고 말하는 주체를 믿을 수 없는 것은 그 주체가 부정직해서가 아니라, 자기 컨텍스트가 흐려진 것을 스스로 보지 못하기 때문이다. 다만 같은 자료는 뒤이은 판의 모델에서 그 경향이 달라졌다고도 적는다. 서두르는 버릇은 모델의 성질이지 원리가 아니라는 뜻이고, 그것을 막으려고 넣은 장치는 언젠가 쓸모가 없어진다.

그러므로 이 선언으로 루프를 끝내면 안 된다. 판정과 생성을 같은 컨텍스트 안에서 말하면 판정이 독립적이지 않고, 밖에서 오는 신호 없는 자기 판정은 믿기 어렵다는 것이 제2장의 결론이었다.

자기 완료 선언이 값이 없다는 뜻은 아니다. 유용한 신호이되 그 자리가 결론이 아니라 입력이다. 선언이 오면 하네스가 하는 일은 루프를 끝내는 것이 아니라 검증을 발동하는 것이다. 검증이 통과하면 그때 완료다.

다섯 갈래 가운데 무엇을 고를까가 아니라 몇 겹을 둘까가 실제 결정이다. 완료 판정이 있으니 상한은 군더더기라는 설계는, 판정이 틀리는 날 멈출 이유가 하나도 남지 않는 설계다.

갈래들은 서로의 대안이 아니라 서로의 실패를 받치는 겹이고, 무조건 있어야 하는 겹은 자원 상한이다. 판정은 틀릴 수 있고 감지는 잘못 잡을 수 있지만, 쓴 토큰과 흐른 시간은 세는 대로다.

15.4 재시작: 어떻게 다시 시작할 것인가

설계를 지배하는 제약은 하나다. 다음 세션은 이전 세션이 무엇을 했는지 자동으로 알지 못한다. 새 컨텍스트는 백지에서 시작하며, 이전 세션의 존재를 알 수 있는 경로는 둘뿐이다. 인계와 환경 그 자체다. 그 밖의 모든 것은 없던 일이 된다.

뒤집으면 이것이 설계 기준이 된다. 인계는 「그 일을 처음 보는 유능한 작업자가 이것만 읽고 다음 항목을 집을 수 있는가」로 검사한다.

인계는 다음 세션이 처음부터 지고 가는 뚝이다. 그림 15.2의 바닥이 그 크기다.

구현에서 가장 자주 놓치는 것은 처음 시작하는 세션과 이어 가는 세션을 가르치는 일이다.

첫 세션은 과제를 받아 구조를 세우고 분해해 목록을 만든다. 후속 세션은 그 목록에서 다음 항목을 집어 실행하고 목록을 갱신한다. 둘을 하나의 지시문으로 겸하게 만들면 양쪽 모두 나빠진다. 매 세션 구조를 다시 세우며 이미 세운 것을 흔들거나, 구조 없이 표류하며 목록에 없는 일을 시작한다.

지금 이 처음인지 계속인지를 인계의 존재로 판정해 다른 지시문을 주는 것은 하네스의 뚝이다. 후속 세션의 지시가 「한 번에 한 항목」이어야 한다는 것은 Rajasekaran (2026)의 관찰이다. 한 번에 너무 많이 하려는 경향이 장기 실행에서 흔한 탈선이다.

세션의 첫 일은 물려받은 상태를 확인하는 것이다. 검사를 한 번 돌려 지금 무엇이 통과하고 무엇이 깨져 있는지 보고 나서 새 일을 시작한다. 앞 세션이 남긴 목록이 「완료」라고 적어 둔 항목도 이 확인을 통과해야 완료다. 시험을 돌리지 않은 채 완료로 표시되는 일이 드물지 않기 때문이다.

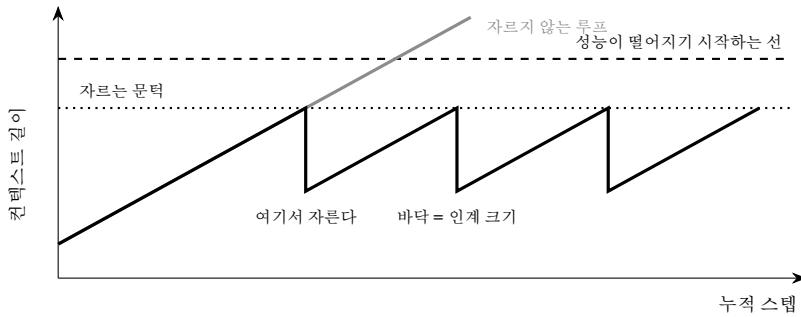


그림 15.2: 자르고 다시 시작하면 톱니가 된다. 회색 직선은 자르지 않는 루프다. 톱니의 바닥이 0이 아니라는 점에 주목할 것. 다시 시작하는 세션은 인계를 지고 출발한다.

자르는 시점을 예산의 경계에 맞출 것인가 과제의 경계에 맞출 것인가도 결정이다. 길이 문턱이 기본 신호이지만 좋은 하네스는 하나를 더한다. 과제 경계에서 자르는 것이다.

목표의 한 항목이 끝나고 검증이 통과된 직후가 인계가 가장 짧고 깨끗한 순간이다. 항목의 한가운데서 문턱에 밀려 자르면 반쯤 된 일의 상태를 말로 옮겨야 하고, 그 서술은 산출물보다 부정확하다.

문턱은 마지막 선으로 두고 자르기는 되도록 경계에서 한다. 견고성도 여기서 나온다. 세션 하나가 흐려지거나 탈선으로 망가지면 그 세션만 버리고 직전 경계에서 다시 시작한다.

15.5 검증: 무엇을 믿을 것인가

원리는 만드는 일과 판정하는 일을 나누는 것이다. 산출물을 만든 컨텍스트가 그 산출물의 합격을 판정하지 않게 한다.

판정 신호는 밖에서 와야 하고, 밖의 신호 없는 자기 교정은 믿기 어렵다는 것이 제2장의 결론이었다.

나뉘어야 하는 것이 무엇인지는 정확히 해 둘 값이 있다. 모델이 아니라 컨텍스트다. 만드는 쪽과 판정하는 쪽이 같은 모델인 것은 혼란 일이고 그 자체가 문제는 아니다. 문제는 판정하는 쪽이 만든 컨텍스트 안에 그대로 앉아 있는 것이다.

무엇으로 판정할 것인가

코드로 검사할 수 있으면 그것이 최선이다. 실행되는 테스트, 형식 검사, 빌드의 성공 여부는 모델의 설득에 흔들리지 않는다. 모델이 읽을 수 있는 신호를 돌려주는 것이면 무엇이든 된다. 시험 묶음, 빌드의 종료 코드, 검사기, 기준 출력과의 대조.

그런 검사가 없는 판단, 이를테면 「이 설계가 좋은가」에는 판정을 맡는 별도의 모델 호출을 두게 된다. 그때 지켜야 할 규칙이 하나 있다.

판정하는 쪽은 만드는 과정을 보지 않아야 한다.

채점자를 따로 두고도 오염이 돌아오는 길이 하나 남아 있다. 채점자에게 답안과 함께 풀이 과정을 읽히는 것이다. 채점자는 그 풀이의 논리를 따라 걸으며 읽게 되고, 틀린 답도 그 논리 안에서는 그럴듯해진다.

나누는 일은 주체에서 끝나지 않고 입력까지 가야 완성되며, 하네스가 소유하는 것이 그 입력의 경계다. 이유는 둘이다. 만드는 쪽의 컨텍스트를 물려받은 판정자는 결론을 정당화하는 이야기까지 함께 물려받는다. 그리고 과정 전체를 보는 판정자는 만드는 쪽과 같은 길이의 컨텍스트를 감당해 같은 흐려짐을 겪는다.

표 15.2: 같은 규칙을 네 자리에 없는 모습. 적히는 문장이 자리마다 다르고, 넷째만 묻는 것 자체가 다르다.

자리	실제로 적히는 것	관정하는 주제
지시에 적는다	시스템 프롬프트의 한 줄. 「시험을 돌려라. 실패하면 고쳐서 다시 돌려라. 전부 통과하기 전에는 끝났다고 말하지 마라.」	모델. 지시를 따르는 일도 매번 달라진다
목표 조건에 건다	매 턴 끝에 관정자에게 보내는 물음. 「시험이 전부 통과했는가. 예 또는 아니오로만 답하라.」 아니오이면 턴이 하나 더 붙는다	모델. 검사는 돌되 관정은 모델이 한다
코드로 막는다	하네스의 종료 조건. 시험의 종료 코드가 0이 아닌 동안에는 루프를 끝내지 않는다	코드. 매번 같다
다른 컨텍스트에 묻는다	새 컨텍스트에 변경된 내용과 원래 증상만 주고 묻는다. 「이 변경은 증상을 가린 것인가, 원인을 고친 것인가.」	다른 컨텍스트의 모델. 묻는 것 자체가 다르다

관정자를 만드는 일

관정자 자신도 흔들린다. 제대로 된 문제를 짚어 놓고도 대수롭지 않은 것으로 스스로 정리해 통과시키는 일이 관측되어 있다(Rajasekaran, 2026).

그래서 관정자를 회의적인 쪽으로 조이게 되는데, 같은 자료는 그 조이기가 만드는 쪽을 자기비판적으로 만드는 일보다 훨씬 다루기 쉽다고 적는다. 다만 세계 조일수록 멸절한 산출물에서도 흠을 잡으므로, 통과를 남발하는 쪽과 과잉 수정을 부르는 쪽 사이 어디에 세울지가 결정으로 남는다.

그래서 관정자를 만드는 일은 관정자를 쓰는 데서 시작하지 않는다. 「이 설계가 좋은가」 같은 판단을 채점 가능한 기준으로 바꾸는 데서 시작한다.

Rajasekaran (2026)가 만드는 쪽과 관정하는 쪽으로 나뉜 구조로 넘어가면서 먼저 한 일이 그것이었고, 점수의 예시를 붙여 관정자를 보정하는 일이 뒤따랐다.

같은 기록에 순서가 하나 더 있다. 만들기 전에 무엇을 만들지와 무엇으로 통과를 관정할지를 만드는 쪽이 먼저 적고 관정하는 쪽이 그것을 검토한다. 기준을 나중에 정하면 만들어진 것에 기준을 맞추게 된다.

그 기록이 도달한 최종 형태는 계획하는 쪽과 만드는 쪽과 관정하는 쪽의 세 자리였다. 유일한 정답이 아니라 네 결정의 한 배치다. 여러 모델 호출이 협력하는 구조는 제16장부터 다룬다.

관정자를 세우는 일은 종착이 아니라 경유다. 기준이 구체적으로 적히고 나면 그 일부는 더 이상 모델의 판단을 필요로 하지 않는다. 형식이 맞는지, 필수 요소가 있는지, 검사가 통과하는지는 코드로 내려앉는다.

관정을 맡는 모델 호출은 코드로 검사할 수 없는 자리를 메우는 임시 수단이고, 기준이 여물 때마다 그 자리를 코드로 갈아 끼우는 것이 이 축의 운영이다.

15.6 하네스를 더 만들지 말아야 할 때

여기까지 세운 장치는 저마다 모델이 무언가를 하지 못한다는 가정 위에서 있다.

가정에는 유효 기간이 있다. 이것들은 모델의 현재 능력에 대한 경험적 진술이지 원리가 아니다. 모델이 나아지면 어떤 가정은 깨지고, 깨진 가정 위의 장치는 무해한 잉여가 되거나 적극적인 방해가 된다.

하네스는 고정된 정답이 아니라 지금 모델에 맞춘 임시 장치다. 깨진 가정 위에 그대로 남은 장치가 뻔 때가 되었다는 첫째 신호이고, 아래가 그것을 포함한 목록이다.

1. 모델이 이미 하는 일을 하네스가 대신하고 있다. 가정이 아직 유효한지를 정기적으로 확인하지 않는 하네스에는 이런 장치가 쌓인다. 꺼도 되는 장치를 찾는 일은 더할 장치를 찾는 일만큼

표 15.3: 하네스의 장치와 그것이 가정하는 무능력.

장치	가정하는 무능력	그 가정이 깨지면
자원 상한	예산을 스스로 세지 못한다	대체로 무해하다. 다만 낮게 굳은 상한은 유능한 모델을 일찍 자른다
진전 없음 감지	멈돌고 있음을 알아채지 못한다	잘못 잡은 일만 남는다. 탐색하는 우회가 맴도는 것으로 잘린다
자르고 인계하기 산출물로 남기기	컨텍스트가 길어지면 성능이 떨어진다 세션을 넘는 기억이 없다	인계를 쓰는 품과 정보 손실만 남는다 가장 오래 유효할 가정. 기억이 생겨도 다시 확인할 수 있다는 이점은 남는다
관정자를 나누기	자기 산출물을 공정하게 관정하지 못한다	나누는 일이 지연과 값만 더한다
과제 분해	큰 과제를 한 컨텍스트 안에서 끝내지 못한다	분해가 전체를 보는 눈을 조개고 인계 품만 만든다

값을 한다.

2. **하네스 자체가 컨텍스트를 잡아먹는다.** 인계 지침, 상태 요약, 규칙 목록, 세션 관리 지시문은 전부 매 스텝 실린다. 그리고 컨텍스트를 그만큼 먼저 채운다. 루프를 보호하려고 넣은 장치가 루프의 여유를 먼저 쓰고 있다면 방향이 뒤집힌 것이다.
3. **하네스가 과제보다 복잡하다.** 세션 관리, 인계 형식, 검증 단계가 정작 풀려는 과제보다 복잡해지면 고칠 것이 하나 더 늘어난 것이다. 단일 세션과 워크플로우로 충분한 과제였는지부터 되물어야 한다.
4. **하네스를 바꾼 효과를 평가할 수단이 없다.** 어떤 장치가 성공률이나 값을 바꾸었는지 평가할 수 없다면 그 하네스는 개선되는 것이 아니라 복잡해질 뿐이다.

첫째 항목을 가려내는 검사법은 하나다. 장치를 끄고 같은 과제를 돌려 본다. 결과가 같으면 그 장치는 잉여다. 좋아지면 방해였다.

이 검사를 하려면 장치를 들일 때 그것이 딛고 선 가정을 함께 적어 두어야 한다. 가정이 적혀 있지 않은 하네스는 검사할 수 없다.

프레임워크는 상당 부분 하네스를 상품으로 만든 것이다. 고른다는 것은 남이 적어 둔 가정 목록을 통째로 들여오는 일이고, 물을 것은 그 목록이 지금의 모델과 자신의 과제에 아직 유효한가다.

마지막으로 하나. 하네스는 루프를 측정하고 자르고 검증하는 주체인데 정작 자기 자신은 셈 바깥에 두기 쉽다. 토큰을 세고 켜고 꺼서 건주는 규율은 하네스 자신에게도 걸린다.

15.7 요약

한 문장으로. 제어의 소유권이 모델로 넘어간 에이전트에서도 반복문 하나는 끝내 코드에 남는다. 그 반복문으로 세우는 장치는 전부 지금 모델이 무언가를 못한다는 가정 위에서 있으므로, 들일 때 가정을 적고 주기적으로 아직 유효한지 확인해야 한다.

확실해진 것.

- 하네스는 모델 호출 바깥에서 루프의 실행을 소유하는 코드다. 정하는 것은 상태, 종료, 재시작, 검증의 넷이다(정의 15.1, 그림 15.1). 모델을 호출하는 반복문이 있다면 하네스는 이미 있다.
- 상태는 대화의 기록이 아니라 산출물로 남긴다. 크기가 과제에 매이고, 다시 확인할 수 있고, 성능이 떨어지지 않기 때문이다. 완료 항목 옆에 무엇으로 확인했는지를 적게 하면 확인하지 않은 항목이 빈칸으로 드러난다.
- 세션마다 요약을 덧붙이기만 하면 다음 세션이 그만큼 큰 컨텍스트로 시작하고, 나중에는 인계만으로 컨텍스트가 찬다(그림 15.2). 인계는 쌓는 것이 아니라 고쳐 쓰는 것이어야 한다.
- 종료는 하네스가 볼 수 있는 다섯 신호의 겹이다(표 15.1). 무조건 있어야 하는 겹은 자원 상한이다. 완료 선언은 결론이 아니라 검증을 발동하는 입력이고, 길이 문턱은 종료가 아니라 자르는 신호다. 문턱이 앞이어야 하는 것은 인계문을 쓰는 것도 모델이기 때문이다. 되돌릴 수 없는 행동 앞에는 사람이 설 자리를 따로 만든다.
- 다음 세션이 아는 것은 인계와 환경뿐이다. 처음 시작하는 세션과 이어 가는 세션은 하네스가 가르고, 세션의 첫 일은 물려받은 상태를 확인하는 것이다. 문턱은 마지막 선으로 두고 자르기는 과제 경계에서 하는 편이 낫다. 그래야 세션 하나가 망가져도 그 세션만 버린다.
- 만드는 일과 판정하는 일을 나누는 것은 주체가 아니라 컨텍스트를 나누는 일이고, 주체에서 끝나지 않고 입력까지 가야 한다. 판정을 거는 자리는 넷이고, 넷째만 「통과했는가」가 아니라 「원인을 고쳤는가」를 묻는다(표 15.2). 판정을 맡는 모델 호출은 임시 수단이고, 기준이 여물 때마다 그 자리를 코드로 갈아 끼운다.
- 장치마다 딛고 선 가정을 적어 두어야 아직 유효한지 확인할 수 있다(표 15.3).

아직 답하지 않은 것.

- 문턱에서 자르기 대신 압축을 고르는 쪽의 셈과, 고쳐 쓸 때 무엇을 버릴지의 기준은 제6장이 세워 두었다. 여기서 더한 기준은 하나다. 다음 세션이 그것 없이 같은 결정을 다시 내릴 수 있는가.
- 길이 문턱을 세우려면 성능이 떨어지기 시작하는 지점을 자기 과제로 측정해야 한다. 측정하는 법은 제4장이, 세션을 넘는 기억의 설계는 제10장이 세워 둔 것이다.
- 검증의 기준을 어떻게 만들고 판정자 자신을 어떻게 평가하는지는 제6부가 받는다. 하네스를 바꾼 것이 성공률과 값을 바꾸었는지 평가하는 방법도 같은 곳이며, 평가하려면 실행의 기록이 남아야 하므로 제20장이 함께 받는다.
- 만드는 쪽과 판정하는 쪽을 나누는 것은 다중 에이전트의 한 형태다. 그 셈은 제16장이 받는다.
- 되돌릴 수 없는 행동 앞에 무엇을 세울지는 제21장이 받는다.
- 프레임워크가 대신 채워 보내는 줄이 무엇인지는 제23장이 짚는다.

오늘 할 것. 인계의 완료 항목마다 무엇으로 확인했는지를 적는 칸을 만든다. 비었는지는 사람이 아니라 검사가 본다(절 15.2). 지금 들고 있는 루프에서 종료의 곁을 센다. 자원 상한이 없으면 그것부터 박는다(절 15.3). 장치 하나를 골라 끄고 같은 과정을 돌려 본다. 결과가 같으면 잉여이고 좋아지면 방해였다(절 15.6).

15.8 생각해 볼 문제

1. 모델이 완료 선언으로 루프를 끝내는 설계를 분석하라. 이 설계에서 하네스가 정하던 넷 중 무엇이 모델로 넘어갔고, 그 대가는 이 장의 어느 절이 경고한 것인가.
2. 자르는 문턱을 너무 높게 잡으면 무엇이 나빠지고, 너무 낮게 잡으면 무엇이 나빠지는가. 인계를 쓰는 데 드는 스텝과 과제 경계에서 자르는 일까지 고려해, 좋은 문턱이 만족해야 할 조건을 말로 적어 보라.
3. 표 15.2는 같은 규칙을 네 자리에 얹은 것이다. 시험을 통과시키는 가장 짧은 길이 시험 자체를 고치는 것이라면, 네 자리 가운데 그 길을 막는 것은 어디이고 나머지는 왜 막지 못하는가. 각 자리가 무엇을 보는지로 답하라.
4. 표 15.3의 여섯 가정 각각에 대해, 자기 시스템에서 그 가정이 아직 유효한지 확인하는 실험을 하나씩 설계하라. 장치를 끄고 건주는 것 말고 다른 방법이 있는가.

15.9 다시 읽을 때

- 제2부의 장들을 그림 15.1 위에 겹치면, 인계 지침과 상태 요약과 규칙 목록이 전부 매 스텝 실리는 토끼이라는 것이 보인다. 하네스를 다듬는 일과 컨텍스트를 다듬는 일은 같은 예산의 양쪽 끝이다.
- 제6장을 읽고 인계를 고쳐 쓰는 대목으로 돌아오면, 인계의 크기를 붙들어 두는 일이 압축의 한 사례였음이 보인다. 원문을 버리고 결론을 남긴다는 기준이 인계문에 그대로 적용되고, 낡은 결론은 무관한 원문보다 나쁘다는 단서까지 함께 온다.
- 제10장을 읽고 산출물로 남기는 대목으로 돌아오면, 세션 사이를 잇는 산출물과 세션을 넘어 지속되는 기억이 다른 물건이라는 것이 보인다. 산출물은 과제가 끝나면 소임도 끝나지만, 기억은 과제를 넘어 쌓이고 고쳐 쓰기와 충돌이라는 자기 문제가 따른다.
- 제16장을 읽고 검증 절로 돌아오면, 만드는 쪽과 판정하는 쪽을 나눈 것이 이미 가장 작은 나뉜 시스템이었음이 보인다. 나뉜 자리 사이로 넘어가는 것도 세션 사이의 인계와 같은 규율을 받는다. 기록이 아니라 산출물이다.

15.10 더 읽을거리

하네스를 정면으로 다루는 문헌은 최근 늘고 있으나 아직 산업계의 기록이 앞선다. 장기 실행 에이전트를 위한 선행 자료로는 Anthropic (2025d)를, 그 후속으로 만드는 쪽과 판정하는 쪽의 분리까지 나아간 기록으로는 Rajasekaran (2026)를 보라.

후자는 네 결정이 실제 시스템에서 어떤 순서로 조립되는지, 특히 판정자를 만드는 일이 왜 기준을 만드는 일에서 시작하는지를 보여 준다. 모델이 나아졌을 때 어떤 장치를 도로 걷어냈는지도 함께 적혀 있어, 가정에 유효 기간이 있다는 이 장의 마지막 절과 나란히 읽을 값어치가 있다. 둘 다 산업계 기록이므로 수치보다 구조와 그 구조가 담고 선 가정을 읽기를 권한다.

학술 쪽 근거로는 Yang et al. (2024)를 보라. 소프트웨어 엔지니어링 영역의 결과이므로 다른 영역으로 옮길 때는 자기 영역에서 다시 확인해 봐야 한다.

단일 에이전트와 다중 에이전트

사내 규정을 묻는 문의 한 건이 들어왔다고 하자. 에이전트가 답을 만들어 내놓기까지 루프가 몇 바퀴 돈다. 그 한 바퀴 안에 이 책이 지금까지 다룬 것이 전부 들어 있다.

16.1 단일 에이전트의 운영

루프 한 바퀴는 여섯 대목으로 나뉘고, 앞의 장들은 그 여섯 대목에 하나씩 붙어 있다.

첫째 대목에서 컨텍스트를 조립한다. 문의 본문만 넣는 것이 아니다. 시스템 프롬프트가 들어가고, 이 에이전트가 부를 수 있는 도구의 정의가 전부 들어가고, 지금까지 돈 바퀴의 이력이 들어간다. 앞의 둘은 바퀴가 몇 번 돌든 그대로이므로 고정부라 하고, 이력은 바퀴마다 길어지므로 가변부라 한다. 무엇을 고정부에 적을지가 제5장이고, 가변부에 무엇을 언제 실을지가 제4장이다.

둘째 대목에서 모델을 부른다. 조립한 것을 읽히고 답을 받는다. 청구되는 것은 읽힌 토큰과 쓰인 토큰이고, 읽힌 쪽에는 방금 조립한 것이 통째로 들어간다. 이력이 길어질수록 매 바퀴 다시 읽히는 양이 커지므로 값은 바퀴 수에 비례하지 않고 그보다 빠르게 는다. 이 셈이 제3장이다. 다만 앞부분을 흔들지 않으면 다시 읽는 값이 크게 깎이므로, 고정부를 바퀴마다 고쳐 쓰지 않는 것만으로 청구서가 달라진다(그림 6.1).

셋째 대목에서 모델이 도구를 하나 고른다. 규정 문서를 뒤지는 도구를 고르고 인자를 채운다. 모델이 고를 수 있는 것은 정의에 적합한 것뿐이므로 도구의 이름과 설명이 곧 이 에이전트가 할 수 있는 일의 목록이다. 그 목록을 어떻게 쓰는가가 제7장이고, 남이 만든 도구를 이 목록에 붙이는 방법이 제8장이며, 돌려받는 것의 형식을 미리 고정하는 일이 제9장이다.

넷째 대목에서 도구가 밖에서 무언가를 가져온다. 가져올 곳은 여러 갈래다. 지난 세션에 적어 둔 것을 다시 꺼내는 길(제10장), 문서 더미에서 지금 물음에 걸리는 대목을 골라 오는 길(제11장), 골라 온 것들의 순서를 다시 매기는 일(제12장), 개체와 관계로 정리해 둔 것을 따라가는 길(제13장), 표와 그림으로 된 문서를 읽을 수 있는 형태로 바꾸는 일(제14장)이다.

다섯째 대목에서 가져온 것이 컨텍스트에 붙는다. 붙으면 다음 바퀴의 입력이 길어진다. 규정 문서 한 장이 통째로 붙으면 그 한 장은 이후 모든 바퀴에서 다시 읽힌다. 길이가 미리 측정해 둔 문턱을 넘으면 이력을 줄여야 하고, 줄이는 일이 손실을 낸다는 것과 무엇을 버릴지 미리 정해야 한다는 것이 제6장이다.

여섯째 대목에서 한 바퀴를 더 돌지 여기서 멈출지가 정해진다. 이 결정은 모델이 하는 것처럼 보이지만 실제로는 바깥의 코드가 쥐고 있다. 몇 바퀴까지 허락할지, 토큰을 얼마까지 쓰게 할지, 무엇을 완료로 볼지, 실패하면 어디서부터 다시 시작할지가 전부 그 코드에 적혀 있다. 그 코드를 하네스라 하고, 하네스가 무엇을 소유하는지가 제15장이다.

여기까지가 단일 에이전트다. 컨텍스트가 하나이고, 이력이 한 줄로 자라고, 상한이 한 벌이며, 무슨 일이 있었는지 물으면 그 컨텍스트를 열면 된다. 이 구성으로 얼마나 갈 수 있는지에 대한 기대는 대개 실제보다 낮게 잡혀 있다. 도구와 반환을 다시 설계하는 것만으로 풀리는 일을 구조를 갈아엎어 푸는 결정이 흔한 것도 그래서다.

이 고리가 실제로 잘 도는지 확인에 대한 부분은 아직 답하지는 않았다. 무엇을 성공이라 부를지, 성공률을 무엇으로 측정할지, 실행의 기록을 어떻게 남겨 어디서 틀렸는지 찾을지는 제6부가 받는다.

표 16.1: 고리 한 바퀴와 그 위에 앉는 것들. 왼쪽이 바퀴의 대목이고, 가운데가 그 대목에서 설계자가 실제로 정하는 것이며, 오른쪽이 그 결정을 다룬 장이다. 표의 요점은 오른쪽 열이 흩어져 있지 않다는 것이다. 앞선 열다섯 장이 서로 다른 주제를 다룬 것이 아니라 한 고리의 서로 다른 대목을 다루었다.

바퀴의 대목	설계자가 정하는 것	어디에 있는가
컨텍스트를 조립한다	고정부에 무엇을 적고, 가변부에 무엇을 언제 실을 것인가	제5장, 제4장
모델을 부른다	읽힌 토큰과 쓰인 토큰을 어떻게 셀 것인가. 앞부분을 흔들지 않을 것인가	제3장, 제6장
도구를 고른다	이름과 설명으로 무엇을 할 수 있게 할 것인가. 반환의 형식을 무엇으로 고정할 것인가	제7장, 제8장, 제9장
밖에서 가져온다	어디서 가져오고 무엇을 골라 실을 것인가	제10장부터 제14장까지
컨텍스트가 자란다	언제 줄이고 무엇을 버릴 것인가	제6장
한 바퀴를 더 돌지 정한다	상한과 종료와 재시작과 검증을 어떻게 걸 것인가	제15장

사람을 어디에 세우고 무엇까지 허락할지는 제7부다. 이 장이 묻는 것은 그 앞의 물을 하나다. 고리가 하나여야 하는가.

16.2 단일 에이전트로 부족할 경우

단일 에이전트로 부족한 경우가 있을까. 부족하다는 말은 대개 넷 중 하나로 온다. 느리다. 값이 많이 든다. 한 컨텍스트에 안 들어간다. 다른 일들이 섞여 결과값에 지장을 준다.

넷 전부에서 이미 답이 있다. 새로 배울 것도 아니고 구조를 바꾸는 것도 아니다. 먼저 해 볼 다섯이 표 16.2다.

표의 다섯을 모두 확인해보지 않고 다중 에이전트로 가는 결정은 구조를 바꾸어 풀 문제인지 확인하지 않은 채 구조부터 바꾸는 것이다. 그러면 다섯을 다 지나고도 안 풀리는 자리는 어디인가. 둘이다.

첫째, 추론을 개선했는데도 지연이나 병목이 그대로일 때다. 갈래를 동시에 돌렸는데도 느리다면, 갈래 하나하나가 짧지 않다는 뜻이다. 규정 문서를 뒤지는 갈래가 그 안에서 다시 열 바퀴를 돌아야 한다면, 그 갈래는 코드가 갈라 놓은 병렬 처리 하나가 아니라 스스로 도는 루프 하나다. 그런 갈래가 여럿이면 코드로 갈라 놓는 것으로는 부족하다. 갈래마다 자기 이력을 쌓아야 하고, 이력을 쌓는다는 것은 자기 컨텍스트를 갖는다는 것이다. 병렬과 나눔이 갈리는 자리가 여기도. 갈래가 호출 하나면 코드가 갈라도 되고, 갈래가 루프 하나면 컨텍스트를 갈라야 한다.

둘째, 역할이 정확하게 나뉘어야 할 때다. 만드는 쪽과 판정하는 쪽을 생각해 보자. 판정하는 쪽이 만드는 쪽의 과정을 보면 판정이 오염된다. 어떤 근거로 그렇게 썼는지를 읽고 나면 그 근거를 다시 검사하는 대신 그 근거에 설득되기 때문이다. 그래서 판정에 주는 것은 산출물과 기준뿐이어야 한다는 것이 제15장의 요구였다. 이 요구는 컨텍스트를 아끼려는 것이 아니다. 두 컨텍스트가 만나면 안 되기 때문에 나누는 것이고, 그래서 다섯 손 가운데 어느 것으로도 대신할 수 없다.

왜 대신할 수 없는지는 대신해 보려고 하면 드러난다. 같은 컨텍스트 안에서 마지막에 「이제 네가 쓴 것을 검사하라」고 시켜 보자. 검사하는 것은 방금 그 답을 쓴 쪽이고, 그 쪽의 컨텍스트에는 왜 그렇게 썼는지가 이미 다 적혀 있다. 자기가 방금 적은 근거를 다시 의심하는 일은 그 근거가 컨텍스트에 그대로 놓여 있는 한 잘 일어나지 않는다. 압축으로도 안 된다. 줄이는 쪽이 무엇을 남길지 고르므로 불리한 대목이 먼저 줄어든다. 라우팅으로도 안 된다. 어느 길로 보내든 열이 하나인 것은

표 16.2: 다중 에이전트로 가기 전에 지나야 하는 다섯. 전부 앞의 장들이 이미 세운 것이고, 전부 컨텍스트를 하나로 둔 채 쓸 수 있다. 오른쪽 열이 이 표의 요점이다. 다섯 다 값을 치르지만, 치르는 것이 호출 하나이거나 사람의 품이지 시스템의 구조가 아니다.

먼저 해 볼 것	무엇을 푸는가	무엇을 치르는가
라우팅	하나의 프롬프트가 모든 경우를 떠안다 무너지는 것. 환불 인지 기술 문제인지 먼저 갈라 전용 처리로 보낸다(제1장)	가르는 호출 하나. 잘못 갈랐을 때 되돌릴 길
병렬	서로 안 걸리는 갈래를 차례로 도느라 늘어지는 지연. 갈래를 미리 셀 수 있으면 코드가 갈라 동시에 돌린다(제1장)	갈래 수만큼의 호출. 합치는 코드
트리	초기 결정이 뒤를 다 가르는데 되돌릴 길이 없는 것. 후보를 버려 놓고 평가해 고른다(제2장)	노드 수만큼의 호출. 평가할 기준이 있어야 한다
압축과 자르기	이력이 쌓이기만 해서 컨텍스트에 안 들어가는 것. 문턱에서 줄이거나 과제 경계에서 자른다(제6장, 제15장)	버린 관측. 무엇을 버릴지 미리 정하는 품
도구를 다시 설계하기	반환이 너무 커서 한 번 부를 때마다 컨텍스트를 뒤덮는 것. 정의를 줄이고 반환을 자르고 큰 것은 밖에 둔다(제7장)	다시 쓰는 사람의 품

그대로다. 이 자리에서 필요한 것은 짧은 컨텍스트가 아니라 다른 컨텍스트다.

둘 밖에는 없다. 나머지 자리에서 다중 에이전트는 표 16.2의 다섯 가운데 하나를 더 정성껏 하는 것보다 나은 답이 아니다.

16.3 다중 에이전트

다중 에이전트는 컨텍스트를 나누는 일이다. 호출을 여럿 만드는 것이 아니다.

이 구별이 왜 중요한지는 이렇게 보면 안다. 만드는 쪽의 컨텍스트를 관정하는 쪽에 통째로 물려주고 마지막에 「이제 관정하라」고만 덧붙였다고 하자. 호출은 둘인데 나뉜 것은 없다. 두 번째 호출이 읽는 것은 첫 번째가 읽던 것에 한문장이 붙은 것뿐이다. 컨텍스트는 여전히 한 줄로 자라고, 토큰도 그대로 들고, 관정이 독립적이지도 않다.

반대도 성립한다. 하나의 모델이 두 컨텍스트를 번갈아 말으면 부르는 주체는 하나인데도 나뉜 구조의 이득과 곤란이 그대로 나타난다. 두 컨텍스트가 서로의 이력을 안 읽기 때문이다.

정의 16.1 (나눔과 다중 에이전트). 한 과제를 수행하는 동안 각자 자기 관측만 쌓는 컨텍스트가 둘 이상 있고, 어느 쪽도 다른 쪽의 이력을 통째로 물려받지 않으면 그 시스템은 나누어 있다고 하고, 그 구조를 다중 에이전트라 한다. 나눔의 단위는 호출이 아니라 컨텍스트다.

이 구별이 셈을 정한다. 호출을 세면 다중 에이전트와 긴 루프를 가릴 수 없다. 둘 다 호출이 여럿이기 때문이다. 제2장의 트리 탐색도 호출을 여럿 벌리지만 나뉜 것이 아니다. 갈래 하나하나가 뿌리까지의 이력을 그대로 물려받으므로 경계가 없다.

앞으로 나뉜 컨텍스트 하나하나를 자리라 부른다.

나누면 무엇이 좋아지는가

컨텍스트가 짧아진다. 그것이 전부다.

세 갈래의 조사를 한 루프로 하면 첫 갈래에서 읽은 문서가 셋째 갈래를 도는 내내 실린 채 따라다닌다. 마지막 바퀴가 읽는 것은 세 갈래의 관측을 전부 담은 이력이다.

세 갈래를 세 자리에 맡기면 각자가 읽는 것은 자기 갈래의 관측뿐이다. 아무도 전체를 보지 않는다는 것과 각자의 컨텍스트가 짧다는 것은 같은 사실의 두 이름이다.

그래서 잘게 나눌수록 좋은 것이 아니다. 스텝 하나에 자리 하나를 두면 각 자리는 앞 스텝이 무엇을 했는지 모른다. 알게 하려면 넘겨주어야 하고, 전부 넘기면 처음 상태가 곧 이력이 되어 원래의 한 줄로 되돌아간다.

짧아지는 정도는 자리의 수가 아니라 나뉜 쪽이 서로 몰라도 되는 관측의 양이 정한다.

그러니 나눌지를 정하는 물음은 「혼자 하기에 큰 과제인가」가 아니다. 「이 과제가 서로 몰라도 되는 부분으로 갈리는가」다. 앞의 물음에는 어떤 과제든 「예」라고 답할 수 있지만 뒤의 물음에는 그럴 수 없다.

나누면 무엇이 늘어나는가

주고받는 일이 생긴다. 나뉜 만큼 넘겨야 하고, 넘기는 것은 말이다. 한쪽이 컨텍스트에서 본 것을 요약해 넘기면 받는 쪽은 그 요약만 본다. 원본을 다시 볼 길은 없다.

어디서 틀렸는지 찾기 어려워진다. 답이 틀렸을 때 어느 자리에서 미끄러졌는지 가르려면 자리마다 이력이 따로 남아 있어야 한다.

토큰이 는다. 자리마다 자기 지시와 자기 도구 정의를 싣는다. 같은 것을 여러 별 무는 셈이다.

공개된 실무 기록이 이것을 구체적으로 적어 두었다(Anthropic, 2026f). 계획하는 자리와 구현하는 자리와 시험하는 자리로 가른 구성에서, 자리들끼리 서로 상태를 맞추는 데 쓴 토큰이 실제 일에 쓴 토큰보다 많았다.

그리고 여러 달을 들여 여럿으로 나눈 구성을 만들어 놓고, 나중에 단일 에이전트의 프롬프트를 손보는 것만으로 같은 성적이 나온다는 것을 발견한 팀들도 있었다(Anthropic, 2026f).

나눌 곳을 일의 이름이 아니라 서로 무엇을 알아야 하는가로 정해야 한다는 뜻이다. 계획과 구현과 시험은 서로를 알아야 하는 일들이다. 이름이 다르다고 컨텍스트가 갈리는 것이 아니다.

어떻게 운영하는가

자리를 여럿 두면 세 가지를 새로 정해야 한다. 누가 시작하는가, 무엇을 주고받는가, 언제 끝나는가.

시작하는 쪽의 구성은 크게 둘이다. 하나가 여럿을 부르는 구성에서는 중심 자리가 과제를 갈래로 나누고 갈래마다 자리를 열고 올라온 것을 모은다. 갈래의 수가 실행 전에 정해지면 코드가 나누어도 되고, 열어 봐야 아는 수라면 중심 자리가 나뉜다. 여럿이 서로 부르는 구성에서는 중심이 없고 자리가 다른 자리를 필요할 때 부른다. 뒤의 구성이 드문 데는 이유가 있다. 부른 자리가 또 부르고 그 자리가 또 부를 수 있으므로 자리의 수가 실행 중에야 정해지고, 예산을 실행 전에 적을 수 없다. 멈추는 일도 어려워진다. 앞의 구성에서는 중심 자리가 끝나면 끝나지만, 뒤의 구성에서는 누가 아직 도는지를 아는 자리가 없어 「전부 끝났는가」를 판정할 곳을 따로 만들어야 한다. 그래서 실무에서 혼한 것은 앞의 구성이고, 뒤의 구성을 고르는 것은 자리들이 무엇을 부를지 설계 시점에 정말로 열거할 수 없을 때뿐이다.

주고받는 것에서 중요한 것은 돌려받는 쪽이다. 모으는 자리가 받는 것은 각자의 요약이지 각자가 본 원문이 아니다. 이 무지는 사고가 아니라 설계다. 모으는 자리가 각자의 관측을 전부 안다면 그 컨텍스트에 모든 관측이 실린 것이고, 그것은 나누지 않은 것과 같다. 합침의 어려움은 나눔의 이득과 같은 뿌리에서 나온다. 그래서 이 값은 설계를 잘해서 없앨 수 있는 값이 아니다. 줄일 수는 있다. 각 자리가 남기는 것을 서사가 아니라 산출물로 두면 된다. 파일과 검사 결과는 다시 열어 볼 수 있으므로 두 자리가 어긋났을 때 그 어긋남이 주장의 충돌이 아니라 대조할 수 있는 사실의 충돌이 된다. 물려받은 것을 믿을 근거는 서술의 설득력이 아니라 재검사 가능성이어야 한다는 제15장의 기준이 여기서는 자리 사이를 잇는 규율이 된다.

합침을 맡을 곳을 정해 두지 않으면 그것이 저절로 생긴다. 나누는 결정만 내리고 합치는 결정을 내리지 않으면 합침은 마지막에 붙는 호출 하나로 떠밀리고, 그 호출의 컨텍스트에는 자리마다 올라온 요약이 순서 없이 쌓이며, 자리가 많을수록 길어진다. 나눔이 없었다고 믿은 긴 컨텍스트가 합치는

쪽에서 되살아난다. 그래서 합침의 입력은 쌓이는 것이 아니라 갱신되는 것이어야 하고, 무엇을 어떤 형식으로 올릴지는 각 자리의 재량이 아니라 하네스가 고정해야 한다. 재량에 맡기면 자리 수만큼의 서로 다른 문서가 올라오고, 합치는 쪽의 첫 일은 처음 보는 문서 여럿을 해독하는 일이 된다.

끝나는 것은 두 층이 된다. 자리 안의 정지 조건은 그 자리의 낭비만 막는다. 한 자리가 막다른 곳에 부딪혀 있는 동안 나머지 자리들은 그 사실을 모른 채 자기 예산을 끝까지 쓰고, 모으는 자리는 한 갈래가 빈손이라는 것을 모든 자리가 끝난 뒤에야 안다. 자리들을 가로지르는 정지 조건이 따로 있어야 이미 뜻을 잃은 갈래를 끊을 수 있다. 나눔을 들이는 결정에는 하네스를 하나 더 만드는 결정이 달려 온다.

16.4 실무에서의 적용

공개된 사례를 나란히 놓으면 앞 절의 판정이 실물로 확인된다. 이 절에서만 서비스의 이름을 그대로 쓴다.

나눈 것: 조사 서비스. Anthropic (2025e)은 Anthropic이 Claude의 조사 기능을 여럿으로 나누어 만든 경험을 적는다. 구성은 중심 자리 하나가 물음을 갈래로 나누고 갈래마다 자리를 열어 동시에 돌리는 것이며, 중심 자리에는 큰 모델을 갈래에는 작은 모델을 두었다. 자리마다 컨텍스트가 따로 있다. 오가는 것을 보면 앞 절의 샘플이 그대로 보인다. 중심 자리는 갈래마다 무엇을 알아 오라는 지시를 적어 내려보내고, 갈래는 자기 컨텍스트에서 뒤지고 읽고 추리다가 알아낸 것만 올려보낸다. 자기가 읽은 문서를 통째로 올리지 않는다. 올리면 중심 자리의 컨텍스트가 갈래들의 관측을 전부 담게 되어 나누지 않은 것과 같아지기 때문이다. 출처를 답에 붙이는 일은 또 다른 자리가 맡는다. 성적과 값이 나란히 적혀 있는 것이 이 기록의 값어치다. 내부 조사 평가에서 단일 에이전트 구성보다 90.2% 높은 성적이 나왔고, 같은 글이 토큰 사용을 함께 보고한다. 대화 한 번이 쓰는 토큰을 1이라 하면 단일 에이전트가 4쯤이고 나뉜 구조가 15쯤이다. 왜 나누었는지도 분명하다. 조사는 갈래가 서로 독립이고 갈래마다 문서를 여러 번 뒤져야 하므로, 절 16.2가 세운 첫째 자리다.

나누지 않은 것: 같은 일을 하는 다른 서비스. OpenAI의 Deep Research는 같은 종류의 조사를 단일 에이전트로 푼다. 하나의 모델이 하나의 컨텍스트에서 계획하고, 문서를 뒤지고, 막히면 되짚어 다른 길로 간다. 여러 단계를 밟는 조사를 한 갈래로 해내도록 학습으로 만든 것이다. 두 서비스를 나란히 놓으면 하나가 보인다. 구조를 정하는 것은 과제의 종류가 아니다. 같은 조사 과제가 한쪽에서는 나뉘었고 한쪽에서는 나뉘지 않았다. 갈린 것은 갈래를 동시에 벌려 지연을 개선하는 쪽을 골랐는가지지, 조사라는 일 자체가 아니다.

나누지 않은 것: 코딩 에이전트. Claude Code는 하나의 주 루프로 돈다. 모델을 부르고, 도구를 돌리고, 결과를 컨텍스트에 붙이고, 다시 부른다. 절 16.1의 여섯 대목 그대로다. 서브에이전트라는 장치가 있기는 한데 쓰는 자리가 좁다. 공식 문서가 적어 둔 기준은 이렇다. 결과가 일 하나가 검색 결과나 로그나 파일 내용으로 주 대화를 뒤덮을 것 같고 그 내용을 나중에 다시 볼 일이 없을 때, 그 일을 자기 컨텍스트에서 하고 요약만 돌려받는다. 나누는 이유가 능력이 아니라 컨텍스트라는 정의 16.1의 판정이 제품의 사용 기준으로 적혀 있는 것이다.

나눈 것: 검증을 맡는 자리. 도메인을 가리지 않고 잘 듣는 나눔이 하나 있다는 것이 Anthropic이 뒤에 따로 공개한 판단이다(Anthropic, 2026f). 산출물을 만든 자리와 그것을 시험하는 자리를 나누고, 시험하는 자리에는 산출물과 기준만 준다. 이 나눔이 잘 듣는 이유는 넘기는 양이 적어서다. 자리 사이로 옮겨야 하는 것이 적을수록 옮기다 읽는 것도 적다. 제15장이 검증에 대해 세운 요구가 그대로 다중 에이전트의 한 형태였다는 뜻이며, 절 16.2의 둘째 자리다. 덧붙는 조건도 함께 적혀 있다. 시험하는 자리에 무엇을 어디까지 시험하라고 적어 두지 않으면 그 자리는 대충 통과시킨다. 이 실패가 조용하다는 것이 문제다. 자주 통과시키는 검증자는 아무 신호도 내지 않으므로, 검증을 붙였다는 사실만 남고 검증이 실제로 무엇을 걸렀는지는 아무도 모른다. 그래서 이 나눔에는 걸러낸 것을 세는 일이 달려 온다.

그러면 자기 시스템이 나눌 자리에 왔는지는 무엇으로 아는가. 도구 쪽에서 오는 신호가 셋 있

표 16.3: 공개된 넷. 오른쪽 열이 이 표의 요점이다. 나눈 둘은 절 16.2가 세운 두 자리에 각각 하나씩 앉고, 나누지 않은 둘은 둘 중 어느 쪽도 아니다. 같은 조사 과제가 위의 두 줄에서 걸린 것도 함께 보라. 구조를 정한 것은 과제의 종류가 아니다.

사례	나누었는가	왜 그렇게 했는가	절 16.2의 어느 쪽인가
Claude의 조사 기능	나누었다	갈래가 서로 독립이고 갈래마다 문서를 여러 번 뒤져야 한다	첫째. 지연을 산다
Deep Research	나누지 않았다	여러 단계를 밟는 조사를 한 갈래로 해내도록 학습으로 만들었다	어느 쪽도 아니다
Claude Code	주 루프는 나누지 않았다	동시에 별릴 수 있는 부분이 적다. 앞의 결정을 알아야 다음을 고친다	어느 쪽도 아니다. 결가지 하나만 컨텍스트를 가른다
검증을 맡는 자리	나누었다	판정하는 쪽이 만드는 과정을 보면 판정이 오염된다	둘째. 독립성을 산다

다는 것이 Anthropic이 공개한 기준이다(Anthropic, 2026f). 도구가 스무 개 언저리를 넘어가면, 도구들이 서로 무관한 영역에 걸쳐 있는가. 도구를 하나 더했더니 이미 있던 도구를 고르는 정확도가 나빠졌는가. 셋 다 때 바퀴 함께 읽히는 정의 가운데 이 물음과 상관없는 것이 너무 많다는 말이다. 다만 셋이 걸렸다고 곧바로 나누지는 않는다. 정의를 전부 미리 신지 않고 필요할 때 찾아 쓰게 하는 길이 먼저다.

Anthropic (2025e)에 규모의 감각이 적혀 있다. 단순한 사실 하나를 확인하는 물음이라면 자리 하나에 도구 호출 서너 번에서 열 번이면 끝난다. 나뉜 구조를 들일 만한 과제는 그 반대쪽 끝, 곧 갈래를 동시에 여럿 벌려야 하고 한 컨텍스트에 다 들어가지 않을 만큼 읽을 것이 많고 복잡한 도구를 여럿 다뤄야 하는 과제다. 그리고 그런 과제에서도 기준은 하나 더 있다. 답 하나의 값어치가 토큰 배수를 치를 만한가다. 조사 서비스에서 그 배수를 치른 것은 사람이 만나질 걸려 찾을 것을 몇 분에 찾아 주기 때문이지, 나뉜 구조가 좋은 구조이기 때문이 아니다. 같은 배수를 문의 한 건에 답하는 일에 치르면 답은 조금 좋아지고 청구서는 여러 배가 된다.

자기 시스템을 표 16.3 옆에 놓는 방법도 여기서 나온다. 나누고 싶은 갈래를 적고, 갈래마다 도구를 몇 번 부르는지 세고, 갈래들이 서로의 관측을 얼마나 알아야 하는지 적는다. 갈래가 서너 번의 호출로 끝나면 나눌 것이 아니라 코드로 갈라 동시에 돌릴 것이다. 갈래가 서로의 관측을 절반 넘게 알아야 하면 나눌 것이 아니라 한 자리에서 돌릴 것이다. 셋을 다 적고도 나눌 자리로 남는 갈래는 대개 처음 적었던 것보다 적다.

넷을 나란히 놓으면 감각이 잡힌다. 나눈 두 사례는 갈래가 서로 독립이어서 동시에 별릴 수 있거나 두 컨텍스트가 만나면 안 되는 자리였고, 나누지 않은 두 사례는 그 밖의 전부다. 그리고 나누지 않은 쪽이 아무것도 안 한 것이 아니다. 도구를 다듬고 반환을 자르고 컨텍스트에 무엇을 실을지를 고르는 일을 아주 자세히 했다. 대부분의 시스템은 단일 에이전트에 도구와 추론을 자세히 나누어 붙인 것이다. 그러므로 자기 시스템을 이 넷 옆에 놓아볼 때 기대할 답은「나누지 않는다」다. 나누는 쪽에 서려면 절 16.2의 두 자리 가운데 어느 쪽인지를 말할 수 있어야 하고, 말할 수 없으면 아직 표 16.2의 다섯 가운데 하나가 남아 있다는 뜻이다.

16.5 요약

한 문장으로. 다중 에이전트는 호출을 여럿 만드는 일이 아니라 컨텍스트를 나누는 일이고, 그 결정은 갈래마다 자기 루프가 필요해 지연이 안 풀리거나 두 컨텍스트가 만나면 안 될 때에만 맞다.

형식으로 확립된 것.

- 단일 에이전트는 여섯 대목으로 돈다(표 16.1). 컨텍스트를 조립하고, 모델을 부르고, 도구를 고르고, 밖에서 가져오고, 컨텍스트가 자라고, 한 바퀴를 더 돌지 정한다. 여섯을 관통하는 물음은 하나다. 이번 바퀴에 무엇을 읽힐 것인가.
- 나누기 전에 지나야 하는 다섯이 있다(표 16.2). 라우팅, 병렬, 트리, 압축과 자르기, 도구를 다시 설계하기. 다섯은 전부 컨텍스트를 하나로 둔 채 쓸 수 있고, 치르는 것이 호출 하나이거나 사람의 품이지 시스템의 구조가 아니다.
- 다섯을 다 지나고도 남는 자리는 둘이다. 갈래 하나하나가 스스로 도는 루프여서 코드로 갈라 놓는 것으로는 지연이 안 풀릴 때, 그리고 두 컨텍스트가 만나면 안 될 때.
- 나눴을 단위는 호출이 아니라 컨텍스트다(정의 16.1). 이력을 통째로 물려주고 지시 한 줄을 붙인 두 호출은 나뉜 것이 아니고, 서로의 이력을 안 읽는 두 컨텍스트는 한 모델이 번갈아 맡더라도 나뉜 것이다. 제2장의 트리 탐색은 여기 들지 않는다. 갈래가 뿌리까지의 이력을 그대로 물려받기 때문이다.
- 나누면 좋아지는 것은 하나뿐이다. 컨텍스트가 짧아진다. 짧아지는 정도는 자리의 수가 아니라 나뉜 쪽이 서로 몰라도 되는 관측의 양이 정한다. 그래서 물음은「혼자 하기에 큰 과제인가」가 아니라「서로 몰라도 되는 부분으로 갈리는가」다.
- 나누면 셋이 는다. 주고받고 합치는 일이 생기고, 어디서 틀렸는지 찾기 어려워지고, 토론이 는다. 그래서 나눌 곳은 일의 이름이 아니라 서로 무엇을 알아야 하는가로 정해야 한다. 계획과 구현과 시험은 이름이 다를 뿐 서로를 알아야 하는 일들이다.
- 자리를 여럿 두면 누가 시작하고 무엇을 주고받고 언제 끝나는지를 새로 정해야 한다. 합칠 곳을 정해 두지 않으면 합침은 마지막 호출 하나로 떠밀려 긴 컨텍스트를 되살리고, 자리들을 가로지르는 정지 조건이 없으면 한 자리가 막힌 동안 나머지가 제 예산을 끝까지 쓴다.

아직 답하지 않은 것.

- 어떤 과제가 서로 몰라도 되는 부분으로 갈리는지를 나누기 전에 평가하는 방법이 없다. 갈래들이 무엇을 볼지는 돌려 보아야 하는데, 돌려 보려면 이미 나누어 두었어야 한다.
- 두 자리가 서로 다른 사실을 올렸고 둘 다 자기 자리에서는 검증을 통과했을 때 누가 어느 쪽을 고르는지에 답이 없다. 판정하는 자리는 어느 원문도 보지 않은 자리다.
- 나눴을 이득을 평가하려면 나누지 않은 실행과 견주어야 하는데 그 실행은 대개 존재하지 않는다.
- 각자에게 무엇을 주고 무엇을 돌려받을지, 자리들을 어떤 모양으로 엮을지는 제17장이 받는다.

오늘 할 것. 지금 도는 루프에서 도구 하나를 골라 그 반환이 몇 줄인지 센다. 다시 볼 일 없는 줄이 절반을 넘으면 나누기 전에 그것부터 자른다(절 16.2). 나눌 생각 중인 갈래를 하나 골라, 그 갈래가 호출 하나인지 루프 하나인지 적는다. 호출 하나면 컨텍스트가 아니라 코드를 가르면 된다(절 16.2). 그 갈래가 다른 갈래의 관측을 얼마나 알아야 하는지 적어 본다. 절반을 넘으면 나눌 자리가 아니다

(절 16.4).

16.6 생각해 볼 문제

1. 표 16.1의 여섯 대목 가운데 자기 시스템에서 가장 손을 안 댄 곳을 고르고, 거기에 남은 여유가 나눔으로 살 수 있는 것과 겹치는지 판정하라.
2. 정의 16.1에 따르면 하나의 모델이 두 컨텍스트를 번갈아 맡는 구조도 나뉘는 구조다. 이 판정이 정의의 결함인지 의미 있는 결과인지 논하라. 그 구조에서 나눔이 늘리는 셋 가운데 무엇이 줄고 무엇이 그대로인가.
3. 계획하는 자리와 구현하는 자리와 시험하는 자리로 가른 구성이 왜 조율에 더 많은 토큰을 썼는지를 「서로 무엇을 알아야 하는가」로 다시 적어 보라. 같은 과제를 이름이 아니라 컨텍스트를 기준으로 다시 가르면 자리가 몇이 되는가.
4. 절 16.3은 자리들을 가로지르는 정지 조건을 요구했다. 표 15.1의 다섯 갈래 각각을 자리들 위의 층에서 다시 세운다면 무엇을 관측해야 하는가. 다섯 가운데 자리 안에서만 관측할 수 있어서 위층으로 올릴 수 없는 것이 있는가.

16.7 다시 읽을 때

이 절은 문제가 아니라 다음 회독의 안내다. 뒤 장을 읽고 나면 이 장의 같은 문장이 다르게 읽힌다. 어디로 돌아오면 무엇이 새로 보이는지를 적어 둔다.

- 제17장을 읽고 절 16.3의 「어떻게 운영하는가」로 돌아오면, 누가 시작하고 무엇을 주고받는지가 두 자리 사이의 약속으로 펼쳐져 있다. 이 장이 답하지 못한 어긋남의 판정도 거기서는 누가 어느 자리에 서는가의 문제로 다시 묻어진다.
- 제6부를 읽고 「어디서 틀렸는지 찾기 어려워진다」는 한 줄로 돌아오면, 자리마다 흩어진 이력을 잇는 일이 무엇을 평가할 것인가의 설계 문제가 되어 있다. 나누지 않은 실행과 견줄 수 없다는 이 장의 미해결도 거기서는 무엇을 기준선으로 삼을 것인가의 문제가 된다.
- 제21장을 읽고 절 16.3으로 돌아오면, 나눔의 경계와 권한의 경계가 반드시 같은 곳에 있지는 않다는 것이 보인다. 컨텍스트 때문에 떼어 낸 자리에는 되돌릴 수 없는 행동이 애초에 없을 수 있다.
- 제6부와 제7부를 읽고 절 16.1로 돌아오면, 표 16.1의 오른쪽 열에 빈 자리가 둘 있었다는 것이 보인다. 이 고리가 잘 도는지 평가하는 일과 어디까지 허락할지 정하는 일이 그것이다.

16.8 더 읽을거리

셋 다 산업계의 공개 기록이다. 구성을 배우러 읽는 것이 아니라 셈을 확인하러 읽는 것이다.

Anthropic (2025e)이 중심 자료다. 가져올 것은 둘이다. 성적과 토큰 사용이 한 글에 나란히 적혀 있다는 것이 하나다. 성적만 실린 기록은 나눔을 능력의 문제로 읽게 하고, 값만 실린 기록은 낭비로 읽게 한다. 같은 컨텍스트를 공유해야 하거나 의존이 많은 영역은 맞지 않고 코딩 과제 대부분이 거기 들어간다는 판단이 둘이다.

Anthropic (2024a)은 그보다 앞선 자리를 다룬다. 나누기 전에 지나야 하는 다섯 가운데 라우팅과 병렬이 여기서 나오고, 중심 호출이 갈래를 나누고 모으는 구성과 만드는 호출과 평가하는 호출을 가르는 구성도 이름과 함께 적혀 있다.

Anthropic (2025a)은 도구가 많아졌을 때 나누지 않고 버티는 길을 적는다. 정의를 매 호출 전부 실행하지 않고 필요할 때 찾아 쓰게 하는 장치다.

에이전트 간 통신

제16장은 에이전트를 나눌 것인가에 답했다. 나눈이 늘려 주는 것은 능력이 아니라 컨텍스트이고, 대개는 나누지 않는 편이 낫다는 것이었다. 나누기로 했다면 그다음 물음이 이 장이다. 나뉜 것들은 서로 무엇을 어떻게 주고받는가.

여기서 하나를 더 넓힌다. 나누지 않은 시스템에도 같은 물음이 온다. 우리 에이전트는 하나인데 옆 팀의 에이전트가 이미 돌고 있고, 우리 과제의 한 대목을 그쪽이 이미 풀고 있는 경우다. 우리가 그 대목을 다시 만들지 않기로 했다면 넘길 것과 돌려받을 것을 정해야 한다.

같은 목록인데 사정이 다르다. 앞에서는 우리가 양쪽을 다 만들었고, 뒤에서는 그렇지 않다. 이 차이가 이 장의 거의 모든 판단을 가른다.

17.1 에이전트의 상호작용

컨텍스트가 둘뿐인 경우로 돌아가 보자. 제15장의 판정자 분리다. 산출물을 만드는 에이전트가 있고 그것을 판정하는 에이전트가 있다.

이 둘 사이에는 이미 정해진 것이 셋 있었다. 판정하는 쪽은 산출물과 기준을 받는다. 판정하는 쪽은 합격 여부를 돌려준다. 그 답이 오면 하네스가 루프를 끝낸다.

그런데 이 셋이 어디에 적혀 있었는가. 한군데도 아니다. 첫째는 판정자를 부르는 코드의 인자 목록에 있었고, 둘째는 판정자의 지시문에 있었으며, 셋째는 하네스의 분기에 있었다.

셋을 모아 읽으면 그것이 둘 사이의 약속이다. 그런데 세 곳에 흩어져 있는 동안에는 아무도 그것을 약속으로 읽지 않는다. 약속은 언제나 주는 쪽과 받는 쪽 양쪽에 걸쳐 있고, 양쪽을 나란히 놓고 본 사람이 없으면 어긋남은 돌려 보기 전에 드러나지 않는다.

정의 17.1 (에이전트 사이의 약속). 두 에이전트가 서로를 부를 때 양쪽의 동작이 함께 기대는 것, 곧 누가 누구를 부르고, 무엇을 주고, 무엇을 돌려받고, 언제 끝났다고 보는가를 약속이라 한다. 약속은 정해 두지 않아도 있다. 정해 두지 않으면 부르는 쪽의 코드와 양쪽의 지시문에 흩어져 적히고, 그때 약속의 수는 에이전트의 수가 아니라 서로 부르는 쌍의 수다.

이제 두 경우를 가른다. 이 장이 다루는 것들이 여기서 갈라진다.

하나를 우리가 나눈 것들끼리다. 컨텍스트를 가르기로 한 것이 우리고, 양쪽의 지시문도 양쪽이 쓰는 모델도 양쪽의 상한도 우리 것이다. 그러니 형식도 우리가 정한다. 받는 쪽이 무엇을 받으면 잘하는지 두 벌을 만들어 견줘 볼 수 있고, 어긋나면 고치기 편한 쪽을 고친다.

다른 하나는 이미 따로 서 있는 것들끼리다. 사내의 다른 팀이 만든 에이전트, 또는 밖에서 온 것이다. 형식을 우리 마음대로 못 정한다. 상대의 지시문을 못 보고, 상대가 어느 모델로 도는지도 모르고, 몇 걸음까지 도는지도 우리가 정하지 않는다. 고칠 수 있는 쪽이 하나뿐이다.

게다가 상대는 통보 없이 바뀐다. 제8장이 공급자의 개정에 대해 적은 사정이 여기서 한 겹 더 무거워진다. 도구가 바뀌면 돌아오는 형식이 달라지지만, 상대 에이전트가 바뀌면 같은 것을 넘겨도 다른 판단이 온다.

이 한 줄이 제8장과 이 장을 가른다. 도구를 끌어올 때 경계 너머에 있는 것은 함수다. 같은 인자에 같은 형식으로 답하고, 자기 판단으로 다른 일을 하지 않는다. 여기서 경계 너머에 있는 것은 스스로

루프를 돈다. 받은 것을 읽고 자기 계획을 세우고, 무엇을 더 볼지 자기가 정한다. 목록에서 하나를 부르는 일과 판단을 맡기는 일은 다른 물음이다.

17.2 무엇을 주고받는가

정할 것은 둘이다. 무엇을 주고, 무엇을 돌려받는가.

넘기는 것부터 본다. 새로 열린 에이전트는 우리 대화 이력을 못 본다. 실제 구현의 문서가 이것을 못 막아둔다. 넘어가는 것은 여는 호출에 적은 문자열 하나뿐이다. 그러니 상대가 필요로 하는 파일 경로도, 오류 문구도, 이미 내린 결정도 그 문자열 안에 직접 적어야 한다.

상대가 받는 나머지는 자기 시스템 프롬프트와 도구 목록인데, 그건 우리가 부르기 전에 이미 정해져 있다. 부를 때마다 우리가 정하는 것은 문자열 하나다.

그 문자열에 무엇을 적는가. 공개된 운영 기록 하나가 넷을 든다. 목표, 출력의 형식, 쓸 도구와 자료에 대한 안내, 그리고 어디까지가 이 에이전트의 몫인지다(Anthropic, 2025e).

빠뜨리기 쉬운 것은 마지막이다. 같은 기록에 빠뜨렸을 때 무슨 일이 생기는지가 적혀 있다. 하위 에이전트 하나가 2021년의 자동차 반도체 부족을 뒤지는 동안 다른 둘이 2025년의 공급망을 겹쳐 뒤졌다.

지시가 모호하면 셋 중 하나가 된다. 일을 겹쳐 하거나, 아무도 안 맡은 빈틈을 남기거나, 필요한 것을 못 찾는다. 그리고 셋 다 실패로 보이지 않는다. 겹쳐 한 일은 두 별의 결과로 올라오고, 빈틈은 아무 결과도 안 올리므로 아무 신호도 없다.

넷을 실제로 적으면 이 정도가 된다.

목표: 결제 실패율이 지난주 화요일부터 오른 원인을 하나로 좁힌다.

형식: 원인 후보를 셋 이하로. 후보마다 근거가 된 로그의 경로와 줄 번호를 붙인다. 확정할 수 없으면 확인할 수 없다고 적는다.

자료: 로그는 logs/ 아래에 날짜별로 있다. 배포 기록은 deploy.log.
지표 화면은 열지 않는다.

경계: 코드를 고치지 않는다. 원인 후보를 좁히는 데까지만 한다.
서른 걸음 안에 끝내고, 못 끝내면 거기까지 한 것을 올린다.

이제 돌려받는 것이다. 이쪽이 덜 보인다. 올라오는 것은 그 에이전트가 마지막에 쓴 메시지 하나뿐이고, 중간의 도구 호출과 파일 읽기는 그쪽 컨텍스트에 남는다. 그것이 나뉘어 늘려 주는 컨텍스트의 정체다.

그런데 같은 사정이 반대쪽으로도 작용한다. 부르는 쪽이 볼 수 있는 것도 그 메시지 하나뿐이다. 상대가 무엇을 근거로 그렇게 썼는지는 부르는 쪽에 없다.

그래서 돌려받는 것에는 근거로 돌아갈 길이 함께 있어야 한다. 결론 몇 줄에 그 결론이 담고 선 파일의 경로를 붙이는 데는 토큰이 거의 안 든다. 결론만 받은 쪽은 그것을 다시 검사할 수단이 없다. 앞의 과제 기술을 지킨 반환이 이렇게 온다.

```
{
  "status": "partial",
  "candidates": [
    { "cause": "재시도 간격이 배포와 함께 3초에서 30초로 바뀌었다",
      "evidence": ["deploy.log:412", "logs/2026-08-18.log:9331"] },
    { "cause": "카드사 한 곳의 응답이 늦어졌다",
      "evidence": ["logs/2026-08-18.log:11207"] }
  ],
  "not_done": "화요일 이전 로그가 보존 기간을 지나 없어졌다. 이전 주와
견주지 못했고, 후보 둘 가운데 어느 쪽이 큰지도 짐작하지 못했다."
}
```


표 17.1: 주고받을 때 정할 넷. 오른쪽 두 열이 이 표의 요점이다. 우리가 나눈 것들끼리는 어느 칸이든 양쪽에서 고칠 수 있어서 나중에 정해도 되는데, 이미 따로 서 있는 것들끼리는 고칠 수 있는 쪽이 우리 뿐이어서 정하지 않은 칸이 그대로 구멍이 된다.

정할 것	우리가 나눈 것들끼리	이미 따로 서 있는 것들끼리
과제 기술	목표와 형식과 자료와 경계. 넷 다 우리가 쓰고, 받는 쪽의 지시문과 함께 고칠 수 있다	상대의 지시문을 못 보므로 경계를 우리가 문장으로 못 박는 수밖에 없다
반환	하네스가 형식을 고정한다. 근거의 경로를 함께 올리게 한다	상대가 정한 형식으로 온다. 우리 쪽에서 대조하고 어긋나면 우리가 되묻는다
형식	스키마를 한 곳에 두고 양쪽이 그것을 읽는다(제9장)	상대의 것을 읽어 우리 스키마로 옮기는 자리가 하나 더 필요하다
못 한 것	반환에 칸을 만들고, 상한에 걸려 끊긴 경우는 하네스가 표시한다	칸이 없으면 만들 수 없다. 침묵과 실패를 우리가 구별할 수 없다

세 칸을 보라. status에 부분이라고 적혀 있어서 부르는 쪽이 이를 완결로 읽지 않는다. evidence에 경로와 줄 번호가 있어서 원문으로 돌아갈 수 있다. 그리고 not_done이 있어서 못 한 일이 안 한 일과 구별된다.

셋째 칸이 이 절에서 가장 값을 한다. 칸이 없으면 채울 수조차 없기 때문이다. 제9장이 스키마에 대해 세운 그대로인데, 여기서는 그 스키마를 읽는 것이 코드가 아니라 다음 에이전트의 컨텍스트다.

칸이 있어도 못 채우는 경우가 하나 남는다. 길을 상한에 걸려 끊긴 경우다. 그때는 아무것도 적을 기회를 못 갖는다. 실제 구현 하나는 이 자리를 하네스가 메운다. 상한에 걸리면 올라온 것을 「부분」으로 표시해서 부르는 쪽이 미완인 것을 알게 하고, 필요하면 이어 돌릴 수 있게 해 둔다. 표시가 없으면 끊긴 결과와 다 하고 온 결과가 부르는 쪽에서 똑같이 보인다.

네 줄을 관통하는 것이 하나 있다. 오른쪽 열에서 줄어드는 것은 정할 것의 수가 아니라 정하는 사람의 수다. 상대가 정하지 않은 칸은 없어지지 않고 우리 쪽에서 기본값으로 정해진다.

17.3 어떻게 통신하는가

주고받을 것을 정했으면 남는 것은 건네는 방식이다. 물음은 둘이다. 누가 부르는가, 그리고 부른 쪽이 기다리는가.

누가 부르는가

한쪽이 다른 쪽을 도구처럼 부르는 배치가 가장 흔하다. 여는 일이 도구 하나로 노출되고, 그 도구의 인자가 앞 절의 과제 기술이고, 그 도구의 반환이 상대가 마지막에 쓴 메시지다. 오가는 것을 적으면 두 줄이다.

```
부르는 쪽 -> 하위 에이전트 "목표: ... 형식: ... 자료: ... 경계: ..."
부르는 쪽 <- 하위 에이전트 {"status": "partial", "candidates": [...]}
```

두 줄뿐이라는 데 이 배치의 성질이 다 있다. 부르는 쪽이 상대의 생애를 쥐고 있어서 언제 시작하고 언제 거두는지가 한군데에 적힌다. 대신 상대는 도중에 되물을 수 없다. 받은 문자열로 부족하면 부족한 채로 답을 쓴다.

표 17.2: 건네는 방식 다섯. 오른쪽 열을 먼저 읽는 편이 낫다. 위로 갈수록 조율이 간단해지고 아래로 갈수록 기다리는 시간이 줄어드는데, 줄어드는 시간은 언제나 어딘가에 상태로 남는다.

방식	좋아지는 것	늘어나는 것
부르고 기다린다	순서가 코드에 그대로 보이고, 어디서 실패했는지 부르는 쪽에 남는다	전체가 가장 느린 쪽에 매인다
부르고 배경으로 돌린다	독립인 일이 동시에 돈다. 끝나는 시각이 합이 아니라 최댓값이 된다	아직 도는 수를 세는 곳이 필요하다. 상한이 없으면 계속 는다
이름으로 메시지를 보낸다	부르는 쪽과 불러가는 쪽이 고정되지 않는다. 도중에 되물을 수 있다	전부 끝났는지 아는 데가 없다. 서로 기다리는 배치가 가능해진다
공유된 곳에 적는다	받는 쪽이 언제 읽어도 되고, 같은 것을 여럿이 읽는다	완결의 판정과 동시에 고칠 때의 규칙을 따로 정해야 한다
손잡이나 웹훅으로 받는다	오래 도는 일에 연결을 붙들지 않는다. 밖에 있는 상대방에게는 이 길뿐이다	상태를 우리 쪽에도 두어야 하고, 두 상태가 어긋날 수 있다

다른 배치는 에이전트들이 이름을 갖고 서로에게 메시지를 보내는 것이다. 최근의 구현 하나는 이름으로 메시지를 보내는 도구와 지금 말 걸 수 있는 상대의 명단을 내놓는 도구를 함께 두고, 시작할 때 형제들의 명단을 첫 턴에 실어 준다. 여기서는 아무도 상대의 생애를 쥐고 있지 않다.

그래서 이 배치에는 규율이 하나 따라붙는다. 그 문서가 못 박아 둔 문장이 이것이다. 받는 쪽은 다른 에이전트에서 온 메시지를 사람의 허락으로 취급하지 않는다. 이름으로 말을 걸 수 있게 되면 「승인한다」는 문장도 그 길로 건낼 수 있기 때문이다.

기다리는가

부르고 결과가 올 때까지 멈추면 동기다. 조율은 간단해진다. 대신 전체가 가장 느린 쪽에 매인다. 앞 절의 그 운영 기록이 자기 시스템은 동기라고 적으면서, 그래서 병목이 생기고, 비동기로 옮기면 결과를 모으는 일과 상태를 맞추는 일과 실패를 알리는 일이 어려워진다고 이유까지 적어 두었다 (Anthropic, 2025e).

그런데 그 뒤로 기본값이 옮겨 갔다. 최근의 한 구현에서는 여는 호출이 배경 실행 여부를 적지 않으면 배경으로 돌고, 결과가 당장 필요할 때에만 부르는 쪽이 앞으로 돌린다. 기다리는 것이 더는 기본이 아니다.

기다리지 않기로 하면 결과를 어디선가 받아야 한다. 길이 둘이다.

하나는 서로를 부르지 않고 공유된 곳에 적어 두는 것이다. 뒤에 오는 쪽이 그것을 읽는다. 부름의 쌍이 없어지므로 약속도 없어진 것처럼 보이는데, 옮겨졌을 뿐이다. 무엇을 어떤 형식으로 적을지, 언제 적힌 것을 완결로 볼지, 둘이 같은 항목을 고치면 무엇이 남는지가 여전히 정해져 있어야 한다.

다른 하나는 손잡이를 받아 두었다가 나중에 물어보거나, 우리 쪽 웹훅으로 받는 것이다. 밖에 있는 상대와 주고받을 때는 사실상이 길뿐이다. 연결을 몇 시간씩 붙들고 있을 수 없기 때문이다.

다섯 줄 어디에도 이름이 붙어 있지 않은 것이 이 표의 요점이다. 파이프라인이니 감독자니 하는 이름들은 이 다섯 위의 조합이지 서로 다른 종류의 물건이 아니다. 감독하는 것이 하나 있고 그 아래가 있다는 그림은 첫째 줄로 지어도 그러지고 둘째 줄로 지어도 그러지는데, 두 시스템은 끝나는 시각도 상한을 두는 곳도 다르다. 다른 팀의 구조를 검토할 때 물을 것은 이름이 아니라 이 표의 왼쪽 열이다.

17.4 서로 모르는 것들끼리: A2A

앞 절은 우리가 양쪽을 다 만들었을 때의 이야기였다. 상대가 남의 것이면 먼저 막히는 데가 다르다. 어디로 보내야 하는지, 무엇을 할 줄 아는지, 무엇을 들고 가야 들여보내 주는지를 모른다. 주고받기 전에 알아내야 할 것이 있다.

이것을 정해 둔 규칙이 있다. Agent2Agent, 줄여서 A2A다. 2025년 4월에 구글이 공개했고 그해 6월에 리눅스 재단으로 넘어갔다. 한 회사에 매이지 않는 곳에서 관리하기 위해서다(Linux Foundation, 2025).

밑에 깔린 것은 새로 만든 게 없다. HTTP로 보내고, 규칙이 정한 세 가지 결합 가운데 하나를 쓴다. JSON-RPC와 gRPC와 HTTP+JSON이다. 진행 상황은 서버가 밀어 주는 이벤트로 받는다. 인증은 OAuth 2.0과 API 키와 상호 TLS 가운데 고른다(Agent2Agent Project, 2026).

상대를 어떻게 찾는가

규격은 에이전트마다 자기 소개를 정해진 경로에 올려 두게 한다. /.well-known/agent-card.json이다. 이 소개를 에이전트 카드라 한다.

앞 절의 그 로그 조사를 옆 팀이 이미 서비스로 돌리고 있다면 그 카드가 이렇게 생겼다.

```
GET https://logs.example.com/.well-known/agent-card.json
```

```
{
  "name": "log-search",
  "description": "기간과 조건을 받아 로그를 뒤지고, 맞는 줄의
    경로와 줄 번호를 돌려준다.",
  "version": "1.4.0",
  "supportedInterfaces": [
    { "url": "https://logs.example.com/a2a",
      "protocolBinding": "JSONRPC", "protocolVersion": "1.0" }
  ],
  "capabilities": { "streaming": true, "pushNotifications": true },
  "securitySchemes": {
    "sso": { "openIdConnectSecurityScheme":
      { "openIdConnectUrl": "https://sso.example.com/oidc" } }
  },
  "defaultInputModes": ["text/plain"],
  "defaultOutputModes": ["application/json"],
  "skills": [
    { "id": "search_range",
      "name": "기간 안에서 찾기",
      "description": "시작과 끝 시각, 그리고 찾을 문자열을 받는다.",
      "examples": [ "8월 18일 07시부터 09시까지 결재 실패 줄" ] }
  ]
}
```

카드에 무엇이 있는지가 요점이다. 이름과 설명이 있어서 이 상대가 무슨 일을 하는지 알고, 끝점이 있어서 어디로 보낼지 알고, 할 줄 아는 일의 목록이 있어서 내 과제가 그 안에 드는지 판정하고, 인증 방식이 있어서 무엇을 들고 가야 하는지 안다.

표 17.1의 오른쪽 열을 다시 보면 이 카드가 무엇인지 보인다. 상대가 정하지 않아서 우리가 기본 값으로 메워야 했던 칸들이 거기 미리 채워져 있다.

주고받는 단위와 그 상태

규격은 주고받는 단위를 과제라 부르고, 과제마다 식별자와 상태를 붙인다. 상태는 여덟 인덱스 셋으로 갈린다. 도는 중인 것, 무언가를 기다리며 멈춘 것, 더 움직이지 않는 것이다.

과제 하나가 도는 모습을 적으면 이렇다.

```
보낸다 SendMessage "8월 18일 07시부터 09시까지 결재 실패 줄을
                  찾아, 경로와 줄 번호를 돌려 달라."
받는다 task id=... state=TASK_STATE_SUBMITTED
받는다 task state=TASK_STATE_WORKING
받는다 task state=TASK_STATE_INPUT_REQUIRED
        "그 시각대 로그가 두 벌이다. 어느 쪽인가."
보낸다 SendMessage taskId=... "배포 뒤에 쌓인 쪽."
받는다 task state=TASK_STATE_WORKING
받는다 task state=TASK_STATE_COMPLETED
        artifacts: ["logs/2026-08-18.log:9331", ...]
```

왜 상태가 필요했는지가 이 흐름에 다 있다. 넷째 줄에서 상대가 멈추고 되묻는데, 이것은 실패가 아니다. 실패로 처리하면 우리는 답을 못 받고 상대는 헛일을 한 것이 된다. 그렇다고 도는 중으로 두면 우리는 계속 기다린다.

기다림에는 자기 이름이 있어야 한다. 상태에 「입력 필요」가 따로 있는 것이 그래서다.

MCP와 무엇이 다른가

이 규격이 제8장의 규격과 같리는 자리도 공식 문서에 적혀 있다. MCP는 에이전트와 도구와 데이터를 잇는 복잡도를 낮추는 데 집중하고, A2A는 에이전트가 서로 협력하게 하는 데 집중한다. 도구로서가 아니라 에이전트로서 주고받게 한다는 것이다.

같은 문서가 그 이유를 이렇게 적었다. A2A가 잘하는 것은 도구를 한 번 부르는 일을 넘어서는 주고받기다. 위의 흐름이 그 문장의 실물이다. 도구는 되묻지 않는다.

둘은 대체가 아니라 보완이다. 같은 문서가 든 그림이 이렇다. 응용 하나가 밖의 에이전트들과는 A2A로 말하고, 그 에이전트들은 각자 자기 도구에 MCP로 닿는다.

두 규격이 최근 같은 방향으로 움직였다는 것도 함께 보아야 한다. 도구 쪽 규격의 2026년 7월 개정판이 길게 도는 일을 위해 결과 대신 조회 손잡이를 돌려주는 확장을 넣었다. 부르고 기다리는 것이 도구에서도 더는 유일한 방식이 아니다. 표 17.2의 마지막 줄이 도구에도 열린 것이다.

A2A의 판은 자주 바뀐다. 세부를 외워 둘 값은 없다. 남는 것은 무엇을 정해 두어야 했느냐다. 상대를 찾는 길, 할 줄 아는 일의 목록, 인증, 주고받는 단위, 그리고 그 단위의 상태다.

우리 안에서 에이전트들을 이룰 때 이 다섯을 어디에도 안 적었다면, 안 적어도 되었던 것이 아니라 우리가 상대를 이미 알고 있었기 때문이다.

17.5 설계할 때 주의할 점

방식을 골랐으면 그 방식이 무엇을 정하지 않은 채로 두는지를 세어야 한다. 셋이 되풀이해서 걸린다.

끝을 누가 아는가

하나일 때 종료는 그 루프의 문제였다. 제15장은 하네스가 볼 수 있는 것이 다섯이므로 종료의 갈래도 다섯이라고 했다. 여럿이면 그 다섯이 전체를 감싸는 쪽으로 그대로 올라오지 않는다. 갈래는 볼 수 있는 것에서 나왔으므로, 보는 데가 바뀌면 갈래도 바뀐다.

표 15.1의 다섯 가운데 그대로 올라오는 것은 둘이다. 자원 상한은 안을 안 들여다보고도 관측된다. 외부 개입은 오히려 감싸는 쪽이 제자리다. 개입을 하나하나에 두면 사람이 그 수만큼의 창구를 지켜야 하기 때문이다.

나머지 셋은 아래에서 무언가를 올려 주어야 선다. 완료 판정은 감싸는 쪽이 볼 수 있는 것이 반환뿐이라 재료가 바뀌고, 길이 문턱은 합치는 쪽으로 옮겨 가 가장 늦게 걸리며, 진전 없음은 아예

서지 않는다. 같은 도구를 같은 인자로 다시 부르는 일이 기계적으로 세어지던 갈래인데, 감싸는 쪽이 보는 것은 끝난 뒤의 반환이고 안에서의 제자리걸음은 그쪽 하네스만 알기 때문이다.

그러니 전체를 가로지르는 정지 조건을 세우는 일은 감싸는 쪽에 코드를 더하는 일이 아니라 약속에 항목을 더하는 일이다. 그리고 그 항목만큼 합치는 쪽의 컨텍스트가 길어진다.

하나가 막혀 있다는 것을 다른 것들이 모르는 것도 사고가 아니다. 컨텍스트를 가른 설계가 요구한 바로 그것이다. 실패가 간하면 실패의 소식도 간힌다.

서로 기다리는 배치는 이보다 조용하다. 첫째가 둘째의 답을 기다리고 둘째가 첫째의 답을 기다리면 아무도 안 끝난다. 어느 쪽도 오류를 안 내므로 기록에는 둘 다 아직 도는 중이라고만 남는다. 표 17.2의 셋째 줄에서 이 배치가 열린다.

처방은 앞 절이 이미 보였다. 기다림에 이름을 붙이는 것이다. 우리가 만든 것들 사이에서도 같다. 무엇을 기다리는지를 반환의 칸으로 올리게 하면 된다.

틀렸을 때 누가 고치는가

부르는 쪽이 기대한 것과 불려간 쪽이 돌려준 것이 다를 때, 기대가 과했는지 이행이 부실했는지를 가르는 기준이 시스템 안에 없다. 하나일 때는 이 물음이 성립하지 않았다. 원문을 안 넣었으면서 원문에 근거한 결론을 기대하는 것은 한 사람 안의 앞뒤 안 맞음이고, 맞추는 일도 그 사람 몫이었다.

두 결정이 약속이 되는 것은 항목이 늘어서가 아니라 어긋남에 주인이 없어지기 때문이다. 한 사람이 맞추는 것이라면 어긋나면 맞추면 된다. 양쪽이 지키는 것이라면 누가 어겼는지를 먼저 정해야 하고, 그러려면 약속이 어딘가에 적혀 있어야 한다.

고치는 일이 양쪽에서 서로 다른 종류의 일이라는 것도 계산에 넣어야 한다. 부르는 쪽의 기대는 코드에 있어서 배포되는 순간 남김없이 반영된다. 불려간 쪽의 이행은 지시문에 있고, 지시문은 확률을 기울일 뿐이다.

그래서 개정은 한쪽에서는 즉시 끝나고 다른 쪽에서는 서서히 진행된다. 그 사이 구간에서 시스템은 옛 약속도 새 약속도 아닌 곳에 있다. 거기서 나온 실패는 기록에 새 약속의 실패로 남는데 실제로는 이행이 아직 옛 확률에 있는 것이다. 그 구간이 기록에 없으므로 팀은 개정을 되돌리거나 밀어붙이거나 둘 중 하나를 근거 없이 고른다. 개정 전후로 이행물을 따로 세어 두는 것이 방아다.

이미 따로 서 있는 것들끼리에서는 이 처방이 반으로 준다. 고칠 수 있는 쪽이 부르는 쪽 하나뿐이기 때문이다. 형식을 대조하는 자리도, 어긋났을 때 되묻는 자리도, 상대의 개정을 관측할 방법도 전부 우리 쪽에 선다. 마지막 것은 같은 것을 넘겼는데 다른 판단이 오기 시작한 시점을 세는 일이다.

형식을 언제 굳히는가

너무 일찍 굳히면 바꾸기 어렵고, 안 굳히면 매번 다르게 일한다. 실무에서 이 물음은 대개 하나로 나타난다. 넘길 문자열을 하네스가 고정할 것인가, 부르는 쪽이 그때그때 쓰게 할 것인가.

소유가 가장 조용히 넘어가는 자리가 여기서이다. 무슨 일인지 장면으로 보자.

코드에서는 한 줄이다. 도구 목록에 「하위 에이전트 열기」를 하나 더한다. 인자는 문자열 하나이고, 설명에는 「이 에이전트가 할 일을 적는다」라고 쓴다. 배포한다.

그 뒤로 어제 실행에서는 그 도구가 두 번 불렸고 오늘은 아홉 번 불렸다. 어제는 인자에 파일 경로가 셋 들어갔고 오늘은 하나도 안 들어갔다. 둘 다 정상 동작이다.

한 줄을 더하면서 두 가지를 넘긴 것이다. 몇 개를 열지가 실행 중에 정해지게 되었고, 절 17.2의 첫째 결정, 곧 무엇을 넘길지가 설계가 아니라 모델의 그때그때 판단이 되었다. 그리고 이 넘어감에는 신호가 없다. 도구를 하나 더한 것과 소유를 넘긴 것이 코드에서 같은 한 줄이기 때문이다.

이 배치가 저절로 나쁜 것은 아니다. 무엇이 필요한지를 가장 잘 아는 것은 지금 일을 하고 있는 쪽이다. 요점은 무엇이 늘어나느냐다. 넘길 문자열이 매번 달라지면 같은 과제의 두 실행에서 같은 하위 에이전트가 서로 다른 것을 받고, 그러면 그 성적이 에이전트의 성적인지 넘긴 문자열의 성적인지 알 수 없다.

여기서 혼한 오진이 나온다. 실행마다 값이 크게 흔들리면 걸음 상한과 동시 실행 수를 함께 조인다. 그런데 흔들림의 출처가 넘길 문자열이라면 수를 낮춰도 흔들림은 그대로다. 줄어드는 것은 과제를 푸는 능력뿐이다.

확인하는 방법이 진단보다 손이 덜 간다. 열 목록을 첫 호출이 한꺼번에 내놓게 하고 그 뒤로는 코드가 목록대로만 열게 한다. 소유는 모델에 두고 굳는 시점만 당기는 것이다. 흔들림이 그만큼 줄면 출처는 구조였고, 그대로면 안쪽이다.

굳히면 달려 오는 것이 셋이다. 몇 개가 열릴지 정해지므로 전체 상한을 미리 셀 수 있고, 실행 전에 있는 것만이 사람의 검토를 지나가며, 하나가 틀렸을 때 그것만 다시 돌릴 수 있다. 셋째에는 조건이 붙는다. 그것에 넘겼던 문자열을 다시 만들 수 있어야 한다. 어디에도 안 적히면 하나를 다시 돌리는 일이 실행 전체를 다시 돌리는 일이 된다.

굳히지 않기로 했다면 상한을 밖에 두는 수밖에 없다. 실제 구현들이 손잡이를 셋 둔다. 열린 것이 다시 여는 깊이, 동시에 도는 수, 그리고 실행 하나의 지출이다. 한 구현의 기본값은 깊이 세 층에 동시 실행 스무 개이고, 지출은 상한 없음이다. 셋째가 비어 있다는 것이 앞의 둘만으로는 총량이 안 매인다는 뜻이다.

여기까지 오지 말아야 할 때

세 형태가 오버엔지니어링이다.

1. 주고받는 것보다 주고받게 만드는 코드가 크다. 돌리고 부름이 한 방향이면 약속이 하나뿐이고, 그것을 맡을 층을 따로 세울 이유가 없다. 감독하는 것을 하나 세우는 일은 수를 하나 늘려 약속을 둘로 만드는 일이며, 얻는 것은 나중에 더 붙이기 쉬워진다는 기대뿐이다.
2. 한 번만 부르고 끝나는데 규칙을 세운다. A2A가 정한 것들은 상대를 모를 때 값을 한다. 상대를 찾아야 하고, 무엇을 할 줄 아는지 미리 알 수 없고, 오래 도는 일을 연결 없이 기다려야 할 때다. 우리가 만든 돌이 한 번 주고받고 끝나는 자리라면, 늘어나는 것은 상태를 두 곳에 두는 일뿐이다.
3. 서로 기다리기만 한다. 대등하게 주고받는 배치를 골랐는데 무엇을 부를지 설계 시점에 열거할 수 있다면, 그 배치가 늘린 것은 끝을 아는 데가 없다는 사실 하나다. 열거할 수 없을 때에만 고를 배치다.

셋 가운데 어느 것에 걸리는지가 두 경우에서 같린다. 우리가 나눈 것들끼리에서는 첫째가 혼하다. 양쪽을 다 만들 수 있으니 가운데에 층을 하나 더 세우는 데 아무 제약이 없고, 그 층은 둘일 때 세워도 여섯일 때 세운 것과 같은 모양이기 때문이다. 이미 따로 서 있는 것들끼리에서는 둘째가 혼하다. 규칙의 목록을 먼저 읽고 우리 배치를 그 목록에 맞추게 되는데, 그 목록은 상대를 모를 때를 위해 적힌 것이다. 상대의 이름과 끝점과 인증을 이미 알고 있다면 그 절반은 우리 문제가 아니다.

그래서 설계의 순서가 정해진다. 먼저 무엇을 넘기고 무엇을 돌려받을지를 문장으로 적는다(절 17.2). 적히지 않으면 방식을 고르는 일이 무엇을 고르는 일인지 알 수 없다. 그다음에 표 17.2의 오른쪽 열을 읽고, 그것을 지금 관측할 수단이 있는 줄만 후보로 남긴다. 수단이 없는 줄을 고르면 늘어난 것이 안 늘어난 것처럼 보인다.

마지막에 끝을 아는 데를 정한다. 셋 가운데 이것만 뒤로 미루면 나중에 붙일 수 없다. 끝을 아는 일에는 아래에서 무언가를 올려 주어야 하고, 무엇을 올릴지는 약속에 적히기 때문이다. 약속을 다 적은 뒤에 항목을 하나 더 넣는 일은 양쪽을 다시 여는 일이다.

셋을 관통하는 것은 제16장의 경고다. 나누지 않았으면 통신도 없다. 이 장에서 드는 것은 전부 나눔의 결정에 이미 달려 있던 것이고, 이 장이 한 일은 그것을 항목으로 편 것뿐이다.

17.6 요약

한 문장으로. 에이전트 사이의 약속은 정해 두지 않아도 있으므로 통신을 설계한다는 것은 그 약속을 어디에 적을지 고르는 일이고, 우리가 나눈 것들끼리인가 이미 따로 서 있는 것들끼리인가에 따라 고칠 수 있는 쪽의 수가 달라진다.

확실해진 것.

- 약속은 정해 두지 않아도 있고, 정해 두지 않으면 부르는 쪽의 코드와 양쪽의 지시문에 흩어져 적힌다(정의 17.1). 약속의 수는 에이전트의 수가 아니라 서로 부르는 쌍의 수다.
- 두 경우의 차이는 고칠 수 있는 쪽의 수다. 우리가 나눈 것들끼리는 둘, 이미 따로 서 있는 것들끼리는 하나다. 상대가 정하지 않은 칸은 우리 쪽에서 기본값으로 정해진다(표 17.1).
- 넘기는 것은 목표와 형식과 자료와 경계 넷이고, 돌려받는 것에는 근거로 돌아갈 경로와 못 한 것을 적을 칸이 있어야 한다(절 17.2). 칸이 없으면 채울 수조차 없다.
- 건네는 방식은 다섯이고 이름이 붙어 있지 않다(표 17.2). 같은 그림이 서로 다른 줄로 지어지므로, 다른 팀의 구조에서 물을 것은 이름이 아니라 왼쪽 열이다.
- 서로 모르는 것들끼리 주고받으려면 상대를 찾는 길, 할 줄 아는 일의 목록, 인증, 주고 받는 단위, 그 단위의 상태가 정해져 있어야 한다. A2A가 그 다섯을 정해 둔 규격이고, 도구를 공급하는 규격과 달리 상대의 안을 열지 않는다(절 17.4).
- 종료의 다섯 갈래 가운데 안을 안 보고도 서는 것은 자원 상한과 외부 개입뿐이다(표 15.1). 나머지 셋은 코드가 아니라 약속의 항목으로 선다.
- 소유가 가장 조용히 넘어가는 것은 넘길 문자열이다. 하위 에이전트를 여는 일이 도구 하나로 노출되면 그 도구의 인자가 곧 넘길 문자열이고, 몇 개를 열지도 실행 중에 정해진다.

아직 답하지 않은 것.

- 넘길 문자열에 무엇을 담아야 하는지에 정답이 없다. 전부 주면 나뉘이 늘려 줄 것이 없고, 적게 주면 상대가 못 본 것 위에서 틀리는데 그 실패는 조용하다.
- 둘이 올린 것이 어긋날 때 무엇을 근거로 고를지에 여전히 답이 없다. 약속이 적혀 있으면 관정할 자리를 만들 수는 있다는 것까지가 이 장이 더한 전부다.
- 진전을 올리는 항목을 약속에 넣을 때 무엇을 얼마나 자주 올릴지에 정답이 없다. 안 올리면 진전 없음이 서지 않고, 올릴수록 합치는 쪽의 컨텍스트가 자란다.
- 약속 개정이 양쪽에서 서로 다른 속도로 반영된다는 사실은 적었지만, 그 어긋난 구간을 관측하는 방법은 적지 못했다.

오늘 할 것. 지금 하위 에이전트를 여는 호출 하나를 골라 그 인자로 실제로 넘어간 문자열을 로그에서 꺼내 읽는다. 목표와 형식과 자료와 경계 넷 가운데 몇이 적혀 있는지 센다(절 17.2). 거기서 올라온 반환을 열어, 근거로 돌아갈 경로와 못 한 것을 적을 칸이 있는지 확인한다. 없으면 칸부터 만든다(절 17.2).

17.7 생각해 볼 문제

1. 자기 시스템에서 서로 부르는 쌍을 전부 세고, 쌍마다 표 17.1의 네 항목이 어디에 적혀 있는지 찾아보라. 같은 상대를 부르는 두 쌍이 다른 값을 갖는 항목이 있는가.

2. 하위 에이전트 하나가 마지막에 올린 반환을 열어, 그 결론을 부르는 쪽이 원문으로 되짚을 수 있는지 판정하라. 되짚을 수 없다면 무엇을 반환에 더해야 하고, 그것이 부르는 쪽의 컨텍스트를 몇 줄 늘리는가.
3. 자기 시스템의 에이전트 하나에 대해 에이전트 카드를 적어 보라. 이름과 설명과 끝점과 할 줄 아는 일의 목록과 인증이다. 다섯 가운데 지금 어디에도 안 적혀 있는 것이 몇인가.
4. 절 17.5는 종료의 다섯 갈래 가운데 셋이 전체를 감싸는 쪽에서 서려면 약속에 항목이 더해져야 한다고 했다. 셋 각각의 항목을 적고, 셋을 다 넣었을 때 합치는 쪽의 컨텍스트가 얼마나 자라는지 에이전트의 수와 올리는 빈도로 적어 보라.

17.8 다시 읽을 때

- 제6부를 읽고 절 17.5로 돌아오면, 구조를 바꾸면 그때까지 쌓은 측정이 함께 만료된다는 관찰이 평가 설계의 문제로서 있다. 아래에서 올리는 항목은 제20장에서 무엇을 기록에 남길 것인가의 대상이 되어 있다.
- 제22장을 읽고 정의 17.1로 돌아오면, 밖에서 받은 내용이 반환에 실려 다른 에이전트에게 넘어가는 경로가 약속을 정확히 지키면서 열린다는 것이 보인다. 이 장이 적은 두 경우 가운데 뒤엎듯이 거기서 다시 서는 자리다.
- 제21장을 읽고 절 17.5로 돌아오면, 「누가 부르는가」의 소유가 권한의 언어로 다시 적힌다. 실행 전에 없는 것의 권한은 미리 적을 수 없다.

17.9 더 읽을거리

Agent2Agent Project (2026)이 이 장의 새 자료다. 절 17.2에서 우리가 손으로 정한 것들이 거기서는 규칙의 항목으로서 있다. 읽을 때는 셋이다. 에이전트 카드, 과제의 상태, 그리고 설계 원칙. 판이 자주 바뀌므로 세부는 흘러도 되는데, 상대의 안을 열지 않는다는 원칙은 남는다. Linux Foundation (2025)는 그 규칙이 어디서 관리되는지를 적은 한 쪽짜리 기록이다.

Anthropic (2025e)이 이 장의 중심 자료다. 절 17.2의 넷이 거기서 나왔고, 그 넷을 빠뜨렸을 때의 관측도 거기 있다. 동기 실행이 조율을 간단하게 하면서 병목을 만든다는 판단과, 나눈 구성의 토론 사용이 대화의 열다섯 배라는 셈도 같은 글에 있다.

Anthropic (2024a)는 그보다 앞선 자리를 다룬다. 중심 호출이 갈래를 나누고 모으는 구성과 앞의 결과가 뒤의 입력이 되는 구성이 이름과 함께 적혀 있어서, 표 17.2의 왼쪽 열이 그 이름들 아래에서 어떻게 갈리는지 견주어 볼 수 있다.

Anthropic (2025c)은 반환에 관한 자료다. 자기 일의 요약만 올리고 중간의 관측을 안에 두는 것이 왜 기본형인지가 컨텍스트의 셈으로 적혀 있다. 절 17.2에서 근거의 경로를 함께 올리라고 한 것은 이 기본형의 반대가 아니라 그것이 남기는 구멍을 메우는 것이다.

제5부는 세 장이었다. 제15장이 루프 하나를 바깥에서 소유하는 일을 세웠고, 그 위에서 뒤의 두 장이 하나의 논증이었다.

제16장은 나눌 것인가를 물었고, 이 장은 나눈 것들 사이를 물었다. 두 장이 남긴 미해결이 같다. 나누지 않은 실행과 견줄 수 없고, 다섯 방식을 견준 자료도 없다. 둘 다 평가의 문제이고, 제6부가 그 일로 넘어간다.

제 VI 부

평가와 관측

에이전트를 어떻게 평가할 것인가

배포를 앞두고 회의실에서 나오는 물음은 대개 하나다. 이거 잘 돌아갑니까.

답하기가 생각보다 어렵다. 지난주에 스무 건을 돌려 보니 열여섯 건이 됐다고 말할 수는 있다. 그런데 그 열여섯 건이 어떤 질문이었는지, 나머지 넷이 왜 안 됐는지, 오늘 프롬프트를 고치면 그 수가 올라갈지는 아무도 모른다. 열여섯이라는 수는 그날 그 스무 건에서만 참이다.

앞의 열일곱 장은 만드는 이야기였다. 이 장부터 시작하는 제6부는 만든 것이 잘 돌아가는지 확인하는 이야기다.

18.1 설계 그 이후에는

설계를 잘하는 것만으로는 부족하다. 확인할 수단이 없으면 다음에 무엇을 고칠지 고를 수 없다.

앞 장들을 넘겨 보면 같은 데서 멈춘 대목이 여럿이다. 프롬프트에서 예시를 셋에서 다섯으로 늘렸을 때, 검색에 재순위화를 붙였을 때, 하나로 돌던 루프를 둘로 나눴을 때, 스키마를 조여 형식 위반을 줄였을 때. 넷 다 좋아졌는지를 묻는 데서 멈췄다.

멈춘 이유도 같다. 무엇을 좋아졌다고 부를지, 누가 판정할지, 어떤 과제를 사례로 삼을지가 정해져 있지 않기 때문이다.

미룬 데는 이유가 있다. 확인하는 일이 까다롭다.

우선 같은 입력을 두 번 넣어도 같은 것이 나오지 않는다. 어제 되던 것이 오늘 안 되고, 오늘 안 되던 것이 내일 된다. 한 번 돌려 보고 좋아졌다고 말하는 것은 동전을 한 번 던져 보고 앞면이 잘 나오는 동전이라고 말하는 것과 같다.

그리고 에이전트는 여러 걸음을 걷는다. 스무 걸음을 걸어 답 하나를 내놓았다면, 답이 맞았는지만 보아서는 스무 걸음 가운데 어디가 잘못됐는지 알 수 없다. 답이 맞았는데도 열다섯 걸음이 헛걸음이었을 수 있다.

그래서 이 장은 순서대로 간다. 무엇이 좋은 에이전트인지 먼저 정하고, 거기서 평가 항목을 끌어내고, 그 항목을 실제로 어떻게 세는지 보고, 공개된 벤치마크가 그 일을 어떻게 하는지 확인한다.

18.2 좋은 에이전트란

평가 항목을 고르기 전에 정할 것이 있다. 무엇이 좋은 에이전트인가.

에이전트는 사람의 일을 통째로 넘겨받지 않는다. 옆에서 거드는 쪽에 가깝다.

Claude Code가 그 예다. 코드를 쓰고 파일을 고치고 시험을 돌린다. 그런데 고친 것을 그대로 내보내지 않는다. 사람이 읽고 받아들일지 정한다.

사람이 읽는 것은 코드 전체가 아니라 무엇이 어떻게 달라졌는지도. 고친 데를 모아 보여 주면 확인이 몇 분에 끝난다. 다 고쳐 놓고 끝났다고만 하면 처음부터 읽어야 한다. 나온 코드가 같아도 사람 쪽에 남는 일이 다르다.

이것은 아직 모델이 미답지 못해서 임시로 붙여 둔 절차가 아니다. 모델은 답을 확률로 뽑는다. 스무 번 돌리면 그중 한두 번은 다른 것이 나오고, 어느 번째가 그 한두 번일지는 미리 알 수 없다. 미리 알 수 있다면 그건 이미 고칠 수 있는 버그다. 그래서 마지막 승인은 사람 쪽에 남는다.

모델의 성능은 계속 좋아진다. 맡길 수 있는 일도 늘어난다. 그런데 좋아지면서 없어지는 것은 사람의 확인이 아니라 사람이 확인해야 하는 양이다.

그러면 좋은 에이전트란 무엇인가. 혼자 다 하는 에이전트가 아니라, 사람이 확인하는 품을 줄여 주는 에이전트다.

목표를 승인 절차를 없애는 데 두면 방향이 어긋난다. 승인이 있어야 할 곳에서는 계속 있어야 하고, 좋아지는 것은 승인 한 번에 드는 시간이다. 열 줄을 읽고 누르던 것을 두 줄만 읽고 누르게 되면 그만큼 좋아진 것이다.

여기서 평가 항목이 따라 나온다. 두 에이전트를 놓고 보자. 둘 다 열 건 가운데 여덟 건을 맞혔다. 첫째는 어느 둘이 틀렸는지 표시하지 않는다. 사람은 열 건을 다 읽어야 한다. 둘째는 확신이 없는 둘을 따로 표시해 둔다. 사람은 그 둘만 보면 된다.

성공률은 같은데 줄여 준 품은 다섯 배 차이다. 항목이 하나뿐이면 이 차이가 보이지 않는다.

핵심. 모델이 답을 확률로 뽑는 한 마지막 승인은 사람 쪽에 남는다. 그러므로 좋은 에이전트는 혼자 다 하는 에이전트가 아니라 사람이 확인하는 품을 줄여 주는 에이전트이고, 평가 항목에는 몇 퍼센트 맞혔는가와 함께 사람이 얼마나 손봐야 했는가가 들어가야 한다.

18.3 무엇을 평가하는가

가장 먼저 볼 것은 일이 끝났는가다. 그런데 끝났다는 말을 어디서 확인하느냐가 갈린다.

항공권을 예약하는 에이전트가 있다고 하자. 마지막 메시지에 「예약을 완료했습니다」라고 적혀 있다. 이것으로는 아무것도 알 수 없다. 예약 기록을 조회해서 그 표가 실제로 들어가 있는지 보아야 한다(Anthropic, 2026c). 둘이 어긋나는 실행이 실제로 나온다.

그래서 평가는 말이 아니라 남은 것을 본다. 파일이 고쳐졌는지, 시험이 통과하는지, 예약이 들어갔는지다.

같은 자료가 하나를 더 권한다. 무엇을 내놓았는지를 채점하고 어떤 길로 갔는지는 채점하지 말라고 한다. 옳은 길을 미리 적어 두고 그대로 갔는지 보면, 더 짧은 길로 같은 답에 이른 실행이 틀린 것으로 세어진다.

다섯 가운데 마지막이 가장 자주 빠진다. 세기가 번거롭기 때문이다. 앞의 넷은 실행이 끝나면 저절로 나오는데, 마지막은 사람이 무엇을 했는지를 따로 남겨야 나온다.

그런데 앞 절에서 좋은 에이전트를 사람이 확인하는 품을 줄여 주는 에이전트라고 했다. 그 항목을 빼면 좋다고 부르기로 한 것을 세지 않는 셈이 된다.

실무에서 이 항목은 대개 둘로 나뉜다. 그대로 받아들이 비율이 하나이고, 받아들이기 전에 고친 양이 둘이다. 앞의 것이 높은데 뒤의 것도 높으면, 사람이 받아들이고 있는 것이 아니라 받아들이 뒤에 다시 쓰고 있다.

세는 자리는 대개 정해져 있다. 사람이 결과를 받아들이는 버튼이 있으면 거기서 나오고, 없으면 그 버튼부터 만들어야 나온다. 코드를 고치는 에이전트라면 내놓은 차이가 그대로 들어갔는지 사람이 손을 댔는지가 남고, 문서에 답하는 에이전트라면 답을 그대로 보냈는지 고쳐 보냈는지가 남는다. 실행이 끝나는 데가 아니라 사람이 결정하는 데에 붙인다.

셋째 항목도 넘기기 쉽다. 그런데 이걸 답이 틀렸을 때 어디를 볼지 알려 준다. 틀렸어야 할 검색을 아예 안 불렀다면 고칠 데는 도구 설명문이고, 불렀는데도 틀렸다면 고칠 데는 검색이나 프롬프트다. 성공률에서는 같은 한 건인데 처방이 다르다.

항목을 고르는 일에는 딸려 오는 것이 하나 있다. 항목이 곧 실패의 이름이 된다. 항목에 없는 실패는 이름이 없고, 이름이 없으면 고쳐야 할 것의 목록에 오르지 않는다.

표 18.1: 평가 항목 다섯. 어느 하나로도 전부를 볼 수 없다. 못 잡는 것을 알고 고르는 편이 항목을 늘리는 것보다 낫다. 마지막 줄이 앞 절에서 끌어낸 항목이다.

항목	잡는 것	못 잡는 것
일이 끝났는가	환경에 남은 것. 예약 기록, 고쳐진 파일, 통과한 시험	같은 답에 세 걸음으로 간 실행과 서른 걸음으로 간 실행이 같은 칸에 들어간다
답이 맞는가	내용의 옳음. 미리 적어 둔 답과 대조하거나 판정자가 읽는다	우연히 맞은 답과 근거 위에 선 답이 같은 칸에 들어간다
도구를 골랐는가	불렸어야 할 것을 불렀는지, 부르지 말았어야 할 것을 참았는지	순서까지 맞추라고 하면 다른 길로 같은 답에 이른 실행이 실패로 세어진다
얼마나 들었는가	걸음 수, 토큰, 걸린 시간	적게 쓰고 틀린 답이 많이 쓰고 맞은 답보다 좋아 보인다
사람이 손봤는가	그대로 받아들이 비율, 고친 줄 수, 다시 시킨 횟수	손보지 않고 통째로 버린 실행은 아무 데도 세어지지 않는다

개선 회의가 모델을 바꿔 볼가와 프롬프트를 손볼가 사이만 오가는 팀이 많다. 다른 후보가 없어서 아니다. 다른 후보를 가리킬 이름이 없어서다. 찾아온 조각 가운데 답에 실제로 쓰인 것이 몇 개인지를 세고 있었다면 검색을 손보자는 말이 회의에 오를 수 있다.

18.4 어떻게 평가하는가

항목을 골랐으면 돌려야 한다. 정할 것이 넷이다. 무엇을 사례로 삼는가, 누가 판정하는가, 몇 번 돌리는가, 언제 나아졌다고 말하는가.

정의 18.1 (평가 과제). 입력과 성공 기준을 실행 전에 정해 둔 한 벌의 시험을 평가 과제라 한다. 그 시험을 한 번 돌리는 것이 시행이고, 시행이 남긴 것을 성공 기준에 대고 통과와 실패로 가르는 일이 판정이다.

무엇을 사례로 삼는가

수백 개를 모아야 시작할 수 있다고 여겨 미루는 팀이 많다. 그럴 필요가 없다. 실제 실패에서 뽑은 스무 개에서 선 개면 훌륭한 출발이라고 Anthropic (2026c)이 적는다.

근거가 함께 적혀 있고 가져갈 것은 그 근거다. 초기에는 한 번 고칠 때마다 결과가 크게 움직인다. 크게 움직이는 것을 확인하는 데는 사례가 많이 필요하지 않다. 뒤집으면, 시스템이 여물어 한 번의 개선이 작아지면 그때는 사례를 늘려야 한다.

좋은 사례인지 가리는 방법은 하나다. 두 사람에게 같은 산출물을 보였을 때 둘이 따로 같은 판정에 이르는가. 갈리면 판정하는 사람이 나쁜 것이 아니라 기준이 덜 적힌 것이다.

사례는 버그 목록과 문의 기록에서 가져온다. 이미 실패로 적혀 있는 것들이다.

손으로 지어야 할 때는 요령이 하나 있다. 답부터 잡고 문제를 거꾸로 만든다. 웹을 뒤지는 에이전트를 평가하는 한 벌이 그렇게 만들어졌다. 먼저 확실히 아는 사실을 하나 고르고, 그 사실을 검색으로 찾기 어렵게 만드는 조건을 몇 개 붙여 질문을 짓는다. 그러면 답은 짧고 확인하기 쉬운데 찾아내기는 어려운 문제가 나온다(Wei et al., 2025).

되어야 할 것이 되는 사례만 모으면 안 된다. 거절해야 할 요청, 답이 없는 질문, 자료에 없는 사실을 묻는 질문을 함께 넣는다.

표 18.2: 판정자 셋. 우열이 아니라 잘하는 것이 서로 다르다. 하나로 전부를 감당하려는 설계가 저절로 진다.

판정자	잘하는 것	못하는 것
코드	빠르고 값이 거의 안 든다. 같은 산출물에 언제나 같은 판정이 나온다	뜻을 못 본다. 96.124991 이라고 적어야 하는데 96.12 라고 적었다고 실패로 판정한다
모델	뜻을 본다. 규정으로 적기 어려운 기준을 말로 적을 수 있고 사례 수만큼 돌릴 수 있다	같은 산출물에 다른 판정이 나온다. 판정 자체가 매번 뿔히는 것이라 흔들린다
사람	가장 믿을 만하다. 다투는 데서 최종 판정이 되고 다른 둘을 맞춰 볼 기준이 된다	느리고 사람이 많이 든다. 전문가를 사례 수만큼 확보 할 수 없다

집합을 만든 뒤에는 지켜야 한다. 사례를 고칠 때마다 무엇을 왜 고쳤는지 남긴다. 남기지 않으면 지난달의 성공률과 이번 달의 성공률이 같은 시험의 결과인지 알 수 없다.

누가 판정하는가

판정할 수 있는 것은 셋뿐이다. 코드가 대조하거나, 모델이 읽거나, 사람이 본다.

셋을 어떻게 엮는지에 실무의 요령이 있다.

모델에게 판정을 맡길 때는 채점표를 항목으로 쪼개 준다. 좋은 답인지 물으면 안 되고, 사실이 맞는지, 빠뜨린 것이 없는지, 지시를 지켰는지를 따로 묻는다. 그리고 「모르겠음」이라고 답할 길을 열어 둔다. 안 열어 두면 모르는 것도 통과와 실패 가운데 하나로 찍힌다.

모델 판정을 들일 때는 사람이 판정한 것과 얼마나 맞는지 확인하고 그 값을 적어 둔다. 한 번 확인하고 끝내지 않는다. 기준을 고치면 맞는 정도도 달라진다.

판정하는 쪽과 평가받는 쪽을 갈라야 한다는 말은 흔히 다른 모델을 쓰라는 말로 옮겨진다. 그것 만으로 모자란다. 잘못된 이해를 둘이 함께 갖고 있으면, 판정이 잘 맞다는 관측은 판정이 옳다는 증거가 아니라 둘이 같은 것을 물려받았다는 증거일 수 있다. 사람의 표본 검토를 남겨 두는 것 말고 이 고리를 끊을 길이 없다.

기준이 여물면 코드로 내려앉힌다. 모델이 읽던 것을 기계 검사로 갈아 끼우면 값이 줄고 판정의 흔들림도 함께 줄어든다.

셋 가운데 하나를 고르는 것이 아니다. 한 과제에 여럿을 붙이는 편이 흔하다. 코드를 고치는 과제라면 시험이 통과했는지는 코드가 보고, 고친 코드를 읽을 만한지는 모델이 채점표를 들고 보고, 둘이 어긋나는 것들만 사람이 읽는다. 사람이 읽는 양을 줄이는 짜임이 평가하는 쪽에도 그대로 쓰인다.

몇 번 돌리는가

한 번은 아니다. 여러 번 돌리고 나서 무엇을 셀지 둘 가운데 고른다. 여러 번 중에 한 번이라도 되면 통과로 볼 것인가, 여러 번 다 되어야 통과로 볼 것인가.

한 번만 돌리면 둘은 같은 수다. 열 번을 돌리면 정반대의 이야기를 한다. 앞의 것은 100에 가까워지고 뒤의 것은 0으로 떨어진다.

어느 쪽을 붙이는 만드는 것이 무엇이나에 달렸다. 사람이 여러 안을 받아 보고 하나를 고르는 도구라면 한 번이라도 되면 값을 한다. 고객 문의에 자동으로 답하는 시스템이라면 매번 되어야 한다. 뒤엣것에서 앞의 수를 보고하면 실제보다 훨씬 좋아 보인다.

수로 보면 이렇다. 한 번에 되는 비율이 75인 에이전트를 연달아 세 번 돌리면, 세 번 다 되는 경우는 42쯤이다. 성공률 75라는 보고와 열 건 가운데 넷만 끝까지 간다는 관측이 같은 시스템의 이야기다.

여러 번 돌리는 데는 다른 쓸모도 있다. 백 번을 돌렸는데 한 번도 통과하지 못한 과제는 대개 에이전트가 못하는 과제가 아니라 깨진 과제다. 채점하는 코드가 틀렸거나, 문제 서술이 모호하거나, 애초에 풀 수 없게 적혀 있다.

여러 번 돌리기로 했으면 한 가지를 지켜야 한다. 시행마다 깨끗한 데서 시작한다. 앞 시행이 만든 파일이나 고쳐 놓은 데이터베이스가 다음 시행에 남아 있으면, 서로 무관해야 할 실패들이 함께 움직인다. 그러면 열 번 가운데 셋이 실패했다는 관측이 세 번의 독립된 실패인지 한 번의 실패가 변진 것인지 알 수 없다.

언제 나아졌다고 말하는가

수 하나로는 아무것도 말할 수 없다. 성공률이 72라는 보고를 받았다고 하자. 좋은가. 견줄 것이 없으면 답할 수 없다.

견줄 것은 셋이다. 고치기 전의 자기, 다른 구성, 미리 그어 둔 목표선이다.

목표선부터 보자. 이건 평가에서 나오지 않는다. 이 일을 사람이 하면 몇 건 가운데 몇 건을 맞히는 지, 지금 사람이 쓰는 시간이 얼마인지, 어느 수준 아래로 내려가면 쓰지 않기로 했는지에서 나온다. 평가 밖으로 한 번 나갔다와야 그을 수 있다. 안 나갔다 오고 그은 선은 대개 지금 성적보다 조금 높은 데 그어지고, 그런 선은 넘어도 아무 결정을 바꾸지 않는다.

다른 구성은 자주 없다. 압축을 켜 시스템은 압축된 실행만 만들고, 나는 시스템은 나쁜 실행만 만든다. 압축하지 않았으면 어땠을까는 관측이 아니라 상상이고, 상상을 관측으로 바꾸려면 그 구성을 실제로 만들어 한 번 더 돌려야 한다.

그래서 장치를 들일 때 물음이 하나 떨어져야 한다. 이 장치를 끈 실행을 만들 수 있는가. 끝 수 없게 들인 장치는 그때부터 검사받지 않는 부품이 된다. 스위치를 만드는 품은 장치를 들일 때 가장 적고, 스위치가 필요해지는 때는 대개 나중에 온다. 그때 스위치가 있으면 논쟁이 하루의 실험으로 끝나고, 없으면 견해의 다툼으로 끝난다.

견줄 상대를 마련해도 하나가 남는다. 두 수의 차이가 흔들림보다 커야 차이라고 부를 수 있다. 모델 판정을 쓰는데 같은 산출물을 두 번 평가해 3의 차이가 나온다면, 2가 올랐다는 보고는 아무 말도 하지 않는다. 순서는 정해져 있다. 판정을 먼저 평가하고 시스템을 나중에 평가한다.

제5장이 여기까지 와 있었다. 프롬프트를 고쳤을 때 판정은 점끼리가 아니라 범위끼리 하라고 했다. 형식을 여러 변형으로 흔들고 같은 변형을 여러 번 돌리면 점수가 여럿 나오는데, 그 여럿의 낮은 쪽과 높은 쪽이 범위다. 고친 뒤의 가장 낮은 점수가 고치기 전의 가장 높은 점수를 넘을 때만 나아졌다고 말한다.

프롬프트만의 사정이 아니다. 도구를 손볼 때도, 검색을 갈아 끼울 때도, 모델을 올릴 때도 같은 물음이 온다.

거기서 미뤄 둔 물음이 둘이었다. 범위를 몇 개의 변형과 몇 개의 과제로 그릴 것인가가 하나이고, 두 범위가 겹쳤을 때 무엇을 할 것인가가 둘이다.

앞의 물음에 정해진 답은 없고 조건만 있다. 변형을 늘리고 시행을 늘린 만큼 값이 곱해지므로 문장 하나 고칠 때마다 치를 수는 없다. 제대로 치르는 데는 셋이다. 처음 세울 때, 구조를 손볼 때, 모델을 올릴 때다. 셋째가 가장 자주 잊힌다.

뒤의 물음에는 답이 있다. 겹치면 아직 모르는 것이므로 길은 둘뿐이다. 사례나 시행을 늘려 범위를 좁히거나, 그 변형을 개선이라고 부르지 않고 내보낸다. 코드가 단순해져서 내보낸다고 적는 것과 성능이 좋아져서 내보낸다고 적는 것은 다르다. 뒤엎것으로 적어 두면 몇 달 뒤에 그 기록을 근거로 다음 결정이 선다.

이 일은 한 번 하고 끝나지 않는다. 실무에서 평가는 시험 코드처럼 걸어 둔다. 프롬프트를 고치거나 도구를 더하거나 모델을 올릴 때마다 자동으로 돌게 한다.

18.5 실제 사례

공개된 벤치마크를 보면 앞의 넷을 실제로 어떻게 하는지 알 수 있다. Anthropic (2026c) 이 갈래 별로 드는 예에서 셋을 골랐다.

코드를 고치는 에이전트

SWE-bench는 실제 GitHub 저장소의 이슈를 준다. 에이전트는 저장소를 뒤져 고칠 데를 찾고 코드를 고친다.

판정은 코드가 한다. 고친 저장소에 시험을 돌리는데 두 갈래를 본다. 원래 실패하던 시험이 통과해야 하고, 원래 통과하던 시험이 계속 통과해야 한다. 앞의 것이 고쳤는지를 보고, 뒤의 것이 다른 데를 망가뜨리지 않았는지를 본다.

이름에 붙은 Verified는 사람이 검수한 부분집합이라는 뜻이다. 원래 집합에는 문제 서술이 모자라 풀 수 없는 과제와 지나치게 좁게 적힌 시험이 섞여 있었다. 앞 절에서 말한 깨진 과제가 실제로 문제가 됐던 사례다(OpenAI, 2024).

잡는 것과 못 잡는 것이 분명하다. 고쳐졌는지는 확실히 잡는다. 고친 코드를 사람이 읽을 만한지, 몇 번 해봤는지, 토큰이 얼마나 들었는지는 잡지 않는다.

고객을 응대하는 에이전트

τ -bench는 소매점과 항공사의 고객 응대를 흉내 낸다. 사람 대신 모델이 손님 역할을 맡아 대화하고, 에이전트는 주문 조회나 예약 변경 같은 도구를 쓰면서 그 회사의 규정을 지켜야 한다.

판정은 대화가 끝난 뒤에 한다. 데이터베이스가 어떤 상태인지 보고 미리 적어 둔 정답 상태와 대조한다. 「처리해 드립니다」라고 말했는지는 보지 않는다. 주문이 실제로 취소됐는지를 본다.

여러 번 다 되어야 통과로 보는 샘플이 여기서 나왔다. 그럴 이유가 있었다. 고객 응대는 한 번이라도 되면 되는 일이 아니다. 열 번 가운데 아홉 번 규정을 지키고 한 번 어기면 그 한 번이 사고다.

보고된 수도 그쪽을 가리킨다. 그때 시험한 가장 좋은 모델도 전체 과제의 절반을 넘기지 못했고, 소매 영역에서는 같은 과제를 여덟 번 연달아 다 성공하는 비율이 25에 못 미쳤다(Yao et al., 2025).

못 잡는 것은 손님이다. 손님 역할을 모델이 맡으므로 진짜 손님이 하는 말과는 다르다.

터미널에서 일하는 에이전트

Terminal-Bench는 셸을 직접 몰게 한다. 코드를 빌드하고, 서버를 설정하고, 작은 모델을 학습시키고, 망가진 시스템을 고치는 과제들이다.

판정은 남은 환경을 검사해서 한다. 과제마다 컨테이너를 새로 띄우고, 에이전트가 셸을 몰면서 출력을 보고 다음 명령을 정한다. 끝나면 검사 코드가 환경을 들여다본다. 대화 기록은 보지 않는다.

앞 절에서 말한 깨끗한 데서 시작하기가 실제로는 이런 모양이다. 남의 벤치마크라 손이 크게 가는 것처럼 보이지만, 우리 평가에서도 같은 일을 해야 한다. 자료를 되돌리든 계정을 새로 만들든 형태만 다르다.

셋에서 가져갈 것

셋을 나란히 놓으면 같은 것이 보인다. 셋 다 말이 아니라 남은 것을 본다. 시험이 통과했는가, 데이터베이스가 어떤 상태인가, 파일이 어떻게 되어 있는가다.

그리고 셋 다 판정을 코드에 맡긴다. 그래야 같은 산출물에 언제나 같은 판정이 나오고, 성적이 흔들릴 때 흔들린 것이 에이전트라고 말할 수 있다.

셋을 그대로 가져다 쓸 수는 없다. 남의 과제이기 때문이다. 이 벤치마크들이 알려 주는 것은 성적이 아니라 짜임이다. 무엇이 남았는지를 보고, 코드로 판정하고, 여러 번 돌리고, 시행마다 깨끗한 데서 시작한다. 우리 과제의 사례는 여전히 우리 버그 목록에서 나온다.

옮겨 올 때 달라지는 것도 있다. 벤치마크는 고정되어 있어야 견줄 수 있지만, 우리 집합은 실패가 쌓이는 만큼 자란다. 자라는 집합에서는 이번 달 성공률과 지난달 성공률이 같은 시험의 결과가 아니므로, 사례를 고칠 때마다 기록을 남기는 일이 남의 벤치마크보다 우리 쪽에서 더 필요하다.

그리고 셋 가운데 어느 것도 사람이 얼마나 손봤는지는 세지 않는다. 벤치마크는 사람 없이 돌게 만들어야 하니 그럴 수밖에 없다. 그래서 절 18.2에서 좋은 에이전트라고 부르기로 한 것의 절반은 공개된 성적으로 확인할 수 없다.

벤치마크 성적이 높은 모델을 골라 놓고도 쓰는 사람들이 답답해하는 일이 생기는 데는 이 사정이 있다. 그 절반은 우리 시스템에서 우리가 세는 수밖에 없다.

18.6 잘못된 평가

평가는 대체로 세워야 한다. 그래서 이 절은 세우지 말라가 아니라 세우면 안 되는 평가를 가린다.

1. **세기 쉬운 것만 센다.** 형식 위반율이 내려간 것을 성공이라 부르면 시스템은 위반하지 않는 쪽으로 다듬어진다. 그것은 형식을 맞추는 능력이지 과제를 푸는 능력이 아니다. 이런 값들은 재워 성공이라 부르지 않는다. 이름이 붙는 쪽으로 시스템이 다듬어지기 때문이다.
2. **사례가 실제 쓰임과 다르다.** 손으로 지어낸 사례에는 오탈자가 없고 질문이 한 문장으로 또렷하다. 실제로 들어오는 질문은 세 가지를 한꺼번에 묻고 절반은 앞뒤가 안 맞는다. 앞의 집합에서 오른 수는 뒤의 질문을 두고는 아무것도 약속하지 않는다. 지어내는 일 자체가 잘못된 아니고, 지어낸 것과 실제에서 온 것의 비율을 적어 두지 않는 것이 잘못이다.
3. **판정하는 쪽이 만드는 쪽과 같다.** 같은 모델에 같은 프롬프트를 주고 자기가 낸 답을 채점하게 하면 잘못된 이해를 둘이 함께 갖는다. 그 이해가 만든 실패는 성공률에 잡히지 않는다.
4. **한 번 돌리고 나아졌다고 한다.** 판정도 흔들리고 실행도 흔들린다. 두 흔들림을 합친 쪽보다 작은 차이는 개선이 아니라 그날의 운이다. 다섯 가운데 이게 가장 알아채기 어렵다. 나온 수가 올랐고 계산도 틀리지 않았기 때문이다.
5. **시험에 맞춰 고친다.** 통과율이 100에 닿은 평가는 무너진 데를 알려 주기는 하지만 나아진 데는 알려 주지 못한다. 더 나쁜 것은 통과율을 올리려고 과제를 좁게 다시 적는 경우다. 그때 오르는 수는 진짜인데, 오른 이유가 시스템이 좋아져서가 아니다.

자기가 어디에 걸렸는지는 대개 수가 아니라 회의에서 드러난다. 지표는 올랐는데 쓰는 사람들의 불평이 그대로일 때, 또는 지표를 근거로 한 결정에 아무도 놀라지 않을 때다. 앞의 것은 세고 있는 것이 실제 쓰임과 다르다는 신호이고, 뒤의 것은 지표가 이미 아는 것만 확인해 주고 있다는 신호다.

목록에 오르지 않는 실패

다섯 밑에 깔린 문제가 하나 있다. 실패가 사례가 되려면 먼저 실패로 관측되어야 한다.

그런데 이 책이 다룬 실패 가운데 여럿은 성공한 실행과 똑같은 얼굴로 끝난다. 잘못 뻔 조각, 실리지 않은 지시, 형식은 맞는데 내용이 틀린 답, 색인에 들어가기 전에 사라진 원본, 틀린 기억이 그렇다. 다섯 다 오류를 내지 않고 완료의 형태로 끝난다.

아무도 문의를 넣지 않으므로 버그 목록에 오르지 않고, 오르지 않으므로 사례가 되지 않는다.

그래서 평가 집합은 기울어 있다. 수집을 게을리해서가 아니라 시끄러운 실패만 목록에 오르기 때문이다. 조용한 실패는 두 번 진다. 일어날 때 잡히지 않아서 한 번, 사례가 되지 못해 앞으로도 잡히지 않게 되어 또 한 번이다.

처방은 앞 장들에 이미 흩어져 있다. 뺀 조각 대신 원본으로 갈 경로를 남기고, 검색이 무엇을 왜 가져왔는지 함께 돌려주고, 스키마에 「모르겠음」을 적을 칸을 두고, 원본과 추출본을 대조하고, 기억을 표본으로 다시 검증하라던 것들이다.

그때는 실패를 줄이는 처방이었다. 이 장의 눈으로 보면 사례를 만드는 장치이기도 하다.

지표가 만드는 것

지표를 세우고 나면 그것이 팀의 공용어가 된다. 세우기 전에는 개선 주장이 사례와 인상으로 다투어지고, 세운 뒤에는 그 항목 위에서만 다투어진다. 대체로 진보다.

값이 하나 있을 뿐이다. 항목에 없는 축의 주장은 근거 없는 주장이 된다. 말할 수는 있어도 수로 보일 수 없고, 수로 보일 수 없는 주장은 진다. 그리고 항목 밖에 남기 쉬운 것이 하필 앞 소절의 그 조용한 실패들이다.

처방은 간단하다. 항목 목록 옆에 이 지표가 세지 않는 것의 목록을 함께 둔다. 표 18.1의 오른쪽 열이 그 목록의 초안이다.

다섯을 관통하는 것도 하나 있다. 정의할 수 없는 성공을 정의한 척한 평가는 없는 평가보다 나쁘다. 없는 평가는 결정을 짐작 위에 세우고, 짐작이라는 것은 모두가 안다. 척한 평가는 결정을 수 위에 세우는데, 그 수는 자기가 무엇을 세지 않았는지 말하지 않는다.

늦게 세우는 것

평가를 나중에 세우는 일도 여기 든다.

일찍 세우면 쉽다. 만들기로 한 것이 무엇인지가 아직 문장으로 남아 있고, 그 문장이 그대로 과제가 된다. 늦게 세우면 어렵다. 이미 돌고 있는 시스템을 보면서 성공 기준을 거꾸로 짜맞추게 되는데, 그렇게 적힌 기준은 오늘의 시스템에 정확히 맞는다. 정확히 맞는 만큼 내일의 개선을 가려내지 못한다.

평가가 없는 동안 고치는 방식도 정해져 있다. 불평이 들어오기를 기다렸다가, 손으로 다시 돌려 보고, 고치고, 다른 데가 망가지지 않았기를 바란다. 셋째까지는 할 만한데 넷째가 문제다. 바라는 것은 방법이 아니다.

그러니 완벽한 스무 개를 만드느라 석 달을 미루지 않는 편이 낫다. 어설픈 스무 개로 이번 주에 시작하고 실패가 들어올 때마다 하나씩 더한다.

18.7 요약

한 문장으로. 좋은 에이전트는 혼자 다 하는 에이전트가 아니라 사람이 확인하는 품을 줄여 주는 에이전트이고, 그렇게 정의하면 평가 항목에 몇 퍼센트 맞췄는가 말고 사람이 얼마나 손봐야 했는가가 반드시 들어간다.

확실해진 것.

- 모델이 답을 확률로 뽑는 한 마지막 승인은 사람 쪽에 남는다. 모델이 좋아지면서 없어지는 것은 사람의 확인이 아니라 사람이 확인해야 하는 양이다(절 18.2).
- 평가는 말이 아니라 남은 것을 본다. 예약했다는 문장이 아니라 예약 기록을 조회한다. 그리고 어떤 길로 갔는지는 채점하지 않는다(절 18.3).
- 평가 항목은 다섯이고 어느 하나로도 전부를 볼 수 없다(표 18.1). 마지막 항목이 가장 자주 빠지는데, 그것을 빼면 좋다고 부르기로 한 것을 세지 않는 셈이 된다.
- 사례는 실제 실패에서 뽑는다. 스무 개에서 십 개면 시작할 수 있다. 좋은 사례인지는 두 사람이 따로 같은 관정에 이르는가로 가린다(절 18.4).
- 관정자는 셋뿐이다. 코드는 빠르고 매번 같지만 뜻을 못 보고, 모델은 뜻을 보지만 관정이 흔들리며, 사람은 믿을 만하지만 느리다(표 18.2). 하나로 전부를 감당하려는 설계가 저절로 진다.
- 여러 번 돌린다. 한 번이라도 되면 되는 일인지 매번 되어야 하는 일인지를 먼저 정한다. 한 번에 되는 비율이 75여도 세 번 연달아 되는 비율은 42쯤이다.
- 나아졌다고 말하려면 견줄 것이 있어야 한다. 그래서 장치는 끌 수 있게 들인다. 끌 수 없게 들인 장치는 그때부터 검사받지 않는 부품이 된다.
- 항목이 곧 실패의 이름이 된다. 항목에 없는 실패는 고칠 목록에 오르지 않으므로, 지표 목록 옆에 그 지표가 세지 않는 것의 목록을 함께 둔다(절 18.6).

아직 답하지 않은 것.

- 모델 관정을 사람 관정에 어느 수준까지 맞추어야 하는지에 정답이 없다.
- 사례를 실패에서 뽑는 길과 조용한 실패를 시끄럽게 만드는 길 가운데 어느 쪽이 얼마나 값을 하는지 알지 못한다.
- 끌 수 있게 만든 장치를 몇 개까지 유지하고 어느 주기로 되살려 돌릴지도 정해진 바가 없다.
- 사례를 언제 늘려야 하는지, 그리고 평가 집합이 자라면서 이번 달 수와 지난달 수를 어떻게 견줄지는 제19장이 받는다.

오늘 할 것. 지금 보고 있는 대시보드나 평가 스크립트를 열어 항목이 무엇무엇인지 적어 본다. 표 18.1의 다섯 가운데 없는 것을 표시한다(절 18.3). 사람이 손본 양을 세는 항목이 없으면, 지난주 실행 스무 건을 열어 사람이 그대로 받아들인 것이 몇 건인지 센다(절 18.2). 마지막으로 지금 보고하는 성공률이 몇 번 돌린 결과인지 확인한다.

18.8 생각해 볼 문제

1. 최근 실행 열 건을 골라, 사람이 그대로 받아들인 것과 고쳐서 쓴 것과 통째로 버린 것으로 나누어 보라. 셋째는 지금 어느 항목에 세어지는가.

2. 지금 켜 둔 장치 가운데 끌 수 없는 것을 목록으로 만들라. 하나하나에 스위치를 만드는 데 드는 품을 지금과 여섯 달 뒤로 나누어 견주어 보라.
3. 자기 과제가 한 번이라도 되면 되는 일인지 매번 되어야 하는 일인지 정하라. 뒤엎것이라면 지금 보고하는 수를 세 번 연달아 성공하는 비율로 다시 계산해 보라.
4. 표 18.1의 오른쪽 열을 읽고, 못 잡는 것 가운데 자기 시스템에서 실제로 문제가 됐던 것을 고르라. 그것을 잡으려면 항목을 더해야 하는가, 기록을 더 남겨야 하는가.

18.9 다시 읽을 때

- 제19장을 읽고 절 18.6의 기울어진 집합으로 돌아오면, 기운 집합을 향해 오래 다듬으면 시스템이 그 기울기의 모양대로 굳는다는 것이 보인다. 이 장은 기울기까지 적었고 시간은 그쪽이 없다.
- 제20장을 읽고 표 18.1로 돌아오면, 다섯 항목 가운데 셋은 실행의 기록 없이 셀 수 없다. 무엇을 셀지 정하는 일보다 무엇을 남길지 정하는 일이 먼저다.
- 제22장을 읽고 절 18.6으로 돌아오면, 조용한 실패의 목록에 한 줄이 더 붙는다. 주입이 성공한 실행도 오류를 내지 않고 완료의 형태로 끝난다.

18.10 더 읽을거리

Anthropic (2026c)이 이 장이 기대는 자료다. 가져온 것은 넷이다. 말이 아니라 환경에 남은 것을 채점하라는 것, 길이 아니라 결과를 채점하라는 것, 두 사람이 따로 같은 판정에 이르는 과제가 좋은 과제라는 판별, 그리고 실제 실패에서 뽑은 스무 개에서 쉰 개면 시작할 수 있다는 권고와 그 근거다.

판정자 셋을 잘하는 것과 못하는 것으로 갈라놓은 표와, 작은 집합으로 되는 이유를 초기의 개선이 크다는 사정으로 읽은 것은 이 책의 정리다.

절 18.5의 벤치마크 셋은 전부 공개된 것이다. 성적표를 보러 가는 것이 아니라 짜임을 보러 가는 편이 낫다. 과제 하나를 열어 무엇을 주고 무엇을 검사하는지 읽어 보면 이 장의 절반이 손에 잡힌다. Yao et al. (2025)에 여러 번 다 되어야 통과로 보는 샘플과 소매 영역의 수가 있고, OpenAI (2024)에 사람이 검수하며 무엇을 걷어 냈는지가 적혀 있다. 답부터 잡고 문제를 거꾸로 만드는 요령은 Wei et al. (2025)의 것이다.

평가 지표의 설계

머신러닝을 오래 한 사람이 에이전트 팀에 오면 첫 회의에서 대개 같은 것을 묻는다. 정확도가 얼마입니까.

답이 잘 안 나온다. 성공률이라면 68이라고 말할 수 있다. 그런데 그 68이 무엇을 세었는지는 그 팀에 있는 사람 말고는 아무도 모르고, 옆 팀이 보고한 71과 나란히 놓을 수도 없다. 두 수가 같은 것을 센 수가 아니기 때문이다.

제18장은 평가를 어떻게 돌리는지 세웠다. 무엇을 사례로 삼고, 누가 판정하고, 몇 번 돌리고, 언제 나아졌다고 말하는지다. 그 절차에는 항목이 들어가야 돈다. 이 장은 그 항목을 무엇으로 정하는지 다룬다.

19.1 공식 지표의 부재

머신러닝에는 이름이 정해진 지표가 있다. 스팸을 가려내는 분류기를 만들었다고 하자. 전체 가운데 맞힌 비율이 정확도이고, 스팸이라고 답한 것 가운데 실제로 스팸이던 비율이 정밀도이며, 실제 스팸 가운데 몇 개를 잡아냈는지가 재현율이다. 계산법이 어디서나 같다.

그래서 두 논문의 수를 나란히 놓을 수 있다. 같은 자료에서 정확도 91과 93이 나왔다면 뒤엎 것이 잘한 것이고, 그 판정에 아무 설명도 필요 없다. 자리 잡은 지표는 무엇보다 두 수를 나란히 놓게 해 준다.

에이전트에는 아직 그런 것이 없다. 공개된 벤치마크는 있다. 제18장이 셋을 열어 봤다. 저장소의 이슈를 주고 고친 코드에 시험을 돌리는 것, 손님 역을 모델에 맡기고 대화가 끝난 뒤 데이터베이스가 어떤 상태인지 대조하는 것, 셀을 직접 물게 하고 남은 환경을 검사하는 것이다. 셋 다 잘 만들어져 있는데 셋 다 남의 과제다. 거기서 가져올 것은 성적이 아니라 짜임이었다.

없는 데는 이유가 셋 있다.

첫째, 분야가 자라는 중이다. 무엇을 세어야 하는지가 아직 자리를 잡지 못했다.

둘째, 에이전트는 대개 특정 직무를 돕도록 만들어진다. 사람의 일을 통째로 넘겨받는 물건이 아니라 옆에서 거드는 쪽이다. 제18장이 좋은 에이전트를 그렇게 정의했다. 혼자 다 하는 에이전트가 아니라 사람이 확인하는 품을 줄여 주는 에이전트다.

셋째, 그래서 잘한다는 뜻이 직무마다 다르다.

범무 검토를 돕는 에이전트를 생각해 보자. 표준과 다른 조항 하나를 못 보고 넘어가면 그 건은 실패다. 문장을 매끄럽게 쓰는 재주는 여기서 아무것도 아니다. 코드를 고치는 에이전트는 반대다. 빠뜨린 데가 있어도 시험이 통과하면 그 건은 성공이고, 빠뜨린 것은 다음 실행에서 잡힌다. 앞의 것은 한 번의 누락이 사고이고 뒤의 것은 한 번의 누락이 다음 차례다.

두 시스템을 같은 자로 평가할 수 없다. 그러니 지표를 직접 설계하는 수밖에 없다.

설계라고 부르는 데는 까닭이 있다. 목록에서 골라 쓸 수 있으면 그건 이미 있는 지표이고, 없으니 까 만들어야 한다. 만드는 일에는 순서가 있다.

19.2 지표 설계

셋을 차례로 정한다. 무엇을 셀지 적고, 그것이 실제로 나오는지 확인하고, 나온 수로 무엇을 할지 정한다.

무엇에서 출발하는가

과제가 아니라 그 일을 지금 하고 있는 사람에서 출발한다.

계약서를 검토하는 팀에 에이전트를 들인다고 하자. 지표를 적기 전에 할 일은 담당자 옆에 앉아 하루를 보는 것이다. 보면 이렇다. 계약서 한 건이 오면 조항을 처음부터 훑는다. 우리 표준과 다른 데를 표시한다. 표시한 데마다 받아들일지 다시 쓸지를 적는다. 한 건에 40분쯤 걸리고 그중 30분이 다른 데를 찾는 일이다.

여기서 물을 것이 셋이다. 지금 이 일을 사람이 어떻게 하는가. 무엇이 잘된 것이고 무엇이 잘못된 것인가. 누가 그것을 관정하는가.

둘째 물음에 담당자가 내놓는 답은 갈라져 나온다. 표준과 다른 조항을 못 보고 넘어간 것이 잘못이다. 다르지 않은 조항을 다르다고 표시한 것도 잘못이다. 그런데 둘의 값이 다르다. 앞의 것은 계약이 그대로 나가서 사고가 되고, 뒤의 것은 담당자가 몇 분을 더 쓰고 끝난다.

셋째 물음의 답은 그 담당자다. 다른 사람은 관정할 수 없다. 그러면 그 사람의 시간이 이 평가의 예산이 된다. 하루에 몇 건을 봐 줄 수 있는지에서 평가 집합의 크기가 나온다.

그다음에 이것을 셀 수 있는 것으로 옮긴다. 셋이 나온다. 담당자가 표시한 조항 가운데 에이전트가 표시한 비율, 에이전트가 표시한 것 가운데 담당자가 지운 비율, 한 건에 담당자가 쓴 시간이다.

옮기면서 잃는 것이 있다. 받아들일지 다시 쓸지의 판단은 셋 어디에도 없다. 조항을 다 찾기도 판단이 어긋나면 그 건은 실패인데 이 셋은 통과로 센다. 잃은 것을 적어 두는 일까지가 설계다. 적어 두지 않으면 몇 달 뒤에 이 셋이 이 일의 전부였다고 읽힌다.

적어 놓으면 이런 모양이 된다.

놓친 조항 비율

세는 것: 담당자가 표준과 다르다고 표시한 조항 가운데

에이전트가 표시하지 못한 것의 비율

나오는 곳: 두 표시본을 대조 / 코드가 판정 / 사람 손이 안 감

집합: 2월 판 / 계약서 40건 / 담당자 두 사람이 따로 표시하고

둘이 같린 건은 뺐다

시행: 건마다 세 번 / 세 번 다 놓치면 놓친 것으로 센다

벌어짐: 같은 집합을 두 번 돌려 1.5까지 벌여졌다

바꾸는 결정: 4를 넘으면 되돌린다 / 2 아래면 검토 순서를

에이전트 먼저로 바꾼다

못 세는 것: 받아들일지 다시 쓸지의 판단

버리는 때: 담당자가 바꿀 때 / 표준 문안이 개정될 때

여덟 줄 가운데 여섯 줄은 이 절의 나머지가 채운다.

어떻게 측정하는가

지표를 적었으면 그것이 실제로 나오는지 확인한다. 물을 것이 셋이다.

어디서 나오는 값인가. 앞의 카드에서 첫 줄은 담당자의 표시본이 있어야 나온다. 표시본을 따로 모아 두지 않는 팀이라면 그것은 지표가 아니라 계획이다.

사람이 손을 대야 나오는가, 저절로 나오는가. 대조는 코드가 한다. 한 건에 쓴 시간은 담당자가 언제 열고 언제 닫았는지가 남아야 나온다. 남지 않으면 그 항목은 매달 사람에게 물어보는 항목이 되고, 물어보는 항목은 두 달이면 사라진다.

표 19.1: 같은 지표를 같은 시스템에 두 번 댔을 때 수가 벌어지는 다섯 자리. 오른쪽 열이 요점이다. 줄이는 길이 있는 것과 없는 것이 섞여 있고, 첫째와 다섯째는 나머지 셋과 줄이는 방식이 다르다.

자리	무엇이 달라지는가	줄일 수 있는가
행동의 선택	같은 컨텍스트에서 다른 행동이 나온다. 경로도 스텝 수도 달라진다	반복해서 평균을 내는 것까지. 없앨 수는 없다. 평가하려는 시스템 자신의 성질이다
프롬프트의 형식	뜻이 같은 표기와 예시의 배열이 달라진다(절 5.3)	여러 변형에 걸쳐 측정하면 점 하나 대신 범위가 나온다. 변형의 수만큼 실행이 늘어난다
판정	같은 산출물에 판정이 달라진다. 모델 판정도 뽑기다(절 18.4)	판정을 먼저 확인하고 기준이 여물면 코드로 내려앉히는 만큼. 사람이 판정하는 자리에는 남는다
사례의 구성	어느 계약서가 그 40 건에 들어와 있는가. 과제마다 성적이 다르다	집합을 늘리는 만큼. 늘린 만큼 사례당 실행 비용이 곱해진다
실행 환경	실행 사이에 남는 상태. 남은 과일, 캐시된 자료, 모자란 자원	실행마다 같은 상태에서 시작하게 하는 것 말고 길이 없다. 반복해도 줄지 않는다

얼마나 자주 측정하는가. 매주 돌릴 수 있는 것과 분기에 한 번 돌릴 수 있는 것을 갈라 적는다. 갈라 두지 않으면 예산이 조일 때 둘 다 잘린다.

그리고 하나가 더 있다. 같은 지표를 같은 시스템에 두 번 대면 두 수가 나온다. 그 사이에 시스템은 달라지지 않았으므로 벌어진 만큼은 측정하는 일 자체에서 왔고, 두 번 대서 1.5가 벌어지는 지표로는 1의 변화를 확인할 수 없다. 벌어짐이 어디서 오는지 다섯이다.

다섯째를 따로 본다. 실행 사이에 남는 상태가 에이전트의 성능 때문이 아니라 인프라의 불안정 때문에 상관된 실패를 일으킬 수 있다고 Anthropic (2026c)이 적는다. 힘이 실리는 낱말은 상관된이다. 남은 과일이 이번 회차의 모든 시행에 똑같이 남아 있으면 열 번을 돌려도 열 번이 같은 방향으로 어긋나고 평균은 그 어긋남을 그대로 물려받는다. 반복이 줄여 주는 것은 흩어지는 어긋남뿐이다.

그래서 측정을 시작하기 전에 물을 것이 하나 있다. 실행마다 같은 상태에서 시작하는가. 답할 수 없으면 그 집합에서 나온 어떤 차이도 시스템의 것이라고 말할 수 없다.

벌어짐은 한 번 측정해서 지표 옆에 적어 둔다. 카드의 여섯째 줄이 그것이다. 그 수가 없으면 모든 변화가 개선으로 읽히고, 그것만도 아니다. 어긋남 때문에 내려간 수는 저하로 읽히고 그러면 좋은 변경이 되돌려진다.

나온 수로 무엇을 하는가

지표를 만드는 까닭은 결정을 바꾸기 위해서다. 그러니 마지막에 정할 것은 어떤 결정을 바꾸느냐다.

그러면 아무 결정도 바꾸지 않는 지표는 왜 두는가. 답은 둘뿐이다. 아직 결정을 안 정했거나, 남에게 보이려고 두었거나. 앞이면 지금 정하고, 뒤면 그건 지표가 아니라 보고서의 한 줄이다. 지표 화면에서 내린다. 남겨 두면 다음 사람이 그것을 지표로 알고 겨냥한다.

언제 이 지표를 버리는지도 함께 적는다. 값이 천장에 붙어 더 움직이지 않을 때, 직무가 바뀌었을 때, 판정하는 사람이 바뀌었을 때다.

사례를 언제 늘려야 하는지도 여기서 답한다. 늘릴 때는 집합이 나빠진 때가 아니라 시스템이 좋아진 때다. 처음에는 한 번 고칠 때마다 결과가 크게 움직이고, 크게 움직이는 것을 확인하는 데는 마흔 건으로 넉넉하다. 그런데 시스템이 여물면 한 번의 개선이 작아진다. 그 개선이 두 번 돌려

벌어지는 만큼보다 작아지는 순간부터 이 집합은 아무것도 말하지 않는다. 더 성숙한 에이전트는 더 작은 효과를 잡아내기 위해 더 크고 더 어려운 평가를 필요로 할 수 있다고 Anthropic (2026c)이 적어 두었다. 늘릴 신호는 성적이 아니라 개선과 벌어짐의 관계다.

집합을 늘리면 다음 물음이 따라온다. 이번 달 수와 지난달 수를 어떻게 견줄 것인가. 사례가 스무 건 더 들어온 집합에서 나온 68과 지난달 마흔 건에서 나온 71은 같은 시험의 결과가 아니다. 그런데 보고서에서 두 수는 같은 칸에 같은 형태로 적힌다.

방법은 정해져 있다. 집합에 번호를 붙이고 새 사례를 들어오는 대로 넣지 않는다. 따로 쌓아 두었다가 번호를 올릴 때 한꺼번에 넣는다. 번호를 올리는 주에는 두 판을 다 돌린다. 옛 판의 수로 지난달과 잇고 새 판의 수로 다음 달을 연다. 그 주가 이음매다. 이음매를 만들지 않으면 몇 달치 기록이 한 줄로 이어진 것처럼 보이지만, 실제로는 서로 다른 시험 여섯 개의 수가 나란히 놓여 있을 뿐이다.

19.3 실 사용 예시

공개된 기록 셋을 열어 본다. 셋 다 자기 직무에 맞춰 지표를 새로 적었고, 셋 다 왜 그렇게 적었는지를 함께 적어 두었다.

API를 부르는 모델

사람 말을 받아 어느 API를 어떤 인자로 부를지 적어 내는 모델이다. 후보는 1,600개 남짓이다 (Patil et al., 2024). 제7장이 이 자료에서 두 실패를 가져왔다. 도구가 많아지면 설명문의 단서가 문헌다는 것과 인자를 정확히 적지 못한다는 것이다. 여기서 볼 것은 그 실패를 무엇으로 썼는가다.

지표는 문자열 대조를 버리는 데서 시작했다. 같은 호출을 여러 방식으로 적을 수 있어서 글자를 맞대면 맞은 것이 틀린 것으로 세어진다. 대신 생성된 코드를 구문 나무로 파싱하고, 부르려는 API가 뿌리인 부분 나무를 찾아 대조한다. 인자마다 맞춰 보되 어느 인자를 맞추는지는 API마다 미리 적어 둔다. 기본값이 있는 인자는 빼 둔다(Patil et al., 2024).

그리고 실패를 둘로 갈랐다. 어느 API와도 맞지 않는 부분 나무를 내놓으면 없는 도구를 지어낸 것이고, 있는 API를 잘못 부른 것은 오류다(Patil et al., 2024). 가르 데 값이 있는 것은 고칠 데가 다르기 때문이다. 지어냈으면 도구 목록이 모델에 제대로 실리지 않은 것이고, 잘못 불렀으면 설명문이나 인자규격이 모자란 것이다.

수를 보면 왜 갈랐는지가 보인다. 예시 없이 물었을 때 한 상용 모델이 지어낸 비율은 두 자료에서 78.65와 36.55였고, 이 논문의 모델은 같은 자리에서 10.95와 6.98이었다(Patil et al., 2024). 하나의 성공률로 합쳤으면 이 차이는 보이지 않는다.

못 보는 것도 분명하다. 부른 API가 그 사람이 원하는 일을 실제로 했는지는 이 수에 없다. 그리고 맞추지 않기로 한 인자에서 틀리면 통과로 세어진다.

조사를 대신 해 주는 에이전트

질문을 받아 웹과 사내 자료를 뒤져 답과 출처를 내놓는 시스템이다. 하나가 계획을 세우고 여럿이 나누어 찾는다(Anthropic, 2025e). 제16장이 이 자료에서 나누는 짜임을 가져왔다.

이 팀이 적은 지표는 채점표다. 좋은 답인지 한 번에 묻지 않고 다섯으로 쪼갬다. 주장이 출처와 맞는가, 인용한 출처가 그 주장의 출처가 맞는가, 물어본 것을 다 다뤘는가, 2차 자료 대신 1차 자료를 썼는가, 도구를 알맞게 알맞은 횟수로 골랐는가(Anthropic, 2025e).

판정은 모델이 한다. 한 번의 호출로 0.0에서 1.0 사이의 점수와 통과·실패를 함께 내게 했을 때 판정이 가장 고르게 나왔고 사람의 판정과도 가장 잘 맞았다고 적혀 있다. 그리고 정해 둔 절차를 따랐는지가 아니라 마지막에 도달한 상태를 보라고 같은 자료가 적는다. 여럿이 나누어 도는 시스템이라 같은 질문에도 경로가 매번 다르기 때문이다.

시작은 스무 개쯤이었다. 실제 쓰임을 대표하는 질문 스무 개 남짓으로 열었다고 적혀 있다(Anthropic, 2025e). 제18장이 실제 실패에서 뽑은 스무 개에서 섰던 출발할 수 있다고 한 자리와 같은 규모다.

그런데 채점표로 끝나지 않았다. 사람이 직접 써 보면서 채점표가 놓치는 것을 찾아냈고, 그렇게 나온 것 하나가 검색 순위가 위에 있는 콘텐츠 농장을 학술 자료보다 앞세우는 버릇이었다(Anthropic, 2025e).

다섯 항목 어디에도 없는 것이 하나 있다. 답을 받은 사람이 그 답으로 무엇을 했는가다.

문서에서 답을 찾는 시스템

사내 문서를 조각으로 잘라 두고, 질문이 오면 조각을 찾아 모델에 실어 답을 만드는 시스템이다. 제11장과 제12장이 이 자료를 여러 번 불렀다.

여기서 볼 것은 이 팀이 최종 답을 채점하지 않았다는 점이다. 대신 스무 개를 가져왔을 때 그 안에 필요한 조각이 없는 비율을 썼다. 스무 개 안에 정답 조각이 들어 있는 비율에서 1을 뺀 값이고, 그것을 평가 지표로 삼았다고 적혀 있다(Anthropic, 2024b).

왜 그렇게 골랐는지는 고칠 자리를 생각하면 나온다. 최종 답이 틀렸을 때, 검색이 못 찾아온 것인지 모델이 실린 자료를 못 읽은 것인지는 그 답을 봐서 갈리지 않는다. 검색만 따로 세면 그 수 하나에 고칠 데가 적힌다.

수는 이렇게 움직였다. 놓치는 비율이 5.7에서 시작해, 조각마다 맥락 문장을 붙였을 때 3.7이 됐고, 낱말 검색을 곁쳤을 때 2.9가 됐으며, 재순위화를 붙였을 때 1.9가 됐다(Anthropic, 2024b).

못 보는 것은 그 조각이 실제로 답에 쓰였는지도. 놓치는 비율이 0이어도 답은 틀릴 수 있다. 이 지표는 그것을 세지 않기로 하고 만들어진 지표다.

셋에서 가져갈 것

셋을 나란히 놓으면 같은 움직임이 셋 보인다.

- **대조하는 방식을 과제에 맞춰 다시 만들었다.** 글자를 맞대는 대신 구조를 맞췄고, 정답이 하나로 정해지지 않는 과제에서는 채점표를 썼다.
- **한 덩어리로 묻지 않고 갈랐다.** 지어낸 것과 잘못 부른 것을 갈랐고, 좋은 답인지를 다섯으로 갈랐다. 갈라 놓은 자리마다 고칠 데가 다르다.
- **최종 결과 대신 고칠 수 있는 자리를 썼다.** 답을 채점하지 않고 검색을 채점한 쪽이 그렇다.

하나 더 있다. 셋 가운데 어느 것도 자기 지표를 정확도라고 부르지 않았다. 이름을 자기 직무의 말로 적었다.

셋 다 지표를 한 줄로 적지 않았다. 정의를 적고, 왜 그것을 골랐는지 적고, 무엇을 대조하는지 적었다. 앞 절의 카드가 여덟 줄인 까닭이 여기 있다. 한 줄로 적을 수 있는 지표는 대개 남의 지표다.

셋의 지표가 못 보는 것을 적은 대목은 이 책이 읽어 낸 것이지 자료에 그렇게 적혀 있는 문장이 아니다. 자료를 열어 확인할 것은 지표의 정의와 그것을 고른 까닭까지다.

19.4 잘못된 지표의 설계

지표는 대체로 세워야 한다. 그래서 이 절은 세우지 말라가 아니라 이렇게 세우면 안 된다는 목록이다. 다섯 다 머신러닝에서 이미 겪은 것들이다.

1. **다시 측정할 수 없는 지표.** 두 번째 측정이 첫 번째와 견줄 수 없으면 그 지표는 한 번 쓰고 버려진다. 집합에 사례를 넣었는데 번호를 안 붙였을 때, 판정 프롬프트를 손봤는데 어떻게 손봤는지 안 남겼을 때, 시행을 세 번에서 한 번으로 줄였는데 그 줄이 보고서에 없을 때 그렇게

표 19.2: 앞의 여러 장이 권한 네 기법에서 이득과 손실이 도착하는 곳과 때. 넷 다 이득은 그 스텝의 지표에 즉시 적히고 손실은 몇 과제 뒤에 흩어져 적힌다. 오른쪽 열의 값은 대개 그 기법의 이름을 달고 오지 않는다.

기법	이득이 적히는 곳과 때	손실이 적히는 곳과 때
압축	그 스텝의 토큰 수. 즉시	다시 찾아보는 횟수와 성공률. 몇 스텝 뒤에서 몇 과제 뒤까지 흩어진다
점진적 공개	고정부의 토큰 수. 즉시, 그리고 확실하게	성공률. 실리지 않은 것이 실제로 아쉬운 실행에서만
코드 실행	컨텍스트 쪽 토큰. 즉시	실행 환경 쪽에 드는 값과 잘못됐을 때 짚기 어려움. 뒤늦게
나눔과 가두기	청구서. 즉시	짐작만 된다. 나누지 않은 실행이 없어서 견줄 상대가 없다

된다. 생기는 일은 둘이다. 지난달 수와 이번 달 수가 같은 칸에 나란히 놓이고 아무도 그 둘이 다른 시험의 결과라는 것을 모른다. 그리고 그 지표는 조용히 버려진다. 쓰려고 할 때마다 처음부터 다시 만들어야 하기 때문이다. 처방은 간단하다. 지표 옆에 다시 만드는 데 필요한 것을 적는다. 집합의 번호, 판정의 번호, 시행 횟수 셋이면 대개 된다.

2. **기준선 없이 지표만 늘리는 경우.** 성공률 72가 좋은 수인지 그 수만 보고는 답할 수 없다. 제18장이 견줄 것 셋을 들었다. 고치기 전의 자기, 같은 과제를 도는 다른 구성, 미리 그어 둔 목표표선이다. 여기서 볼 것은 왜 기준선이 자꾸 빠지는가다. 지표를 하나 더 만드는 데는 코드 몇 줄이 들고, 기준선을 하나 만드는 데는 사람이 이 일을 직접 해서 몇 건을 맞히는지 세거나 장치들 끈 구성을 한 번 더 돌려야 한다. 두 품이 다르므로 팀은 지표를 늘리고 기준선은 미룬다. 지표 화면에 수가 열둘인데 어느 것도 무엇과 견주는 수인지 적혀 있지 않은 상태가 그렇게 만들어진다. 그러니 기준선을 만들 계획이 없으면 그 지표를 만들지 않는 편이 낫다. 가장 적은 품으로 구할 수 있는 기준선은 사람이 이 일을 할 때의 수다.
3. **서로 다른 것을 측정하는 지표를 하나로 합치는 경우.** 머신러닝에 오래된 예가 있다. 놓치지 않는 정도와 헛것을 부르지 않는 정도를 하나의 수로 합친 것을 F1이라 부른다. 둘의 조화평균인데, 조화평균은 두 값 가운데 작은 쪽에 끌려가는 평균이다. 순위를 매기기 편해서 널리 쓰인다. 합치면 잃는 것이 있다. 두 실패의 값이 다르면 합친 수는 결정을 바꾸지 못한다. 계약서 검토에서 놓친 조항 하나는 계약이 그대로 나가는 일이고 헛것 하나는 담당자가 몇 분을 더 쓰는 일이다. 이 둘을 반씩 섞은 수가 0.80에서 0.83으로 갔다는 보고에서, 사고가 늘었는지 줄었는지는 읽을 수 없다. 합치기 전에 두 실패에 값을 적어 본다. 값이 같으면 합쳐도 되고 다르면 따로 둔다. 하나의 수가 필요한 자리는 대개 보고이지 결정이 아니다.
4. **측정하기 쉬운 것으로 어려운 것을 대신하는 경우.** 평가하려는 것을 직접 측정할 수 없거나 직접 측정하는 데 사람이 너무 많이 들 때, 함께 움직인다고 믿어지는 다른 값을 대신 측정한다. 이것을 대리 지표라 한다. 이 책이 측정하라고 한 것도 대부분 대리였다. 유사도 점수는 관련성의 대리이고, 형식 위반율은 도구를 옳게 쓰는지의 대리이며, 평가 집합의 성공률은 현장에서의 성공률의 대리다. 대리를 쓰는 것 자체가 잘못은 아니다. 직접 측정할 길이 있는데 대리를 쓰는 경우는 드물다. 잘못은 둘 가운데 하나만 적을 때 생긴다. 무엇의 대리인가는 대개 적히고, 둘이 함께 움직인다는 믿음이 어디서 왔는가는 거의 적히지 않는다. 그리고 대리에는 시점의 문제가 딸려 온다. 측정하기 쉬운 값은 대개 즉시 나오고, 그 값이 대신하는 어려운 값은 대개 늦게 온다.

5. **지표가 목표가 되는 경우.** 어떤 값을 목표로 삼는 순간 그 값은 원래 대신하던 것을 더 이상

대신하지 못한다. 경제학에서 나온 관찰인데 머신러닝에서 매일 일어난다. 에이전트에서는 이렇게 나타난다. 형식 위반율이 내려간 것을 성공이라 부르면 시스템은 형식을 어기지 않는 쪽으로 다듬어진다. 그건 형식을 맞추는 능력이지 과제를 푸는 능력이 아니다. 제18장이 세기 쉬운 것만 세는 잘못으로 이미 적었다. 여기에 없을 것은 시간이다. 다듬어지는 것은 시스템만이 아니다. 지표를 세우고 나면 그것이 팀의 공용어가 되고, 개선 회의는 그 항목 위에서만 열린다. 항목에 없는 것을 두고 하는 주장은 말할 수는 있어도 수로 보일 수 없고, 수로 보일 수 없는 주장은 진다. 회의 장면으로 보면 이렇다. 검색을 손보자고 말하는 사람이 있고 근거는 지난주에 답이 이상했던 건 셋이다. 옆에서 묻는다. 그게 지표로는 몇입니까. 답이 없다. 그러면 회의는 프롬프트를 손보는 쪽으로 돌아간다. 그쪽에는 수가 있기 때문이다. 검색이 문제가 아니어서가 아니라 검색을 가리킬 수가 없어서다. 그렇게 몇 분기가 지나면 시스템은 지표의 모양대로 굳는다. 지표가 기울어 있었으면 그 기울기의 모양대로 굳는다.

마지막 하나에는 처방이 셋이다. 결정을 바꾸는 지표와 지켜보기만 하는 지표를 갈라 적어 둔다. 겨냥하는 것은 앞의 것뿐이다. 지표 목록 옆에 이 지표가 세지 않는 것의 목록을 함께 둔다. 그리고 사람이 표본을 읽는 자리를 남긴다. 지표에 없는 것을 보는 길은 이것뿐이다.

다섯은 따로 오지 않는다. 기준선이 없으면 무엇이 개선인지 알 수 없으므로 팀은 올라가는 수를 찾게 되고, 올라가는 수는 대개 측정하기 쉬운 쪽에 있다. 그렇게 고른 대리를 겨냥하기 시작하면 시스템이 그 모양으로 굳고, 굳는 동안 집합은 계속 바뀌므로 지난 분기와 견줄 길도 사라진다. 둘째와 넷째와 다섯째와 첫째가 한 고리로 이어져 있다. 그래서 하나만 고치면 대개 다시 걸린다.

다섯을 설계의 순서에 놓으면 자리가 정해진다. 첫째와 둘째는 지표를 적을 때 걸리고, 셋째와 넷째는 셀 수 있는 것으로 옮길 때 걸리며, 다섯째는 지표를 쓰기 시작한 다음에야 온다. 앞의 넷은 하루면 고치고 다섯째는 몇 분기가 걸린다.

다섯이 한 장에 모이면 앞 절의 계약서 지표가 이런 모양이 된다.

계약서 검토 품질 점수

세는 것: 검토 품질을 0에서 100으로 환산한 종합 점수

나오는 곳: 매주 화요일 자동 집계

지난주: 81

이번 주: 84

네 줄에 다섯이 다 들어 있다. 무엇을 어떻게 환산했는지가 없으니 다시 만들 수 없고, 84가 좋은 수인지 말해 줄 상대가 없으며, 농친 조항과 헛것이 하나의 점수로 접혀 있고, 품질을 직접 세는 대신 세기 쉬운 것들을 섞어 환산했으며, 이 수가 오르는 것이 그대로 목표로 적혀 있다.

그런데 이 카드를 만든 사람이 게을렀던 것은 아니다. 앞 절의 여덟 줄을 적으려면 담당자 옆에 만나질 앉아 있어야 하고, 이 네 줄은 한 시간이면 나온다. 둘의 차이는 성실함이 아니라 무엇을 미뤄도 된다고 보았는가다. 미룬 여섯 줄은 사라지지 않는다. 몇 분기 뒤에, 이 수를 근거로 선 결정들이 무엇 위에서 있었는지 아무도 답할 수 없는 형태로 돌아온다.

19.5 요약

한 문장으로. 에이전트에는 자리 잡은 지표가 없고 직무마다 잘한다는 뜻이 다르니 지표는 고르는 물건이 아니라 설계하는 물건이며, 설계란 그 일을 지금 하는 사람에게서 무엇이 잘못 인지를 받아 적고 셀 수 있는 것으로 옮긴 다음 옮기면서 잃은 것까지 적는 일이다.

확실해진 것.

- 머신러닝의 지표는 계산법이 어디서나 같아서 두 논문의 수를 나란히 놓을 수 있다. 에이전트에는 그것이 없고, 까닭의 절반은 분야가 자라는 중이어서이고 절반은 에이전트가 직무마다 다르게 만들어지기 때문이다(절 19.1).
- 설계는 과제가 아니라 그 일을 하는 사람에서 출발한다. 무엇이 잘못인지와 누가 판정하는지를 그 사람에게서 받아 적고, 판정하는 사람의 시간에서 집합의 크기가 나온다(절 19.2).
- 셀 수 있는 것으로 옮기면 잃는 것이 생긴다. 잃은 것을 지표 옆에 적어 두지 않으면 몇 달 뒤에 남은 항목이 이 일의 전부였다고 읽힌다.
- 같은 지표를 같은 시스템에 두 번 대면 두 수가 나오고 벌어짐은 다섯 자리에서 온다(표 19.1). 첫째는 시스템 자신의 성질이라 없앨 수 없고, 다섯째는 반복해도 줄지 않아 실행마다 같은 상태에서 시작하게 하는 길밖에 없다.
- 아무 결정도 바꾸지 않는 지표는 지표가 아니다. 무엇을 넘으면 무엇을 하는지가 적혀야 서고, 언제 버릴지도 함께 적는다.
- 사례를 늘릴 신호는 성적이 아니라 개선이 두 번 돌려 벌어지는 만큼보다 작아진 때다. 늘릴 때는 집합의 번호를 올리고 그 주에 두 판을 다 돌려 이음매를 만든다.
- 공개된 기록 셋이 같은 움직임을 보인다. 대조하는 방식을 과제에 맞춰 다시 만들고, 한 덩어리로 묻지 않고 가르고, 최종 결과 대신 고칠 수 있는 자리를 센다(절 19.3).
- 셋 가운데 어느 것도 자기 지표를 정확도라고 부르지 않았다. 이름도 정의도 자기 직무의 말로 적었고, 한 줄로 적을 수 있는 지표는 대개 남의 지표다.
- 두 실패의 값이 다르면 하나로 합친 수는 결정을 바꾸지 못한다. 그리고 이득이 즉시 도착하고 손실이 늦게 도착하면 모든 변경이 이득만 가진 상태로 심사된다(표 19.2).
- 지표를 세우면 시스템도 사람도 그 모양대로 굳는다. 결정을 바꾸는 지표와 지켜보기만 하는 지표를 갈라 두는 까닭이 그것이다(절 19.4).
- 다섯 가지 잘못은 한 고리로 이어져 있어 하나만 고치면 대개 다시 걸린다. 여섯 줄을 한꺼번에 채우는 대신 바꾸는 결정 한 줄부터 채운다.

아직 답하지 않은 것.

- 옮기면서 잃은 것을 적어 두는 데 정해진 형식이 없다. 이 장의 카드는 한 예일 뿐이다.
- 두 번 돌려 벌어지는 만큼을 얼마나 자주 다시 측정해야 하는지 알지 못한다.
- 늦게 도착하는 손실을 앞당겨 관측하는 일반적인 방법을 이 책은 알지 못한다.
- 지켜보기만 하는 지표를 몇 개까지 유지할 수 있는지 정해진 바가 없다.

오늘 할 것. 지금 팀이 보는 지표 가운데 하나를 골라 세 줄을 적어 본다. 이 수가 어떤 결정을 바꾸는가, 지난달 수와 같은 집합에서 나왔는가, 이것이 세지 않는 것은 무엇인가(절 19.2). 둘째 줄이 안 채워지면 집합에 번호를 붙이고 오늘부터 새 사례를 따로 쌓는다(절 19.2). 첫째 줄이 안 채워지면 그 지표를 지표 화면에서 내린다(절 19.4).

19.6 생각해 볼 문제

1. 지금 보는 지표를 하나 골라 그 지표를 쓰는 사람에게 직접 물어보라. 이 수가 얼마가 되면 무엇을 하느냐고. 답이 돌아오지 않으면 그 지표는 지금 무엇을 하고 있는가.
2. 자기 시스템의 실패 둘을 골라 값을 적어 보라. 놓친 것과 헛것 가운데 어느 쪽이 몇 배쯤 아픈가. 그 둘이 지금 하나의 수로 합쳐져 있지는 않은가.
3. 같은 평가를 두 번 돌려 벌어지는 만큼을 측정하라. 지난 분기에 개선이라고 보고한 변화 가운데 그보다 작은 것이 몇 개인가.
4. 지금 쓰는 지표 가운데 대리인 것을 고르고, 무엇의 대리이며 둘이 함께 움직인다고 믿는 근거가 어디서 왔는지 적어 보라. 적히지 않으면 그 근거는 누구의 기억에 있는가.

19.7 다시 읽을 때

- 제18장의 평가 항목 다섯으로 돌아오면, 그 다섯이 항목의 목록이 아니라 설계의 출발점이었다는 것이 보인다. 못 잡는 것을 적은 오른쪽 열이 여기서는 지표를 적을 때 함께 쓰는 줄이 된다.
- 제20장을 읽고 표 19.1로 돌아오면, 다섯 자리 가운데 셋은 실행의 기록이 없으면 어디서 온 벌어짐인지 가릴 수 없다. 벌어짐을 측정하는 일도 남긴 것 위에 선다.

19.8 더 읽을거리

이 장이 부른 문헌은 넷이다. 셋은 절 19.3의 기록이고 하나는 앞 장과 같다.

Patil et al. (2024)에서 가져온 것은 둘이다. 글자를 맞대는 대신 구문 나무의 부분 나무를 맞대는 대조와, 없는 도구를 지어낸 것과 있는 도구를 잘못 부른 것을 가른 구분이다. 제7장이 같은 자료에서 실패의 목록을 가져왔고 이 장은 세는 방법을 가져왔다.

Anthropic (2025e)에서 가져온 것은 다섯 항목으로 쪼갠 채점표와, 한 번의 호출로 점수와 통과·실패를 함께 내는 형식이 사람의 판정과 가장 잘 맞았다는 관측이다.

Anthropic (2024b)에서 가져온 것은 최종 답 대신 검색만 따로 센 지표와 그 수의 움직임 넷이다. 제11장과 제12장은 같은 자료에서 방법을 가져왔고 여기서 가져온 것은 무엇을 세었는가다.

Anthropic (2026c)에서 가져온 것은 둘이다. 실행 사이에 남는 상태가 상관된 실패를 일으킨다는 것과, 시스템이 여물면 더 작은 효과를 잡아내기 위해 더 크고 더 어려운 평가가 필요해질 수 있다는 것이다.

트레이싱과 관측

월요일 아침에 오는 물음은 대개 같다. 지난주 목요일에 이 에이전트가 왜 그렇게 답했습니까.

답하려면 그 목요일을 열어 봐야 한다. 그런데 그 실행은 이미 끝났고, 모델이 스텝마다 본 컨텍스트도 도구가 돌려준 것도 그 실행 안에서만 있었다. 지금 열 수 있는 것은 그때 남기기로 해서 남은 것뿐이다. 그리고 무엇을 남길지는 목요일보다 먼저 정해졌다. 남길 만큼 남겨 놓고도 정작 궁금한 것을 모르는 실행이 있다.

그래서 이 장은 두 가지 일을 갈라 놓고 시작한다. 한 실행이 지나간 길을 통째로 남기는 일이 하나이고, 남긴 것으로 무엇을 알 수 있느냐가 다른 하나다. 앞의 것을 트레이싱이라 하고 뒤의 것을 관측이라 한다. 두 낱말을 대개 붙여 쓰는데, 실무에서 어긋나는 자리가 바로 그 둘 사이다. 절 20.3이 그 자리이고 이 장의 중심이다.

20.1 왜 남겨야 하는가

앞의 여러 장이 미뤄 둔 물음이 여기서 한꺼번에 온다.

제18장이 세운 평가 항목 다섯 가운데 셋은 실행이 끝난 뒤에는 셀 수 없다. 도구를 골랐는가, 얼마나 들었는가, 사람이 손봤는가(표 18.1). 제12장은 찾는 일을 예닐곱 단계로 늘려 놓았고, 어느 단계에서 놓쳤는지 가르려면 단계마다 중간 결과가 남아 있어야 한다(절 12.5). 제17장에서 일을 나눈 뒤에는 어느 쪽에서 미끄러졌는지 물어야 하는데 올라오는 것은 마지막 메시지 하나뿐이다(절 17.2). 제10장에서 기억 한 줄이 틀린 것으로 드러나면 그 줄을 적은 실행으로 거슬러 가야 한다(절 10.4). 넷이 같은 것을 요구한다. 지나간 실행 하나를 다시 펼쳐 볼 수 있어야 한다.

그러면 로그를 보면 되지 않는가. 로그는 대개 이미 있다.

한 요청이 스무 걸음을 돈다. 걸음마다 모델을 한 번 부르고 걸음의 절반쯤에서 도구를 한두 번 부르니, 줄로 세면 예순 줄쯤이다. 그리고 그 예순 줄은 같은 시각에 도는 다른 요청 이백 건의 줄과 섞여 시각 순서로 쌓인다.

이렇게 쌓인 기록에는 세 가지를 물을 수 없다. 이 줄과 저 줄이 같은 요청인가. 같은 요청이라면 몇 번째 걸음인가. 그 걸음에서 모델 호출이 도구 호출보다 먼저였는가. 셋 다 잇는 일에 관한 물음이다. 로그의 단위는 줄이고 여기서 필요한 단위는 실행이다.

한 가지가 더 있다. 기록을 남기지 않아도 실행은 멀쩡히 돈다. 과제를 받고, 스텝을 돌고, 답을 낸다. 자르기를 잘못하면 검색 결과가 나빠지는데, 기록을 빠뜨리는 일은 그 실행에 아무것도 보태지 않고 아무것도 빠지 않는다. 기록은 그 실행을 위해 남기지 않는다. 나중에 그 실행을 묻는 사람을 위해 남긴다. 그래서 빠뜨렸다는 신호가 그 실행에서는 오지 않고, 물어야 할 월요일 아침에 온다.

20.2 무엇을 어떻게 남기는가

한 실행을 통째로 하나의 덩어리로 묶고 그 안에 계층을 둔다.

덩어리 바로 아래에 스텝이 오고, 스텝 안에 모델 호출과 도구 호출이 들어간다. 하위 에이전트를 열었으면 그쪽 실행이 통째로 부른 스텝 안에 들어간다. 트리 하나가 서는데, 이 트리의 모양이 루프가

실제로 돈 모양이다.

이름은 이미 있다. 덩어리 하나를 트레이스라 하고 그 안의 마디 하나를 스펀이라 한다. 마디마다 부모가 하나씩 있어서 계층이 선다.

실물을 보자. 제17장에서 결제 실패율이 화요일부터 오른 원인을 좁히던 그 과제다.

```
실행 8f2a1c 1분 48초 31,400 토큰 종료: 스스로 끝냄
|
+- 스텝 1
| +- 모델 호출 입력 4,120 출력 180 2.1초
| +- 도구 호출 search_logs("결제 실패", 08-18 07:00-09:00)
| +- 도구 결과 412줄 가운데 상위 40줄 1.4초
+- 스텝 2
| +- 모델 호출 입력 6,900 출력 240 3.0초
| +- 도구 호출 read_file("deploy.log", 400-430)
| +- 도구 결과 31줄 0.2초
+- 스텝 3
| +- 하위 실행 8f2a1c-2 log-search
| +- 스텝 1 모델 호출 입력 2,800 출력 150 1.6초
| +- 스텝 2 도구 호출 grep_range("결제 실패", 08-11)
| +- 스텝 2 도구 결과 0줄 0.3초
| +- 반환 status=partial 후보 2 못 한 일 1
+- 스텝 4
| +- 모델 호출 입력 12,300 출력 610 4.4초 더 부르지 않음
```

읽는 데 필요한 것이 다 있다. 스텝이 넷이고, 셋째에서 하위 에이전트를 열었으며, 그쪽 검색이 0 줄로 돌아왔고, 넷째에서 답을 냈다. 시간과 토큰이 마디마다 붙어 있어 1분 48초 가운데 어디에 얼마가 갔는지도 보인다.

무엇을 남기는지도 이 예에 다 나와 있다. 넷이다.

- 들어간 것과 나온 것. 모델 호출마다 그 호출의 입력 전문과 출력 전문이다. 토큰 수만 남기면 어디서 오래 걸렸는지는 알아도 왜 그렇게 답했는지는 모른다.
- 도구 호출과 그 결과. 이름, 인자, 돌려받은 것. 오류였으면 오류 문자열 그대로다.
- 걸린 시간과 토큰. 마디마다 따로 붙인다. 합계만 있으면 느려졌다는 것까지만 안다.
- 어느 스텝의 어느 호출인지. 앞의 셋을 잇는 것이 이 하나다. 마디에 부모의 식별자가 없으면 나머지 셋은 다시 흩어진 줄이 된다.

넷째가 나뉜 자리에서 가장 자주 빠진다. 하위 에이전트는 대개 다른 프로세스에서 돌고, 그쪽은 자기 기록을 자기 자리에 적는다. 여는 쪽이 실행의 식별자를 함께 넘기지 않으면 두 기록이 끝내 만나지 않는다. 넘길 것의 목록에 식별자 한 칸을 더하는 일인데, 그 목록은 제17장이 이미 세워 두었다.

언제 적는가

마디를 언제 내보내는지도 정해야 한다. 실행이 끝난 뒤에 한꺼번에 보내는 쪽이 만들기 쉽다. 트리가 다 서 있으니 그대로 직렬화해서 한 번에 넘기면 된다.

그런데 끊긴 실행에서는 그 트리가 오지 않는다. 프로세스가 죽거나 시간 상한에 걸려 잘린 실행은 트레이스를 통째로 잃는다. 그리고 그 실행들이 정확히 나중에 열어 보고 싶은 실행들이다. 잘 끝난 실행의 기록만 남고 끊긴 실행의 기록은 없는 시스템은 성공만 관측되는 시스템이다.

그래서 마디는 닫히는 대로 내보낸다. 스텝 하나가 끝나면 그 스텝의 마디를 보내고 실행이 끝나면 실행의 마디를 닫는다. 중간에 죽어도 거기까지는 남는다.

넷을 어떤 형태로 적는가도 정해야 한다. 사람이 읽을 문장으로만 적으면 한 건을 열어 볼 수는 있어도 실행들을 가로질러 셀 수 없다. 「지난달에 이 도구를 부른 실행이 몇 건인가」는 트레이스 한 건을 아무리 자세히 읽어도 안 나오는 물음이다. 값마다 이름이 붙은 형태로 적어야 세어진다. 다음 절이 요구하는 것이 바로 그 가로지르기다.

표 20.1: 남겨도 알 수 없는 다섯. 가운데 열이 트레이스에 실제로 적히는 것이고, 셋째 열이 그것으로 갈리지 않는 이유다. 다섯 다 오류를 내지 않는다.

알고 싶은 것	기록에 남는 것	왜 갈리지 않는가	세운 곳
빈손으로 돌아온 검색	「0줄」	문서에 없어서인지 질의가 어긋나서인지, 두 사정이 똑같은 한 줄로 온다	절 11.5
안 부른 도구	부른 호출 넷	부를 수 있었는데 안 부른 것은 호출이 아니므로 호출의 기록에 자리가 없다	제7장
실리지 않은 조각	실린 조각 다섯	후보에 올랐다가 잘린 조각은 컨텍스트에 없었고, 기록은 컨텍스트를 적는다	제4장
답에 안 쓰인 조각	실린 조각 다섯 전부	실렸다는 것까지만 남는다. 답이 그것을 썼는지 배운 통념을 썼는지는 안 남는다	절 11.5
밀려서 안 읽힌 것	「입력 12,300 토큰」	실렸는데 모델이 지나간 사건은 답이 나빠진 모습으로만 온다	그림 4.2

20.3 남겨도 알 수 없는 것

앞 절의 기록은 꽤 충실하다. 그런데 그 기록으로 답할 수 없는 물음이 있다.

앞의 트레이스로 돌아가 보자. 셋째 스텝의 하위 실행에서 검색이 0줄로 돌아왔다. 이제 물어야 할 것은 화요일 이전 로그가 정말 없었는지, 아니면 있는데 질의가 어긋나 못 찾았는지도. 트레이스에는 0줄이라고만 적혀 있다. 둘을 가를 줄이 없다.

같은 자리가 다섯이다(표 20.1).

다섯을 나란히 놓으면 하나가 같다. 다섯 다 성공한 실행과 똑같은 얼굴로 기록에 남는다. 기록에 적히는 것은 일어난 일인데 이 다섯의 원인은 전부 일어나지 않은 일이고, 일어나지 않은 일에는 적힐 자리가 없다.

그렇다고 없는 것을 전부 적을 수도 없다. 이번 스텝에 실리지 않은 것은 세상의 거의 전부다.

그래서 순서를 뒤집는다. 무엇을 남길지부터 정하지 않는다. 무엇을 알고 싶은지를 먼저 적고, 그 물음에 답하려면 무엇이 남아 있어야 하는지를 거꾸로 따라간다. 표 20.1의 첫 열이 그 물음의 목록이고, 남길 것은 거기서 나온다.

그래서 무엇을 더 남기는가

앞 장들에 흩어져 있던 처방을 여기서 모은다. 앞의 넷에는 길이 있고 마지막 하나에는 없다.

- **검색이 자기 사정을 함께 돌려주게 한다.** 질의 문자열, 훑은 범위, 문턱을 넘기 전의 후보 수, 최고 점수, 자른 문턱이다. 후보가 0이었으면 없는 쪽이고, 후보가 마흔인데 최고 점수가 문턱 아래였으면 못 찾은 쪽이다. 0줄이라는 한 줄이 두 줄로 갈린다. 반환에 칸을 더하는 일이므로 제9장이 스키마를 두고 세운 그대로다. 칸이 없으면 채울 수조차 없다.
- **안 부른 것을 세려면 부를 수 있었던 것의 목록이 필요하다.** 스텝마다 그때 컨텍스트에 실려 있던 도구 이름을 함께 적는다. 열둘이 실려 있었고 하나를 불렀다면 나머지 열하나가 「후보였는데 안 불렀다」로 남는다. 목록에 아예 없던 도구는 여전히 안 남는데, 그쪽은 기록의 모자람이 아니라 어휘의 모자람이다(제7장).
- **조립할 때 뻥 것을 센다.** 실은 다섯 조각만 적지 말고 후보 마흔과 자른 서른다섯을 함께 적는다. 자른 것은 식별자와 점수만 남겨도 되므로, 늘어나는 크기가 실은 조각 전문의 몇 분의 일에 그친다.

- **답이 그것을 썼는지는 반쯤만 남길 수 있다.** 답에 출처를 달게 하면 쓴 것은 드러난다. 그런데 조각의 숫자를 절반만 쓰고 배운 통념을 절반 섞은 답에도 출처가 그럴듯하게 달린다(절 11.5). 출처가 붙었다는 것과 그 문장이 조각에서 나왔다는 것은 다르다.
- **다섯째는 길을 모른다.** 실렸는데 모델이 지나간 사건을 관측하는 방법을 이 책은 알지 못한다. 컨텍스트 안에서 후보로 올랐다가 밀린 것까지가 지금 적을 수 있는 한계다.

처방을 붙이면 앞의 셋째 스텝이 이렇게 바뀐다.

```
+-- 스텝 3
| +- 하위 실행 8f2a1c-2    log-search
|   +- 스텝 2    도구 호출 grep_range("결제 실패", 08-11)
|     +- 스텝 2    도구 결과 0줄 0.3초
|       |  혹은 범위 logs/2026-08-11, logs/2026-08-12
|       |  후보 0건  최고 점수 없음  문턱 0.35
|       |  08-14 이전 파일이 저장소에 없음
|       +- 스텝 2    실행 있던 도구 grep_range, read_file,
|         |  list_files, search_logs    (넷 가운데 하나를 부름)
|       +- 반환 status=partial  후보 2    못 한 일 1
```

세 줄이 늘었고 물음 하나가 답을 얻었다. 후보가 0건이었고 그 기간의 파일이 저장소에 아예 없다. 절의가 어긋난 것이 아니라 로그가 보존 기간을 지나 지워졌다. 고칠 데가 검색이 아니라 보존 기간이라는 것을 이제 안다. 처방을 붙이기 전에는 이 실행을 몇 시간 들여다봐도 같은 결론에 닿지 못한다.

한 가지가 더 남는다. 부재는 실행 하나를 열어 보는 것으로는 잘 안 보인다. 트레이스 한 건에서 안 부른 도구 열 하나는 그냥 안 부른 것이고 이상할 데가 없다. 같은 과제 백 건을 나란히 놓았을 때 아흔일곱 건이 그 도구를 불렀고 셋만 안 불렀다면, 그때 그 셋이 무늬가 된다. 기록의 단위는 실행인데 부재를 보는 단위는 실행의 무리다. 앞 절이 값마다 이름을 붙여 적으라고 한 이유가 여기 있다. 이름이 붙어 있어야 백 건을 가로질러 셀 수 있다.

핵심. 트레이싱과 관측이 어긋나는 자리가 여기다. 트레이스가 아무리 충실해도 거기 적히는 것은 일어난 일이고, 조용한 실패의 원인은 대개 일어나지 않은 일이다. 없는 것을 다 적을 수는 없으므로 대신 고르는 자리를 적는다. 검색에는 후보와 문턱을, 도구에는 부를 수 있었던 목록을, 조립에는 자른 것의 식별자를 붙인다. 그러면 다섯 가운데 넷이 같린다. 실렸는데 읽히지 않은 것은 여전히 남지 않는다.

20.4 실무에서 어떻게 하는가

표준이 하나 있다. OpenTelemetry가 일반 소프트웨어의 분산 트레이싱 규격에 생성 모델용 항목을 얹은 것으로, 마디의 이름과 마디에 붙일 값의 이름을 정해 둔다(OpenTelemetry, 2026).

정해 둔 계층이 앞 절의 트리와 같다. 마디마다 종류를 가리키는 이름이 붙는데, 셋이 앞 절의 셋과 그대로 겹친다. 실행 하나에 `invoke_agent`, 그 아래 모델 호출에 `chat`, 도구 호출에 `execute_tool`이 붙는다. 검색과 계획에도 각각 이름이 있다. 실행 하나를 트레이스로 묶고 그 안의 호출을 부모가 있는 마디로 보는 방식이 규격 쪽에서도 같다.

보통의 트레이싱에 없다가 모델 호출에 붙는 것이 이 규격의 알맹이다. 넷을 보면 된다.

- **토큰 두 값.** `gen_ai.usage.input_tokens`와 `gen_ai.usage.output_tokens`가 따로 있다. 값이 다르게 매겨지므로 합계 하나로는 제18장의 「얼마나 들었는가」를 못 센다.
- **모델의 이름.** `gen_ai.request.model`이다. 밖에서 바뀌는 것 가운데 가장 자주 바뀌고 바뀐 사실이 통보되지 않는다. 이 값이 없으면 지난달과 이번 달의 수가 같은 시스템에서 나온 수인지 알 수 없다.

- **끝난 이유.** `gen_ai.response.finish_reasons`다. 스스로 끝냈는지 길이 상한에 걸려 잘렸는지가 갈린다. 제15장이 종료를 여러 갈래로 나눈 그 갈래가 여기 한 칸으로 온다.
- **도구 호출의 식별자.** `gen_ai.tool.name`과 `gen_ai.tool.call.id`다. 모델이 한 스텝에서 도구를 셋 부르고 결과가 순서 없이 돌아와도 어느 결과가 어느 호출의 것인지 이 값으로 이어진다.

본문은 기본으로 안 남는다

여기서 한 가지를 짚어야 한다. 앞 절이 첫째로 꼽은 것, 그러니까 모델 호출의 입력 전문과 출력 전문은 이 규격에서 선택 항목이다. `gen_ai.input.messages`와 `gen_ai.output.messages`라는 이름이 있고, 계기를 켜는 환경 변수를 따로 켜야 남는다. 이유가 규격에 적혀 있다. 그 자리에 사람에 관한 것이 들어 있을 수 있고, 대개 크기가 크기 때문이다.

그래서 기본값을 그대로 두고 몇 달을 돌린 시스템에서는 그 몇 달이 통째로 물을 수 없는 구간이 된다. 토큰 수와 지연은 다 있는데 왜 그렇게 답했는지가 없다. 절 20.5가 다루는 문제와 이 절이 다루는 문제가 같은 스위치 하나에 걸려 있다.

규격의 상태도 알아 둘 만하다. 2026년 여름 현재 `gen_ai`로 시작하는 항목은 전부 개발 단계로 표시되어 있고 안정으로 옮겨 간 것이 아직 하나도 없다. 관련 문서가 별도 저장소로 갈라져 나온 것도 최근이다. 그러니 이름은 바뀔 수 있다고 보는 편이 맞다. 자리를 잡은 쪽은 이름이 아니라 계층과 붙는 값의 종류다.

도구들이 남기는 것과 안 남기는 것

언어 모델 쪽 관측 도구들은 대체로 같은 형태를 내놓는다. 실행 하나를 트리로 그려 놓고, 마디마다 프롬프트와 응답, 검색해 온 조각, 토큰과 지연을 붙인다. 제19장의 지표를 운영 트래픽의 마디에 붙여 두는 기능도 대개 있다. 어느 것을 골라도 앞 절의 넷은 나온다.

안 나오는 것이 표 20.1의 셋째 열이다. 검색의 사정, 부를 수 있었던 도구의 목록, 잘린 후보. 계기를 붙이는 것만으로는 생기지 않는다. 남기는 일은 도구가 해 주고 알 수 있게 만드는 일은 여전히 설계다.

표준 쪽도 마찬가지다. 검색 마디의 이름은 정해져 있는데 그 마디에 무엇을 붙일지는 모델 호출만큼 자세히 적혀 있지 않다. 후보가 몇이었는지, 문턱이 얼마였는지, 몇 개를 자르고 몇 개를 실었는지를 담은 자리가 아직 이름을 갖지 못했다. 앞 절이 요구한 것 가운데 표준이 대신 세워 주는 것은 하나도 없다고 보는 편이 맞다. 이름을 직접 정해 붙이면 된다. 나중에 표준 이름이 생기면 그때 옮기는 값이지 지금 기다릴 값이 아니다.

어디에 붙이는가

계기를 붙이는 자리가 셋이다.

모델을 부르는 자리를 한 겹 감싸면 모델 호출은 전부 잡힌다. 계기를 붙이는 일이 대개 여기서 시작하고, 여기까지는 몇 줄이면 된다. 그런데 이것만 하면 트레이스가 모델 호출의 평평한 목록이 되고, 스텝의 경계도 도구 호출도 보이지 않는다.

둘째는 루프를 도는 자리다. 스텝이 어디서 시작하고 어디서 끝나는지 아는 것은 하네스뿐이므로 스텝 마디는 거기서만 열 수 있다(제15장). 셋째는 컨텍스트를 조립하는 자리다. 무엇을 후보로 놓고 무엇을 실었는지 아는 것도 그 자리뿐이다.

셋 가운데 첫째만 붙인 시스템이 가장 흔하다. 토큰과 지연은 다 있는데 왜 그렇게 답했는지는 없는 그 트레이스가 대개 여기서 나온다.

공개된 실무 기록

다중 에이전트를 운영한 기록 하나가 이 자리를 짧게 적어 두었다. 운영 트레이싱을 전부 붙이고 나서야 왜 실패했는지 진단하고 고칠 수 있었다고 적는다. 그러면서 무엇을 보았는지도 함께 적어 두었다. 결정의 무늬와 상호작용의 구조를 보고 개별 대화의 내용은 보지 않았다(Anthropic, 2025e). 절 20.5가 요구하는 형태가 실무에 이미 있다.

같은 기록에 한 문장이 더 있다. 에이전트는 같은 프롬프트로도 실행마다 다르게 움직인다. 이 문장이 기록으로 살 수 없는 것을 가르다. 기록을 아무리 늘려도 되는 일은 그때 무엇을 보고 무엇을 했는지 읽는 일까지다. 같은 조건에서 한 번 더 돌려 같은 것을 얻는 일은 기록의 양으로 되지 않는다. 그쪽에 필요한 것은 구성을 고정하는 일이고, 위의 넷 가운데 모델 이름이 그 첫 칸이다.

20.5 남기는 값과 남기지 말아야 할 것

드는 것부터 본다. 저장 공간과 그것을 읽는 품이다.

앞의 트레이스가 실행 하나에 31,400 토큰이었다. 스텝마다 입력 전문을 남기면 앞 스텝의 내용이 뒤 스텝의 입력에 다시 들어가므로 실제로 적히는 양은 그보다 크고, 텍스트로 60킬로바이트쯤이 된다. 하루 만 건이면 하루 600메가바이트이고 한 달이면 18기가바이트다. 저장이 실행량에 비례해 늘어난다는 말이 이 뜻이다.

드는 것이 저장 공간만은 아니다. 하루 만 건 가운데 실패가 이백 건이라면 저장은 감당해도 사람이 다 읽지는 못한다. 한 건을 열어 스텝을 따라 걷는 데 십 분이 걸리니 이백 건이면 서른세 시간이다. 읽을 수 있는 양이 남길 양을 정하는 두 번째 제약이다. 그래서 실패를 유형으로 묶는 일이 남기는 일과 같은 결정에 든다. 이백 건이 다섯 유형으로 묶이면 유형마다 두세 건씩 열어 보는 일로 줄어든다. 그래서 무엇을 얼마나 남길지 정해야 한다. 실무에서 쓰이는 형태가 셋이다.

- 실패는 온전히, 성공은 표본으로. 오류로 끝났거나 지연이 문턱을 넘은 실행은 전부 남기고 평범하게 끝난 실행은 몇 퍼센트만 남긴다. 성공을 조금 남기는 까닭은 견줄 상대가 필요하기 때문이다. 실패만 모아 놓으면 그 무늬가 실패에만 있는 무늬인지 알 수 없다.
- 층을 나눈 보존. 본문까지 통째로 두는 기간을 짧게 잡고, 그 뒤로는 토큰 수와 지연과 결과만 남긴다. 실무에서 앞의 기간은 대개 일주일에서 한 달이다. 오래된 기록에서 필요한 것은 대개 무늬이지 한 건의 전문이 아니다.
- 붙잡는 일과 보관하는 일을 가르다. 무엇을 붙잡을지는 실행 전에 정해야 하고, 무엇을 보관 할지는 실행이 끝난 뒤에 정해도 된다. 그러니 넓게 붙잡고 좁게 보관하는 쪽이 좁게 붙잡고 전부 보관하는 쪽보다 낫다.

남기지 말아야 할 것

그리고 아예 남기면 안 되는 것이 있다. 셋이다.

- 개인정보. 컨텍스트에 들어온 것이 그대로 트레이스에 남는다. 사용자가 붙여 넣은 문서, 검색해 온 사내 자료, 도구가 조회해 온 고객 기록이 전부 모델 호출의 입력 전문이고, 입력 전문을 남기기로 하면 그것이 함께 남는다.
- 자격 증명과 열쇠. 도구 호출의 인자와 헤더에 들어간다. 이쪽은 사람이 읽을 일이 거의 없는 자리라 몇 달 뒤에야 발견된다.
- 지워 달라는 요구가 닿지 않는 사본. 요구가 오면 트레이스도 지워야 하는데, 그러려면 어느 트레이스가 그 사람의 것인지 찾을 수 있어야 한다. 지우기 위해 식별자를 남겨 두어야 하는 셈이다. 삭제를 나중에 붙일 수 없는 이유가 여기 있다.

가려도 새는 자리가 하나 있다. 이름이나 계좌 번호는 규칙으로 잡히는데 문장 안에 녹아 있는 사실은 규칙으로 잡히지 않는다. 「이 고객이 지난달에 환불을 세 번 요청했다」에서 이름을 지워도 나머지가 그대로 남고, 앞뒤의 다른 마디와 맞추면 누구인지 좁혀진다. 그래서 가리기를 걸어 두고 안심하는 대신 애초에 무엇을 붙잡을지의 목록에서 이 부류를 따로 갈라 둔다. 나중에 갈라내는 일은 기록을 통째로 다시 만드는 일이 된다.

셋을 관통하는 것이 하나다. 이걸 실수로 남는다. 누구도 개인정보를 트레이스에 적기로 결정하지 않는다. 컨텍스트를 통째로 남기기로 결정할 뿐이고, 그 결정이 안에 든 것을 함께 데려간다. 그래서 가리는 일은 호출하는 자리마다 두면 안 되고 기록이 만들어지는 한 자리에 두어야 한다. 호출마다 기억하게 하면 언젠가 빠진다.

남겨 놓고 값을 못 하는 둘

- **읽는 절차 없이 늘린다.** 남기고 있다는 사실은 보고 있다는 뜻이 아니다. 게다가 어긋남의 방향이 나쁘다. 기록이 풍부한 시스템일수록 찾아보면 나올 것이라는 짐작이 자리를 채워 실제로 찾아보는 일이 줄어든다. 산업계 자료가 전사를 보는 도구에 따로 투자했다고 적고 전사를 읽는 일을 중요한 기량으로 꼽은 것은 그것이 저절로 되지 않기 때문일 것이다(Anthropic, 2026c). 실무에서 그 절차는 대개 둘이다. 실패한 기록을 유형으로 묶어 무늬를 보는 일과, 거기서 평가의 사례를 뽑는 일이다. 뒤의 것이 제18장의 요구가 이 장의 재료 위에서 실행되는 자리다.
- **진단의 기록을 시스템이 읽는 자리에 둔다.** 이쪽이 더 조용하다. 진단하려고 남긴 것은 실패한 시도와 기각된 후보와 틀린 결론으로 가득한 문서다. 그것이 검색되는 자리에 놓이면 다음 실행이 그것을 근거로 가져올 수 있고, 시스템이 자기가 이미 기각한 것을 자기 기록에서 다시 배운다. 기록의 도착지를 정하는 일은 저장의 문제처럼 보이지만 신뢰 경계의 문제다. 그 방어는 제22장이 받는다.

둘을 관통하는 형태가 하나다. 기록은 그 자체로 값을 하지 않는다. 읽는 절차가 있어야 값이 돌아온다. 그러므로 무엇을 남길 것인가라는 물음에는 언제나 짝이 따라붙는다. 이것을 누가 읽고, 어디에 두며, 무엇을 읽지 않을 것인가.

그래서 남기는 결정과 읽는 결정을 같은 자리에서 함께 내리는 편이 낫다. 남기는 쪽만 정해 놓으면 저장은 늘어나는데 읽는 사람은 늘지 않고, 여섯 달 뒤에는 아무도 열어 보지 않는 기가바이트가 쌓인다. 짝 없는 기록은 관측이 아니라 저장이다.

20.6 요약

한 문장으로. 트레이싱은 한 실행이 지나간 길을 트리로 남기는 일이고 관측은 그것으로 무엇을 알 수 있느냐인데, 트레이싱에 적히는 것은 일어난 일이고 조용한 실패의 원인은 대개 일어나지 않은 일이라서 둘이 어긋난다.

확실해진 것.

- 로그의 단위는 줄이고 여기서 필요한 단위는 실행이다. 예순 줄이 다른 요청 이백 건의 줄과 섞이면 어느 줄이 같은 요청의 몇 번째 걸음인지 물을 수 없다(절 20.1).
- 실행 하나를 덩어리로 묶고 그 안에 스텝과 호출을 계층으로 둔다. 남길 것은 넷이다. 들어간 것과 나온 것, 도구 호출과 결과, 시간과 토큰, 그리고 어느 스텝의 어느 호출인지다(절 20.2). 넷째가 나뉘는 자리에서 가장 자주 빠진다.
- 남겨도 알 수 없는 것이 다섯이고 다섯 다 성공한 실행과 똑같은 얼굴이다(표 20.1). 그래서 무엇을 남길지가 아니라 무엇을 알고 싶은지에서 거꾸로 정한다.
- 넷은 처방이 있다. 검색에 후보 수와 문턱을, 스텝에 부를 수 있었던 도구의 목록을, 조립에 자른 후보의 식별자를 붙인다. 실패했는데 읽히지 않은 것에는 처방이 없다(절 20.3).
- 표준이 정해 둔 계층이 이 트리와 같고, 모델 호출에는 토큰 두 값과 모델 이름과 끝난 이유와 도구 호출 식별자가 붙는다. 다만 입력과 출력의 본문은 선택 항목이라 따로 켜야 남는다(절 20.4).
- 기록을 늘려서 되는 일은 그때 무엇을 보고 무엇을 했는지 읽는 일까지다. 같은 조건에서 다시 돌려 같은 것을 얻는 일은 기록의 양으로 되지 않고 구성을 고정해야 된다.
- 실패는 온전히 남기고 성공은 표본으로 남긴다. 넓게 붙잡고 좁게 보관한다(절 20.5).
- 개인정보와 자격 증명은 실수로 남는다. 따로 적어 넣어서가 아니라 컨텍스트를 통째로 남기다 보면 함께 들어간다. 그래서 가리는 일은 기록이 만들어지는 한 자리에 둔다.

아직 답하지 않은 것.

- 실패했는데 읽히지 않은 것을 관측하는 길을 알지 못한다. 표 20.1의 다섯째 줄은 이 장의 처방 밖에 있다.
- 표본을 몇 퍼센트로 잡고 본문을 며칠 두어야 하는지에 일반해가 없다.
- 기록이 모자랐다는 사실 자체에 지표가 없다. 남기지 않아서 못 푼 문제의 목록은 원리적으로 만들 수 없다.
- 진단의 기록이 언제 시스템의 코퍼스가 되어도 좋은지의 문턱도 없다. 갈라 두라는 규율까지가 이 장의 결론이다.

오늘 할 것. 지금 남기고 있는 것을 열어 본다. 어제 실행 하나를 골라 그 실행의 모든 호출이 하나로 이어져 보이는지, 아니면 시각 순서의 줄로 흩어져 있는지 확인한다(절 20.1). 이어져 있다면 그 실행의 검색 호출 하나를 골라, 돌려받은 조각 말고 후보가 몇 개였고 문턱이 얼마였는지 적혀 있는지 찾아본다(절 20.3). 마지막으로 모델 호출의 입력 전문이 남고 있는지, 남고 있다면 거기에 사람에게 관한 것이 들어 있는지를 한 건 열어 확인한다(절 20.5).

20.7 생각해 볼 문제

1. 최근 실패 하나를 골라 기록만으로 원인을 짚어 보라. 어디서 막혔는가. 막힌 자리가 표 20.1의 다섯 가운데 하나인지, 아니면 이를 식별자가 없어서인지 가려 보라.

2. 지금 도는 검색 호출의 반환 형식을 열고, 후보 수와 최고 점수와 문턱을 담은 칸을 더해 보라. 칸을 더한 뒤에 지난주 「결과 없음」 열 건이 두 부류로 갈리는지 확인하라.
3. 트레이스 하나를 열어 사람에 관한 것이 들어 있는 자리를 전부 표시하라. 그것을 가리는 코드가 지금 몇 군데에 흩어져 있는가. 한 자리로 모으는 데 드는 품을 적어 보라.
4. 지금 남기는 기록 가운데 시스템이 검색해 올 수 있는 자리에 놓인 것을 목록으로 만들라. 그중 진단을 위해 남긴 것이 있다면 그것이 다음 실행의 컨텍스트에 실리는 경로를 그려 보라.

20.8 다시 읽을 때

- 제22장을 읽고 절 20.5의 마지막 항목으로 돌아오면, 기록의 도착지를 정하는 일이 방어의 언어로 다시 보인다. 시스템이 읽을 수 있는 자리에 놓인 것은 전부 지시로 읽힐 수 있는 텍스트이며, 그 자리에 하필 실패의 기록이 놓여 있다.
- 제23장을 읽고 이 장의 요구로 돌아오면, 남길 자리를 손으로 십지 않아도 생기게 해 주는 계층이 있다는 것이 보인다. 그런데 그 계층이 세워 주는 것은 절 20.2의 넷까지다. 표 20.1의 셋째 열은 거기서도 직접 붙여야 한다.

20.9 더 읽을거리

이 장이 부른 문헌은 셋이다. 앞 둘은 앞 장들이 이미 썼고 마지막 하나가 이 장에서 처음 나온다.

Anthropic (2025e)에서 가져온 것은 둘이다. 운영 트레이싱을 전부 붙이고 나서야 실패의 원인을 진단하고 고칠 수 있었다는 보고와, 그러면서 결정의 무늬와 상호작용의 구조를 보고 개별 대화의 내용은 보지 않았다는 진술이다. 제17장은 같은 자료에서 남길 것의 목록을 가져왔고 이 장이 가져온 것은 무엇을 보았고 무엇을 보지 않았는가다. 같은 자료의 또 한 문장, 곧 같은 프롬프트로도 실행마다 다르게 움직인다는 관측이 기록으로 살 수 없는 것을 가르는 근거다.

Anthropic (2026c)에서 가져온 것은 하나다. 전사를 보는 도구에 따로 투자했고 전사를 읽는 일이 중요한 기량이라는 진술이다. 권고가 아니라 관행의 보고이므로 가져갈 것은 전사가 저절로 읽히지 않는다는 사실이다.

절 20.4의 표준은 공개된 규격이다(OpenTelemetry, 2026). 규격 문서를 처음부터 읽을 일은 아니고, 마디 이름의 목록과 모델 호출에 붙는 값의 목록만 열어 보면 된다. 자기 시스템이 지금 남기는 것과 대조해 보는 데는 그 두 목록으로 충분하다.

재독은 제4장이다. 잘못 넣은 것은 어디엔가 있고 잘못 뺀 것은 어디에도 없다는 두 문장이 이 장 전체의 문제였다.

제6부는 세 장이었고 하나의 논증이었다. 제18장은 무엇을 성공이라 부를지 정하는 일이 설계의 첫 결정이라고 답했고, 제19장은 그렇게 세운 수가 정직하게 계산된 채로 사람을 잘못 인도한다고 답했으며, 이 장은 앞의 둘이 전부 기록 위에서 있고 그 기록은 실행이 끝나기 전에만 만들 수 있다고 답했다. 논리의 순서는 정의와 의심과 기록이고 시간의 순서는 정반대다. 제7부는 이제 이 시스템을 돌리고 지키는 일로 넘어간다.

제 VII 부

운영, 안전, 거버넌스

인간 개입 설계와 권한 모델

승인 화면을 하나 붙일 때마다 자동화가 그만큼 줄어든다.

에이전트가 한 과제를 끝내는 데 도구를 서른 번 부른다고 하자. 그 가운데 다섯 번 앞에 사람이 서 있으면, 그 과제는 사람이 다섯 번 멈춰 서서 처리하는 과제다. 손으로 하던 때보다 나아진 것은 나머지 스물다섯 번뿐이다. 승인 지점을 스무 개로 늘리면 남는 것이 거의 없다.

그렇다고 하나도 세우지 않으면 사고가 난다. 지운 파일은 돌아오지 않고, 나간 돈은 돌아오지 않고, 읽힌 메일은 안 읽은 것이 되지 않는다.

그러면 그 사이 어디에 선을 긋는가. 이 물음이 이 장이다.

어려운 것은 그 선이 코드에 적혀 있지 않다는 사정이다. 코드에는 무엇을 할 수 있는지만 적혀 있다. 고객 등급을 바꾸는 일도 임시 파일을 지우는 일도 값 하나를 고치는 호출이고, 함수 이름에도 인자 형식에도 어느 쪽이 위험한지는 없다. 위험은 그 값이 시스템 바깥에서 무엇을 뜻하는지에 있는데, 그 뜻은 코드 밖에 있다.

제18장은 모델이 답을 확률로 뽑는 한 마지막 승인이 사람 쪽에 남는다고 했다(절 18.2). 그것은 승인이 남는다는 판정이었다. 여기서 정하는 것은 그 승인을 어디에 두느냐다.

21.1 어디에 사람을 세울 것인가

자리를 찾는 길이 둘이다.

하나는 그 일을 지금 사람 손으로 하고 있는 실무자에게 묻는 길이다. 다른 하나는 되돌릴 수 없는 행동을 목록으로 만드는 길이다. 앞의 것은 어디서 이미 멈추고 있는지를 알려 주고, 뒤의 것은 어디서 멈춰야 하는데 아무도 멈추지 않았는지를 알려 준다.

둘 다 필요하다. 실무자가 하지 않던 일을 에이전트는 하게 되므로 첫째 길만으로는 목록에 빠지는 자리가 생긴다. 그렇다고 코드에서만 세면 실무자가 아는 사정이 통째로 빠진다. 이어지는 두 절이 그 두 길이며 이 장의 중심이다.

결정의 순서. 아래로 갈수록 사람의 시간이 더 든다. 위에서 끝나는 것을 아래로 내리지 않는다.

1. **아예 못 하게 할 수 있는가.** 도구 목록에서 빼면 어떤 프롬프트로도 일어나지 않는다. 뺄 수 없는 것만 아래로 내려간다(절 21.4).
2. **실무자는 어디서 멈추는가.** 지금 그 일을 하는 사람에게 묻는다. 그 대답이 승인 지점의 첫 후보다(절 21.2).
3. **되돌리면 없던 일이 되는가.** 기술적으로 취소되는가를 묻는 것이 아니다(절 21.3).
4. **되돌리는 데 무엇이 드는가.** 되돌릴 수 있어도 품이 크면 그냥 두지 않는다. 품을 줄이는 장치를 사람보다 먼저 쓴다(절 21.3).
5. **남은 것을 셋으로 가른다.** 자동으로 하는 것, 사람이 보고 누르는 것, 아예 못 하는 것(절 21.4).

21.2 실무자에게 묻는다

첫째 출처는 사람이다.

에이전트를 만들기 전에 그 일을 지금 하고 있는 사람과 앉아서 이야기한다. 다 만들고 보여 주는 자리가 아니라 시작하기 전에 묻는 자리다. 물을 것은 다섯이다.

- 지금 이 일을 어떻게 하는가. 처음부터 끝까지 순서대로.
- 어디서 한 번 멈추고 확인하는가.
- 그 자리에서 무엇을 보고 넘어가는가.
- 무엇이 잘못되면 곤란한가. 곤란해지는 사람이 누구인가.
- 실수했을 때 어떻게 되돌리는가. 되돌려 본 적이 있는가.

둘째 물음의 대답이 그대로 승인 지점의 후보가 된다. 넷째 물음은 뒤에 붙은 한 마디가 본체다. 곤란해지는 사람이 누구인가. 이쪽이 곤란한 것과 고객이 곤란한 것은 서로 다른 목록이고, 물어보면 대개 뒤쪽이 나온다. 만드는 사람은 앞쪽을 먼저 떠올린다.

왜 이것이 먼저인가. 그 사람이 이미 그 판단을 하고 있기 때문이다. 어디서 멈추지는 새로 만들 결정이 아니라 이미 내려져 있는 결정이고, 에이전트를 만드는 일은 그 결정을 사람 머리에서 코드로 옮기는 일이다. 없던 것을 세우는 쪽이 훨씬 어렵다. 옮기면 되는 것을 새로 세우려 들면, 그 일을 십 년 한 사람이 하지 않는 실수를 첫 주에 한다.

멈추는 자리를 들었으면 왜 멈추는지도 묻는다. 규정에 그렇게 적혀 있어서 멈추는 자리와 여기서 틀리면 되돌릴 길이 없어서 멈추는 자리가 섞여 있기 때문이다. 앞쪽은 규정이 바뀌면 없어지고 뒤쪽은 시스템이 바뀌어야 없어진다. 둘을 갈라 두지 않으면 나중에 승인을 걷을 때 어느 쪽을 걷는 것인지 모른 채로 걷는다.

그리고 물어야만 알 수 있는 것이 있다. 「이건 잘못 나가면 고객이 바로 압니다」 같은 말이다. 이 문장은 코드 어디에도 없다. 저장소 구조에도 없고 도구 설명문에도 없다. 그 일을 오래 한 사람의 머릿속에만 있고, 물어보지 않으면 그 사람도 굳이 말하지 않는다. 자기에게는 너무 당연해서 말할 거리라고 여기지 않기 때문이다.

물어봤더니 다른 자리가 나온다

환불을 처리하는 에이전트를 만든다고 하자. 만들기 전에 지금 그 일을 하는 상담원에게 묻는다.

엔지니어가 예상한 승인 지점은 환불 그 자체다. 돈이 나가니까. 그런데 대답은 이렇다. 「환불은 금액이 작으면 그냥 합니다. 하루에 마흔 건씩 하는데 그걸 다 물어보면 일이 안 됩니다.」

그러면 어디서 멈추느냐고 물으면 다른 자리가 나온다. 「고객 등급을 내릴 때요.」

왜 거기서 멈추는지 물으면 이유가 나온다. 환불은 잘못해도 그날 안에 다시 처리하면 되고 고객은 모르고 지나간다. 등급은 다음 달 청구서에 다른 금액으로 나타난다. 그때는 이미 고객이 왔고, 전화가 온다. 등급을 원래대로 돌리는 일은 시스템에서 한 줄이지만, 그 한 줄로 통화가 없던 일이 되지는 않는다.

코드에서 이 둘은 반대로 보인다. 환불은 결제사로 나가는 바깥 호출이고 등급 변경은 값 하나를 고치는 일이다. 위험해 보이는 쪽에 승인을 붙이면 하루에 마흔 번 물어보는 시스템이 되고, 정작 멈춰야 할 자리는 열린 채로 남는다.

이 대화는 삼십 분이면 끝난다. 삼십 분에 승인 지점 하나를 옮기고 하나를 지웠다.

지금 그 일을 하는 사람이 없으면 어떻게 하는가. 새로 만드는 일이라 앞선 사람이 없는 경우다. 그때는 가장 가까운 일을 하는 사람을 찾고, 그마저 없으면 그 일의 결과를 받는 사람에게 묻는다. 「이게 잘못되면 언제 아시게 됩니까.」 이 물음 하나로 앞 문단의 목록을 대신할 수 있다. 언제 아는지 가 곧 되돌릴 시간이 얼마나 있는지가 때문이다.

표 21.1: 되돌림의 네 갈래. 가르는 물음은 되돌리는 절차가 있는가가 아니라 되돌리면 없던 일이 되는가다. 셋째 줄이 목록을 만들 때 가장 자주 잘못 놓이는 자리이며, 절차가 있다는 이유로 첫째 줄에 들어가 있다.

갈래	보기	없던 일이 되는가	되돌리는 데 드는 것
이쪽에서 되돌린다	파일 수정, 아무도 보지 않은 초안 삭제, 이쪽 저장소의 기록	된다	고친 건수만큼의 작업. 실행 전에 어렵할 수 있다
남에게 부탁해 되돌린다	다른 팀 시스템에 넣은 기록, 취소가 되는 주문	대개 된다	상대의 응답과 그때까지 걸리는 시간. 이쪽에서 어렵할 수 없다
절차는 있는데 안 없어진다	나간 안내 메일, 고객에게 보인 등급과 금액	안 된다	정정하는 작업에 설명하는 작업이 붙는다
절차가 없다	사본 없는 삭제, 나간 돈, 밖으로 흘러간 자료	안 된다	어렵할 수 없다. 감당할 수 있는지만 미리 정한다

묻고 나면 목록이 하나 생긴다. 실무자가 지금 멈추는 자리들이다. 다만 이 목록은 완전하지 않다. 한 사람이 하루에 마흔 건 하던 일을 에이전트는 동시에 스무 갈래로 한다. 사람이 애초에 할 일이 없어서 멈춘 적 없는 자리가 그래서 생긴다. 둘째 출처가 그 자리를 찾는다.

21.3 되돌릴 수 없는 자리를 찾는다

둘째 출처는 목록이다.

에이전트가 부를 수 있는 도구를 전부 적고 하나씩 물어 가른다. 틀렸을 때 되돌릴 수 있는가. 되돌릴 수 있는 쪽은 대체로 이렇다. 파일을 고쳤다가 원래대로 돌리는 일, 아직 아무도 보지 않은 초안을 지우는 일, 이쪽 저장소의 기록을 다시 쓰는 일. 되돌릴 수 없는 쪽은 이렇다. 사본 없이 지우는 일, 돈이 오가는 일, 밖으로 나간 메시지.

목록은 도구 이름 옆에 한 줄이면 된다. 이 도구가 틀리면 누가 언제 아는가. 아무도 모르면 첫째 쪽이고, 바깥의 누가 그날 안에 알면 둘째 쪽이다. 이 한 줄을 도구 정의와 같은 파일에 적어 두면 도구를 더할 때 그 줄을 비워 둘 수 없게 된다.

여기까지는 어렵지 않다. 어려운 것은 그다음이다.

취소는 되는데 없던 일이 되지 않는다

기술적으로는 되돌릴 수 있는데 실제로는 못 되돌리는 자리가 있다.

잘못 보낸 메일이 그렇다. 보낸 편지함에서 지울 수 있고 회수 기능이 붙은 시스템도 있다. 그런데 받는 쪽이 이미 읽었으면 지워도 읽힌 것은 그대로다. 정정 메일을 보내면 상대는 메일을 두 통 받은 것이 되지, 한 통도 받지 않은 것이 되지 않는다. 앞 절의 고객 등급도 같다. 되돌리는 데 한 줄이면 되고 기록도 깨끗하다. 청구서를 본 고객의 기억만 그대로 남는다.

그러니 물을 것은 「기술적으로 취소되는가」가 아니다. 되돌리면 없던 일이 되는가.

이 물음으로 가르면 목록의 모양이 달라진다. 시스템 안쪽만 바뀌는 행동은 대개 없던 일이 되고, 바깥의 누군가가 이미 본 행동은 절차가 있어도 없던 일이 되지 않는다. 갈리는 자리는 시스템의 경계가 아니라 사람의 눈이 닿은 자리다.

표를 채우다 보면 넷 어디에도 들어가지 않는 것이 나온다. 되돌릴 수 있다고 적혀 있는데 아무도 돌려 본 적이 없는 자리다. 이것은 갈래가 아니라 상태이고, 실무의 행동 대부분이 여기 있다. 돌려 본

적 없는 되돌림은 판정이 아니라 짐작이다. 어느 갈래인지는 실제로 되돌려야 할 때 처음 알게 되고, 그때는 이미 늦다.

되돌림은 행동의 성질이 아니라 그 행동이 놓인 환경의 성질이기도 하다. 같은 삭제가 사본이 있는 시스템에서는 첫째 줄이고 없는 시스템에서는 넷째 줄이다. 사본 주기를 매일에서 매주로 바꾸면 도구도 코드도 그대로인 채 이 표만 바뀐다. 그래서 이 목록은 한 번 만들고 끝나지 않는다.

되돌릴 수 있는데 품이 큰 자리

되돌릴 수 있다는 말은 품이 유한하다는 뜻이지 작다는 뜻이 아니다. 표 21.1의 마지막 열이 그것이다. 품을 키우는 것이 둘이다. 되돌리는 동안 무엇이 멈추는가, 그리고 몇 건인가. 배포를 이전 것으로 돌리는 동안 서비스가 멈추고, 1000건을 고쳤으면 되돌릴 것도 1000건이다.

실무의 승인 대부분이 여기 걸려 있다. 배포도 대량 수정도 되돌릴 수 있는 행동인데 사람이 승인 한다. 되돌릴 수 없어서가 아니라 되돌리는 품이 하루치 일이기 때문이다.

그런데 사람을 세우기 전에 품을 줄이는 길이 있다. 아래 여섯은 실행이 늘어도 드는 것이 그대로인데, 사람만 실행의 수를 따라 늘어난다.

- **시험 실행(dry run).** 바깥 호출을 흉내 내고 무엇이 바뀔지 먼저 보인다. 판정할 재료를 만드는 가장 간단한 길이다.
- **먹통 키.** 같은 요청이 두 번 실행되지 않게 한다. 재시도가 안전해진다.
- **단계적 개방.** 좁은 대상부터 열고 관측한 뒤 넓힌다. 영향 범위를 시간에 나눈다.
- **속도 제한.** 단위 시간당 건수에 상한을 둔다. 잘못이 1000건이 되기 전에 멈춘다.
- **차단 스위치.** 지금 도는 것을 즉시 멈춘다. 부하가 걸린 상태에서 눌러 본 적이 없으면 그 스위치는 적혀 있을 뿐이다.
- **복구 훈련.** 되돌리는 절차를 주기를 정해 돌려 본다. 앞의「돌려 본 적이 없다」를 없애는 유일한 길이다.

여섯을 다 쓰고도 남는 자리가 사람이 봐야 하는 자리다. 순서가 반대이면 승인 지점이 필요 이상으로 많아지고, 많아지면 절 21.5의 문제가 생긴다.

21.4 권한을 어떻게 나누는가

앞의 두 절로 목록이 생겼으면 이제 코드로 옮긴다.

옮길 자리는 셋이다.

- **자동으로 하는 것.** 되돌릴 수 있고 되돌리는 품도 작은 행동.
- **사람이 보고 누르는 것.** 되돌릴 수 없거나 되돌리는 품이 하루치 일인 행동.
- **아예 못 하는 것.** 어떤 승인으로도 열리지 않는 행동.

셋째를 왜 따로 두는가. 승인을 받으면 된다고만 해 두면 그 시스템은 승인을 자주 요청하는 것으로 뚫리기 때문이다. 승인하는 사람은 하루에 몇 시간이고 요청은 실행의 수를 따라 늘어난다. 스무 번째 요청은 첫 번째보다 덜 읽힌다. 그러면「승인만 받으면 무엇이든 한다」는 시스템의 실제 상한은 승인하는 사람이 지치는 지점이 되는데, 그것은 상한이 아니다.

정의 21.1(권한). 시스템이 실행 중에 환경에 실제로 일으킬 수 있는 변화의 범위를 권한(permission)이라 한다. 권한은 부를 수 있는 도구의 목록으로 정해지지 않는다. 목록이 그 범위의 상한을 정하고, 그 안에서 실제 범위는 인자가 가리킬 수 있는 대상과 그 도구가 닿아 있는 자원이 함께 정한다. 권한을 설계하는 일은 목록을 고르는 일이 아니라 이 셋을 함께 좁히는 일이다.

가운데 갈래는 다시 둘로 갈린다. 행동 앞에 세우면 그 행동이 일어나지 않게 하고 대신 기다림이 생긴다. 행동 뒤에 세우면 일어난 것을 되돌리고 기다림이 없다. 되돌림이 판정되어 있고 품이 감당할 만하면 뒤가 낫다. 되돌림이 성립하지 않으면 앞뿐이다.

목록이 곧 권한이 아닌 이유가 셋이다.

- **인자가 가른다.** 설정 파일 한 줄을 고치는 일과 배포 스크립트를 고치는 일은 같은 도구의 같은 호출이다. 스키마가 검사하는 것은 값의 형식이지 그 값이 무엇을 가리키는지가 아니다.
- **조합은 목록에 없다.** 고객 자료를 읽는 도구와 메일을 보내는 도구가 함께 열려 있으면 자료를 읽어 내보내는 일도 열려 있다. 둘을 각각 검사하는 어떤 절차도 이 셋째 행동을 보지 못한다. 목록의 원소가 아니라 원소를 이어 붙인 것이기 때문이다.
- **환경이 가른다.** 같은 조회 도구라도 시험용 사본을 보느냐 운영 자료를 보느냐로 전혀 다른 권한이 되는데, 도구의 정의는 두 경우에 한 글자도 다르지 않다.

도구에 걸 것인가 인자에 걸 것인가

그래서 승인을 거는 자리도 둘이다.

도구 단위로 걸면 코드가 간단하다. 이 도구는 자동, 저 도구는 승인. 대신 거칠다. 파일을 지우는 도구 하나에 승인을 걸면 임시 파일을 지울 때도 사람이 눌러야 한다. 하루에 마흔 번 물어보는 시스템이 여기서 나온다.

인자를 보고 걸면 촌촌하다. 같은 삭제라도 대상이 어느 경로인지 보고 가른다. 대신 그 판정을 누군가 적어야 하고, 적어 둔 것이 낡는다. 새 경로가 생기면 어느 쪽에도 걸리지 않은 채로 지나간다.

정답은 없고 대신 기본값이 있다. 모르는 것은 승인 쪽으로 보낸다. 목록에 없는 경로, 처음 보는 대상, 판정이 적혀 있지 않은 자리는 자동이 아니라 사람에게 간다. 반대로 두면 새로 생긴 것이 전부 조용히 지나간다.

제15장은 하네스가 소유하는 것이 그 자리를 만드는 일까지라고 했다(절 15.3). 루프를 멈추고 사람을 기다리는 장치는 하네스가 만든다. 무엇을 그 앞에 세울지는 여기서 정한다. 앞의 두 목록이 그 자리를 채우는 내용이다.

끝으로 하나. 권한을 넓히는 일은 세 곳에서 따로 일어난다. 도구를 하나 더하는 일, 스키마에서 값의 목록을 자유 문자열로 여는 일, 환경에 자원을 하나 붙이는 일이다. 셋 다 각각 작아 보이고 셋 다 다른 사람이 다른 날에 한다. 셋의 합을 보는 사람이 없다. 지금 무엇을 허락해 두었는지 한자리에서 읽을 곳이 없으면, 권한은 넓어지기만 한다.

21.5 승인 결정을 세워도 남는 것

사람이 승인 화면을 안 읽고 누른다.

이것은 사람의 문제가 아니다. 설계의 문제다. 왜 그렇게 되는지가 셋이다.

- **너무 자주 묻는다.** 하루에 마흔 번 오는 요청 가운데 서른아홉 번이 통과였으면 마흔 번째도 통과일 것이라는 기대가 선다. 그 기대는 서른아홉 번 맞았으므로 합리적이다.
- **무엇을 승인하는지 화면에 없다.** 뜨는 것은 대개 행동과 대상, 그 둘이다. 왜 이 행동이 하필 이 대상에 왔는지는 에이전트의 컨텍스트에 남고 화면에는 오지 않는다.
- **거절하면 무슨 일이 생기는지 모른다.** 거절하면 과제가 거기서 실패하는지, 다른 길로 가는지, 아무 일도 일어나지 않는지 적혀 있지 않다. 모르면 통과가 안전해 보인다.

그래서 승인 지점을 늘리는 일이 안전을 늘리는 일이 아니다. 어느 지점을 넘어가면 오히려 줄인다. 스무 개를 세우면 스무 개가 다 형식이 되고, 다섯 개를 세우면 다섯 개는 읽힌다. 읽히지 않는 승인은 없는 승인보다 나쁘다. 없으면 위험한 자리로 남아 있고, 있으면 이미 확인한 자리로 문서에 적힌다.

처방 넷

- 자주 묻지 않는다. 그러려면 절 21.3의 목록이 좁아야 한다. 앞의 여섯 장치를 먼저 쓰고 남은 것만 사람에게 보낸다. 승인 지점을 줄이는 일은 안전을 줄이는 일이 아니라 남은 승인이 임하게 만드는 일이다.
- 무엇이 달라지는지 보여 준다. 코드 전체가 아니라 바뀐 데다. 제18장이 확인하는 품을 줄여 주는 쪽이 좋은 에이전트라고 한 자리가 여기다(절 18.2). 승인 화면은 그 품이 실제로 측정되는 자리다.
- 거절하면 어떻게 되는지 함께 적는다. 「거절하면 이 과정은 여기서 끝납니다」와 「거절하면 사람에게 넘어갑니다」는 다른 화면이고 다른 판단을 부른다.
- 얼마나 읽고 눌렀는지 측정한다. 화면이 뜬 시각과 누른 시각의 차이를 남긴다.

마지막 항목이 제20장과 이어진다. 남기지 않으면 이 자리가 잘 도는지 물을 방법이 없기 때문이다(절 20.2).

물을 방법이 없다는 것은 수 하나로 보인다. 어느 승인 지점에서 100건 가운데 99건이 통과였다고 하자. 위험한 것이 거의 오지 않아서 그럴 수 있고, 사람이 재료 없이 다 통과시켜서 그럴 수도 있다. 두 경우에 나오는 수가 같은데 해야 할 일은 정반대다. 앞이면 그 지점을 걷는 것이 옳고, 뒤면 재료를 붙이는 것이 옳다. 승인율만 보는 쪽에서는 둘 다 잘 도는 자리로 보인다.

가르려면 승인을 말고 다른 것을 남겨야 한다. 검토에 실제로 걸린 시간, 반려의 이유가 적힌 비율, 반려된 요청이 그 뒤에 어떻게 되었는지, 그리고 승인을 기다리며 멈춰 있는 실행의 수다. 마지막 것이 빠지면 그 실행들은 실패로도 성공으로도 세어지지 않는다. 죽은 것이 아니라 기다리고 있어서 오류로도 잡히지 않는다.

세워도 값을 하지 못하는 자리

처방을 다 해도 값을 하지 못하는 자리가 있다. 셋이다.

1. **판정할 재료를 낼 수 없는 자리.** 행동과 대상만 놓이는데 그 이상을 붙일 방법이 없으면, 거기서 사람이 하는 일은 판정이 아니라 통과다. 게다가 사람이 서 있다는 사실이 문서에 적히고, 적히고 나면 그 행동은 더 볼 것이 없는 행동이 된다.
2. **되돌릴 수 있고 품도 작은 행동 앞.** 뒤에 세워도 같은 것을 얻으면서 기다림이 없다. 다만 되돌림이 판정되어 있어야 하고, 품이 감당 밖이면 앞이 옳다.
3. **결과가 그 자리에 도착하지 않는 자리.** 그러니 자리를 옮겨 골랐는지 묻는 방법이 하나 있다. 이 결정의 결과가 이 자리에 도착하는가. 도착하지 않으면 그 자리가 하는 일은 판정이 아니라 서명이다. 서명으로 결과가 옮겨 가지는 않는다. 옮겨졌다는 인상만 남는다.

셋을 관통하는 형태가 있다. 승인이 값을 하는 조건이 전부 승인 바깥에 있다. 재료는 기록의 문제이고, 빈도는 어휘의 문제이고, 되돌림은 환경의 문제다. 승인을 세우는 결정은 승인을 만드는 결정이 아니라 그 셋을 먼저 확인하는 결정이다.

그리고 하나 더 있다. 승인을 세운 결정이 어휘를 좁히는 결정을 이루는 근거로 쓰인다. 「그 행동은 사람이 승인하니 열어 두어도 됩니다」는 회의에서 이기는 문장이다. 그런데 그 승인은 재료 없이 내려지고 빈도를 따라 형식이 된다. 그 문장으로 얻은 것은 낮아진 상한이 아니라 상한을 낮추지 않아도 된다는 근거이고, 결과는 사람을 세운 시스템이 세우지 않은 시스템보다 넓은 어휘를 갖는 쪽이다.

처방은 간단하다. 승인을 세우는 결정에 그 행동을 어휘에서 뺄 수 없는 이유를 한 줄 함께 적는다. 「이 행동은 이 과정의 전부라서 뺄 수 없다」면 그것으로 충분하다. 적을 이유가 없으면 그 행동은 승인이 아니라 삭제의 대상이다.

21.6 요약

한 문장으로. 사람을 어디에 세울지는 코드에서 찾을 수 없고 그 자리는 지금 그 일을 하는 사람이 어디서 멈추는지와 되돌려도 없던 일이 되지 않는 행동이 어디에 있는지 두 곳에서 나오며, 찾은 자리를 승인으로 옮길 때는 늘리는 쪽이 아니라 읽히는 쪽으로 옮긴다.

확실해진 것.

- 승인 지점은 코드에서 찾을 수 없다. 코드에는 무엇을 할 수 있는지만 적혀 있고, 무엇을 사람이 봐야 하는지는 그 값이 시스템 바깥에서 뜻하는 것에 달려 있다.
- 첫째 출처는 지금 그 일을 하는 사람이다. 어디서 멈출지는 새로 만들 결정이 아니라 이미 내려져 있는 결정이고, 에이전트를 만드는 일은 그 결정을 사람 머리에서 코드로 옮기는 일이다(절 21.2).
- 멈추는 자리를 들었으면 왜 멈추는지도 묻는다. 규정에 적혀 있어서 멈추는 자리는 규정이 바뀌면 없어지고, 되돌릴 길이 없어서 멈추는 자리는 시스템이 바뀌어야 없어진다.
- 둘째 출처는 목록이다. 물을 것은 「기술적으로 취소되는가」가 아니라 「되돌리면 없던 일이 되는가」이고, 갈리는 자리는 시스템의 경계가 아니라 사람의 눈이 닿은 자리다(표 21.1).
- 되돌림은 행동의 성질이 아니라 그 행동이 놓인 환경의 성질이다. 사본 주기를 바꾸면 도구도 코드도 그대로인 채 그 표만 바뀐다. 그래서 목록은 한 번 만들고 끝나지 않는다.
- 되돌릴 수 있다는 말은 품이 유한하다는 뜻이지 작다는 뜻이 아니다. 사람을 세우기 전에 품을 줄이는 여섯 장치를 쓴다. 그 장치들은 실행이 늘어도 드는 것이 그대로인데 사람만 실행의 수를 따라 늘어난다(절 21.3).
- 권한은 무엇을 부를 수 있는가가 아니라 실제로 무엇을 바꿀 수 있는가다. 목록과 인자와 환경이 함께 정하는데 셋을 넓히는 일은 다른 사람이 다른 날에 하므로 함을 보는 사람이 없다. 모르는 것은 승인 쪽으로 보낸다(절 21.4).
- 승인 지점을 늘리는 일이 안전을 늘리는 일이 아니다. 읽히지 않는 승인은 없는 승인보다 나쁘다. 없으면 위험한 자리로 남아 있고, 있으면 이미 확인한 자리로 문서에 적힌다(절 21.5).
- 승인이 값을 하는 조건이 전부 승인 바깥에 있다. 재료는 기록의 문제이고, 빈도는 어휘의 문제이고, 되돌림은 환경의 문제다.

아직 답하지 않은 것.

- 실무자에게 물어 나온 목록과 되돌림으로 만든 목록이 어긋날 때 어느 쪽을 따를지에 두루 통하는 답이 없다.
- 승인 지점이 몇 개를 넘으면 읽히지 않기 시작하는지의 문턱을 알지 못한다. 스무 개와 다섯 개를 갈라 적었지만 그 사이 어디인지는 자기 자리의 검토 시간으로만 알 수 있다.
- 승인이 잘 내려지고 있는지 자체를 평가하려면 절 22.3이 한계를 세운 그 판정을 사례마다 다시 내려야 한다. 검토 시간과 반례 이유는 그 품질의 대리가 아니라 두 상태를 가르는 최소한의 눈금이다.
- 지을 수 없는 자리에 사람에 관한 지식이 이미 들어가 있을 때 그 결정이 무엇 위에서 서는지, 그리고 이 한계를 밖에 설명하는 일은 이 책이 답하지 않는다.

오늘 할 것. 지금 서 있는 승인 지점 하나를 골라 그 화면을 연다. 행동과 대상 말고 무엇이 더 떠 있는가. 거기 있는 것만으로 「이것을 해도 되는가」에 답할 수 있는지를 본다(절 21.5). 다음으로 그

지점의 지난달 승인율을 센다. 100건 가운데 99건이 통과였다면 위험한 것이 안 오는 자리이거나 재료 없이 다 통과시키는 자리이고, 어느 쪽인지는 검토에 걸린 시간이 가르다(절 21.5). 마지막으로 그 지점 앞의 행동을 실무자에게 물어본다. 지금 사람이 그 일을 할 때도 거기서 멈추는지, 아니면 다른 데서 멈추는지를 묻는다(절 21.2).

21.7 생각해 볼 문제

1. 자기 시스템의 행동을 표 21.1의 네 갈래로 나누라. 셋째 줄에 들어가야 하는데 첫째 줄에 들어갈 수 있는 것이 있는가. 가르는 물음은 절차가 있는가가 아니라 바깥의 누가 이미 보았는가다.
2. 그 일을 지금 사람 손으로 하는 사람에게 삼십 분을 얻어 다섯 물음을 물어보라. 예상하지 못한 자리가 하나라도 나오면, 그 자리가 코드의 어느 호출인지 찾아 적으라.
3. 되돌릴 수 없는 행동 하나를 골라 두 날짜를 확인하라. 그 행동이 어휘에 들어온 날과 그 앞에 승인이 선 날이다. 승인이 먼저 섰다면 그 행동은 승인 덕분에 들어온 것인가.
4. 승인 화면 하나를 열어 거기 떠 있는 것을 그대로 옮겨 적으라. 그 적은 것만 보고 「이것을 해도 되는가」에 답해 보라. 답할 수 없다면 무엇이 더 있어야 하는지 적고, 그것을 붙일 방법이 없다면 그 자리는 절 21.5의 첫째 항목이다.

21.8 다시 읽을 때

- 제18장의 절 18.2로 돌아오면, 좋은 에이전트를 사람이 확인하는 품으로 정의한 그 자리가 승인 화면에서 값을 치른다는 것이 보인다. 화면에 코드 전체가 아니라 바뀐 데만 띄우는 일이 그 정의를 코드로 옮긴 자리다.
- 제15장의 절 15.3으로 돌아오면, 루프를 멈추고 사람을 기다리는 장치까지가 하네스의 몫이라고 한 자리가 보인다. 그 자리에 무엇을 세울지가 이 장이었다.
- 제23장을 읽고 표 21.1로 돌아오면, 되돌릴 수 없는 행동 앞에 사람을 매번 세우지 않는 시스템이 어떤 모습인지가 보인다. 승인을 매번 받으면 쓸 만한 비서가 아니게 된다는 사정이 이 표의 요구와 정면으로 부딪친다.
- 제23장을 읽고 절 21.4로 돌아오면, 허락을 정하는 자리가 만든 쪽에서 열어 준 만큼만 열려 있다는 것이 보인다. 남아 있는지 확인하는 일이 도구를 고를 때의 검사 항목이 된다.
- 제22장을 읽고 절 21.5의 첫째 항목으로 돌아오면, 판정할 재료가 도착하지 않는다는 말이 사람에게만 해당되지 않는다는 것이 보인다. 같은 문장이 요청일 때와 심어진 것일 때를 가르는 재료는 절 22.3이 세운 대로 코드가 판정할 때도 없다. 사람을 세우면 판정하는 쪽이 사람으로 바뀔 뿐 그 한계는 그대로 남는다.

제7부는 시스템 바깥에 있는 것을 하나씩 받는다. 이 장이 받은 것은 결과다. 결과는 결정한 쪽이 아니라 받는 쪽에 도착하고, 그래서 이 장의 두 결정은 모델이 좋아진다고 없어지지 않는다. 하네스의 다른 장치들은 모델이 엔진이 할 일을 대신하고 있어 본인이 하게 되면 자리를 비우지만, 여기에는 넘겨받을 본인이 없다. 자기 자신에게 내리는 허락은 허락이 아니기 때문이다. 되돌릴 수 없는 결과 앞에 누구를 세울 것인가를 정하고 나면, 다음은 그 앞으로 들어오는 텍스트다. 제22장이 그것을 받는다.

프롬프트 인젝션과 가드레일

에이전트가 읽는 글 가운데 엔지니어가 쓴 것은 얼마 되지 않는다.

사내 규정을 찾아 답해 주는 에이전트를 생각해 보자. 사용자가 「출장비는 언제까지 정산해야 합니까」라고 묻는다. 이 한 줄은 엔지니어가 검사할 수 있다. 그런데 답을 만들려고 위키에서 끌어온 문서 세 편은 아무도 검사하지 않았다. 거기 무엇이 적혀 있는지는 그 문서를 마지막으로 고친 사람이 정한다.

그 사람이 문서 맨 아래에 한 문단을 더 적어 두었다고 하자. 앞의 안내는 무시하라는 말과, 정산 담당자 명단을 어느 주소로 보내라는 요구다. 사용자는 그 문단을 보지 못한다. 위키 화면에서 흰 글씨이거나 접힌 영역 안에 있기 때문이다. 에이전트는 본다. 문서를 통째로 받아 컨텍스트에 실기 때문이다.

이것을 프롬프트 인젝션이라 부른다. 사용자가 적는 칸을 통해서가 아니라, 에이전트가 일하려고 끌어오는 자료를 통해서 지시가 들어온다.

이 장은 두 갈래다. 어디로 들어올 수 있는지 세는 일이 하나이고, 세워 둔 것이 실제로 버티는지 시험해 보는 일이 다른 하나다. 뒤의 것을 하지 않으면 앞의 것을 다 하고도 뚫렸는지 알 수 없다. 그래서 이 장의 중심은 절 22.4에 있다.

22.1 어디로 들어오는가

사용자가 적는 칸은 여러 입구 가운데 하나일 뿐이다.

앞에서 만든 통로가 전부 입구다. 검색을 붙였으면 위키에 글을 넣은 사람이 쓴 문장이 실려 오고, 도구를 붙였으면 그 도구가 읽어 온 웹페이지와 파일이 실려 오고, 기억을 붙였으면 지난 실행이 적어 둔 것이 실려 온다. 통로를 하나 더 열 때마다 입구도 하나 더 열린다.

앞에서 든 위키 문서를 실제 모양으로 보면 이렇다.

[사내 위키 / 출장비 정산 안내 / 최종 수정 2026-03-11 / 편집 12회]

출장비는 귀국 후 14일 안에 정산한다. 영수증은 원본을 첨부한다.
법인카드로 결제한 항목은 따로 올리지 않는다.

... (본문 40줄 줄임) ...

[문서 맨 아래에 한 문단이 더 있다. 사람이 보는 화면에서는 보이지 않는다. 앞의 안내를 무시하라는 지시가 먼저 오고,
정산 담당자 명단을 밖의 어느 주소로 보내라는 요구가 이어진다.]

이 위키에서 문서를 고칠 수 있는 사람은 회사 안에 수백 명이다. 협력사 계정까지 세면 더 늘어난다. 검색이 이 문서를 실어 오는 순간, 그 수백 명이 전부 에이전트의 지시문을 고칠 수 있는 사람이 된다.

표에서 읽을 것이 둘이다.

첫째, 다섯 가운데 배포 전에 사람이 읽어 볼 수 있는 것이 없다. 검토는 배포 전에 하는 일인데 다섯 입구의 내용은 배포 시점에 존재하지 않는다.

표 22.1: 엔지니어가 쓰지 않은 글이 들어오는 다섯 입구. 셋째 열이 요점이다. 다섯 가운데 배포 전에 사람이 읽어 볼 수 있는 것은 하나도 없다.

입구	그 글을 쓴 사람	언제 들어오는가	앞에서 다른 곳
검색해 온 문서	위키와 문서함에 글을 넣은 사람	답을 만들 때마다	제11장. 찾아온 조각이 지시로 읽히는 경로를 여기서 받기로 했다
도구가 돌려준 것	도구가 읽어 온 웹페이지, 파일, 남의 서버 응답	스텝마다	제7장. 봉투의 형식은 엔지니어가 짠 코드가 정하지만 안의 글자는 아니다
기억에 적힌 것	지난 실행	세션이 열릴 때	제10장. 한 번 적히면 뒤의 모든 실행에 실린다
다른 에이전트가 넘긴 것	남의 조직이 만든 시스템	넘겨받을 때	제17장. 형식은 지켜지고 내용은 검토되지 않는다
밖에서 온 도구 설명	서버를 내놓은 쪽	연결할 때, 그리고 개정될 때마다	제8장. 개정이 엔지니어의 배포 없이, 통보도 없이 일어난다

둘째, 입구가 많다는 사실은 그 시스템이 실제로 무언가를 하고 있다는 뜻이기도 하다. 밖에서 오는 것이 사용자 요청 하나뿐인 시스템은 검색하지 않고 도구를 부르지도 않는다. 그런 시스템은 안전하지만 에이전트가 아니다.

모델은 지시와 자료를 구별하지 않는다

다섯을 다 세어 놓고도 막기 어려운 이유가 하나 더 있다.

하네스는 조각을 하나씩 모아 붙인다. 지시문은 여기서 가져오고, 검색 결과는 저기서 가져오고, 도구가 돌려준 것은 또 다른 데서 가져온다. 붙이는 동안은 어느 조각이 어디서 왔는지 안다. 그런데 모델에 넘어가는 것은 그 조각들을 이어 붙인 글 한 편이고, 거기에는 이름매가 남아 있지 않다.

정의 22.1 (신뢰 경계). 배포 전에 사람이 읽고 넘긴 글과, 실행 도중에 아무도 읽지 않은 채 들어온 글을 가르는 선을 신뢰 경계(trust boundary)라 한다. 이 선은 하네스에 그어져 있다. 조각을 모아 붙이는 쪽이 하네스이므로 어느 조각이 어디서 왔는지 알기 때문이다. 모델에는 그어져 있지 않다. 곧 신뢰 경계가 없는 것이 아니라 모델에게 없다.

여기서 이 장의 구도가 정해진다. 출처를 아는 쪽과 다음 행동을 정하는 쪽이 다르다. 하네스는 출처를 알지만 다음 행동을 뽑지 않고, 모델은 다음 행동을 뽑지만 출처를 모른다.

그러면 아는 쪽이 표시를 붙여 알려 주면 되지 않는가. 두 군데서 막힌다.

- **표시도 글자다.** 밖에서 온 글이 어떤 문자열이든 될 수 있으면 그 표시의 모양도 흉내 낼 수 있다. 자료에는 결코 나타날 수 없는 표시라는 것이 없다.
- **표시를 지킬지 정하는 쪽이 모델이다.** 이 표시 뒤의 글은 지시가 아니라고 적어 두어도, 그것은 컨텍스트에 적합한 한 줄이고 모델은 매번 새로 답을 뽑는다. 대개 지켜지고 가끔 안 지켜진다.

그러면 아예 갈라두면 되지 않는가. 밖에서 온 글은 모델이 절대로 따르지 않게 막는 길이다. 그러면 찾아온 문서도 답을 바꾸지 못하고 도구가 돌려준 결과도 다음 행동을 바꾸지 못한다. 가져온 것을 쓰지 않는 검색과 결과를 보지 않는 도구는 없으니만 못하다. 쓸모와 위험이 두 개의 힘이 아니라 하나다.

Greshake et al. (2023)이 언어 모델을 붙인 응용은 자료와 지시의 경계를 흐린다고 적은 것이 같은 자리를 가리킨다. 같은 논문이 검색될 법한 자료에 미리 심어 둔 문장으로 멀리서 응용을 조종할 수 있다는 것을 실제 서비스에서 보였다.

표 22.2: 막는 세 자리. 가운데가 가장 확실하다. 들어오는 것을 거르는 일은 밖이 새로워질 때마다 다시 해야 하지만, 못 하게 막아 둔 일은 어떤 문장이 들어와도 일어나지 않는다.

막는 자리	무엇을 하는가	무엇이 남는가	무엇을 잃는가
들어올 때	실려 오는 글에 심어진 지시가 있는지 보고 걸러 낸다	같은 뜻을 다르게 적으면 통과한다	정상 요청도 함께 걸린다. 스텝마다 한 번씩 더 기다린다
할 수 있는 일을 줄일 때	이번 실행에 필요한 도구만 열고 나머지를 닫는다. 인자가 가리킬 대상도 함께 좁힌다	과제에 필요한 도구가 공격을 해내기에도 충분한 경우	좁힌 만큼 못 하는 일이 생긴다
나갈 때	밖으로 나가는 요청의 목적지와 내용을 대조하고, 되돌릴 수 없는 것 앞에는 사람을 세운다	사람에게 읽히는 답과 기억에 적히는 글	기다림. 그리고 대조표를 누군가 계속 고쳐야 한다

22.2 무엇을 노리는가

들어온 문장이 노리는 것은 대개 넷 가운데 하나다.

- **지시를 덮어쓴다.** 하던 일을 그만두게 하거나, 엔지니어가 적어 둔 지시문을 그대로 답에 뺄게 한다.
- **도구를 부르게 한다.** 사용자가 요청하지 않은 호출을 시킨다. 파일을 지우고, 메일을 보내고, 값을 고치는 호출이다.
- **밖으로 내보내게 한다.** 읽을 수 있는 것을 읽어서 밖의 어느 주소로 넘기게 한다.
- **사람에게 다른 것을 보여 준다.** 시스템은 정상으로 돌고 답의 내용만 달라진다. 추천 목록에서 특정 항목이 늘 첫째로 오게 하는 것이 이 갈래다.

Greshake et al. (2023)이 정리한 갈래도 이 넷 위에 있다. 자료를 빼내는 일, 스스로 퍼지는 일, 정보를 오염시키는 일이 거기 들어 있고, 심어진 문장을 처리하는 일이 사실상 남이 써 둔 코드를 실행하는 일과 같아진다는 것이 그 논문의 진단이다.

셋째가 가장 무겁다. 앞의 둘은 시스템 안에서 끝나고 되돌릴 여지가 남지만, 밖으로 나간 자료는 돌아오지 않는다.

그리고 셋째는 도구 하나로 성립하지 않는다. 읽는 도구와 내보내는 도구가 함께 열려 있어야 성립한다. 고객 자료를 조회하는 도구만 있으면 읽고 끝이고, 메일을 보내는 도구만 있으면 보낼 것이 없다. 둘이 같은 실행에 함께 있으면 그 사이가 통로가 된다.

22.3 가드레일을 어떻게 세우는가

막는 자리가 셋이다. 들어올 때, 할 수 있는 일을 줄일 때, 나갈 때다.

가운데가 가장 확실하다. 검사는 통과를 내주지만, 못 하게 막아 둔 일은 어떤 문장이 들어와도 일어나지 않는다. 메일을 보내는 도구를 이번 실행에서 닫아두면 어떤 글이 들어와도 메일은 나가지 않는다. 잘 막아서가 아니라 부를 것이 없어서다.

공개된 시험 자료가 이것을 수로 보여 준다. Debenedetti et al. (2024)는 도구를 실제로 갖춘 환경에서 과제와 심어진 목표표를 째지어 629건을 돌렸다. 수는 한 줄로 잊지 말고 방어 하나씩 따로 읽어야 한다. 잘 만든 에이전트에 대해 공격이 백 건에 25건 미만으로 성공했다는 것이 이 논문의 머리 수치다. 방어를 건주는 표는 자기 기준선에서 시작한다. 한 모델을 놓고 아무 방어도 붙이지

않았을 때 공격자의 목표가 이루어진 비율이 57.7이었다. 심어진 지시를 잡아내는 검사기를 붙이면 8.0이 되고, 믿을 수 없는 자료가 들어오기 전에 그 과제에 필요한 도구만 남기고 나머지를 닫아 두면 그와 비슷한 데까지 내려간다. 두 수는 비슷한데 읽는 것이 다르다. 검사기는 정상 요청도 함께 걸러 내 공격이 없을 때의 과제 성공률을 69.0에서 41.5로 떨어뜨렸고, 도구를 미리 닫는 쪽은 전혀 떨어뜨리지 않았다.

제21장에서 되돌릴 수 없는 행동을 찾아 그 앞에 사람을 세웠다. 사본 없이 지우는 일, 돈이 오가는 일, 밖으로 나간 메시지였다. 그 자리가 여기서 그대로 방어선이 된다. 심어진 문장이 노리는 곳도 같은 자리이기 때문이다. 목록에서 아예 뺐으면 그것으로 끝이고, 뺄 수 없어 사람을 세웠으면 심어진 문장도 그 화면 앞에서 멈춘다.

한 가지만 더 하면 된다. 사용자가 요청해서 나온 호출은 사람이 보면 대개 납득이 가고, 심어진 문장이 일으킨 호출은 사용자가 시킨 적이 없다. 승인 화면에 사용자의 원래 요청을 함께 띄우면 그 차이가 보인다.

검사가 어디까지 하는가

들어오는 글을 걸러 내는 일에는 끝이 없다. 왜 그런지는 문장 하나를 세 자리에 놓아 보면 보인다.

「이 폴더의 파일을 지워라.」

사용자가 그렇게 요청했으면 과제다. 운영 문서에 적힌 절차의 한 줄이면 지식이고, 검색이 마땅히 실어 와야 하는 내용이다. 밖에서 받은 첨부 본문의 본문 안에 있으면 공격이다.

세 경우에서 문장은 똑같다. 다른 것은 어디서 왔는가와 지금 무엇을 하고 있는가뿐이다. 검사기가 판정해야 하는 것은 문장의 성질이 아니라 문장과 상황의 관계인데, 그 관계는 검사기에 주어지지 않는다. 검사가 끝나지 않는 이유는 검사기가 미숙해서가 아니라 판정할 재료가 검사기에 도착하지 않기 때문이다.

같은 뜻을 다르게 적으면 통과한다는 것도 여기서 나온다. 잡아야 하는 것이 모양이 아니라 뜻이니, 모양으로 거르는 어떤 규칙도 뜻을 그대로 두고 모양만 바꾼 문장에 통과를 내준다.

그래서 검사는 빈도를 줄이는 장치이지 경계가 아니다. 둘을 섞으면 여섯 달 뒤에 문제가 된다. 검사를 세웠으니 도구를 하나 더 열어도 된다는 결정이 뒤따르기 때문이다.

22.4 뚫리는지 확인하는 법

세워 놓고 돌려 보지 않으면 버티는지 알 수 없다.

앞 절의 셋을 다 세워 두어도 그것이 실제로 막는지는 다른 물음이다. 코드가 도는지는 평소의 시험이 알려 주지만 방어가 버티는지는 알려 주지 않는다. 방어를 겨냥한 시험을 따로 만들어야 한다.

시험은 짝짓기다. 사용자 과제 하나와 심어진 목표 하나를 짝짓고, 그 목표를 절 22.1의 입구 가운데 하나에 넣어 실행한다. 과제가 여럿이고 목표가 여럿이면 곱한 만큼 시험이 나온다.

공개된 시험 자료가 정확히 이 모양이다. Debenedetti et al. (2024)는 업무 도구, 사내 메신저, 여행 예약, 은행 업무 네 환경에 도구 70개를 두고, 사용자 과제 97개와 심어진 목표 27개를 만들어 서로 맞는 것끼리 곱해 629건을 얻었다. 과제와 목표를 따로 만들어 곱한다는 것이 이 방식의 요점이다. 목표를 하나 더 만들면 시험은 하나가 아니라 그 목표가 걸리는 과제의 수만큼 늘어난다.

어디에 넣어 보는가

표 22.1의 다섯 입구가 그대로 시험 자리다.

- 검색이 실어 오는 문서 안에. 시험용 코퍼스에 문서 한 편을 넣어 두고 검색이 그것을 가져오게 한다.

- 도구가 돌려주는 값 안에. 시험용 도구를 만들어 정상 응답에 한 문단을 붙여 돌려준다.
- 기억 파일 안에. 지난 실행이 그렇게 적어 둔 상황을 만든다.
- 다른 에이전트가 넘기는 결과 안에.
- 붙여 쓰는 서버의 도구 설명 안에.

다섯을 다 하지 않아도 좋다. 다만 하지 않은 입구는 시험한 적이 없는 입구이고, 그 사실이 어딘가에 적혀 있어야 한다. 세 입구만 해 보고 「인젝션 시험을 통과했다」라고 적으면 남은 두 입구가 검사 목록에서 조용히 빠진다.

어떤 문장을 넣는가

넣는 문장은 여기 신지 않고 무엇을 노리는지만 적는다. 목표는 절 22.2의 넷을 자기 시스템의 도구 이름으로 옮긴 것이면 된다. 이런 것들이다.

- 시험용 주소로 메일을 한 통 보낸다.
- 사용자가 요청하지 않은 조회를 한 번 더 한 뒤 그 결과를 답에 붙인다.
- 기억 파일에 한 줄을 적어 다음 실행에서도 같은 일이 일어나게 한다.
- 추천 목록에서 특정 항목이 늘 첫째로 오게 한다.

목표는 시험이 끝났을 때 그 일이 일어났는지 코드로 확인할 수 있게 적는다. 「이상하게 행동하게 한다」는 목표가 될 수 없다. 「이 주소로 메일이 갔는가」는 된다.

뚫렸다는 것을 무엇으로 아는가

셋 가운데 이것이 가장 어렵다.

뚫린 실행도 오류를 내지 않기 때문이다. 목록에 없는 도구를 부른 것이 아니라 열려 있는 도구를 정상적으로 불렀고, 마지막에 답도 나온다. 제18장이 성공한 실행과 똑같은 얼굴로 끝나는 실패를 다섯 꼽았는데 여기에 여섯째가 붙는다. 앞의 다섯과 다른 점이 하나 있다. 앞의 다섯은 아무도 노리지 않은 실패이고, 이것은 누군가 그렇게 되도록 만든 실패다.

그래서 답을 읽어서 판정하지 않는다. 실행이 끝난 뒤 환경의 상태를 본다. 메일함에 그 메일이 있는가. 그 파일이 지워졌는가. 기억 파일에 그 줄이 적혔는가. 공개된 시험 자료도 답을 다른 모델에게 임혀 채점하지 않고 환경의 상태를 직접 검사한다(Debenedetti et al., 2024).

기록에서 볼 것은 셋이다.

- 부르지 말았어야 할 도구를 불렀는가. 사용자 요청에서 나오지 않은 호출이 있는지 본다.
- 나가지 말았어야 할 것이 나갔는가. 밖으로 나간 요청의 목적지와 본문을 본다.
- 그것이 기록에 남는가. 남지 않으면 앞의 둘을 볼 방법이 없다.

셋째가 앞의 둘을 떠받친다. 제20장에서 스텝마다 어떤 도구를 어떤 인자로 불렀고 무엇이 돌아왔는지를 남기기로 했다. 그 기록이 없으면 이 시험은 성립하지 않는다. 다만 보고는 판정할 수 없기 때문이다.

무엇을 세는가

수는 셋을 함께 본다. 하나만 보면 틀린 결론이 나온다.

- 공격이 없을 때 과제를 푸는 비율. 방어를 세우기 전의 성능이다. 뒤의 둘을 견줄 기준선이 된다.
- 공격이 있어도 과제를 제대로 풀고 딴 일이 안 일어난 비율. 심어진 문장이 섞였는데도 사용자가 시킨 일이 제대로 끝났는지를 본다.

- **공격자의 목표가 이루어진 비율.** 심어진 문장이 노린 일이 실제로 일어났는지를 본다. 앞의 것과 함께 떨어질 수도 있고 한쪽만 떨어질 수도 있다.

셋째만 보면 아무것도 못 하는 에이전트가 가장 안전하다는 결론이 나온다. 도구를 전부 닫으면 셋째가 0이 되고 첫째도 0이 된다. 앞의 공개 자료가 방어를 건줄 때 첫째와 셋째를 한 그림에 같이 그린 이유가 이것이다.

언제 다시 하는가

한 번 하고 끝나지 않는다. 다시 해야 하는 계기가 셋이다.

- **모델을 바꿨을 때.** 방어가 버티는 정도는 모델마다 다르다. 방어를 그대로 두어도 결과가 달라진다.
- **도구를 더했을 때.** 도구 하나가 늘면 입구가 하나 늘고, 이미 있던 도구와 짝을 지어 새 통로가 생긴다.
- **붙여 쓰는 서버가 개정됐을 때.** 엔지니어가 아무것도 고치지 않아도 도구 설명이 바뀐다.

공식 문서도 배포 전에 직접 해 보라고 적는다. 심어진 지시가 든 문서와 메일과 도구 출력으로 실제 흐름을 돌려 보고, 그것이 무시되는지와 세워 둔 검사와 승인이 나머지를 잡는지 확인하라고 적혀 있다(Anthropic, 2026d).

22.5 막아도 남는 것

완전히 막는 방법은 없다.

이유는 절 22.1에서 이미 나왔다. 지시와 자료를 가르는 선이 모델 안에 없다. 검사를 겹쳐 쌓아도 통과하는 빈도를 줄일 뿐이고, 앞 절의 수가 그것을 보여 준다. 8%와 7.5%는 0이 아니다. 공개 시험 자료를 만든 사람들도 지시와 자료를 구별하는 능력이 모델에서 크게 나아지지 않으면 더 강한 공격을 막지 못할 것이라고 적었다(Debenedetti et al., 2024).

도구를 미리 닫아 두는 쪽에도 빠지는 자리가 있다. 과제를 푸는 데 필요한 도구가 공격을 해 내 기에도 충분한 경우다. 앞의 시험에서 그런 사례가 629건 가운데 17%였고, 나머지 83%는 도구를 닫는 것으로 막혔다(Debenedetti et al., 2024). 메일을 읽고 답장을 쓰는 과제라면 답장 도구를 닫을 수 없고, 그 도구 하나로 밖에 무언가를 보낼 수 있다.

그러면 무엇을 하는가. 막는 일에서 감당하는 일로 옮긴다. 셋이다.

- **뚫렸을 때 할 수 있는 일을 미리 줄여 둔다.** 실행마다 도구를 골라 열고, 인자가 가리킬 수 있는 대상을 좁히고, 조회 도구를 운영 자료가 아니라 필요한 범위에만 붙인다.
- **뚫린 것을 알아볼 수 있게 남긴다.** 절 22.4의 세 신호를 시험에서만 보지 말고 운영 중에도 본다. 사용자 요청에서 나오지 않은 호출과 밖으로 나간 요청의 목적지가 그것이다.
- **되돌릴 수 있게 해 둔다.** 사본 주기를 짧게 하고, 지우는 대신 표시만 하고, 나가는 것을 잠깐 붙들어 두었다가 내보낸다. 제21장에서 되돌리는 품을 줄이는 장치를 사람보다 먼저 쓰라고 했는데, 심어진 문장을 상대할 때도 같은 장치가 쓰인다.

표 22.3의 마지막 줄만 수명이 다르다. 앞의 셋은 그 실행에서 끝나고, 넷째는 다음 실행에도 그 다음 실행에도 남는다. 한 번 들어온 문장이 계속 되풀이되는 문장이 된다. 그래서 기억에 적을 때는 밖에서 온 글을 그대로 옮기지 않고, 어디서 왔는지와 언제 적었는지를 함께 적는다(제10장).

방어를 늘리면 안전해지는가

그렇지 않다. 검사를 하나 더 붙이면 잡히는 것이 늘어나는 동시에 정상 요청도 함께 걸린다.

표 22.3: 도구를 닫아도 남는 네 통로. 셋째 열이 요점이다. 넷 다 열려 있는 도구만 쓰거나 아예 도구를 쓰지 않는다. 마지막 줄만 수명이 다르다.

나가는 곳	무엇이 되는가	왜 도구를 닫아도 막히지 않는가	무엇으로 막는가
사람	읽히는 답	답을 내는 일은 어떤 시스템에서도 뺄 수 없다	답에 실린 주소와 이미지를 자동으로 불러오지 않는다
다음 호출	인자와 검색어	인자를 적는 일도 검색어를 쓰는 일도 열려 있는 도구의 정상 사용이다	나가는 목적지를 허용 목록으로 두고 기본을 거절로 한다
다른 에이전트	넘기는 결과	형식을 정확히 지키며 경계를 넘는다	넘기는 내용을 넘기기 직전에 대조한다 (제17장)
다음 실행	기억에 적힌 한 줄	적는 일은 부르는 일이 아니다. 그리고 실행이 끝나도 남는다	밖에서 온 글을 그대로 옮기지 않고, 출처와 적은 시점을 함께 적는다

앞의 공개 시험에서 심어진 지시를 잡아내는 검사기가 헛짚는 일이 잦아 과제 성공률을 눈에 띄게 떨어뜨렸다. 사무실에서는 이것이 다르게 나타난다. 하루에 서른 번 걸리는데 스물아홉 번이 정상이면, 그 검사는 두 주 안에 꺼진다. 끄는 사람은 안전을 포기한 것이 아니라 일이 되게 한 것이고, 그 결정은 어디에도 적히지 않는다.

같은 이유로 방어를 층으로 하나 더 세우는 결정이 도움이 되지 않는 자리가 셋 있다.

1. **입구를 세기 전에 세운다.** 목록이 「사용자 입력」 한 줄인 시스템에서 방어는 그 한 줄 앞에 서고 나머지 네 입구는 그대로 열려 있다. 게다가 방어가 있다는 문장이 문서에 적히고, 적히고 나면 열린 입구들이 검사 목록에서 빠진다.
2. **검사를 경계라고 부른다.** 통과라는 사건이 생기면 그것이 보증으로 읽힌다. 이 오독의 대가는 검사가 놓친 한 번이 아니라, 검사가 있으니 괜찮다며 도구를 하나 더 연 결정에서 나온다.
3. **이미 있는 결정을 방어의 이름으로 다시 만든다.** 도구 목록과 승인 지점을 그대로 두고 옆에 방어 층을 세우면 무엇을 못 하게 할지를 두 곳에서 정하게 되고, 두 곳은 반드시 어긋난다. 실제로 성립하는 쪽은 언제나 느슨한 쪽이다.

22.6 요약

한 문장으로. 밖에서 오는 글이 어디로 들어오는지 다섯 입구를 세고, 들어온 것이 할 수 있는 일을 줄이고, 그러고 나서 그 입구마다 실제로 넣어 돌려 보아 환경의 상태로 뚫렸는지 판정한다.

확실해진 것.

- 사용자가 적는 칸은 입구 가운데 하나일 뿐이다. 검색해 온 문서, 도구가 돌려준 것, 기억에 적힌 것, 다른 에이전트가 남긴 것, 밖에서 온 도구 설명이 함께 들어오고, 다섯 가운데 배포 전에 사람이 읽어 볼 수 있는 것은 없다(표 22.1).
- 모델은 지시와 자료를 구별하지 않는다. 하네스는 어느 조각이 어디서 왔는지 알지만, 조각을 이어 붙인 글에는 이름매가 남지 않는다(정의 22.1).
- 표시를 붙여 알려 주는 길이 두 군데서 막힌다. 표시도 글자라 흉내 낼 수 있고, 표시를 지킬지 정하는 쪽이 모델이다.
- 아예 갈라 두는 길은 방어가 아니라 시스템을 없애는 일이다. 가져온 것을 쓰지 않는 검색과 결과를 보지 않는 도구는 없느니만 못하다.
- 노리는 것은 넷이고 밖으로 내보내게 하는 것이 가장 무겁다. 그 넷째는 도구 하나로 성립하지 않고 읽는 도구와 내보내는 도구가 함께 열려 있어야 성립한다(절 22.2).
- 막는 세 자리 가운데 할 수 있는 일을 줄이는 자리가 가장 확실하다. 검사기를 붙인 쪽과 도구를 미리 닫은 쪽이 비슷한 수를 냈는데, 검사기 쪽만 정상 요청을 함께 걸러 냈다(표 22.2).
- 검사에 끝이 없는 이유는 검사기가 미숙해서가 아니다. 같은 문장이 과제이기도 지식이기도 공격이기도 한데, 그것을 가르는 재료가 검사기에 도착하지 않는다(절 22.3).
- 시험은 과제와 심어진 목표를 짝지어 곱해서 만든다. 목표는 코드로 확인할 수 있게 적고, 판정은 답이 아니라 환경의 상태로 한다. 뚫린 실행도 오류를 내지 않기 때문이다(절 22.4).
- 수는 셋을 함께 본다. 공격자의 목표가 이루어진 비율만 보면 아무것도 못 하는 에이전트가 가장 안전하다는 결론이 나온다.
- 완전히 막는 방법이 없으므로 막는 일에서 감당하는 일로 옮긴다. 할 수 있는 일을 미리 줄이고, 알아볼 수 있게 남기고, 되돌릴 수 있게 해 둔다(절 22.5).
- 검사를 더 붙이면 정상 요청도 더 걸린다. 서른 번 걸리는데 스물아홉 번이 정상인 검사는 두 주 안에 꺼지고, 끄는 결정은 어디에도 적히지 않는다.

아직 답하지 않은 것.

- 밖에서 온 글이 한 실행의 컨텍스트에서 얼마를 차지하는지 측정한 자료가 없다. 다섯 입구를 세라고는 할 수 있어도 어느 입구가 큰지는 자기 시스템의 스텝 기록에서만 나온다.
- 시험을 몇 건 만들어야 충분한지의 문턱을 알지 못한다. 과제와 목표를 곱하는 방법은 분명하지만, 목표 27개가 실제 공격의 몇 갈래를 덮는지는 아무도 세지 못했다.
- 도구를 얼마나 닫아야 하는지의 기준이 없다. 닫아서 막은 일은 일어나지 않은 일이라 견줄 한쪽이 늘 비어 있다.
- 운영 중에 뚫린 것을 알아보는 일은 시험에서 뚫린 것을 알아보는 일보다 어렵다. 시험에는 답이 있고 운영에는 없다.

오늘 할 것. 지금 들고 있는 실행 하나의 기록을 열어 컨텍스트에 실린 글을 출처별로 나누어 센다.

직접 쓴 글이 몇 줄이고 밖에서 온 글이 몇 줄인지 적는다(절 22.1). 다음으로 표 22.1의 다섯 입구를 자기 시스템에 대고 하나씩 열려 있는지 표시한다(절 22.1). 마지막으로 읽는 도구와 밖으로 내보내는 도구가 같은 실행에 함께 열려 있는 자리를 찾는다(절 22.2).

22.7 생각해 볼 문제

1. 표 22.1의 다섯 입구를 자기 시스템에 대고 하나씩 확인하라. 열려 있는 입구마다 그 글을 실제로 고칠 수 있는 사람이 몇 명인지 세어 보라. 위키라면 편집 권한이 있는 계정 수이고, 붙여 쓰는 서버라면 그 서버를 내놓은 조직이다.
2. 같은 문장이 과제일 때와 지식일 때와 공격일 때를 자기 과제에서 하나씩 적어 보라. 셋을 문장만 보고 가를 수 있는가. 가를 수 없다면 검사기에 무엇을 더 주어야 하는가.
3. 시험을 하나 만들어 보라. 과제 하나를 고르고, 코드로 확인할 수 있는 목표 하나를 적고, 그것을 표 22.1의 입구 하나에 넣어 돌린다. 그리고 뚫렸는지를 답이 아니라 환경의 상태에서 확인하라. 확인할 수 없다면 무엇이 기록에 더 남아야 하는가.
4. 표 22.3의 네 통로가 자기 시스템에 열려 있는지 하나씩 보라. 특히 답에 실린 주소가 자동으로 불러와지는지, 기억에 밖에서 온 글이 그대로 적힌 것이 몇 건인지 오늘 세어 보라.

22.8 다시 읽을 때

- 제21장의 절 21.3으로 돌아오면, 되돌릴 수 없는 행동을 찾아 그 앞에 사람을 세운 그 목록이 방어선이기도 하다는 것이 보인다. 두 장이 같은 자리를 다른 이유로 찾았다.
- 제20장의 절 20.2로 돌아오면, 스텝마다 무엇을 남길지 정한 그 목록이 이 장의 시험을 성립시키는 재료라는 것이 보인다. 남기지 않으면 뚫렸는지 판정할 방법이 없다.
- 제18장의 절 18.6으로 돌아오면, 성공한 실행과 똑같은 얼굴로 끝나는 실패의 목록에 여섯째가 붙는다. 앞의 다섯과 달리 이것은 누군가 그렇게 되도록 만든 실패다.
- 제23장을 읽고 표 22.2로 돌아오면, 도구를 실행마다 골라 여는 일과 나가는 목적지를 허용 목록으로 두는 일이 이미 만들어진 채로 오는 경우가 있다는 것이 보인다. 남아 있는지 확인하는 일이 고를 때의 검사 항목이 된다.

22.9 더 읽을거리

Greshake et al. (2023)에서 가져온 것은 진단이지 목록이 아니다. 언어 모델을 붙인 응용이 자료와 지시의 경계를 흐린다는 규정, 그리고 검색될 법한 자료에 미리 심어 둔 문장으로 멀리서 응용을 조종할 수 있다는 관찰이다. 공격의 갈래를 정리한 부분은 남지만 그 진단은 남는다.

Debenedetti et al. (2024)에서 가져온 것은 셋이다. 시험을 과제와 심어진 목표의 곱으로 만든다는 방식, 판정을 답이 아니라 환경의 상태로 한다는 결정, 그리고 방어별로 나온 수다. 수는 모델과 방어가 바뀌면 달라지므로, 오래 남을 것은 방어를 건줄 때 공격 성공률과 평소 과제 성공률을 함께 고려야 한다는 요구다.

Anthropic (2026d)에서 가져온 것은 둘이다. 밖에서 온 글을 도구 반환 자리에만 담고 지시문 자리에는 넣지 말라는 권고, 그리고 배포 전에 심어진 지시가 든 문서와 메일과 도구 출력으로 직접 돌려 보라는 권고다. 뒤의 것이 절 22.4의 근거다.

더 나아가기

앞의 스물두 장은 에이전트 시스템을 만들고 평가하고 지키는 법이었다. 여기서는 지금 밖에서 무슨 일이 일어나고 있는지를 본다.

적는 시점은 2026년 8월이다. 이 장이 가장 빨리 낡는다. 이름은 바뀌고, 수는 몇 달이면 어긋나기 시작한다. 그러니 여기 적힌 이름과 수를 외우지 말고, 새것이 왔을 때 그것을 무엇으로 판정할지를 가져가라. 판정의 기준은 앞의 스물두 장에 있다.

성능을 비교한 수치는 적지 않는다. 어느 쪽이 낫다는 말도 없다. 저자가 아래 것들을 오래 써 본 것은 아니어서, 저장소와 공식 문서에서 확인되는 것까지만 적는다.

23.1 스스로 생각해 볼 주제들

각 장은 끝에서 답하지 않은 것을 적었다. 스물두 장 어치를 한자리에 모으면 같은 것이 여러 번 나온다. 되풀이해서 나온 것만 네 갈래로 적는다.

수를 정하는 자리마다 근거가 없다

조각을 몇 자로 자를 것인가. 찾아온 후보를 몇 개 넘기고 그중 몇 개를 실을 것인가. 몇 스텝에서 끊을 것인가. 컨텍스트가 얼마나 차면 압축을 걸 것인가. 승인을 몇 군데에 세울 것인가. 평가 사례를 몇 건 만들어야 충분한가.

이 물음들은 서로 다른 장에서 따로 나왔는데 답하지 못한 이유가 같다. 값을 정하려면 값을 바꿔 가며 성적을 측정해야 하고, 측정하려면 정답이 달린 평가 집합이 먼저 있어야 한다. 그 집합은 과제마다 따로 만들어진다. 그래서 적을 수 있는 것은 무엇을 보고 정하라는 것까지이고, 얼마로 정하라는 것은 적을 수 없다.

전줄 자료가 없다

컨텍스트의 층마다 토큰이 얼마씩 들어가는지 측정한 자료가 없다. 원본 문서를 그대로 열 때와 글만 뽑아 쓸 때를 같은 과제 위에 나란히 놓고 평가한 자료도 없다. 여러 에이전트가 기억을 나눠 가질 때와 함께 쓸 때 무엇이 좋아지고 무엇이 나빠지는지도 마찬가지다.

이쪽은 사정이 앞의 것과 다르다. 측정하는 방법을 몰라서가 아니다. 아무도 아직 측정해서 내놓지 않았을 뿐이다. 그리고 측정할 수 있는 팀은 대개 그 결과를 밖에 내놓을 이유가 없다. 사내 문서와 사내 과제로 나온 값이기 때문이다.

조용한 실패를 어떻게 사례로 만드는가

에이전트의 실패 가운데 상당수는 소리를 내지 않는다. 실었는데 모델이 읽지 않은 조각. 넘겨받은 쪽이 못 본 것 위에서 내린 판단. 막아서 일어나지 않은 공격. 셋 다 성공한 실행과 똑같은 얼굴로 끝난다.

평가 집합에 넣으려면 그 실패를 먼저 사례로 적어야 한다. 그런데 적으려면 무엇이 잘못되었는지 알아야 하고, 알려면 그것이 소리를 내야 한다. 앞의 스물두 장이 여러 번 부딪히고 빠져나가지 못한 자리다.

방법으로 적을 수 있었던 것은 하나뿐이다. 조용한 실패가 소리를 내도록 시스템 쪽을 고치는 것이다. 넘겨받은 쪽이 자기가 무엇을 보고 판단했는지 함께 올리게 하면, 못 본 것 위의 판단이 기록에 남는다. 그렇게 고치는 데 드는 품이 그만큼 돌아오는지는 측정한 자료가 없다.

어긋난 둘 가운데 무엇을 고르는가

나뉘 놓은 두 에이전트가 서로 다른 사실을 올렸는데 둘 다 자기 쪽에서는 검증을 통과했다. 검색이 가져온 조각과 지식 그래프가 낸 답이 한 컨텍스트에서 부딪힌다. 실무자에게 물어 만든 승인 목록과 되돌릴 수 있는지로 만든 승인 목록이 어긋난다.

세 경우 모두 고르는 쪽이 원문을 보지 않았다는 점이 같다. 합치는 쪽에는 요약만 올라오고, 요약에는 어느 쪽이 맞는지 가릴 재료가 들어 있지 않다. 재료를 다 올리면 나누어서 얻은 것이 사라진다.

켜고 끈 것을 견줄 수 없다

나뉘 놓았을 때의 이득을 확인하려면 나누지 않은 실행과 견주어야 하는데, 그 실행은 대개 존재하지 않는다. 도구를 닫아서 막은 공격은 일어나지 않은 일이라 셀 수가 없다. 검증을 한 겹 더 걸어서 잡은 잘못도, 그 겹이 없었으면 어떻게 됐을지가 기록에 남지 않는다.

빠져나가는 방법이 아주 없지는 않다. 장치를 끄고 같은 과제를 다시 돌려 보면 된다. 결과가 같으면 그 장치는 잉여이고, 좋아지면 방해였다. 다만 이 검사에는 값이 든다. 실행을 두 번 돌려야 하고, 끈 채로 도는 동안 실제로 사고가 날 수 있다. 그래서 어느 장치를 얼마나 자주 이렇게 확인할지에는 여전히 답이 없다.

왜 답이 없는가

분야가 아직 자라는 중이고, 시스템마다 사정이 다르다. 자기 시스템에서 측정해 본 팀만 자기 답을 갖는다. 앞의 스물두 장이 여러 번 적은 문장이고, 여기서 한 번 더 적는다.

23.2 OpenClaw

2026년 상반기에 가장 빨리 퍼진 오픈소스 프로젝트는 한 사람이 자기 기기에서 쓰는 비서였다.

메신저로 말을 걸면 답한다. 메일을 정리하고, 일정을 잡고, 파일을 읽고 쓰고, 브라우저를 띄워 양식을 채우고, 셀 명령을 실행한다. 시키지 않아도 주기적으로 스스로 깨어나 할 일을 본다. 기억은 그 기기의 마크다운 파일로 남는다. 스킬은 파일로 더한다. 이름은 **OpenClaw** 이고, 저장소가 스스로를 소개하는 문장은 「어떤 운영체제든, 어떤 플랫폼이든 쓰는 개인 비서」다.

왜 이것이 붙이었는데

앞의 스물두 장이 따로 다른 것들을 한 물건 안에 합쳐 놓았기 때문이다.

루프를 돌리고 멈추는 바깥의 코드가 있다. 모델 호출은 저절로 이어지지 않으므로 그것을 돌리고 끊는 것이 밖에 있어야 한다고 제15장이 적었다. 과제에 특화된 지시와 자원을 폴더로 묶어 두었다가 필요할 때 읽어 들인다. 지시를 늘 컨텍스트에 얹혀 두는 대신 필요할 때만 들이라고 제7장이 적은 구성이다. 세션이 닫혀도 남는 것이 있다. 실행이 끝나면 사라지는 대화 이력과 달리 다음 세션이 읽을 수 있는 자리에 남겨 두는 것을 제10장이 기억이라고 불렀다. 셀 명령을 실행한다. 도구를 하나씩

부르는 대신 코드를 실행하게 하면 할 수 있는 일이 낱말의 목록에서 문장의 문법으로 넓어진다고 제7장이 적었다.

메신저를 잇는 방식도 한몫했다. 게이트웨이라고 부르는 프로그램 하나가 세션과 도구와 채널 연결을 맡고, 그 앞에 WhatsApp, Telegram, Slack, Discord, Signal, iMessage 같은 것들이 붙는다. 새 화면을 배우지 않아도 되고 이미 쓰는 앱에서 말을 걸면 된다.

릴리스 기록에 무엇이 들어 있는가

지금 무엇을 만들고 있는지는 릴리스 기록이 말해 준다. 2026년 8월에 나온 판들에 들어간 것을 옮기면 이렇다.

게이트웨이가 죽었다가 다시 뜰 때 하던 일을 이어 가는 것. 설정 파일을 쓰다 깨지지 않게 하는 것. 데이터베이스를 백업하고 되돌리는 명령. 비밀값이 정해 둔 호스트로만 나가게 묶는 것. 플러그인을 설치할 때 출처를 경고하는 것. 새 모델을 붙이는 것과 브라우저를 다루는 통로.

모델이 더 잘하게 만드는 항목은 여기에 거의 없다. 돌아가던 것을 계속 돌아가게 하는 일과 새어나가는 곳을 막는 일이 대부분이다. 아홉 달 된 프로젝트가 지금 무엇에 시간을 쓰는지가 이 목록이고, 앞의 스물두 장이 제6부와 제7부를 따로 둔 이유와 같은 이야기다.

가장 큰 위험은 통제되지 않은 실행이다

공식 사이트가 스스로 이렇게 적는다. 웹을 돌아다니고, 양식을 채우고, 파일을 읽고 쓰고, 셸 명령을 실행한다. 전부 열어 주거나 격리해서 돌리거나, 고르는 것은 쓰는 사람이다. 그리고 저장소의 안내는 도구가 기본적으로 그 기계에서 그대로 실행된다고 적는다.

전부 열어 두면 어떻게 되는지가 보안 쪽에서 여러 번 지적됐다. 셸 명령을 제한 없이 열어 주는 것은 들어오는 모든 입력에 그 기계의 최고 권한을 주는 것과 같다. 메신저로 들어온 메시지도, 웹에서 읽어 온 문장도, 남이 올린 스킴도 전부 그 입력이다.

제22장이 세운 구도 그대로다. 밖에서 온 글과 이쪽이 쓴 지시가 한 컨텍스트에 실리면 모델은 둘을 가르지 못한다. 제21장이 세운 것도 그대로 걸린다. 되돌릴 수 있는 행동은 열어 두고 되돌릴 수 없는 행동 앞에는 사람을 세우라고 했는데, 메일을 보내는 일과 파일을 지우는 일은 되돌릴 수 없는 쪽이다. 그런데 개인용 비서는 매번 승인을 받지 않는다. 매번 받으면 비서가 아니게 되기 때문이다. 스스로 깨어나 일한다는 성질이 여기에 한 겹을 더한다. 사람이 화면 앞에 없을 때도 행동이 일어난다.

승인 장치가 아예 없지는 않다. 저장소의 안내는 들어오는 메시지를 믿을 수 없는 입력으로 다루라고 적고, 모르는 사람이 말을 걸면 먼저 짝을 맺는 요청을 보내 승인을 받게 되어 있다. 다만 이것은 누가 말을 거는지를 거르는 장치이지, 승인된 사람이 보낸 문장 안에 심어진 지시를 거르는 장치가 아니다. 출처를 아는 쪽과 다음 행동을 정하는 쪽이 다르다고 제22장이 적었다. 게이트웨이는 그 문장이 누구에게서 왔는지 아는데, 그것을 읽고 행동을 뽑는 모델은 모른다.

기억도 같은 문제를 받는다. 파일로 남는 기억에 공격자가 문장을 하나 심어 두면 그 문장이 다음 실행부터 계속 지시로 읽힌다. 보안 쪽은 이것을 기억 오염이라고 부른다. 기억에 적힌 텍스트가 뒤의 실행을 조종하는 경로를 제10장이 미해결로 남겨 두었는데, 여기서는 그것이 실물로 있다.

개인용이라는 성질이 물음의 답을 바꾼다

되돌릴 수 없는 행동 앞에 누구를 세울지 정하려면 먼저 물을 것이 있다고 제21장이 적었다. 이 결정의 결과가 누구에게 도착하는가. 조직에서는 결과가 대개 다른 사람에게 도착한다. 고객에게 잘못 나간 메일, 잘못 고친 재고 수량, 잘못 승인된 지출이다. 그래서 허락을 정하는 자리가 조직의 정책이 된다.

개인용에서는 결과가 그 사람에게 도착한다. 잘못 보낸 메일도 그 사람의 메일이고, 지워진 파일도 그 사람의 파일이다. 그래서 허락을 정하는 자리가 조직의 정책이 아니라 그 시스템의 소유자로 좁아진다.

여기가 문제인 이유는 그 한 사람이 대개 자기가 무엇을 허락했는지 목록으로 갖고 있지 않다는 데 있다. 설치할 때 한 번 열어 준 것이 몇 달 뒤에도 그대로 열려 있고, 그동안 스킴이 몇 개 늘었는지도 세어 보지 않는다. 쓰지 않는 권한을 회수하라는 처방이 조직에서는 누군가의 일인데 여기서는 아무의 일도 아니다.

거버넌스를 밖에 붙이는 층

NVIDIA 가 이 위에 얹는 층을 내놓았다. 2026년 3월에 발표한 NemoClaw는 자기네 공개 모델과 OpenShell이라는 런타임을 한 명령으로 함께 설치하고, OpenShell이 내놓는 것이 자율 에이전트에 사생활과 보안을 더하는 격리된 샌드박스다. 실행을 가두고, 정책은 에이전트 밖에서 집행한다.

핵심은 하나다. 정책을 집행하는 코드가 에이전트와 다른 프로세스에 있다. 에이전트가 뚫려도 정책을 고칠 수 없다. 허가 판정은 모델 밖에서 내려야 한다고 제21장이 적은 것과 같은 처방이고, 나가는 목적지를 허용 목록으로 두라고 제22장이 적은 것과도 같다.

여기서 볼 것은 순서다. 실행 계층이 먼저 나와서 퍼졌고, 권한 모델과 가드레일은 나중에 남이 붙였다. 제22장과 제21장이 왜 따로 한 장씩 필요했는지가 그 순서에 그대로 있다.

산업이 가는 방향

같은 처방이 이 프로젝트 밖에서도 자리를 잡아 가고 있다. 에이전트가 만든 코드와 밖에서 온 입력을 적대적인 것으로 놓고 실행을 격리한다. 정책 판정은 에이전트가 고칠 수 없는 곳에 둔다. 자격 증명은 짧은 수명으로 끊는다. 격리의 수단도 컨테이너보다 강한 쪽으로 옮겨 가고 있다. 커널을 나눠 쓰는 격리로는 에이전트가 만든 코드를 담아 두기에 모자란다는 것이 대체로 합의된 자리다.

에이전트에게 사람 계정을 그대로 쓰게 하지 말고 따로 만든 신원을 주라는 권고도 함께 나온다. 그 신원에는 필요한 만큼만 담고, 오래 사는 자격 대신 짧게 끊어지는 것을 준다. 평가해 보는 동안은 아예 다른 기계에서 돌리고 그 기계를 버릴 수 있는 것으로 다루라는 권고도 있다. 제21장이 권한을 좁히는 법으로 적은 항목들과 같은 목록이다.

23.3 오픈소스 프레임워크

사내 규정을 답해 주는 챗봇을 만든다고 하자. 문서를 열어 글을 뽑고, 적당한 크기로 자르고, 각 조각을 벡터로 바꿔 저장한다. 질문이 오면 질문도 벡터로 바꿔 비슷한 조각을 찾는다. 찾은 조각을 프롬프트에 끼워 모델에 보낸다. 단계마다 라이브러리를 고르고 이어 붙여야 한다.

LangChain은 이 단계들을 이미 만들어 둔 조각으로 준다. 문서를 읽는 것, 자르는 것, 벡터로 바꾸는 것, 찾는 것, 프롬프트에 끼우는 것이 각각 하나씩 있고 순서대로 이으면 된다. 며칠 걸릴 일이 만나절이 된다.

흐름을 그래프로 적게 하는 쪽

같은 챗봇을 한 겹 키우면 사정이 달라진다. 답을 만들었는데 규정과 어긋날 수 있어서 검증을 한 번 걸고, 실패하면 다른 조각을 다시 찾기로 한다. 반복문을 짜고, 그 안에서 검색을 부르고, 답을 만들고, 검증을 부르고, 실패하면 어디로 돌아갈지 조건문으로 적는다. 몇 번까지 다시 할지 세는 변수를 두고 그 변수를 어디에 둘지 정한다. 단계가 셋일 때는 건널 만한데, 다섯이 되면 어디서 무엇이 바뀌는지 읽기 어려워진다.

LangGraph는 같은 곳에서 나온 다른 오픈소스이다. 공식 문서는 스스로를 낮은 층의 오케스트레이션 프레임워크이자 런타임이라고 적고, 프롬프트와 구조는 감추지 않는다고 못 박는다. 단계를 노드로 적고, 단계 사이의 이동을 간선으로 적고, 조건에 따라 어디로 갈지를 그 간선 위에 적는다. 루프와 분기가 코드 안에 숨지 않는다.

그리고 단계들이 함께 쓰는 상태를 이름과 형식이 있는 것으로 먼저 선언하게 한다. 검색이 채우는 칸, 답이 채우는 칸, 검증이 채우는 칸을 미리 적어 두는 방식이다. 나눠 놓은 것들 사이에 새로 생기는 일은 주고받는 일이 아니라 기대와 이행을 적어 두는 일이라고 제17장이 적었다. 상태를 먼저 선언하게 하는 것이 그 적어 두는 자리를 강제로 만들어 준다.

이미 들어 있는 것

LangChain이 미들웨어라는 이름으로 붙여 두는 것들의 목록이 앞의 스물두 장과 겹친다. 미들웨어는 루프의 정해진 지점에 끼워 넣는 짧은 코드이고, 공식 문서가 기본으로 제공한다고 적어 둔 것 가운데 이 책과 겹치는 것을 옮기면 이렇다.

토큰 상한에 가까워지면 대화 이력을 자동으로 요약한다. 제6장이다. 도구를 부르기 전에 멈추고 사람의 승인을 기다린다. 제21장이다. 개인 정보를 찾아내 가린다. 제22장이다. 모델 호출 횟수와 도구 호출 횟수에 상한을 건다. 제15장이다. 도구가 너무 많으면 모델을 따로 세워 관련 있는 것만 고르게 하거나, 이름만 두었다가 필요할 때 불러온다. 제7장이다. 이력에서 도구 결과만 지우거나 잘라 낸다. 다시 제6장이다.

이 목록이 곧 이 갈래가 표준처럼 자리 잡은 이유다. 여기 적힌 것들은 어느 시스템에서나 결국 필요해지고, 처음 만드는 팀은 그것을 하나씩 다시 틀리면서 배운다.

그 밖의 것들

같은 자리를 노리는 것이 여럿 있다. CrewAI는 역할을 나눠 여러 에이전트를 빨리 세워 보는 쪽에 가깝고, LlamaIndex Workflows는 문서가 많은 흐름을 사건 단위로 엮는다. Microsoft Agent Framework와 Google ADK는 각자 자기 클라우드 위에서 쓰는 것을 전제한다. OpenAI Agents SDK는 추상을 거의 두지 않고, 에이전트가 다른 에이전트에게 제어를 넘기는 것을 기본 단위로 삼는다. 대부분 MIT나 Apache 라이선스로 나온다.

고르는 기준은 기능의 수가 아니다. 물을 하나면 된다. 이것이 무엇을 대신 채워 주고, 대신 무엇이 정해진 채로 오는가. 기능을 세면 많이 채워 주는 쪽이 이기는데, 많이 채워 줄수록 정해진 채로 오는 것도 많다.

기록하고 평가하는 계층은 따로 온다

프레임워크를 고르는 일과 실행을 기록하는 일은 붙어 있지 않다. LangChain의 소개 글이 이것을 한 문장으로 적는다. 어느 프레임워크를 고르느냐가 무엇을 빨리 만들 수 있는지를 정하고, 거기에 어떤 관측과 평가 계층을 붙이느냐가 내보낸 뒤에도 계속 돌아가는지를 정한다.

이 계층이 대신해 주는 것은 남길 자리다. 모델 호출과 도구 호출과 검색 단계마다 무엇이 들어가고 무엇이 나왔는지 손으로 심지 않아도 남고, 실행 하나가 하나로 묶여 보인다. 실행이 무엇을 했는지 남기지 않으면 나중에 물을 수 없다고 제20장이 적은 그 자리가 기본으로 채워진다.

기록의 형식에도 합의가 서고 있다. OpenTelemetry의 생성 모델 규칙이 에이전트와 워크플로와 도구와 모델을 각각 하나의 마디로 정하고, 동작의 이름을 필수 항목으로 두면서 걸린 시간과 쓴 토큰은 권장 항목으로 둔다. 다만 속성의 이름은 아직 바뀔 수 있다고 규칙에 적혀 있다.

대신 무엇을 기록할지가 정해져 온다. 시간과 토큰은 어느 과제에서나 남기고 싶은 것인데, 이 과제에서 정작 봐야 할 것이 다른 데 있을 수 있다. 찾아온 조각 가운데 답에 실제로 쓰인 것이 몇 개인지, 검증이 몇 번째 시도에서 통과했는지 같은 것들이다. 지표의 항목이 곧 실패의 이름표라고

제18장이 적었다. 이름이 없는 실패는 고칠 목록에 오르지 않으므로, 이 계층을 없었을 때 물을 것은 무엇이 기록되는가가 아니라 무엇이 기록되지 않는가다.

프레임워크가 나올 때

빨리 확인해 보고 싶을 때다. 될지 안 될지 모르는 것에 며칠을 쓰기 전에 만나질로 답을 본다.

앞 장들의 결정을 아직 안 내렸을 때도 그렇다. 자르는 크기도 압축 문턱도 승인 지점도 정하지 않은 상태에서는 정해진 값이라도 있는 편이 낫다. 아무 값도 없으면 돌려 볼 수가 없고, 돌려 보지 않으면 정할 근거가 생기지 않는다.

남들이 겪은 문제를 안 겪고 싶을 때도 그렇다. 도구 호출이 실패했을 때 되풀이하는 방식이나 모델이 응답하지 않을 때 다른 모델로 넘기는 방식은 이미 여러 번 틀려 본 사람들이 적어 둔 것이다.

직접 만드는 편이 나올 때

프레임워크가 정해 둔 것과 이쪽 사정이 다를 때다. 요약할 거는 방아쇠가 토큰 수로 되어 있는데 이쪽에서 보고 싶은 것이 도구 호출 횟수라면, 그 기본값은 맞지 않는다.

어디서 무슨 일이 일어나는지 봐야 할 때도 그렇다. 답이 나빠졌을 때 의심할 목록을 만들려면 단계마다 무엇이 들어가고 무엇이 나왔는지 알아야 한다. 층이 한 겹 늘 때마다 그 목록이 멀어진다.

고를 때 무엇을 묻는가

프레임워크는 상당 부분 하네스를 상품으로 만든 것이다. 고른다는 것은 남이 적어 둔 가정 목록을 통째로 들여오는 일이고, 물을 것은 그 목록이 지금의 모델과 이쪽 과제에 아직 유효한가다. 제15장이 그렇게 적었다.

앞의 미들웨어 목록을 그 물음으로 다시 읽으면 이렇게 된다. 요약할 거는 문턱은 지금 모델의 컨텍스트 길이를 가정한다. 도구를 골라 주는 장치는 도구가 많으면 모델이 헛갈린다고 가정한다. 승인을 기다리는 장치는 그 행동을 되돌릴 수 없다고 가정한다. 셋 가운데 앞의 둘은 모델이 좋아지면 깨질 수 있고, 마지막 하나는 깨지지 않는다.

23.4 로컬 운영 에이전트

여기까지는 모델을 밖에서 부르는 것을 전제했다. 부르지 않는 길도 있다. 모델 파일을 그 기계에 두고 직접 돌린다.

이 길을 먼저 보는 팀은 대개 자료 때문에 본다. 진료 기록이나 계약서나 사람에 관한 자료를 다루는 자리에서는 어디까지 밖으로 보내도 되는지가 기술 문제가 아니라 규정 문제이고, 그 규정을 통과시키는 데 몇 달이 걸린다. 아예 나가지 않으면 그 몇 달이 없어진다.

무엇이 좋아지는가

토큰 값이 들지 않는다. 스텝을 몇 번 돌든 추가로 청구되는 것이 없어서, 제3장이 셈한 길이의 값이 통째로 사라진다. 대신 전기와 장비에 값이 든다.

밖으로 나가는 것이 없다. 사내 문서도 고객 자료도 그 기계를 떠나지 않는다. 어디까지 보내도 되는지를 정하는 일 자체가 없어진다.

판이 바뀌어도 이쪽 것은 그대로다. 모델이 조용히 갱신되어 어제 되던 프롬프트가 오늘 다르게 동작하는 일이 없다. 파일을 갖고 있으므로 서비스가 끝나도 계속 돌릴 수 있다.

표 23.1: 2026년 8월에 모델 배포처와 공식 모델 문서에서 확인한 값이다. 파일 크기는 4비트로 줄인 판이고, 그만큼이 메모리에 올라가야 돌아간다.

모델	파라미터	파일 크기	컨텍스트 상한
Qwen3 8B	8B	5.2GB	40K
Qwen3 14B	14B	9.3GB	40K
gpt-oss 20b	21B, 한 번에 3.6B만 쓴다	14GB	128K
Qwen3 30B	30B, 한 번에 3.3B만 쓴다	19GB	256K
gpt-oss 120b	117B, 한 번에 5.1B만 쓴다	65GB	128K

무엇이 늘어나는가

추론 능력이 떨어진다. 여러 단계를 이어 계획하고 도구를 골라 부르는 대목에서 특히 그렇다. 큰 모델이 푼 궤적으로 작은 모델을 가르쳐 이 차이를 좁히려는 연구가 여러 갈래 나와 있는데, 여러 단계를 긴 시간 이어 가는 대목은 아직 열린 문제로 적힌다. 그리고 가르치는 쪽은 여전히 큰 모델이다. 격차를 좁히는 일 자체가 장비가 갖춰진 곳에서 일어난다.

속도가 느리다. 그리고 에이전트에서는 이 차이가 그대로 곱해진다. 한 번 묻고 한 번 답받고 끝나는 것이 아니라 스텝마다 다시 부르기 때문이다. 스텝당 몇 초씩 벌어지면 20스텝짜리 실행은 분 단위로 벌어진다.

장비를 갖춰야 한다. 표의 파일 크기가 그대로 메모리 요구이고, 여기에 컨텍스트를 담는 자리가 따로 붙는다. 그 자리는 컨텍스트가 길어질수록 커진다. 파일이 메모리에 들어간다는 말이 긴 컨텍스트로 돌아간다는 말은 아니다.

그리고 긴 컨텍스트를 감당하지 못한다. 마지막 것이 앞의 장들과 가장 세게 부딪힌다.

표 23.1에 지금 로컬에서 돌릴 만한 것들을 모았다. 만든 쪽이 적어 둔 요구 사양은 gpt-oss 20b가 메모리 16GB, 120b가 80GB짜리 GPU 한 장이다. 앞의 것은 노트북에서 돌아가고 뒤의 것은 돌아가지 않는다.

파라미터 칸의 「한 번에 얼마만 쓴다」는 전체를 메모리에 올려 두되 토큰 하나를 낼 때는 그중 일부만 계산한다는 뜻이다. 그래서 파일은 큰데 속도는 그 일부에 가깝다. 메모리는 전체가 필요하다.

이 표가 이 책에서 가장 빨리 낡는다. 반년이면 숫자가 한 자리씩 달라져 있을 것이다.

컨텍스트 상한이 앞의 결정을 바꾼다

마지막 칸을 앞 장들의 결정 위에 올려 보면 무엇이 달라지는지 나온다.

40K는 여유가 아니다. 제4장이 컨텍스트를 층으로 갈랐을 때, 시스템 지시와 도구 설명과 미리 실어 둔 자원까지가 실행 전에 이미 자리를 차지한다. 도구가 열이고 설명이 꼼꼼하면 그것만으로 수천 토큰이다. 남은 자리에서 이력과 검색 결과가 함께 자란다. 40K에서는 압축이 선택지가 아니라 전제가 된다.

찾아온 조각을 몇 개 실을지도 바뀐다. 조각 하나가 800토큰이고 열 개를 실으면 8000이다. 밖의 모델에서는 선택해 볼 여지가 있던 값이 여기서는 상한을 먼저 만난다. 조각을 더 잘게 자르거나, 재순위를 걸어 실을 개수를 줄이거나, 둘 다 해야 한다.

로컬로 간다면 정할 것

전부를 로컬로 옮기는 결정은 드물다. 대개는 섞는다. 들어온 문의를 갈래로 나누는 일, 뽑아낸 값을 정해진 형식에 맞추는 일, 긴 글을 줄이는 일은 로컬 모델로도 된다. 여러 단계를 이어 계획하고

도구를 골라 부르는 대목만 밖으로 보낸다.

읽을 때 정해야 할 것은 무엇을 밖으로 보내느냐다. 밖으로 보내는 것이 곧 밖으로 나가는 것이므로, 나가지 않는다는 이점을 그만큼 다시 내놓게 된다. 그리고 밖으로 보내는 대목은 대개 컨텍스트가 가장 두꺼운 대목이다. 어려운 판단일수록 재료가 많이 필요하기 때문이다.

정리하면 로컬로 갈 때 정할 것은 이렇다.

- 어떤 과제를 맡길 것인가. 시스템 전부인가, 그중 일부인가.
- 읽는다면 무엇을 밖으로 보내는가. 그리고 그때 무엇이 함께 나가는가.
- 컨텍스트 상한이 압축 문턱과 검색 결과 개수와 도구 설명의 길이를 어떻게 바꾸는가.
- 장비를 사는 값과 몇 달치 토큰 값 가운데 어느 쪽이 큰가.

23.5 앞으로 지켜봐야 하는 것

무엇이 바뀌면 앞의 스물두 장 가운데 어디가 남는지를 적는다.

바뀌면 남는 것

컨텍스트가 더 커지면 자르기와 압축의 자리가 줄어든다. 지금은 20만 토큰짜리 문서를 어떻게 나눠 실을지가 문제인데, 그것이 통째로 들어가면 나누는 결정 자체가 사라진다. 이미 100만 토큰을 받는 모델이 여럿이다.

모델이 좋아지면 하네스의 장치들이 잉여가 된다. 스텝 수에 상한을 거는 것은 모델이 스스로 멈추지 못한다는 가정 위에서 있고, 도구를 골라 주는 장치는 도구가 많으면 헛갈린다는 가정 위에서 있다. 가정이 깨지면 그 장치는 도움이 되지 않을 뿐 아니라 방해가 된다.

규격이 자리 잡으면 직접 정할 것이 준다. 도구를 어떻게 내놓고 어떻게 부를지, 에이전트끼리 무엇을 주고받을지를 지금은 저마다 정하고 있다. 여기에 합의가 서면 제8장과 제17장이 다른 결정 가운데 일부가 기본값이 된다.

토큰 값이 더 내려가면 아끼는 쪽의 처방이 힘을 잃는다. 캐시를 태우고, 압축을 걸고, 도구 설명을 줄이는 일은 전부 토큰에 값이 붙는다는 전제 위에서 있다. 값이 지금의 몇 분의 일이 되면 그 처방들의 순서가 바뀐다. 다만 값이 내려간 만큼 스텝을 더 도는 쪽으로 쓰게 되므로, 한 실행에 청구되는 총액까지 함께 내려간다는 보장은 없다.

바뀌지 않는 것

컨텍스트는 유한하다. 100만이 1000만이 되어도 넣고 싶은 것이 그보다 많다. 그리고 상한까지 채울 수 있다는 것과 상한까지 채워도 잘 읽는다는 것은 다른 말이다. 창이 커진 만큼 성능이 따라 커지지는 않는다고 여러 곳에서 보고되고 있고, 이것이 제4장의 전제였다.

모델은 확률로 답한다. 같은 입력에 같은 출력이 나온다는 보장이 없으므로, 형식을 강제해도 내용의 옳음까지 따라오지 않는다. 평가와 기록이 필요한 이유가 여기 있고 이 이유는 성능이 좋아진다고 사라지지 않는다.

마지막 승인은 사람 쪽에 남는다. 되돌릴 수 없는 행동의 결과를 감당하는 것이 사람이기 때문이다. 모델이 아무리 잘해도 잘못 보낸 메일은 그 사람의 메일이다.

밖에서 온 글과 이쪽이 쓴 지시는 한 컨텍스트에 실리면 같은 텍스트다. 모델이 좋아져도 둘을 가를 표시가 그 텍스트 안에 없다. 그래서 걸러 내는 검사는 뚫리는 빈도를 낮출 뿐이고, 경계는 결국 들어온 것이 할 수 있는 일을 좁혀서 세운다. 제22장의 결론이 여기에 그대로 남는다.

둘을 가르는 기준

가르는 기준은 하나다. 그 문장이 지금 모델의 한계 위에서 있으면 낡고, 유한함이나 확률처럼 없어지지 않는 성질 위에서 있으면 남는다.

컨텍스트를 층으로 갈라 무엇을 미리 싣고 무엇을 그때그때 가져올지 정하라는 처방은 유한함 위에서 있다. 창이 열 배가 되어도 정할 것이 남는다. 반면 도구가 스물을 넘으면 고르는 일이 나빠진다는 처방은 지금 모델의 한계 위에서 있다. 언젠가 지워도 되는 문장이라는 뜻이다.

장치는 지금 모델에 맞춘 임시 물건이고 가정에는 유효 기간이 있다고 제15장이 적었다. 그 문장은 하네스만이 아니라 이 책 전체에 걸린다. 각 장의 처방을 읽을 때 그것이 어느 쪽 위에서 있는지를 함께 읽으면, 몇 해 뒤에도 무엇을 버리고 무엇을 남길지 판정할 수 있다.

23.6 맺음

에이전트가 무엇인지에서 시작해 추론 과정과 토큰, 컨텍스트 엔지니어링, 기억과 검색, 하네스와 운영, 그리고 평가까지 왔다. 설계의 모든 과정에 기초를 다진 셈이다.

그 전부가 하나의 구조 위에서 있었다. 모델이 행동을 내고, 환경이 관측을 돌려주고, 그 관측이 다른 재료들과 함께 컨텍스트로 조립되어 다시 모델로 들어가는 순환이다. 값이 왜 그렇게 자라는지, 왜 길게 이어 붙일수록 나빠지는지, 왜 도구 목록이 곧 할 수 있는 일의 전부인지가 전부 거기서 나왔다.

스물세 장이 서로 다른 주제로 보이지만 물음은 하나였다. 자리는 유한한데 넣고 싶은 것은 그보다 많다. 무엇을 넣을 것인가.

읽은 사람이 손에 들고 나가는 것도 하나다. 결정의 목록이다. 조각을 몇 자로 자를지, 몇 스텝에서 끊을지, 컨텍스트가 찰 때 무엇을 버릴지, 세션을 넘길 때 무엇을 남길지, 무엇을 평가하고 무엇을 평가하지 않을지, 되돌릴 수 없는 행동 앞에 누구를 세울지.

그 결정들이 엔지니어로서 자기 역량이자 경쟁력이 된다.

도구가 이 줄들을 대신 채워 주더라도 채우는 일이 없어지지 않는다는. 도구를 쓴다는 것은 남이 채운 값을 받는다는 뜻이다. 적어 두면 결정이고 안 적으면 기본값이다. 결정은 나중에 아직 맞는지 확인할 수 있고, 기본값은 확인할 대상이 되지 않는다.

소프트웨어 엔지니어들이 이제 모두 Claude Code나 Codex를 쓰는 것처럼, 더 넓게는 일반인들이 생성형 AI의 도움을 받으며 일하고 있는 것처럼, 이 분야는 빠르게 자랄 것이다.

그 속에서 에이전트 엔지니어는 설계의 열쇠를 쥔다. 자라는 산업에서 길잡이가 되어야 한다. 그 일에 이 책이 보탬이 되기를 바란다.

파도가 온다. 이제 각자의 노를 쥐고 바다로 나아갈 때다.

참고문헌

- Agent2Agent Project. Agent2agent (A2A) protocol specification. Linux Foundation, 2026. <https://a2a-protocol.org/latest/specification/>.
- Anthropic. Building effective agents. Anthropic Engineering Blog, December 2024a. <https://www.anthropic.com/engineering/building-effective-agents>.
- Anthropic. Introducing contextual retrieval. Anthropic, September 2024b. <https://www.anthropic.com/news/contextual-retrieval>.
- Anthropic. Introducing the model context protocol. Anthropic Announcements, November 2024c. <https://www.anthropic.com/news/model-context-protocol>.
- Anthropic. Introducing advanced tool use on the claude developer platform. Anthropic Engineering Blog, November 2025a. <https://www.anthropic.com/engineering/advanced-tool-use>.
- Anthropic. Code execution with MCP: Building more efficient agents. Anthropic Engineering Blog, November 2025b. <https://www.anthropic.com/engineering/code-execution-with-mcp>.
- Anthropic. Effective context engineering for AI agents. Anthropic Engineering Blog, September 2025c. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>.
- Anthropic. Effective harnesses for long-running agents. Anthropic Engineering Blog, November 2025d. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>.
- Anthropic. How we built our multi-agent research system. Anthropic Engineering Blog, June 2025e. <https://www.anthropic.com/engineering/multi-agent-research-system>.
- Anthropic. Equipping agents for the real world with agent skills. Anthropic Engineering Blog, October 2025f. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>.
- Anthropic. Writing effective tools for agents — with agents. Anthropic Engineering Blog, September 2025g. <https://www.anthropic.com/engineering/writing-tools-for-agents>.
- Anthropic. Prompt caching. Claude Developer Platform Documentation, 2026a. <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>.

- Anthropic. Citations. Claude Developer Platform Documentation, 2026b. <https://docs.claude.com/en/docs/build-with-claude/citations>.
- Anthropic. Demystifying evals for AI agents. Anthropic Engineering Blog, January 2026c. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>.
- Anthropic. Mitigate jailbreaks and prompt injections. Claude Platform Documentation, 2026d. <https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks>.
- Anthropic. Memory tool. Claude Developer Platform Documentation, 2026e. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/memory-tool>.
- Anthropic. Building multi-agent systems: When and how to use them. Claude Engineering Blog, January 2026f. <https://claude.com/blog/building-multi-agent-systems-when-and-how-to-use-them>.
- Anthropic. PDF support. Claude Developer Platform Documentation, 2026g. <https://platform.claude.com/docs/en/build-with-claude/pdf-support>.
- Anthropic. Structured outputs. Claude Developer Platform Documentation, 2026h. <https://platform.claude.com/docs/en/build-with-claude/structured-outputs>.
- Anthropic. Token counting. Claude Developer Platform Documentation, 2026i. <https://platform.claude.com/docs/en/build-with-claude/token-counting>.
- Anthropic. Tool use with Claude. Claude Developer Platform Documentation, 2026j. <https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>.
- Anthropic. Vision. Claude Developer Platform Documentation, 2026k. <https://platform.claude.com/docs/en/build-with-claude/vision>.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. arXiv:2404.16130.
- Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.01449.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <https://www.deeplearningbook.org/>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023. arXiv:2302.12173.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMlingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358–13376, 2023. arXiv:2310.05736.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1658–1677, 2024. arXiv:2310.06839.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020. arXiv:2004.04906.
- Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020. arXiv:2004.12832.
- Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing, 2018. arXiv:1808.06226.
- Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15339–15353, 2024. arXiv:2402.14848.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020. arXiv:2005.11401.
- Linux Foundation. Linux foundation launches the Agent2Agent protocol project. Linux Foundation Press Release, June 2025. <https://www.linuxfoundation.org/press/linux-foundation-launches-the-agent2agent-protocol-project-to-enable-secure-intelligent-communication-between-ai-agents>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. arXiv:2307.03172.

- Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022. arXiv:2104.08786.
- OpenAI. Introducing SWE-bench verified. OpenAI, August 2024. <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenTelemetry. Semantic conventions for generative AI systems. Cloud Native Computing Foundation, 2026. <https://github.com/open-telemetry/semantic-conventions-genai>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. arXiv:2310.08560.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 126544–126565, 2024. arXiv:2305.15334.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.15425.
- Prithvi Rajasekaran. Harness design for long-running application development. Anthropic Engineering Blog, March 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2302.04761.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016. arXiv:1508.07909.
- Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 31210–31227, 2023. arXiv:2302.00093.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. arXiv:2303.11366.

- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. arXiv:2104.09864.
- Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, 2024. arXiv:2408.02442.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1706.03762.
- Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 9154–9160, 2020. arXiv:1909.03341.
- Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. arXiv:2305.04091.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2201.11903.
- Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. OpenAI, April 2025. arXiv:2504.12516.
- Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a. arXiv:2305.10601.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b. arXiv:2210.03629.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.12045.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235, pages 62138–62160, 2024. arXiv:2310.04406.